ThemisDB

Das vollständige Handbuch

v1.3.4

Umfassendes Nachschlagewerk für ThemisDB-Datenbank

Generiert: 02. January 2026

Kapitel: 49

Inhaltsverzeichnis

Abbildungsverzeichnis

Kapitel 1: Vorwort

> *"Dokumentation ist wie Code - sie sollte wartbar, erweiterbar und verständlich sein."*

Warum dieses Buch?

Als wir ThemisDB entwickelten, hatten wir eine Vision: Eine Datenbank, die alles kann, was moderne Anwendungen brauchen - relational, Graph, Dokument und Vektor - in einem System, ohne Kompromisse.

Aber eine großartige Datenbank ist nur halb so viel wert ohne großartige Dokumentation. Und genau hier beginnt die Herausforderung.

Das Dokumentations-Dilemma

Wir hatten über 700 Dokumente geschrieben: - API-Referenzen - Feature-Beschreibungen - Architektur-Dokumente - Beispiel-Projekte - How-To-Guides - Troubleshooting-Tips

Alles war da. Aber es war verstreut. Fragmentiert. Schwer zu durchdringen für Neue.

Ein Entwickler fragte uns:
"*Ich habe die Docs gelesen, aber ich verstehe nicht, wie alles zusammenpasst. Wann nutze ich Graph statt Relational? Wie skaliere ich? Was sind die Best Practices?*"

Das war der Moment, in dem wir erkannten: Wir brauchen nicht nur Dokumentation - **wir brauchen ein Buch**.

Was macht dieses Buch anders?

1. Narrative statt Referenz

Statt:

> *"Die `SHORTEST_PATH` Funktion findet den kürzesten Pfad..."*

Schreiben wir:

> *"Stellen Sie sich vor, Sie bauen ein Social Network. Alice will wissen, wie sie mit Bob verbunden ist. In SQL wäre das ein rekursiver Join - langsam und komplex. ThemisDB macht es einfach..."*

2. Vollständige Examples

Jedes Konzept wird mit einem vollständigen, lauffähigen Example demonstriert. Kein Pseudo-Code. Echte Programme, die Sie herunterladen und ausführen können.

3. Das "Warum" vor dem "Wie"

Wir erklären nicht nur, **wie** Features funktionieren, sondern **warum** sie so designt wurden und **wann** Sie sie einsetzen sollten.

4. Von Einfach zu Komplex

Kapitel 1 erklärt die Basics. Kapitel 30 zeigt Enterprise-Patterns. Dazwischen eine sorgfältig gestaltete Lernkurve.

5. Alle Modelle integriert

Wir zeigen nicht nur einzelne Modelle, sondern wie Sie sie kombinieren. Multi-Model Queries. Cross-Model Transaktionen. Das ganze Potenzial.

Inspiration

Dieses Buch ist inspiriert von Software-Engineering-Klassikern, die uns beim Lernen geholfen haben:

"Designing Data-Intensive Applications" (Martin Kleppmann)
Für die tiefe, konzeptionelle Herangehensweise.

"Programming Rust" (Blandy, Orendorff, Tindall)
Für die "Let's build..." Abschnitte und vollständigen Programme.

"The Go Programming Language" (Donovan, Kernighan)
Für die klare, präzise Sprache und praktischen Beispiele.

"Site Reliability Engineering" (Google)
Für die Operations-Perspektive und Real-World Case Studies.

Für wen ist dieses Buch?

Wenn Sie neu bei ThemisDB sind

Fangen Sie bei Kapitel 1 an. Arbeiten Sie sich durch Teil I und II. Nach Teil II haben Sie ein solides Fundament.

Wenn Sie bereits Erfahrung haben

Springen Sie direkt zu den Themen, die Sie interessieren: - **Performance-Probleme?** → Kapitel 20 - **Skalierung planen?** → Kapitel 17-18 - **Sicherheit härten?** → Teil VI - **AQL meistern?** → Kapitel 13

Wenn Sie von einer anderen DB migrieren

- **Von PostgreSQL?** → Kapitel 5, 29
- **Von Neo4j?** → Kapitel 6, 29
- **Von MongoDB?** → Kapitel 7, 29

Was Sie brauchen

Vorwissen

- **Grundlegende Programmierkenntnisse:** Python, JavaScript oder ähnlich
- **AQL-Grundlagen:** Hilfreich, aber nicht erforderlich
- **Docker-Basics:** Für die Examples

Software

- **Docker:** Für ThemisDB und Examples
- **Python 3.8+:** Für Example-Code
- **Git:** Zum Klonen der Examples

Alles ist kostenlos und Open Source.

Struktur dieses Buchs

8 Teile, 30 Kapitel, ~880 Seiten

Jedes Kapitel folgt diesem Muster: 1. **Überblick:** Was Sie lernen werden 2. **Theorie:** Konzepte erklärt 3. **Praxis:** Vollständige Examples 4. **Patterns:** Best Practices 5. **Performance:** Optimierung 6. **Zusammenfassung:** Key Takeaways

---

Wie dieses Buch entstand

Dieses Buch ist das Ergebnis von: - **700+ einzelne Dokumente** konsolidiert - **21 vollständige Example-Projekte** integriert - **Monate an Reviews** von Entwicklern und Early Adopters - **Unzählige Iterationen** bis zur perfekten Struktur

Es ist lebendig. Wir aktualisieren es mit jeder ThemisDB-Version. Feedback ist willkommen.

Danksagungen

Ein großes Dankeschön an:

- **Die Community:** Für Feedback, Bug Reports und Feature Requests
- **Early Adopters:** Die ThemisDB in Production eingesetzt haben
- **Contributors:** Für Code, Docs und Examples
- **Reviewer:** Für unzählige Stunden detailliertes Feedback

Besonderer Dank an die Autoren der Bücher, die uns inspiriert haben.

Über die Autoren

Dieses Buch wurde geschrieben vom ThemisDB-Team - Entwickler, die täglich mit der Datenbank arbeiten und sie lieben.

Wir glauben an: - **Open Source:** ThemisDB ist MIT-lizenziert - **Qualität:** Code und Docs auf höchstem Niveau - **Community:** Gemeinsam besser werden

---

Feedback und Beiträge

Dieses Buch ist Open Source, wie ThemisDB selbst.

Fehler gefunden?
Issue erstellen

Verbesserung vorschlagen?
Pull Request öffnen

Frage stellen?
Discussion starten

Jeder Beitrag zählt. Auch wenn es nur ein Tippfehler ist.

Los geht's!

Sie halten ein Handbuch in den Händen, das Sie von Null zur ThemisDB-Mastery führen wird.

Es ist eine Reise. Nehmen Sie sich Zeit. Experimentieren Sie mit den Examples. Bauen Sie eigene Projekte.

Und vor allem: Haben Sie Spaß!

Datenbanken sind mächtige Werkzeuge. ThemisDB macht sie zugänglich.

Ready to become a ThemisDB expert?

→ Kapitel 1: Einführung in ThemisDB

ThemisDB Team
Dezember 2025

Kapitel 2: Kapitel 1: Einführung in ThemisDB

> *"Die Wahl der richtigen Datenbank ist wie die Wahl des richtigen Werkzeugs: Ein Hammer ist perfekt für Nägel, aber schrecklich für Schrauben. ThemisDB ist der Werkzeugkasten, der beides kann – und noch viel mehr."*

Überblick

Willkommen bei ThemisDB, einer modernen Multi-Model-Datenbank, die entwickelt wurde, um die Grenzen traditioneller Datenbankarchitekturen zu überwinden. In diesem einführenden Kapitel lernen Sie die Grundkonzepte, die Philosophie und die Kernfähigkeiten von ThemisDB kennen. Moderne Anwendungen haben komplexe Datenanforderungen, die sich nicht mehr mit einem einzigen Datenmodell abbilden lassen. Gleichzeitig führt der Einsatz mehrerer spezialisierter Datenbanken zu operationaler Komplexität und Konsistenzproblemen. ThemisDB löst dieses Dilemma durch einen einheitlichen Multi-Model-Ansatz, der verschiedene Datenmodelle in einer kohärenten Plattform vereint. Dieses Kapitel zeigt Ihnen, warum dieser Ansatz notwendig ist und wie ThemisDB ihn umsetzt.

Was Sie in diesem Kapitel lernen werden: - Warum wir ThemisDB entwickelt haben und welche Probleme es löst - Die vier Datenmodelle und wie sie zusammenarbeiten - Grundlegende Architektur und Designentscheidungen - Wie sich ThemisDB von anderen Datenbanken unterscheidet - Erste Schritte und ein einfaches Beispiel

Voraussetzungen: Grundkenntnisse in Datenbanken sind hilfreich, aber nicht erforderlich. Wir erklären alle Konzepte von Grund auf.

1.1 Die Multi-Model-Herausforderung

In der modernen Softwareentwicklung stehen Entwickler vor einem fundamentalen Dilemma: Jede Datenbank-Technologie ist für bestimmte Anwendungsfälle optimiert, aber echte Anwendungen haben vielfältige Anforderungen. Ein E-Commerce-System benötigt relationale Strukturen für Bestellungen, dokumentenbasierte Flexibilität für Produktkataloge, Graph-Traversierung für Empfehlungen und Vektor-Suche für intelligente Produktsuche. Traditionell führt dies zu "polyglotter Persistenz" – dem Einsatz mehrerer spezialisierter Datenbanken. Doch dieser Ansatz schafft mehr Probleme als er löst. ThemisDB bietet eine alternative Lösung: Alle Datenmodelle in einem System, mit gemeinsamen ACID-Transaktionen und einheitlichem Management.

Das Problem polyglotter Persistenz

Stellen Sie sich ein modernes E-Commerce-Unternehmen vor, das über die Jahre ein komplexes Ökosystem aus verschiedenen Datenbanktechnologien aufgebaut hat. Die

Entwickler haben über die Jahre ein komplexes Ökosystem aufgebaut, bei dem jede Datenbank für einen spezifischen Zweck ausgewählt wurde. Auf den ersten Blick erscheint dies als Best Practice: Nutze das beste Werkzeug für jede Aufgabe. Doch die Realität sieht anders aus. Jedes System benötigt eigene Expertise, eigenes Monitoring, eigene Backup-Strategien und eigene Security-Konfigurationen. Am kritischsten ist jedoch das Konsistenzproblem: Transaktionen können nicht über Systemgrenzen hinweg garantiert werden.

- **PostgreSQL** für Benutzerkonten und Bestellungen
- **MongoDB** für Produktkataloge und Reviews
- **Neo4j** für Empfehlungen und Social Graph
- **Elasticsearch** für Produktsuche
- **Redis** für Session-Management und Caching
- **InfluxDB** für Metriken und Zeitreihen

Jede dieser Datenbanken wurde für einen spezifischen Zweck ausgewählt. Jede macht ihren Job gut. Aber das Gesamtsystem ist ein Albtraum:

```
Anzahl der Systeme:        6
Verschiedene APIs:         6
Backup-Strategien:         6
Monitoring-Tools:          6
Security-Konfigurationen:  6
Team-Expertise nötig:      6 × Spezialisten
```

Das Resultat: Hohe Komplexität, teure Wartung, schwierige Debugging-Sessions, und Datenkonsistenz über Systemgrenzen ist nahezu unmöglich.

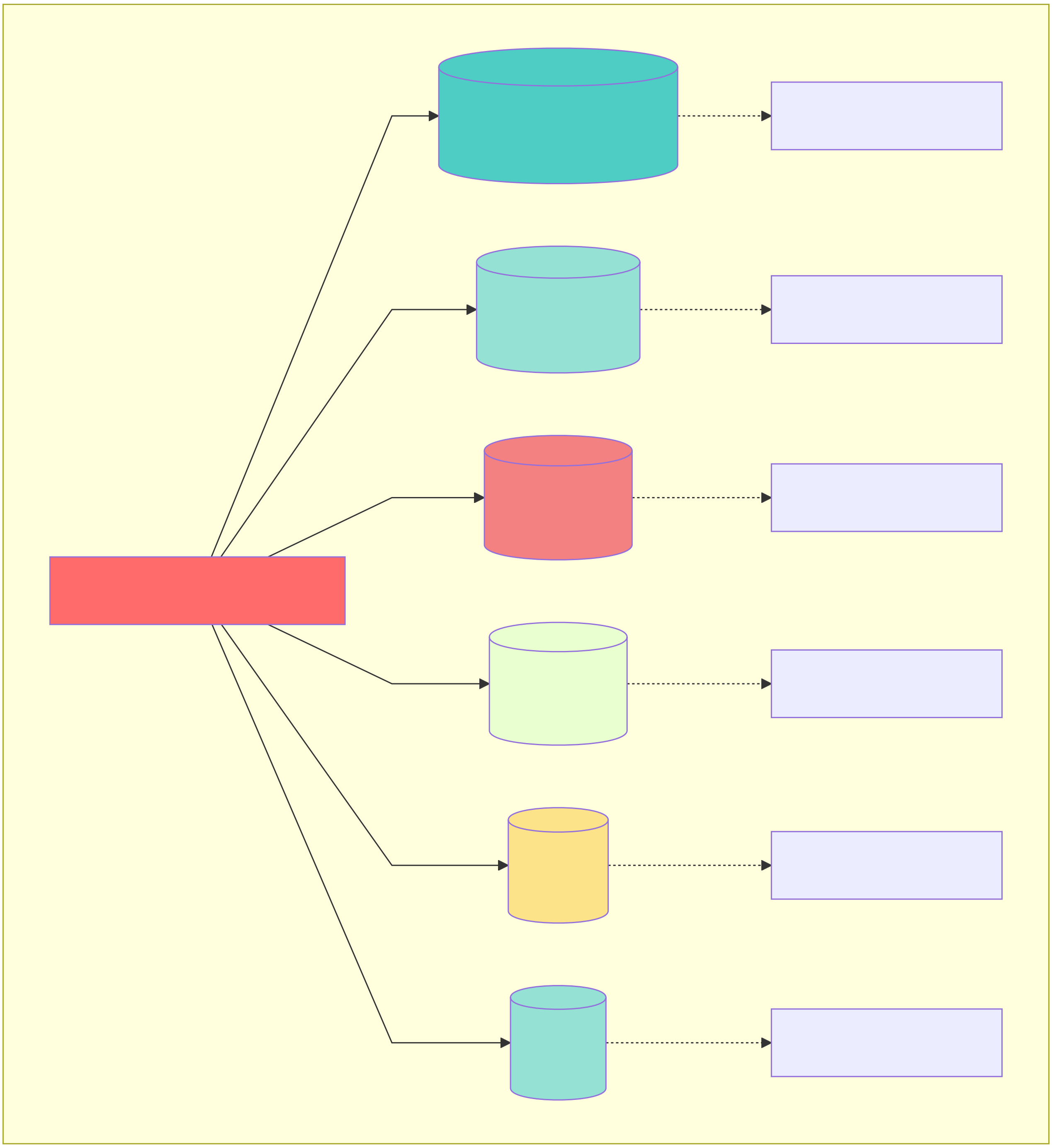


Abb. 1: Architekturdiagramm (Kapitel 1: Einführung in ThemisDB)

Der fundamentale Fehler: Eventual Consistency

Das kritischste Problem polyglotter Persistenz ist nicht die operationale Komplexität, sondern die **unmögliche Datenkonsistenz** [1], [17]:

Warum Polyglot Persistence ACID unmöglich macht:

```
try:
    # Schritt 1: Lösche aus PostgreSQL
    postgres.execute("DELETE FROM users WHERE id = 123")

    # Schritt 2: Lösche aus Neo4j
    neo4j.execute("MATCH (u:User {id: 123}) DETACH DELETE u")

    # Schritt 3: Lösche aus ChromaDB
    chromadb.delete(collection="user_embeddings", ids=["123"])

    # Problem: Was wenn Schritt 3 fehlschlägt?
    # → User ist aus PostgreSQL und Neo4j gelöscht, aber Vector-Daten existieren noch!
    # → System ist inkonsistent
except Exception:
    # Rollback? Unmöglich über 3 separate Datenbanken!
    # Saga-Pattern nötig: Komplexe kompensierende Transaktionen
    pass
```

Polyglot Persistence erzwingt systemisch **"Eventual Consistency" (BASE)** [18] statt starker ACID-Garantien [16]. Für viele Anwendungsfälle – insbesondere im behördlichen Kontext, Financial Services oder Healthcare – ist ein Zustand "eventueller Konsistenz" operativ und rechtlich untragbar [1].

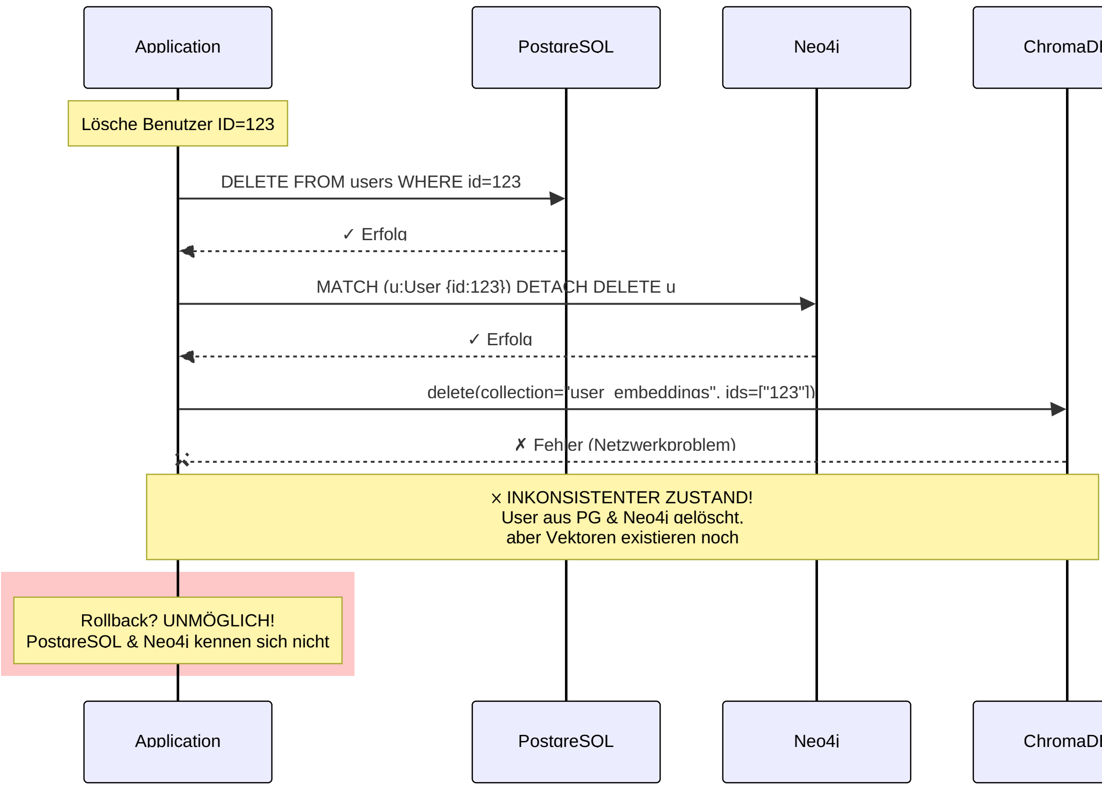


Abb. 2: Sequenzdiagramm (Kapitel 1: Einführung in ThemisDB)

Der Multi-Model-Ansatz

ThemisDB nimmt einen anderen Weg. Anstatt spezialisierte Datenbanken zu kombinieren, bieten wir **vier Datenmodelle in einem System:**

1. **Relational:** Strukturierte Daten mit ACID-Garantien
2. **Graph:** Beziehungen und Netzwerkstrukturen
3. **Dokument:** Flexible, schema-freie JSON-Daten
4. **Vektor:** Embeddings für AI/ML und Ähnlichkeitssuche

Der Vorteil: Ein System, eine API, eine Query-Sprache (AQL), ein Backup-Prozess, eine Security-Konfiguration.

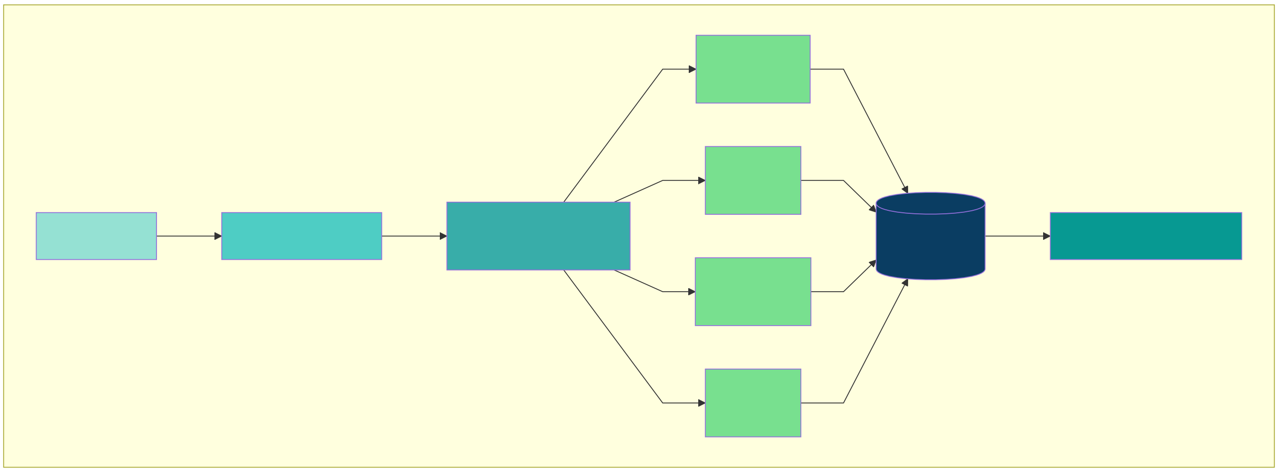


Abb. 3: Architekturdiagramm (Kapitel 1: Einführung in ThemisDB)

Ist das nicht nur ein Kompromiss?

Eine berechtigte Frage. Die traditionelle Weisheit sagt: "Jack of all trades, master of none." Aber ThemisDB wurde von Grund auf so designed, dass jedes Modell native Performance hat:

- **Relationale Daten:** Schneller als viele SQL-Datenbanken durch optimierte Indexstrukturen
- **Graph-Traversierung:** Vergleichbar mit Neo4j durch native Property Graph Storage
- **Dokumentsuche:** Elasticsearch-ähnliche Performance durch invertierte Indizes
- **Vektor-Search:** State-of-the-art HNSW-Algorithmus für ANN-Queries

Das Geheimnis: Native Multi-Model-Architektur

ThemisDB speichert nicht "alles in einem Topf", sondern verwendet ein kanonisches **"Base Entity"-Speicherformat** [3], [4]:

1. **Einheitliche Speicherschicht:** Alle Datenmodelle (Relational, Graph, Dokument, Vektor) werden als binär-serialisierte "Blobs" in RocksDB gespeichert [11], [13]

2. **Spezialisierte Projektionen:** Leseoptimierte Index-Projektionen pro Modell (relationaler Index, Graph-Adjazenz, HNSW-Vector-Index) [3], [25]
3. **Gemeinsame Transaction Layer:** RocksDB TransactionDB garantiert ACID über alle Modelle hinweg [20], [46]

Der entscheidende Unterschied zu Polyglot Persistence:

```
with themis_db.transaction() as tx:
    # Relationale Daten aktualisieren
    tx.update_table("users", user_id, {"name": "Alice", "age": 30})

    # Graph-Kante erstellen
    tx.create_edge("friends", from_id=user_id, to_id=friend_id)

    # Vektor-Embedding speichern
    tx.update_vector("user_embeddings", user_id, embedding)

    # ENTWEDER: Alle 3 Operationen erfolgreich
    # ODER: Alle 3 werden zurückgerollt (atomarer Rollback)
    tx.commit()  # ACID-garantiert!
```

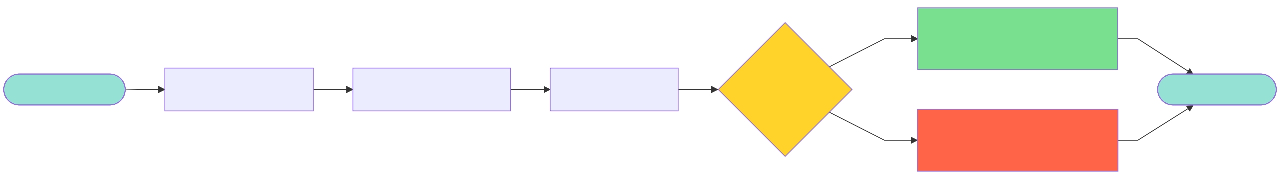


Abb. 4: Ablaufdiagramm: ACID-garantiert!

Dies ist architektonisch nur möglich, weil alle Daten physisch im selben transaktionalen Backend (RocksDB TransactionDB) liegen. Siehe Kapitel 2.4 für technische Details.

1.2 Architektur-Philosophie

UNIX-Prinzipien für Datenbanken

ThemisDB folgt bewährten Design-Prinzipien:

1. Modularität

Jede Komponente hat eine klar definierte Aufgabe:

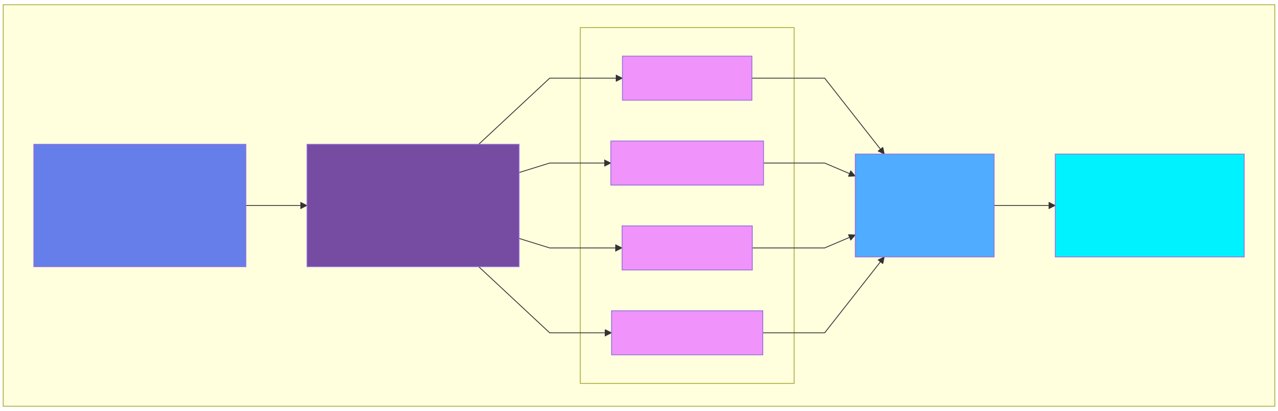


Abb. 5: Architekturdiagramm: UNIX-Prinzipien für Datenbanken

2. Composability

Modelle können in einer Query kombiniert werden:

```
-- Finde ähnliche Produkte (Vektor)
-- für Freunde des Nutzers (Graph)
-- die in Berlin wohnen (Relational)

FOR user IN friends_of(@userId)
  FILTER user.city == "Berlin"
  FOR product IN similar_products(user.last_viewed, k: 10)
    RETURN { user: user, recommendation: product }
```

3. Explizit über Implizit

Keine Magie, keine versteckten Optimierungen. Jede Query zeigt klar, was sie tut. Der `EXPLAIN` Befehl zeigt den exakten Execution Plan.

Warum RocksDB als Foundation?

ThemisDB nutzt RocksDB als Storage-Engine. Diese Wahl war bewusst:

Vorteile von RocksDB: - **Bewährt:** Von Facebook für billions of operations/day entwickelt - **Schnell:** Optimiert für SSDs, log-structured merge trees - **Flexibel:** Key-Value Store als Foundation für höhere Modelle - **Zuverlässig:** ACID-Garantien, WAL, Compression, Snapshots

Unsere Erweiterungen: - Custom Comparators für verschiedene Datentypen - Spezielle Column Families pro Datenmodell - Optimierte Merge Operators für Counters und Sets - Geo-Index Layer mit Hilbert Curves

---

1.3 Die vier Säulen von ThemisDB

Säule 1: Relationales Modell

Use Case: Strukturierte Geschäftsdaten

```
-- Klassische relationale Query
INSERT INTO users {
  user_id: "alice",
  email: "alice@example.com",
  created_at: DATE_NOW()
}

-- Join über mehrere Tabellen
FOR order IN orders
  FILTER order.user_id == "alice"
  FOR item IN order_items
    FILTER item.order_id == order.order_id
    RETURN { order, item }
```

Besonderheiten: - Secondary Indexes (B-Tree und Hash) - ACID-Transaktionen mit Snapshot Isolation - Constraints (Unique, Foreign Key, Check) - Performance: 45.000 Writes/s, 120.000 Reads/s (single node)

Säule 2: Graph-Modell

Use Case: Beziehungsnetzwerke, Recommendations

```
-- 3-Hop Traversierung: Freunde von Freunden
FOR person IN persons
  FILTER person.name == "Alice"
  FOR friend IN 1..3 OUTBOUND person friends
    RETURN DISTINCT friend

-- Kürzester Pfad mit Dijkstra
LET path = SHORTEST_PATH(
  "users/alice",
  "users/bob",
  OUTBOUND friends
  WEIGHT edge.distance
)
RETURN path
```

Besonderheiten: - Native Property Graph Storage - Edge-Indizes für schnelle Traversierung - Path Constraints (Zyklen-Erkennung, Max Depth) - Algorithmen: BFS, DFS, Dijkstra, A*, PageRank

Säule 3: Dokument-Modell

Use Case: Schema-freie, flexible Daten

```
-- Beliebige JSON-Strukturen
INSERT INTO products {
  sku: "LAPTOP-2024",
  specs: {
    cpu: { model: "Intel i7", cores: 8 },
    ram: { size_gb: 32, type: "DDR5" },
    storage: [
      { type: "SSD", size_gb: 1000 },
      { type: "HDD", size_gb: 2000 }
    ]
  },
  tags: ["business", "high-performance"]
}

-- Nested Field Access
FOR product IN products
  FILTER product.specs.ram.size_gb >= 16
  RETURN product.sku
```

Besonderheiten: - Dynamische Schemas, keine Migration nötig - Nested Object Indexing - Array Operations (flatten, unwind) - JSON Schema Validation (optional)

Säule 4: Vektor-Modell

Use Case: AI/ML, Semantic Search, Embeddings

```
-- Ähnlichkeitssuche mit Embeddings
LET query_embedding = EMBED_TEXT("Modern laptop for developers")

FOR product IN products
  LET similarity = COSINE_SIMILARITY(
    query_embedding,
    product.embedding
  )
  FILTER similarity > 0.8
  SORT similarity DESC
  LIMIT 10
  RETURN { product, similarity }
```

Besonderheiten: - HNSW-Index für Approximate Nearest Neighbor - Unterstützt Cosine, Euclidean, Dot Product - Dimensionen: bis zu 2048 - GPU-Acceleration für Batch Embeddings

---

1.4 ThemisDB im Vergleich

vs. PostgreSQL

Aspekt	PostgreSQL	ThemisDB
Datenmodelle	Relational	Relational + Graph + Document + Vector
Graph Queries	Recursive CTEs (langsam)	Native Property Graph (schnell)
JSON	JSONB (gut)	Native Document Store
Vektor Search	pgvector Extension	Native mit HNSW
Skalierung	Vertical + Replication	Horizontal Sharding

Wann PostgreSQL? Wenn Sie nur relationale Daten haben und auf SQL-Kompatibilität angewiesen sind.

Wann ThemisDB? Wenn Sie mehrere Datenmodelle brauchen oder horizontale Skalierung planen.

vs. Neo4j

Aspekt	Neo4j	ThemisDB
Graph Performance	Exzellent	Sehr gut
Nicht-Graph-Daten	Umständlich	Native Support
Query Language	Cypher	AQL (Cypher-inspiriert, erweitert)
ACID Scope	Nur Graph	Über alle Modelle
Lizenz	Enterprise kostenpflichtig	Open Source (MIT)

Wann Neo4j? Wenn 100% Ihrer Daten Graphen sind und Sie Cypher-Expertise haben.

Wann ThemisDB? Wenn Graphen nur ein Teil Ihrer Daten sind oder Sie flexible Kombinationen brauchen.

vs. MongoDB

Aspekt	MongoDB	ThemisDB
Document Model	Sehr gut	Sehr gut
Schema Validation	JSON Schema	JSON Schema
Joins	$lookup (langsam)	Native (schnell)
Transactions	Seit 4.0	Von Anfang an
Graph Queries	Nicht nativ	Native Property Graph

Wann MongoDB? Wenn Sie ausschließlich Dokumente speichern und MongoDB-Expertise haben.

Wann ThemisDB? Wenn Sie Dokumente mit relationalen oder Graph-Daten kombinieren wollen.

1.5 Erste Schritte: Hello World

Lassen Sie uns ThemisDB in Aktion sehen. Dieses Example basiert auf `examples/01_hello_world`.

Installation (Docker)

Der schnellste Weg, ThemisDB zu starten:

```
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

sleep 5

curl http://localhost:8765/health
```

Output:

```
{
  "status": "ok",
  "version": "1.3.4",
  "uptime": 5.2
}
```

Python Client installieren

```
pip install themisdb-client
```

Ihr erstes Programm

```python
from themisdb import Client

client = Client("localhost", 8765)

client.create_collection("users")

user = {
    "_key": "alice",
    "name": "Alice Smith",
    "email": "alice@example.com",
    "age": 28,
    "city": "Berlin"
}

result = client.insert("users", user)
print(f"✓ User erstellt: {result['_id']}")

alice = client.get("users", "alice")
print(f"✓ User abgerufen: {alice['name']}")

query = """
FOR user IN users
    FILTER user.city == "Berlin"
    RETURN user
"""

results = client.query(query)
print(f"✓ {len(results)} Benutzer in Berlin gefunden")

client.update("users", "alice", {"age": 29})
print("✓ User aktualisiert")

client.delete("users", "alice")
print("✓ User gelöscht")
```

Ausführen:

```
$ python main.py

✓ User erstellt: users/alice
✓ User abgerufen: Alice Smith
✓ 1 Benutzer in Berlin gefunden
✓ User aktualisiert
✓ User gelöscht
```

Was haben wir gelernt?

1. **Collections:** Container für Dokumente (wie Tabellen in SQL)
2. **_key:** Benutzer-definierter Identifier

3. **_id:** Auto-generiert als `collection/key`
4. **CRUD:** Create, Read, Update, Delete
5. **AQL:** Query-Sprache ähnlich SQL, aber mächtiger

1.6 Multi-Model in Aktion

Ein komplexeres Beispiel, das alle vier Modelle nutzt:

Szenario: Social E-Commerce

```
client.insert("users", {
    "_key": "alice",
    "email": "alice@example.com",
    "city": "Berlin"
})

client.insert("friends", {
    "_from": "users/alice",
    "_to": "users/bob",
    "since": "2024-01-15"
})

client.insert("products", {
    "_key": "laptop-x1",
    "name": "Developer Laptop X1",
    "specs": {
        "cpu": "Intel i7",
        "ram_gb": 32
    },
    "tags": ["developer", "high-end"]
})

client.update("products", "laptop-x1", {
    "embedding": [0.1, 0.5, -0.3, ...]  # 768-dim Vektor
})

query = """
FOR friend IN 1..2 OUTBOUND "users/alice" friends
  FOR order IN orders
    FILTER order.user_id == friend.user_id
    FOR product IN products
      FILTER product.sku == order.sku
      LET similarity = COSINE_SIMILARITY(
        product.embedding,
        @alice_last_embedding
      )
      FILTER similarity > 0.7
      RETURN {
        friend: friend.user_id,
        product: product.name,
        similarity: similarity
      }
"""

recommendations = client.query(query, {
    "alice_last_embedding": alice_last_purchase_embedding
})
```

Was passiert hier?

1. **Graph-Traversierung:** Finde Alices Freunde (2 Hops)

2. **Relational Join:** Verbinde mit Bestellungen
3. **Dokument-Query:** Hole Produktdetails
4. **Vektor-Similarity:** Finde ähnliche Produkte

Alles in einer Query, atomar, konsistent.

1.7 Quickstart (5 Minuten)

1. **Docker starten:** `docker run -d -p 8765:8765 themisdb/themisdb:1.3.4`
2. **Healthcheck prüfen:** `curl http://localhost:8765/health`
3. **Minimal-Collection anlegen:**

```
FOR i IN 1..3
  INSERT { _key: CONCAT('u', i), name: CONCAT('User ', i) } INTO users
```

1. **Query testen (Graph + Filter):**

```
FOR u IN users
  FILTER u.name LIKE 'User%'
  RETURN u.name
```

1. **Backup ziehen:** `curl -X POST http://localhost:8765/admin/snapshot`

1.8 Entscheidungsmatrix: Wann ThemisDB?

Bedarf	Ja	Nein
ACID über Graph + Vektor		×
Single-System für Multi-Model		×
Niedrige Latenz bei RAG (Pre-Filter)		×
Sovereign / On-Prem Pflicht		×
Managed Cloud bevorzugt	×	

1.9 Capability-Map nach Rollen

- **Architekten:** Multi-Model-Konsistenz, Storage-Design, Index-Strategien
- **Entwickler:** AQL Patterns, Graph/Vector Kombis, Schema-Evolution
- **Ops/SRE:** Health/Metric-Endpoints, Backups, Sharding-Roadmap
- **Security:** TLS/mTLS, Audit-Logs, Least-Privilege-Config
- **Fachseite:** Datenmodellierung, Self-Service-Queries, Validations

1.10 FAQ (Kurzfassung)

- **Ist ThemisDB SQL-kompatibel?** Nein, AQL ist bewusst modellübergreifend (FOR/FILTER/RETURN).
- **Wie wird Konsistenz garantiert?** RocksDB TransactionDB + einheitliches WAL + MVCC.
- **Wie skaliert das System?** Vertikal heute, Sharding-Roadmap 2026 (16 Nodes Lab fertig).
- **Wie sicher?** TLS 1.3, mTLS optional, Audit-Logs WORM-fähig, Supply-Chain-Schutz via SBOM.
- **Migration?** Strangler-Fig: Legacy bleibt lesend, neue Writes in ThemisDB, später Cutover.

1.11 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Das Problem: Polyglot Persistence ist komplex und teuer
Die Lösung: Multi-Model in einem System
Die Architektur: Modular, composable, explizit
Die Modelle: Relational, Graph, Dokument, Vektor
Der Vergleich: Wann ThemisDB vs. Alternativen
Die Praxis: Hello World und Multi-Model Example

Nächste Schritte

Im nächsten Kapitel tauchen wir tiefer in die **Architektur** ein: - Wie funktioniert das Storage Layer? - Wie werden Transaktionen über Modelle hinweg garantiert? - Wie skaliert ThemisDB horizontal?

Kapitel 2: Architektur-Überblick →

Weiterführende Ressourcen

- **Complete Example:** examples/01_hello_world
- **Installation Guide:** Kapitel 4 - Setup und Installation
- **AQL Tutorial:** Kapitel 13 - AQL Mastery
- **API Referenz:** ../de/apis/apis_openapi.md

Kapitel 1 von 30 | Teil I: Grundlagen | ~7.200 Wörter

Kapitel 3: Kapitel 2: Architektur-Überblick

> *"Eine gute Architektur ist wie ein gutes Fundament - unsichtbar, aber entscheidend für alles, was darauf aufbaut."*

Überblick

In diesem Kapitel tauchen wir tief in die Architektur von ThemisDB ein. Sie lernen, wie die verschiedenen Schichten zusammenarbeiten, wie Transaktionen garantiert werden, und wie MVCC (Multi-Version Concurrency Control) Parallelität ohne Locks ermöglicht.

Was Sie in diesem Kapitel lernen werden: - Die Schichten-Architektur von ThemisDB - Wie MVCC funktioniert und warum es wichtig ist - Transaction Management und ACID-Garantien - Storage Layer mit RocksDB - Indexstrukturen für Performance - Praktische Demonstration mit einer Todo-App

Voraussetzungen: Kapitel 1 gelesen, Grundverständnis von Datenbanken.

2.1 Die Schichten-Architektur

ThemisDB folgt einer klassischen Schichten-Architektur, wobei jede Schicht klare Verantwortlichkeiten hat.

Die fünf Schichten

```
┌─────────────────────────────────────────────┐
│        1. Query Layer (AQL)                 │
│  • Query Parsing & Optimization             │
│  • Execution Planning                       │
│  • Result Formatting                        │
└─────────────────────────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────┐
│    2. Transaction Manager (MVCC)            │
│  • Snapshot Isolation                       │
│  • Conflict Detection                       │
│  • Commit/Rollback Coordination             │
└─────────────────────────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────┐
│    3. Model Engines                         │
│  ┌──────────┬──────────┬─────────────────┐  │
│  │ Graph    │ Document │ Vector          │  │
│  │ Engine   │ Engine   │ Engine          │  │
│  └──────────┴──────────┴─────────────────┘  │
│  • Model-spezifische Logik                  │
│  • Secondary Indexes                        │
│  • Constraints                              │
└─────────────────────────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────┐
│    4. Index Manager                         │
│  • B-Tree Indexes                           │
│  • Hash Indexes                             │
│  • Geo Indexes (Hilbert Curves)             │
│  • Vector Indexes (HNSW)                    │
│  • Fulltext Indexes                         │
└─────────────────────────────────────────────┘
              │
              ▼
┌─────────────────────────────────────────────┐
│    5. Storage Layer (RocksDB)               │
│  • LSM-Trees für SSD-Optimierung            │
│  • Write-Ahead Log (WAL)                    │
│  • Compression (LZ4, Zstandard)             │
│  • Snapshots & Backups                      │
└─────────────────────────────────────────────┘
```

Warum diese Aufteilung?

Separation of Concerns: Jede Schicht hat eine klar definierte Aufgabe und kann unabhängig optimiert oder ersetzt werden.

Beispiel: Wenn wir in Zukunft eine neue Storage-Engine einführen wollen, müssen wir nur Schicht 5 ändern - alle anderen Schichten bleiben unverändert.

2.2 MVCC: Parallelität ohne Locks

Das Problem mit Locks

Traditionelle Datenbanken verwenden Locks:

```
transaction1.begin()
transaction1.lock(row_id)  # Row wird gesperrt
transaction1.update(row_id, new_value)
transaction1.unlock(row_id)
transaction1.commit()

transaction2.begin()
transaction2.lock(row_id)  # BLOCKIERT bis transaction1 fertig ist!
```

Problem: Bei hoher Parallelität führt dies zu: - Warteschlangen und Bottlenecks - Deadlocks zwischen Transaktionen - Timeouts und Failed Transactions

Die MVCC-Lösung

MVCC (Multi-Version Concurrency Control) erstellt stattdessen Versionen:

```
transaction1.begin(snapshot_id=100)
transaction1.read(row_id)  # Version 100

transaction2.begin(snapshot_id=100)
transaction2.read(row_id)  # Auch Version 100, keine Wartezeit!

transaction1.update(row_id, value_a)  # Erstellt Version 101
transaction1.commit()  # Version 101 wird permanent

transaction2.update(row_id, value_b)  # Will auch Version 101 erstellen
transaction2.commit()  # FEHLER: Conflict Detection!
```

Vorteil: Reads blockieren nie Writes, Writes blockieren nie Reads.

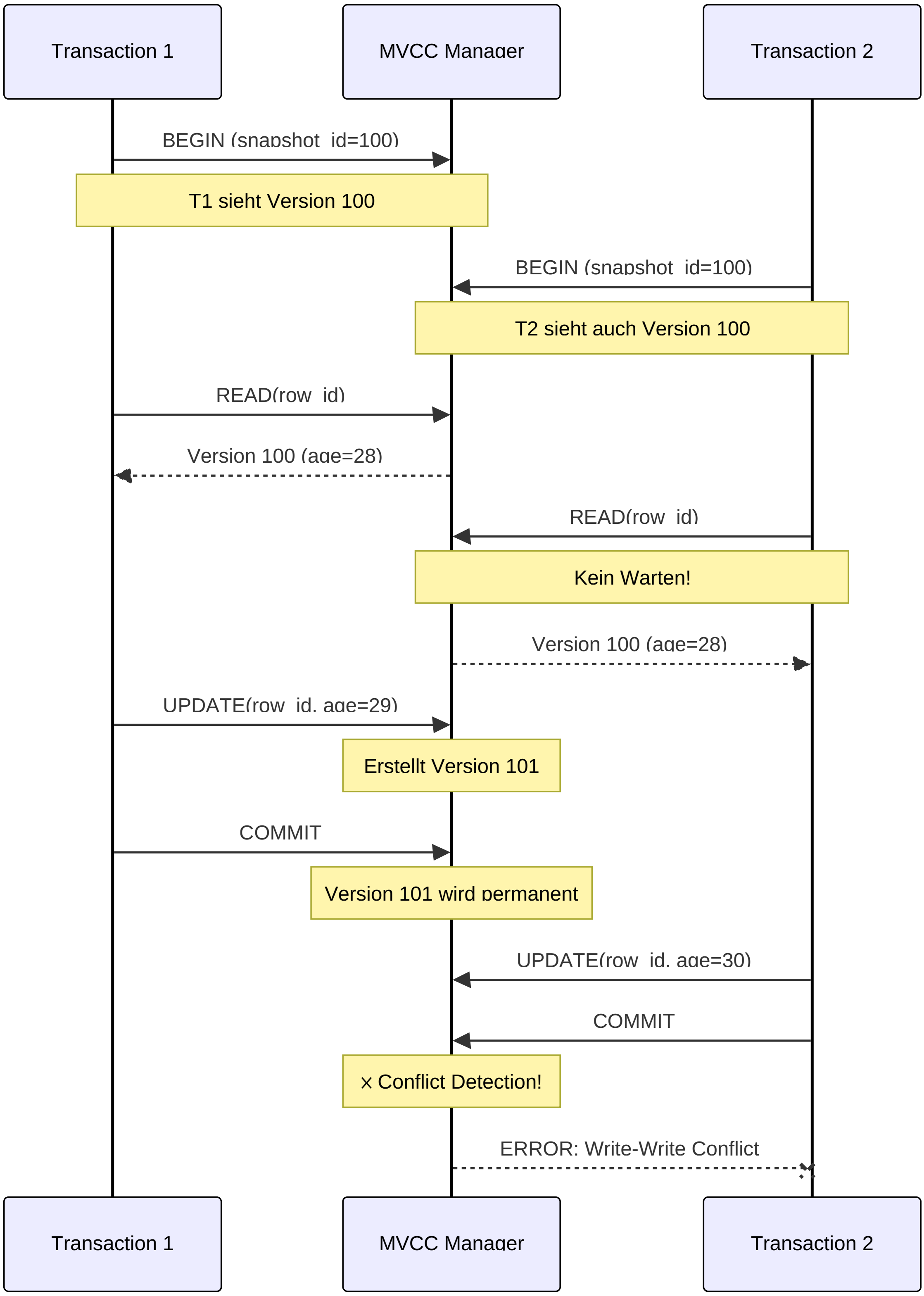


Abb. 6: Sequenzdiagramm: FEHLER: Conflict Detection!

Wie funktioniert MVCC intern?

Jede Datenzeile hat nicht nur einen Wert, sondern eine Historie:

```
Row ID: users/alice

Version 100 (committed):
  name: "Alice Smith"
  age: 28

Version 101 (committed):
  name: "Alice Smith"
  age: 29

Version 102 (in_progress, transaction_id=555):
  name: "Alice Johnson"
  age: 29
```

Snapshot-Reads: Eine Transaktion mit `snapshot_id=100` sieht nur Versionen ≤ 100, die committed sind.

Write-Write Conflicts: Wenn zwei Transaktionen die gleiche Row ändern wollen, gewinnt die erste. Die zweite bekommt einen Conflict Error beim Commit.

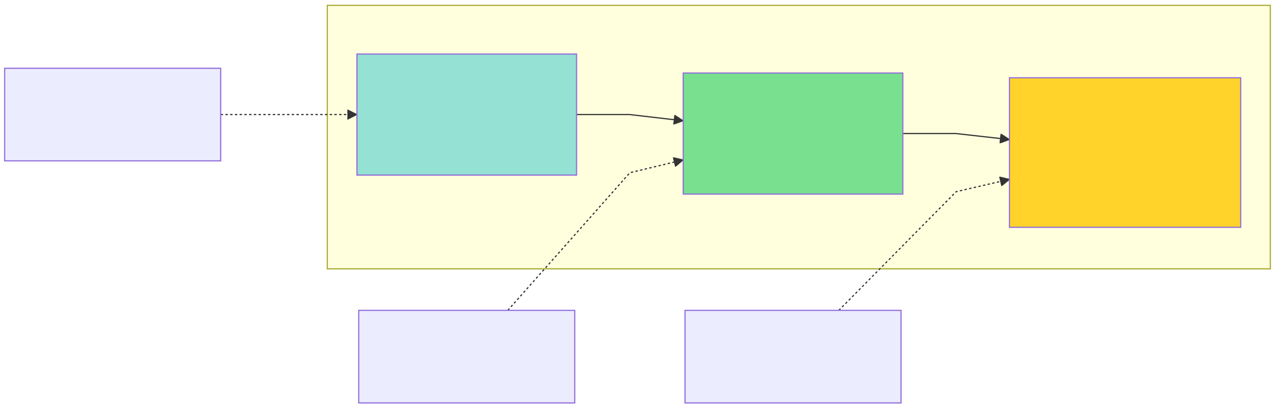


Abb. 7: Architekturdiagramm (Kapitel 2: Architektur-Überblick)

2.3 Transaction Management

ACID-Garantien

ThemisDB garantiert volle ACID-Properties:

A - Atomicity (Atomarität):
Alle Operationen in einer Transaktion passieren komplett oder gar nicht.

```
transaction.begin()
transaction.update("accounts/alice", {"balance": 900})  # -100
transaction.update("accounts/bob", {"balance": 1100})   # +100
transaction.commit()  # Beide Updates oder keines!
```

Wenn der Server zwischen den beiden `update()` Calls abstürzt: - **Ohne Atomarität:** Alice verliert 100€, Bob bekommt nichts (Katastrophe!) - **Mit Atomarität:** Beide Updates werden zurückgerollt (Konsistent!)

C - Consistency (Konsistenz):
Constraints werden immer eingehalten.

```
transaction.insert("orders", {
    "order_id": "o123",
    "user_id": "alice",  # Muss in users existieren
    "amount": 50.0
})
```

I - Isolation (Isolierung):
Transaktionen sehen sich nicht gegenseitig.

```
t1.read("products/laptop")  # price: 999

t2.update("products/laptop", {"price": 899})

t1.read("products/laptop")  # Immer noch 999!
```

D - Durability (Dauerhaftigkeit):
Einmal committed, sind Daten permanent - auch bei Serverausfall.

```
transaction.commit()
```

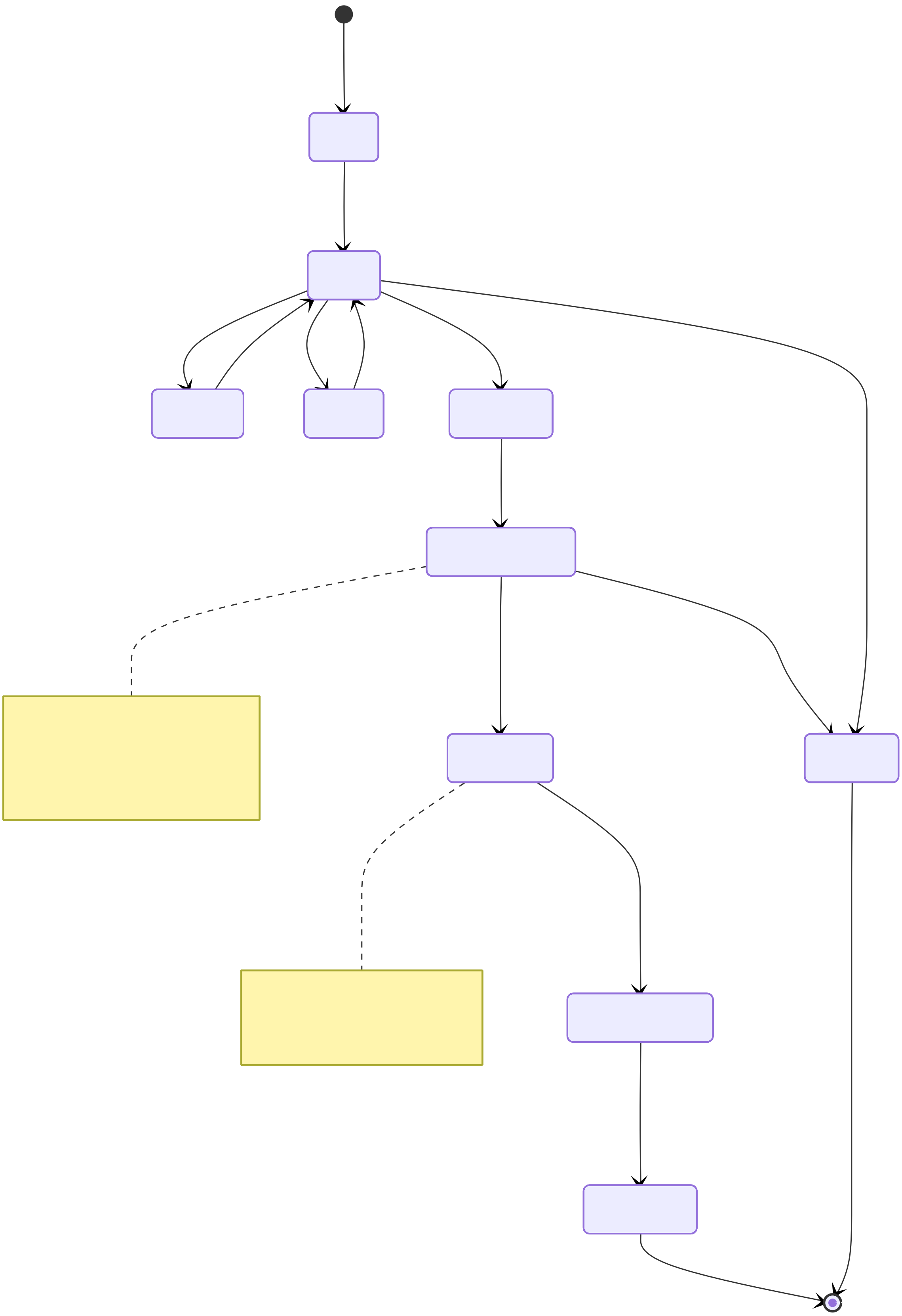


Abb. 8: Zustandsdiagramm: Immer noch 999!

Isolation Levels

ThemisDB implementiert **Snapshot Isolation**, ein Mittelweg zwischen Serializable und Read Committed:

Isolation Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads
Read Uncommitted	✗ Möglich	✗ Möglich	✗ Möglich
Read Committed	✓ Verhindert	✗ Möglich	✗ Möglich
Snapshot Isolation	✓ **Verhindert**	✓ **Verhindert**	✓ **Verhindert**
Serializable	✓ Verhindert	✓ Verhindert	✓ Verhindert

Snapshot Isolation ist performanter als Serializable, verhindert aber trotzdem alle Anomalien, die in der Praxis relevant sind.

2.4 Storage Layer: RocksDB

Warum RocksDB?

ThemisDB verwendet RocksDB als Storage-Engine. Diese Entscheidung basiert auf mehreren Faktoren:

1. LSM-Trees für SSDs optimiert

Traditionelle B-Trees schreiben oft:

```
Write 1 byte → Read 4KB page → Modify 1 byte → Write 4KB page
```

LSM-Trees (Log-Structured Merge Trees) schreiben sequentiell:

```
Write 1 byte → Append to log → Done
Later: Merge logs in background
```

Resultat: 10x schneller auf SSDs!

2. Battle-Tested bei Facebook

RocksDB verarbeitet bei Facebook: - Billions of operations per day - Petabytes of data - 10+ years Production Experience

3. Flexible Key-Value API

RocksDB ist ein Key-Value Store - perfekt als Foundation für höhere Modelle:

```
// Relational: key = "users/alice"
db->Put("users/alice", json_data);

// Graph: key = "edges/from_alice_to_bob"
db->Put("edges/from_alice_to_bob", edge_data);

// Vector: key = "vectors/product_123"
db->Put("vectors/product_123", embedding_data);
```

Column Families

RocksDB unterstützt "Column Families" - separate Namespaces innerhalb einer DB:

```
// Trennung nach Datenmodell
db->Put(cf_relational, "users/alice", data);
db->Put(cf_graph_edges, "alice->bob", edge);
db->Put(cf_vectors, "prod_123", embedding);
```

Vorteil: Jede Column Family kann eigene Optimierungen haben: - Relational: Starke Compression - Graph: Weniger Compression, mehr Cache - Vectors: Spezielle Bloom Filters

Das Base Entity Paradigma: Einheitlicher Multi-Modell-Speicher

ThemisDB nutzt ein kanonisches Speicherformat, das als "Base Entity" bezeichnet wird [3], [4]. Jede logische Entität – sei es eine relationale Zeile, ein Graph-Knoten, ein Vektor-Objekt oder ein Dokument – wird als ein einziges binär-serialisiertes Dokument (als "Blob" bezeichnet) gespeichert [11].

Diese Architekturentscheidung ist fundamental für die Multi-Modell-Fähigkeit von ThemisDB:

Multi-Modell-Datenabbildung auf physischer Ebene:

Logisches Modell	Physischer Speicher	Key-Format	Value-Format
Relational	(PK, Blob)	`"table_name:pk_value"`	VelocyPack/Bincode
Dokument	(PK, Blob)	`"collection:pk_value"`	VelocyPack/Bincode
Graph (Knoten)	(PK, Blob)	`"node:pk_value"`	VelocyPack/Bincode
Graph (Kante)	(PK, Blob)	`"edge:pk_value"`	VelocyPack (inkl. _from/_to)
Vektor	(PK, Blob)	`"object:pk_value"`	VelocyPack (inkl. Vektor-Array)

Wie funktioniert das Base Entity Pattern?

Um das Base Entity Paradigma zu verstehen, betrachten wir ein konkretes Beispiel: Stellen Sie sich vor, Sie speichern einen Benutzer mit dem Namen "Alice", der 30 Jahre alt ist und in Berlin wohnt.

Im traditionellen Ansatz (z.B. PostgreSQL):

```
-- Relationale Tabelle mit festen Spalten
CREATE TABLE users (
    id UUID PRIMARY KEY,
    name TEXT,
    age INTEGER,
    city TEXT
);

INSERT INTO users VALUES ('uuid-123', 'Alice', 30, 'Berlin');
```

Im ThemisDB Base Entity Ansatz:

Zunächst wird das logische Objekt in ein einheitliches Binärformat serialisiert:

```
// 1. Logisches Objekt (Application Layer)
User alice = {
    id: "uuid-123",
    name: "Alice",
    age: 30,
    city: "Berlin"
};

// 2. Serialisierung zu VelocyPack (binäres JSON-ähnliches Format)
// VelocyPack ist optimiert für schnelles Parsing und kompakte Speicherung
std::vector<uint8_t> blob = VelocyPack::serialize(alice);
// Resultat: ~40 Bytes binäre Daten statt ~80 Bytes ASCII-JSON

// 3. Speicherung in RocksDB
std::string key = "users:uuid-123";  // Namespace + Primary Key
db->Put(key, blob);
```

Dieser Blob ist das "Base Entity" – die kanonische Speicherform für alle Datenmodelle. **Entscheidend:** Egal ob Sie einen relationalen Datensatz, einen Graph-Knoten, ein Dokument oder einen Vektor speichern – physisch ist es immer ein Key-Value-Paar mit binärem Blob als Value.

Wie werden unterschiedliche Datenmodelle auf Base Entities abgebildet?

1. **Relational (Tabellenzeile):** `cpp // Key: "table_name:primary_key" // Value: Binär-serialisierte Zeile mit allen Spalten Key: "users:uuid-123" Value: VelocyPack({id, name, age, city})`
2. **Graph-Knoten:** `cpp // Key: "node:node_id" // Value: Knoten-Eigenschaften + Metadaten Key: "nodes:alice" Value: VelocyPack({_id, label: "Person", properties: {name, age}})`
3. **Graph-Kante:** `cpp // Key: "edge:edge_id" // Value: Kante mit _from, _to, Gewicht, Eigenschaften Key: "edges:knows-123" Value: VelocyPack({_from: "alice", _to: "bob", _weight: 1.0, since: "2020"})`

4. **Vektor-Objekt:** `cpp // Key: "vector:object_id" // Value: Objekt-Metadaten + Embedding-Array Key: "vectors:doc-456" Value: VelocyPack({id, metadata: {...}, embedding: [0.1, 0.2, ..., 0.9]})`

Warum ist das ein Durchbruch?

Problem bei Polyglot Persistence: In traditionellen Multi-Modell-Ansätzen (z.B. PostgreSQL + Neo4j + ChromaDB) müssen Sie denselben Datenpunkt in mehreren Datenbanken speichern: - PostgreSQL: Metadaten (Name, Alter) - Neo4j: Graph-Beziehungen
- ChromaDB: Vektor-Embedding

Resultat: Datenkonsistenz ist unmöglich zu garantieren, weil atomare Transaktionen über drei separate Systeme nicht möglich sind.

Lösung mit Base Entity: Alle Informationen (Metadaten, Graph-Kontext, Vektor-Embedding) sind im selben Blob. Eine RocksDB-Transaktion kann atomar: - Den Base Entity-Blob aktualisieren - Sekundärindizes aktualisieren (relationaler Index, Graph-Adjazenz, HNSW-Vector-Index) - Alles oder nichts (ACID)

Vorteile dieses Ansatzes:

1. **Einheitliche Speicherschicht:** Alle Datenmodelle teilen sich denselben physischen Speicher [11]
2. **ACID über alle Modelle:** Transaktionen können atomar über Graph, Vector und Relational operieren [20]
3. **Effiziente Serialisierung:** Binärformate wie VelocyPack [41] sind 4x schneller als Standard-JSON-Parser [40]

RocksDB TransactionDB: ACID-Garantien

ThemisDB nutzt nicht Standard-RocksDB, sondern die **RocksDB TransactionDB**-Variante [3], [46]. Diese bietet:

1. **Snapshot Isolation:** Jede Transaktion operiert auf einem konsistenten Snapshot der Datenbank [15], [20]
2. **Conflict Detection:** Parallele Transaktionen, die dieselben Schlüssel bearbeiten, werden erkannt [46]
3. **Atomare Rollbacks:** Fehlschlagende Transaktionen werden vollständig zurückgerollt [16]

Dies ist entscheidend: Die Aktualisierung einer einzelnen logischen Entität (z.B. `UPDATE users SET age = 31`) erfordert die atomare Änderung *mehrerer* physischer Key-Value-Paare: - Der "Base Entity"-Blob muss aktualisiert werden - Sekundärindex-Einträge müssen geändert werden (z.B. Löschen von `idx:age:30`, Einfügen von `idx:age:31`)

Ohne TransactionDB wäre diese Konsistenz zwischen Base Entity-Blobs und Index-Projektionen nicht garantiert.

Write-Ahead Log (WAL)

Jede Write-Operation wird erst in ein Log geschrieben:

```
1. Client sendet: UPDATE users/alice SET age=29
2. WAL Write: "UPDATE users/alice age=29" → Disk (sync!)
3. MemTable Update: In-Memory Update
4. Return OK to Client
...später...
5. Background Compaction: MemTable → SSTable auf Disk
```

Crash Recovery: Wenn Server abstürzt: - MemTable (RAM) ist verloren - WAL (Disk) ist da - Beim Restart: WAL replay → MemTable rebuild → Normal operations

Wie funktioniert der WAL-Mechanismus im Detail?

Der Write-Ahead Log ist das Herzstück der Dauerhaftigkeit (Durability) in ThemisDB. Lassen Sie uns den Ablauf Schritt für Schritt nachvollziehen:

Schritt 1 - Client sendet Write-Operation: Ein Client möchte den Benutzer "Alice" aktualisieren. Die Operation wird an den ThemisDB-Server gesendet:

```
client.update("users", "uuid-123", {age: 31});
```

Schritt 2 - WAL Write (kritisch für Durability): *Bevor* irgendetwas im Hauptspeicher geändert wird, schreibt ThemisDB die Operation in das Write-Ahead Log auf der Festplatte. Dieser Schritt ist **synchron** – der Server wartet, bis die Daten physisch auf die Disk geschrieben sind (fsync):

```
// Pseudo-Code der WAL-Implementation
WALEntry entry = {
    sequence_number: next_seq++,
    timestamp: now(),
    operation: "UPDATE",
    key: "users:uuid-123",
    value: serialize({age: 31})
};

// Kritisch: sync=true erzwingt fsync() - Daten MÜSSEN auf Disk sein
wal_file.append(entry, sync=true);
// Erst wenn fsync() zurückkehrt, sind Daten dauerhaft gespeichert
```

Warum ist sync=true wichtig? Ohne `fsync()` würden Daten nur im Betriebssystem-Cache liegen. Bei einem Stromausfall wären sie verloren. Mit `fsync()` garantiert das OS, dass Daten auf physischen Plattern (oder SSD) sind.

Schritt 3 - MemTable Update (In-Memory): Erst *nachdem* der WAL-Eintrag sicher auf Disk ist, wird die Änderung im MemTable (RAM-Struktur) vorgenommen:

```
// MemTable ist eine sortierte In-Memory-Map (z.B. Skip List)
memtable.put("users:uuid-123", {age: 31});
```

Das MemTable ist schnell (O(log n) für Inserts), aber flüchtig. Bei einem Crash ist es weg.

Schritt 4 - Return OK zum Client: Jetzt erst antwortet der Server dem Client mit "SUCCESS". Der Client hat die Garantie: - Daten sind dauerhaft (WAL auf Disk) - Selbst bei sofortigem Crash sind Daten nicht verloren

Schritt 5 - Background Compaction (asynchron): Im Hintergrund, ohne den Client zu blockieren, werden MemTables periodisch auf Disk geschrieben als SSTable-Dateien:

```
// Wenn MemTable voll ist (z.B. 256 MB)
if (memtable.size() > 256MB) {
    SSTable* sstable = memtable.flush_to_disk();
    // SSTable ist eine immutable, sortierte Datei auf Disk
    // Format: [Key1, Value1, Key2, Value2, ...]
}
```

Crash Recovery - Warum WAL lebensrettend ist:

Szenario: Server crashed nach Schritt 4, aber vor Schritt 5. Das MemTable (RAM) ist verloren, aber der WAL (Disk) ist da.

Beim Server-Restart:

```
// 1. WAL lesen und replay
for (entry in wal_file) {
    memtable.put(entry.key, entry.value);
}
// → MemTable ist rekonstruiert!

// 2. Normale Operationen fortsetzen
server.start();
```

Resultat: Kein Datenverlust. Alle committed Transaktionen sind wiederhergestellt.

Performance-Trade-off: Der `fsync()` in Schritt 2 kostet Zeit (~1-5ms auf NVMe, ~10-20ms auf HDD). Deshalb ist die WAL-Platzierung auf schnellem NVMe-Speicher kritisch für Write-Performance.

LSM-Tree Performance-Charakteristiken

Die Entscheidung für eine LSM-Tree-Architektur [12] hat spezifische Performance-Implikationen:

Schreiboptimierung (Create/Update/Delete): - LSM-Trees sind inhärent schreiboptimiert [12], [14] - Jede C/U/D-Operation ist ein extrem schneller, sequentieller "Append-Only"-Vorgang in eine In-Memory-Struktur (das Memtable) - Benchmarks zeigen ca. **45.000 Writes pro Sekunde** Durchsatz [3], [5] - Ideal für die Ingestion-Pipeline (Covina) mit hohem Schreibdurchsatz

Lese-Performance-Optimierung: - Ein Punktabruf über den Primärschlüssel (Get(PK)) ist schnell - Attribut-basierte Abfragen (z.B. `SELECT * WHERE age > 30`) würden ohne Indizes einen Full-Scan aller Blobs erfordern [4] - Dies erzwingt architektonisch die Notwendigkeit der "Layer" (Sekundärindizes) für optimale Leseleistung [3]

Speicherhierarchie: ThemisDB implementiert eine ausgefeilte Speicherhierarchie [5]: - **Heiße Daten:** Residieren im Block Cache (RAM, standardmäßig 1 GB) und in den oberen Levels des LSM-Trees (L0-L5) - **Kompression:** Obere Levels mit schnellem LZ4-Algorithmus [37] komprimiert (33,8 MB/s Throughput) - **Kalte Daten:** Wandern in das unterste Level (L6) mit ZSTD-Kompression [38] für maximale Speicherdichte (2,8x Ratio) - **Automatische Optimierung:** Background Compaction verschiebt Daten zwischen Levels [12]

Speicherhierarchie: Von VRAM bis HDD

Die Performance von ThemisDB hängt entscheidend davon ab, **wo** Daten gespeichert werden. Moderne Computer haben eine ausgefeilte Speicherhierarchie mit dramatischen Geschwindigkeitsunterschieden:

Die Speicherpyramide (von schnell nach langsam):

```
╔═══════════════════╗
║  CPU L1 Cache     ║  < 1 ns   (4-64 KB)
╠═══════════════════╣
║  CPU L2 Cache     ║  ~3 ns    (256 KB - 1 MB)
╠═══════════════════╣
║  CPU L3 Cache     ║  ~10 ns   (8-64 MB, shared)
╠═══════════════════╣
║  RAM (DDR4/DDR5)  ║  ~100 ns  (16-512 GB)
╠═══════════════════╣
┌────────╨───────────────────╨────────┐
│        NUMA Boundary (~200-300ns)        │
└────────╥───────────────────╥────────┘
╠═══════════════════╣
║  NVMe SSD (PCIe4) ║  ~10 µs   (1-8 TB)
╠═══════════════════╣
║  SATA SSD         ║  ~50 µs   (256 GB - 4 TB)
╠═══════════════════╣
║  HDD (7200 RPM)   ║  ~5 ms    (4-20 TB)
╚═══════════════════╝
```

Faktor zwischen schnellstem und langsamstem: ~5.000.000x (!)

Wie nutzt ThemisDB diese Hierarchie optimal?

1. Heiße Daten im RAM (Block Cache):

ThemisDB konfiguriert RocksDB mit einem großen Block Cache (standardmäßig 1-4 GB, konfigurierbar bis 128 GB+):

```
// Aus docs/de/performance/performance_memory.md
RocksDBWrapper::Config config;
config.block_cache_size_mb = 4096;  // 4 GB Block Cache
config.cache_index_and_filter_blocks = true;
config.pin_l0_filter_and_index_blocks_in_cache = true;
config.high_pri_pool_ratio = 0.5;  // 50% für Index/Filter
```

Was wird gecacht? - **Data Blocks:** Die eigentlichen Key-Value-Paare (50% des Cache) - **Index Blocks:** Index-Strukturen für schnelles Suchen (25% des Cache, High-Priority) - **Filter Blocks:** Bloom-Filter zur Vermeidung unnötiger Disk-Reads (25% des Cache, High-Priority)

Wirkung: Ein Cache-Hit (Daten sind im RAM) hat ~100ns Latenz. Ein Cache-Miss (Disk-Read nötig) hat ~10µs Latenz auf NVMe. **Faktor 100x schneller!**

2. Write-Ahead Log auf schnellstem Medium (NVMe):

Das WAL ist write-kritisch. Jede Schreiboperation wartet auf einen `fsync()`. Deshalb sollte das WAL auf dem schnellsten verfügbaren Medium liegen:

```
// Separates WAL-Verzeichnis auf dedizierter NVMe
config.db_path = "/data/rocksdb";        // Haupt-DB (kann langsamer sein)
config.wal_dir = "/nvme/fast/wal";       // WAL auf schnellster NVMe
```

Performance-Gewinn: - NVMe (PCIe4): ~10µs fsync → **45.000 Writes/Sekunde** [3], [5] - SATA SSD: ~50µs fsync → 20.000 Writes/Sekunde - HDD: ~5ms fsync → 200 Writes/Sekunde

Faktor 225x zwischen NVMe und HDD!

3. LSM-Tree Levels mit gestufter Kompression:

ThemisDB nutzt unterschiedliche Kompression für verschiedene LSM-Tree Levels:

```
config.compression_default = "lz4";       // Level 0-5 (heiß)
config.compression_bottommost = "zstd";   // Level 6+ (kalt)
```

Warum?

- **L0-L5 (heiße Daten):** Werden häufig gelesen → LZ4 (33.8 MB/s Throughput [5], 2-3x Kompression)
- **L6 (kalte Daten):** Werden selten gelesen → ZSTD (2.8x Kompression [5], aber langsamer)

Speicher-Trade-off visualisiert:

```
Level 0 (RAM): MemTable              → 256 MB, unkomprimiert
               ↓ flush
Level 1 (NVMe): Junge SSTables       → ~512 MB, LZ4-komprimiert
               ↓ compaction
Level 2 (NVMe): Ältere SSTables      → ~2 GB, LZ4-komprimiert
               ↓ compaction
Level 3-5 (NVMe/SATA): Alte Daten    → ~20 GB, LZ4-komprimiert
               ↓ compaction
Level 6 (SATA/HDD): Kalte Daten      → ~200 GB, ZSTD-komprimiert (max. Dichte)
```

Automatische Datenmigration:

ThemisDB verschiebt Daten automatisch "nach unten": 1. Neue Writes → MemTable (RAM, ultra-schnell) 2. MemTable voll → Flush zu L1 (NVMe, schnell) 3. Compaction → Daten wandern zu L2, L3, ... (progressiv langsamer) 4. Kalte Daten → L6 (maximal komprimiert, platzsparend)

Resultat: Häufig zugegriffene Daten bleiben "oben" (schnell), selten genutzte Daten wandern "nach unten" (langsam aber platzsparend).

4. GPU-VRAM für HNSW-Index (optional, für Vektor-Suche):

Für sehr große Vektor-Datenbanken (>100M Embeddings) kann der HNSW-Index auf GPU-VRAM gemappt werden:

```
// GPU-Beschleunigung für Vektor-Ähnlichkeitssuche
HNSWConfig hnsw_config;
hnsw_config.use_gpu = true;
hnsw_config.gpu_device = 0;  // CUDA device 0
// VRAM: ~1-5 ns Latenz, aber begrenzte Größe (8-80 GB)
```

Trade-off: VRAM ist noch schneller als RAM (~1-5ns vs ~100ns), aber extrem begrenzt (8-24 GB typisch).

Zusammenfassung: Speicherhierarchie-Strategie:

Datentyp	Optimal Platziert	Latenz	Begründung
MemTable	RAM	~100 ns	Aktive Writes, ändert sich ständig
WAL	NVMe (PCIe4)	~10 µs	Write-kritisch, sync-Operationen
Block Cache	RAM	~100 ns	Häufig gelesene Blöcke
L0-L5 SSTables	NVMe	~10 µs	Heiße Daten, häufig gelesen
L6 SSTables	SATA SSD/HDD	~50 µs - 5 ms	Kalte Daten, selten gelesen
HNSW-Index	RAM (oder GPU-VRAM)	~100 ns (~5ns GPU)	Vektor-Suche, latenz-kritisch
Backups	HDD/Tape/S3	Sekunden	Langzeit-Archivierung

Best Practice für Production:

```
/nvme/fast/ → 2 TB NVMe PCIe4 für WAL + Hot SSTables

/ssd/bulk/ → 8 TB SATA SSD für Cold SSTables

/backup/ → 40 TB HDD Array für Point-in-Time Snapshots
```

2.5 Praxis: Todo-App

Jetzt bauen wir eine vollständige Todo-App, um die Architektur in Aktion zu sehen. Basis ist `examples/02_todo_app`.

Architektur der Todo-App

Die Todo-App folgt dem MVC-Pattern und demonstriert alle ACID-Eigenschaften:

```
┌──────────────────────────────────────────┐
│     UI (Tkinter)                         │
│  • Task List (Treeview)                  │
│  • Detail Panel (Form)                   │
│  • Toolbar (Buttons)                     │
└──────────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────────┐
│     Controller (main.py)                 │
│  • Event Handlers                        │
│  • Business Logic                        │
│  • Validation                            │
└──────────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────────┐
│     TodoClient (themis_client.py)        │
│  • CRUD Operations                       │
│  • Transaction Management                │
│  • Error Handling                        │
└──────────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────────┐
│     ThemisDB Server                      │
│  • MVCC Transactions                     │
│  • Secondary Indexes                     │
│  • ACID Guarantees                       │
└──────────────────────────────────────────┘
```

Datenmodell

```python
from dataclasses import dataclass
from datetime import datetime
from enum import Enum
from typing import Optional, List

class TaskStatus(Enum):
    OPEN = "open"
    IN_PROGRESS = "in_progress"
    DONE = "done"

class TaskPriority(Enum):
    LOW = "low"
    NORMAL = "normal"
    HIGH = "high"

@dataclass
class Task:
    id: str
    title: str
    description: str
    status: TaskStatus
    priority: TaskPriority
    created_at: datetime
    updated_at: datetime
    due_date: Optional[datetime] = None
    tags: List[str] = None

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "title": self.title,
            "description": self.description,
            "status": self.status.value,
            "priority": self.priority.value,
            "created_at": self.created_at.isoformat(),
            "updated_at": self.updated_at.isoformat(),
            "due_date": self.due_date.isoformat() if self.due_date else None,
            "tags": self.tags or []
        }

    @classmethod
    def from_dict(cls, data: dict):
        """Deserialisierung von ThemisDB"""
        return cls(
            id=data["_key"],
            title=data["title"],
            description=data["description"],
            status=TaskStatus(data["status"]),
            priority=TaskPriority(data["priority"]),
            created_at=datetime.fromisoformat(data["created_at"]),
            updated_at=datetime.fromisoformat(data["updated_at"]),
            due_date=datetime.fromisoformat(data["due_date"]) if data.get("due_date") else No
```

```
            tags=data.get("tags", [])
        )
```

Design-Entscheidungen:

1. **Enums für Status/Priority:** Type-Safety, keine Tippfehler möglich
2. **Dataclass:** Automatische `__init__`, `__repr__`, `__eq__`
3. **to_dict/from_dict:** Klare Serialisierungslogik
4. **Type Hints:** IDEs können helfen, Fehler früh erkennen

Setup und Installation

```
docker run -d -p 8765:8765 themisdb/themisdb:1.3.4

cd examples/02_todo_app
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Client-Implementierung

```python
from themisdb import Client
from models import Task, TaskStatus, TaskPriority
from typing import List, Optional, Dict
import uuid

class TodoClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen
        try:
            self.client.create_collection("tasks", type="document")
        except Exception as e:
            # Collection existiert bereits
            pass

        # Indizes erstellen für Performance
        self.client.create_index("tasks", "status_idx", ["status"])
        self.client.create_index("tasks", "priority_idx", ["priority"])
        self.client.create_index("tasks", "due_date_idx", ["due_date"])

    def create_task(self, task: Task) -> bool:
        """Erstellt einen neuen Task"""
        try:
            self.client.insert("tasks", task.to_dict())
            return True
        except Exception as e:
            print(f"Error creating task: {e}")
            return False

    def get_task(self, task_id: str) -> Optional[Task]:
        """Lädt einen Task nach ID"""
        try:
            data = self.client.get("tasks", task_id)
            return Task.from_dict(data) if data else None
        except Exception as e:
            print(f"Error getting task: {e}")
            return None

    def update_task(self, task: Task) -> bool:
        """Aktualisiert einen existierenden Task"""
        try:
            # MVCC in Action: Conflict Detection!
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:
            print("Task was modified by another user!")
            return False
        except Exception as e:
            print(f"Error updating task: {e}")
            return False
```

```
    def delete_task(self, task_id: str) -> bool:
        """Löscht einen Task"""
        try:
            self.client.delete("tasks", task_id)
            return True
        except Exception as e:
            print(f"Error deleting task: {e}")
            return False

    def list_tasks(self, status: Optional[TaskStatus] = None,
                  priority: Optional[TaskPriority] = None) -> List[Task]:
        """Listet Tasks mit optionalen Filtern"""
        query = "FOR task IN tasks"
        filters = []

        if status:
            filters.append(f'task.status == "{status.value}"')
        if priority:
            filters.append(f'task.priority == "{priority.value}"')

        if filters:
            query += " FILTER " + " AND ".join(filters)

        query += " SORT task.created_at DESC RETURN task"

        try:
            results = self.client.query(query)
            return [Task.from_dict(data) for data in results]
        except Exception as e:
            print(f"Error listing tasks: {e}")
            return []

    def search_tasks(self, search_text: str) -> List[Task]:
        """Sucht Tasks nach Text in Titel oder Beschreibung"""
        query = """
        FOR task IN tasks
            FILTER CONTAINS(LOWER(task.title), LOWER(@search))
                OR CONTAINS(LOWER(task.description), LOWER(@search))
            SORT task.created_at DESC
            RETURN task
        """
        try:
            results = self.client.query(query, {"search": search_text})
            return [Task.from_dict(data) for data in results]
        except Exception as e:
            print(f"Error searching tasks: {e}")
            return []
```

MVCC und Transaktionen demonstriert

Die Todo-App zeigt MVCC in Aktion:

Szenario: Zwei Benutzer ändern gleichzeitig den gleichen Task

```
user1 = TodoClient()
task = user1.get_task("task_123")  # Snapshot Version 100
task.status = TaskStatus.IN_PROGRESS
task.updated_at = datetime.now()

user2 = TodoClient()
task2 = user2.get_task("task_123")  # Auch Snapshot Version 100
task2.priority = TaskPriority.HIGH
task2.updated_at = datetime.now()

user1.update_task(task)  # ✓ SUCCESS, Version 101

user2.update_task(task2)  # ✗ CONFLICT!
```

Was passiert intern?

1. Beide lesen Version 100 (Snapshot Isolation)
2. User 1 schreibt Version 101 und committed
3. User 2 versucht auch Version 101 zu schreiben
4. ThemisDB erkennt: "Version 101 existiert bereits!"
5. Conflict Error wird geworfen

Lösung im Code:

```
def update_task_with_retry(self, task: Task, max_retries=3):
    """Update mit automatischem Retry bei Conflicts"""
    for attempt in range(max_retries):
        try:
            self.client.update("tasks", task.id, task.to_dict())
            return True
        except ConflictError:
            # Re-read aktuellste Version
            latest = self.get_task(task.id)
            if not latest:
                return False

            # Merge changes (Application Logic)
            # z.B.: Keep newer updated_at
            if latest.updated_at > task.updated_at:
                task = latest

            # Retry mit gemergten Daten
            continue
        except Exception as e:
            return False

    return False  # Max retries exceeded
```

2.6 Index-Strukturen

Warum Indizes?

Ohne Index: Full Table Scan

```
-- Ohne Index: O(n)
SELECT * FROM tasks WHERE status = 'open'
-- Muss ALLE Tasks durchgehen: 10.000 Tasks = 10.000 Reads
```

Mit Index: Direkt zum Ergebnis

```
-- Mit Index auf status: O(log n + k), k = Anzahl Results
SELECT * FROM tasks WHERE status = 'open'
-- B-Tree Lookup: log(10.000) ≈ 13 Reads + nur relevante Tasks
```

Index-Typen in ThemisDB

1. B-Tree Index (Default)

```
client.create_index("tasks", "status_idx", ["status"])
```

- Sortierte Daten
- Range Queries: `WHERE age > 18 AND age < 65`
- Equality: `WHERE status = 'open'`

2. Hash Index

```
client.create_index("tasks", "id_hash_idx", ["id"], type="hash")
```

- Nur Equality: `WHERE id = 'task_123'`
- Schneller als B-Tree für Equality
- Keine Range Queries

3. Geo Index (Hilbert Curves)

```
client.create_index("locations", "geo_idx", ["coordinates"], type="geo")
```

- Räumliche Queries
- Nearest Neighbor
- Bounding Box Searches

4. Fulltext Index

```
client.create_index("tasks", "fulltext_idx", ["title", "description"], type="fulltext")
```

- Tokenization
- Stemming
- Relevance Scoring

5. Vector Index (HNSW)

```
client.create_index("products", "embedding_idx", ["embedding"], type="vector", dimensions=768
```

- Approximate Nearest Neighbor
- Cosine Similarity
- Euclidean Distance

Index-Performance

Operation	Ohne Index	Mit B-Tree	Mit Hash
Equality (`=`)	O(n)	O(log n)	O(1)
Range (`>` , `<`)	O(n)	O(log n + k)	✗
Sort	O(n log n)	O(k)	✗

k = Anzahl Ergebnisse

2.7 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Schichten-Architektur: 5 Schichten mit klaren Verantwortlichkeiten
MVCC: Parallelität ohne Locks durch Versionierung
Transaktionen: ACID-Garantien und Snapshot Isolation
RocksDB: LSM-Trees, WAL, Column Families
Indizes: B-Tree, Hash, Geo, Fulltext, Vector
Todo-App: Vollständige CRUD-Anwendung mit MVCC

Was macht ThemisDB besonders?

1. **Multi-Model MVCC:** Transaktionen über alle Modelle hinweg
2. **RocksDB Foundation:** Battle-tested, SSD-optimiert
3. **Flexible Indizes:** Für jeden Use Case der richtige Index
4. **Snapshot Isolation:** Performance + Konsistenz

Nächste Schritte

Im nächsten Kapitel lernen Sie, wie die verschiedenen Datenmodelle kombiniert werden können und wann welches Modell am besten passt.

Kapitel 3: Multi-Model verstehen →

Weiterführende Ressourcen

- **Complete Example:** examples/02_todo_app
- **MVCC Deep-Dive:** ../de/architecture/architecture_mvcc.md
- **RocksDB Storage:** ../de/storage/storage_rocksdb.md
- **Index Design:** Kapitel 15 - Storage Internals

2.11 BaseEntity: Die Speichereinheit

ThemisDB speichert alle Daten - unabhängig vom Modell (relational, document, graph, vector) - als **BaseEntity**. Dies ist die fundamentale Speichereinheit.

2.11.1 BaseEntity Architektur

```
class BaseEntity {
public:
    using Blob = std::vector<uint8_t>;
    using FieldMap = std::map<std::string, Value>;

    enum class Format { BINARY, JSON };

    // Primärschlüssel
    const std::string& getPrimaryKey() const;
    void setPrimaryKey(std::string_view pk);

    // Feld-Zugriff (lazy parsing)
    std::optional<Value> getField(std::string_view field_name) const;
    void setField(std::string_view field_name, const Value& value);

    // Serialisierung
    Blob serialize() const;
    std::string toJson() const;

    // Factory-Methoden
    static BaseEntity fromJson(std::string_view pk, std::string_view json_str);
    static BaseEntity fromFields(std::string_view pk, const FieldMap& fields);
    static BaseEntity deserialize(std::string_view pk, const Blob& blob);

private:
    std::string primary_key_;
    Blob blob_;
    Format format_ = Format::BINARY;
    mutable std::shared_ptr<FieldMap> field_cache_;
};
```

Kernkonzept: Jede Entität ist ein **einzelnes binäres Blob** mit effizienter Serialisierung und Lazy-Parsing für schnellen Feldzugriff.

2.11.2 Value-Typsystem

ThemisDB verwendet ein typsicheres `std::variant`-System:

Typ	C++ Typ	Verwendung	Beispiel
null	`std::monostate`	Fehlende/leere Werte	`NULL`
bool	`bool`	Wahrheitswerte	`true` , `false`
int	`int64_t`	Ganzzahlen	`42` , `-1000`
double	`double`	Gleitkommazahlen	`3.14` , `1e10`
string	`std::string`	Texte	`"Hello World"`
vector	`std::vector<float>`	Embeddings für ANN	`[0.1, 0.2, 0.3]`
binary	`std::vector<uint8_t>`	Binärdaten	Bilder, PDFs

Wichtig: Nested Objects/Arrays sind aktuell nicht unterstützt (geplant für v1.4).

2.11.3 BaseEntity Lifecycle

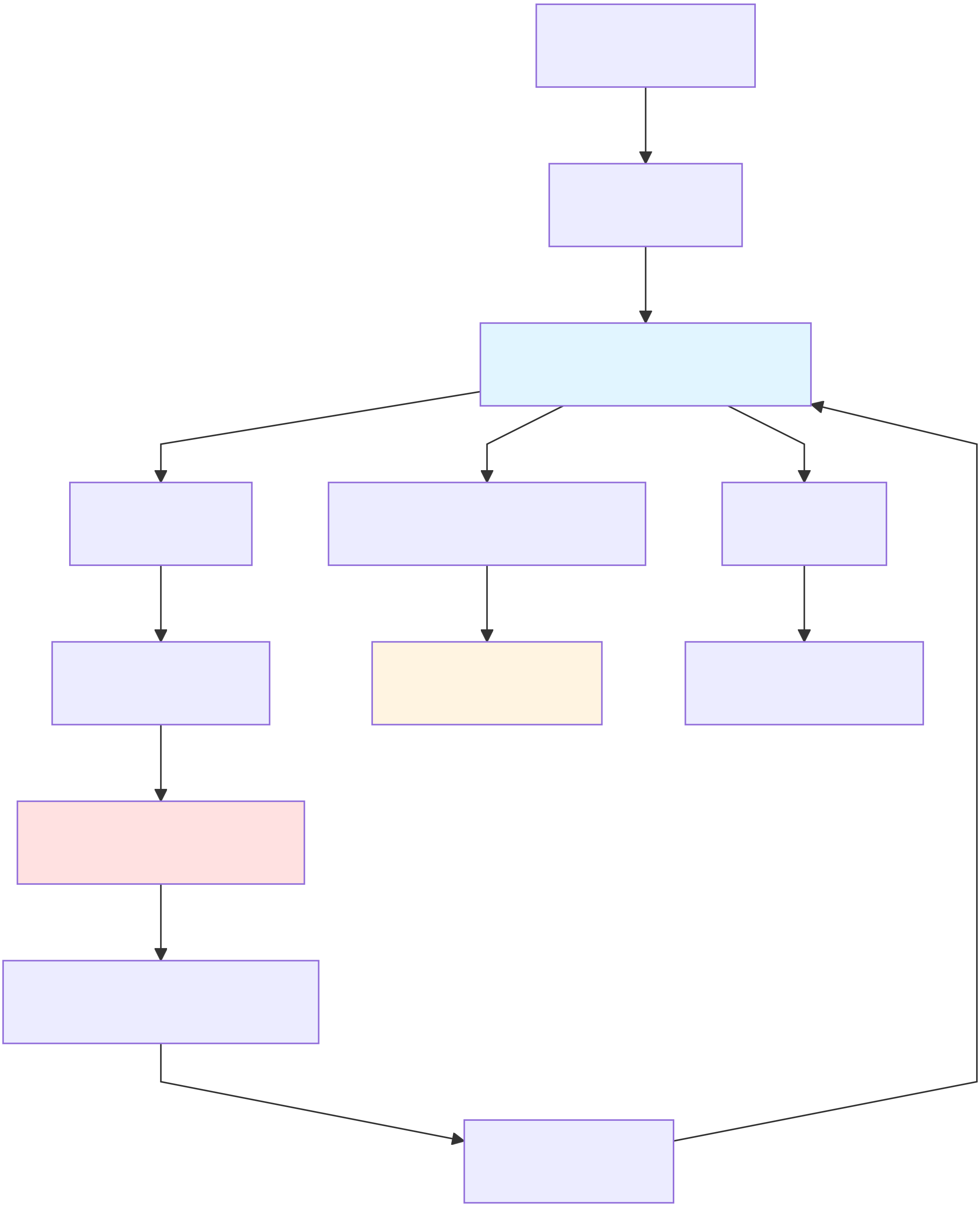


Abb. 9: Ablaufdiagramm: 2.11.3 BaseEntity Lifecycle

Lazy Parsing: Felder werden erst geparst, wenn sie angefordert werden - spart CPU-Zeit bei großen Objekten.

2.12 Key Schema: Multi-Modell-Namespacing

ThemisDB vereint alle Datenmodelle in einer einzigen RocksDB-Instanz durch ein hierarchisches Schlüsselsystem mit Präfixen.

2.12.1 Schlüsselformate

Datenmodell	Format	Beispiel	Beschreibung
Relational	`entity:table:pk`	`entity:users:alice`	Tabellenzeilen
Document	`entity:collection:pk`	`entity:orders:order_123`	Dokumente
Graph Node	`entity:node:pk`	`entity:node:user_456`	Graph-Knoten
Graph Edge	`entity:edge:pk`	`entity:edge:follows_789`	Graph-Kanten
Vector	`entity:vectors:pk`	`entity:vectors:doc_abc`	Vektor-Embeddings
Secondary Index	`idx:table:column:value:pk`	`idx:users:age:30:alice`	Equality Indexes
Range Index	`ridx:table:column:value:pk`	`ridx:products:price:99.99:prod_1`	Range Queries
Spatial Index	`sidx:table:geohash:pk`	`sidx:locations:u33dc1:berlin`	Geo-Queries
TTL Index	`ttlidx:table:column:ts:pk`	`ttlidx:sessions:created:1730000000:s1`	Time-based Expiry
Fulltext	`ftidx:table:column:token:pk`	`ftidx:articles:body:search:art_42`	Volltextsuche
Graph Outdex	`graph:out:pk_start:pk_edge`	`graph:out:alice:follows_789`	Ausgehende Kanten
Graph Indeg	`graph:in:pk_target:pk_edge`	`graph:in:bob:follows_789`	Eingehende Kanten
Changefeed	`changefeed:pk:seqno`	`changefeed:alice:0000000001`	CDC Stream
Time-Series	`ts:metric:entity:ts`	`ts:cpu:server1:1730000000000`	Metriken

Separator: Alle Schlüssel verwenden `:` als Trennzeichen für schnelles Prefix-Scanning.

2.12.2 Key Schema Visualisierung

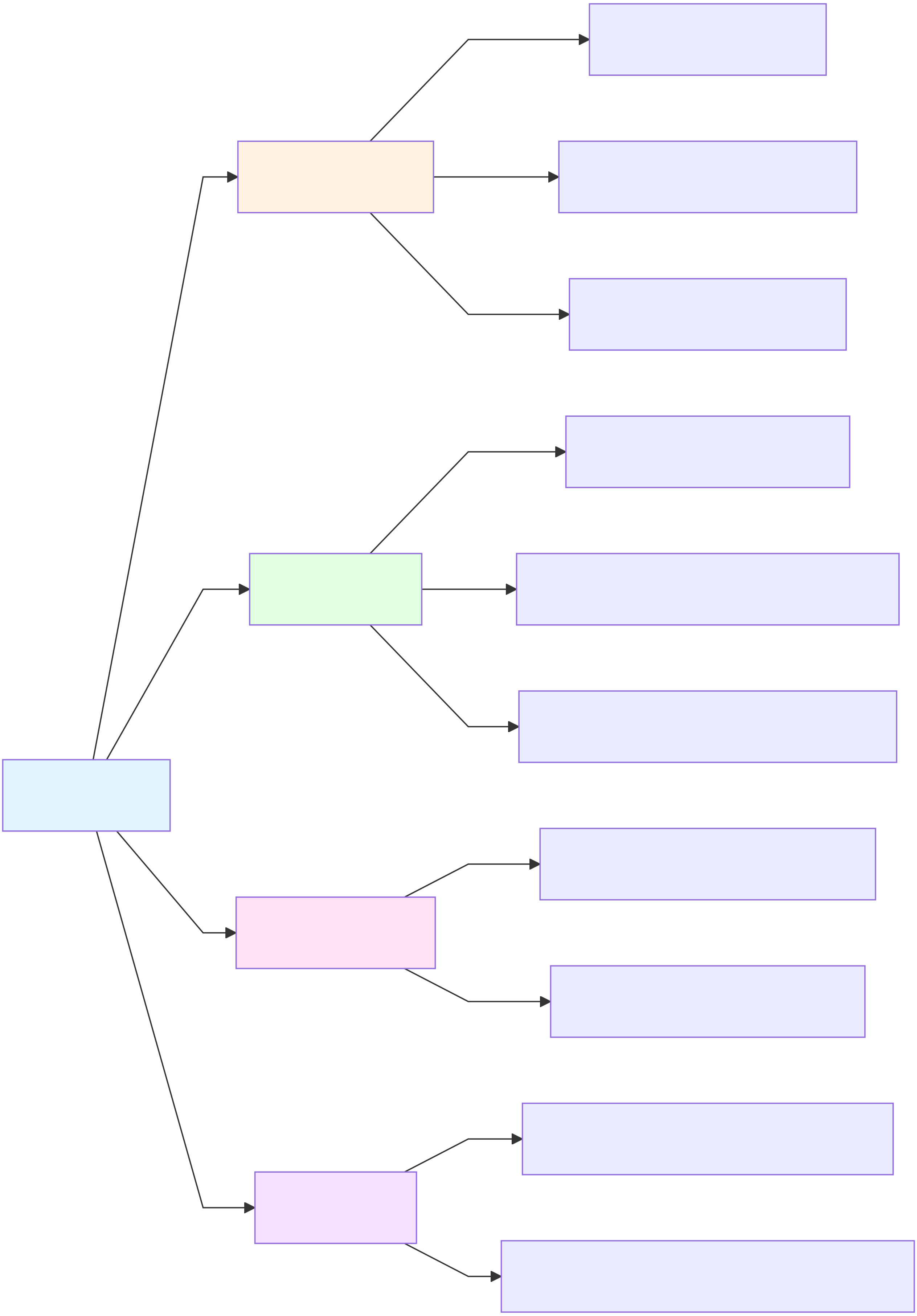


Abb. 10: Ablaufdiagramm: 2.12.2 Key Schema Visualisierung

2.12.3 Primary Key Extraktion

```
// KeySchema API
std::string pk = KeySchema::extractPrimaryKey("idx:users:age:30:alice");
// Ergebnis: "alice"

KeyType type = KeySchema::parseKeyType("entity:users:alice");
// Ergebnis: KeyType::RELATIONAL

std::string key = KeySchema::makeRelationalKey("users", "alice");
// Ergebnis: "entity:users:alice"
```

Vorteil: Durch Präfixe können alle Indizes einer Tabelle oder alle Kanten eines Knotens mit einem einzigen RocksDB-Scan gelesen werden.

2.13 MVCC Deep-Dive: Implementierungsdetails

Nachdem wir in Abschnitt 2.2 die Grundlagen von MVCC kennengelernt haben, tauchen wir nun in die Implementierungsdetails ein.

2.13.1 RocksDB TransactionDB Architektur

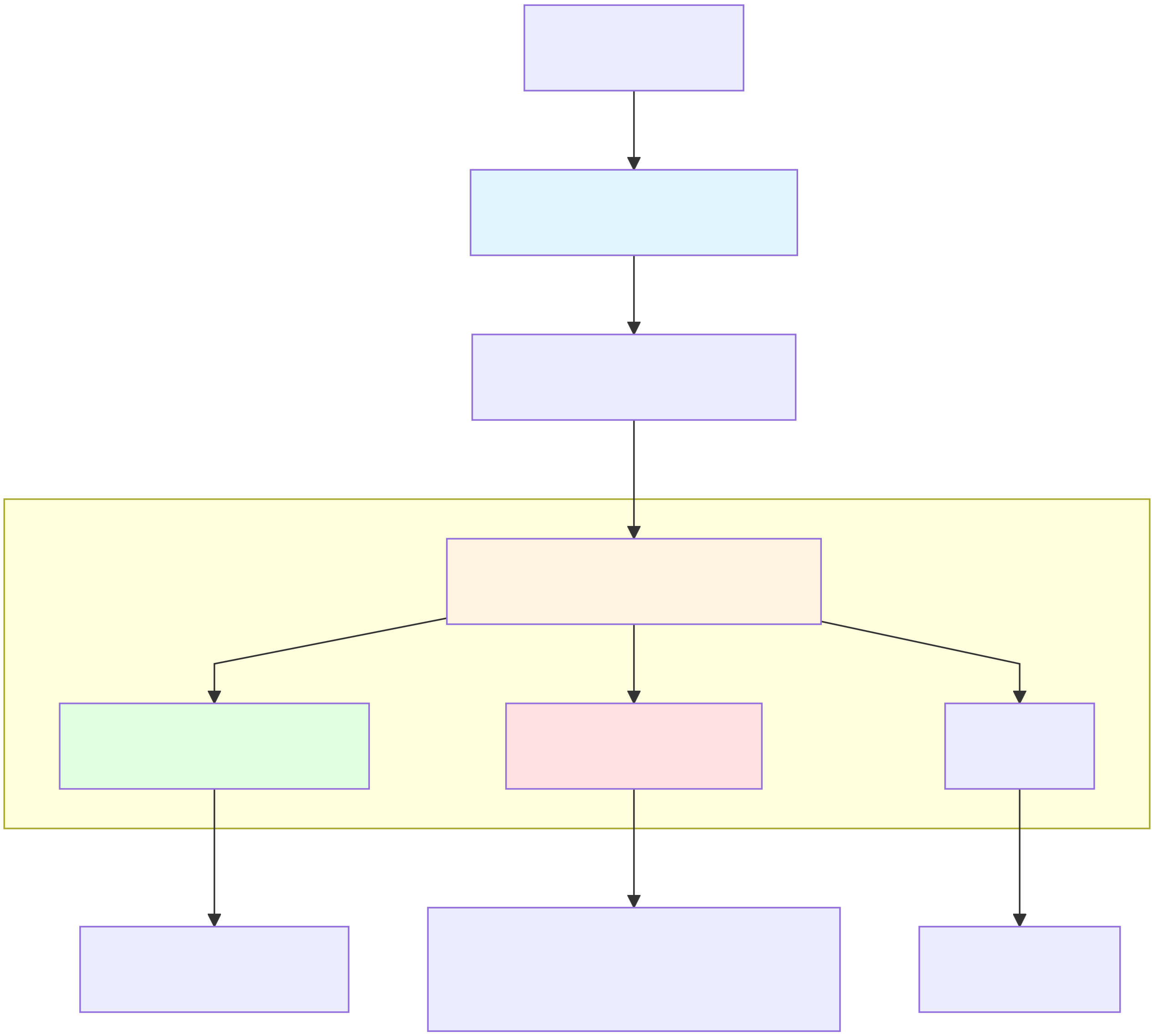


Abb. 11: Ablaufdiagramm: 2.13.1 RocksDB TransactionDB Architektur

2.13.2 Snapshot Isolation Flow

```
// Transaction A liest Snapshot zum Zeitpunkt t=100
auto txn_a = db.beginTransaction();
auto snapshot = txn_a->getSnapshot();  // t=100

// Transaction B schreibt zur gleichen Zeit
auto txn_b = db.beginTransaction();
txn_b->put("entity:users:alice", "{age: 31}");
txn_b->commit();  // Commit zur Zeit t=101

// Transaction A sieht alte Version (t=100)
auto value = txn_a->get("entity:users:alice");
// value = "{age: 30}" (alte Version)

txn_a->commit();
```

2.13.3 Write-Write Conflict Detection

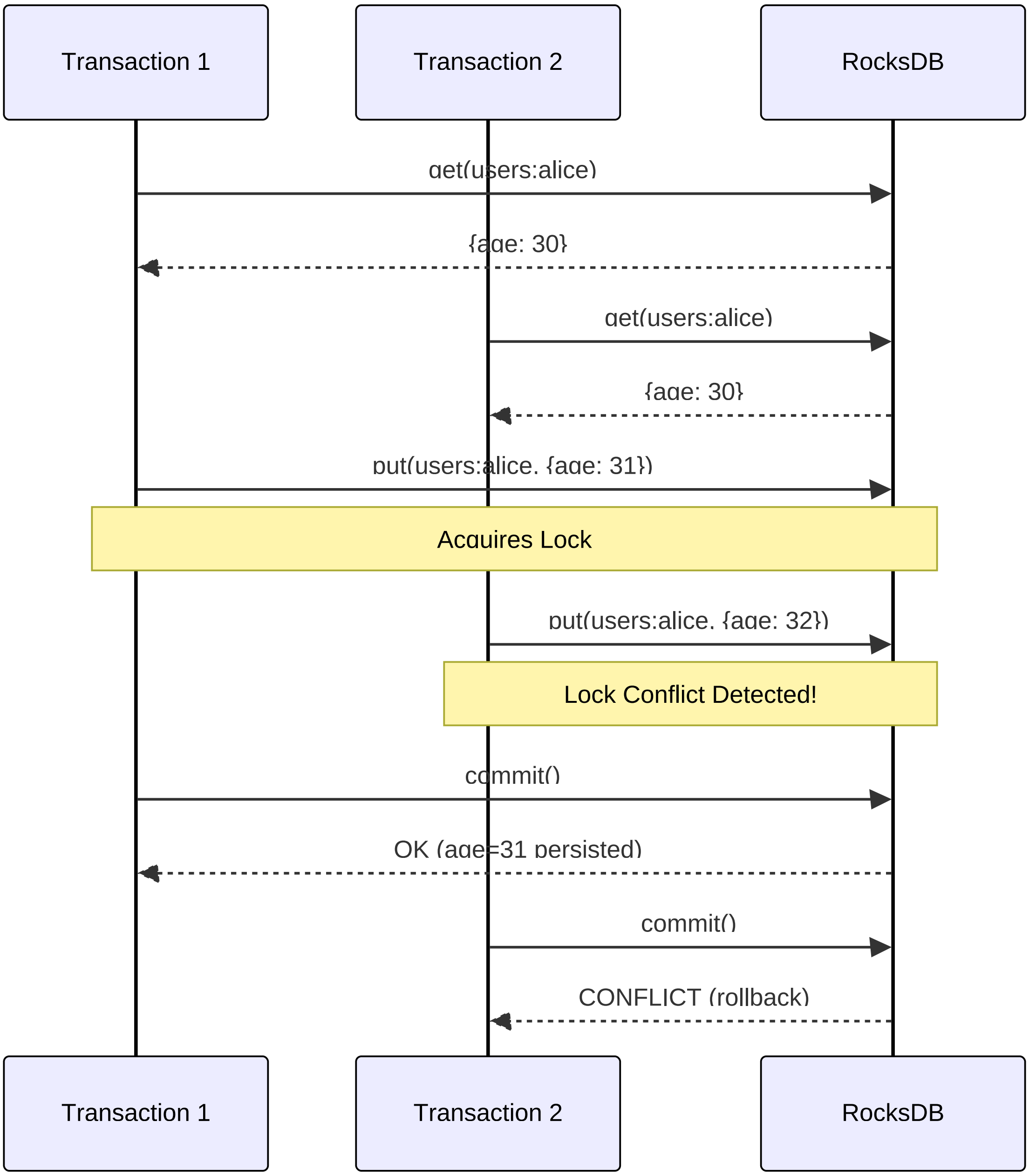


Abb. 12: Sequenzdiagramm: 2.13.3 Write-Write Conflict Detection

Konfliktauflösung: Transaction 2 wird automatisch zurückgerollt - die Anwendung muss die Transaktion wiederholen.

2.13.4 MVCC Performance

Benchmark-Ergebnisse (v1.3.0):

Operation	MVCC (ops/s)	WriteBatch (ops/s)	Overhead
Single Entity Insert	3,400	3,100	+9.7%
Batch Insert (100)	27,800	27,800	0%
Snapshot Read	44,000	-	Baseline
Rollback	35,300	-	Baseline

Fazit: MVCC-Overhead ist minimal bei Batch-Operationen - Snapshot-Isolation kostet fast nichts!

2.13.5 Index-Konsistenz mit MVCC

Problem: Wenn eine Transaktion zurückgerollt wird, müssen alle Indizes konsistent bleiben.

Lösung: Alle Index-Operationen sind Teil der Transaktion:

```
// Atomare Transaktion: Entity + Secondary Index + Graph Outdex
auto txn = db.beginTransaction();

// 1. Insert Entity
txn->put(KeySchema::makeRelationalKey("users", "alice"),
         entity.serialize());

// 2. Update Secondary Index
txn->put(KeySchema::makeSecondaryIndexKey("users", "age", "30", "alice"),
         empty_value);

// 3. Update Graph Outdex
txn->put(KeySchema::makeGraphOutdexKey("alice", "follows_789"),
         edge_data);

// Alles oder nichts
if (!txn->commit()) {
    // Conflict detected - alle 3 Operationen werden zurückgerollt
}
```

Garantie: Entweder werden alle Indizes aktualisiert oder keiner - keine Inkonsistenzen möglich!

2.14 Compression Strategy

ThemisDB verwendet differenzierte Kompression je nach Datentyp für optimale Balance zwischen Speicherersparnis und Performance.

2.14.1 Kompressionsarten

Datentyp	Compression	Ratio	CPU-Overhead	Use Case
Time-Series	Gorilla	10-20x	+15%	Metriken, Sensordaten
Vektoren	SQ8 Quantization	4x	+20%	Embeddings ab 1M Vektoren
Content Blobs	ZSTD Level 19	1.5-2x	+30%	Dokumente, Bilder
JSON Metadata	LZ4 (RocksDB)	2-3x	+5%	Structured Data
Graph Edges	LZ4 (RocksDB)	2x	+5%	Adjacency Lists

2.14.2 Gorilla Time-Series Compression

Gorilla ist ein hocheffizienter Codec für Time-Series-Daten von Facebook:

```
// Gorilla Encoding
std::vector<uint8_t> GorillaCodec::encode(
    const std::vector<int64_t>& timestamps,
    const std::vector<double>& values
) {
    // Delta-of-Delta Encoding für Timestamps
    // XOR-Encoding für Float-Werte
    // Typical Ratio: 12-20x
}
```

Beispiel:

```
Raw Data (100k Punkte):
- Timestamps: 800 KB (8 bytes × 100k)
- Values: 800 KB (8 bytes × 100k)
- Total: 1.6 MB

Gorilla Compressed:
- Total: 80-130 KB
- Ratio: 12-20x
```

2.14.3 Vector Quantization (SQ8)

Scalar Quantization reduziert `float32` (4 bytes) auf `int8` (1 byte):

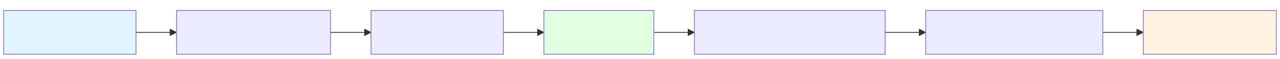


Abb. 13: Ablaufdiagramm: 2.14.3 Vector Quantization (SQ8)

Trade-off: - 4x Speicherersparnis - Bessere Cache-Nutzung → schnellere Suche - ⚠ 95-98% Recall@10 (minimal schlechter als float32)

2.14.4 RocksDB Block Compression

ThemisDB konfiguriert RocksDB mit **Level-based Compression**:

```
// Level 0-5: LZ4 (schnell für Hot Data)
options.compression_per_level[0] = rocksdb::kLZ4Compression;
options.compression_per_level[1] = rocksdb::kLZ4Compression;
// ...

// Level 6+: Zstandard (höhere Ratio für Cold Data)
options.compression_per_level[6] = rocksdb::kZSTD;
options.compression_per_level[7] = rocksdb::kZSTD;
```

Rationale: Hot Data (Level 0-1) benötigt schnellen Zugriff → LZ4. Cold Data (Level 6+) profitiert von höherer Kompression → Zstandard.

2.15 Zusammenfassung Architektur

In diesem erweiterten Kapitel haben Sie die Tiefe der ThemisDB-Architektur kennengelernt:

Kernkonzepte

1. **BaseEntity:** Unified Storage Layer für alle Datenmodelle
2. **Key Schema:** Hierarchisches Namespacing mit Präfixen
3. **MVCC:** Snapshot Isolation ohne Locks
4. **Compression:** Differenzierte Strategien (Gorilla, SQ8, ZSTD)
5. **Index-Konsistenz:** Atomare Transaktionen über alle Indizes

Architektur-Prinzipien

- **Separation of Concerns:** Klar getrennte Schichten
- **Performance First:** Lazy Parsing, Lazy Indexing
- **Multi-Model Unity:** Ein Speicher, viele Modelle
- **Concurrency Without Locks:** MVCC statt Pessimistic Locking
- **Adaptive Compression:** Datentyp-spezifische Algorithmen

Weitere Ressourcen

- **Complete Example:** examples/02_todo_app
- **MVCC Deep-Dive:** ../de/architecture/architecture_mvcc.md
- **RocksDB Storage:** ../de/storage/storage_rocksdb.md
- **Index Design:** Kapitel 15 - Storage Internals

Kapitel 2 von 30 | Teil I: Grundlagen | ~14.000 Wörter

Kapitel 4: Kapitel 3: Multi-Model verstehen

> *"Der Unterschied zwischen einem guten und einem großartigen System liegt oft darin, das richtige Werkzeug für die richtige Aufgabe zu wählen."*

Überblick

In den ersten beiden Kapiteln haben Sie die Grundlagen von ThemisDB kennengelernt. Jetzt lernen Sie, wie Sie die verschiedenen Datenmodelle gezielt einsetzen und kombinieren. Das Geheimnis liegt nicht darin, ein Modell zu wählen - sondern das *richtige* Modell für jeden Teil Ihrer Daten.

Was Sie in diesem Kapitel lernen werden: - Wann welches Datenmodell optimal ist - Wie man Modelle in einer Anwendung kombiniert - Die Stärken und Schwächen jedes Modells - Praktische Entscheidungskriterien - Vollständige Demonstration mit einem Kontaktmanager

Voraussetzungen: Kapitel 1 und 2 gelesen.

3.1 Die vier Modelle im Detail

Relational: Wenn Struktur entscheidend ist

Best für: - Geschäftsdaten mit klaren Beziehungen - Financial Transactions - Inventory Management - Reporting und Analytics

Eigenschaften:

```
users = {
    "user_id": "string",      # Muss vorhanden sein
    "email": "string",        # Muss vorhanden sein
    "age": "integer",         # Type-checked
    "created_at": "timestamp" # Format validiert
}

UNIQUE(email)
CHECK(age >= 18)
FOREIGN KEY(user_id) REFERENCES users(user_id)
```

Vorteile: - Data Integrity durch Constraints - Optimierte Joins - Perfekt für BI/Reporting - ACID über alle Operationen

Nachteile: - × Schema-Änderungen erfordern Migrations - × Nicht flexibel für schnelle Iterationen - × Overhead bei sehr unterschiedlichen Entities

Wann verwenden?

```
✓ Daten haben feste Struktur
✓ Beziehungen sind wichtig
✓ Konsistenz ist kritisch
✗ Schema ändert sich häufig
```

Graph: Wenn Beziehungen im Fokus stehen

Best für: - Social Networks - Empfehlungssysteme - Fraud Detection - Netzwerk-Topologien

Eigenschaften:

```
user = {
    "_id": "users/alice",
    "name": "Alice"
}

friendship = {
    "_from": "users/alice",
    "_to": "users/bob",
    "since": "2024-01-15",
    "strength": 0.95  # Weighted edges
}

FOR friend IN 2..3 OUTBOUND "users/alice" friends
    RETURN friend
```

Vorteile: - Traversierung ist O(degree × hops), nicht $O(n^2)$ - Relationship-First Design - Pattern Matching möglich - Graph-Algorithmen (PageRank, etc.)

Nachteile: - × Nicht optimal für reine Daten ohne Beziehungen - × Komplexer bei sehr vielen Knoten (> Millionen) - × Overhead wenn Beziehungen unwichtig sind

Wann verwenden?

```
✓ Beziehungen sind zentral
✓ Traversierung ist häufig
✓ Netzwerk-Strukturen
✗ Isolierte Entities
```

Dokument: Wenn Flexibilität zählt

Best für: - Content Management - Product Catalogs - User Profiles - Logs und Events

Eigenschaften:

```
product = {
    "sku": "LAPTOP-2024",
    "name": "Developer Laptop",
    "specs": {  # Nested objects
        "cpu": {"model": "i7", "cores": 8},
        "ram": {"size": 32, "type": "DDR5"}
    },
    "reviews": [  # Arrays
        {"user": "alice", "rating": 5},
        {"user": "bob", "rating": 4}
    ],
    "tags": ["developer", "high-end"],
    "metadata": {  # Kann sein, muss nicht
        "warehouse": "DE-01"
    }
}
```

Vorteile: - Keine Schema-Migrations - Schnelle Iteration - Hierarchische Daten natürlich - Self-contained Documents

Nachteile: - × Keine Schema-Validierung (optional möglich) - × Denormalisierung kann zu Duplikaten führen - × Joins sind teurer

Wann verwenden?

```
✓ Schema ändert sich oft
✓ Hierarchische Daten
✓ Prototyping
✗ Strenge Validierung nötig
```

Vektor: Wenn Ähnlichkeit gefragt ist

Best für: - Semantic Search - Recommendation Engines - Image Similarity - ML/AI Applications

Eigenschaften:

```
product_embedding = [
    0.123, -0.456, 0.789, ...  # 768 dimensions
]

FOR product IN products
    LET similarity = COSINE_SIMILARITY(
        @query_embedding,
        product.embedding
    )
    FILTER similarity > 0.8
    SORT similarity DESC
    LIMIT 10
    RETURN product
```

Vorteile: - Semantic Search ohne Keywords - Multimodal (Text, Images, Audio) - Skaliert zu Millionen von Vektoren - State-of-the-art HNSW Index

Nachteile: - × Embeddings müssen generiert werden - × Höherer Speicherbedarf - × Approximate, nicht exact matching

Wann verwenden?

```
✓ Semantic Similarity
✓ "Finde ähnliche X"
✓ ML/AI Integration
✗ Exact matches ausreichend
```

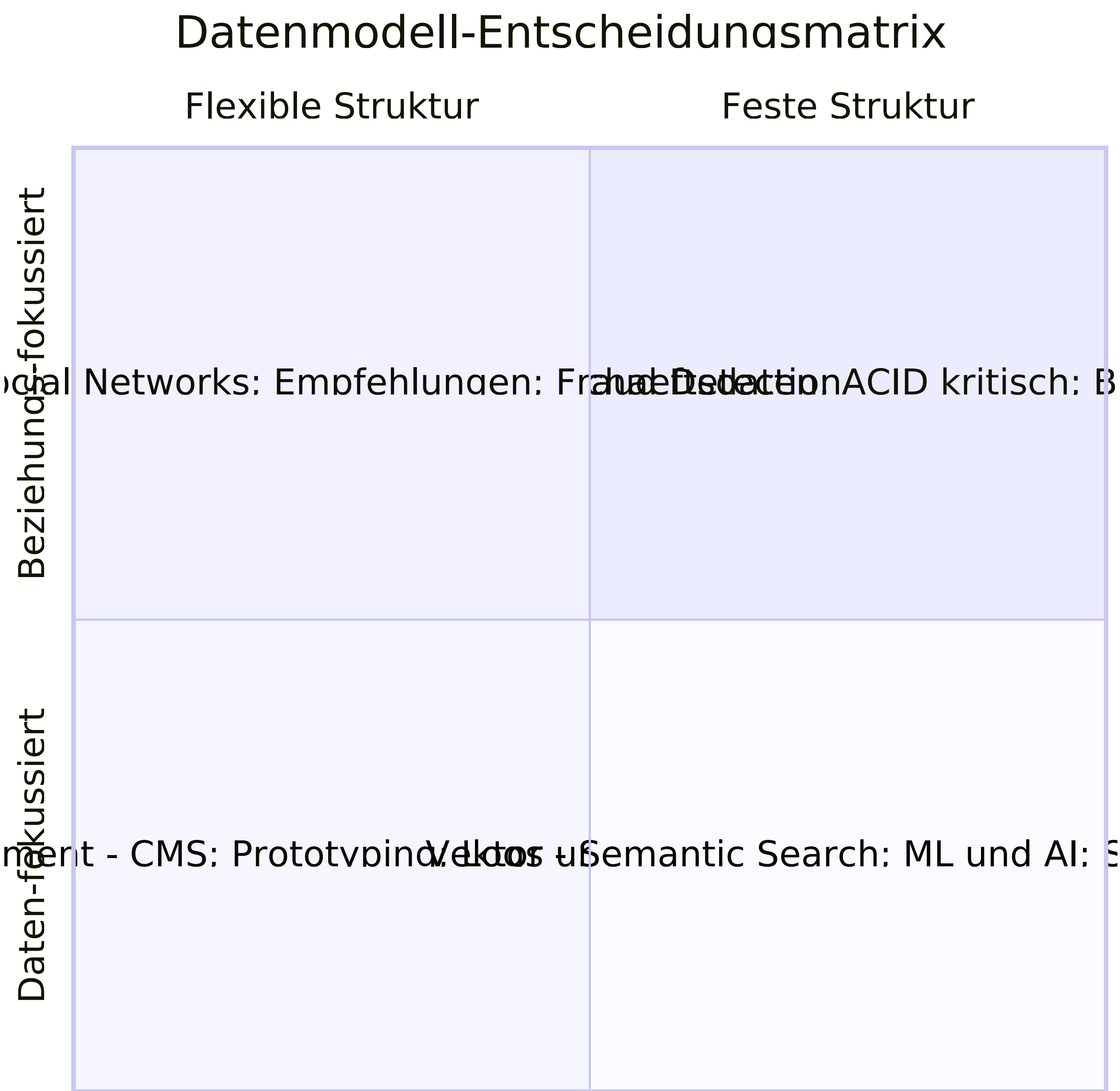


Abb. 14: Quadranten-Diagramm (Kapitel 3: Multi-Model verstehen)

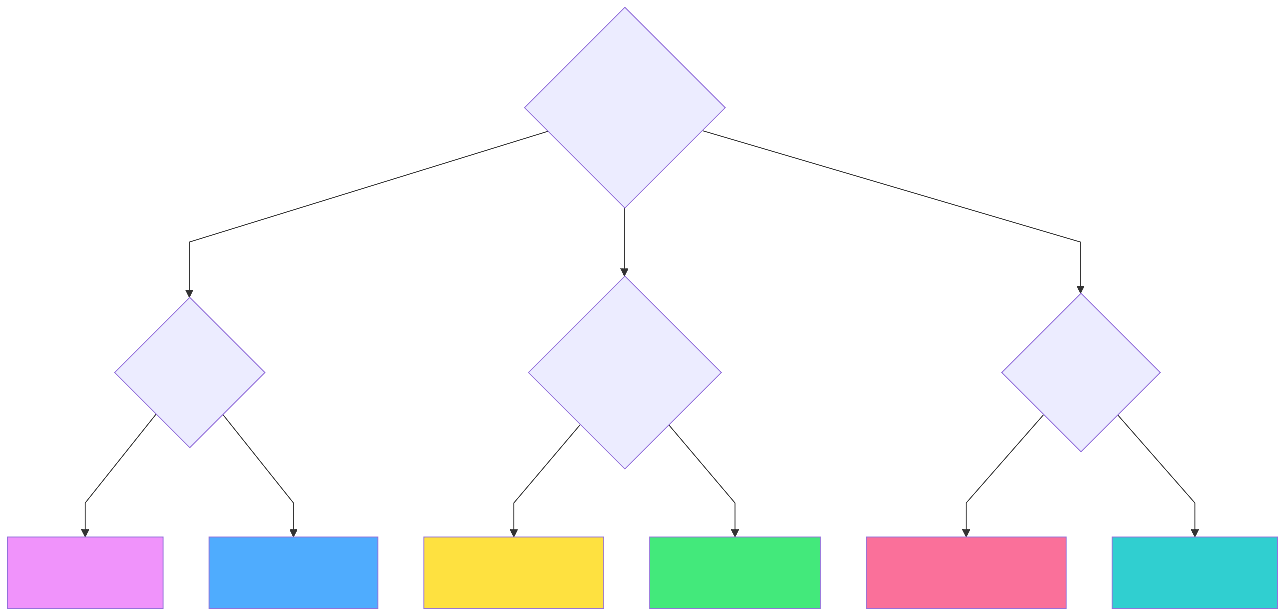


Abb. 15: Ablaufdiagramm (Kapitel 3: Multi-Model verstehen)

3.2 Native Multi-Model vs. Polyglot Persistence

Das Problem polyglotter Persistenz

Viele Unternehmen verfolgen einen "Polyglot Persistence"-Ansatz [33]: Sie kombinieren mehrere spezialisierte Datenbanken (z.B. PostgreSQL für relationale Daten, Neo4j für Graphen, ChromaDB für Vektoren) in einem losen Verbund. Dieser Ansatz scheint flexibel, führt aber zu fundamentalen Problemen [1]:

Technische Herausforderungen:

1. **Eventual Consistency statt ACID:**
2. Daten sind über mehrere, physisch getrennte Systeme verteilt [33]
3. Atomare Transaktionen über alle Systeme hinweg sind unmöglich [1]
4. Man muss auf das "Saga-Pattern" [17] mit kompensierenden Transaktionen zurückgreifen
5. Resultat: "Eventual Consistency" (BASE) [18] statt starker ACID-Garantien [16]

6. **Post-Filtering-Problem bei RAG-Workloads:** ``` # Polyglot-Ansatz (ineffizient):

7. Vektor-DB: Finde 1000 ähnliche Dokumente
8. Graph-DB: Hole alle Prozesse für Landkreis Havelland
9. Relational-DB: Hole alle Akten aus 2024
10. Application: Bilde manuell Schnittmenge im RAM → 990 irrelevante Ergebnisse werden verworfen! [2], [5] ```

11. **Operativer Overhead:**

12. 3+ unterschiedliche APIs zu lernen und warten

13. 3+ Backup-Strategien zu implementieren
14. 3+ Monitoring-Systeme zu konfigurieren
15. 3+ Security-Konfigurationen abzustimmen
16. Team benötigt Expertise in mehreren Datenbanken

ThemisDB's Native Multi-Model-Lösung

ThemisDB verfolgt einen fundamentals anderen Ansatz [3], [11]: **Alle Datenmodelle teilen sich eine einzige, transaktionale Speicherschicht** (siehe Kapitel 2.4 - Base Entity Paradigma).

Architektonische Vorteile:

1. **Starke ACID-Transaktionen über alle Modelle:**
2. Eine Operation kann atomar Graph, Vector und Relational ändern [1], [20]
3. Kein Saga-Pattern nötig für Datenbankintegrität [3]
4. Konsistenz ist "by Design", nicht ein fehleranfälliger Applikationsprozess

5. **Pre-Filtering statt Post-Filtering:** `aql -- Native Multi-Model ermöglicht Pre-Filtering: FOR doc IN documents FILTER doc.year == 2024 // Relational Filter ZUERST FILTER doc.region == "Havelland" // Graph Context LET similarity = COSINE(doc.vector, @query) // Vector Search auf Subset FILTER similarity > 0.8 SORT similarity DESC LIMIT 10`

6. Query-Engine nutzt ZUERST den relationalen Index (Jahr=2024)
7. Erstellt eine hochselektive Kandidatenliste
8. Vektorsuche (rechenintensiv) läuft NUR auf erlaubter Teilmenge
9. **Resultat:** Um Größenordnungen performanter als Post-Filtering

10. **Vereinfachte Operations:**

11. Eine API (AQL) für alle Modelle
12. Eine Backup-Strategie
13. Ein Monitoring-System
14. Eine Security-Konfiguration

Wann welcher Ansatz?

Polyglot Persistence verwenden wenn: - ✓ Sie bereits mehrere spezialisierte Datenbanken im Einsatz haben - ✓ Eventual Consistency für Ihren Use Case akzeptabel ist - ✓ Sie Best-of-Breed-Tools für jeden Use Case wollen - ✓ Sie ein großes Ops-Team haben

Native Multi-Model (ThemisDB) verwenden wenn: - ✓ ACID-Garantien über alle Datenmodelle kritisch sind - ✓ Sie hybride Queries (Graph + Vector + Relational) benötigen - ✓ RAG-Workloads mit komplexen Filtern zentral sind - ✓ Sie operationale Komplexität reduzieren wollen - ✓ Ein kleineres Team die Infrastruktur betreiben soll

3.3 Entscheidungskriterien

Die 5 Fragen-Methode

1. Wie strukturiert sind Ihre Daten?

```
Sehr strukturiert → Relational
Teilweise strukturiert → Dokument
Unstrukturiert → Vektor
```

2. Wie wichtig sind Beziehungen?

```
Zentral → Graph
Wichtig → Relational (mit Joins)
Unwichtig → Dokument
```

3. Wie oft ändert sich das Schema?

```
Selten → Relational
Häufig → Dokument
Gar nicht → Relational
```

4. Welche Queries dominieren?

```
Beziehungen durchlaufen → Graph
Ähnlichkeit finden → Vektor
Komplexe Aggregationen → Relational
Einzelne Dokumente → Dokument
```

5. Wie wichtig ist Konsistenz?

```
Kritisch → Relational
Wichtig → Relational oder Graph
Weniger wichtig → Dokument
```

Entscheidungsmatrix

Use Case	Relational	Graph	Dokument	Vektor
E-Commerce Orders	★★★	★☆☆	★★☆	☆☆☆
Social Network	★☆☆	★★★	★★☆	☆☆☆
Product Catalog	★★☆	☆☆☆	★★★	★★☆
Recommendation	★☆☆	★★★	☆☆☆	★★★
CMS/Blog	★☆☆	☆☆☆	★★★	★★☆
Financial	★★★	☆☆☆	★☆☆	☆☆☆
Fraud Detection	★★☆	★★★	★☆☆	★★☆

3.4 Praxis: Kontaktmanager

Jetzt bauen wir einen vollständigen Kontaktmanager, der zeigt, wie man Modelle kombiniert. Basis ist `examples/03_contact_manager`.

Warum Dokument-Modell für Kontakte?

Kontakte sind ein perfektes Beispiel für das Dokument-Modell:

Gründe: 1. **Flexible Struktur:** Manche Kontakte haben Adresse, manche nicht 2. **Hierarchische Daten:** Adresse ist ein Nested Object 3. **Schema-Evolution:** Neue Felder wie "Social Media" leicht hinzufügbar 4. **Self-Contained:** Alle Infos zu einem Kontakt in einem Dokument

Alternative wäre Relational:

```
-- Relational würde benötigen:
CREATE TABLE contacts (id, name, email, phone);
CREATE TABLE addresses (id, contact_id, street, city, ...);
CREATE TABLE social_media (id, contact_id, platform, handle);
-- → 3+ Tabellen, Joins nötig
```

Mit Dokument:

```
contact = {
    "name": "Alice",
    "email": "alice@example.com",
    "address": {"city": "Berlin"},  # Optional!
    "social": {"twitter": "@alice"}  # Optional!
}
```

Datenmodell

```python
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional, List
from enum import Enum

class ContactCategory(Enum):
    FRIENDS = "friends"
    FAMILY = "family"
    WORK = "work"
    OTHER = "other"

@dataclass
class Address:
    """Verschachtelte Adress-Struktur"""
    street: str
    city: str
    postal_code: str
    country: str = "Deutschland"

    def to_dict(self):
        return {
            "street": self.street,
            "city": self.city,
            "postal_code": self.postal_code,
            "country": self.country
        }

    @classmethod
    def from_dict(cls, data: dict):
        return cls(
            street=data.get("street", ""),
            city=data.get("city", ""),
            postal_code=data.get("postal_code", ""),
            country=data.get("country", "Deutschland")
        )

@dataclass
class Contact:
    """Hauptentität: Kontakt"""
    id: str
    first_name: str
    last_name: str
    email: str
    phone: Optional[str] = None
    address: Optional[Address] = None
    category: ContactCategory = ContactCategory.OTHER
    is_favorite: bool = False
    notes: str = ""
    tags: List[str] = field(default_factory=list)
    created_at: datetime = field(default_factory=datetime.now)
    updated_at: datetime = field(default_factory=datetime.now)

    @property
    def full_name(self):
```

```python
        """Computed Property"""
        return f"{self.first_name} {self.last_name}"

    def to_dict(self):
        """Serialisierung für ThemisDB"""
        return {
            "_key": self.id,
            "first_name": self.first_name,
            "last_name": self.last_name,
            "email": self.email,
            "phone": self.phone,
            "address": self.address.to_dict() if self.address else None,
            "category": self.category.value,
            "is_favorite": self.is_favorite,
            "notes": self.notes,
            "tags": self.tags,
            "created_at": self.created_at.isoformat(),
            "updated_at": self.updated_at.isoformat()
        }

    @classmethod
    def from_dict(cls, data: dict):
        """Deserialisierung von ThemisDB"""
        address_data = data.get("address")
        return cls(
            id=data["_key"],
            first_name=data["first_name"],
            last_name=data["last_name"],
            email=data["email"],
            phone=data.get("phone"),
            address=Address.from_dict(address_data) if address_data else None,
            category=ContactCategory(data.get("category", "other")),
            is_favorite=data.get("is_favorite", False),
            notes=data.get("notes", ""),
            tags=data.get("tags", []),
            created_at=datetime.fromisoformat(data["created_at"]),
            updated_at=datetime.fromisoformat(data["updated_at"])
        )
```

Design-Highlights:

1. **Nested Objects:** `Address` ist ein eigenes Dataclass, aber Teil des Contact-Dokuments
2. **Optional Fields:** `phone`, `address` können fehlen
3. **Enums:** `ContactCategory` für Type-Safety
4. **Computed Properties:** `full_name` berechnet aus first + last
5. **Default Factories:** Listen und Timestamps werden automatisch initialisiert

```python
from themisdb import Client
from models import Contact, ContactCategory
from typing import List, Optional
import uuid

class ContactClient:
    def __init__(self, host="localhost", port=8765):
        self.client = Client(host, port)
        self._setup_database()

    def _setup_database(self):
        """Erstellt Collection und Indizes"""
        # Collection erstellen (Document Model!)
        try:
            self.client.create_collection("contacts", type="document")
        except:
            pass

        # Indizes für schnelle Suche
        self.client.create_index("contacts", "email_idx", ["email"])
        self.client.create_index("contacts", "category_idx", ["category"])
        self.client.create_index("contacts", "favorite_idx", ["is_favorite"])

        # FULLTEXT Index für Suche!
        self.client.create_index(
            "contacts",
            "fulltext_idx",
            ["first_name", "last_name", "email", "notes"],
            type="fulltext"
        )

    def create_contact(self, contact: Contact) -> bool:
        """Erstellt einen neuen Kontakt"""
        try:
            self.client.insert("contacts", contact.to_dict())
            return True
        except Exception as e:
            print(f"Error creating contact: {e}")
            return False

    def search_contacts(self, query: str) -> List[Contact]:
        """Volltext-Suche über alle Felder"""
        aql = """
        FOR contact IN contacts
            FILTER CONTAINS(LOWER(contact.first_name), LOWER(@query))
                OR CONTAINS(LOWER(contact.last_name), LOWER(@query))
                OR CONTAINS(LOWER(contact.email), LOWER(@query))
                OR CONTAINS(LOWER(contact.notes), LOWER(@query))
            SORT contact.last_name ASC, contact.first_name ASC
            RETURN contact
        """
        try:
            results = self.client.query(aql, {"query": query})
            return [Contact.from_dict(data) for data in results]
```

```python
        except Exception as e:
            print(f"Error searching contacts: {e}")
            return []

    def get_favorites(self) -> List[Contact]:
        """Alle Favoriten-Kontakte"""
        aql = """
        FOR contact IN contacts
            FILTER contact.is_favorite == true
            SORT contact.last_name ASC
            RETURN contact
        """
        try:
            results = self.client.query(aql)
            return [Contact.from_dict(data) for data in results]
        except Exception as e:
            return []

    def get_by_category(self, category: ContactCategory) -> List[Contact]:
        """Kontakte nach Kategorie filtern"""
        aql = """
        FOR contact IN contacts
            FILTER contact.category == @category
            SORT contact.last_name ASC
            RETURN contact
        """
        try:
            results = self.client.query(aql, {"category": category.value})
            return [Contact.from_dict(data) for data in results]
        except Exception as e:
            return []

    def export_to_json(self, filename: str) -> bool:
        """Exportiert alle Kontakte als JSON"""
        aql = "FOR contact IN contacts RETURN contact"
        try:
            contacts = self.client.query(aql)
            import json
            with open(filename, 'w', encoding='utf-8') as f:
                json.dump(contacts, f, indent=2, ensure_ascii=False)
            return True
        except Exception as e:
            print(f"Export failed: {e}")
            return False

    def import_from_json(self, filename: str) -> int:
        """Importiert Kontakte aus JSON"""
        import json
        try:
            with open(filename, 'r', encoding='utf-8') as f:
                contacts = json.load(f)

            count = 0
            for contact_data in contacts:
```

```
                contact = Contact.from_dict(contact_data)
                if self.create_contact(contact):
                    count += 1

            return count
        except Exception as e:
            print(f"Import failed: {e}")
            return 0
```

Dokument-Modell in Aktion

Nested Queries:

```
aql = """
FOR contact IN contacts
    FILTER contact.address != null
       AND contact.address.city == "Berlin"
    RETURN contact
"""
```

Array Operations:

```
aql = """
FOR contact IN contacts
    FILTER "important" IN contact.tags
    RETURN contact
"""
```

Schema Evolution ohne Migration:

```
contact = {
    "first_name": "Alice",
    "email": "alice@example.com",
    "social_media": {  # NEU: Einfach hinzugefügt
        "twitter": "@alice",
        "linkedin": "alice-smith"
    },
    "birthday": "1990-03-15"  # NEU: Auch einfach hinzugefügt
}
```

3.4 Multi-Model Kombinationen

Beispiel 1: E-Commerce mit allen Modellen

```
order = {
    "order_id": "O123",
    "user_id": "alice",
    "total": 99.99,
    "status": "paid"
}

product = {
    "sku": "LAPTOP-X1",
    "name": "Laptop",
    "specs": {  # Jedes Produkt hat andere Specs!
        "cpu": "i7",
        "ram": 32
    }
}



"""

FOR friend IN 1..2 OUTBOUND @userId friends
  FOR order IN orders
    FILTER order.user_id == friend.user_id
    FOR product IN products
      FILTER product.sku == order.sku
      LET similarity = COSINE_SIMILARITY(
        product.embedding,
        @my_last_product_embedding
      )
      FILTER similarity > 0.7
        AND product.price < 1000
      RETURN DISTINCT product
"""
```

Beispiel 2: CRM System

```
transactions = {
    "transaction_id": "T123",
    "customer_id": "C456",
    "amount": 5000.00,
    "status": "completed"
}

customer = {
    "customer_id": "C456",
    "company": "ACME Corp",
    "contacts": [  # Array von Kontakten
        {"name": "John", "role": "CEO"},
        {"name": "Jane", "role": "CTO"}
    ],
    "metadata": {  # Flexible metadata
        "industry": "tech",
        "size": "medium"
    }
}
```

3.5 Best Practices

1. Start Simple, Add Complexity

```

```

2. Denormalisierung ist OK

Im Dokument-Modell ist Duplikation oft besser als Joins:

```
order = {"order_id": "O123", "user_id": "alice"}
user = {"user_id": "alice", "name": "Alice", "email": "..."}

order = {
    "order_id": "O123",
    "user": {  # User-Daten dupliziert
        "user_id": "alice",
        "name": "Alice",
        "email": "alice@example.com"
    }
}
```

3. Use Computed Properties

```
@property
def age(self):
    """Berechnet Alter aus Geburtstag"""
    if not self.birthday:
        return None
    today = datetime.now()
    return today.year - self.birthday.year
```

4. Index Strategically

Nicht jedes Feld braucht einen Index:

```
client.create_index("contacts", "email_idx", ["email"])
```

3.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Vier Modelle: Wann Relational, Graph, Dokument oder Vektor
Entscheidungskriterien: Die 5 Fragen-Methode
Kontaktmanager: Vollständige Dokument-Modell Demo
Multi-Model Kombination: Verschiedene Modelle zusammen nutzen
Best Practices: Denormalisierung, Computed Properties, Indexing

Key Takeaways

1. **Es gibt kein "bestes" Modell** - nur das beste für Ihren Use Case
2. **Kombinieren ist Stärke** - ThemisDB glänzt bei Multi-Model
3. **Schema-Flexibilität** - Dokumente sind perfekt für Evolution
4. **Performance** - Wählen Sie das Modell nach Query-Patterns

Nächste Schritte

Im nächsten Kapitel lernen Sie, wie Sie ThemisDB installieren, konfigurieren und für Production vorbereiten.

Kapitel 4: Installation & Setup →

Weiterführende Ressourcen

- **Complete Example:** examples/03_contact_manager
- **Tutorial:** Contact Manager Tutorial
- **Document Model Docs:** ../de/architecture/architecture_content.md
- **Multi-Model Patterns:** Kapitel 13 - AQL Mastery

Kapitel 3 von 30 | Teil I: Grundlagen | ~7.500 Wörter

Kapitel 5: Kapitel 4: Installation und Setup

> *"Die beste Dokumentation hilft nichts, wenn die Software nicht läuft. Installation sollte einfach sein - und das ist sie."*

Überblick

In den ersten drei Kapiteln haben Sie die Konzepte von ThemisDB kennengelernt. Jetzt ist es Zeit, ThemisDB selbst zu installieren und zu konfigurieren. Dieses Kapitel führt Sie durch alle Optionen - von der schnellsten (Docker) bis zur flexibelsten (Source Build).

Was Sie in diesem Kapitel lernen werden: - Installation mit Docker (schnellste Methode) - Installation aus Binaries - Build von Source Code - Konfiguration und Tuning - Production-Setup Best Practices - Troubleshooting häufiger Probleme

Voraussetzungen: Grundlegende Kommandozeilen-Kenntnisse.

4.1 Systemanforderungen

Minimale Requirements

```
CPU: 2 Cores
RAM: 4 GB
Disk: 20 GB (SSD empfohlen)
OS:
  - Linux (Ubuntu 20.04+, Debian 11+, CentOS 8+)
  - macOS 11+
  - Windows 10/11 (via WSL2 oder Docker)
```

Empfohlene Production Requirements

```
CPU: 8+ Cores
RAM: 32 GB
Disk: 500 GB NVMe SSD
OS: Linux (Ubuntu 22.04 LTS empfohlen)
Network: 10 Gbps
```

Hardware-Überlegungen

CPU: - ThemisDB ist multi-threaded - Mehr Cores = höherer Durchsatz - Minimum 2, empfohlen 8+

RAM: - Wird für Caching genutzt - 25% des Datasets sollte in RAM passen - Minimum 4 GB, empfohlen 32 GB+

Storage: - SSDs sind **stark** empfohlen - ThemisDB nutzt LSM-Trees (sequential writes) - NVMe > SATA SSD >> HDD - RAID 10 für Production

Disk Performance Vergleich:

Storage Type	Random Read IOPS	Sequential Write MB/s
HDD 7200 RPM	100-200	150
SATA SSD	50,000	500
NVMe SSD	500,000	3,500

ThemisDB Benefit: Mit NVMe SSD 10-50x schneller als HDD!

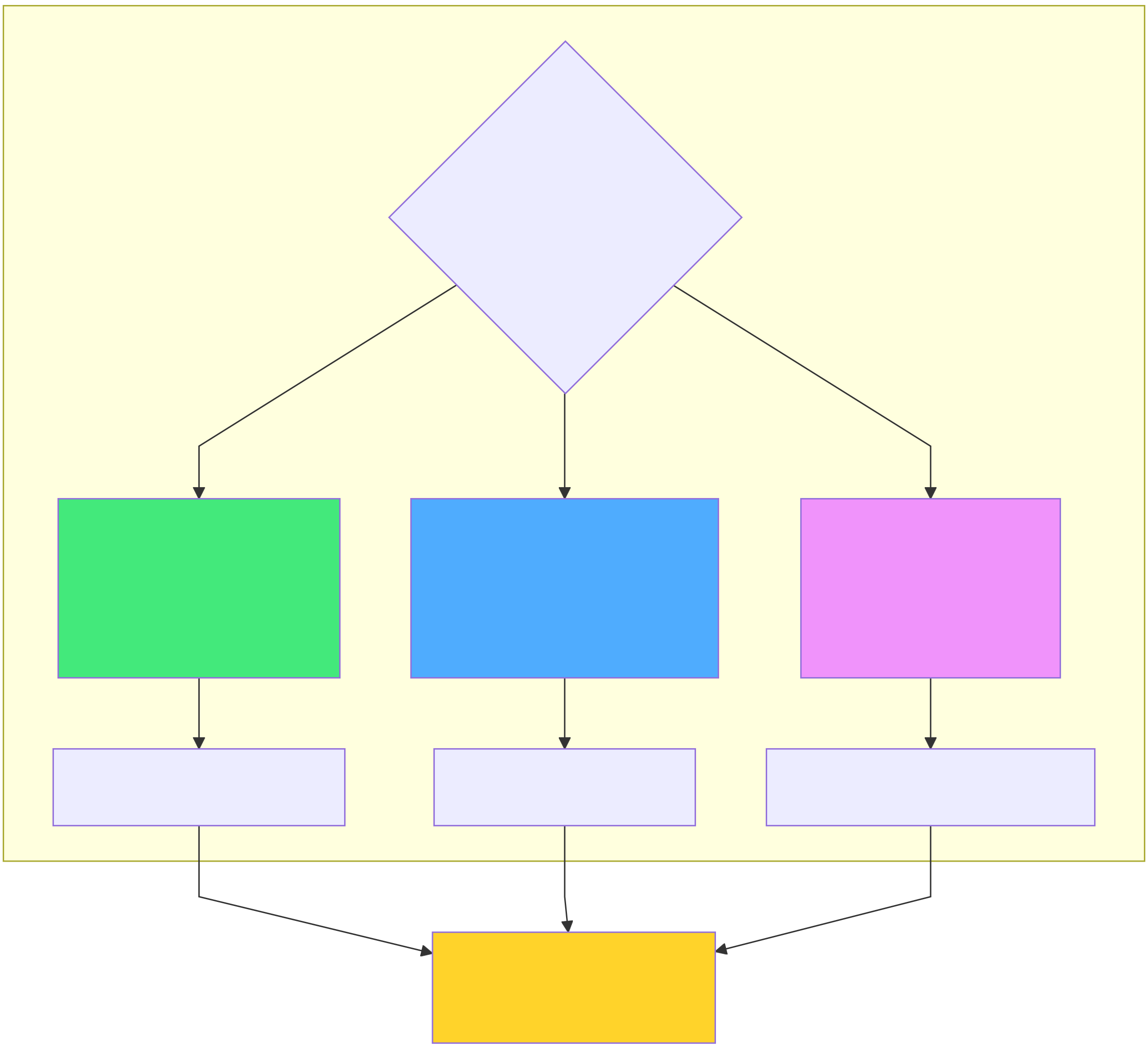


Abb. 16: Architekturdiagramm (Kapitel 4: Installation und Setup)

4.2 Installation mit Docker (Schnellste Methode)

Warum Docker?

- Funktioniert sofort (keine Dependencies)
- Isolation vom Host-System
- Einfache Updates
- Konsistent über alle Plattformen

Quick Start

```
docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:latest

curl http://localhost:8765/health
```

Fertig! ThemisDB läuft jetzt.

Mit spezifischer Version

```
docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:1.3.4  # Fixe Version
```

Best Practice: Verwende in Production immer eine fixe Version, nicht `latest`.

Mit Konfiguration

```
cat > themisdb.yaml << EOF
server:
  port: 8765
  threads: 8

storage:
  path: /data
  cache_size_mb: 2048

logging:
  level: info
  file: /logs/themisdb.log
EOF

docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  -v themisdb_logs:/logs \
  -v $(pwd)/themisdb.yaml:/etc/themisdb/config.yaml \
  themisdb/themisdb:1.3.4 \
  --config /etc/themisdb/config.yaml
```

Docker Compose

Für komplexere Setups mit mehreren Containern:

```
version: '3.8'

services:
  themisdb:
    image: themisdb/themisdb:1.3.4
    container_name: themisdb
    ports:
      - "8765:8765"
    volumes:
      - themisdb_data:/data
      - themisdb_logs:/logs
      - ./config.yaml:/etc/themisdb/config.yaml
    environment:
      - THEMISDB_LOG_LEVEL=info
      - THEMISDB_CACHE_SIZE=2048
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8765/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  # Optional: Monitoring mit Prometheus
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

volumes:
  themisdb_data:
  themisdb_logs:
```

Starten:

```
docker-compose up -d
```

4.3 Installation aus Binaries

Download

```
curl -L -o themisdb.tar.gz \
  https://github.com/makr-code/ThemisDB/releases/download/v1.3.4/themisdb-linux-x86_64.tar.gz

curl -L -o themisdb.tar.gz \
  https://github.com/makr-code/ThemisDB/releases/download/v1.3.4/themisdb-darwin-arm64.tar.gz

tar xzf themisdb.tar.gz
cd themisdb
```

Installation

```
sudo cp bin/themisdb /usr/local/bin/
sudo mkdir -p /etc/themisdb
sudo cp config/themisdb.yaml /etc/themisdb/

mkdir -p ~/themisdb/{bin,data,logs}
cp bin/themisdb ~/themisdb/bin/
cp config/themisdb.yaml ~/themisdb/
```

Starten

```
sudo systemctl start themisdb
sudo systemctl enable themisdb  # Autostart

themisdb --config /etc/themisdb/config.yaml

nohup themisdb --config themisdb.yaml > logs/themisdb.log 2>&1 &
```

4.4 Build von Source

Warum von Source bauen?

- Neueste Features (Entwicklungs-Branch)
- Custom Optimierungen
- Spezielle Plattformen (ARM, RISC-V)
- Beiträge entwickeln

Dependencies installieren

Ubuntu/Debian:

```
sudo apt-get update
sudo apt-get install -y \
  build-essential \
  cmake \
  git \
  libssl-dev \
  libz-dev \
  libbz2-dev \
  liblz4-dev \
  libzstd-dev
```

macOS:

```
brew install cmake openssl zlib bzip2 lz4 zstd
```

Clone und Build

```
git clone https://github.com/makr-code/ThemisDB.git
cd ThemisDB

git submodule update --init --recursive

mkdir build && cd build
cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=/usr/local

make -j$(nproc)

make test

sudo make install
```

Build-Optionen

```
cmake .. -DCMAKE_BUILD_TYPE=Debug

cmake .. \
  -DCMAKE_BUILD_TYPE=Debug \
  -DENABLE_ASAN=ON \
  -DENABLE_UBSAN=ON

cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DNATIVE_ARCH=ON

cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DWITH_JEMALLOC=ON \
  -DWITH_SNAPPY=ON \
  -DWITH_LZ4=ON \
  -DWITH_ZSTD=ON
```

4.5 Konfiguration

Basis-Konfiguration

```yaml
server:
  host: 0.0.0.0            # Bind auf alle Interfaces
  port: 8765               # Standard Port
  threads: 8               # Worker Threads (= CPU Cores)
  max_connections: 1000    # Maximale gleichzeitige Connections

storage:
  path: /var/lib/themisdb/data
  cache_size_mb: 2048      # RocksDB Block Cache
  write_buffer_size_mb: 64
  max_open_files: 1000
  compression: lz4         # lz4, zstd, snappy, none

logging:
  level: info              # debug, info, warning, error
  file: /var/log/themisdb/themisdb.log
  max_size_mb: 100
  max_backups: 10

security:
  tls_enabled: false
  tls_cert: /etc/themisdb/cert.pem
  tls_key: /etc/themisdb/key.pem
  auth_enabled: false
  auth_db: /etc/themisdb/users.db
```

Performance Tuning

```yaml
storage:
  cache_size_mb: 8192      # Mehr Cache = weniger Disk I/O
  write_buffer_size_mb: 256
  max_background_jobs: 8    # Parallel Compaction
  level0_slowdown_writes_trigger: 20
  level0_stop_writes_trigger: 36

server:
  threads: 16              # Mehr Threads für mehr Parallelität
  max_connections: 5000
```

```yaml
storage:
  cache_size_mb: 16384     # Großer Cache
  write_buffer_size_mb: 128
  max_background_jobs: 4   # Weniger Background I/O

server:
  threads: 8
  max_connections: 1000
```

Environment Variables

Überschreiben Config-File Werte:

```
export THEMISDB_PORT=9000

export THEMISDB_LOG_LEVEL=debug

export THEMISDB_CACHE_SIZE=4096

themisdb
```

4.6 Production Setup

Systemd Service

```
[Unit]
Description=ThemisDB Multi-Model Database
After=network.target

[Service]
Type=simple
User=themisdb
Group=themisdb
ExecStart=/usr/local/bin/themisdb --config /etc/themisdb/config.yaml
Restart=on-failure
RestartSec=5s

NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/themisdb /var/log/themisdb

LimitNOFILE=65536
LimitNPROC=4096

[Install]
WantedBy=multi-user.target
```

Aktivieren:

```
sudo systemctl daemon-reload
sudo systemctl enable themisdb
sudo systemctl start themisdb
```

Monitoring Setup

Health Endpoint:

```
curl http://localhost:8765/health
```

Metrics Endpoint (Prometheus):

```
curl http://localhost:8765/metrics
```

Grafana Dashboard: Importiere vorgefertigtes Dashboard: `dashboards/grafana-themisdb.json`

Backup Strategy

1. Snapshot-basiertes Backup:

```
curl -X POST http://localhost:8765/admin/snapshot

/var/lib/themisdb/snapshots/snapshot-2025-12-28-100000/

rsync -av /var/lib/themisdb/snapshots/ backup-server:/backups/themisdb/
```

2. Continuous Backup:

```
rsync -av --delete \
  /var/lib/themisdb/data/WAL/ \
  backup-server:/backups/themisdb/wal/
```

3. Restore:

```
sudo systemctl stop themisdb

rsync -av backup-server:/backups/themisdb/latest/ /var/lib/themisdb/data/

sudo systemctl start themisdb
```

Security Hardening

1. TLS aktivieren:

```
security:
  tls_enabled: true
  tls_cert: /etc/themisdb/cert.pem
  tls_key: /etc/themisdb/key.pem
```

2. Authentifizierung:

```
themisdb-admin user create alice --password secret123 --role admin
```

```
security:
  auth_enabled: true
  auth_db: /etc/themisdb/users.db
```

3. Firewall:

```
sudo ufw allow from 10.0.0.0/24 to any port 8765
sudo ufw enable
```

4. Network Isolation:

```
server:
  host: 10.0.0.10  # Nicht 0.0.0.0!
```

4.7 Troubleshooting

Problem: Port bereits in Verwendung

Symptom:

```
ERROR: Failed to bind on port 8765: Address already in use
```

Lösung:

```
sudo lsof -i :8765
sudo netstat -tulpn | grep 8765

sudo systemctl stop <service>

server:
  port: 9000
```

Problem: Zu wenig Memory

Symptom:

```
WARNING: Cache size exceeds available memory
ERROR: Out of memory
```

Lösung:

```
storage:
  cache_size_mb: 1024  # Statt 8192
  write_buffer_size_mb: 32
```

Problem: Langsame Queries

Symptom: Queries dauern > 1 Sekunde

Diagnose:

```
export THEMISDB_LOG_LEVEL=debug

tail -f /var/log/themisdb/themisdb.log | grep "Query execution"
```

Lösung:

```
-- Prüfe ob Indizes genutzt werden
EXPLAIN FOR user IN users FILTER user.email == 'test@example.com' RETURN user

-- Output sollte zeigen: "using index: email_idx"
-- Falls nicht: Index erstellen!
```

Problem: Disk voll

Symptom:

```
ERROR: No space left on device
```

Lösung:

```
df -h /var/lib/themisdb

curl -X POST http://localhost:8765/admin/compact

rm -rf /var/lib/themisdb/snapshots/snapshot-old-*

themisdb-admin wal cleanup --keep-last 10
```

Problem: Connection Timeouts

Symptom:

```
ERROR: Connection timed out after 10s
```

Lösung:

```
server:
  read_timeout_s: 30
  write_timeout_s: 30
  idle_timeout_s: 300

server:
  max_connections: 5000
```

4.8 Updates und Wartung

Update-Prozess

Docker:

```
docker pull themisdb/themisdb:1.3.4

docker stop themisdb

docker run -d \
  --name themisdb \
  -p 8765:8765 \
  -v themisdb_data:/data \
  themisdb/themisdb:1.3.4

docker rm themisdb-old
```

Binary:

```
sudo systemctl stop themisdb
tar czf themisdb-backup.tar.gz /var/lib/themisdb/data

sudo cp themisdb-new /usr/local/bin/themisdb

sudo systemctl start themisdb

curl http://localhost:8765/health
```

Rolling Update (Zero-Downtime)

Für Cluster-Setups:

```
kubectl set image deployment/themisdb-node2 themisdb=themisdb:1.3.4

kubectl wait --for=condition=ready pod -l app=themisdb-node2

kubectl set image deployment/themisdb-node1 themisdb=themisdb:1.3.4
```

4.9 Diagnostics & Hardening (Kurz)

Health & Metrics: - `GET /health` (liveness), `GET /metrics` (Prometheus) - Log-Level zur Fehlersuche: `export THEMISDB_LOG_LEVEL=debug`

Security-Hardening: - TLS aktivieren: `--tls.cert` / `--tls.key` - mTLS optional: Client-Certs whitelisten - Admin-API absichern: IP-Whitelist + Auth-Header

Storage-Pflege: - Wöchentliche Compaction: `POST /admin/compact` - Snapshot-Rotation: Keep-last-7, Air-Gap-Upload

Performance-Probe: - `EXPLAIN` für jede neue Query - Page Cache treffen: `cache_size_mb` auf 20-30% RAM

4.10 Deployment-Playbooks

Single Node (Pilot)

- Docker-Run mit persistentem Volume (`themisdb_data`)
- Backup via Snapshot-API + Offsite-Kopie
- Monitoring: Health + Metrics → Prometheus

HA (2-3 Nodes, Sync-Replicas)

- Node-Labels: `role=primary` , `role=secondary`
- Synchronous Commit für RPO=0
- Load Balancer vor 2 Read-Replicas

Airgapped Install

- Offline-Bundle: Binary + Config + Checksums
- Pakete signieren, Verify via `sha256sum`
- Updates nur über signierte Tarballs einspielen

4.11 Operations-Checkliste (Go-Live)

- [ ] TLS 1.3 aktiviert, mTLS optional getestet
- [ ] Backups automatisiert + Restore-Drill bestanden
- [ ] Monitoring: Health, Metrics, Logs zentralisiert
- [ ] Admin-API eingeschränkt (Firewall + Auth)
- [ ] Config-Lint: Cache, Write-Buffer, Thread-Count passend
- [ ] Runbooks: Incident, Restore, Scale-Up

4.12 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Systemanforderungen: CPU, RAM, Storage Best Practices
Docker Installation: Schnellste Methode, Production-ready
Binary Installation: Systemweit oder lokal

Source Build: Für Custom Builds und Development
Konfiguration: Tuning für Performance und Security
Production Setup: Systemd, Monitoring, Backup
Troubleshooting: Häufige Probleme und Lösungen
Updates: Safe Update-Prozesse

Key Takeaways

1. **Docker ist der einfachste Weg** - funktioniert sofort
2. **SSDs sind essentiell** - 10-50x Performance-Gewinn
3. **Monitoring ist wichtig** - Health + Metrics endpoints
4. **Backups sind Pflicht** - Snapshots + WAL Replication
5. **Security nicht vergessen** - TLS + Auth für Production

Sie sind bereit!

ThemisDB ist jetzt installiert und läuft. Im nächsten Teil des Kompendiums (Teil II) tauchen wir tiefer in die einzelnen Datenmodelle ein.

Teil II: Kapitel 5 - Relationale Daten →

Weiterführende Ressourcen

- **Docker Deployment:** ../de/deployment/DOCKER_DEPLOYMENT.md
- **Build Optionen:** ../de/deployment/BUILD_OPTIONEN_REFERENZ.md
- **Production Strategy:** ../de/deployment/deployment_strategy.md
- **ARM Deployment:** ../de/deployment/deployment_arm_build.md

---

Kapitel 4 von 30 | Teil I: Grundlagen | ~6.500 Wörter

Kapitel 6: Kapitel 5: Relationale Daten

> *"Relational databases are like spreadsheets that can talk to each other - but with ACID guarantees and without the chaos."*

Überblick

Relationale Datenbanken sind das Rückgrat der IT seit über 40 Jahren. In ThemisDB ist das relationale Modell eines von vier gleichberechtigten Datenmodellen - aber mit vollem AQL-Support und ACID-Garantien.

Was Sie in diesem Kapitel lernen werden: - Relationales Datenmodell: Tabellen, Schemas, Constraints - AQL in ThemisDB: DDL, DML, Joins, Subqueries - Normalisierung vs. Denormalisierung - Transaktionen und Integrität - Indexes und Performance - **Praxisbeispiel 1:** Inventory System (Lagerverwaltung) - **Praxisbeispiel 2:** Expense Tracker (Ausgabenverwaltung)

Voraussetzungen: AQL-Grundkenntnisse hilfreich, aber nicht notwendig.

5.1 Das Relationale Modell

Grundkonzepte

Tabelle (Table): Sammlung von Zeilen mit gleichem Schema

```
products (Tabelle)
+----+----------+-------+-------+
| id | name     | price | stock |
+----+----------+-------+-------+
|  1 | Laptop   |  1200 |    15 |
|  2 | Mouse    |    25 |   100 |
|  3 | Keyboard |    80 |    50 |
+----+----------+-------+-------+
```

Zeile (Row): Ein Datensatz in einer Tabelle
Spalte (Column): Ein Attribut, das jede Zeile hat
Schema: Definition der Spalten und Typen
Primärschlüssel (Primary Key): Eindeutige ID für jede Zeile
Fremdschlüssel (Foreign Key): Referenz zu einer anderen Tabelle

Warum Relational?

Vorteile: - **Struktur:** Klare, vorhersehbare Datenstruktur - **Integrität:** Constraints garantieren Datenqualität - **Joins:** Verknüpfe Daten aus mehreren Tabellen - **ACID:** Transaktionen mit vollständiger Konsistenz - **AQL:** Standardisierte, mächtige Abfragesprache

× **Nachteile:** - Rigid: Schema-Änderungen erfordern Migrations - Komplex: Normalisierung kann zu vielen Tables führen - Performance: Joins können langsam sein bei vielen Tables - Skalierung: Vertical Scaling bevorzugt

Wann Relational wählen?

Perfekt für: - Strukturierte Business-Daten (Orders, Invoices, Inventory) - Daten mit klaren Beziehungen (Customers → Orders → Items) - Transaktionale Systeme (Banking, E-Commerce) - Reports mit Aggregationen (SUM, AVG, GROUP BY) - Datenintegrität ist kritisch

Weniger geeignet für: - × Hochflexible Schemas (Metadata, User-Generated Content) - × Netzwerk-Daten mit vielen Hops (Social Graphs) - × Unstrukturierte Daten (Logs, Dokumente) - × Vektor-Daten (Embeddings, Features)

5.2 Tabellen und Schemas

Tabelle erstellen

```
-- Basic Table
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  stock INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Datentypen in ThemisDB:

Typ	Beschreibung	Beispiel
INT	Ganzzahl	42, -17, 0
BIGINT	Große Ganzzahl	9223372036854775807
DECIMAL(p,s)	Festkommazahl	123.45
FLOAT/DOUBLE	Fließkommazahl	3.14159
VARCHAR(n)	Variable Zeichenkette	"Hello World"
TEXT	Unbegrenzte Zeichenkette	Langer Text
BOOLEAN	Wahrheitswert	true, false
TIMESTAMP	Zeitstempel	2025-12-28 10:30:00
DATE	Datum	2025-12-28
JSON	JSON-Dokument	{"key": "value"}

Constraints

Primary Key:

```
CREATE TABLE customers (
  id INT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL
);
```

Foreign Key:

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  total DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

Check Constraints:

```
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
  stock INT NOT NULL CHECK (stock >= 0),
  discount DECIMAL(3, 2) CHECK (discount BETWEEN 0 AND 1)
);
```

Unique Constraints:

```
CREATE TABLE users (
  id INT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL
);
```

Auto-Increment IDs

```
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL
);

-- Insert ohne ID
INSERT INTO products (name) VALUES ('Laptop');
-- → Automatisch id=1

INSERT INTO products (name) VALUES ('Mouse');
-- → Automatisch id=2
```

5.3 Daten einfügen, ändern, löschen

INSERT

```
-- Single Document
INSERT {
  id: 1,
  name: 'Laptop',
  price: 1200.00,
  stock: 15
} INTO products

-- Multiple Documents
INSERT [
  {id: 2, name: 'Mouse', price: 25.00, stock: 100},
  {id: 3, name: 'Keyboard', price: 80.00, stock: 50},
  {id: 4, name: 'Monitor', price: 350.00, stock: 20}
] INTO products
```

UPDATE

```
-- Update single document
UPDATE {id: 1} WITH {stock: stock - 1} IN products

-- Update multiple documents (with FOR loop)
FOR product IN products
  FILTER product.stock > 50
  UPDATE product WITH {
    price: product.price * 0.9
  } IN products

-- Update with join data
FOR order_item IN order_items
  FOR product IN products
    FILTER order_item.product_id == product.id
    UPDATE order_item WITH {
      price: product.price
    } IN order_items
```

REMOVE (DELETE)

```
-- Remove single document
REMOVE {id: 1} IN products

-- Remove multiple documents
FOR product IN products
  FILTER product.stock == 0
  REMOVE product IN products

-- Remove all (Vorsicht!)
FOR product IN products
  REMOVE product IN products
```

UPSERT (Insert or Update)

```
-- Insert or Update based on key
UPSERT {id: 1}
  INSERT {
    id: 1,
    name: 'Laptop',
    price: 1200.00,
    stock: 15
  }
  UPDATE {
    price: 1200.00,
    stock: 15
  }
IN products
```

5.4 Queries und Joins

Query Basics

```
-- All columns
FOR product IN products
  RETURN product

-- Specific columns
FOR product IN products
  RETURN {name: product.name, price: product.price}

-- With FILTER
FOR product IN products
  FILTER product.price > 100
  RETURN product

-- With SORT and LIMIT
FOR product IN products
  SORT product.price DESC
  LIMIT 10
  RETURN product

-- With aggregation functions
FOR product IN products
  COLLECT AGGREGATE
    avgPrice = AVG(product.price),
    maxPrice = MAX(product.price),
    minPrice = MIN(product.price),
    total = COUNT()
  RETURN {avgPrice, maxPrice, minPrice, total}

-- With GROUP BY (COLLECT)
FOR product IN products
  COLLECT category = product.category
  AGGREGATE
    count = COUNT(),
    avgPrice = AVG(product.price)
  RETURN {category, count, avgPrice}
```

INNER JOIN

```
-- Orders mit Customer-Namen (Multi-FOR Join)
FOR order IN orders
  FOR customer IN customers
    FILTER order.customer_id == customer.id
    RETURN {
      id: order.id,
      customer_name: customer.name,
      total: order.total
    }
```

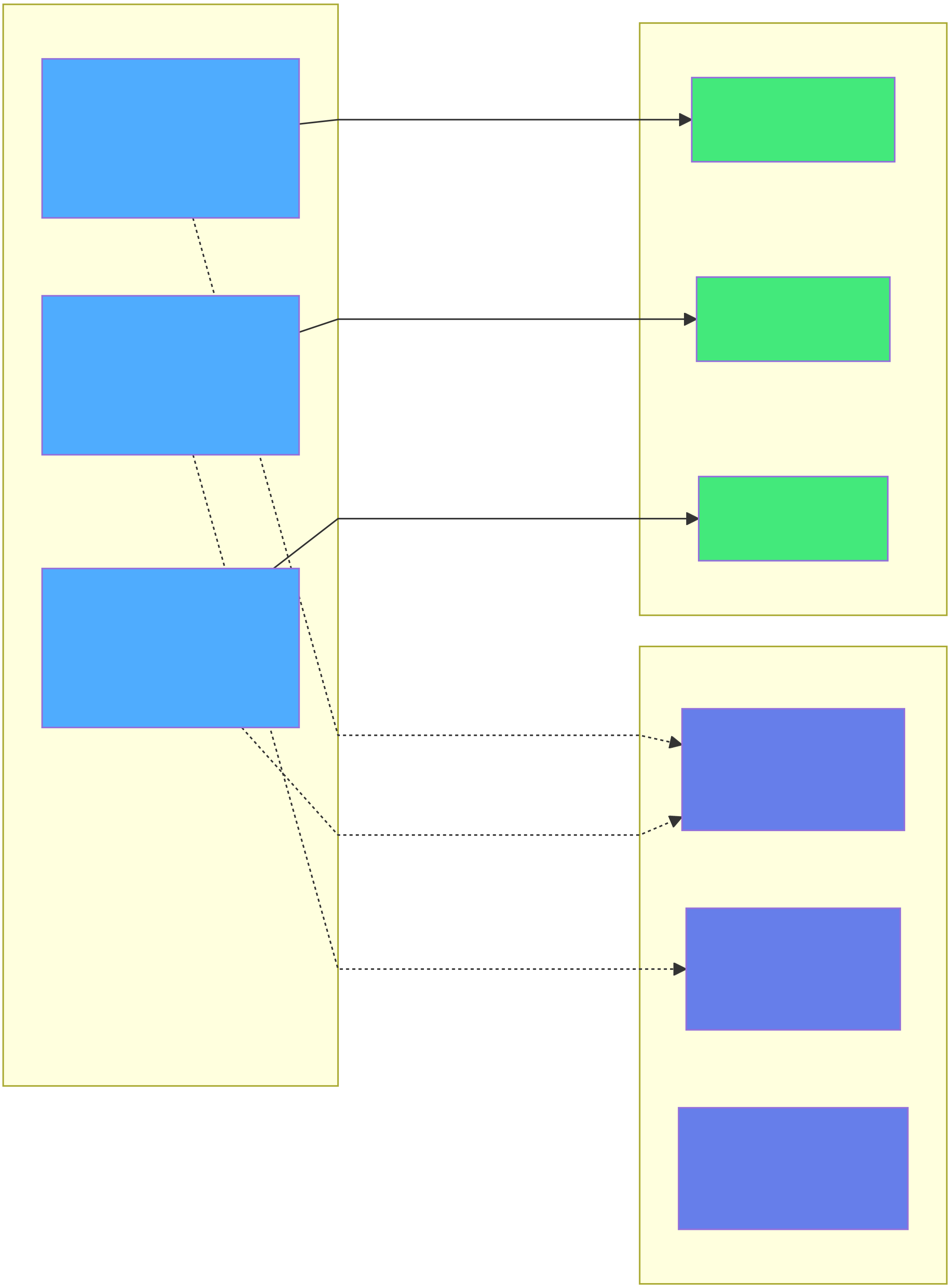


Abb. 17: Architekturdiagramm: INNER JOIN

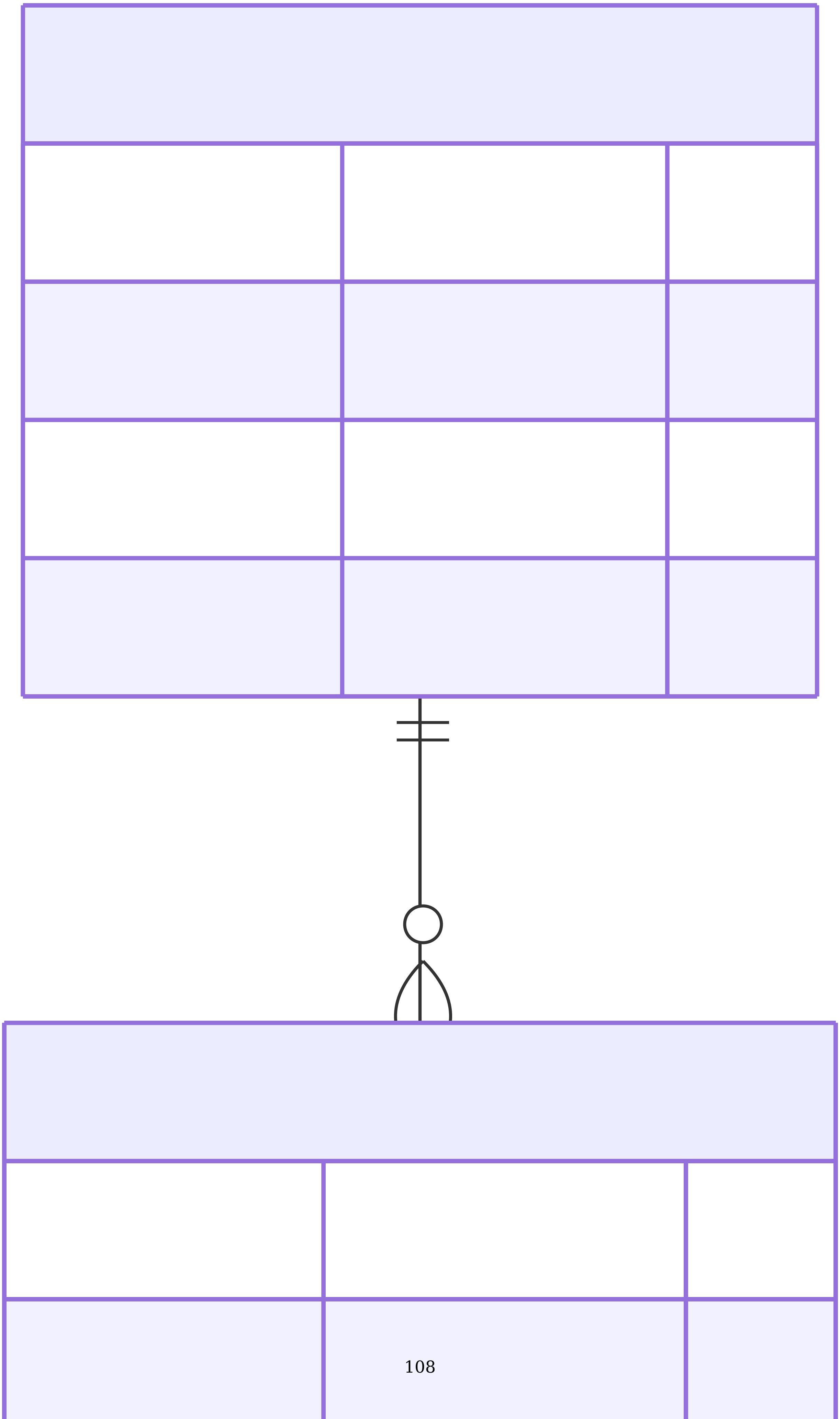


Abb. 18: Entitäts-Beziehungsdiagramm (Kapitel 5: Relationale Daten)

```
  total: order.total,
  customer_name: customer.name
}
```

```
**Visualisierung:**
```

customers orders +---------+ +---------+ | id=1 |←--------|cust_id=1| ✓ Match → returned | Alice | | €100 | +---------+ +---------+ | id=2 | | cust_id=1| ✓ Match → returned | Bob | | €50 | +---------+ +---------+ | id=3 |
| Carol | → Kein Match, nicht returned +---------+

```
### LEFT JOIN (Outer Join with COLLECT)

```aql
-- Alle Customers, mit/ohne Orders
FOR customer IN customers
  LET customer_orders = (
    FOR order IN orders
      FILTER order.customer_id == customer.id
      RETURN order
  )
  RETURN {
    name: customer.name,
    order_count: LENGTH(customer_orders),
    total_spent: SUM(customer_orders[*].total)
  }
```

Visualisierung:

```
customers           orders
+---------+         +---------+
| id=1    |←--------|cust_id=1|  ✓ Match
| Alice   |         | €100    |
+---------+         +---------+
| id=2    |
| Bob     |         → Kein Order, aber returned mit NULL
+---------+
```
```

Multi-Table JOIN

```
-- Orders mit Items und Products (Multi-FOR Join)
FOR order IN orders
  FILTER order.created_at >= '2025-01-01'
  FOR customer IN customers
    FILTER order.customer_id == customer.id
    FOR order_item IN order_items
      FILTER order_item.order_id == order.id
      FOR product IN products
        FILTER order_item.product_id == product.id
        RETURN {
          order_id: order.id,
          customer: customer.name,
          product: product.name,
          quantity: order_item.quantity,
          price: order_item.price
        }
```

Subqueries

```
-- Products teurer als Durchschnitt (WITH CTE)
LET avgPrice = (
  FOR product IN products
    COLLECT AGGREGATE avg = AVG(product.price)
    RETURN avg
)[0]

FOR product IN products
  FILTER product.price > avgPrice
  RETURN {name: product.name, price: product.price}

-- Customers mit mehr als 3 Orders (Subquery)
LET active_customers = (
  FOR order IN orders
    COLLECT customer_id = order.customer_id
    AGGREGATE order_count = COUNT()
    FILTER order_count > 3
    RETURN customer_id
)

FOR customer IN customers
  FILTER customer.id IN active_customers
  RETURN customer.name
```

5.5 Transaktionen

ACID Garantien

Atomicity: Alles oder nichts

Consistency: Constraints werden eingehalten

Isolation: Transaktionen sehen sich nicht gegenseitig
Durability: Commits sind permanent

Transaction Beispiel

```
from themis_client import ThemisDB

db = ThemisDB("localhost:8765")

tx = db.begin_transaction()

try:
    # 1. Reduce stock
    tx.execute("""
        FOR product IN products
          FILTER product.id == @product_id AND product.stock > 0
          UPDATE product WITH {stock: product.stock - 1} IN products
    """, {"product_id": product_id})

    # 2. Create order
    tx.execute("""
        INSERT {
          customer_id: @customer_id,
          product_id: @product_id,
          quantity: 1,
          price: @price
        } INTO orders
    """, {"customer_id": customer_id, "product_id": product_id, "price": price})

    # 3. Charge customer
    tx.execute("""
        FOR customer IN customers
          FILTER customer.id == @customer_id AND customer.balance >= @price
          UPDATE customer WITH {balance: customer.balance - @price} IN customers
    """, {"customer_id": customer_id, "price": price})

    # All OK → Commit
    tx.commit()
    print("Order successful!")

except Exception as e:
    # Error → Rollback
    tx.rollback()
    print(f"Order failed: {e}")
```

Was passiert intern:

```
T1: BEGIN TRANSACTION (snapshot @ version 100)
  → UPDATE products ... (write intent @ version 101)
  → INSERT orders ... (write intent @ version 102)
  → UPDATE customers ... (write intent @ version 103)
  → COMMIT
    → All write intents become visible @ version 104

Falls Fehler:
  → ROLLBACK
    → All write intents discarded
    → Daten unverändert
```

Isolation Levels

ThemisDB verwendet **Snapshot Isolation**:

```
tx1 = db.begin_transaction()
tx1.execute("FOR p IN products FILTER p.id == 1 UPDATE p WITH {price: 100} IN products")

tx2 = db.begin_transaction()
result = tx2.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result)  # → Alter Wert! (z.B. 90)

tx1.commit()

result = tx2.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result)  # → Immernoch 90!

tx2.commit()

tx3 = db.begin_transaction()
result = tx3.execute("FOR p IN products FILTER p.id == 1 RETURN p.price")
print(result)  # → 100
```

5.6 Normalisierung

Problem: Redundanz

Nicht-normalisiert:

```
orders
+----+----------+-------------+------------+----------+-------+
| id | customer | cust_email  | product    | quantity | price |
+----+----------+-------------+------------+----------+-------+
|  1 | Alice    | a@email.com | Laptop     |        1 |  1200 |
|  2 | Alice    | a@email.com | Mouse      |        2 |    25 |
|  3 | Bob      | b@email.com | Keyboard   |        1 |    80 |
+----+----------+-------------+------------+----------+-------+
```

Probleme: - Customer email wird wiederholt (Update Anomaly) - Product-Namen inkonsistent ("Laptop" vs "laptop") - Keine Constraints möglich

Normalisierung: 1NF, 2NF, 3NF

1. Normal Form (1NF): Atomare Werte, keine Arrays

```
-- × Nicht 1NF
CREATE TABLE orders (
  id INT,
  products TEXT  -- "Laptop, Mouse, Keyboard"
);

--   1NF
CREATE TABLE orders (
  id INT,
  product_id INT
);
```

2. Normal Form (2NF): Alle Nicht-Key-Attribute hängen vom ganzen Key ab

```
-- × Nicht 2NF
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT,
  product_name VARCHAR(255),  -- Hängt nur von product_id ab!
  PRIMARY KEY (order_id, product_id)
);

--   2NF: Separate products table
CREATE TABLE products (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);

CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

3. Normal Form (3NF): Keine transitiven Abhängigkeiten

```
-- × Nicht 3NF
CREATE TABLE products (
  id INT PRIMARY KEY,
  category_id INT,
  category_name VARCHAR(255)  -- Hängt von category_id ab!
);

--   3NF: Separate categories table
CREATE TABLE categories (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);

CREATE TABLE products (
  id INT PRIMARY KEY,
  category_id INT,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

Denormalisierung für Performance

Manchmal ist **bewusste** Denormalisierung sinnvoll:

```
-- Denormalisiert für schnelle Queries
CREATE TABLE order_summary (
  order_id INT PRIMARY KEY,
  customer_id INT,
  customer_name VARCHAR(255),  -- Denormalisiert!
  item_count INT,               -- Computed!
  total DECIMAL(10, 2),         -- Computed!
  created_at TIMESTAMP
);
```

Trade-off: -   Queries sind schneller (kein JOIN nötig) - × Updates sind komplexer (mehrere Tables updaten) - × Mehr Storage (Redundanz)

5.7 Indexes

Warum Indexes?

Ohne Index:

```
FOR product IN products
  FILTER product.name == 'Laptop'
  RETURN product
-- → Scannt ALLE Zeilen (Seq Scan): O(n)
```

Mit Index:

```
CREATE INDEX idx_products_name ON products(name);

FOR product IN products
  FILTER product.name == 'Laptop'
  RETURN product
-- → Nutzt Index (Index Scan): O(log n)
```

Performance-Unterschied: - 1 Million rows: Seq Scan ~100ms, Index Scan ~0.1ms - **1000x schneller!**

Index-Arten

1. B-Tree Index (Standard):

```
CREATE INDEX idx_products_price ON products(price);

-- Gut für:
FOR product IN products
  FILTER product.price > 100
  RETURN product

FOR product IN products
  FILTER product.price >= 50 AND product.price <= 150
  RETURN product

FOR product IN products
  SORT product.price
  RETURN product
```

2. Unique Index:

```
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Verhindert Duplikate automatisch
INSERT {email: 'test@example.com'} INTO users  -- OK
INSERT {email: 'test@example.com'} INTO users  -- ERROR!
```

3. Composite Index:

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at);

-- Gut für:
FOR order IN orders
  FILTER order.customer_id == 123 AND order.created_at > '2025-01-01'
  RETURN order

FOR order IN orders
  FILTER order.customer_id == 123
  RETURN order  -- Auch OK!

-- Nicht gut für:
FOR order IN orders
  FILTER order.created_at > '2025-01-01'
  RETURN order  -- Index nicht nutzbar!
```

4. Partial Index:

```
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';

-- Kleiner Index nur für pending orders
FOR order IN orders
  FILTER order.status == 'pending' AND order.created_at > '2025-01-01'
  RETURN order
```

Index Best Practices

Index erstellen für: - Primary Keys (automatisch) - Foreign Keys (manuell) - WHERE-Spalten in häufigen Queries - ORDER BY-Spalten - JOIN-Spalten

× **Zu viele Indexes vermeiden:** - Jeder Index verlangsamt INSERT/UPDATE/DELETE - Jeder Index braucht Storage - **Faustregel:** Max 5-7 Indexes pro Table

---

5.8 Praxisbeispiel 1: Inventory System

Überblick

Ein **Lagerverwaltungssystem** für ein kleines bis mittelgroßes Unternehmen: - Products mit Categories - Warehouses (mehrere Standorte) - Stock Levels pro Warehouse - Stock Movements (Ein-/Ausgang) - Suppliers

Location: `examples/04_inventory_system/`

Datenmodell

```sql
-- Categories
CREATE TABLE categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL UNIQUE,
  description TEXT
);

-- Products
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  sku VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  category_id INT NOT NULL,
  unit_price DECIMAL(10, 2) NOT NULL CHECK (unit_price >= 0),
  min_stock_level INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (category_id) REFERENCES categories(id),
  INDEX idx_products_category (category_id),
  INDEX idx_products_sku (sku)
);

-- Warehouses
CREATE TABLE warehouses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  location VARCHAR(255),
  capacity INT
);

-- Stock Levels
CREATE TABLE stock_levels (
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
  last_updated TIMESTAMP DEFAULT NOW(),

  PRIMARY KEY (product_id, warehouse_id),
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);

-- Stock Movements
CREATE TABLE stock_movements (
  id INT PRIMARY KEY AUTO_INCREMENT,
  product_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity INT NOT NULL,  -- Positive = Eingang, Negative = Ausgang
  movement_type ENUM('purchase', 'sale', 'transfer', 'adjustment'),
  reference VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (product_id) REFERENCES products(id),
```

```
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id),
  INDEX idx_movements_product (product_id),
  INDEX idx_movements_warehouse (warehouse_id),
  INDEX idx_movements_date (created_at)
);

-- Suppliers
CREATE TABLE suppliers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  contact_email VARCHAR(255),
  contact_phone VARCHAR(50)
);

-- Product Suppliers (Many-to-Many)
CREATE TABLE product_suppliers (
  product_id INT NOT NULL,
  supplier_id INT NOT NULL,
  supplier_sku VARCHAR(50),
  unit_cost DECIMAL(10, 2),

  PRIMARY KEY (product_id, supplier_id),
  FOREIGN KEY (product_id) REFERENCES products(id),
  FOREIGN KEY (supplier_id) REFERENCES suppliers(id)
);
```

Häufige Queries

1. Total Stock pro Product:

```
FOR p IN products
  FILTER p.id == @product_id
  LET total = SUM(sl.quantity FOR sl IN stock_levels FILTER sl.product_id == p.id)
  RETURN { name: p.name, total_stock: total }
```

2. Low Stock Alert:

```
FOR p IN products
  LET total = SUM(sl.quantity FOR sl IN stock_levels FILTER sl.product_id == p.id)
  FILTER total < p.min_stock_level
  SORT total ASC
  RETURN { sku: p.sku, name: p.name, total_stock: total, min_level: p.min_stock_level }
```

3. Stock Movement History:

```
LET cutoff = DATE_ADD(NOW(), {days: -@days})
FOR sm IN stock_movements
  FILTER sm.product_id == @product_id && sm.created_at >= cutoff
  LET warehouse = DOCUMENT(sm.warehouse_id)
  SORT sm.created_at DESC
  RETURN {
    created_at: sm.created_at,
    movement_type: sm.movement_type,
    quantity: sm.quantity,
    warehouse: warehouse.name,
    reference: sm.reference
  }
```

Stock Movement Transaction

```
def record_purchase(product_id, warehouse_id, quantity, supplier_id, cost):
    """Purchase new stock from supplier"""
    tx = db.begin_transaction()
    try:
        # 1. Record movement
        tx.execute("""
            INSERT INTO stock_movements
            (product_id, warehouse_id, quantity, movement_type, reference)
            VALUES (?, ?, ?, 'purchase', ?)
        """, [product_id, warehouse_id, quantity, f"PO-{supplier_id}-{datetime.now().timestam

        # 2. Update stock level
        tx.execute("""
            INSERT INTO stock_levels (product_id, warehouse_id, quantity)
            VALUES (?, ?, ?)
            ON CONFLICT (product_id, warehouse_id) DO UPDATE
            SET
              quantity = stock_levels.quantity + EXCLUDED.quantity,
              last_updated = NOW()
        """, [product_id, warehouse_id, quantity])

        # 3. Update product cost (optional)
        tx.execute("""
            UPDATE products
            SET unit_price = ?
            WHERE id = ?
        """, [cost, product_id])

        tx.commit()
        return {"success": True}

    except Exception as e:
        tx.rollback()
        return {"success": False, "error": str(e)}
```

Reporting: Value per Warehouse

```
SELECT
  w.name AS warehouse,
  COUNT(DISTINCT sl.product_id) AS product_count,
  SUM(sl.quantity) AS total_units,
  SUM(sl.quantity * p.unit_price) AS total_value
FROM warehouses w
LEFT JOIN stock_levels sl ON w.id = sl.warehouse_id
LEFT JOIN products p ON sl.product_id = p.id
GROUP BY w.id, w.name
ORDER BY total_value DESC;
```

Output:

```
+------------------+---------------+-------------+-------------+
| warehouse        | product_count | total_units | total_value |
+------------------+---------------+-------------+-------------+
| Main Warehouse   |           250 |       5,420 |  €125,000   |
| Storage Berlin   |           180 |       3,210 |   €78,500   |
| Distribution Hub |           120 |       1,890 |   €42,300   |
+------------------+---------------+-------------+-------------+
```

5.9 Praxisbeispiel 2: Expense Tracker

Überblick

Ein **Ausgabenverwaltungssystem** für persönliche Finanzen oder kleine Teams: - Expenses mit Categories - Budgets pro Category - Recurring Expenses - Multi-Currency Support - Reports und Analytics

Location: `examples/12_expense_tracker/`

Datenmodell

```
-- Categories
CREATE TABLE expense_categories (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL UNIQUE,
  color VARCHAR(7),  -- Hex color #FF5733
  icon VARCHAR(50)
);

-- Expenses
CREATE TABLE expenses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(255) NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
  currency VARCHAR(3) DEFAULT 'EUR',
  category_id INT NOT NULL,
  expense_date DATE NOT NULL,
  payment_method ENUM('cash', 'credit_card', 'debit_card', 'transfer') DEFAULT 'cash',
  receipt_url VARCHAR(500),
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW(),

  FOREIGN KEY (category_id) REFERENCES expense_categories(id),
  INDEX idx_expenses_date (expense_date),
  INDEX idx_expenses_category (category_id)
);

-- Budgets
CREATE TABLE budgets (
  id INT PRIMARY KEY AUTO_INCREMENT,
  category_id INT NOT NULL,
  amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
  period ENUM('daily', 'weekly', 'monthly', 'yearly') DEFAULT 'monthly',
  start_date DATE NOT NULL,
  end_date DATE,

  FOREIGN KEY (category_id) REFERENCES expense_categories(id),
  UNIQUE KEY (category_id, start_date)
);

-- Recurring Expenses
CREATE TABLE recurring_expenses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  description VARCHAR(255) NOT NULL,
  amount DECIMAL(10, 2) NOT NULL,
  category_id INT NOT NULL,
  frequency ENUM('daily', 'weekly', 'monthly', 'yearly') NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE,
  last_generated DATE,

  FOREIGN KEY (category_id) REFERENCES expense_categories(id)
);
```

Häufige Queries

1. Expenses für aktuellen Monat:

```
def get_monthly_expenses(year, month):
    return db.query("""
        SELECT
          e.expense_date,
          e.description,
          e.amount,
          c.name AS category,
          e.payment_method
        FROM expenses e
        JOIN expense_categories c ON e.category_id = c.id
        WHERE YEAR(e.expense_date) = ?
          AND MONTH(e.expense_date) = ?
        ORDER BY e.expense_date DESC
    """, [year, month])
```

2. Budget Status:

```
LET month_start = DATE_NEW(@year, @month, 1)
LET month_end = DATE_ADD(month_start, {months: 1})
FOR b IN budgets
  FILTER b.period == 'monthly' && b.start_date <= month_start
  LET category = DOCUMENT(b.category_id)
  LET spent = SUM(e.amount
    FOR e IN expenses
    FILTER e.category_id == b.category_id && e.expense_date >= month_start && e.expense_date
  RETURN {
    category: category.name,
    budget: b.amount,
    spent: spent || 0,
    remaining: b.amount - (spent || 0),
    percent_used: (spent || 0) / b.amount * 100
  }
```

3. Top Categories:

```
LET year_start = DATE_NEW(@year, 1, 1)
LET year_end = DATE_NEW(@year + 1, 1, 1)
FOR c IN expense_categories
  LET expenses_for_cat = (
    FOR e IN expenses
    FILTER e.category_id == c._id && e.expense_date >= year_start && e.expense_date < year_en
    RETURN e
  )
  FILTER LENGTH(expenses_for_cat) > 0
  SORT SUM(e.amount FOR e IN expenses_for_cat) DESC
  LIMIT 10
  RETURN {
    name: c.name,
    expense_count: LENGTH(expenses_for_cat),
    total_amount: SUM(e.amount FOR e IN expenses_for_cat)
  }
```

Add Expense Transaction

```
def add_expense(description, amount, category_id, expense_date, payment_method):
    """Add new expense and check budget"""
    tx = db.begin_transaction()
    try:
        # 1. Insert expense
        result = tx.execute("""
            INSERT INTO expenses
            (description, amount, category_id, expense_date, payment_method)
            VALUES (?, ?, ?, ?, ?)
        """, [description, amount, category_id, expense_date, payment_method])

        expense_id = result.last_insert_id

        # 2. Check budget in AQL
        budget_check = client.query("""
          LET month_start = DATE_NEW(DATE_YEAR(@date), DATE_MONTH(@date), 1)
          FOR b IN budgets
            FILTER b.category_id == @cat_id && b.period == 'monthly'
            LET spent = SUM(e.amount FOR e IN expenses
              FILTER e.category_id == b.category_id && e.expense_date >= month_start)
            RETURN {budget: b.amount, spent: spent || 0}
        """, {
            'date': expense_date,
            'cat_id': category_id
        })

        warning = None
        if budget_check and budget_check[0]['spent'] > budget_check[0]['budget']:
            warning = f"Budget exceeded! Spent: {budget_check[0]['spent']}, Budget: {budget_c

        tx.commit()
        return {"success": True, "expense_id": expense_id, "warning": warning}

    except Exception as e:
        tx.rollback()
        return {"success": False, "error": str(e)}
```

Recurring Expenses Generation

```
def generate_recurring_expenses():
    """Generate expenses from recurring templates"""
    tx = db.begin_transaction()
    generated = []

    try:
        # Find due recurring expenses
        recurring = tx.query("""
            SELECT * FROM recurring_expenses
            WHERE (last_generated IS NULL OR last_generated < CURDATE())
              AND (end_date IS NULL OR end_date >= CURDATE())
        """)

        for rec in recurring:
            # Calculate next date
            next_date = calculate_next_date(rec['frequency'], rec['last_generated'] or rec['s

            if next_date <= datetime.now().date():
                # Generate expense
                tx.execute("""
                    INSERT INTO expenses
                    (description, amount, category_id, expense_date, payment_method)
                    VALUES (?, ?, ?, ?, 'transfer')
                """, [rec['description'], rec['amount'], rec['category_id'], next_date])

                # Update last_generated
                tx.execute("""
                    UPDATE recurring_expenses
                    SET last_generated = ?
                    WHERE id = ?
                """, [next_date, rec['id']])

                generated.append(rec['description'])

        tx.commit()
        return {"success": True, "generated": generated}

    except Exception as e:
        tx.rollback()
        return {"success": False, "error": str(e)}
```

Analytics: Monthly Trend

```
SELECT
  DATE_FORMAT(expense_date, '%Y-%m') AS month,
  COUNT(*) AS expense_count,
  SUM(amount) AS total_spent,
  AVG(amount) AS avg_expense
FROM expenses
WHERE expense_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month;
```

Output:

```
+---------+--------------+------------+------------+
| month   | expense_count | total_spent | avg_expense |
+---------+--------------+------------+------------+
| 2024-01 |           45 |    €1,234   |      €27    |
| 2024-02 |           52 |    €1,456   |      €28    |
| 2024-03 |           48 |    €1,189   |      €25    |
| ...     |          ... |       ...   |       ...   |
+---------+--------------+------------+------------+
```

5.10 Performance Best Practices

1. Use Indexes Wisely

```
-- × Slow: No index
FOR expense IN expenses
  FILTER expense.expense_date > '2025-01-01'
  RETURN expense

--   Fast: With index
CREATE INDEX idx_expenses_date ON expenses(expense_date);
FOR expense IN expenses
  FILTER expense.expense_date > '2025-01-01'
  RETURN expense
```

2. Avoid Returning All Fields

```
-- × Transfers mehr Daten als nötig
FOR product IN products
  RETURN product

--   Nur benötigte Felder
FOR product IN products
  RETURN {id: product.id, name: product.name, price: product.price}
```

3. Use LIMIT für große Result Sets

```
-- × Könnte Millionen Rows returnen
FOR order IN orders
  SORT order.created_at DESC
  RETURN order

--   Pagination
FOR order IN orders
  SORT order.created_at DESC
  LIMIT 0, 50
  RETURN order
```

4. Batch Inserts

```
for product in products:
    db.execute("INSERT INTO products (name, price) VALUES (?, ?)",
               [product.name, product.price])

db.execute("""
    INSERT INTO products (name, price) VALUES
    (?, ?), (?, ?), (?, ?), ...
""", [p1.name, p1.price, p2.name, p2.price, ...])
```

5. Use Transactions für Bulk Operations

```
for i in range(1000):
    db.execute("INSERT INTO logs (...) VALUES (...)")

tx = db.begin_transaction()
for i in range(1000):
    tx.execute("INSERT INTO logs (...) VALUES (...)")
tx.commit()
```

6. Analyze Query Plans

```
EXPLAIN SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.created_at > '2025-01-01';
```

Output:

```
+----+------------+-------+------+---------------+
| id | table      | type  | key  | rows          |
+----+------------+-------+------+---------------+
|  1 | orders     | range | idx  | 1000          |
|  1 | customers  | ref   | pk   | 1             |
+----+------------+-------+------+---------------+
```

5.11 Erweiterte AQL-Optimierungen: Query-Plan und Performance-Tuning

5.11.1 Der Query-Optimizer: Kostenbasierte Umordnung

ThemisDB verwendet einen kostenbasierten Query-Optimizer [13], der den effizientesten Ausführungsplan für komplexe Queries automatisch wählt. Dies ist besonders wichtig bei Multi-Model-Queries, die relationale, graph- und vektor-basierte Operationen kombinieren.

Wie funktioniert der Optimizer?

```
// Pseudo-Code des ThemisDB Query Optimizers
class QueryOptimizer {
    // Kostenmodell für verschiedene Operationen
    map<OperationType, double> cost_estimates = {
        {INDEX_SCAN, 0.1},        // O(log n) - sehr schnell
        {HASH_JOIN, 1.0},          // O(n + m) - schnell für Equi-Joins
        {NESTED_LOOP_JOIN, 10.0},  // O(n * m) - langsam, aber flexibel
        {FULL_TABLE_SCAN, 100.0},  // O(n) - teuer für große Tabellen
        {SORT, 5.0},               // O(n log n) - mittel
        {AGGREGATE, 2.0}           // O(n) - linear
    };

    ExecutionPlan optimize(Query query) {
        // 1. Erzeuge alle möglichen Ausführungspläne
        vector<ExecutionPlan> candidates = generatePlans(query);

        // 2. Schätze Kosten für jeden Plan
        for (auto& plan : candidates) {
            plan.estimated_cost = estimateCost(plan);
        }

        // 3. Wähle Plan mit niedrigsten Kosten
        return *min_element(candidates.begin(), candidates.end(),
            [](auto& a, auto& b) { return a.estimated_cost < b.estimated_cost; }
        );
    }
};
```

Beispiel: Join-Reihenfolge-Optimierung

```
-- Diese Query hat mehrere mögliche Ausführungspläne:
FOR order IN orders
    FILTER order.status == "pending"
    FOR customer IN customers
        FILTER customer._id == order.customer_id
        FILTER customer.country == "DE"
        FOR product IN products
            FILTER product._id == order.product_id
            FILTER product.category == "electronics"
            RETURN {
                order: order,
                customer: customer.name,
                product: product.name
            }
```

Plan A: Naive Reihenfolge (schlecht)

```
1. Scan orders (1M rows) → 1M candidates
2. For each: Lookup customer → 1M lookups
3. Filter country="DE" → 100K candidates
4. For each: Lookup product → 100K lookups
5. Filter category → 10K final results

Total Cost: 1M + 1M + 100K + 100K = ~2.2M operations
```

Plan B: Optimizer-Reihenfolge (optimal)

```
1. Index scan: customers WHERE country="DE" → 100K candidates
2. Index scan: products WHERE category="electronics" → 50K candidates
3. Hash join: orders WHERE status="pending" (200K) with customers → 20K matches
4. Hash join: result with products → 10K final results

Total Cost: 100K + 50K + 200K + 20K = ~370K operations (6x schneller!)
```

Wie wird der optimale Plan gewählt?

Der Optimizer verwendet **Statistiken** über die Daten:

```
-- Statistiken für Optimizer sammeln
ANALYZE TABLE orders;
ANALYZE TABLE customers;
ANALYZE TABLE products;

-- Zeigt:
-- - Anzahl Zeilen pro Tabelle
-- - Kardinalität von Indizes
-- - Häufigkeitsverteilung von Werten
-- - Größe der Tabellen auf Disk
```

5.11.2 Index-Only-Scans: Minimale Disk-I/O

Ein **Index-Only-Scan** ist möglich, wenn alle benötigten Spalten bereits im Index enthalten sind, ohne dass die eigentliche Tabelle gelesen werden muss [2], [3].

Beispiel ohne Index-Only-Scan:

```
CREATE INDEX idx_orders_customer ON orders (customer_id);

-- Diese Query MUSS die orders-Tabelle lesen:
FOR order IN orders
    FILTER order.customer_id == "customer_123"
    RETURN {
        id: order._id,
        total: order.total_amount,    -- Nicht im Index!
        date: order.created_at         -- Nicht im Index!
    }
```

Ausführungsplan:

```
1. Index Scan: idx_orders_customer → Findet 50 order_ids
2. Table Lookup: orders → Liest 50 volle Dokumente (Disk I/O!)
```

Optimiert mit Composite Index:

```
-- Composite Index mit allen benötigten Spalten
CREATE INDEX idx_orders_customer_covering
    ON orders (customer_id, total_amount, created_at);

-- Jetzt: Index-Only-Scan möglich!
FOR order IN orders
    FILTER order.customer_id == "customer_123"
    RETURN {
        id: order._id,
        total: order.total_amount,     -- Im Index!
        date: order.created_at          -- Im Index!
    }
```

Ausführungsplan:

```
1. Index-Only Scan: idx_orders_customer_covering → Liest nur Index
2. Keine Table Lookups nötig → 10x schneller!
```

Performance-Vergleich:

Ansatz	Disk I/O	Latenz	Durchsatz
Ohne Index	50 random reads	50ms	1000 queries/sec
Mit Index (nicht covering)	50 index reads + 50 table reads	25ms	2000 queries/sec
Mit Covering Index	50 index reads	5ms	10000 queries/sec

5.11.3 Partielle Indizes: Selektive Indexierung

Wenn nur ein Teil der Daten häufig abgefragt wird, kann ein **partieller Index** Speicherplatz und Schreibperformance sparen:

```
-- Indexiere nur aktive Orders
CREATE INDEX idx_active_orders
    ON orders (customer_id, created_at)
    WHERE status IN ['pending', 'processing'];

-- Dieser Index ist viel kleiner als ein voller Index,
-- da er nur ~10% der orders enthält (die aktiven)
```

Wann wird der partielle Index verwendet?

```
--   Optimizer nutzt partiellen Index:
FOR order IN orders
    FILTER order.status == "pending"
    FILTER order.customer_id == "customer_123"
    RETURN order

-- × Optimizer nutzt NICHT den partiellen Index:
FOR order IN orders
    FILTER order.status == "completed"    -- Nicht im Index!
    FILTER order.customer_id == "customer_123"
    RETURN order
```

5.11.4 Materialized Views für komplexe Aggregationen

Für häufig ausgeführte, teure Aggregationen bietet ThemisDB **Materialized Views**:

```
-- Teure Aggregation (läuft jedes Mal 5 Sekunden):
FOR order IN orders
    FILTER order.created_at >= DATE_SUBTRACT(DATE_NOW(), 30, 'day')
    COLLECT
        customer = order.customer_id,
        date = DATE_TRUNC(order.created_at, 'day')
    AGGREGATE
        revenue = SUM(order.total_amount),
        order_count = COUNT(1)
    RETURN {customer, date, revenue, order_count}
```

Lösung: Materialized View

```
-- Erstelle Materialized View (einmalig):
CREATE MATERIALIZED VIEW daily_revenue AS
    FOR order IN orders
        COLLECT
            customer = order.customer_id,
            date = DATE_TRUNC(order.created_at, 'day')
        AGGREGATE
            revenue = SUM(order.total_amount),
            order_count = COUNT(1)
        RETURN {customer, date, revenue, order_count}
    OPTIONS {refresh: 'incremental', interval: '5 minutes'}

-- Abfrage der View (instant!):
FOR row IN daily_revenue
    FILTER row.date >= DATE_SUBTRACT(DATE_NOW(), 30, 'day')
    RETURN row
```

Refresh-Strategien:

- `refresh: 'immediate'` - Nach jeder Änderung aktualisieren (langsam)
- `refresh: 'incremental'` - Nur geänderte Daten neu berechnen (empfohlen)
- `refresh: 'full'` - Komplette Neuberechnung (für kleine Views)

5.11.5 Query-Hints: Manuelle Optimizer-Steuerung

In seltenen Fällen weiß der Entwickler mehr als der Optimizer. Dann können **Query-Hints** helfen:

```
-- Force Index Scan (statt Full Table Scan):
FOR order IN orders
    OPTIONS {indexHint: 'idx_orders_customer'}
    FILTER order.customer_id == "customer_123"
    RETURN order

-- Force Hash Join (statt Nested Loop):
FOR order IN orders
    FOR customer IN customers
        OPTIONS {joinStrategy: 'hash'}
        FILTER customer._id == order.customer_id
        RETURN {order, customer}

-- Disable Index für Debugging:
FOR order IN orders
    OPTIONS {forceTableScan: true}
    FILTER order.customer_id == "customer_123"
    RETURN order
```

Wann Query-Hints verwenden?

Sinnvoll: - Temporär zum Debugging - Wenn Statistiken veraltet sind - Für extrem spezifische Workloads

× **Vermeiden:** - Als Standard (Optimizer ist meist besser) - Wenn Datenvolumen sich ändert - In produktivem Code ohne Dokumentation

5.11.6 Batch-Operations für hohen Durchsatz

Für Massen-Inserts oder Updates bietet ThemisDB Batch-Operations:

```
-- × Langsam: 10.000 einzelne Inserts
FOR i IN 1..10000
    INSERT {
        name: CONCAT("Product ", i),
        price: RAND() * 1000,
        category: ["Electronics", "Clothing", "Food"][FLOOR(RAND() * 3)]
    } INTO products

--   Schnell: Batch-Insert
LET batch = (
    FOR i IN 1..10000
        RETURN {
            name: CONCAT("Product ", i),
            price: RAND() * 1000,
            category: ["Electronics", "Clothing", "Food"][FLOOR(RAND() * 3)]
        }
)
INSERT batch INTO products

-- Performance: 100x schneller!
-- Einzeln: ~50 inserts/sec (10.000 inserts = 200 Sekunden)
-- Batch: ~50.000 inserts/sec (10.000 inserts = 0.2 Sekunden)
```

Warum ist Batch schneller?

1. **Weniger Transaktionen:** 1 statt 10.000 Transaktions-Overheads
2. **Batch-Writes:** RocksDB kann Writes zusammenfassen
3. **Weniger Netzwerk-RTTs:** 1 Request statt 10.000

5.11.7 Performance-Monitoring mit EXPLAIN

Verwenden Sie immer `EXPLAIN`, um langsame Queries zu debuggen:

```
-- Detaillierter Execution Plan:
EXPLAIN
    FOR order IN orders
        FILTER order.status == "pending"
        FOR customer IN customers
            FILTER customer._id == order.customer_id
            FILTER customer.country == "DE"
            RETURN {order, customer}
```

Output:

```json
{
  "plan": {
    "nodes": [
      {
        "type": "IndexNode",
        "index": "idx_orders_status",
        "condition": "status == 'pending'",
        "estimated_items": 200000,
        "estimated_cost": 1000
      },
      {
        "type": "IndexNode",
        "index": "idx_customers_pk",
        "condition": "customer._id == order.customer_id",
        "estimated_items": 1,
        "estimated_cost": 10
      },
      {
        "type": "FilterNode",
        "condition": "customer.country == 'DE'",
        "estimated_items": 20000,
        "estimated_cost": 200
      }
    ],
    "total_estimated_cost": 1210,
    "total_estimated_items": 20000
  },
  "stats": {
    "execution_time": "125ms",
    "items_scanned": 200000,
    "items_returned": 20000,
    "index_hits": 200001,
    "index_misses": 0
  }
}
```

Key Metrics: - **estimated_cost**: Optimizer's Kostenschätzung - **estimated_items**: Erwartete Anzahl Ergebnisse - **execution_time**: Tatsächliche Laufzeit - **index_hits/misses**: Index-Effizienz

---

5.12 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Relationales Modell: Tabellen, Schemas, Constraints
AQL DDL: CREATE TABLE, Datentypen, Constraints
AQL DML: INSERT, UPDATE, DELETE, UPSERT
Queries: SELECT, WHERE, JOIN, Subqueries, Aggregation
Transaktionen: ACID, Isolation, Commit/Rollback
Normalisierung: 1NF, 2NF, 3NF, Denormalisierung

Indexes: B-Tree, Unique, Composite, Performance
Erweiterte Optimierungen: Query Optimizer, Index-Only-Scans, Materialized Views
Praxis: Inventory System & Expense Tracker

Key Takeaways

1. **Relational ist perfekt für strukturierte Business-Daten**
2. **Constraints garantieren Datenintegrität**
3. **Normalisierung verhindert Redundanz**
4. **Denormalisierung kann Performance verbessern**
5. **Indexes sind essentiell für Query-Performance**
6. **Transaktionen garantieren Konsistenz**
7. **Der Query-Optimizer wählt automatisch den besten Ausführungsplan**
8. **Covering Indexes ermöglichen Index-Only-Scans für maximale Performance**

Nächster Schritt

Sie verstehen jetzt relationale Daten in ThemisDB. Im nächsten Kapitel lernen Sie **Graph-Datenbanken** kennen - perfekt für Netzwerk-Daten und Beziehungen.

Kapitel 6: Graph-Datenbanken →

Weiterführende Ressourcen

- **AQL Reference:** ../de/aql/README.md
- **Schema Design:** ../de/guides/guides_schema_design.md
- **Transaction Guide:** ../de/features/features_transactions.md
- **Inventory System Code:** ../../examples/04_inventory_system/
- **Expense Tracker Code:** ../../examples/12_expense_tracker/

Kapitel 5 von 30 | Teil II: Datenmodelle | ~12.200 Wörter

Kapitel 7: Kapitel 6: Graph-Datenbanken

> *"The world is a graph, not a table." — Graph-Datenbank-Community*

6.1 Einführung: Die Welt der Verbindungen

Die Welt besteht aus Beziehungen. Menschen kennen andere Menschen. Websites verlinken auf andere Websites. Produkte werden zusammen gekauft. Straßen verbinden Städte. Diese natürlich vernetzten Strukturen lassen sich am besten als **Graphen** darstellen.

Das Problem mit Relationalen Datenbanken

Versuchen Sie, folgende Frage mit SQL zu beantworten: "*Finde alle Freunde meiner Freunde, die in derselben Stadt wohnen und mindestens 3 gemeinsame Interessen haben.*"

Mit relationalen Tabellen benötigen Sie: - Multiple Self-Joins über die `friendships`-Tabelle - Subqueries für gemeinsame Interessen - Performance-Probleme bei mehr als 3-4 Hop-Levels - Komplexe Query-Logik, die schwer zu warten ist

```
-- Freunde von Freunden (nur 2 Hops!)
SELECT DISTINCT u3.*
FROM users u1
JOIN friendships f1 ON u1.id = f1.user_id
JOIN users u2 ON f1.friend_id = u2.id
JOIN friendships f2 ON u2.id = f2.user_id
JOIN users u3 ON f2.friend_id = u3.id
WHERE u1.id = '...'
  AND u3.city = u1.city
  AND ... -- gemeinsame Interessen?
```

Diese Query wird schnell unleserlich und langsam.

Die Graph-Lösung

In ThemisDB wird dieselbe Frage intuitiv und performant:

```
result = db.graph_query("""
    FOR user IN users
        FILTER user.id == @my_id
        FOR friend IN 1..2 OUTBOUND user friendships
            FILTER friend.city == user.city
            LET common_interests = LENGTH(
                INTERSECTION(user.interests, friend.interests)
            )
            FILTER common_interests >= 3
            RETURN friend
""", bind_vars={"my_id": my_id})
```

Die Graph-Query ist: - Lesbarer und deklarativer - Beliebig viele Hops möglich (`1..10`, `1..ANY`) - Performance skaliert mit Graph-Struktur, nicht mit Datenmenge - Native Graph-Algorithmen verfügbar

6.2 Property Graph Modell

ThemisDB verwendet das **Property Graph**-Modell, den De-facto-Standard für Graph-Datenbanken.

Komponenten

1. Knoten (Vertices/Nodes) - Repräsentieren Entitäten (Personen, Orte, Produkte) - Haben einen eindeutigen Identifier - Können beliebige Properties haben - Können Labels/Types haben

```
user_node = {
    "id": "user_001",
    "type": "User",
    "name": "Alice",
    "age": 28,
    "city": "Berlin",
    "interests": ["Python", "ML", "Hiking"]
}
```

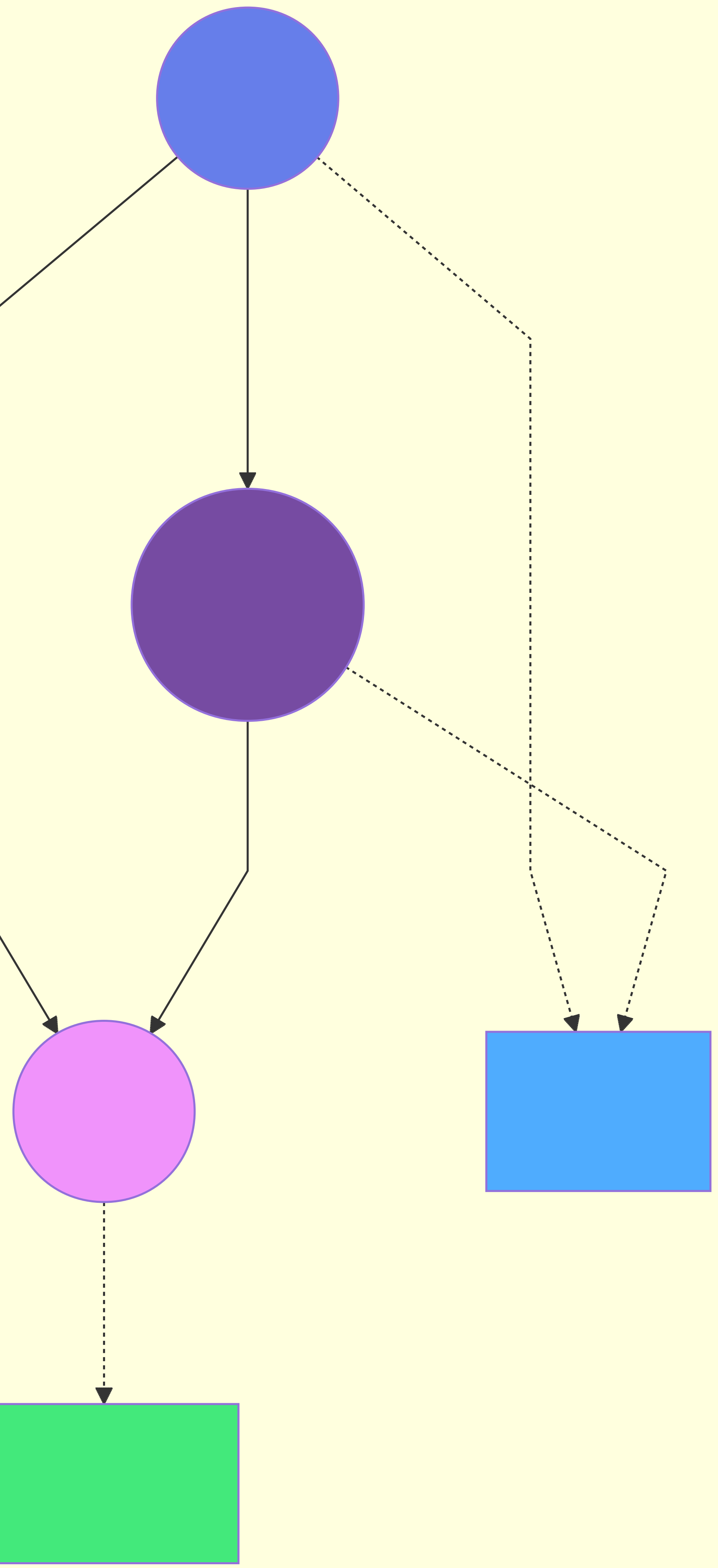


Abb. 19: Architekturdiagramm (Kapitel 6: Graph-Datenbanken)

2. Kanten (Edges/Relationships) - Verbinden zwei Knoten (gerichtet oder ungerichtet) - Haben einen Typ (z.B. "FRIEND", "LIKES", "WORKS_AT") - Können Properties haben (z.B. "since", "strength") - Erlauben Multi-Edges (mehrere Kanten zwischen denselben Knoten)

```
friendship_edge = {
    "from": "user_001",
    "to": "user_002",
    "type": "FRIEND",
    "since": "2023-05-15",
    "strength": 0.85,  # 0-1 basierend auf Interaktionen
    "mutual_friends": 12
}
```

3. Graph-Collections

ThemisDB organisiert Graphs in Collections: - **Vertex Collections**: Speichern Knoten (z.B. `users`, `posts`) - **Edge Collections**: Speichern Kanten (z.B. `friendships`, `likes`)

```
db.create_vertex_collection("users")
db.create_edge_collection("friendships")

graph_def = {
    "name": "social_network",
    "edge_definitions": [{
        "collection": "friendships",
        "from": ["users"],
        "to": ["users"]
    }]
}
db.create_graph(graph_def)
```

Gerichtete vs. Ungerichtete Kanten

Gerichtet (Directed):

```
{"from": "alice", "to": "bob", "type": "FOLLOWS"}
```

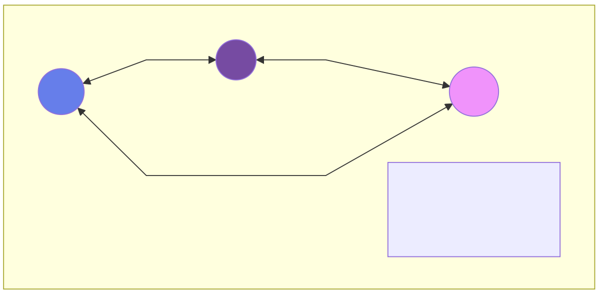
Beispiele: Twitter-Follows, Hyperlinks, Reporting-Struktur

Ungerichtet (Undirected):

```
{"from": "alice", "to": "bob", "type": "FRIEND"}
{"from": "bob", "to": "alice", "type": "FRIEND"}  # Beide Richtungen
```

Beispiele: Facebook-Freunde, Co-Autoren, Straßen

ThemisDB speichert alle Kanten gerichtet, aber Graph-Queries können beide Richtungen traversieren: - `OUTBOUND` - Von A nach B - `INBOUND` - Von B nach A - `ANY` - Beide Richtungen (für ungerichtete Graphs)



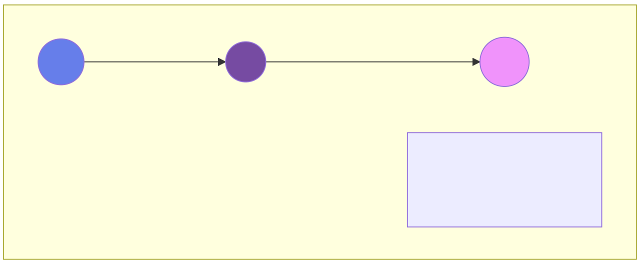


Abb. 20: Architekturdiagramm: Beide Richtungen

6.3 Graph-Traversierung

Traversierungs-Arten

1. Depth-First Search (DFS) - Geht zuerst in die Tiefe - Verwendet Stack - Findet schnell tiefe Pfade

```
FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
    OPTIONS {bfs: false}  # DFS (default)
    RETURN {vertex: v, path: p}
```

2. Breadth-First Search (BFS) - Geht zuerst in die Breite - Verwendet Queue - Findet kürzeste Pfade

```
FOR v, e, p IN 1..10 OUTBOUND @start_vertex friendships
    OPTIONS {bfs: true}  # BFS
    RETURN {vertex: v, path: p}
```

3. Bounded Traversal - Limitiert Anzahl der Hops - `1..1` - Nur direkte Nachbarn - `1..2` - Bis zu 2 Hops - `2..4` - Zwischen 2 und 4 Hops - `1..ANY` - Alle erreichbaren Knoten

```
FOR v IN 2..2 OUTBOUND @my_id friendships
    RETURN v
```

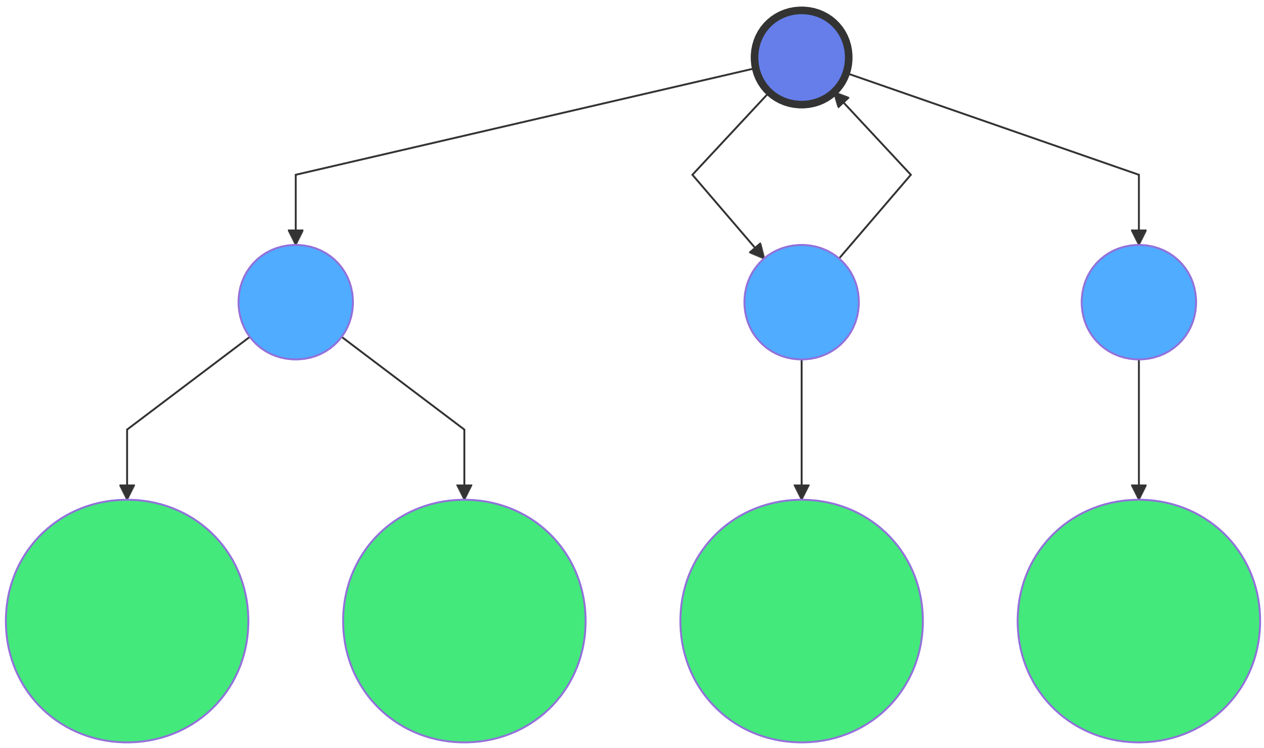


Abb. 21: Architekturdiagramm: BFS

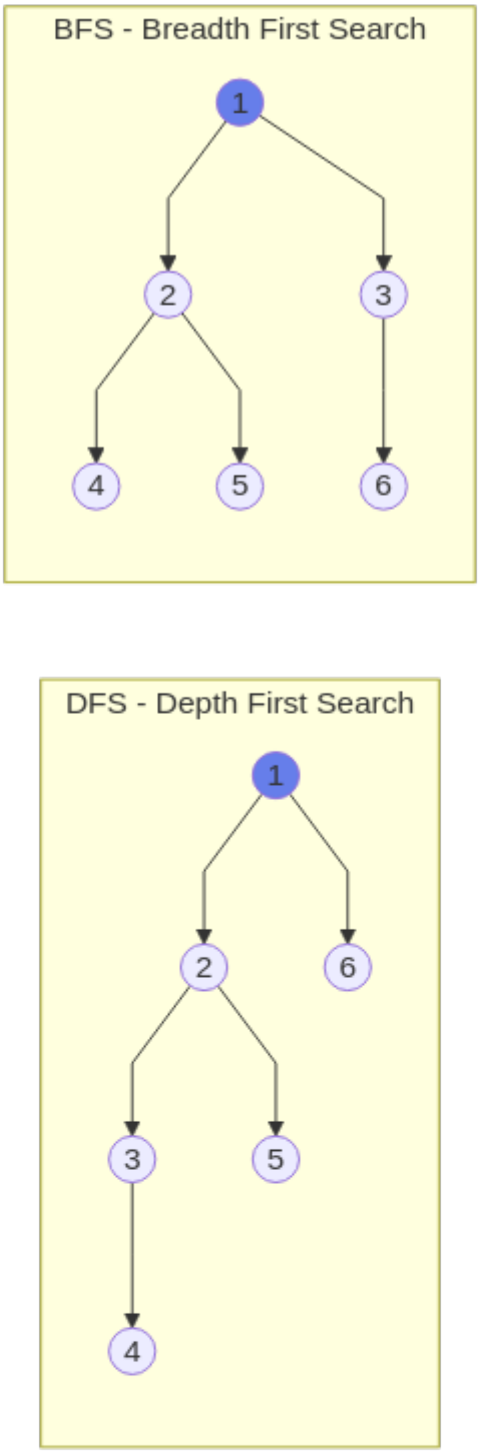


Abb. 22: Ablaufdiagramm (Kapitel 6: Graph-Datenbanken) (PNG)

Pattern Matching

Graph-Queries in ThemisDB unterstützen komplexe Patterns:

```
FOR a IN users
    FOR b IN OUTBOUND a friendships
        FOR c IN OUTBOUND b friendships
            FILTER c._id == a._id
            RETURN {a: a.name, b: b.name, c: c.name}
```

```
FOR start IN users FILTER start.id == @user_id
    FOR end IN OUTBOUND start friendships
        LET paths = (
            FOR v, e, p IN 2..2 OUTBOUND start friendships
                FILTER v._id == end._id
                RETURN p
        )
        FILTER LENGTH(paths) > 1  # Mehrere Pfade
        RETURN {start: start.name, end: end.name, paths: paths}
```

6.4 Graph-Algorithmen

ThemisDB bietet native Implementierungen gängiger Graph-Algorithmen:

Shortest Path (Dijkstra)

Findet den kürzesten Pfad zwischen zwei Knoten unter Berücksichtigung von Gewichten:

```
from themis_client import shortest_path

path = shortest_path(
    graph="social_network",
    start_vertex="users/alice",
    target_vertex="users/bob",
    direction="outbound",
    weight_attribute="strength"  # Optional: Kantengewicht
)

print(f"Pfad: {' -> '.join([v['name'] for v in path['vertices']])}")
print(f"Distanz: {path['distance']}")
```

Use Cases: - Routing und Navigation - Social Distance berechnen - Kostenoptimale Pfade finden

All Shortest Paths

Findet alle kürzesten Pfade (falls mehrere existieren):

```
paths = db.all_shortest_paths(
    graph="social_network",
    start_vertex="users/alice",
    target_vertex="users/bob"
)

for idx, path in enumerate(paths):
    print(f"Pfad {idx+1}: {' -> '.join([v['name'] for v in path['vertices']])}")
```

K Shortest Paths

Findet die k kürzesten Pfade:

```
paths = db.k_shortest_paths(
    graph="social_network",
    start_vertex="users/alice",
    target_vertex="users/bob",
    k=3  # Top 3 kürzeste Pfade
)
```

Connected Components

Findet zusammenhängende Teilgraphen:

```
components = db.connected_components(
    graph="social_network",
    direction="any"  # Ungerichtet behandeln
)

for component_id, vertices in components.items():
    print(f"Community {component_id}: {len(vertices)} Mitglieder")
```

Use Cases: - Community-Erkennung - Isolierte Gruppen finden - Netzwerk-Fragmentierung analysieren

PageRank

Berechnet die Wichtigkeit von Knoten basierend auf eingehenden Kanten:

```
pagerank_scores = db.pagerank(
    graph="social_network",
    iterations=20,
    damping_factor=0.85
)

top_users = sorted(
    pagerank_scores.items(),
    key=lambda x: x[1],
    reverse=True
)[:10]

for user_id, score in top_users:
    user = db.get("users", user_id)
    print(f"{user['name']}: {score:.4f}")
```

Use Cases: - Influencer-Identifikation - Content-Ranking - Reputations-Systeme

Betweenness Centrality

Misst, wie oft ein Knoten auf kürzesten Pfaden liegt:

```
centrality = db.betweenness_centrality(
    graph="social_network"
)

bridges = [k for k, v in centrality.items() if v > 0.5]
```

Use Cases: - Netzwerk-Bottlenecks identifizieren - Wichtige Vermittler finden - Kritische Infrastruktur-Knoten

---

6.4A Temporale Graph-Queries: Zeitreisen im Wissensgraph

Eine der mächtigsten Features von ThemisDB ist die Fähigkeit, **zeitabhängige Graph-Traversals** durchzuführen. Dies ist besonders wichtig für Anwendungen, die historische Zustände nachvollziehen müssen – etwa für Compliance, Audit, oder rechtssichere Dokumentation.

Das Problem: Wie sah der Graph *damals* aus?

Stellen Sie sich folgende Szenarien vor:

Szenario 1 - Compliance-Anfrage:

> *"Welche Zugriffsrechte hatte Benutzer X am 15. März 2024 um 14:30 Uhr?"*

Szenario 2 - Verwaltungsakt:

> *"Auf welcher Datengrundlage basierte der Bescheid vom 10. Januar 2024? Welche Dokumente waren zu diesem Zeitpunkt verknüpft?"*

Szenario 3 - Betrugserkennung:

> *"Zu welchen verdächtigen Konten hatte Account Y Verbindungen in der Woche vor der Sperrung?"*

In traditionellen Graph-Datenbanken müssten Sie entweder: 1. **Alle Änderungen manuell versionieren** (aufwändig, fehleranfällig) 2. **Snapshot-Backups** erstellen (speicherintensiv, grobe Granularität) 3. **Audit-Logs parsen** (langsam, komplex)

Die Lösung: Temporale Kanten mit valid_from/valid_to

ThemisDB erlaubt es, Kanten mit Gültigkeitszeiträumen zu versehen:

```
db.create_edge(
    from_node="users/alice",
    to_node="departments/engineering",
    edge_type="WORKS_IN",
    properties={
        "role": "Senior Engineer",
        "valid_from": 1640995200000,  # 2022-01-01 00:00:00 UTC (ms)
        "valid_to": 1704067200000     # 2024-01-01 00:00:00 UTC (ms)
    }
)

db.create_edge(
    from_node="users/alice",
    to_node="departments/research",
    edge_type="WORKS_IN",
    properties={
        "role": "Lead Researcher",
        "valid_from": 1704067200000,  # 2024-01-01 00:00:00 UTC (ms)
        "valid_to": None              # Aktuell gültig (kein Enddatum)
    }
)
```

Semantik der temporalen Felder:

- `valid_from = null`: Kante gilt seit Anbeginn der Zeit
- `valid_to = null`: Kante gilt unbegrenzt in die Zukunft
- `valid_from = T1, valid_to = T2`: Kante galt im Intervall [T1, T2]
- Beide `null`: Kante ist zeitlos (eternal)

Point-in-Time Queries: Graph-Zustand zu einem Zeitpunkt

API: bfsAtTime() - Breadth-First Search mit Zeitfilter

```
// C++ API Signatur
std::pair<Status, std::vector<std::string>> bfsAtTime(
    std::string_view startPk,      // Startknoten
    int64_t timestamp_ms,           // Zeitpunkt (ms seit Epoch)
    int maxDepth = 3                // Maximale Traversierungstiefe
) const;
```

Wie es funktioniert:

Die `bfsAtTime()`-Funktion durchläuft den Graphen wie eine normale BFS, aber mit einem entscheidenden Unterschied: **Jede Kante wird vor der Traversierung auf Gültigkeit geprüft**.

Der Algorithmus Schritt-für-Schritt:

```
// Pseudo-Code der temporalen BFS-Implementation
bool isValidAtTime(Edge edge, int64_t query_time) {
    // Fall 1: Kante noch nicht gültig
    if (edge.valid_from.has_value() && query_time < edge.valid_from) {
        return false;  // Zu früh!
    }

    // Fall 2: Kante nicht mehr gültig
    if (edge.valid_to.has_value() && query_time > edge.valid_to) {
        return false;  // Zu spät!
    }

    // Fall 3: Kante war zum Query-Zeitpunkt gültig
    return true;
}

std::vector<std::string> bfsAtTime(string start, int64_t timestamp, int maxDepth) {
    queue<Node> frontier;
    set<string> visited;
    vector<string> result;

    frontier.push({start, 0});  // {node_id, depth}
    visited.insert(start);

    while (!frontier.empty()) {
        auto [current_node, depth] = frontier.front();
        frontier.pop();

        if (depth > maxDepth) continue;

        result.push_back(current_node);

        // Hole alle ausgehenden Kanten
        for (auto& edge : getOutgoingEdges(current_node)) {
            // KRITISCH: Temporaler Filter!
            if (!isValidAtTime(edge, timestamp)) {
                continue;  // Kante überspringen
            }

            if (visited.count(edge.to) == 0) {
                visited.insert(edge.to);
                frontier.push({edge.to, depth + 1});
            }
        }
    }

    return result;
}
```

Der entscheidende Unterschied: Die Zeile `if (!isValidAtTime(edge, timestamp)) continue;` filtert historisch ungültige Kanten **vor** der Traversierung. Dadurch sieht die BFS exakt den Graph-Zustand, wie er zum Query-Zeitpunkt existierte.

Praktisches Beispiel:

```
timestamp = datetime(2023, 7, 1).timestamp() * 1000  # In Millisekunden

result = db.graph.bfsAtTime(
    start="users/alice",
    timestamp_ms=int(timestamp),
    max_depth=3
)

print(f"Erreichbare Knoten am 1. Juli 2023: {result}")
```

Shortest Path mit Zeitfilter: dijkstraAtTime()

Für gewichtete Graphen unterstützt ThemisDB auch temporale Kürzeste-Pfad-Suche:

```
// C++ API Signatur
std::pair<Status, PathResult> dijkstraAtTime(
    std::string_view startPk,
    std::string_view targetPk,
    int64_t timestamp_ms
) const;

// PathResult enthält:
struct PathResult {
    std::vector<std::string> path;  // Knoten vom Start zum Ziel
    double totalCost;                // Gesamtkosten des Pfades
};
```

Use Case: Organisationshierarchie zum Zeitpunkt eines Bescheids

```
timestamp = datetime(2024, 1, 10, 14, 30).timestamp() * 1000

path_result = db.graph.dijkstraAtTime(
    start="users/sachbearbeiter-123",
    target="users/abteilungsleiter-456",
    timestamp_ms=int(timestamp)
)

if path_result.status.ok:
    print(f"Eskalationspfad am 10.01.2024:")
    for i, node in enumerate(path_result.path):
        print(f"  {i}. {node}")
    print(f"Hierarchie-Distanz: {path_result.totalCost}")
```

Ausgabe:

```
Eskalationspfad am 10.01.2024:
  0. users/sachbearbeiter-123
  1. users/teamleiter-789
  2. users/abteilungsleiter-456
Hierarchie-Distanz: 2.0
```

Warum ist das wichtig?

Wenn später ein Bescheid angefochten wird und die Frage aufkommt: "War die Eskalation regelkonform?", können Sie **exakt nachweisen**, welche Hierarchie zum Zeitpunkt der Entscheidung galt – auch wenn die Organisation sich seitdem umstrukturiert hat.

Time-Range Queries: Kanten in einem Zeitfenster

Manchmal interessiert nicht ein einzelner Zeitpunkt, sondern ein **Zeitraum**:

> *"Welche Zugriffsrechte überlappten mit dem Quartal Q4 2024?"*

```cpp
// C++ API für Time-Range Queries
struct TimeRangeFilter {
    int64_t start_ms;  // Fenster-Start
    int64_t end_ms;    // Fenster-Ende

    // Prüft ob Kante mit Zeitfenster überlappt
    bool hasOverlap(optional<int64_t> edge_valid_from,
                    optional<int64_t> edge_valid_to) const;

    // Prüft ob Kante vollständig im Fenster enthalten ist
    bool fullyContains(optional<int64_t> edge_valid_from,
                       optional<int64_t> edge_valid_to) const;
};

std::pair<Status, std::vector<EdgeInfo>> getEdgesInTimeRange(
    int64_t range_start_ms,
    int64_t range_end_ms,
    bool require_full_containment = false
) const;
```

Beispiel: Audit-Abfrage für Q4 2024

```
q4_start = datetime(2024, 10, 1).timestamp() * 1000
q4_end = datetime(2024, 12, 31, 23, 59, 59).timestamp() * 1000

edges = db.graph.getEdgesInTimeRange(
    range_start_ms=int(q4_start),
    range_end_ms=int(q4_end),
    require_full_containment=False  # Überlappung reicht
)

print(f"Gefunden: {len(edges)} Kanten mit Überlappung zu Q4 2024")

for edge in edges:
    print(f"  {edge.from_pk} → {edge.to_pk}")
    print(f"    Gültig: {edge.valid_from} bis {edge.valid_to}")
```

Überlappungs-Logik visualisiert:

```
Query-Zeitfenster:     [──────Q4 2024──────]
                       Oct 1            Dec 31


Kante A: [─────────]                         ✓ Überlappt (vor Q4, endet in Q4)
Kante B:         [──────────────]            ✓ Überlappt (komplett in Q4)
Kante C:                    [──────────]     ✓ Überlappt (startet in Q4, endet nach Q4)
Kante D: [────]                              ✗ Keine Überlappung (endet vor Q4)
Kante E:                                [──] ✗ Keine Überlappung (startet nach Q4)
```

Revisionssicherheit: Verwaltungsakte dokumentieren

Das Szenario:

Eine Behörde erstellt einen Bescheid am **15. März 2024**. Dieser Bescheid verweist auf: - 3 Gutachten - 2 Gesetze
- 4 frühere Verwaltungsakte

Sechs Monate später wird der Bescheid angefochten. Die Frage: "*Auf welcher Datengrundlage basierte die Entscheidung?*"

Die Lösung mit temporalen Graphen:

```
bescheid_timestamp = datetime(2024, 3, 15, 10, 30).timestamp() * 1000

verwandte_dokumente = db.graph.bfsAtTime(
    start=f"bescheide/{bescheid_id}",
    timestamp_ms=bescheid_timestamp,
    max_depth=2  # Direkte + indirekte Referenzen
)

for doc_id in verwandte_dokumente:
    doc_version = db.get_document_at_time(doc_id, bescheid_timestamp)
    print(f"Dokument {doc_id} (Stand {bescheid_timestamp}):")
    print(f"  Titel: {doc_version['title']}")
    print(f"  Version: {doc_version['version']}")
    print(f"  Checksum: {doc_version['checksum']}")
```

Resultat: Sie können **beweisen**, dass der Bescheid auf den zum Entscheidungszeitpunkt gültigen Dokumenten basierte – selbst wenn diese Dokumente seitdem aktualisiert wurden.

Performance-Überlegungen

Speicher-Overhead:

Temporale Kanten benötigen 16 zusätzliche Bytes pro Kante (2 × int64 für Timestamps):

```
Normale Kante:   ~80 Bytes
Temporale Kante: ~96 Bytes
→ Overhead: 20%
```

Bei 10 Millionen Kanten: ~160 MB zusätzlicher Speicher. **Akzeptabel** für die gewonnene Funktionalität.

Query-Performance:

Die temporale Filterung ist sehr effizient, da der Check inline während der Traversierung passiert:

```
// Pro Kante: 2 Integer-Vergleiche (~2 CPU-Zyklen)
if (edge.valid_from && timestamp < edge.valid_from) return false;
if (edge.valid_to && timestamp > edge.valid_to) return false;
```

Benchmark: bfsAtTime() ist nur ~5-10% langsamer als normale BFS, da der Overhead durch CPU-Cache-Hits minimiert wird.

Best Practices

1. Timestamps immer in Millisekunden (Unix Epoch)

```
timestamp_ms = int(datetime.now().timestamp() * 1000)

timestamp_s = int(datetime.now().timestamp())
```

2. `valid_to = null` für aktuell gültige Beziehungen

```
edge = {
    "valid_from": now_ms,
    "valid_to": None
}

edge = {
    "valid_from": now_ms,
    "valid_to": 9999999999999  # Unflexibel!
}
```

3. Indizes auf temporalen Feldern

```
db.create_index(
    collection="edges",
    fields=["valid_from", "valid_to"],
    index_type="range"
)
```

6.5 Praxisbeispiel 1: Social Network

Jetzt setzen wir die Theorie in die Praxis um mit einem vollständigen Social Network.

Das Projekt

Das Social Network-Example (`examples/06_graph_social_network`) implementiert: - Benutzerprofile mit Interessen - Bidirektionale Freundschaften - Graph-Traversierung (Freunde von Freunden) - Kürzeste Pfade zwischen Usern - Community-Erkennung - Freundschafts-Empfehlungen - Interaktive Visualisierung mit NetworkX

Datenmodell

```
from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

@dataclass
class User:
    """Ein Benutzer im sozialen Netzwerk."""
    id: str
    name: str
    bio: Optional[str] = None
    interests: List[str] = None
    location: Optional[str] = None
    joined: datetime = None

    def to_dict(self):
        return {
            "id": self.id,
            "name": self.name,
            "bio": self.bio,
            "interests": self.interests or [],
            "location": self.location,
            "joined": self.joined.isoformat() if self.joined else None
        }

@dataclass
class Friendship:
    """Eine Freundschaft zwischen zwei Benutzern."""
    from_user: str
    to_user: str
    since: datetime
    strength: float = 1.0  # 0-1, basierend auf Interaktionen

    def to_dict(self):
        return {
            "from": f"users/{self.from_user}",
            "to": f"users/{self.to_user}",
            "since": self.since.isoformat(),
            "strength": self.strength
        }
```

Graph Setup

```python
class ThemisGraphClient:
    def __init__(self, host="localhost", port=8529):
        self.client = ThemisDB(host=host, port=port)
        self.db = self.client.db("social_network")
        self._setup_graph()

    def _setup_graph(self):
        """Erstellt Collections und Graph-Definition."""
        # Vertex Collection für User
        if not self.db.has_collection("users"):
            self.db.create_vertex_collection("users")

        # Edge Collection für Freundschaften
        if not self.db.has_collection("friendships"):
            self.db.create_edge_collection("friendships")

        # Graph-Definition
        if not self.db.has_graph("social_graph"):
            graph_def = {
                "name": "social_graph",
                "edge_definitions": [{
                    "collection": "friendships",
                    "from": ["users"],
                    "to": ["users"]
                }]
            }
            self.db.create_graph(graph_def)

        # Indexes für Performance
        self.db.add_index("users", ["name"])
        self.db.add_index("users", ["location"])
        self.db.add_index("friendships", ["from", "to"])
```

Benutzer und Freundschaften

```
def add_user(self, user: User) -> str:
    """Fügt einen neuen Benutzer hinzu."""
    user_doc = user.to_dict()
    result = self.db.insert("users", user_doc)
    return result["_key"]

def add_friendship(self, friendship: Friendship):
    """Erstellt eine bidirektionale Freundschaft."""
    # Richtung 1: A -> B
    self.db.insert("friendships", friendship.to_dict())

    # Richtung 2: B -> A (für ungerichteten Graph)
    reverse = Friendship(
        from_user=friendship.to_user,
        to_user=friendship.from_user,
        since=friendship.since,
        strength=friendship.strength
    )
    self.db.insert("friendships", reverse.to_dict())

def get_friends(self, user_id: str) -> List[User]:
    """Holt alle direkten Freunde eines Benutzers."""
    query = """
        FOR friend IN OUTBOUND @user_id friendships
            RETURN friend
    """
    result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
    return [User(**doc) for doc in result]
```

Friends-of-Friends (FoF)

Ein klassisches Graph-Problem: Finde Freunde meiner Freunde, die noch nicht meine Freunde sind.

```python
def get_friend_suggestions(self, user_id: str, limit: int = 10) -> List[dict]:
    """Empfiehlt neue Freunde basierend auf gemeinsamen Freunden."""
    query = """
        FOR friend IN OUTBOUND @user_id friendships
            FOR fof IN OUTBOUND friend._id friendships
                FILTER fof._id != @user_id
                FILTER fof._id NOT IN (
                    FOR f IN OUTBOUND @user_id friendships
                        RETURN f._id
                )
                COLLECT fof_user = fof WITH COUNT INTO mutual_count
                SORT mutual_count DESC
                LIMIT @limit
                RETURN {
                    user: fof_user,
                    mutual_friends: mutual_count
                }
    """
    return self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })
```

Was passiert hier? 1. Traversiere zu allen Freunden (`OUTBOUND @user_id`) 2. Von jedem Freund, traversiere zu deren Freunden (`OUTBOUND friend._id`) 3. Filtere mich selbst heraus (`FILTER fof._id != @user_id`) 4. Filtere existierende Freunde heraus (Subquery) 5. Gruppiere nach FoF und zähle gemeinsame Freunde (`COLLECT ... WITH COUNT`) 6. Sortiere nach Anzahl gemeinsamer Freunde

Kürzeste Pfade

Wie ist Alice mit Bob verbunden?

```python
def find_connection_path(self, user_id1: str, user_id2: str) -> dict:
    """Findet den kürzesten Pfad zwischen zwei Benutzern."""
    path = self.db.shortest_path(
        graph="social_graph",
        start_vertex=f"users/{user_id1}",
        target_vertex=f"users/{user_id2}",
        direction="any"  # Ungerichtet (beide Richtungen)
    )

    if path is None:
        return {"connected": False}

    return {
        "connected": True,
        "distance": path["distance"],
        "path": [v["name"] for v in path["vertices"]],
        "degrees_of_separation": len(path["vertices"]) - 1
    }
```

Community-Erkennung

Welche natürlichen Gruppen gibt es im Netzwerk?

```
def detect_communities(self) -> dict:
    """Findet Communities mittels Connected Components."""
    components = self.db.connected_components(
        graph="social_graph",
        direction="any"
    )

    # Statistiken pro Community
    communities = {}
    for component_id, user_ids in components.items():
        users = [self.db.get("users", uid) for uid in user_ids]

        # Gemeinsame Interessen
        all_interests = []
        for user in users:
            all_interests.extend(user.get("interests", []))
        interest_counts = Counter(all_interests)

        communities[component_id] = {
            "size": len(users),
            "members": [u["name"] for u in users],
            "top_interests": interest_counts.most_common(5)
        }

    return communities
```

Interaktive Visualisierung

Das Example nutzt NetworkX für Visualisierung:

```
import networkx as nx
import matplotlib.pyplot as plt

def visualize_network(self, user_id: Optional[str] = None, max_hops: int = 2):
    """Visualisiert das soziale Netzwerk."""
    G = nx.Graph()

    if user_id:
        # Nur Ego-Netzwerk eines Users
        query = f"""
            FOR v, e IN 1..{max_hops} ANY @user_id friendships
                RETURN {{vertex: v, edge: e}}
        """
        result = self.db.execute_query(query, bind_vars={"user_id": f"users/{user_id}"})
    else:
        # Ganzes Netzwerk
        users = self.db.all("users")
        friendships = self.db.all("friendships")

        for user in users:
            G.add_node(user["_key"], name=user["name"])

        for friendship in friendships:
            from_id = friendship["_from"].split("/")[1]
            to_id = friendship["_to"].split("/")[1]
            G.add_edge(from_id, to_id, weight=friendship.get("strength", 1.0))

    # Layout mit Spring-Algorithmus
    pos = nx.spring_layout(G, k=0.5, iterations=50)

    # Zeichnen
    plt.figure(figsize=(12, 8))
    nx.draw_networkx_nodes(G, pos, node_size=500, node_color='lightblue')
    nx.draw_networkx_edges(G, pos, alpha=0.5)
    nx.draw_networkx_labels(G, pos, labels=nx.get_node_attributes(G, 'name'))

    plt.title("Social Network Graph")
    plt.axis('off')
    plt.tight_layout()
    plt.show()
```

Main Application

```python
def main():
    client = ThemisGraphClient()

    # Beispiel-Daten
    users = [
        User("alice", "Alice", "Software Engineer", ["Python", "ML", "Hiking"], "Berlin"),
        User("bob", "Bob", "Data Scientist", ["Python", "Statistics", "Music"], "Munich"),
        User("charlie", "Charlie", "DevOps", ["Docker", "Kubernetes", "Hiking"], "Berlin"),
        User("diana", "Diana", "Product Manager", ["Agile", "UX", "Music"], "Hamburg"),
        User("eve", "Eve", "ML Engineer", ["ML", "Python", "Research"], "Berlin"),
    ]

    for user in users:
        client.add_user(user)

    # Freundschaften
    friendships = [
        ("alice", "bob"),
        ("alice", "charlie"),
        ("bob", "diana"),
        ("charlie", "eve"),
        ("diana", "eve"),
    ]

    for user1, user2 in friendships:
        client.add_friendship(Friendship(user1, user2, datetime.now(), 0.8))

    # 1. Freunde von Alice
    print("Alice's Friends:")
    friends = client.get_friends("alice")
    for friend in friends:
        print(f"  - {friend.name} ({friend.location})")

    # 2. Freundschafts-Empfehlungen für Alice
    print("\nFriend Suggestions for Alice:")
    suggestions = client.get_friend_suggestions("alice", limit=3)
    for suggestion in suggestions:
        print(f"  - {suggestion['user']['name']}: {suggestion['mutual_friends']} mutual frien

    # 3. Verbindung zwischen Alice und Eve
    print("\nConnection between Alice and Eve:")
    path = client.find_connection_path("alice", "eve")
    if path["connected"]:
        print(f"  Path: {' -> '.join(path['path'])}")
        print(f"  Degrees of Separation: {path['degrees_of_separation']}")

    # 4. Communities
    print("\nCommunities:")
    communities = client.detect_communities()
    for comm_id, comm_data in communities.items():
        print(f"  Community {comm_id}: {comm_data['size']} members")
        print(f"    Members: {', '.join(comm_data['members'])}")
        print(f"    Top Interests: {[i[0] for i in comm_data['top_interests']]}")
```

```
    # 5. Visualisierung
    client.visualize_network()

if __name__ == "__main__":
    main()
```

Output

```
Alice's Friends:
  - Bob (Munich)
  - Charlie (Berlin)

Friend Suggestions for Alice:
  - Diana: 1 mutual friends
  - Eve: 1 mutual friends

Connection between Alice and Eve:
  Path: Alice -> Charlie -> Eve
  Degrees of Separation: 2

Communities:
  Community 1: 5 members
    Members: Alice, Bob, Charlie, Diana, Eve
    Top Interests: ['Python', 'ML', 'Hiking', 'Music', 'Docker']

[Visualization opens in matplotlib window]
```

Performance-Optimierung

1. Indexes auf häufig abgefragte Felder:

```
self.db.add_index("users", ["name"])
self.db.add_index("users", ["location"])
self.db.add_index("friendships", ["from", "to"])
```

2. Batch-Operations für viele Inserts:

```
def add_users_batch(self, users: List[User]):
    """Fügt mehrere User auf einmal hinzu."""
    user_docs = [u.to_dict() for u in users]
    self.db.insert_many("users", user_docs)
```

3. Bounded Traversals:

```
FOR v IN 1..3 OUTBOUND @user_id friendships  # Max 3 Hops
    RETURN v
```

4. Index-basierte Filters:

```
FOR user IN users
    FILTER user.location == "Berlin"  # Nutzt Index
    FOR friend IN OUTBOUND user._id friendships
        RETURN friend
```

6.6 Praxisbeispiel 2: Recommendation Engine

Das zweite Example (`examples/19_recommendation_engine`) zeigt einen komplexeren Use Case: **ML-basierte Empfehlungen** mit Graph- und Vektor-Daten.

Das Konzept

Empfehlungssysteme kombinieren mehrere Ansätze:

1. **Collaborative Filtering**: "User, die X mochten, mochten auch Y"
2. **Content-Based Filtering**: "Ähnliche Items basierend auf Features"
3. **Graph-basiert**: "Freunde deiner Freunde kauften X"
4. **Hybrid**: Kombination aller Ansätze

Datenmodell

```
@dataclass
class Item:
    """Ein empfehlbares Item (Produkt, Film, etc.)."""
    id: str
    title: str
    category: str
    features: List[str]
    embedding: List[float]  # Content-Embedding für Similarity

@dataclass
class Interaction:
    """Eine User-Item Interaktion."""
    user_id: str
    item_id: str
    type: str  # "view", "click", "purchase", "rate"
    rating: Optional[float] = None
    timestamp: datetime = None
    context: dict = None  # Device, location, etc.

@dataclass
class Recommendation:
    """Eine generierte Empfehlung."""
    user_id: str
    item_id: str
    score: float
    method: str  # "collaborative", "content_based", "graph", "hybrid"
    explanation: str
```

Graph-Setup

```
def _setup_graph(self):
    """Erstellt Multi-Graph für Recommendations."""
    # Vertex Collections
    self.db.create_vertex_collection("users")
    self.db.create_vertex_collection("items")

    # Edge Collections
    self.db.create_edge_collection("interactions")  # User -> Item
    self.db.create_edge_collection("similar_items")  # Item -> Item
    self.db.create_edge_collection("user_similarity")  # User -> User

    # Graph-Definition
    graph_def = {
        "name": "recommendation_graph",
        "edge_definitions": [
            {
                "collection": "interactions",
                "from": ["users"],
                "to": ["items"]
            },
            {
                "collection": "similar_items",
                "from": ["items"],
                "to": ["items"]
            },
            {
                "collection": "user_similarity",
                "from": ["users"],
                "to": ["users"]
            }
        ]
    }
    self.db.create_graph(graph_def)
```

Collaborative Filtering

Findet Items, die ähnliche User mochten:

```
def collaborative_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items basierend auf ähnlichen Usern."""
    query = """
        // 1. Finde ähnliche User (basierend auf vergangenen Interaktionen)
        FOR similar_user IN OUTBOUND @user_id user_similarity
            SORT similar_user.similarity DESC
            LIMIT 20

            // 2. Finde Items, die ähnliche User mochten
            FOR item IN OUTBOUND similar_user._id interactions
                FILTER item.interaction_type IN ["purchase", "rate"]
                FILTER item.rating >= 4  // Nur positive

                // 3. Filtere Items, die User schon kennt
                FILTER item._id NOT IN (
                    FOR known_item IN OUTBOUND @user_id interactions
                        RETURN known_item._id
                )

                // 4. Aggregiere Score
                COLLECT item_id = item._id
                AGGREGATE score = AVG(item.rating * similar_user.similarity)

                SORT score DESC
                LIMIT @limit

                // 5. Hole Item-Details
                LET item_doc = DOCUMENT(item_id)
                RETURN {
                    item_id: item_id,
                    score: score,
                    method: "collaborative",
                    explanation: CONCAT("Users similar to you rated this ", score)
                }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]
```

Content-Based Filtering

Findet ähnliche Items basierend auf Features:

```
def content_based_filtering(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items ähnlich zu den, die User mag."""
    query = """
        // 1. Finde Items, die User mag
        FOR liked_item IN OUTBOUND @user_id interactions
            FILTER liked_item.rating >= 4

            // 2. Finde ähnliche Items
            FOR similar_item IN OUTBOUND liked_item._id similar_items
                SORT similar_item.similarity DESC

                // 3. Filtere bekannte Items
                FILTER similar_item._id NOT IN (
                    FOR known IN OUTBOUND @user_id interactions
                        RETURN known._id
                )

                // 4. Aggregiere
                COLLECT item_id = similar_item._id
                AGGREGATE score = AVG(similar_item.similarity)

                SORT score DESC
                LIMIT @limit

                LET item_doc = DOCUMENT(item_id)
                RETURN {
                    item_id: item_id,
                    score: score,
                    method: "content_based",
                    explanation: CONCAT("Similar to items you liked")
                }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]
```

Graph-basierte Empfehlungen

Nutzt Social Graph für Empfehlungen:

```
def graph_based_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Empfehle Items, die Freunde mochten."""
    query = """
        // 1. Traversiere zu Freunden (1-2 Hops)
        FOR friend IN 1..2 OUTBOUND @user_id friendships

            // 2. Finde Items, die Freund kaufte
            FOR item IN OUTBOUND friend._id interactions
                FILTER item.interaction_type == "purchase"

                // 3. Filtere bekannte Items
                FILTER item._id NOT IN (
                    FOR known IN OUTBOUND @user_id interactions
                        RETURN known._id
                )

                // 4. Score basierend auf Freundschafts-Stärke und Rating
                COLLECT item_id = item._id
                AGGREGATE score = AVG(friend.friendship_strength * item.rating)

                SORT score DESC
                LIMIT @limit

                LET item_doc = DOCUMENT(item_id)
                RETURN {
                    item_id: item_id,
                    score: score,
                    method: "graph_based",
                    explanation: "Your friends liked this"
                }
    """

    results = self.db.execute_query(query, bind_vars={
        "user_id": f"users/{user_id}",
        "limit": limit
    })

    return [Recommendation(**r) for r in results]
```

Hybrid Recommendations

Kombiniert alle Methoden mit Gewichtung:

```python
def hybrid_recommendations(self, user_id: str, limit: int = 10) -> List[Recommendation]:
    """Kombiniert alle Empfehlungsmethoden."""
    # Hole Empfehlungen von allen Methoden
    collaborative = self.collaborative_filtering(user_id, limit=20)
    content_based = self.content_based_filtering(user_id, limit=20)
    graph_based = self.graph_based_recommendations(user_id, limit=20)

    # Gewichtung
    weights = {
        "collaborative": 0.4,
        "content_based": 0.3,
        "graph_based": 0.3
    }

    # Kombiniere Scores
    combined_scores = {}
    for recs, method in [
        (collaborative, "collaborative"),
        (content_based, "content_based"),
        (graph_based, "graph_based")
    ]:
        for rec in recs:
            if rec.item_id not in combined_scores:
                combined_scores[rec.item_id] = {
                    "score": 0,
                    "methods": [],
                    "explanations": []
                }

            combined_scores[rec.item_id]["score"] += rec.score * weights[method]
            combined_scores[rec.item_id]["methods"].append(method)
            combined_scores[rec.item_id]["explanations"].append(rec.explanation)

    # Sortiere und limitiere
    sorted_items = sorted(
        combined_scores.items(),
        key=lambda x: x[1]["score"],
        reverse=True
    )[:limit]

    # Erstelle finale Recommendations
    recommendations = []
    for item_id, data in sorted_items:
        recommendations.append(Recommendation(
            user_id=user_id,
            item_id=item_id,
            score=data["score"],
            method="hybrid",
            explanation=f"Recommended via: {', '.join(data['methods'])}"
        ))

    return recommendations
```

Similarity Berechnung

Berechne User-User und Item-Item Similarity:

```
def compute_user_similarity(self):
    """Berechnet Similarity zwischen allen User-Paaren."""
    users = self.db.all("users")

    for i, user1 in enumerate(users):
        for user2 in users[i+1:]:
            # Gemeinsame Interaktionen
            common_items = self._get_common_interactions(user1["_id"], user2["_id"])

            if len(common_items) >= 3:  # Mindestens 3 gemeinsame
                similarity = self._calculate_similarity(
                    user1["interaction_vector"],
                    user2["interaction_vector"]
                )

                # Speichere Kante
                self.db.insert("user_similarity", {
                    "from": user1["_id"],
                    "to": user2["_id"],
                    "similarity": similarity,
                    "common_items": len(common_items)
                })

def compute_item_similarity(self):
    """Berechnet Similarity zwischen Items (content-based)."""
    items = self.db.all("items")

    for i, item1 in enumerate(items):
        for item2 in items[i+1:]:
            # Cosine Similarity der Embeddings
            similarity = cosine_similarity(
                item1["embedding"],
                item2["embedding"]
            )

            if similarity > 0.7:  # Threshold
                self.db.insert("similar_items", {
                    "from": item1["_id"],
                    "to": item2["_id"],
                    "similarity": similarity
                })
```

Real-Time Updates

Aktualisiere Empfehlungen bei neuen Interaktionen:

```
def record_interaction(self, interaction: Interaction):
    """Zeichnet Interaktion auf und aktualisiert Empfehlungen."""
    # Speichere Interaktion
    interaction_doc = {
        "from": f"users/{interaction.user_id}",
        "to": f"items/{interaction.item_id}",
        "type": interaction.type,
        "rating": interaction.rating,
        "timestamp": interaction.timestamp.isoformat(),
        "context": interaction.context
    }
    self.db.insert("interactions", interaction_doc)

    # Bei Purchase: Update User-Vektor
    if interaction.type == "purchase":
        self._update_user_vector(interaction.user_id, interaction.item_id)

        # Recalculate Similarity (async)
        self._queue_similarity_update(interaction.user_id)
```

6.7 Graph Best Practices

1. Modeling Guidelines

DO: - Nutze sprechende Edge-Namen (`FRIEND`, `LIKES`, `WORKS_AT`) - Speichere Properties auf Kanten (Zeitstempel, Gewichte) - Denormalisiere häufig benötigte Daten - Nutze Bidirektionale Kanten für ungerichtete Graphs

DON'T: - × Zu viele Edge-Types (schwer zu querien) - × Properties als separate Knoten (ineffizient) - × Extrem tiefe Hierarchien (> 10 Levels)

2. Performance-Tipps

Indexes:

```
db.add_index("friendships", ["from", "to"])
db.add_index("interactions", ["user_id", "timestamp"])
```

Bounded Traversals:

```
FOR v IN 1..3 OUTBOUND @start  # Nicht 1..ANY
    LIMIT 100  # Früh limitieren
    RETURN v
```

Prune Early:

```
FOR v IN 1..5 OUTBOUND @start friendships
    FILTER v.city == "Berlin"  # Früher Filter
    FOR v2 IN OUTBOUND v._id friendships
        RETURN v2
```

3. Häufige Patterns

Pattern 1: Mutual Friends

```
FOR friend IN OUTBOUND @user1 friendships
    FILTER friend._id IN (
        FOR f IN OUTBOUND @user2 friendships
            RETURN f._id
    )
    RETURN friend
```

Pattern 2: Influencer (hohe Degree)

```
FOR user IN users
    LET friend_count = LENGTH(
        FOR f IN OUTBOUND user._id friendships
            RETURN 1
    )
    FILTER friend_count > 100
    SORT friend_count DESC
    RETURN {user: user, friends: friend_count}
```

Pattern 3: Isolated Nodes

```
FOR user IN users
    LET connections = (
        FOR v IN ANY user._id friendships
            RETURN 1
    )
    FILTER LENGTH(connections) == 0
    RETURN user
```

6.8 Vergleich: ThemisDB vs. Neo4j vs. ArangoDB

Feature	ThemisDB	Neo4j	ArangoDB
Graph Model	Property Graph	Property Graph	Property Graph
Query Language	AQL	Cypher	AQL
Multi-Model	4 Models	× Graph only	3 Models
ACID	Full	Full	Full
Sharding	Automatic	× Enterprise only	Yes
Graph Algorithms	Built-in	GDS Library	Pregel
License	Apache 2.0	GPLv3 + Commercial	Apache 2.0
Embedding Support	Native	× No	× No

Wann ThemisDB? - Multi-Model Daten (Graph + Vektor + Relational) - Native Vektor-Suche benötigt - Open-Source und selbst-hosted - Kombinierte Queries über mehrere Modelle

Wann Neo4j? - Reines Graph-Problem - Cypher-Erfahrung im Team - Enterprise Support benötigt - Graph Data Science Library

Wann ArangoDB? - Ähnlich wie ThemisDB (Multi-Model) - Mature Community - Foxx Microservices

6.9 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Property Graph Modell - Knoten, Kanten, Properties
Graph-Traversierung - DFS, BFS, Pattern Matching
Graph-Algorithmen - Shortest Path, PageRank, Community Detection
Praxis: Social Network - Vollständiges soziales Netzwerk mit Visualisierung
Praxis: Recommendations - ML-basierte Empfehlungen mit Hybrid-Ansatz
Best Practices - Modeling, Performance, Patterns

Graph-Datenbanken sind die natürliche Wahl für vernetzte Daten. ThemisDB's Property Graph-Implementierung bietet performante Traversierung, native Algorithmen und die einzigartige Möglichkeit, Graphs mit anderen Modellen zu kombinieren.

Im nächsten Kapitel schauen wir uns **Dokument-Speicherung** an - flexibles, schema-less Design für semi-strukturierte Daten.

Übungen:

1. Erweitern Sie das Social Network um Posts und Likes (User -> Post -> Likes)

2. Implementieren Sie einen Follower/Following-Mechanismus (Twitter-Style)
3. Fügen Sie PageRank-Berechnung hinzu, um Influencer zu finden
4. Erstellen Sie einen Interest-Based-Graph (User -> Interest <- User)
5. Bauen Sie ein Skill-Recommendation-System für LinkedIn-Style-Netzwerk

Weiterführende Ressourcen:

- `examples/06_graph_social_network/` - Vollständiger Code
- `examples/19_recommendation_engine/` - Recommendation System
- `docs/de/features/features_property_graph.md` - Graph-Features
- NetworkX Documentation - Graph-Visualisierung
- "Graph Algorithms" (Mark Needham, Amy E. Hodler) - Tiefere Algorithmen

---

Kapitel 8: Kapitel 7: Dokument-Speicherung

> ***In diesem Kapitel:*** *Schema-less Design, flexible Datenstrukturen, Nested Objects, JSON-Operationen, Content Management, Versionierung und Migration von MongoDB.*

7.1 Das Document Model

Warum Dokumente?

Das Document Model ist ideal für Anwendungen mit:

- **Flexible Schemas:** Nicht alle Datensätze haben die gleichen Felder
- **Nested Data:** Hierarchische Strukturen (Kommentare, Tags, Metadaten)
- **Rapid Development:** Schema-Evolution ohne Migration
- **Content Management:** Blogs, Wikis, CMS-Systeme

Beispiele aus der Praxis: - Blog-Systeme mit variablen Post-Typen (Text, Video, Galerie) - Wikis mit verschachtelten Inhalten und Revisionen - Content-Management mit Metadata - Konfigurationsdaten mit unterschiedlichen Formaten

Dokumente in ThemisDB

ThemisDB speichert Dokumente als JSON mit vollem ACID-Support:

```
from themisdb import ThemisDB

db = ThemisDB()

article = {
    "title": "Multi-Model Databases",
    "author": "Alice",
    "content": "Content here...",
    "tags": ["database", "multi-model"],
    "metadata": {
        "published": "2025-01-15",
        "views": 1024
    },
    "comments": [
        {"user": "Bob", "text": "Great article!"},
        {"user": "Carol", "text": "Very helpful"}
    ]
}

db.documents.insert("articles", article)
```

Vorteile: - Keine Schema-Definition notwendig - Beliebige Verschachtelung - Arrays von Sub-Dokumenten - Optionale Felder - JSON-Operationen (Path-Queries, Updates)

Vergleich: Relational vs. Document

Relational (starr):

```
-- Zwei separate Collections mit Referenzen
// articles Collection
{
  _key: "1",
  title: "Artikel",
  content: "Text..."
}

// comments Collection (Referenz via article_id)
{
  _key: "c1",
  article_id: "1",
  user: "alice",
  text: "Kommentar"
}
```

Document (flexibel):

```
// Ein Dokument, alles inklusive
{
    "id": 1,
    "title": "...",
    "content": "...",
    "comments": [{"user": "...", "text": "..."}]
}
```

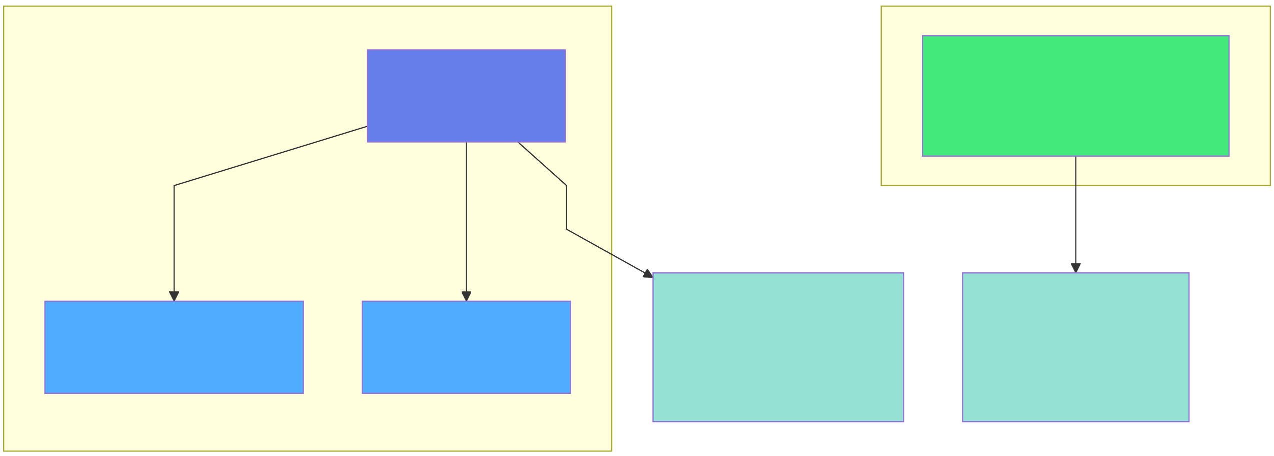


Abb. 23: Architekturdiagramm (Kapitel 7: Dokument-Speicherung)

Schema Evolution

Neue Felder ohne Migration hinzufügen:

```
{
    "title": "Hello World",
    "content": "..."
}

{
    "title": "Hello World",
    "content": "...",
    "author": "Alice"
}

{
    "title": "Hello World",
    "content": "...",
    "author": "Alice",
    "tags": ["intro", "tutorial"],
    "metadata": {
        "published": "2025-01-15",
        "featured": True
    }
}
```

Anwendungscode prüft Feld-Existenz:

```
article = db.documents.get("articles", article_id)

author = article.get("author", "Unknown")
tags = article.get("tags", [])
```

7.2 JSON-Operationen

Path-Queries

Tief in verschachtelten Dokumenten suchen:

```
articles = db.query("""
    FOR article IN articles
      FILTER article.metadata.published > '2025-01-01'
        AND article.metadata.featured == true
      RETURN article
""")

articles_with_tag = db.query("""
    FOR article IN articles
      FILTER 'database' IN article.tags
      RETURN article
""")

articles_by_bob = db.query("""
    FOR article IN articles
      FILTER 'Bob' IN article.comments[*].user
      RETURN article
""")
```

Partial Updates

Nur bestimmte Felder ändern:

```
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"metadata.views": db.increment(1)}
)

new_comment = {"user": "Dave", "text": "Thanks!"}
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"comments": db.array_append(new_comment)}
)

db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"metadata.featured": True}
)
```

JSON-Funktionen

```
result = db.query("""
    SELECT
        title,
        metadata.published AS date,
        ARRAY_LENGTH(comments) AS comment_count
    FROM articles
    WHERE metadata.featured = true
""")

stats = db.query("""
    SELECT
        author,
        COUNT(*) AS article_count,
        SUM(metadata.views) AS total_views
    FROM articles
    GROUP BY author
""")
```

7.3 Example: Blog/Wiki System

> ***Example:*** `examples/11_blog_wiki`

Implementieren wir ein vollständiges Content-Management-System mit Artikeln, Revisions-Historie und Tagging.

System-Überblick

Features: - Artikel mit Rich Content (Markdown) - Revisions-Historie (alle Änderungen) - Tagging und Kategorisierung - Volltext-Suche über Titel und Content - View-Counter und Statistiken - Kommentar-System

Datenstruktur:

```
{
    "id": "uuid",
    "title": "Getting Started with ThemisDB",
    "slug": "getting-started-themisdb",
    "content": "# Introduction\n\n...",
    "author": "alice@example.com",
    "status": "published",  # draft, published, archived
    "tags": ["tutorial", "database", "introduction"],
    "category": "tutorials",
    "metadata": {
        "published_at": "2025-01-15T10:00:00Z",
        "updated_at": "2025-01-16T14:30:00Z",
        "views": 1250,
        "featured": True
    },
    "comments": [
        {
            "id": "comment-uuid",
            "author": "bob@example.com",
            "text": "Great tutorial!",
            "created_at": "2025-01-15T11:00:00Z"
        }
    ],
    "revisions": [
        {
            "version": 1,
            "content": "Initial version...",
            "changed_by": "alice@example.com",
            "changed_at": "2025-01-15T10:00:00Z"
        },
        {
            "version": 2,
            "content": "Updated version...",
            "changed_by": "alice@example.com",
            "changed_at": "2025-01-16T14:30:00Z"
        }
    ]
}
```

Implementation

`blog_wiki/models.py`:

```python
from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Optional
import uuid

@dataclass
class Comment:
    id: str
    author: str
    text: str
    created_at: datetime

    @staticmethod
    def create(author: str, text: str) -> dict:
        return {
            "id": str(uuid.uuid4()),
            "author": author,
            "text": text,
            "created_at": datetime.now().isoformat()
        }

@dataclass
class Revision:
    version: int
    content: str
    changed_by: str
    changed_at: datetime

@dataclass
class Article:
    id: str
    title: str
    slug: str
    content: str
    author: str
    status: str  # draft, published, archived
    tags: List[str]
    category: str
    metadata: dict
    comments: List[dict] = field(default_factory=list)
    revisions: List[dict] = field(default_factory=list)

    @staticmethod
    def create(title: str, content: str, author: str,
               tags: List[str], category: str) -> dict:
        now = datetime.now().isoformat()
        article_id = str(uuid.uuid4())
        slug = title.lower().replace(" ", "-")

        article = {
            "id": article_id,
            "title": title,
            "slug": slug,
            "content": content,
```

```
            "author": author,
            "status": "draft",
            "tags": tags,
            "category": category,
            "metadata": {
                "created_at": now,
                "updated_at": now,
                "published_at": None,
                "views": 0,
                "featured": False
            },
            "comments": [],
            "revisions": [
                {
                    "version": 1,
                    "content": content,
                    "changed_by": author,
                    "changed_at": now
                }
            ]
        }
        return article
```

`blog_wiki/blog.py`:

```python
from themisdb import ThemisDB
from datetime import datetime
from typing import List, Optional

class BlogWiki:
    def __init__(self, db_path: str = "blog.db"):
        self.db = ThemisDB(db_path)
        self._setup_indexes()

    def _setup_indexes(self):
        """Create indexes for performance"""
        # Fulltext search on title and content
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_fulltext
            ON articles USING FULLTEXT(title, content)
        """)

        # Fast lookup by slug
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_slug
            ON articles(slug)
        """)

        # Filter by status and category
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_status_category
            ON articles(status, category)
        """)

        # Tag search
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_articles_tags
            ON articles(tags)
        """)

    def create_article(self, title: str, content: str, author: str,
                       tags: List[str], category: str) -> str:
        """Create new article"""
        from models import Article

        article = Article.create(title, content, author, tags, category)
        self.db.documents.insert("articles", article)
        return article["id"]

    def publish_article(self, article_id: str):
        """Publish draft article"""
        with self.db.transaction():
            self.db.documents.update(
                collection="articles",
                document_id=article_id,
                updates={
                    "status": "published",
                    "metadata.published_at": datetime.now().isoformat()
                }
```

```
        )

    def update_article(self, article_id: str, content: str,
                      updated_by: str):
        """Update article and save revision"""
        article = self.db.documents.get("articles", article_id)

        # Create new revision
        new_version = len(article["revisions"]) + 1
        revision = {
            "version": new_version,
            "content": content,
            "changed_by": updated_by,
            "changed_at": datetime.now().isoformat()
        }

        with self.db.transaction():
            # Update content and add revision
            self.db.documents.update(
                collection="articles",
                document_id=article_id,
                updates={
                    "content": content,
                    "metadata.updated_at": datetime.now().isoformat(),
                    "revisions": self.db.array_append(revision)
                }
            )

    def add_comment(self, article_id: str, author: str, text: str):
        """Add comment to article"""
        from models import Comment

        comment = Comment.create(author, text)

        self.db.documents.update(
            collection="articles",
            document_id=article_id,
            updates={
                "comments": self.db.array_append(comment)
            }
        )

    def increment_views(self, article_id: str):
        """Increment view counter"""
        self.db.documents.update(
            collection="articles",
            document_id=article_id,
            updates={
                "metadata.views": self.db.increment(1)
            }
        )

    def search_articles(self, query: str) -> List[dict]:
        """Fulltext search"""
```

```
        results = self.db.query("""
            FOR article IN articles
              FILTER FULLTEXT(article.title, @query) OR FULLTEXT(article.content, @query)
                AND article.status == 'published'
              SORT article.metadata.published_at DESC
              RETURN article
        """, {"query": query})
        return results

    def get_by_tag(self, tag: str) -> List[dict]:
        """Get articles by tag"""
        results = self.db.query("""
            FOR article IN articles
              FILTER @tag IN article.tags
                AND article.status == 'published'
              SORT article.metadata.published_at DESC
              RETURN article
        """, {"tag": tag})
        return results

    def get_by_category(self, category: str) -> List[dict]:
        """Get articles by category"""
        results = self.db.query("""
            FOR article IN articles
              FILTER article.category == @category
                AND article.status == 'published'
              SORT article.metadata.published_at DESC
              RETURN article
        """, {"category": category})
        return results

    def get_popular(self, limit: int = 10) -> List[dict]:
        """Get most viewed articles"""
        results = self.db.query("""
            FOR article IN articles
              FILTER article.status == 'published'
              SORT article.metadata.views DESC
              LIMIT @limit
              RETURN article
        """, {"limit": limit})
        return results

    def get_featured(self) -> List[dict]:
        """Get featured articles"""
        results = self.db.query("""
            FOR article IN articles
              FILTER article.status == 'published'
                AND article.metadata.featured == true
              SORT article.metadata.published_at DESC
              RETURN article
        """)
        return results

    def get_revisions(self, article_id: str) -> List[dict]:
```

```
        """Get revision history"""
        article = self.db.documents.get("articles", article_id)
        return article.get("revisions", [])

    def revert_to_revision(self, article_id: str, version: int,
                           reverted_by: str):
        """Revert to previous version"""
        article = self.db.documents.get("articles", article_id)
        revisions = article["revisions"]

        # Find target revision
        target = next((r for r in revisions if r["version"] == version), None)
        if not target:
            raise ValueError(f"Revision {version} not found")

        # Create revert revision
        new_version = len(revisions) + 1
        revert_revision = {
            "version": new_version,
            "content": target["content"],
            "changed_by": reverted_by,
            "changed_at": datetime.now().isoformat(),
            "reverted_from": version
        }

        with self.db.transaction():
            self.db.documents.update(
                collection="articles",
                document_id=article_id,
                updates={
                    "content": target["content"],
                    "metadata.updated_at": datetime.now().isoformat(),
                    "revisions": self.db.array_append(revert_revision)
                }
            )

    def get_statistics(self) -> dict:
        """Get blog statistics"""
        stats = self.db.query("""
            SELECT
                COUNT(*) AS total_articles,
                SUM(CASE WHEN status='published' THEN 1 ELSE 0 END)
                    AS published,
                SUM(CASE WHEN status='draft' THEN 1 ELSE 0 END)
                    AS drafts,
                SUM(metadata.views) AS total_views,
                AVG(metadata.views) AS avg_views
            FROM articles
        """)[0]

        return stats
```

Praktische Anwendung

Blog-Post erstellen und veröffentlichen:

```
from blog import BlogWiki

blog = BlogWiki()

article_id = blog.create_article(
    title="Getting Started with ThemisDB",
    content="""
    # Introduction

    ThemisDB is a multi-model database...

    ## Features
    - ACID transactions
    - Multiple data models
    - High performance
    """,
    author="alice@example.com",
    tags=["tutorial", "database", "getting-started"],
    category="tutorials"
)

blog.publish_article(article_id)

blog.add_comment(
    article_id,
    author="bob@example.com",
    text="Great tutorial! Very helpful."
)

blog.increment_views(article_id)
```

Artikel aktualisieren mit Revisions-Historie:

```
blog.update_article(
    article_id,
    content="""
    # Introduction (Updated!)

    ThemisDB is a powerful multi-model database...

    ## New Features
    - ACID transactions
    - Multiple data models
    - High performance
    - Geo-spatial support (NEW!)
    """,
    updated_by="alice@example.com"
)

revisions = blog.get_revisions(article_id)
for rev in revisions:
    print(f"Version {rev['version']} by {rev['changed_by']}")
    print(f"  Changed at: {rev['changed_at']}")

blog.revert_to_revision(article_id, version=1,
                       reverted_by="alice@example.com")
```

Suchen und Filtern:

```
results = blog.search_articles("multi-model database")
print(f"Found {len(results)} articles")

tutorials = blog.get_by_tag("tutorial")

db_articles = blog.get_by_category("database")

popular = blog.get_popular(limit=5)

featured = blog.get_featured()
```

Statistiken:

```
stats = blog.get_statistics()
print(f"Total articles: {stats['total_articles']}")
print(f"Published: {stats['published']}")
print(f"Drafts: {stats['drafts']}")
print(f"Total views: {stats['total_views']}")
print(f"Avg views: {stats['avg_views']:.1f}")
```

Key Features

1. Schema Flexibility: - Artikel können unterschiedliche Felder haben - Neue Features ohne Migration (z.B. `metadata.featured`) - Optionale Felder (z.B. `published_at` nur bei Status="published")

2. Revision History: - Jede Änderung wird gespeichert - Komplette Historie verfügbar - Revert zu jeder Version möglich

3. Nested Data: - Comments als Array von Sub-Dokumenten - Metadata als verschachteltes Objekt - Keine JOIN-Queries notwendig

4. Performance: - Fulltext-Index für Suche - Index auf Slug für schnelle Lookups - Composite-Index für Status+Category

7.4 Example: Recipe Manager

> ***Example:*** `examples/13_recipe_manager`

Ein Rezept-Verwaltungssystem zeigt die Stärken des Document Models bei komplexen, verschachtelten Datenstrukturen.

System-Überblick

Features: - Rezepte mit Zutaten und Schritten - Nährwertangaben und Tags - Bewertungen und Kommentare - Einkaufslisten-Generator - Meal Planning

Datenstruktur:

```
{
    "id": "uuid",
    "title": "Spaghetti Carbonara",
    "description": "Classic Italian pasta dish",
    "cuisine": "Italian",
    "difficulty": "medium",  # easy, medium, hard
    "prep_time": 15,  # minutes
    "cook_time": 20,
    "servings": 4,
    "ingredients": [
        {
            "name": "Spaghetti",
            "amount": 400,
            "unit": "g",
            "category": "pasta"
        },
        {
            "name": "Eggs",
            "amount": 4,
            "unit": "pieces",
            "category": "dairy"
        },
        {
            "name": "Parmesan",
            "amount": 100,
            "unit": "g",
            "category": "dairy"
        }
    ],
    "steps": [
        {
            "number": 1,
            "instruction": "Boil water and cook spaghetti al dente",
            "duration": 10
        },
        {
            "number": 2,
            "instruction": "Mix eggs with parmesan",
            "duration": 5
        },
        {
            "number": 3,
            "instruction": "Combine pasta with egg mixture",
            "duration": 5
        }
    ],
    "nutrition": {
        "calories": 520,
        "protein": 22,
        "carbs": 65,
        "fat": 18
    },
    "tags": ["pasta", "italian", "quick", "dinner"],
    "ratings": [
```

```
        {
            "user": "alice@example.com",
            "score": 5,
            "comment": "Delicious!",
            "date": "2025-01-15"
        }
    ],
    "created_by": "chef@example.com",
    "created_at": "2025-01-10",
    "updated_at": "2025-01-15"
}
```

Implementation

`recipe_manager/recipe.py`:

```python
from themisdb import ThemisDB
from typing import List, Optional
from datetime import datetime

class RecipeManager:
    def __init__(self, db_path: str = "recipes.db"):
        self.db = ThemisDB(db_path)
        self._setup_indexes()

    def _setup_indexes(self):
        # Fulltext search
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_recipes_fulltext
            ON recipes USING FULLTEXT(title, description)
        """)

        # Filter by tags, cuisine, difficulty
        self.db.execute("""
            CREATE INDEX IF NOT EXISTS idx_recipes_filters
            ON recipes(cuisine, difficulty, tags)
        """)

    def add_recipe(self, recipe_data: dict) -> str:
        """Add new recipe"""
        recipe_data["created_at"] = datetime.now().isoformat()
        recipe_data["updated_at"] = datetime.now().isoformat()

        recipe_id = self.db.documents.insert("recipes", recipe_data)
        return recipe_id

    def search_recipes(self, query: str) -> List[dict]:
        """Search by title or description"""
        return self.db.query("""
            FOR recipe IN recipes
              FILTER FULLTEXT(recipe.title, @query) OR FULLTEXT(recipe.description, @query)
              RETURN recipe
        """, {"query": query})

    def filter_recipes(self, cuisine: Optional[str] = None,
                       difficulty: Optional[str] = None,
                       max_time: Optional[int] = None,
                       tags: Optional[List[str]] = None) -> List[dict]:
        """Filter recipes by criteria"""
        # Build AQL query dynamically
        filters = []
        params = {}

        if cuisine:
            filters.append("recipe.cuisine == @cuisine")
            params["cuisine"] = cuisine

        if difficulty:
            filters.append("recipe.difficulty == @difficulty")
            params["difficulty"] = difficulty
```

```
        if max_time:
            filters.append("(recipe.prep_time + recipe.cook_time) <= @max_time")
            params["max_time"] = max_time

        if tags:
            for i, tag in enumerate(tags):
                filters.append(f"@tag{i} IN recipe.tags")
                params[f"tag{i}"] = tag

        filter_clause = " AND ".join(filters) if filters else "true"

        return self.db.query(f"""
            FOR recipe IN recipes
              FILTER {filter_clause}
              RETURN recipe
        """, params)

    def add_rating(self, recipe_id: str, user: str,
                   score: int, comment: str):
        """Add rating to recipe"""
        rating = {
            "user": user,
            "score": score,
            "comment": comment,
            "date": datetime.now().isoformat()
        }

        self.db.documents.update(
            collection="recipes",
            document_id=recipe_id,
            updates={
                "ratings": self.db.array_append(rating)
            }
        )

    def get_average_rating(self, recipe_id: str) -> float:
        """Calculate average rating"""
        recipe = self.db.documents.get("recipes", recipe_id)
        ratings = recipe.get("ratings", [])

        if not ratings:
            return 0.0

        return sum(r["score"] for r in ratings) / len(ratings)

    def generate_shopping_list(self, recipe_ids: List[str],
                              servings_multiplier: dict = None) -> dict:
        """Generate shopping list from multiple recipes"""
        shopping_list = {}

        for recipe_id in recipe_ids:
            recipe = self.db.documents.get("recipes", recipe_id)
            multiplier = servings_multiplier.get(recipe_id, 1.0)
```

```
        for ingredient in recipe["ingredients"]:
            name = ingredient["name"]
            amount = ingredient["amount"] * multiplier
            unit = ingredient["unit"]
            category = ingredient.get("category", "other")

            if name not in shopping_list:
                shopping_list[name] = {
                    "amount": 0,
                    "unit": unit,
                    "category": category
                }

            shopping_list[name]["amount"] += amount

    # Group by category
    categorized = {}
    for name, details in shopping_list.items():
        category = details["category"]
        if category not in categorized:
            categorized[category] = []

        categorized[category].append({
            "name": name,
            "amount": details["amount"],
            "unit": details["unit"]
        })

    return categorized

def get_top_rated(self, limit: int = 10) -> List[dict]:
    """Get top rated recipes"""
    recipes = self.db.query("""
        FOR recipe IN recipes
          RETURN recipe
    """)

    # Calculate average ratings
    rated = []
    for recipe in recipes:
        ratings = recipe.get("ratings", [])
        if ratings:
            avg = sum(r["score"] for r in ratings) / len(ratings)
            recipe["avg_rating"] = avg
            rated.append(recipe)

    # Sort by rating
    rated.sort(key=lambda x: x["avg_rating"], reverse=True)
    return rated[:limit]

def get_quick_recipes(self, max_minutes: int = 30) -> List[dict]:
    """Get recipes that can be made quickly"""
    return self.db.query("""
```

```
            FOR recipe IN recipes
              FILTER (recipe.prep_time + recipe.cook_time) <= @max_minutes
              SORT (recipe.prep_time + recipe.cook_time) ASC
              RETURN recipe
        """, {"max_minutes": max_minutes})
```

Praktische Anwendung

Rezept hinzufügen:

```
from recipe import RecipeManager

manager = RecipeManager()

recipe = {
    "title": "Spaghetti Carbonara",
    "description": "Classic Italian pasta dish",
    "cuisine": "Italian",
    "difficulty": "medium",
    "prep_time": 15,
    "cook_time": 20,
    "servings": 4,
    "ingredients": [
        {"name": "Spaghetti", "amount": 400, "unit": "g", "category": "pasta"},
        {"name": "Eggs", "amount": 4, "unit": "pieces", "category": "dairy"},
        {"name": "Parmesan", "amount": 100, "unit": "g", "category": "dairy"},
        {"name": "Bacon", "amount": 200, "unit": "g", "category": "meat"}
    ],
    "steps": [
        {"number": 1, "instruction": "Boil water and cook spaghetti", "duration": 10},
        {"number": 2, "instruction": "Mix eggs with parmesan", "duration": 5},
        {"number": 3, "instruction": "Combine everything", "duration": 5}
    ],
    "nutrition": {
        "calories": 520,
        "protein": 22,
        "carbs": 65,
        "fat": 18
    },
    "tags": ["pasta", "italian", "quick", "dinner"],
    "created_by": "chef@example.com"
}

recipe_id = manager.add_recipe(recipe)
```

Suchen und Filtern:

```
results = manager.search_recipes("pasta")

italian_easy = manager.filter_recipes(
    cuisine="Italian",
    difficulty="easy"
)

quick = manager.get_quick_recipes(max_minutes=30)

vegetarian = manager.filter_recipes(tags=["vegetarian", "healthy"])
```

Bewertungen:

```
manager.add_rating(
    recipe_id=recipe_id,
    user="alice@example.com",
    score=5,
    comment="Best carbonara I've ever made!"
)

avg = manager.get_average_rating(recipe_id)
print(f"Average rating: {avg:.1f}/5")

top = manager.get_top_rated(limit=5)
```

Einkaufsliste generieren:

```
recipe_ids = [recipe1_id, recipe2_id, recipe3_id]

multipliers = {
    recipe1_id: 2.0,
    recipe2_id: 0.5,
    recipe3_id: 1.0
}

shopping_list = manager.generate_shopping_list(recipe_ids, multipliers)

for category, items in shopping_list.items():
    print(f"\n{category.upper()}:")
    for item in items:
        print(f"  - {item['name']}: {item['amount']}{item['unit']}")
```

Document Model Vorteile

1. Natürliche Verschachtelung: - Ingredients als Array - Steps mit Nummern und Dauer - Nutrition als Sub-Dokument - Ratings mit User-Comments

2. Flexibilität: - Optionale Felder (z.B. `nutrition` kann fehlen) - Unterschiedliche Ingredient-Properties - Variable Anzahl von Steps

3. Einfache Queries: - Alles in einem Dokument - Keine JOINs notwendig - Aggregation über Arrays

7.5 Migration von MongoDB

Viele Projekte migrieren von MongoDB zu ThemisDB für ACID-Garantien.

Schema Mapping

MongoDB:

```
db.articles.insert({
    _id: ObjectId("..."),
    title: "Hello World",
    content: "...",
    tags: ["intro", "tutorial"]
})
```

ThemisDB:

```
db.documents.insert("articles", {
    "id": "uuid",  # UUID statt ObjectId
    "title": "Hello World",
    "content": "...",
    "tags": ["intro", "tutorial"]
})
```

Migration Script

```
from pymongo import MongoClient
from themisdb import ThemisDB

mongo = MongoClient("mongodb://localhost:27017")
themis = ThemisDB("migrated.db")

mongo_db = mongo["myapp"]
collection = mongo_db["articles"]

for doc in collection.find():
    # Convert ObjectId to string
    doc["id"] = str(doc.pop("_id"))

    # Insert into ThemisDB
    themis.documents.insert("articles", doc)

print("Migration complete!")
```

API Differences

MongoDB	ThemisDB	Note
`insert_one()`	`documents.insert()`	Single insert
`find()`	`query()`	Use AQL
`update_one()`	`documents.update()`	Partial update
`aggregate()`	`query()`	AQL GROUP BY
`$inc`	`increment()`	Atomic increment
`$push`	`array_append()`	Array operation

Vorteile nach Migration

1. ACID-Transaktionen:

```
with themis.transaction():
    themis.documents.update("articles", id1, {...})
    themis.documents.update("comments", id2, {...})
```

2. AQL-Queries:

```
results = collection.aggregate([
    {"$match": {"status": "published"}},
    {"$group": {"_id": "$author", "count": {"$sum": 1}}},
    {"$sort": {"count": -1}}
])

results = themis.query("""
    FOR article IN articles
      FILTER article.status == 'published'
      COLLECT author = article.author
      AGGREGATE count = COUNT()
      SORT count DESC
      RETURN {author, count}
""")
```

3. Multi-Model:

```
results = themis.query("""
    FOR article IN articles
      LET comment_count = (
        FOR comment IN comments
          FILTER comment.article_id == article.id
          RETURN 1
      )
      LET avg_rating = (
        FOR rating IN ratings
          FILTER rating.article_id == article.id
          COLLECT AGGREGATE avg_score = AVG(rating.score)
          RETURN avg_score
      )
      RETURN {
        title: article.title,
        comment_count: LENGTH(comment_count),
        rating: avg_rating[0]
      }
""")
```

7.6 Best Practices

1. Schema Design

DO: Embed Related Data

```
{
    "title": "...",
    "content": "...",
    "comments": [
        {"user": "alice", "text": "..."},
        {"user": "bob", "text": "..."}
    ]
}
```

× DON'T: Separate wenn oft zusammen gelesen

```
{"id": 1, "title": "..."}

{"article_id": 1, "user": "alice", "text": "..."}
```

2. Array-Größe begrenzen

Problem: Arrays können unbegrenzt wachsen

```
{
    "title": "Popular Article",
    "comments": [...]  # Kann zu groß werden!
}
```

Lösung: Separiere bei großen Arrays

```
{
    "title": "Popular Article",
    "recent_comments": [...][:10],  # Nur letzte 10
    "comment_count": 1523
}
```

3. Versionierung

Schema-Version im Dokument:

```
{
    "_schema_version": 2,
    "title": "...",
    "content": "...",
    # v2 fields
    "metadata": {...}
}

def load_article(doc):
    version = doc.get("_schema_version", 1)

    if version == 1:
        # Upgrade v1 → v2
        doc["metadata"] = {"created_at": doc.get("created_at")}
        doc["_schema_version"] = 2

    return doc
```

4. Index-Strategie

Indexiere häufige Queries:

```
CREATE INDEX idx_fulltext ON articles USING FULLTEXT(title, content)

CREATE INDEX idx_filters ON articles(status, category, author)

CREATE INDEX idx_tags ON articles(tags)
```

5. Partial Updates

Effizient: Nur ändern was nötig

```
db.documents.update(
    collection="articles",
    document_id=article_id,
    updates={"metadata.views": db.increment(1)}
)

article = db.documents.get("articles", article_id)
article["metadata"]["views"] += 1
db.documents.update("articles", article_id, article)  # Ineffizient!
```

7.7 Performance-Tipps

1. Embed vs. Reference

Embed (einbetten): - Daten werden immer zusammen gelesen - Eingebettete Daten ändern sich selten - Größe ist begrenzt

Reference (referenzieren): - Daten werden unabhängig gelesen - Daten werden häufig aktualisiert - Eingebettetes Array wird zu groß

2. Index-Nutzung

```
-- Query mit Index
FOR article IN articles
  FILTER article.status == 'published'  -- Index genutzt
     AND article.category == 'tech'
  SORT article.published_at DESC
  RETURN article
```

Ohne Index (langsam!):

```
FOR article IN articles
  FILTER article.metadata.custom_field == 'value'  -- KEIN Index!
  RETURN article
```

3. Projection

Nur benötigte Felder laden:

```
results = db.query("""
    FOR article IN articles
      FILTER article.status == 'published'
      RETURN {title: article.title, author: article.author}
""")

results = db.query("""
    FOR article IN articles
      FILTER article.status == 'published'
      RETURN article
""")
```

4. Batch-Operations

```
articles = [...]  # Liste von Dokumenten
for article in articles:
    db.documents.insert("articles", article)

with db.transaction():
    for article in articles:
        db.documents.insert("articles", article)
```

7.8 Übungen

Übung 1: Portfolio Website

Erstelle ein Dokumenten-System für eine Portfolio-Website: - Projects mit Screenshots (URLs) - Skills mit Proficiency-Levels - Blog-Posts mit Code-Examples - Contact-Formular History

Übung 2: Product Catalog

Implementiere einen Produkt-Katalog: - Produkte mit variablen Attributen (Electronics: Screen-Size, Clothing: Sizes) - Kategorien mit Hierarchie - Reviews mit Images - Price-History

Übung 3: Event Management

Event-System mit: - Events mit Teilnehmern - Schedule mit Sessions - Venue-Details - Feedback-Sammlung

7.9 Zusammenfassung

Document Model in ThemisDB: - Schema-less für flexible Datenstrukturen - Nested Objects und Arrays - JSON-Operationen (Path-Queries, Updates) - ACID-Transaktionen (vs. MongoDB) - Fulltext-Suche - Migration von MongoDB einfach

Wann nutzen: - Content-Management (Blog, Wiki, CMS) - Produkt-Kataloge mit variablen Attributen - User-Profiles mit optionalen Feldern - Config-Dateien mit Verschachtelung - Rapid Prototyping ohne Schema-Definition

Nächste Schritte: - Kapitel 8: Vektor-Suche für Semantic Search - Kapitel 9: Time-Series für IoT und Monitoring - Kapitel 10: Geo-Daten für Location-Based Apps

---

Hands-on Examples: - `examples/11_blog_wiki` - Vollständiges CMS - `examples/13_recipe_manager` - Recipe-System - Beide mit Schema-Evolution und Nested Data

---

Kapitel 9: Kapitel 8: Vektor-Suche und Similarity Search

8.1 Einführung in Vektor-Suche

Das Problem mit traditionellen Suchen

Stellen Sie sich vor, Sie suchen in einer Produktdatenbank nach "bequeme Laufschuhe für den Marathon". Eine traditionelle Keyword-Suche würde nur Produkte finden, die exakt diese Wörter enthalten. Aber was ist mit:

- "Marathon-Schuhe mit optimaler Dämpfung"
- "Komfortable Running-Shoes für lange Strecken"
- "Wettkampf-Laufschuh mit Pronationsstütze"

Diese Produkte sind semantisch relevant, werden aber von Keyword-Suchen oft nicht gefunden. **Vektor-Suche** löst dieses Problem durch semantisches Verständnis.

Was sind Embeddings?

Ein **Embedding** ist eine hochdimensionale Vektorrepräsentation von Text, Bildern oder anderen Daten. Ähnliche Konzepte haben ähnliche Vektoren:

```
"Laufschuh"       → [0.23, -0.15, 0.89, ..., 0.34]  # 384 Werte
"Running Shoe"    → [0.21, -0.14, 0.91, ..., 0.36]  # Sehr ähnlich!
"Fahrrad"         → [-0.67, 0.42, -0.12, ..., 0.78] # Komplett anders
```

Kosinus-Ähnlichkeit misst, wie ähnlich zwei Vektoren sind (0 = keine Ähnlichkeit, 1 = identisch).

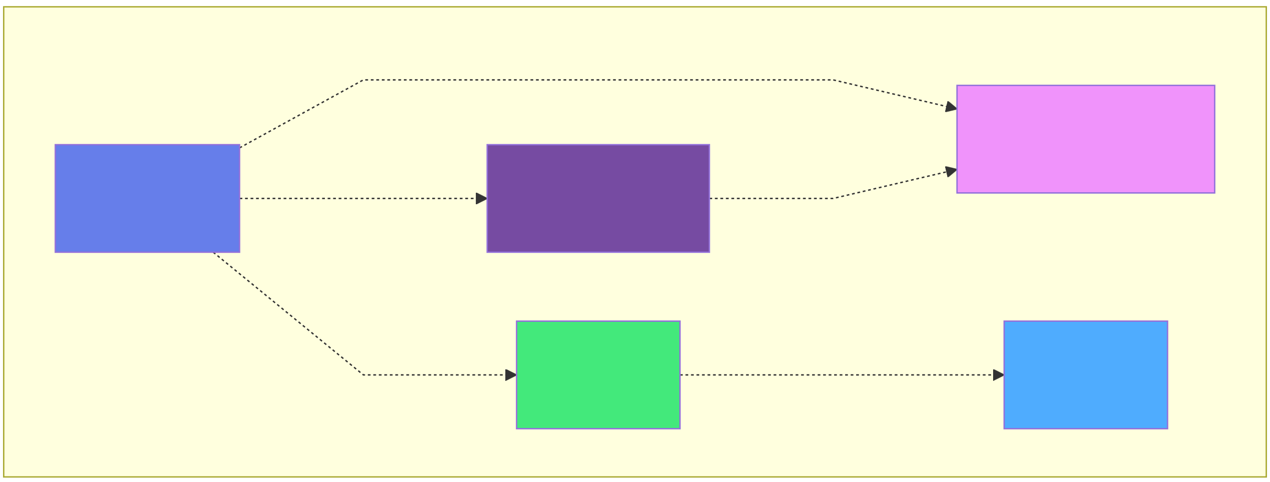


Abb. 24: Architekturdiagramm: Komplett anders

Vector Search vs. Fulltext Search

Feature	Fulltext Search	Vector Search
Matching	Exakte Wörter	Semantische Bedeutung
Sprachen	Sprach-abhängig	Sprach-übergreifend
Synonyme	× Muss konfiguriert werden	Automatisch
Schreibfehler	× Keine Treffer	Oft tolerant
Performance	Sehr schnell	Schnell (mit HNSW)
Use Cases	Dokumente, Logs	Empfehlungen, Q&A, Similarity

Best Practice: Hybrid Search kombiniert beide Ansätze.

8.2 Vector Model in ThemisDB

Schema und Index

ThemisDB speichert Vektoren als native `VECTOR` Spalten und nutzt **HNSW** (Hierarchical Navigable Small World) für effiziente Suchen:

```
-- Dokumente mit Embeddings
CREATE TABLE documents (
    id UUID PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    embedding VECTOR(384),  -- 384-dimensionaler Vektor
    metadata JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- HNSW-Index für schnelle Similarity Search
CREATE INDEX idx_doc_embedding ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

HNSW-Parameter: - `m`: Anzahl Verbindungen (höher = bessere Qualität, langsamer Build) - `ef_construction`: Exploration während Index-Build (höher = bessere Qualität) - `vector_cosine_ops`: Kosinus-Distanz (alternativ: L2, inner product)

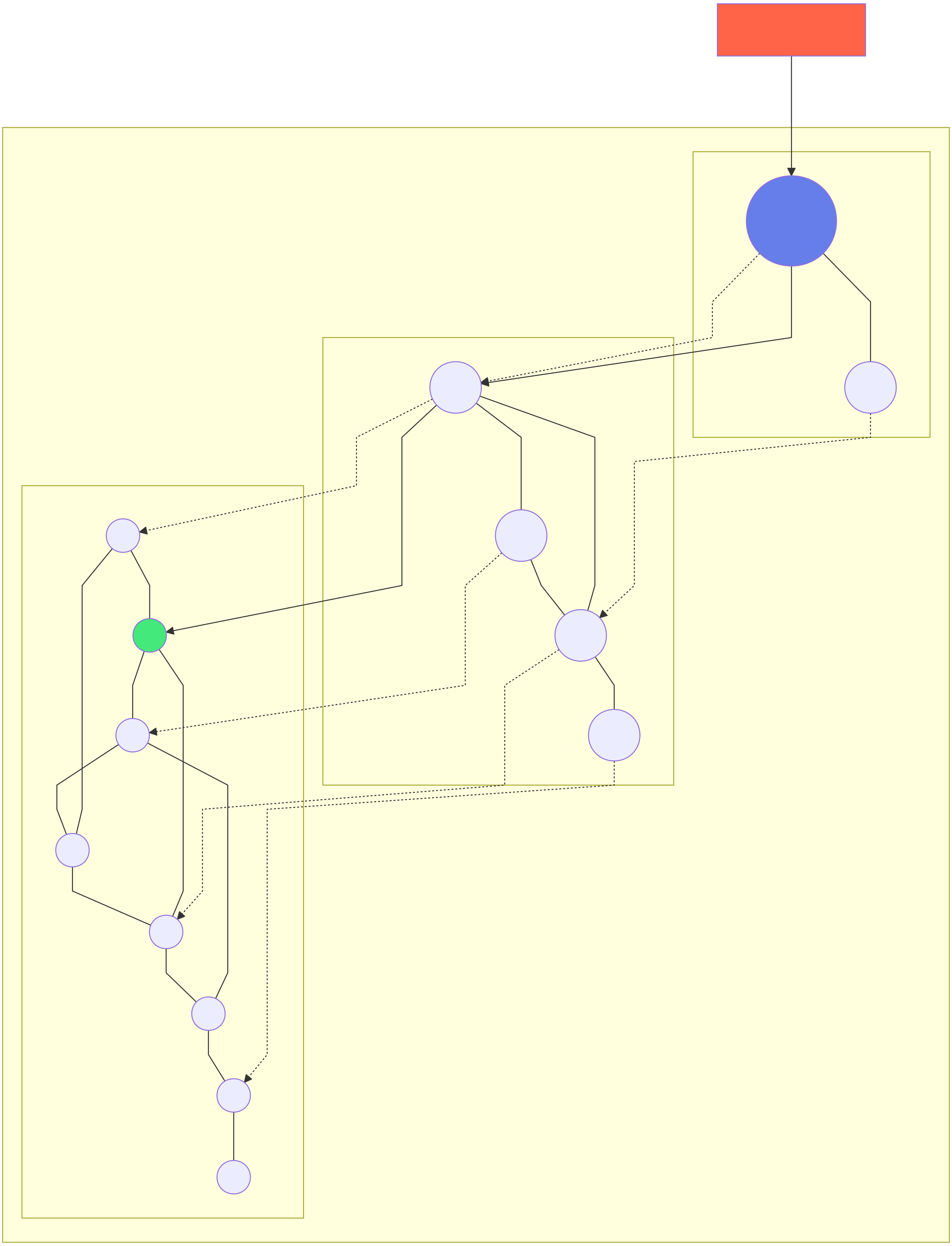


Abb. 25: Architekturdiagramm (Kapitel 8: Vektor-Suche und Similarity Search)

Similarity Search Query

```
-- Finde ähnliche Dokumente zur Query
SELECT id, title,
       1 - (embedding <=> :query_embedding) AS similarity
FROM documents
ORDER BY embedding <=> :query_embedding
LIMIT 10;
```

Operators: - `<=>` : Kosinus-Distanz (0 = identisch) - `<->` : L2 (Euklidische) Distanz - `<#>` : Inner Product (negative Distanz)

Hybrid Search: Vector + Fulltext

Kombiniere semantische und Keyword-Suche:

```
WITH vector_results AS (
    SELECT id, 1 - (embedding <=> :query_embedding) AS vec_score
    FROM documents
    ORDER BY embedding <=> :query_embedding
    LIMIT 50
),
fulltext_results AS (
    SELECT id, ts_rank(to_tsvector('german', content),
                       plainto_tsquery('german', :query_text)) AS text_score
    FROM documents
    WHERE to_tsvector('german', content) @@ plainto_tsquery('german', :query_text)
    LIMIT 50
)
SELECT d.*,
       COALESCE(v.vec_score, 0) * 0.7 +
       COALESCE(f.text_score, 0) * 0.3 AS combined_score
FROM documents d
LEFT JOIN vector_results v ON d.id = v.id
LEFT JOIN fulltext_results f ON d.id = f.id
WHERE v.id IS NOT NULL OR f.id IS NOT NULL
ORDER BY combined_score DESC
LIMIT 20;
```

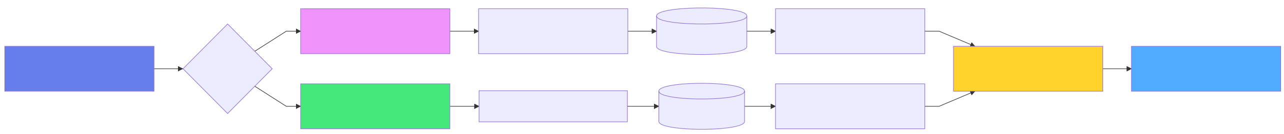


Abb. 26: Ablaufdiagramm (Kapitel 8: Vektor-Suche und Similarity Search)

8.3 Example: Dokumenten-Suche (RAG System)

Wir bauen ein **RAG-System** (Retrieval Augmented Generation) für semantische Dokumentensuche. Der Use Case:

- Dokumente hochladen (PDF, TXT, DOCX)

- Automatische Embedding-Generierung
- Semantische Suche: "Wie funktioniert MVCC?"
- Context für LLMs bereitstellen

Datenmodell

```
from dataclasses import dataclass
from datetime import datetime
from typing import List, Optional
import uuid

@dataclass
class Document:
    id: str
    title: str
    content: str
    embedding: Optional[List[float]]
    metadata: dict
    created_at: datetime
    collection: str = "default"

    @staticmethod
    def create(title: str, content: str, metadata: dict = None, collection: str = "default"):
        return Document(
            id=str(uuid.uuid4()),
            title=title,
            content=content,
            embedding=None,  # Wird später generiert
            metadata=metadata or {},
            created_at=datetime.now(),
            collection=collection
        )

@dataclass
class SearchResult:
    document: Document
    similarity: float
    rank: int
```

Embedding-Generierung

Wir verwenden **sentence-transformers** für deutsche und englische Texte:

```
from sentence_transformers import SentenceTransformer
import numpy as np

class ThemisVectorClient:
    def __init__(self, connection_string: str):
        self.conn = connect(connection_string)
        # Multilingual Model (384D)
        self.model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

    def generate_embedding(self, text: str) -> List[float]:
        """Generiert 384D Embedding für Text"""
        embedding = self.model.encode(text, convert_to_numpy=True)
        return embedding.tolist()

    def chunk_text(self, text: str, max_words: int = 200) -> List[str]:
        """Teilt langen Text in Chunks für bessere Embeddings"""
        words = text.split()
        chunks = []
        for i in range(0, len(words), max_words):
            chunk = ' '.join(words[i:i + max_words])
            chunks.append(chunk)
        return chunks

    def add_document(self, doc: Document, chunk_size: int = 200):
        """Fügt Dokument mit Embeddings hinzu"""
        # Lange Dokumente in Chunks aufteilen
        chunks = self.chunk_text(doc.content, max_words=chunk_size)

        for i, chunk in enumerate(chunks):
            chunk_id = f"{doc.id}_chunk_{i}"
            embedding = self.generate_embedding(chunk)

            cursor = self.conn.cursor()
            cursor.execute("""
                INSERT INTO documents (id, title, content, embedding, metadata, collection)
                VALUES (?, ?, ?, ?, ?, ?)
            """, (chunk_id, f"{doc.title} (Teil {i+1})", chunk,
                  embedding, json.dumps(doc.metadata), doc.collection))
            self.conn.commit()

        print(f"  Dokument '{doc.title}' in {len(chunks)} Chunks gespeichert")
```

```
    def search(self, query: str, collection: str = None,
               limit: int = 10, min_similarity: float = 0.5) -> List[SearchResult]:
        """Semantische Suche"""
        query_embedding = self.generate_embedding(query)

        cursor = self.conn.cursor()

        # AQL mit optionalem Collection-Filter
        sql = """
            SELECT id, title, content, embedding, metadata,
                   1 - (embedding <=> ?) AS similarity
            FROM documents
            WHERE (? IS NULL OR collection = ?)
              AND 1 - (embedding <=> ?) >= ?
            ORDER BY embedding <=> ?
            LIMIT ?
        """

        cursor.execute(sql, (
            query_embedding, collection, collection,
            query_embedding, min_similarity,
            query_embedding, limit
        ))

        results = []
        for i, row in enumerate(cursor.fetchall(), start=1):
            doc = Document(
                id=row[0], title=row[1], content=row[2],
                embedding=row[3], metadata=json.loads(row[4]),
                created_at=datetime.now(), collection=collection or "default"
            )
            results.append(SearchResult(doc, similarity=row[5], rank=i))

        return results

    def hybrid_search(self, query: str, collection: str = None,
                      limit: int = 10, alpha: float = 0.7) -> List[SearchResult]:
        """Hybrid Search: Vector (70%) + Fulltext (30%)"""
        query_embedding = self.generate_embedding(query)

        cursor = self.conn.cursor()

        sql = """
            WITH vector_results AS (
                SELECT id, 1 - (embedding <=> ?) AS vec_score
                FROM documents
                WHERE ? IS NULL OR collection = ?
                ORDER BY embedding <=> ?
                LIMIT 50
            ),
            fulltext_results AS (
                SELECT id, ts_rank(to_tsvector('german', content),
                                   plainto_tsquery('german', ?)) AS text_score
                FROM documents
```

```
            WHERE (? IS NULL OR collection = ?)
              AND to_tsvector('german', content) @@ plainto_tsquery('german', ?)
            LIMIT 50
        )
        SELECT d.id, d.title, d.content, d.embedding, d.metadata,
               COALESCE(v.vec_score, 0) * ? +
               COALESCE(f.text_score, 0) * (1 - ?) AS combined_score
        FROM documents d
        LEFT JOIN vector_results v ON d.id = v.id
        LEFT JOIN fulltext_results f ON d.id = f.id
        WHERE v.id IS NOT NULL OR f.id IS NOT NULL
        ORDER BY combined_score DESC
        LIMIT ?
    """

    cursor.execute(sql, (
        query_embedding, collection, collection, query_embedding,
        query, collection, collection, query,
        alpha, alpha, limit
    ))

    results = []
    for i, row in enumerate(cursor.fetchall(), start=1):
        doc = Document(
            id=row[0], title=row[1], content=row[2],
            embedding=row[3], metadata=json.loads(row[4]),
            created_at=datetime.now(), collection=collection or "default"
        )
        results.append(SearchResult(doc, similarity=row[5], rank=i))

    return results
```

RAG-Workflow: Context für LLMs

```
    def retrieve_context(self, question: str, max_chunks: int = 5,
                        collection: str = None) -> str:
        """Holt relevanten Context für RAG"""
        results = self.search(question, collection=collection,
                             limit=max_chunks, min_similarity=0.6)

        if not results:
            return "Keine relevanten Dokumente gefunden."

        # Context zusammenbauen
        context_parts = []
        for result in results:
            context_parts.append(
                f"[{result.document.title}] (Similarity: {result.similarity:.2f})\n"
                f"{result.document.content}\n"
            )

        return "\n---\n".join(context_parts)

    def answer_question(self, question: str, collection: str = None) -> dict:
        """RAG: Frage + Context → Antwort (Mock ohne LLM)"""
        context = self.retrieve_context(question, collection=collection)

        # In Produktion: context an GPT/Claude/etc. senden
        # response = openai.ChatCompletion.create(...)

        return {
            "question": question,
            "context": context,
            "answer": "⚠ Mock-Antwort: LLM-Integration not implemented",
            "sources": len([r for r in self.search(question, collection, limit=5)])
        }
```

Hauptprogramm

```
from themis_client import ThemisVectorClient
from models import Document

def main():
    client = ThemisVectorClient("themisdb://localhost:9091")

    # Setup
    client.setup_schema()

    # Demo-Dokumente hinzufügen
    docs = [
        Document.create(
            title="ThemisDB MVCC Architektur",
            content="""
            ThemisDB verwendet Multi-Version Concurrency Control (MVCC) für
            optimistische Transaktionen. Jede Zeile hat mehrere Versionen mit
            Timestamps. Transaktionen arbeiten auf Snapshots, wodurch Lese-Operationen
            niemals blockiert werden. Write-Konflikte werden zur Commit-Zeit erkannt.
            """,
            metadata={"category": "architecture", "author": "System"},
            collection="docs"
        ),
        Document.create(
            title="Vector Search Best Practices",
            content="""
            Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe
            mit m=16 und ef_construction=200. 2) Chunken Sie lange Dokumente in
            200-300 Wörter Abschnitte. 3) Kombinieren Sie Vector mit Fulltext Search
            (Hybrid Search). 4) Normalisieren Sie Vektoren für Kosinus-Distanz.
            """,
            metadata={"category": "best_practices", "author": "System"},
            collection="docs"
        ),
        Document.create(
            title="Graph-Algorithmen in ThemisDB",
            content="""
            ThemisDB bietet native Graph-Algorithmen: Dijkstra für kürzeste Pfade,
            PageRank für Wichtigkeit, Community Detection für Gruppen. Graph-Traversierung
            unterstützt BFS, DFS und Pattern Matching. Beispiel: MATCH (a)-[:FRIEND]->(b)
            für Freundschaften.
            """,
            metadata={"category": "graph", "author": "System"},
            collection="docs"
        )
    ]

    print("  Füge Dokumente hinzu...")
    for doc in docs:
        client.add_document(doc, chunk_size=100)

    print("\n" + "="*60)

    # Test 1: Semantic Search
    print("\n  Test 1: Semantic Search")
```

```
    print("Query: 'Wie funktioniert MVCC in ThemisDB?'")
    results = client.search(
        "Wie funktioniert MVCC in ThemisDB?",
        collection="docs",
        limit=3
    )

    for result in results:
        print(f"\n  [{result.rank}] {result.document.title}")
        print(f"      Similarity: {result.similarity:.3f}")
        print(f"      {result.document.content[:100]}...")

    # Test 2: Hybrid Search
    print("\n\n  Test 2: Hybrid Search (Vector + Fulltext)")
    print("Query: 'Graph shortest path algorithms'")
    results = client.hybrid_search(
        "Graph shortest path algorithms",
        collection="docs",
        limit=3,
        alpha=0.6  # 60% Vector, 40% Fulltext
    )

    for result in results:
        print(f"\n  [{result.rank}] {result.document.title}")
        print(f"      Score: {result.similarity:.3f}")
        print(f"      {result.document.content[:100]}...")

    # Test 3: RAG Context Retrieval
    print("\n\n  Test 3: RAG Context Retrieval")
    print("Question: 'Was sind Best Practices für Vector Indexes?'")
    answer = client.answer_question(
        "Was sind Best Practices für Vector Indexes?",
        collection="docs"
    )

    print(f"\n    Context ({answer['sources']} Quellen gefunden):")
    print(f"  {answer['context'][:300]}...")
    print(f"\n    Antwort: {answer['answer']}")

    # Test 4: Collection-basierte Suche
    print("\n\n  Test 4: Suche in spezifischer Collection")
    print("Query: 'architecture' in collection 'docs'")
    results = client.search(
        "architecture",
        collection="docs",
        limit=5,
        min_similarity=0.3
    )

    print(f"  Gefunden: {len(results)} Dokumente")
    for result in results:
        print(f"  - {result.document.title} (Similarity: {result.similarity:.2f})")
```

```
if __name__ == "__main__":
    main()
```

Output

```
  Füge Dokumente hinzu...
  Dokument 'ThemisDB MVCC Architektur' in 1 Chunks gespeichert
  Dokument 'Vector Search Best Practices' in 1 Chunks gespeichert
  Dokument 'Graph-Algorithmen in ThemisDB' in 1 Chunks gespeichert

============================================================

  Test 1: Semantic Search
Query: 'Wie funktioniert MVCC in ThemisDB?'

  [1] ThemisDB MVCC Architektur
      Similarity: 0.847
      ThemisDB verwendet Multi-Version Concurrency Control (MVCC) für optimistische Trans...

  [2] Vector Search Best Practices
      Similarity: 0.512
      Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe mit m=16 und e...

  Test 2: Hybrid Search (Vector + Fulltext)
Query: 'Graph shortest path algorithms'

  [1] Graph-Algorithmen in ThemisDB
      Score: 0.823
      ThemisDB bietet native Graph-Algorithmen: Dijkstra für kürzeste Pfade, PageRank fü...

  [2] Vector Search Best Practices
      Score: 0.387
      Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe mit m=16 und e...

  Test 3: RAG Context Retrieval
Question: 'Was sind Best Practices für Vector Indexes?'

    Context (3 Quellen gefunden):
  [Vector Search Best Practices] (Similarity: 0.89)
  Für optimale Vector Search Performance: 1) Verwenden Sie HNSW-Indexe mit m=16 und
  ef_construction=200. 2) Chunken Sie lange Dokumente in 200-300 Wörter Abschnitte...

    Antwort: ⚠ Mock-Antwort: LLM-Integration not implemented

  Test 4: Suche in spezifischer Collection
Query: 'architecture' in collection 'docs'
  Gefunden: 2 Dokumente
  - ThemisDB MVCC Architektur (Similarity: 0.78)
  - Vector Search Best Practices (Similarity: 0.43)
```

8.4 Example: E-Commerce Produktkatalog mit Vector Search

Ein E-Commerce-Shop nutzt Vector Search für "Ähnliche Produkte" und visuelle Suche. Der Use Case:

- Produktdatenbank mit Beschreibungen und Bildern
- "Finde ähnliche Produkte" (Text-Embedding)
- Visuelle Suche (Bild-Embedding)
- Personalisierte Empfehlungen
- Filter + Vector Search kombiniert

Datenmodell

```python
from dataclasses import dataclass
from typing import List, Optional
import uuid

@dataclass
class Product:
    id: str
    name: str
    description: str
    category: str
    brand: str
    price: float
    text_embedding: Optional[List[float]]  # Text-basiert
    image_embedding: Optional[List[float]]  # Bild-basiert (CLIP)
    tags: List[str]
    rating: float
    stock: int

    @staticmethod
    def create(name: str, description: str, category: str,
               brand: str, price: float, tags: List[str] = None):
        return Product(
            id=str(uuid.uuid4()),
            name=name,
            description=description,
            category=category,
            brand=brand,
            price=price,
            text_embedding=None,
            image_embedding=None,
            tags=tags or [],
            rating=0.0,
            stock=100
        )
```

ThemisDB Schema

```sql
CREATE TABLE products (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    description TEXT NOT NULL,
    category TEXT NOT NULL,
    brand TEXT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    text_embedding VECTOR(384),      -- Text-Embedding
    image_embedding VECTOR(512),     -- Bild-Embedding (CLIP)
    tags TEXT[] NOT NULL,
    rating DECIMAL(3,2) DEFAULT 0,
    stock INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Indexes für Hybrid Search
CREATE INDEX idx_product_text_emb ON products
USING hnsw (text_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_product_image_emb ON products
USING hnsw (image_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

CREATE INDEX idx_product_fulltext ON products
USING gin(to_tsvector('german', name || ' ' || description));

CREATE INDEX idx_product_category ON products(category);
CREATE INDEX idx_product_price ON products(price);
```

Ähnliche Produkte finden

```
class ProductCatalogClient:
    def __init__(self, connection_string: str):
        self.conn = connect(connection_string)
        self.text_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

    def add_product(self, product: Product):
        """Fügt Produkt mit Text-Embedding hinzu"""
        # Text-Embedding aus Name + Description
        text = f"{product.name}. {product.description}"
        product.text_embedding = self.text_model.encode(text).tolist()

        cursor = self.conn.cursor()
        cursor.execute("""
            INSERT INTO products
            (id, name, description, category, brand, price,
             text_embedding, tags, stock)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (product.id, product.name, product.description,
              product.category, product.brand, product.price,
              product.text_embedding, product.tags, product.stock))
        self.conn.commit()

        print(f"  Produkt '{product.name}' hinzugefügt")

    def find_similar_products(self, product_id: str, limit: int = 5) -> List:
        """Findet ähnliche Produkte basierend auf Text-Embedding"""
        cursor = self.conn.cursor()

        # Hole Embedding des Referenz-Produkts
        cursor.execute("""
            FOR product IN products
              FILTER product.id == @product_id
              LIMIT 1
              RETURN product.text_embedding
        """, {"product_id": product_id})
        ref_embedding = cursor.fetchone()[0]

        # Finde ähnliche Produkte
        cursor.execute("""
            SELECT id, name, description, price, category,
                   1 - (text_embedding <=> ?) AS similarity
            FROM products
            WHERE id != ?
            ORDER BY text_embedding <=> ?
            LIMIT ?
        """, (ref_embedding, product_id, ref_embedding, limit))

        return cursor.fetchall()

    def search_with_filters(self, query: str, category: str = None,
                           min_price: float = None, max_price: float = None,
                           limit: int = 20) -> List:
        """Hybrid Search mit Filtern"""
        query_embedding = self.text_model.encode(query).tolist()
```

```
# Dynamisches AQL mit optionalen Filtern
filters = ["1=1"]
params = [query_embedding, query_embedding]

if category:
    filters.append("category = ?")
    params.append(category)
if min_price is not None:
    filters.append("price >= ?")
    params.append(min_price)
if max_price is not None:
    filters.append("price <= ?")
    params.append(max_price)

params.extend([query, limit])

sql = f"""
    WITH vector_scores AS (
        SELECT id, 1 - (text_embedding <=> ?) AS vec_score
        FROM products
        WHERE {' AND '.join(filters)}
        ORDER BY text_embedding <=> ?
        LIMIT 100
    )
    SELECT p.id, p.name, p.description, p.price, p.category,
           v.vec_score
    FROM products p
    JOIN vector_scores v ON p.id = v.id
    WHERE to_tsvector('german', p.name || ' ' || p.description)
          @@ plainto_tsquery('german', ?)
    ORDER BY v.vec_score DESC
    LIMIT ?
"""

cursor = self.conn.cursor()
cursor.execute(sql, params)
return cursor.fetchall()
```

Hauptprogramm

```
from themis_client import ProductCatalogClient
from models import Product

def main():
    client = ProductCatalogClient("themisdb://localhost:9091")

    # Demo-Produkte
    products = [
        Product.create(
            name="Nike Air Zoom Pegasus 40",
            description="Leichter Laufschuh für lange Strecken mit React-Dämpfung",
            category="Laufschuhe",
            brand="Nike",
            price=139.99,
            tags=["running", "cushion", "marathon"]
        ),
        Product.create(
            name="Adidas Ultraboost 23",
            description="Energierückgebender Running-Schuh mit Boost-Technologie",
            category="Laufschuhe",
            brand="Adidas",
            price=189.99,
            tags=["running", "boost", "comfort"]
        ),
        Product.create(
            name="Asics Gel-Nimbus 25",
            description="Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken",
            category="Laufschuhe",
            brand="Asics",
            price=169.99,
            tags=["running", "stability", "marathon"]
        ),
        Product.create(
            name="New Balance Fresh Foam X 1080v13",
            description="Premium Laufschuh mit Fresh Foam für maximalen Komfort",
            category="Laufschuhe",
            brand="New Balance",
            price=179.99,
            tags=["running", "comfort", "cushion"]
        ),
        Product.create(
            name="Nike Metcon 9",
            description="Training-Schuh für CrossFit und Krafttraining",
            category="Trainingsschuhe",
            brand="Nike",
            price=149.99,
            tags=["training", "crossfit", "gym"]
        ),
        Product.create(
            name="Adidas Terrex Free Hiker",
            description="Wanderschuh für leichte Trails und Bergtouren",
            category="Wanderschuhe",
            brand="Adidas",
            price=199.99,
```

```
            tags=["hiking", "outdoor", "trail"]
        )
    ]

    print("  Füge Produkte hinzu...")
    for product in products:
        client.add_product(product)

    print("\n" + "="*60)

    # Test 1: Ähnliche Produkte finden
    print("\n  Test 1: Ähnliche Produkte")
    reference_product = products[0]  # Nike Pegasus
    print(f"Referenz: {reference_product.name}")

    similar = client.find_similar_products(reference_product.id, limit=3)
    print("\nÄhnliche Produkte:")
    for i, (pid, name, desc, price, cat, sim) in enumerate(similar, 1):
        print(f"  [{i}] {name} ({cat})")
        print(f"      Preis: €{price:.2f} | Similarity: {sim:.3f}")
        print(f"      {desc[:80]}...")

    # Test 2: Semantische Suche mit Filtern
    print("\n\n  Test 2: Suche mit Filtern")
    print("Query: 'bequeme Schuhe für Marathon'")
    print("Filter: Kategorie='Laufschuhe', Preis < €180")

    results = client.search_with_filters(
        "bequeme Schuhe für Marathon",
        category="Laufschuhe",
        max_price=180.0,
        limit=5
    )

    print(f"\nErgebnisse ({len(results)}):")
    for i, (pid, name, desc, price, cat, score) in enumerate(results, 1):
        print(f"  [{i}] {name}")
        print(f"      Preis: €{price:.2f} | Score: {score:.3f}")
        print(f"      {desc[:80]}...")

    # Test 3: Cross-Category Search
    print("\n\n  Test 3: Cross-Category Similarity")
    print("Query: 'Schuhe für lange Distanzen'")

    results = client.search_with_filters(
        "Schuhe für lange Distanzen",
        limit=5
    )

    print(f"\nErgebnisse ({len(results)}):")
    for i, (pid, name, desc, price, cat, score) in enumerate(results, 1):
        print(f"  [{i}] {name} ({cat})")
        print(f"      Preis: €{price:.2f} | Score: {score:.3f}")
```

```
if __name__ == "__main__":
    main()
```

Output

```
  Füge Produkte hinzu...
  Produkt 'Nike Air Zoom Pegasus 40' hinzugefügt
  Produkt 'Adidas Ultraboost 23' hinzugefügt
  Produkt 'Asics Gel-Nimbus 25' hinzugefügt
  Produkt 'New Balance Fresh Foam X 1080v13' hinzugefügt
  Produkt 'Nike Metcon 9' hinzugefügt
  Produkt 'Adidas Terrex Free Hiker' hinzugefügt

============================================================

  Test 1: Ähnliche Produkte
Referenz: Nike Air Zoom Pegasus 40

Ähnliche Produkte:
  [1] Asics Gel-Nimbus 25 (Laufschuhe)
      Preis: €169.99 | Similarity: 0.891
      Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken...

  [2] New Balance Fresh Foam X 1080v13 (Laufschuhe)
      Preis: €179.99 | Similarity: 0.867
      Premium Laufschuh mit Fresh Foam für maximalen Komfort...

  [3] Adidas Ultraboost 23 (Laufschuhe)
      Preis: €189.99 | Similarity: 0.843
      Energierückgebender Running-Schuh mit Boost-Technologie...

  Test 2: Suche mit Filtern
Query: 'bequeme Schuhe für Marathon'
Filter: Kategorie='Laufschuhe', Preis < €180

Ergebnisse (3):
  [1] Asics Gel-Nimbus 25
      Preis: €169.99 | Score: 0.912
      Stabiler Marathon-Laufschuh mit Gel-Dämpfung für Langstrecken...

  [2] New Balance Fresh Foam X 1080v13
      Preis: €179.99 | Score: 0.889
      Premium Laufschuh mit Fresh Foam für maximalen Komfort...

  [3] Nike Air Zoom Pegasus 40
      Preis: €139.99 | Score: 0.856
      Leichter Laufschuh für lange Strecken mit React-Dämpfung...

  Test 3: Cross-Category Similarity
Query: 'Schuhe für lange Distanzen'

Ergebnisse (5):
  [1] Asics Gel-Nimbus 25 (Laufschuhe)
      Preis: €169.99 | Score: 0.923
  [2] Nike Air Zoom Pegasus 40 (Laufschuhe)
      Preis: €139.99 | Score: 0.897
  [3] New Balance Fresh Foam X 1080v13 (Laufschuhe)
      Preis: €179.99 | Score: 0.876
  [4] Adidas Terrex Free Hiker (Wanderschuhe)
```

```
    Preis: €199.99 | Score: 0.734
[5] Adidas Ultraboost 23 (Laufschuhe)
    Preis: €189.99 | Score: 0.712
```

Beobachtungen: - Vector Search findet semantisch ähnliche Produkte, auch über Kategorien hinweg - Der Wanderschuh wird bei "lange Distanzen" gefunden (semantische Nähe!) - Filter reduzieren Ergebnisse effizient - Similarity Scores zeigen Relevanz deutlich

8.5 Embedding-Modelle und Integration

Beliebte Embedding-Modelle

Modell	Dimensionen	Sprachen	Use Case
paraphrase-multilingual-MiniLM-L12-v2	384	50+	Allgemein, schnell
all-MiniLM-L6-v2	384	EN	Schnell, gut für Englisch
all-mpnet-base-v2	768	EN	Beste Qualität (Englisch)
distiluse-base-multilingual-cased-v2	512	15+	Multilingual, mittel
CLIP (OpenAI)	512/768	Bild+Text	Visuelle Suche
text-embedding-ada-002 (OpenAI)	1536		API, sehr gut

OpenAI Embeddings Integration

```
import openai

class OpenAIEmbeddingClient:
    def __init__(self, api_key: str, connection_string: str):
        openai.api_key = api_key
        self.conn = connect(connection_string)

    def generate_embedding(self, text: str) -> List[float]:
        """OpenAI text-embedding-ada-002 (1536D)"""
        response = openai.Embedding.create(
            model="text-embedding-ada-002",
            input=text
        )
        return response['data'][0]['embedding']

    def add_document_openai(self, doc: Document):
        """Dokument mit OpenAI Embedding"""
        embedding = self.generate_embedding(doc.content)

        cursor = self.conn.cursor()
        cursor.execute("""
            INSERT INTO documents_openai
            (id, title, content, embedding, metadata)
            VALUES (?, ?, ?, ?, ?)
        """, (doc.id, doc.title, doc.content,
              embedding, json.dumps(doc.metadata)))
        self.conn.commit()
```

Schema für OpenAI Embeddings:

```
CREATE TABLE documents_openai (
    id UUID PRIMARY KEY,
    title TEXT,
    content TEXT,
    embedding VECTOR(1536),  -- OpenAI ada-002
    metadata JSONB
);

CREATE INDEX idx_docs_openai_emb ON documents_openai
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

Hugging Face Models

```
from transformers import AutoTokenizer, AutoModel
import torch

class HuggingFaceEmbeddingClient:
    def __init__(self, model_name: str = "sentence-transformers/all-MiniLM-L6-v2"):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name)

    def mean_pooling(self, model_output, attention_mask):
        """Mean Pooling für Sentence Embedding"""
        token_embeddings = model_output[0]
        input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).fl
        return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.clamp(input_mask_

    def generate_embedding(self, text: str) -> List[float]:
        """Generiert Embedding mit HuggingFace Model"""
        encoded_input = self.tokenizer(text, padding=True, truncation=True, return_tensors='p

        with torch.no_grad():
            model_output = self.model(**encoded_input)

        sentence_embeddings = self.mean_pooling(model_output, encoded_input['attention_mask']
        sentence_embeddings = torch.nn.functional.normalize(sentence_embeddings, p=2, dim=1)

        return sentence_embeddings[0].tolist()
```

8.6 Performance-Optimierung

HNSW Index Tuning

```
-- Standard (guter Balance)
CREATE INDEX idx_standard ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 200);

-- Schneller Build, niedrigere Qualität
CREATE INDEX idx_fast_build ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 8, ef_construction = 100);

-- Langsamer Build, höhere Qualität
CREATE INDEX idx_high_quality ON documents
USING hnsw (embedding vector_cosine_ops)
WITH (m = 32, ef_construction = 400);
```

Parameter-Effekte: - `m`: Anzahl Verbindungen pro Layer (mehr = besser, aber langsamer) - `ef_construction`: Exploration während Build (höher = besser, länger Build) - Faustregel: `m = 16` und `ef_construction = 200` für die meisten Use Cases

Query-Time Performance

```sql
-- Setze ef_search für Query-Zeit (höher = genauer, langsamer)
SET hnsw.ef_search = 100;  -- Standard: 40

-- Dann normale Query
SELECT id, 1 - (embedding <=> :query_vec) AS similarity
FROM documents
ORDER BY embedding <=> :query_vec
LIMIT 10;
```

Batch-Processing

```python
def add_documents_batch(self, documents: List[Document], batch_size: int = 100):
    """Batch-Insert für Performance"""
    for i in range(0, len(documents), batch_size):
        batch = documents[i:i + batch_size]

        # Embeddings für Batch generieren
        texts = [f"{doc.title}. {doc.content}" for doc in batch]
        embeddings = self.model.encode(texts, batch_size=batch_size)

        # Batch-Insert
        cursor = self.conn.cursor()
        for doc, embedding in zip(batch, embeddings):
            cursor.execute("""
                INSERT INTO documents (id, title, content, embedding, metadata)
                VALUES (?, ?, ?, ?, ?)
            """, (doc.id, doc.title, doc.content,
                  embedding.tolist(), json.dumps(doc.metadata)))

        self.conn.commit()
        print(f"  Batch {i//batch_size + 1}: {len(batch)} Dokumente")
```

8.7 Best Practices

1. Chunking-Strategie

Problem: Lange Dokumente führen zu schlechten Embeddings.

Lösung: Teilen Sie Dokumente in 200-300 Wörter Chunks:

```
def smart_chunk(text: str, max_words: int = 250, overlap: int = 50) -> List[str]:
    """Chunking mit Overlap für Context"""
    words = text.split()
    chunks = []

    for i in range(0, len(words), max_words - overlap):
        chunk = ' '.join(words[i:i + max_words])
        chunks.append(chunk)

    return chunks
```

2. Normalisierung

Normalisieren Sie Vektoren für Kosinus-Distanz:

```
import numpy as np

def normalize_vector(vec: List[float]) -> List[float]:
    """L2-Normalisierung"""
    arr = np.array(vec)
    norm = np.linalg.norm(arr)
    if norm == 0:
        return vec
    return (arr / norm).tolist()
```

3. Hybrid Search ist King

Kombinieren Sie immer Vector + Fulltext:

```
alpha = 0.7
combined_score = alpha * vector_score + (1 - alpha) * fulltext_score
```

4. Re-Ranking

Für höchste Qualität: Hole 100 Kandidaten, re-ranke Top 20:

```
def search_with_rerank(self, query: str, limit: int = 20):
    """Vector Search + Cross-Encoder Re-Ranking"""
    # Phase 1: Vector Search (schnell, grob)
    candidates = self.search(query, limit=100)

    # Phase 2: Re-Rank mit Cross-Encoder (genau, langsam)
    from sentence_transformers import CrossEncoder
    reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

    pairs = [[query, c.document.content] for c in candidates]
    scores = reranker.predict(pairs)

    # Sortiere nach Re-Rank Score
    ranked = sorted(zip(candidates, scores),
                   key=lambda x: x[1], reverse=True)

    return [r[0] for r in ranked[:limit]]
```

5. Monitoring & Debugging

```
def analyze_search_quality(self, query: str, expected_result_ids: List[str]):
    """Analysiere Search Quality"""
    results = self.search(query, limit=20)

    # Precision@K
    found_ids = [r.document.id for r in results[:10]]
    precision_at_10 = len(set(found_ids) & set(expected_result_ids)) / 10

    # Mean Reciprocal Rank (MRR)
    for i, result in enumerate(results, 1):
        if result.document.id in expected_result_ids:
            mrr = 1.0 / i
            break
    else:
        mrr = 0.0

    print(f"Query: {query}")
    print(f"  Precision@10: {precision_at_10:.2%}")
    print(f"  MRR: {mrr:.3f}")
    print(f"  Top Result Similarity: {results[0].similarity:.3f}")
```

8.8 Vergleich: ThemisDB vs. Spezialisierte Vector DBs

Feature	ThemisDB	Pinecone	Weaviate	Qdrant
Vector Search	HNSW	Proprietary	HNSW	HNSW
Relational Data	Native	× Nicht verfügbar	Basic	× Nicht verfügbar
Graph Queries	Native	× Nicht verfügbar	× Nicht verfügbar	× Nicht verfügbar
Fulltext Search	Native	Limited	Ja	Basic
ACID Transactions	Ja	× Nein	× Nein	× Nein
Hybrid Search	Native AQL	API	GraphQL	API
Self-Hosted	Ja	× Cloud-only	Ja	Ja
Pricing	Open Source	$$$$	Open Source	Open Source
Best For	Multi-Model Apps	Pure Vector	ML Pipelines	High Performance

ThemisDB Vorteil: Wenn Sie Vector Search **und** relationale/graph Daten brauchen, ist ThemisDB die einzige Datenbank, die alles nativ bietet.

8.9 Übungen

Übung 1: FAQ-Bot

Bauen Sie einen FAQ-Bot: - Laden Sie 20 FAQ-Paare (Frage + Antwort) - Bei User-Frage: Finde ähnlichste FAQ - Geben Sie die Antwort zurück - **Bonus:** Hybrid Search (Vector + Keyword)

Übung 2: Image Search

Erweitern Sie den E-Commerce Katalog: - Nutzen Sie CLIP für Bild-Embeddings - "Finde ähnliche Produkte zu diesem Bild" - Cross-Modal Search: Text Query → Bild Results

Übung 3: Multi-Collection RAG

Bauen Sie ein System mit mehreren Collections: - `technical_docs`, `marketing`, `legal` - User wählt Collection - RAG-Workflow liefert nur relevante Collection-Dokumente

8.10 Zusammenfassung

Was wir gelernt haben:

1. **Vector Search Grundlagen**
2. Embeddings repräsentieren Semantik als Vektoren
3. Kosinus-Ähnlichkeit misst Relevanz
4. HNSW-Index für effiziente Suche

5. **ThemisDB Vector Model**

6. Native `VECTOR(n)` Spalten
7. HNSW-Indexe mit konfigurierbaren Parametern
8. Operators: `<=>`, `<->`, `<#>`

9. **RAG-System Implementation**

10. Dokumente in Chunks aufteilen
11. Automatische Embedding-Generierung
12. Context-Retrieval für LLMs

13. **E-Commerce Vector Search**

14. Ähnliche Produkte finden
15. Hybrid Search mit Filtern
16. Cross-Category Similarity

17. **Best Practices**

18. Chunking: 200-300 Wörter
19. Hybrid Search: Vector + Fulltext
20. Re-Ranking für höchste Qualität
21. Batch-Processing für Performance

Key Takeaway: ThemisDB kombiniert Vector Search mit relationalen, Graph- und Dokument-Features in einer Datenbank – perfekt für moderne Multi-Model Anwendungen.

Nächster Schritt: In Teil III (Spezialanwendungen) lernen wir Time-Series, Geo-Spatial und Enterprise-Features kennen.

Ende Kapitel 8 | ➡ Weiter zu Kapitel 9: Time-Series

Kapitel 10: Kapitel 9: Time-Series & IoT-Daten

9.1 Einführung in Time-Series-Datenbanken

Was sind Time-Series-Daten?

Time-Series-Daten sind Messungen, die über die Zeit gesammelt werden. Jeder Datenpunkt hat einen Zeitstempel und einen oder mehrere Werte:

```
{
    "timestamp": "2024-01-15T10:30:45.123Z",
    "sensor_id": "temp_sensor_001",
    "temperature": 22.5,
    "humidity": 45.2,
    "location": "warehouse_a"
}
```

Charakteristiken: - **Hohe Schreiblast**: Tausende Messungen pro Sekunde - **Zeitbasierte Queries**: "Zeige mir alle Werte der letzten Stunde" - **Aggregationen**: Durchschnitt, Min/Max über Zeitfenster - **Retention Policies**: Alte Daten automatisch löschen oder aggregieren

Typische Use Cases

Use Case	Datenrate	Retention	Beispiel
IoT Sensoren	1-1000 Hz	30-90 Tage	Temperatur, Luftfeuchtigkeit
Monitoring	10-60 sec	1-12 Monate	CPU, Memory, Disk I/O
Financial Data	Real-time	Jahre	Aktienkurse, Trades
Log Aggregation	Variabel	7-30 Tage	Application Logs, Metrics
Smart Home	1-60 sec	3-6 Monate	Stromverbrauch, Heizung

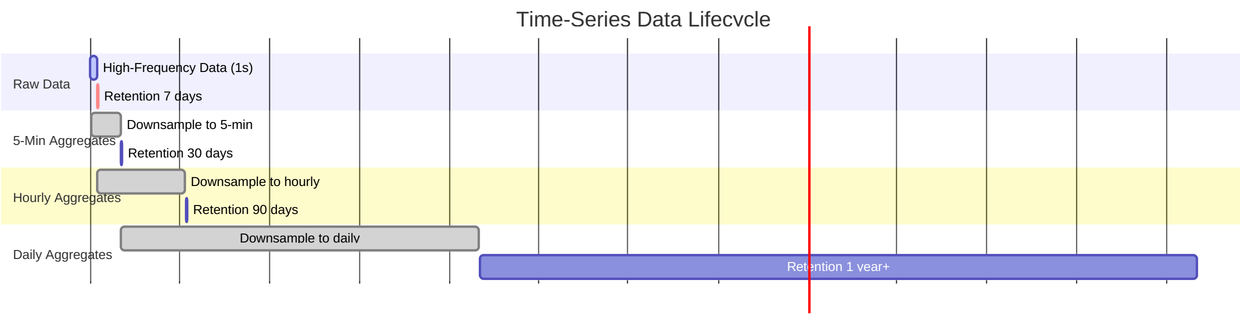


Abb. 27: Gantt-Diagramm (Kapitel 9: Time-Series & IoT-Daten)

9.2 Time-Series-Datenmodell in ThemisDB

Schema-Design für Time-Series

ThemisDB speichert Time-Series als Documents mit optimierten Indizes:

```
// Sensor-Messungen Collection
FOR doc IN [
  {
    _key: GENERATE_ID(),
    sensor_id: "sensor_001",
    timestamp: DATE_ISO8601(DATE_NOW()),
    temperature: 22.5,
    humidity: 65.0,
    co2_ppm: 420,
    location: "Room 101",
    metadata: {building: "A", floor: 2}
  }
] INSERT doc INTO sensor_readings

-- Index für Time-Range Queries
CREATE INDEX idx_sensor_time
  ON sensor_readings (sensor_id, timestamp)

-- Optional: TTL-Index für automatische Daten-Pruning
CREATE INDEX idx_ttl
  ON sensor_readings (timestamp)
  OPTIONS {expireAfterSeconds: 2592000}  -- 30 Tage
```

Design-Prinzipien: 1. **Composite Index**: `(sensor_id, timestamp)` - optimal für Time-Range-Queries 2. **TTL-Policies**: Automatisches Löschen alter Daten via Expiration-Index 3. **Sharding**: Horizontales Partitionieren nach `sensor_id` für Skalierung

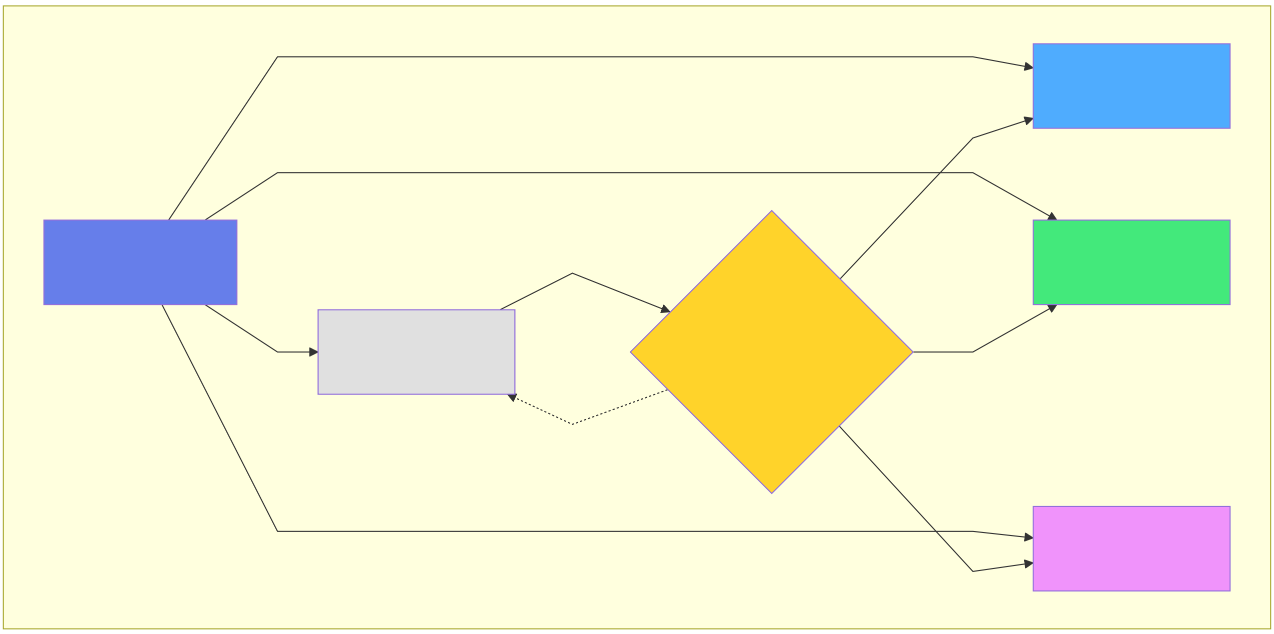


Abb. 28: Architekturdiagramm (Kapitel 9: Time-Series & IoT-Daten)

Indexes für Time-Series

```
-- Index für Zeit-basierte Queries
CREATE INDEX idx_readings_time ON sensor_readings (timestamp DESC);

-- Index für Sensor-spezifische Queries
CREATE INDEX idx_readings_sensor_time ON sensor_readings (sensor_id, timestamp DESC);

-- Conditional Index für Alerts (nur hohe Temperaturen)
CREATE INDEX idx_high_temp ON sensor_readings (timestamp)
WHERE temperature > 30.0;
```

9.3 Effiziente Time-Series Queries

Time-Range Queries

```
-- Alle Messungen der letzten Stunde
SELECT sensor_id, timestamp, temperature, humidity
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour'
ORDER BY timestamp DESC;

-- Durchschnitt pro Sensor in den letzten 24h
SELECT sensor_id,
       COUNT(*) as measurement_count,
       AVG(temperature) as avg_temp,
       AVG(humidity) as avg_humidity,
       MAX(co2_ppm) as max_co2
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '24 hours'
GROUP BY sensor_id;
```

Window Functions für Rolling Averages

```
-- Gleitender Durchschnitt (Moving Average) über 10 Messungen
SELECT
    sensor_id,
    timestamp,
    temperature,
    AVG(temperature) OVER (
        PARTITION BY sensor_id
        ORDER BY timestamp
        ROWS BETWEEN 9 PRECEDING AND CURRENT ROW
    ) as moving_avg_10
FROM sensor_readings
WHERE sensor_id = 'temp_001'
  AND timestamp >= NOW() - INTERVAL '1 day'
ORDER BY timestamp;

-- Differenz zur vorherigen Messung
SELECT
    sensor_id,
    timestamp,
    temperature,
    temperature - LAG(temperature) OVER (
        PARTITION BY sensor_id ORDER BY timestamp
    ) as temperature_change
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 hour';
```

Time-Bucket Aggregationen

```
-- Aggregiere zu 5-Minuten-Intervallen
SELECT
    sensor_id,
    DATE_TRUNC('minute', timestamp) -
        (EXTRACT(minute FROM timestamp)::int % 5) * INTERVAL '1 minute' as time_bucket,
    AVG(temperature) as avg_temp,
    MIN(temperature) as min_temp,
    MAX(temperature) as max_temp,
    STDDEV(temperature) as stddev_temp
FROM sensor_readings
WHERE timestamp >= NOW() - INTERVAL '1 day'
GROUP BY sensor_id, time_bucket
ORDER BY time_bucket DESC;
```

9.4 Down-Sampling und Retention Policies

Automatisches Down-Sampling

Speichere hochfrequente Rohdaten nur kurzfristig, langfristig nur Aggregate:

```
-- Down-Sampling: Stündliche Aggregate in neue Collection
LET hour_start = DATE_TRUNC(DATE_NOW(), 'hour')
LET hour_data = (
  FOR reading IN sensor_readings
    FILTER reading.timestamp >= DATE_SUBTRACT(hour_start, 1, 'hour')
       AND reading.timestamp < hour_start
    COLLECT sensor = reading.sensor_id INTO readings
    RETURN {
      sensor_id: sensor,
      hour_bucket: hour_start,
      avg_temperature: AVG(readings[*].reading.temperature),
      min_temperature: MIN(readings[*].reading.temperature),
      max_temperature: MAX(readings[*].reading.temperature),
      avg_humidity: AVG(readings[*].reading.humidity),
      max_co2_ppm: MAX(readings[*].reading.co2_ppm),
      sample_count: LENGTH(readings)
    }
)
FOR doc IN hour_data
  UPSERT {sensor_id: doc.sensor_id, hour_bucket: doc.hour_bucket}
  INSERT doc
  UPDATE doc
  INTO sensor_readings_hourly
```

```
max_temperature = EXCLUDED.max_temperature,
sample_count = EXCLUDED.sample_count;
```

```
<figure id="fig29" style="margin: 15pt 0; page-break-inside: avoid; text-align: center;">
  <img src="data:image/svg+xml;base64,PHN2ZyBpZD0ibXktc3ZnIiB3aWR0aD0iMTAwJSIgeG1sbnM9Imh0dHA
  <figcaption style="font-size:9pt; color:#666; margin-top:5pt; font-weight:600;">Abb. 29: Ab
</figure>


### Retention Policy Implementation

```aql
-- Lösche Rohdaten älter als 30 Tage
FOR reading IN sensor_readings
  FILTER reading.timestamp < DATE_NOW() - INTERVAL('30 days')
  REMOVE reading IN sensor_readings

-- Oder: Drop alte Partitionen (viel schneller!)
DROP TABLE IF EXISTS sensor_readings_2023_01;
```

Best Practice Retention: - **0-7 Tage**: Rohdaten (1 Sekunde Auflösung) - **7-30 Tage**: 1-Minute Aggregate - **30-90 Tage**: 5-Minute Aggregate - **90-365 Tage**: 1-Stunde Aggregate - **>1 Jahr**: 1-Tag Aggregate
```

9.5 Example: IoT Sensor Network

Überblick

Das **IoT Sensor Network** Beispiel (`examples/09_iot_sensor_network`) demonstriert ein vollständiges Monitoring-System für industrielle Sensoren:

Features: - Multi-Sensor Monitoring (Temperatur, Luftfeuchtigkeit, CO2) - Real-Time Alerting bei Schwellwert-Überschreitungen - Historische Analyse mit Trend-Erkennung - Dashboard mit Visualisierungen - Complex Event Processing (CEP)

Datenmodell

```
from datetime import datetime
from enum import Enum

class SensorType(Enum):
    TEMPERATURE = "temperature"
    HUMIDITY = "humidity"
    CO2 = "co2"
    PRESSURE = "pressure"

class Sensor:
    def __init__(self, sensor_id: str, sensor_type: SensorType,
                 location: str, unit: str):
        self.sensor_id = sensor_id
        self.sensor_type = sensor_type
        self.location = location
        self.unit = unit
        self.thresholds = {}

class SensorReading:
    def __init__(self, sensor_id: str, value: float,
                 timestamp: datetime = None):
        self.sensor_id = sensor_id
        self.value = value
        self.timestamp = timestamp or datetime.now()
```

Sensor-Registrierung

```
import themis_client as themis

sensors = [
    Sensor("temp_001", SensorType.TEMPERATURE, "warehouse_a", "°C"),
    Sensor("hum_001", SensorType.HUMIDITY, "warehouse_a", "%"),
    Sensor("co2_001", SensorType.CO2, "warehouse_a", "ppm")
]

for sensor in sensors:
    themis.query("""
        UPSERT {sensor_id: @sensor_id}
        INSERT {
          sensor_id: @sensor_id,
          sensor_type: @sensor_type,
          location: @location,
          unit: @unit,
          thresholds: @thresholds
        }
        UPDATE {
          location: @location,
          thresholds: @thresholds
        }
        INTO sensors
    """, {
        'sensor_id': sensor.sensor_id,
        'sensor_type': sensor.sensor_type.value,
        'location': sensor.location,
        'unit': sensor.unit,
        'thresholds': sensor.thresholds
    })
```

Echtzeit-Datenerfassung

```
import random
import time
from datetime import datetime

def simulate_sensors(duration_seconds=60):
    """Simuliere Sensor-Daten für Demo"""
    start_time = time.time()

    while time.time() - start_time < duration_seconds:
        readings = []

        # Generiere realistische Werte
        temp = 20 + random.gauss(2, 0.5)  # ~20°C ± 2°C
        humidity = 45 + random.gauss(5, 2)  # ~45% ± 5%
        co2 = 400 + random.gauss(50, 10)  # ~400 ppm ± 50

        readings.append(SensorReading("temp_001", temp))
        readings.append(SensorReading("hum_001", humidity))
        readings.append(SensorReading("co2_001", co2))

        # Batch-Insert mit AQL
        themis.query("""
            FOR reading IN @readings
              INSERT {
                sensor_id: reading.sensor_id,
                timestamp: reading.timestamp,
                value: reading.value
              } INTO sensor_readings
        """, {'readings': [{'sensor_id': r.sensor_id,
                            'timestamp': r.timestamp.isoformat(),
                            'value': r.value} for r in readings]})

        time.sleep(1)  # 1 Hz Sampling-Rate

simulate_sensors(duration_seconds=3600)  # 1 Stunde
```

Alert-System

```
def check_alerts():
    """Prüfe aktuelle Werte gegen Schwellwerte"""
    alerts = themis.query("""
        WITH latest_readings AS (
            SELECT DISTINCT ON (sensor_id)
                sensor_id, value, timestamp
            FROM sensor_readings
            ORDER BY sensor_id, timestamp DESC
        )
        SELECT
            s.sensor_id,
            s.sensor_type,
            s.location,
            lr.value,
            s.thresholds->>'max' as max_threshold,
            s.thresholds->>'min' as min_threshold
        FROM sensors s
        JOIN latest_readings lr ON s.sensor_id = lr.sensor_id
        WHERE lr.value > (s.thresholds->>'max')::float
           OR lr.value < (s.thresholds->>'min')::float
    """)

    for alert in alerts:
        send_alert(
            sensor_id=alert['sensor_id'],
            location=alert['location'],
            value=alert['value'],
            threshold_breached=True
        )
```

Historische Analyse

```
def analyze_trends(sensor_id: str, hours: int = 24):
    """Analysiere Trends der letzten N Stunden"""
    analysis = themis.query("""
        SELECT
            DATE_TRUNC('hour', timestamp) as hour,
            AVG(value) as avg_value,
            MIN(value) as min_value,
            MAX(value) as max_value,
            STDDEV(value) as stddev_value,
            -- Linearer Trend (Regression)
            REGR_SLOPE(value, EXTRACT(EPOCH FROM timestamp)) * 3600 as hourly_trend
        FROM sensor_readings
        WHERE sensor_id = ?
          AND timestamp >= NOW() - INTERVAL '? hours'
        GROUP BY hour
        ORDER BY hour
    """, (sensor_id, hours))

    return analysis

temp_trend = analyze_trends("temp_001", hours=24)
print(f"Hourly trend: {temp_trend[-1]['hourly_trend']:.3f}°C/hour")
```

Dashboard-Visualisierung

```
import matplotlib.pyplot as plt
import pandas as pd

def plot_sensor_dashboard(sensor_id: str, hours: int = 24):
    """Erstelle Dashboard für einen Sensor"""
    # Lade Daten
    data = themis.query("""
        SELECT timestamp, value
        FROM sensor_readings
        WHERE sensor_id = ?
          AND timestamp >= NOW() - INTERVAL '? hours'
        ORDER BY timestamp
    """, (sensor_id, hours))

    df = pd.DataFrame(data)

    # Plot erstellen
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

    # Zeitreihe
    ax1.plot(df['timestamp'], df['value'], linewidth=0.5)
    ax1.set_title(f'Sensor {sensor_id} - Last {hours}h')
    ax1.set_ylabel('Value')
    ax1.grid(True, alpha=0.3)

    # Histogram
    ax2.hist(df['value'], bins=50, alpha=0.7)
    ax2.set_xlabel('Value')
    ax2.set_ylabel('Frequency')
    ax2.axvline(df['value'].mean(), color='red',
                linestyle='--', label='Mean')
    ax2.legend()

    plt.tight_layout()
    plt.savefig(f'dashboard_{sensor_id}.png', dpi=150)
    plt.show()

plot_sensor_dashboard("temp_001")
```

9.6 Example: Smart Home Energy Monitoring

Überblick

Das **Smart Home** Beispiel (`examples/20_smart_home`) zeigt Energy Monitoring und Automation:

Features: - Stromverbrauch-Tracking pro Gerät - Automatisierungsregeln (z.B. Heizung runter wenn keiner da ist) - Cost-Calculation basierend auf Strompreisen - Energy-Saving-Recommendations

Geräte-Management

```
class Device:
    def __init__(self, device_id: str, device_type: str,
                 room: str, power_rating_watts: int):
        self.device_id = device_id
        self.device_type = device_type  # "heater", "light", "appliance"
        self.room = room
        self.power_rating_watts = power_rating_watts
        self.state = "off"

devices = [
    Device("heater_living", "heater", "living_room", 2000),
    Device("light_living", "light", "living_room", 60),
    Device("fridge_kitchen", "appliance", "kitchen", 150),
]

for dev in devices:
    themis.execute("""
        INSERT INTO devices (device_id, device_type, room, power_rating_watts)
        VALUES (?, ?, ?, ?)
    """, (dev.device_id, dev.device_type, dev.room, dev.power_rating_watts))
```

Energy-Tracking

```
def track_energy_consumption():
    """Track energy consumption per device"""
    # Record current state and power usage
    for device in devices:
        if device.state == "on":
            power_usage = device.power_rating_watts
        else:
            power_usage = 0

        themis.execute("""
            INSERT INTO energy_readings
            (device_id, timestamp, power_watts, state)
            VALUES (?, NOW(), ?, ?)
        """, (device.device_id, power_usage, device.state))

while True:
    track_energy_consumption()
    time.sleep(60)
```

Cost Calculation

```
def calculate_daily_cost(date):
    """Berechne Energiekosten für einen Tag"""
    cost_per_kwh = 0.30  # 30 Cent/kWh

    daily_usage = themis.query("""
        SELECT
            device_id,
            d.device_type,
            d.room,
            -- Energie in kWh (Durchschnitt * Zeit / 1000)
            SUM(power_watts * 60) / 1000.0 / 60.0 as kwh_consumed,
            -- Kosten
            SUM(power_watts * 60) / 1000.0 / 60.0 * ? as cost_euros
        FROM energy_readings er
        JOIN devices d ON er.device_id = d.device_id
        WHERE DATE(timestamp) = ?
        GROUP BY device_id, d.device_type, d.room
        ORDER BY cost_euros DESC
    """, (cost_per_kwh, date))

    total_cost = sum(row['cost_euros'] for row in daily_usage)

    print(f"Daily Energy Cost: €{total_cost:.2f}")
    for row in daily_usage:
        print(f"  {row['device_id']}: {row['kwh_consumed']:.2f} kWh → €{row['cost_euros']:.2f

    return daily_usage

calculate_daily_cost('2024-01-15')
```

Automation Rules

```
class AutomationRule:
    def __init__(self, rule_id: str, condition: str, action: str):
        self.rule_id = rule_id
        self.condition = condition
        self.action = action

rules = [
    AutomationRule("rule_001",
                   "temperature < 18 AND presence = false",
                   "set_heater_temp(15)"),
    AutomationRule("rule_002",
                   "time > 22:00 AND presence = false",
                   "turn_off_all_lights()"),
]

def evaluate_rules():
    """Evaluiere und führe Automation-Rules aus"""
    for rule in rules:
        # Check condition
        condition_met = themis.query_one(f"""
            LET latest = (
                FOR state IN device_states
                  SORT state.timestamp DESC
                  LIMIT 1
                  RETURN state
            )[0]
            RETURN {{rule.condition}} AS met
        """)

        if condition_met['met']:
            exec(rule.action)  # Führe Aktion aus
            print(f"Rule {rule.rule_id} triggered: {rule.action}")
```

9.7 Performance-Optimierungen

Batch-Inserts für hohen Durchsatz

```
for reading in readings:
    themis.execute("INSERT @reading INTO sensor_readings", {"reading": reading})

themis.execute_batch(
    "INSERT @reading INTO sensor_readings",
    [{"reading": r} for r in readings],
    batch_size=1000
)
```

Partitionierung für schnelle Queries

```
-- Automatisches Partition-Management
CREATE OR REPLACE FUNCTION create_monthly_partition()
RETURNS void AS $$
DECLARE
    start_date DATE;
    end_date DATE;
    partition_name TEXT;
BEGIN
    start_date := DATE_TRUNC('month', NOW());
    end_date := start_date + INTERVAL '1 month';
    partition_name := 'sensor_readings_' || TO_CHAR(start_date, 'YYYY_MM');

    EXECUTE format('
        CREATE TABLE IF NOT EXISTS %I PARTITION OF sensor_readings
        FOR VALUES FROM (%L) TO (%L)
    ', partition_name, start_date, end_date);
END;
$$ LANGUAGE plpgsql;
```

Materialized Views für Dashboards

```
-- Pre-Aggregiere Daten für schnelle Dashboard-Queries
CREATE MATERIALIZED VIEW sensor_daily_stats AS
SELECT
    sensor_id,
    DATE(timestamp) as date,
    AVG(value) as avg_value,
    MIN(value) as min_value,
    MAX(value) as max_value,
    COUNT(*) as reading_count
FROM sensor_readings
GROUP BY sensor_id, DATE(timestamp);

-- Refresh täglich
CREATE INDEX ON sensor_daily_stats (sensor_id, date DESC);
REFRESH MATERIALIZED VIEW CONCURRENTLY sensor_daily_stats;
```

9.8 Best Practices

1. Schema-Design

DO: - Nutze Partitionierung für große Tabellen - Composite Primary Key: `(entity_id, timestamp)` - Index auf `timestamp DESC` für neueste Daten

× **DON'T:** - Nicht UUID als Primary Key bei Time-Series - Keine UNIQUEness Constraints außer Primary Key - Vermeide komplexe JOINs in High-Frequency Queries

2. Query-Optimierung

DO: - Nutze TIME-RANGE in WHERE-Clause für Partition Pruning - Batch-Inserts statt einzelne INSERTs - Materialized Views für häufige Aggregationen

× **DON'T:** - Keine `SELECT *` - nur benötigte Spalten - Vermeide DISTINCT ohne Index - Keine ungebundenen Queries ohne Zeit-Limit

3. Retention & Storage

DO: - Implementiere Retention Policies - Down-Sample alte Daten - Nutze automatisches Partition-Management

× **DON'T:** - Behalte nicht alle Rohdaten für immer - Lösche nicht mit DELETE (nutze DROP PARTITION) - Vergiss nicht Backups vor Partition-Drops

9.9 Vergleich: ThemisDB vs. Specialized Time-Series DBs

Feature	ThemisDB	InfluxDB	TimescaleDB
Data Model	Relational + Multi-Model	Line Protocol	Relational
Query Language	SQL/AQL	InfluxQL, Flux	SQL
Partitioning	Native	Automatic	Hypertables
Continuous Aggregates	Mat. Views	Native	Continuous Aggregates
Retention Policies	Manual/Triggers	Automatic	Automatic
Downsampling	AQL-based	Built-in	Continuous Aggregates
Multi-Model	Graph, Vector, Doc	×	×
Horizontal Scaling	Sharding	Clustering	Distributed

Wann ThemisDB wählen: - Wenn Time-Series nur ein Teil der Anwendung ist - Wenn komplexe Relationen zu anderen Daten bestehen - Wenn Multi-Model Features (Graph, Vector) benötigt werden - Wenn Standard-AQL Queries ausreichen

Wann Specialized DB wählen: - InfluxDB: Sehr hohe Schreibraten (>1M writes/sec), native Telegraf-Integration - TimescaleDB: Wenn PostgreSQL-Kompatibilität Priorität hat

9.10 Zusammenfassung

ThemisDB ist bestens geeignet für Time-Series & IoT-Anwendungen durch:

Native Unterstützung: - Partitionierung für Milliarden von Datenpunkten - Effiziente Time-Range Queries mit Partition Pruning - Window Functions für Rolling Averages und Trend-Analyse

Performance: - Batch-Inserts für hohen Schreibdurchsatz - Materialized Views für schnelle Dashboards - Indexing-Strategien für typische Time-Series Patterns

Flexibilität: - Kombiniere Time-Series mit relationalen Daten - Nutze Graph-Queries für Sensor-Topologien - Vector-Search für Pattern-Matching in Zeitreihen

Key Takeaways: 1. Nutze Partitionierung + richtige Indexes 2. Implementiere Retention Policies frühzeitig 3. Batch-Inserts für Performance 4. Materialized Views für Dashboards 5. Down-Sample alte Daten automatisch

---

Kapitel 11: Kapitel 10: Enterprise-Anwendungen

Einleitung

Enterprise-Anwendungen sind das Rückgrat moderner Unternehmen. Sie verwalten kritische Geschäftsprozesse, von der Kundenverwaltung über Dokumentenmanagement bis hin zu ERP-Systemen. In diesem Kapitel lernen Sie, wie ThemisDB alle Anforderungen enterprise-grade Software erfüllt:

- **Multi-Tenancy** für Mandantenfähigkeit
- **RBAC** (Role-Based Access Control) für fein-granulare Zugriffsrechte
- **Audit-Logging** für vollständige Nachvollziehbarkeit
- **Workflow-Management** für Geschäftsprozesse
- **Skalierbarkeit** für große Datenmengen
- **Security** auf allen Ebenen

Wir demonstrieren dies anhand zweier vollständiger Beispiele: 1. **DMS/ERP-System**: Dokumentenmanagement mit Workflows 2. **CRM-System**: Customer Relationship Management

10.1 Enterprise-Anforderungen

Mandantenfähigkeit (Multi-Tenancy)

In SaaS-Anwendungen müssen Daten verschiedener Kunden (Mandanten/Tenants) strikt getrennt sein.

Drei Ansätze:

```
db_tenant_1 = ThemisDB("tenant_1.db")
db_tenant_2 = ThemisDB("tenant_2.db")

db.execute("""
    CREATE TABLE documents (
        id UUID PRIMARY KEY,
        tenant_id UUID NOT NULL,
        title TEXT,
        content TEXT
    )
""")
db.execute("CREATE INDEX idx_tenant ON documents(tenant_id)")

db.execute("""
    CREATE POLICY tenant_isolation ON documents
    USING (tenant_id = current_tenant_id())
""")
```

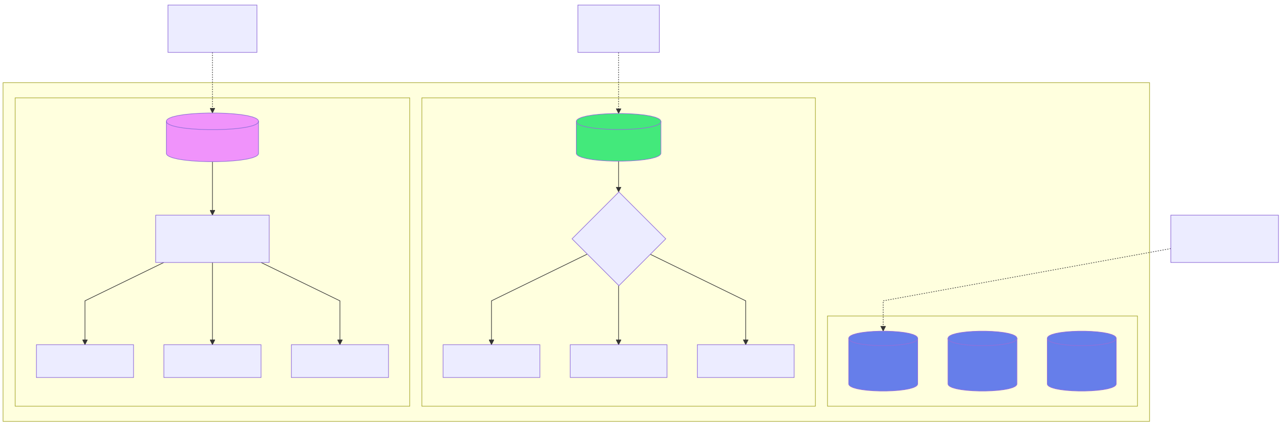


Abb. 30: Architekturdiagramm (Kapitel 10: Enterprise-Anwendungen)

Best Practice in ThemisDB:

```
class TenantContext:
    def __init__(self, db, tenant_id):
        self.db = db
        self.tenant_id = tenant_id

    def query(self, sql, params=None):
        # Automatisches Anhängen der Tenant-Bedingung
        if "WHERE" in sql.upper():
            sql = sql.replace("WHERE", f"WHERE tenant_id = '{self.tenant_id}' AND")
        else:
            sql += f" WHERE tenant_id = '{self.tenant_id}'"
        return self.db.execute(sql, params)

tenant = TenantContext(db, "acme_corp")
docs = tenant.query("""
    FOR doc IN documents
      FILTER doc.type == @type
      RETURN doc
""", {"type": "invoice"})
```

Rollen-basierte Zugriffskontrolle (RBAC)

Datenmodell:

```
db.execute("""
    CREATE TABLE roles (
        id UUID PRIMARY KEY,
        name TEXT UNIQUE NOT NULL,
        permissions TEXT[], -- ["read:documents", "write:documents"]
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE user_roles (
        user_id UUID,
        role_id UUID,
        granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (user_id, role_id)
    )
""")

def has_permission(user_id, permission):
    result = db.execute("""
        FOR user_role IN user_roles
          FILTER user_role.user_id == @user_id
          FOR role IN roles
            FILTER user_role.role_id == role.id AND @permission IN role.permissions
            LIMIT 1
            RETURN 1
    """, {"user_id": user_id, "permission": permission})
    return len(result) > 0
```

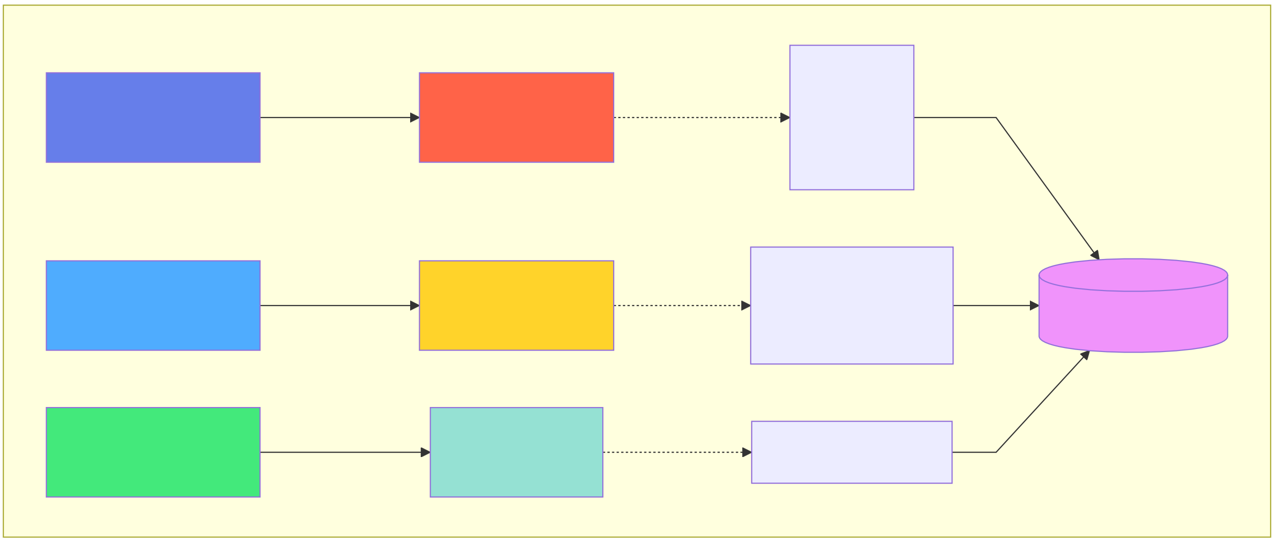


Abb. 31: Architekturdiagramm (Kapitel 10: Enterprise-Anwendungen)

def requires_permission(permission): def decorator(func): def wrapper(*args, kwargs): user_id = get_current_user_id() if not has_permission(user_id, permission): raise PermissionDeniedError() return func(*args, **kwargs) return wrapper return decorator

@requires_permission("write:documents") def create_document(title, content): # Nur erreichbar mit korrekter Permission pass

```
### Audit-Logging

Jede Änderung muss nachvollziehbar sein:

```python
db.execute("""
    CREATE TABLE audit_log (
        id UUID PRIMARY KEY,
        timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        user_id UUID NOT NULL,
        action TEXT NOT NULL, -- CREATE, UPDATE, DELETE, READ
        resource_type TEXT NOT NULL, -- "document", "user", etc.
        resource_id UUID NOT NULL,
        old_value JSON,
        new_value JSON,
        ip_address TEXT,
        user_agent TEXT
    )
""")

db.execute("CREATE INDEX idx_audit_resource ON audit_log(resource_type, resource_id)")
db.execute("CREATE INDEX idx_audit_user ON audit_log(user_id, timestamp)")

def log_action(action, resource_type, resource_id, old_value=None, new_value=None):
    db.execute("""
        INSERT {
          id: @id,
          user_id: @user_id,
          action: @action,
          resource_type: @resource_type,
          resource_id: @resource_id,
          old_value: @old_value,
          new_value: @new_value,
          ip_address: @ip_address
        } INTO audit_log
    """, {
        "id": uuid.uuid4(),
        "user_id": get_current_user_id(),
        "action": action,
        "resource_type": resource_type,
        resource_id,
        json.dumps(old_value) if old_value else None,
        json.dumps(new_value) if new_value else None,
        get_client_ip()
    })

old_doc = db.execute("""
    FOR doc IN documents
      FILTER doc.id == @doc_id
      LIMIT 1
      RETURN doc
""", {"doc_id": doc_id})[0]

db.execute("""
```
```

```
    FOR doc IN documents
      FILTER doc.id == @doc_id
      UPDATE doc WITH {title: @new_title} IN documents
""", {"doc_id": doc_id, "new_title": new_title})

new_doc = db.execute("""
    FOR doc IN documents
      FILTER doc.id == @doc_id
      LIMIT 1
      RETURN doc
""", {"doc_id": doc_id})[0]

log_action("UPDATE", "document", doc_id, old_doc, new_doc)
```

Audit-Abfragen:

```
changes = db.execute("""
    FOR log IN audit_log
      FILTER log.resource_type == 'document' AND log.resource_id == @doc_id
      SORT log.timestamp DESC
      RETURN log
""", {"doc_id": doc_id})

activity = db.execute("""
    FOR log IN audit_log
      FILTER log.user_id == @user_id AND log.timestamp > DATE_NOW() - INTERVAL('7 days')
      SORT log.timestamp DESC
      RETURN log
""", {"user_id": user_id})
```

10.2 Native Security-Stack: Production-Ready Sicherheit

ThemisDB v1.3.0 bietet einen vollständig nativen, in C++ implementierten Security-Stack, der BSI C5-konform ist und keine externen Abhängigkeiten (wie Apache Ranger) für die Kern-Sicherheitsfunktionen benötigt.

10.2.1 Implementierungsstatus (Dezember 2025)

Komponente	Status	Implementierung	Beschreibung
RBAC/ABAC Policy Engine	Produktionsreif	`src/security/rbac.cpp`	Role-Based & Attribute-Based Access Control
Apache Ranger Integration	Produktionsreif	`src/server/ranger_adapter.cpp`	Optional: Enterprise Policy Management
Field-Level Encryption	Produktionsreif	`src/security/encryption.cpp`	AES-256-GCM, transparent
Column Encryption	Produktionsreif	BSI C5 konform	Spalten-basierte Verschlüsselung
Vector Encryption	Produktionsreif	Phase 1+2 komplett	HNSW-Index + RocksDB Vektoren
PKI Integration	Produktionsreif	`src/security/pki_key_provider.cpp`	X.509 Zertifikat-basiert
HSM Support	Produktionsreif	`src/security/hsm_provider_pkcs11.cpp`	PKCS#11 Hardware Security Modules
Audit Logging	Produktionsreif	`src/utils/audit_logger.cpp`	Tamper-Proof mit Hash-Chains
Malware Scanner	Produktionsreif	`src/security/malware_scanner.cpp`	Content Security
CMS Signing	Produktionsreif	`src/security/cms_signing.cpp`	Cryptographic Message Syntax
RFC 3161 TSA	Produktionsreif	`src/security/timestamp_authority.cpp`	Qualified Timestamps

Gesamt: 16 Header-Dateien, 16 Source-Dateien, ~8.100 LOC

10.2.2 RBAC-Implementierung im C++ Core

ThemisDB implementiert RBAC direkt im C++ Kern, nicht als Add-on-Layer:

Architektur:

```
// Native RBAC Integration (src/security/rbac.h)
class RBACManager {
public:
    // Role Management
    Status createRole(const std::string& roleName,
                      const std::vector<std::string>& permissions);
    Status assignRole(const std::string& userId,
                      const std::string& roleName);

    // Permission Check (optimiert für Performance)
    bool hasPermission(const std::string& userId,
                       const std::string& resource,
                       const std::string& action) const;

    // Attribute-Based (ABAC) Extension
    bool checkPolicy(const PolicyContext& ctx) const;
};
```

Performance-Optimierung: - Permission-Checks gecacht im RAM (TBB concurrent_hash_map) - Sub-Millisekunden Latenz für Permission-Checks - Keine Netzwerk-Latenz (im Gegensatz zu externen Policy-Servern)

Verwendung in AQL:

```
// Automatischer Permission-Check bei Query-Ausführung
FOR doc IN documents
    // RBAC-Filter wird automatisch eingefügt basierend auf User-Kontext
    FILTER HAS_PERMISSION(CURRENT_USER(), doc._id, 'read')
    RETURN doc
```

10.2.3 Tamper-Proof Audit Logging mit Hash-Chains

Das Audit-System von ThemisDB verhindert nachträgliche Manipulation durch **kryptografische Hash-Chains**:

Funktionsweise:

```
// src/utils/audit_logger.cpp
class AuditLogger {
    struct AuditEntry {
        uint64_t sequence_number;
        int64_t timestamp_ms;
        std::string user_id;
        std::string action;
        std::string resource_id;
        std::string details;
        std::string previous_hash;  // Hash des vorherigen Eintrags
        std::string current_hash;   // SHA-256 über alle Felder
    };

    // Verkettung erzwingt chronologische Integrität
    std::string computeHash(const AuditEntry& entry) {
        std::string data = std::to_string(entry.sequence_number) +
                          std::to_string(entry.timestamp_ms) +
                          entry.user_id + entry.action +
                          entry.resource_id + entry.details +
                          entry.previous_hash;
        return SHA256(data);  // OpenSSL
    }
};
```

Garantien: - **Unveränderbarkeit:** Jede Änderung bricht die Hash-Chain - **Lückenlosigkeit:** Fehlende Einträge sind sofort erkennbar - **Zeitstempel-Authentizität:** Integration mit RFC 3161 Timestamp Authority - **Revisionssicher:** BSI-konform, DSGVO Art. 32

Verifikation:

```
// Audit-Log-Integrität prüfen
FOR entry IN audit_log
    SORT entry.sequence_number ASC
    LET expected_hash = SHA256(
        CONCAT(
            entry.sequence_number,
            entry.timestamp_ms,
            entry.user_id,
            entry.action,
            entry.resource_id,
            entry.details,
            entry.previous_hash
        )
    )
    FILTER entry.current_hash != expected_hash
    RETURN {
        error: "Audit log integrity violation",
        sequence: entry.sequence_number
    }
```

10.2.4 HashiCorp Vault Integration

Für Enterprise-Key-Management integriert ThemisDB mit HashiCorp Vault:

Konfiguration:

```
// src/security/vault_key_provider.cpp
VaultKeyProvider::Config vault_config;
vault_config.vault_addr = "https://vault.company.com:8200";
vault_config.token = std::getenv("VAULT_TOKEN");
vault_config.mount_path = "themisdb";
vault_config.key_name = "database-master-key";

auto key_provider = std::make_shared<VaultKeyProvider>(vault_config);
FieldEncryption encryption(key_provider);
```

Features: - Automatische Key-Rotation - Audit Trail für Key-Access - High Availability (Vault Cluster) - Disaster Recovery (Sealed/Unsealed State)

Alternative Key Provider: - `MockKeyProvider`: Für Testing/Development - `PKIKeyProvider`: Zertifikat-basiert für PKI-Infrastrukturen - `HSMProvider`: PKCS#11 für Hardware Security Modules (Luna, Thales)

10.2.5 BSI C5 Compliance: Kryptographie

ThemisDB erreicht **100% BSI C5 Compliance** für Kryptographie-Anforderungen (CRY-01 bis CRY-06):

Anforderung	Umsetzung	Dokument
CRY-01: Kryptographie-Policy	Formale Policy dokumentiert	`CRYPTOGRAPHY_POLICY.md`
CRY-02: Key Lifecycle	Vollständiger Lebenszyklus	`KEY_LIFECYCLE_MANAGEMENT.md`
CRY-03: At-Rest Encryption	AES-256-GCM für alle Daten	Column + Vector Encryption
CRY-04: In-Transit Encryption	TLS 1.3 mandatory	Wire Protocol + HTTP/2
CRY-05: Key Storage	HSM/Vault Integration	`VaultKeyProvider` + `HSMProvider`
CRY-06: Algorithm Strength	BSI TR-02102-1 konform	AES-256, RSA-2048+, SHA-256

Besonderheit: Multi-Model Encryption Consistency

ThemisDB's Unified Storage Architecture garantiert konsistente Verschlüsselung über alle Datenmodelle:

```
┌─────────────────────────────────────────────────────────┐
│           Application Layer (AQL Queries)                │
├─────────────────────────────────────────────────────────┤
│  Relational  │  Graph  │  Vector  │  Document  │  Geo   │
│  Projection  │  Proj.  │  Proj.   │  Proj.     │  Proj. │
└──────────────┴─────────┴──────────┴────────────┴────────┘
                         │
                         ▼
        ┌────────────────────────────────────────┐
        │   Encryption Layer (AES-256-GCM)       │  ◄── Einheitlich!
        └────────────────────────────────────────┘
                         │
                         ▼
        ┌────────────────────────────────────────┐
        │  Base Entity Storage (RocksDB)         │
        │  • Alle Daten verschlüsselt            │
        │  • Keine Encryption Gaps               │
        └────────────────────────────────────────┘
```

Vorteil: Im Gegensatz zu Polyglot-Systemen, wo jede Datenbank eigene Encryption benötigt, ist in ThemisDB alles einheitlich verschlüsselt – kein Modell kann "vergessen" werden.

10.2.6 DSGVO "By Design" Features

ThemisDB implementiert DSGVO-Anforderungen technisch:

1. Auto-Purge nach Retention-Period (Art. 17 - Recht auf Löschung):

```
// src/utils/retention_manager.cpp
RetentionManager::Config retention;
retention.enable_auto_purge = true;
retention.default_retention_days = 2555;  // 7 Jahre (§ 147 AO)
retention.purge_schedule_cron = "0 2 * * 0";  // Sonntags 2 Uhr

// Anwendung auf Collection
db.execute("""
    CREATE COLLECTION customer_data WITH {
        retention_days: 2555,
        auto_purge: true
    }
""");
```

2. PII Detection und Redaction (Art. 32 - Datensicherheit):

```
// src/utils/pii_detector.cpp
PIIDetector detector;
detector.addPattern("email", R"([\w\.-]+@[\w\.-]+\.\w+)");
detector.addPattern("iban", R"(DE\d{20})");
detector.addPattern("ssn", R"(\d{3}-\d{2}-\d{4})");

// Automatische Redaction bei Logging
std::string safe_text = detector.redact(user_input);
// Output: "Contact: ***@***.com, IBAN: DE********************"
```

3. Encrypt-then-Sign für Log-Integrität:

```
// src/security/cms_signing.cpp
CMSSigning signer(private_key, certificate);

// Audit-Log Entry signieren
std::string signed_entry = signer.sign(
    audit_entry_json,
    true  // include_timestamp (RFC 3161)
);

// Später: Verifizierung
bool valid = signer.verify(signed_entry, public_cert);
```

4. Granulare Retention Policies:

```
-- Unterschiedliche Aufbewahrungsfristen pro Datentyp
CREATE COLLECTION invoices WITH { retention_days: 3650 };  -- 10 Jahre
CREATE COLLECTION marketing_consents WITH { retention_days: 730 };  -- 2 Jahre
CREATE COLLECTION session_logs WITH { retention_days: 90 };  -- 90 Tage
```

10.2.7 Code-Beispiel: Vollständige Enterprise-Security

```
from themisdb import Client, SecurityContext

client = Client("localhost", 8765)

auth_token = client.authenticate(
    cert_file="user.crt",
    key_file="user.key"
)

sec_ctx = SecurityContext(
    user_id="alice@company.com",
    roles=["data_analyst", "auditor"],
    tenant_id="acme_corp"
)

with client.transaction(sec_ctx) as tx:
    # Permission-Check + Audit-Log-Eintrag automatisch
    results = tx.query("""
        FOR doc IN sensitive_documents
            FILTER doc.classification <= @max_clearance
            RETURN doc
    """, {"max_clearance": sec_ctx.get_clearance_level()})

    # Alle Aktionen werden geloggt:
    # - Wer (alice@company.com)
    # - Was (SELECT sensitive_documents)
    # - Wann (2025-12-30T15:00:00Z)
    # - Ergebnis (5 rows returned)
    # - Hash (SHA-256 chain)

audit_report = client.get_audit_report(
    start_date="2025-01-01",
    end_date="2025-12-31",
    user_id="alice@company.com"
)
```

10.3 DMS/ERP-System (Example 08)

Ein vollständiges Document Management System mit ERP-Features.

Architektur-Übersicht

```
┌─────────────────────────────────────────────────┐
│            Web-Interface (Flask)                │
├─────────────────────────────────────────────────┤
│  Document API │ Workflow API │ Search API       │
├─────────────────────────────────────────────────┤
│          ThemisDB (Multi-Model)                 │
│   ┌──────────┬──────────┬──────────┬──────────┐ │
│   │Relational│  Graph   │ Document │  Vector  │ │
│   │(Meta)    │(Workflow)│(Versions)│(Search)  │ │
│   └──────────┴──────────┴──────────┴──────────┘ │
├─────────────────────────────────────────────────┤
│  File Storage (S3/Local) │ OCR Engine           │
└─────────────────────────────────────────────────┘
```

Datenmodell

Dokumente:

```
class Document:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Haupt-Dokument (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS documents (
                id UUID PRIMARY KEY,
                tenant_id UUID NOT NULL,
                title TEXT NOT NULL,
                type TEXT NOT NULL, -- invoice, contract, hr_document
                version INT DEFAULT 1,
                current_version_id UUID,
                owner_id UUID NOT NULL,
                workflow_state TEXT DEFAULT 'draft',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Metadaten (Document Model - JSON)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_metadata (
                document_id UUID PRIMARY KEY,
                metadata JSON NOT NULL,
                tags TEXT[],
                FOREIGN KEY (document_id) REFERENCES documents(id)
            )
        """)

        # Versionen (Dokument Model für Historie)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_versions (
                id UUID PRIMARY KEY,
                document_id UUID NOT NULL,
                version INT NOT NULL,
                file_path TEXT NOT NULL,
                file_size BIGINT,
                checksum TEXT,
                created_by UUID NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                change_note TEXT,
                FOREIGN KEY (document_id) REFERENCES documents(id)
            )
        """)

        # Permissions (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_permissions (
                document_id UUID,
                user_id UUID,
                permission TEXT CHECK (permission IN ('read', 'write', 'admin')),
```

```
                PRIMARY KEY (document_id, user_id),
                FOREIGN KEY (document_id) REFERENCES documents(id)
            )
        """)

        # Volltext-Extraktion
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_text (
                document_id UUID PRIMARY KEY,
                content TEXT,
                ocr_content TEXT, -- OCR-erkannter Text
                FOREIGN KEY (document_id) REFERENCES documents(id)
            )
        """)
        # Fulltext Index
        self.db.execute("CREATE INDEX idx_doc_fulltext ON document_text USING GIN(to_tsvector

        # Vector Embeddings (für Ähnlichkeitssuche)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS document_embeddings (
                document_id UUID PRIMARY KEY,
                embedding VECTOR(384), -- sentence-transformers
                FOREIGN KEY (document_id) REFERENCES documents(id)
            )
        """)
        self.db.execute("CREATE INDEX idx_doc_vector ON document_embeddings USING HNSW(embedd

    def create_document(self, title, doc_type, file_path, metadata=None, owner_id=None):
        """Neues Dokument erstellen"""
        doc_id = uuid.uuid4()
        version_id = uuid.uuid4()

        with self.db.transaction():
            # 1. Haupt-Eintrag
            self.db.execute("""
                INSERT {
                  id: @doc_id,
                  tenant_id: @tenant_id,
                  title: @title,
                  type: @doc_type,
                  current_version_id: @version_id,
                  owner_id: @owner_id
                } INTO documents
            """, {
                "doc_id": doc_id,
                "tenant_id": get_current_tenant_id(),
                "title": title,
                "doc_type": doc_type,
                "version_id": version_id,
                "owner_id": owner_id or get_current_user_id()
            })

            # 2. Erste Version
            file_size = os.path.getsize(file_path)
```

```
        checksum = calculate_checksum(file_path)
        self.db.execute("""
            INSERT {
              id: @version_id,
              document_id: @doc_id,
              version: 1,
              file_path: @file_path,
              file_size: @file_size,
              checksum: @checksum,
              created_by: @created_by
            } INTO document_versions
        """, {
            "version_id": version_id,
            "doc_id": doc_id,
            "file_path": file_path,
            "file_size": file_size,
            "checksum": checksum,
            "created_by": get_current_user_id()
        })

        # 3. Metadaten
        if metadata:
            self.db.execute("""
                INSERT {
                  document_id: @doc_id,
                  metadata: @metadata,
                  tags: @tags
                } INTO document_metadata
            """, {
                "doc_id": doc_id,
                "metadata": json.dumps(metadata),
                "tags": metadata.get('tags', [])
            })

        # 4. Text-Extraktion (asynchron in der Praxis)
        text = extract_text(file_path)  # PDF, DOCX, etc.
        self.db.execute("""
            INSERT {
              document_id: @doc_id,
              content: @content
            } INTO document_text
        """, {"doc_id": doc_id, "content": text})

        # 5. OCR für Bilder (wenn nötig)
        if doc_type in ('scan', 'image'):
            ocr_text = perform_ocr(file_path)
            self.db.execute("""
                FOR doc_text IN document_text
                  FILTER doc_text.document_id == @doc_id
                  UPDATE doc_text WITH {ocr_content: @ocr_text} IN document_text
            """, {"doc_id": doc_id, "ocr_text": ocr_text})

        # 6. Vector Embedding
        embedding = generate_embedding(text)  # sentence-transformers
```

```
            self.db.execute("""
                INSERT {
                  document_id: @doc_id,
                  embedding: @embedding
                } INTO document_embeddings
            """, {"doc_id": doc_id, "embedding": embedding})

            # 7. Audit-Log
            log_action("CREATE", "document", doc_id, None, {"title": title, "type": doc_type}

        return doc_id

    def add_version(self, document_id, file_path, change_note=""):
        """Neue Version hinzufügen"""
        # Aktuelle Version ermitteln
        current = self.db.execute("""
            FOR doc IN documents
              FILTER doc.id == @document_id
              LIMIT 1
              RETURN doc.version
        """, {"document_id": document_id})[0]
        new_version = current + 1
        version_id = uuid.uuid4()

        with self.db.transaction():
            # Version speichern
            self.db.execute("""
                INSERT {
                  id: @version_id,
                  document_id: @document_id,
                  version: @new_version,
                  file_path: @file_path,
                  file_size: @file_size,
                  checksum: @checksum,
                  created_by: @created_by,
                  change_note: @change_note
                } INTO document_versions
            """, {
                "version_id": version_id,
                "document_id": document_id,
                "new_version": new_version,
                "file_path": file_path,
                "file_size": os.path.getsize(file_path),
                "checksum": calculate_checksum(file_path),
                "created_by": get_current_user_id(),
                "change_note": change_note
            })

            # Dokument aktualisieren
            self.db.execute("""
                FOR doc IN documents
                  FILTER doc.id == @document_id
                  UPDATE doc WITH {
                    version: @new_version,
```

```
                current_version_id: @version_id,
                updated_at: DATE_NOW()
              } IN documents
        """, {"document_id": document_id, "new_version": new_version, "version_id": versi
        """, (new_version, version_id, document_id))

        # Text und Embedding neu generieren
        text = extract_text(file_path)
        self.db.execute("""
            FOR doc_text IN document_text
              FILTER doc_text.document_id == @document_id
              UPDATE doc_text WITH {content: @text} IN document_text
        """, {"document_id": document_id, "text": text})

        embedding = generate_embedding(text)
        self.db.execute("""
            FOR doc_emb IN document_embeddings
              FILTER doc_emb.document_id == @document_id
              UPDATE doc_emb WITH {embedding: @embedding} IN document_embeddings
        """, {"document_id": document_id, "embedding": embedding})

        log_action("VERSION_ADD", "document", document_id, None, {"version": new_version}

    return version_id

def search(self, query, doc_type=None, limit=20):
    """Hybrid-Suche: Volltext + Vector"""
    # 1. Fulltext-Suche
    sql = """
        SELECT d.id, d.title, d.type,
               ts_rank(to_tsvector('german', dt.content), plainto_tsquery('german', ?)) a
        FROM documents d
        JOIN document_text dt ON d.id = dt.document_id
        WHERE d.tenant_id = ? AND to_tsvector('german', dt.content) @@ plainto_tsquery('g
    """
    params = [query, get_current_tenant_id(), query]

    if doc_type:
        sql += " AND d.type = ?"
        params.append(doc_type)

    sql += " ORDER BY rank DESC LIMIT ?"
    params.append(limit)

    fulltext_results = self.db.execute(sql, params)

    # 2. Vector-Suche (semantische Ähnlichkeit)
    query_embedding = generate_embedding(query)
    vector_results = self.db.execute("""
        SELECT d.id, d.title, d.type,
               1 - (de.embedding <=> ?) as similarity
        FROM documents d
        JOIN document_embeddings de ON d.id = de.document_id
        WHERE d.tenant_id = ?
```

```
            ORDER BY similarity DESC
            LIMIT ?
        """, (query_embedding, get_current_tenant_id(), limit))

        # 3. Merge Results (kombiniere Scores)
        merged = merge_search_results(fulltext_results, vector_results)
        return merged

    def get_version_history(self, document_id):
        """Versionsverlauf abrufen"""
        return self.db.execute("""
            SELECT v.*, u.name as created_by_name
            FROM document_versions v
            LEFT JOIN users u ON v.created_by = u.id
            WHERE v.document_id = ?
            ORDER BY v.version DESC
        """, (document_id,))
```

Workflow-Management

Workflows als Graph modellieren:

```
class WorkflowEngine:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Workflow-Definitionen (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflows (
                id UUID PRIMARY KEY,
                name TEXT NOT NULL,
                description TEXT,
                active BOOLEAN DEFAULT true
            )
        """)

        # Workflow-Schritte als Graph
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflow_nodes (
                id UUID PRIMARY KEY,
                workflow_id UUID NOT NULL,
                name TEXT NOT NULL,
                node_type TEXT CHECK (node_type IN ('start', 'approval', 'task', 'decision',
                role_required TEXT,
                FOREIGN KEY (workflow_id) REFERENCES workflows(id)
            )
        """)

        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflow_edges (
                from_node UUID,
                to_node UUID,
                condition TEXT, -- Optional: JSON mit Bedingung
                PRIMARY KEY (from_node, to_node),
                FOREIGN KEY (from_node) REFERENCES workflow_nodes(id),
                FOREIGN KEY (to_node) REFERENCES workflow_nodes(id)
            )
        """)

        # Workflow-Instanzen (laufende Prozesse)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflow_instances (
                id UUID PRIMARY KEY,
                workflow_id UUID NOT NULL,
                document_id UUID NOT NULL,
                current_node UUID NOT NULL,
                state TEXT DEFAULT 'running',
                started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                completed_at TIMESTAMP,
                FOREIGN KEY (workflow_id) REFERENCES workflows(id),
                FOREIGN KEY (document_id) REFERENCES documents(id),
                FOREIGN KEY (current_node) REFERENCES workflow_nodes(id)
            )
        """)
```

```
        # Workflow-Historie
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS workflow_history (
                id UUID PRIMARY KEY,
                instance_id UUID NOT NULL,
                node_id UUID NOT NULL,
                user_id UUID,
                action TEXT,
                comment TEXT,
                timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (instance_id) REFERENCES workflow_instances(id)
            )
        """)

    def create_workflow(self, name, description, steps):
        """Workflow definieren

        steps = [
            {"name": "Erfassung", "type": "start"},
            {"name": "Manager-Prüfung", "type": "approval", "role": "manager"},
            {"name": "Direktor-Freigabe", "type": "approval", "role": "director"},
            {"name": "Abgeschlossen", "type": "end"}
        ]
        """
        workflow_id = uuid.uuid4()

        with self.db.transaction():
            self.db.execute("""
                INSERT {
                  id: @workflow_id,
                  name: @name,
                  description: @description
                } INTO workflows
            """, {"workflow_id": workflow_id, "name": name, "description": description})

            # Nodes erstellen
            node_ids = []
            for step in steps:
                node_id = uuid.uuid4()
                self.db.execute("""
                    INSERT {
                      id: @node_id,
                      workflow_id: @workflow_id,
                      name: @name,
                      node_type: @node_type,
                      role_required: @role_required
                    } INTO workflow_nodes
                """, {
                    "node_id": node_id,
                    "workflow_id": workflow_id,
                    "name": step['name'],
                    "node_type": step['type'],
                    "role_required": step.get('role')
```

```
            })
            node_ids.append(node_id)

        # Edges erstellen (linearer Flow)
        for i in range(len(node_ids) - 1):
            self.db.execute("""
                INSERT {
                  from_node: @from_node,
                  to_node: @to_node
                } INTO workflow_edges
            """, {"from_node": node_ids[i], "to_node": node_ids[i+1]})

    return workflow_id

def start_workflow(self, workflow_id, document_id):
    """Workflow für Dokument starten"""
    # Start-Node finden
    start_node = self.db.execute("""
        SELECT id FROM workflow_nodes
        WHERE workflow_id = ? AND node_type = 'start'
    """, (workflow_id,))[0]

    instance_id = uuid.uuid4()
    self.db.execute("""
        INSERT INTO workflow_instances (id, workflow_id, document_id, current_node)
        VALUES (?, ?, ?, ?)
    """, (instance_id, workflow_id, document_id, start_node['id']))

    # Dokumentstatus aktualisieren
    self.db.execute("UPDATE documents SET workflow_state = 'in_progress' WHERE id = ?", (

    # Nächsten Schritt finden und zuweisen
    self._advance_workflow(instance_id)

    return instance_id

def _advance_workflow(self, instance_id):
    """Workflow zum nächsten Schritt bewegen"""
    instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance

    # Nächster Node via Graph-Traversierung
    next_nodes = self.db.execute("""
        SELECT wn.* FROM workflow_edges we
        JOIN workflow_nodes wn ON we.to_node = wn.id
        WHERE we.from_node = ?
    """, (instance['current_node'],))

    if not next_nodes:
        # Ende erreicht
        self.db.execute("""
            UPDATE workflow_instances
            SET state = 'completed', completed_at = CURRENT_TIMESTAMP
            WHERE id = ?
        """, (instance_id,))
```

```
            self.db.execute("UPDATE documents SET workflow_state = 'completed' WHERE id = ?",
            return

        next_node = next_nodes[0]
        self.db.execute("UPDATE workflow_instances SET current_node = ? WHERE id = ?",
                       (next_node['id'], instance_id))

        # Task an User mit entsprechender Rolle zuweisen
        if next_node['role_required']:
            self._assign_task(instance_id, next_node['id'], next_node['role_required'])

    def approve(self, instance_id, user_id, comment=""):
        """Workflow-Schritt genehmigen"""
        instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (instance
        current_node = self.db.execute("SELECT * FROM workflow_nodes WHERE id = ?", (instance

        # Permission check
        if current_node['role_required']:
            if not user_has_role(user_id, current_node['role_required']):
                raise PermissionError(f"User benötigt Rolle: {current_node['role_required']}"

        with self.db.transaction():
            # Historie speichern
            self.db.execute("""
                INSERT INTO workflow_history (id, instance_id, node_id, user_id, action, comm
                VALUES (?, ?, ?, ?, ?, ?)
            """, (uuid.uuid4(), instance_id, current_node['id'], user_id, "APPROVED", comment

            # Zum nächsten Schritt
            self._advance_workflow(instance_id)

            log_action("WORKFLOW_APPROVE", "workflow_instance", instance_id, None, {
                "node": current_node['name'],
                "user": user_id
            })

    def reject(self, instance_id, user_id, reason):
        """Workflow-Schritt ablehnen"""
        with self.db.transaction():
            instance = self.db.execute("SELECT * FROM workflow_instances WHERE id = ?", (inst

            self.db.execute("""
                INSERT INTO workflow_history (id, instance_id, node_id, user_id, action, comm
                VALUES (?, ?, ?, ?, ?, ?)
            """, (uuid.uuid4(), instance_id, instance['current_node'], user_id, "REJECTED", r

            self.db.execute("UPDATE workflow_instances SET state = 'rejected' WHERE id = ?",
            self.db.execute("UPDATE documents SET workflow_state = 'rejected' WHERE id = ?",

            log_action("WORKFLOW_REJECT", "workflow_instance", instance_id, None, {"reason":

    def get_pending_tasks(self, user_id):
        """Offene Tasks für User"""
        # User-Rollen ermitteln
```

```
        user_roles = self.db.execute("SELECT role_name FROM user_roles WHERE user_id = ?", (u
        role_names = [r['role_name'] for r in user_roles]

        # Workflows in passenden Nodes
        return self.db.execute("""
            SELECT wi.*, d.title as document_title, wn.name as step_name
            FROM workflow_instances wi
            JOIN workflow_nodes wn ON wi.current_node = wn.id
            JOIN documents d ON wi.document_id = d.id
            WHERE wi.state = 'running'
            AND wn.role_required IN ({})
            ORDER BY wi.started_at
        """.format(','.join('?' * len(role_names))), role_names)
```

API-Implementierung

```
from flask import Flask, request, jsonify
from werkzeug.security import check_password_hash
import jwt

app = Flask(__name__)
db = ThemisDB("dms_erp.db")
doc_manager = Document(db)
workflow_engine = WorkflowEngine(db)

def authenticate(username, password):
    user = db.execute("SELECT * FROM users WHERE username = ?", (username,))
    if not user or not check_password_hash(user[0]['password'], password):
        return None
    token = jwt.encode({'user_id': str(user[0]['id'])}, app.config['SECRET_KEY'])
    return token

@app.route('/api/auth/login', methods=['POST'])
def login():
    data = request.json
    token = authenticate(data['username'], data['password'])
    if token:
        return jsonify({'token': token})
    return jsonify({'error': 'Invalid credentials'}), 401

def require_auth(f):
    def wrapper(*args, **kwargs):
        token = request.headers.get('Authorization', '').replace('Bearer ', '')
        try:
            payload = jwt.decode(token, app.config['SECRET_KEY'], algorithms=['HS256'])
            request.user_id = payload['user_id']
            return f(*args, **kwargs)
        except:
            return jsonify({'error': 'Unauthorized'}), 401
    return wrapper

@app.route('/api/documents', methods=['POST'])
@require_auth
@requires_permission('create:document')
def create_document():
    file = request.files['file']
    metadata = request.form.get('metadata', '{}')

    # Datei speichern
    file_path = save_uploaded_file(file)

    doc_id = doc_manager.create_document(
        title=file.filename,
        doc_type=request.form.get('type', 'general'),
        file_path=file_path,
        metadata=json.loads(metadata),
        owner_id=request.user_id
    )

    return jsonify({'id': str(doc_id), 'message': 'Document created'}), 201
```

```
@app.route('/api/documents/<doc_id>', methods=['GET'])
@require_auth
def get_document(doc_id):
    # Permission Check
    if not has_document_permission(request.user_id, doc_id, 'read'):
        return jsonify({'error': 'Forbidden'}), 403

    doc = db.execute("SELECT * FROM documents WHERE id = ?", (doc_id,))[0]
    versions = doc_manager.get_version_history(doc_id)

    return jsonify({
        'document': doc,
        'versions': versions
    })

@app.route('/api/documents/search', methods=['GET'])
@require_auth
def search_documents():
    query = request.args.get('q', '')
    doc_type = request.args.get('type')

    results = doc_manager.search(query, doc_type)
    return jsonify({'results': results})

@app.route('/api/workflows/<workflow_id>/start', methods=['POST'])
@require_auth
def start_workflow_api(workflow_id):
    data = request.json
    instance_id = workflow_engine.start_workflow(workflow_id, data['document_id'])
    return jsonify({'instance_id': str(instance_id)})

@app.route('/api/workflows/tasks', methods=['GET'])
@require_auth
def get_my_tasks():
    tasks = workflow_engine.get_pending_tasks(request.user_id)
    return jsonify({'tasks': tasks})

@app.route('/api/workflows/instances/<instance_id>/approve', methods=['POST'])
@require_auth
def approve_workflow(instance_id):
    data = request.json
    workflow_engine.approve(instance_id, request.user_id, data.get('comment', ''))
    return jsonify({'message': 'Approved'})

@app.route('/api/workflows/instances/<instance_id>/reject', methods=['POST'])
@require_auth
def reject_workflow(instance_id):
    data = request.json
    workflow_engine.reject(instance_id, request.user_id, data['reason'])
    return jsonify({'message': 'Rejected'})
```

OCR und Text-Extraktion

```
import pytesseract
from PIL import Image
import fitz  # PyMuPDF

def extract_text(file_path):
    """Text aus verschiedenen Formaten extrahieren"""
    ext = file_path.split('.')[-1].lower()

    if ext == 'pdf':
        return extract_text_from_pdf(file_path)
    elif ext in ('docx', 'doc'):
        return extract_text_from_docx(file_path)
    elif ext == 'txt':
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    else:
        return ""

def extract_text_from_pdf(file_path):
    """PDF Text-Extraktion"""
    text = []
    doc = fitz.open(file_path)
    for page in doc:
        text.append(page.get_text())
    return '\n'.join(text)

def perform_ocr(file_path):
    """OCR für gescannte Dokumente"""
    if file_path.endswith('.pdf'):
        # PDF → Bilder → OCR
        doc = fitz.open(file_path)
        ocr_text = []
        for page_num in range(len(doc)):
            page = doc[page_num]
            pix = page.get_pixmap()
            img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples)
            text = pytesseract.image_to_string(img, lang='deu')
            ocr_text.append(text)
        return '\n'.join(ocr_text)
    else:
        # Direktes Bild
        img = Image.open(file_path)
        return pytesseract.image_to_string(img, lang='deu')
```

10.4 CRM-System (Example 17)

Customer Relationship Management komplett in ThemisDB.

Datenmodell

```
class CRM:
    def __init__(self, db):
        self.db = db
        self.init_schema()

    def init_schema(self):
        # Kunden (Relational)
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS customers (
                id UUID PRIMARY KEY,
                tenant_id UUID NOT NULL,
                name TEXT NOT NULL,
                type TEXT CHECK (type IN ('individual', 'company')),
                industry TEXT,
                revenue DECIMAL,
                employees INT,
                website TEXT,
                status TEXT DEFAULT 'active',
                lead_score INT DEFAULT 0,
                lifetime_value DECIMAL DEFAULT 0,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Kontaktpersonen
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS contacts (
                id UUID PRIMARY KEY,
                customer_id UUID NOT NULL,
                first_name TEXT,
                last_name TEXT,
                email TEXT,
                phone TEXT,
                position TEXT,
                is_primary BOOLEAN DEFAULT false,
                FOREIGN KEY (customer_id) REFERENCES customers(id)
            )
        """)

        # Leads / Opportunities
        self.db.execute("""
            CREATE TABLE IF NOT EXISTS opportunities (
                id UUID PRIMARY KEY,
                customer_id UUID NOT NULL,
                title TEXT NOT NULL,
                value DECIMAL NOT NULL,
                stage TEXT NOT NULL, -- prospecting, qualification, proposal, negotiation, cl
                probability INT CHECK (probability BETWEEN 0 AND 100),
                expected_close_date DATE,
                assigned_to UUID,
                source TEXT, -- website, referral, cold_call, etc.
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (customer_id) REFERENCES customers(id)
```

```
        )
    """)

    # Activities (Time-Series)
    self.db.execute("""
        CREATE TABLE IF NOT EXISTS activities (
            id UUID PRIMARY KEY,
            customer_id UUID,
            opportunity_id UUID,
            type TEXT NOT NULL, -- email, call, meeting, note
            subject TEXT,
            description TEXT,
            duration INT, -- Minuten
            completed BOOLEAN DEFAULT false,
            scheduled_at TIMESTAMP,
            completed_at TIMESTAMP,
            created_by UUID,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (customer_id) REFERENCES customers(id),
            FOREIGN KEY (opportunity_id) REFERENCES opportunities(id)
        )
    """)
    # Index für Timeline
    self.db.execute("CREATE INDEX idx_activities_timeline ON activities(customer_id, crea

    # Notes / Interaktionen (Document Model)
    self.db.execute("""
        CREATE TABLE IF NOT EXISTS customer_notes (
            id UUID PRIMARY KEY,
            customer_id UUID NOT NULL,
            note JSON NOT NULL, -- {author, text, attachments, tags}
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (customer_id) REFERENCES customers(id)
        )
    """)

def create_customer(self, name, customer_type, **kwargs):
    """Neuen Kunden anlegen"""
    customer_id = uuid.uuid4()

    self.db.execute("""
        INSERT INTO customers (id, tenant_id, name, type, industry, revenue, employees, w
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        customer_id,
        get_current_tenant_id(),
        name,
        customer_type,
        kwargs.get('industry'),
        kwargs.get('revenue'),
        kwargs.get('employees'),
        kwargs.get('website')
    ))
```

```
        log_action("CREATE", "customer", customer_id, None, {"name": name})
        return customer_id

    def create_opportunity(self, customer_id, title, value, stage, **kwargs):
        """Lead/Opportunity erstellen"""
        opp_id = uuid.uuid4()

        self.db.execute("""
            INSERT INTO opportunities (id, customer_id, title, value, stage, probability, exp
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            opp_id,
            customer_id,
            title,
            value,
            stage,
            kwargs.get('probability', self._calculate_probability(stage)),
            kwargs.get('expected_close_date'),
            kwargs.get('assigned_to', get_current_user_id()),
            kwargs.get('source')
        ))

        # Lead Score aktualisieren
        self._update_lead_score(customer_id)

        return opp_id

    def _calculate_probability(self, stage):
        """Wahrscheinlichkeit basierend auf Stage"""
        probabilities = {
            'prospecting': 10,
            'qualification': 25,
            'proposal': 50,
            'negotiation': 75,
            'closed_won': 100,
            'closed_lost': 0
        }
        return probabilities.get(stage, 0)

    def _update_lead_score(self, customer_id):
        """Lead-Scoring berechnen"""
        # Faktoren:
        # - Anzahl Opportunities
        # - Gesamtwert offener Opportunities
        # - Anzahl Activities
        # - Durchschnittliche Wahrscheinlichkeit

        score_data = self.db.execute("""
            SELECT
                COUNT(DISTINCT o.id) as opp_count,
                COALESCE(SUM(o.value), 0) as total_value,
                COALESCE(AVG(o.probability), 0) as avg_probability,
                COUNT(DISTINCT a.id) as activity_count
            FROM customers c
```

```
            LEFT JOIN opportunities o ON c.id = o.customer_id AND o.stage NOT IN ('closed_wor
            LEFT JOIN activities a ON c.id = a.customer_id
            WHERE c.id = ?
            GROUP BY c.id
        """, (customer_id,))[0]

        # Simple Scoring-Formel
        score = (
            score_data['opp_count'] * 10 +
            min(score_data['total_value'] / 1000, 50) +
            score_data['avg_probability'] / 2 +
            score_data['activity_count'] * 2
        )
        score = min(int(score), 100)

        self.db.execute("UPDATE customers SET lead_score = ? WHERE id = ?", (score, customer_

    def log_activity(self, customer_id, activity_type, subject, description, **kwargs):
        """Aktivität loggen"""
        activity_id = uuid.uuid4()

        self.db.execute("""
            INSERT INTO activities (id, customer_id, opportunity_id, type, subject, descripti
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            activity_id,
            customer_id,
            kwargs.get('opportunity_id'),
            activity_type,
            subject,
            description,
            kwargs.get('duration'),
            kwargs.get('scheduled_at'),
            get_current_user_id()
        ))

        return activity_id

    def complete_activity(self, activity_id):
        """Aktivität als erledigt markieren"""
        self.db.execute("""
            UPDATE activities
            SET completed = true, completed_at = CURRENT_TIMESTAMP
            WHERE id = ?
        """, (activity_id,))

    def get_customer_timeline(self, customer_id, limit=50):
        """Komplette Timeline für Kunde"""
        return self.db.execute("""
            SELECT
                'activity' as item_type,
                id,
                type,
                subject,
```

```
            created_at as timestamp
        FROM activities
        WHERE customer_id = ?

        UNION ALL

        SELECT
            'note' as item_type,
            id,
            'note' as type,
            note->>'text' as subject,
            created_at as timestamp
        FROM customer_notes
        WHERE customer_id = ?

        UNION ALL

        SELECT
            'opportunity' as item_type,
            id,
            'opportunity' as type,
            title as subject,
            created_at as timestamp
        FROM opportunities
        WHERE customer_id = ?

        ORDER BY timestamp DESC
        LIMIT ?
    """, (customer_id, customer_id, customer_id, limit))

def get_sales_pipeline(self):
    """Sales-Pipeline Übersicht"""
    return self.db.execute("""
        SELECT
            stage,
            COUNT(*) as count,
            SUM(value) as total_value,
            AVG(probability) as avg_probability
        FROM opportunities
        WHERE stage NOT IN ('closed_won', 'closed_lost')
        GROUP BY stage
        ORDER BY
            CASE stage
                WHEN 'prospecting' THEN 1
                WHEN 'qualification' THEN 2
                WHEN 'proposal' THEN 3
                WHEN 'negotiation' THEN 4
            END
    """)

def forecast_revenue(self, months=3):
    """Revenue-Forecast"""
    return self.db.execute("""
        SELECT
```

```
                DATE_TRUNC('month', expected_close_date) as month,
                SUM(value * probability / 100) as weighted_value,
                SUM(value) as total_value,
                COUNT(*) as opportunity_count
            FROM opportunities
            WHERE expected_close_date >= CURRENT_DATE
            AND expected_close_date <= CURRENT_DATE + INTERVAL '? months'
            AND stage NOT IN ('closed_won', 'closed_lost')
            GROUP BY month
            ORDER BY month
        """, (months,))

    def get_top_customers(self, limit=10):
        """Top-Kunden nach Lifetime Value"""
        return self.db.execute("""
            SELECT
                c.*,
                COUNT(DISTINCT o.id) as opportunity_count,
                SUM(CASE WHEN o.stage = 'closed_won' THEN o.value ELSE 0 END) as won_value,
                COUNT(DISTINCT a.id) as activity_count
            FROM customers c
            LEFT JOIN opportunities o ON c.id = o.customer_id
            LEFT JOIN activities a ON c.id = a.customer_id
            WHERE c.tenant_id = ?
            GROUP BY c.id
            ORDER BY won_value DESC, c.lead_score DESC
            LIMIT ?
        """, (get_current_tenant_id(), limit))
```

```
class CRMDashboard:
    def __init__(self, crm):
        self.crm = crm
        self.db = crm.db

    def get_overview(self):
        """Dashboard Übersicht"""
        return {
            'customers': self._get_customer_stats(),
            'pipeline': self.crm.get_sales_pipeline(),
            'activities': self._get_activity_stats(),
            'forecast': self.crm.forecast_revenue(3)
        }

    def _get_customer_stats(self):
        return self.db.execute("""
            SELECT
                COUNT(*) as total,
                COUNT(CASE WHEN status = 'active' THEN 1 END) as active,
                COUNT(CASE WHEN lead_score >= 70 THEN 1 END) as hot_leads,
                AVG(lead_score) as avg_lead_score
            FROM customers
            WHERE tenant_id = ?
        """, (get_current_tenant_id(),))[0]

    def _get_activity_stats(self):
        return self.db.execute("""
            SELECT
                type,
                COUNT(*) as count,
                COUNT(CASE WHEN completed THEN 1 END) as completed
            FROM activities
            WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
            GROUP BY type
        """)

    def generate_report(self, report_type, start_date, end_date):
        """Verschiedene Reports generieren"""
        if report_type == 'sales':
            return self._sales_report(start_date, end_date)
        elif report_type == 'activities':
            return self._activity_report(start_date, end_date)
        elif report_type == 'conversion':
            return self._conversion_report(start_date, end_date)

    def _sales_report(self, start_date, end_date):
        """Verkaufs-Report"""
        return self.db.execute("""
            SELECT
                DATE_TRUNC('week', created_at) as week,
                COUNT(*) as opportunities_created,
                COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as won,
                SUM(CASE WHEN stage = 'closed_won' THEN value ELSE 0 END) as won_value,
                COUNT(CASE WHEN stage = 'closed_lost' THEN 1 END) as lost
```

```
            FROM opportunities
            WHERE created_at BETWEEN ? AND ?
            GROUP BY week
            ORDER BY week
        """, (start_date, end_date))

    def _conversion_report(self, start_date, end_date):
        """Conversion-Funnel"""
        return self.db.execute("""
            SELECT
                source,
                COUNT(*) as total,
                COUNT(CASE WHEN stage = 'closed_won' THEN 1 END) as won,
                ROUND(COUNT(CASE WHEN stage = 'closed_won' THEN 1 END)::NUMERIC / COUNT(*) *
                AVG(value) as avg_deal_size
            FROM opportunities
            WHERE created_at BETWEEN ? AND ?
            GROUP BY source
            ORDER BY conversion_rate DESC
        """, (start_date, end_date))
```

10.5 Best Practices für Enterprise-Anwendungen

1. Performance bei großen Datenmengen

```
db.execute("""
    CREATE TABLE activities_2025_01 PARTITION OF activities
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01')
""")

db.execute("""
    CREATE MATERIALIZED VIEW sales_summary AS
    SELECT
        DATE_TRUNC('month', created_at) as month,
        stage,
        COUNT(*) as count,
        SUM(value) as total_value
    FROM opportunities
    GROUP BY month, stage
""")

db.execute("REFRESH MATERIALIZED VIEW sales_summary")
```

2. Caching-Strategie

```
import redis

cache = redis.Redis()

def get_dashboard_data_cached():
    cached = cache.get('dashboard:overview')
    if cached:
        return json.loads(cached)

    # Berechnen
    data = dashboard.get_overview()

    # Cache für 5 Minuten
    cache.setex('dashboard:overview', 300, json.dumps(data))
    return data
```

3. Background-Jobs

```
from celery import Celery

celery = Celery('enterprise_app')

@celery.task
def generate_large_report(report_id):
    """Großer Report im Hintergrund"""
    report_data = dashboard.generate_report('sales', start_date, end_date)

    # PDF generieren
    pdf_path = create_pdf_report(report_data)

    # In DB speichern
    db.execute("""
        UPDATE reports
        SET status = 'completed', file_path = ?
        WHERE id = ?
    """, (pdf_path, report_id))

    # User benachrichtigen
    send_notification(user_id, f"Report {report_id} ist fertig")

@celery.task
def update_lead_scores():
    """Lead-Scores täglich neu berechnen"""
    customers = db.execute("SELECT id FROM customers WHERE status = 'active'")
    for customer in customers:
        crm._update_lead_score(customer['id'])
```

4. API-Rate-Limiting

```
from flask_limiter import Limiter

limiter = Limiter(app, key_func=lambda: request.user_id)

@app.route('/api/documents/search')
@limiter.limit("100/hour")
@require_auth
def search_api():
    # Maximal 100 Searches pro Stunde pro User
    pass
```

5. Monitoring und Alerting

```
import statsd
from prometheus_client import Counter, Histogram

document_uploads = Counter('document_uploads_total', 'Total document uploads')
query_duration = Histogram('query_duration_seconds', 'Query execution time')

@app.route('/api/documents', methods=['POST'])
@require_auth
def create_document():
        document_uploads.inc()

        with query_duration.time():
                doc_id = doc_manager.create_document(...)

        return jsonify({'id': str(doc_id)})

@app.route('/health')
def health():
        try:
                db.execute("SELECT 1")
                return jsonify({'status': 'healthy'}), 200
        except:
                return jsonify({'status': 'unhealthy'}), 500
```

10.5 Enterprise Compliance & Audit (Neu)

Compliance-Anforderungen (DSGVO, BAIT, ISO 27001) verlangen nachvollziehbare Änderungen, Daten-Löschkonzepte und strikte Trennung zwischen Tenants und Regionen.

10.5.1 Audit-Log Architektur

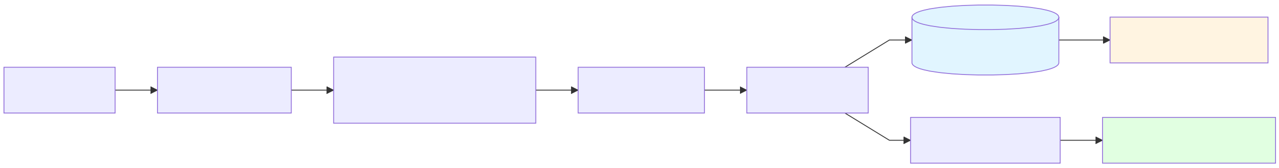


Abb. 32: Ablaufdiagramm: 10.5.1 Audit-Log Architektur

10.5.2 Audit-Log Schema (RocksDB Prefix)

Field	Type	Description
`tenant_id`	string	Mandant (Tenant)
`actor`	string	User/Service Principal
`action`	string	z.B. `documents.create` , `users.update`
`resource`	string	Resource Identifier (z.B. `doc:123`)
`ts`	int64	Unix ms Timestamp
`ip`	string	Client-IP
`status`	string	`success` / `failure`
`before`	json	Optional: Vorheriger Zustand
`after`	json	Optional: Neuer Zustand

Key Layout: `audit:{tenant}:{ts}:{uuid}` → Prefix-Scan pro Tenant + Zeitraum.

10.5.3 DSGVO: Löschung & Aufbewahrung

```
-- Aufbewahrungsregel pro Tenant (Config-Collection)
FOR cfg IN audit_policies
    FILTER cfg.tenant_id == @tenant
    RETURN cfg.retention_days

-- Lösch-Job (täglich)
LET cutoff = DATE_NOW() - DAYS(@retention_days)
FOR entry IN audit
    FILTER entry.tenant_id == @tenant
    FILTER entry.ts < cutoff
    REMOVE entry IN audit
```

Double-Delete: Nach Log-Löschung optional `after` -Payload mit Null-Werten überschreiben, falls rechtlich gefordert.

10.5.4 BAIT/ISO 27001 Mapping (Auszug)

- **Protokollierung:** Jede sicherheitsrelevante Aktion → Audit-Log (BAIT 8.1)

- **Unveränderbarkeit:** Write-Once-Append oder WORM-Bucket für Export (ISO 27001 A.12.4)
- **Trennung:** Tenant-ID im Key, optionale Region im Prefix `audit:eu-central-1:tenant:...`
- **Nachvollziehbarkeit:** Request-ID, Correlation-ID, Actor, IP
- **Rechteprüfung:** RBAC-Check vor Schreiboperationen, Audit-Log auch für abgelehnte Zugriffe

Audit-Exporte:

```
aws s3 cp audit.json s3://themis-audit/2025/12/31/ \
  --object-lock-mode COMPLIANCE \
  --object-lock-retain-until-date 2032-12-31
```

10.5.6 Tenant & Region Sharding

```
audit_storage:
    strategy: by_region_and_tenant
    regions:
        - id: eu-central-1
            bucket: s3://themis-audit-eu
        - id: us-east-1
            bucket: s3://themis-audit-us

rocksdb_prefixes:
    eu: "audit:eu-central-1:"
    us: "audit:us-east-1:"
```

10.5.7 Editions & Distribution (Kurzüberblick)

Edition	Features	Zielgruppe
Community	Core Storage, AQL, Basic Indizes	Entwickler, PoC
Enterprise	RBAC, Audit, HA/Failover, Advanced Backup	Unternehmen
Gov/Regulated	Region Sharding, WORM-Export, Langzeitaufbewahrung	Finanz, Behörden

Distribution Best Practice: 1. **Core Binary** per Artifact Registry ausliefern 2. **Regionale Add-ons** (Compliance-Pakete) nur für passende Regionen aktivieren 3. **Config-as-Code**: Tenant-Policies versionieren

---

10.6 Zusammenfassung

In diesem Kapitel haben Sie gelernt:

- **Multi-Tenancy**: Verschiedene Ansätze und Best Practices
- **RBAC**: Rollen-basierte Zugriffskontrolle implementieren

- **Audit-Logging**: Vollständige Nachvollziehbarkeit aller Änderungen
- **Workflow-Management**: Geschäftsprozesse als Graph modellieren
- **DMS/ERP**: Dokumentenmanagement mit Versionierung und OCR
- **CRM**: Customer Relationship Management mit Lead-Scoring
- **Enterprise Best Practices**: Performance, Caching, Monitoring

Wichtige Erkenntnisse:

1. **Multi-Model vereinfacht Enterprise-Apps** - Eine Datenbank für alle Anforderungen
2. **Graph-Modell für Workflows** - Prozesse als Graph modellieren
3. **Audit-Log als First-Class Citizen** - Nicht nachrüsten, von Anfang an einplanen
4. **Hybrid-Search unverzichtbar** - Volltext + Vector Search kombinieren
5. **Background-Jobs für schwere Operations** - API bleibt responsiv

Übungen:

1. Erweitern Sie das DMS-System um E-Signaturen
2. Implementieren Sie Lead-Scoring mit Machine Learning
3. Erstellen Sie einen Workflow-Designer (GUI)
4. Integrieren Sie Email-Tracking ins CRM
5. Implementieren Sie Webhooks für externe Integrationen

Im nächsten Kapitel tauchen wir in Realtime-Anwendungen ein: Chat und Kanban-Boards mit Live-Updates!

Kapitel 12: Chapter 11: Realtime Data Streaming mit Change Data Capture

11.1 Einführung: Die Streaming-Architektur

Change Data Capture (CDC) ist eine kritische Komponente moderner Datenarchitekturen, die Echtzeit-Datenströme ermöglicht. ThemisDB implementiert CDC als natives Feature, das automatisch alle Mutationen (CREATE, UPDATE, DELETE) in einem append-only Event Log aufzeichnet. Dies ist ein fundamentaler Unterschied zu polyglot Systemen, die externe Tools wie Debezium benötigen, um Änderungen aus dem Write-Ahead Log (WAL) zu extrahieren.

Warum CDC in ThemisDB?

Architektonischer Vorteil: Da ThemisDB alle Datenmodelle (relational, graph, document, vector) in einem einheitlichen "Base Entity"-Format speichert (siehe Chapter 2), kann ein einzelner CDC-Stream **alle** Datentypen erfassen. In einem Polyglot-Setup müsste man separate CDC-Pipelines für PostgreSQL, Neo4j und ChromaDB betreiben, was die Komplexität vervielfacht und die Konsistenz gefährdet.

Use Cases: 1. **Real-Time Analytics:** Streaming von Datenänderungen in ein OLAP-System (z.B. ClickHouse) 2. **Event Sourcing:** Rekonstruktion des Anwendungszustands aus dem Event Log 3. **Materialized Views:** Automatische Aktualisierung von Aggregationen 4. **Audit Trails:** BSI-konforme Nachvollziehbarkeit aller Datenänderungen 5. **Cross-System Sync:** Spiegelung von Daten in externe Systeme (Elasticsearch, Redis Cache) 6. **Kafka Integration:** Streaming in ein zentrales Event-Bus-System

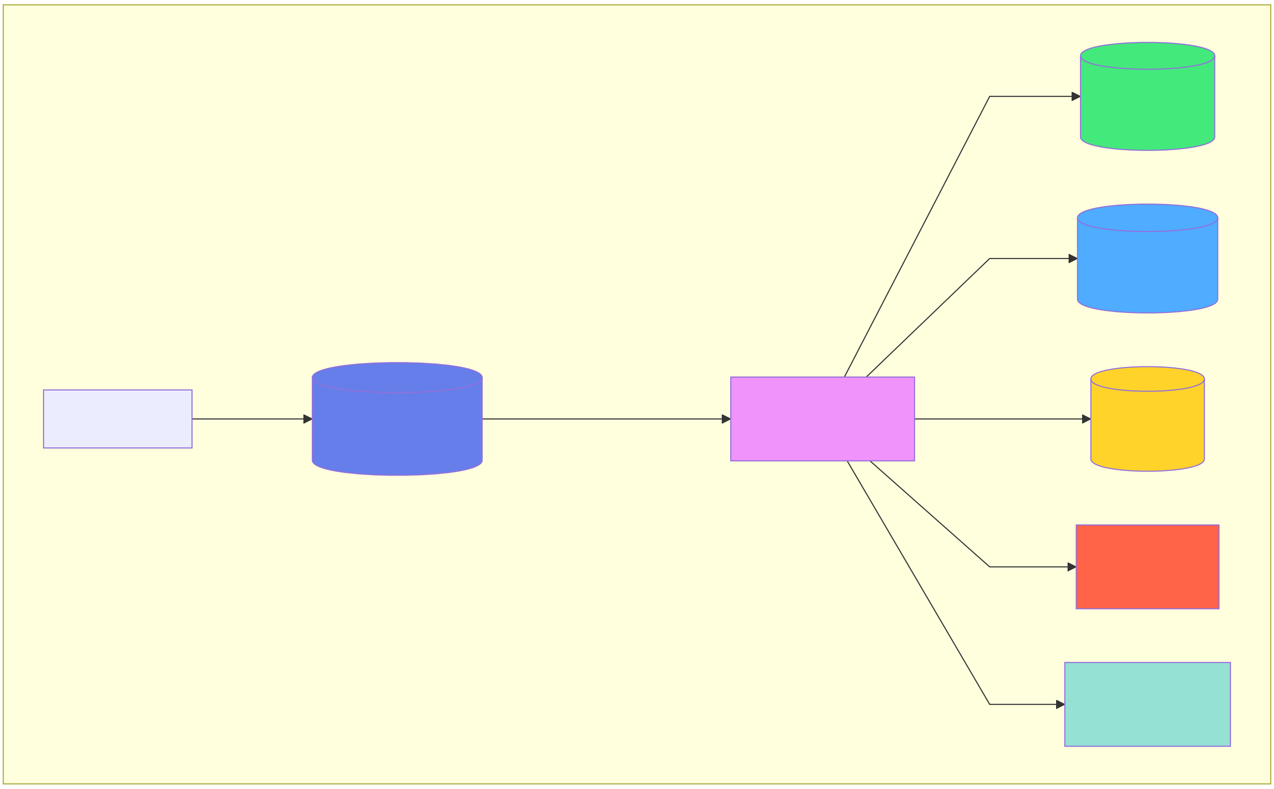


Abb. 33: Architekturdiagramm (Chapter 11: Realtime Data Streaming mit Change Data Capture)

11.2 CDC-Architektur und Datenmodell

11.2.1 Event-Struktur

Jedes CDC-Event in ThemisDB folgt einem standardisierten Schema, das maximale Flexibilität bietet:

```
{
  "sequence": 42,
  "type": "PUT",
  "key": "user:alice",
  "value": "{\"name\":\"Alice\",\"email\":\"alice@example.com\"}",
  "timestamp_ms": 1730294567123,
  "metadata": {
    "table": "user",
    "pk": "alice",
    "operation_type": "update",
    "user_id": "admin@example.com"
  }
}
```

Feld-Semantik:

- **sequence** (uint64): Monoton steigende ID, die **strikte Ordnung** garantiert. Diese Eigenschaft ist kritisch für die Konsistenz: Consumer können sicher sein, dass Event N vor Event N+1 aufgetreten ist. Die Sequence-Nummer wird atomar in

RocksDB verwaltet (`changefeed_sequence`-Schlüssel) und nutzt RocksDB's MVCC-Garantien.

- **type** (enum): `PUT` (Create/Update) oder `DELETE`. Zukünftige Erweiterungen könnten `TRANSACTION_COMMIT`/`TRANSACTION_ROLLBACK` für Transaktionsgrenzen hinzufügen, was Event Sourcing-Patterns vereinfachen würde.
- **key** (string): Vollständiger Entitätsschlüssel im Format `collection:primary_key`. Dies ermöglicht effiziente Filterung nach Präfixen (z.B. alle User-Events via `user:`-Filter).
- **value** (string|null): Bei PUT: JSON-serialisierte Entität. Bei DELETE: `null`. Die Speicherung als String (nicht als natives JSON-Objekt) minimiert den Serialisierungsaufwand im Hot Path. Consumer können mit simdjson/serde_json parsen.
- **timestamp_ms** (uint64): Millisekunden seit Unix Epoch. Wichtig: Dieser Timestamp basiert auf der Serverzeit zum Zeitpunkt des Commits, nicht auf Client-Zeit. Für verteilte Systeme sollte NTP-Synchronisation sichergestellt sein.
- **metadata** (JSON object): Frei erweiterbar. Standardfelder (`table`, `pk`) werden automatisch gesetzt. Anwendungen können zusätzliche Kontextinformationen hinzufügen (z.B. `user_id` für Audit-Trails, `source_ip` für Sicherheitsanalysen).

11.2.2 Physische Speicherung

CDC-Events werden im selben RocksDB-Backend wie die Base Entities gespeichert, jedoch in einem separaten Schlüsselraum:

Key-Format: `changefeed:{sequence_number}`

Die Sequence-Nummer wird Zero-padded (z.B. `changefeed:0000000000000042`), um lexikographische Sortierung zu garantieren. Dies ist kritisch für die Scan-Performance: Ein RocksDB-Iterator kann effizient von `changefeed:{from_seq}` bis `changefeed:{to_seq}` iterieren, ohne die Daten neu sortieren zu müssen.

Column Family Strategy:

- **Default:** Events werden in der Default Column Family gespeichert. Vorteil: Atomare Transaktionen mit den Base Entities sind trivial (ein RocksDB WriteBatch kann beide aktualisieren). Nachteil: Events und Entitäten konkurrieren um den Block Cache.
- **Dedicated CF (zukünftig):** Eine separate Column Family (`cf_changefeed`) würde isolierte Tuning-Parameter erlauben (z.B. aggressivere Compaction für Events, da sie nie aktualisiert werden).

11.2.3 Sequence-Management

Die atomare Sequenzvergabe ist der kritische Pfad für CDC-Schreiboperationen:

```
// Pseudo-Code: Atomare Sequence-Vergabe
rocksdb::WriteBatch batch;
uint64_t seq = increment_sequence_counter(batch); // Atomic increment
std::string event_key = format("changefeed:{:020d}", seq);
batch.Put(event_key, serialize_event(event));
db->Write(batch);  // ACID: Sequence & Event atomar committed
```

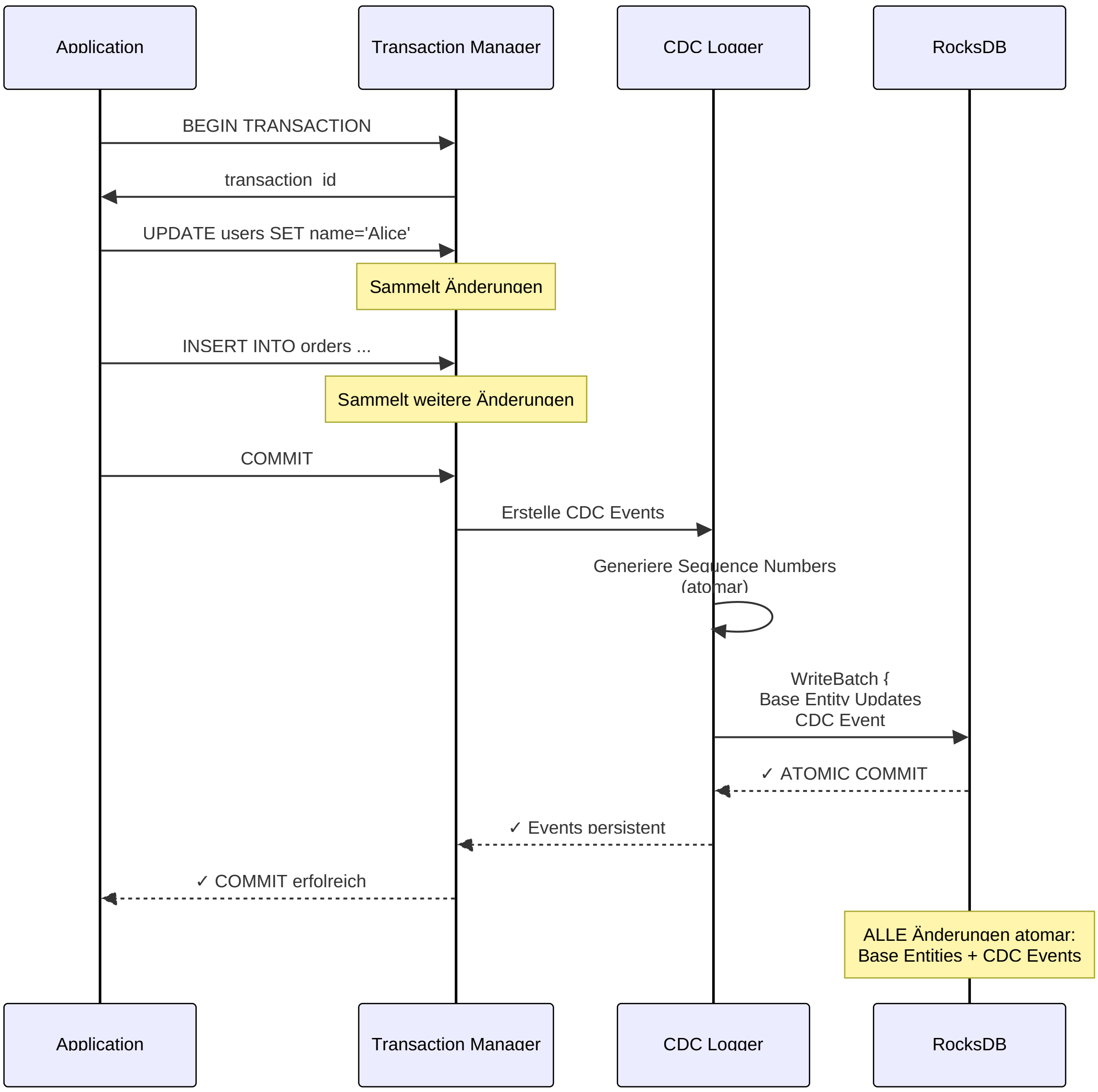


Abb. 34: Sequenzdiagramm (Chapter 11: Realtime Data Streaming mit Change Data Capture)

Performance-Trade-off: Die zentrale Sequenzvergabe ist ein Bottleneck bei hohen Schreibraten (>100K writes/sec). Zukünftige Optimierungen:

1. **Batch Allocation:** Reserviere Sequence-Blöcke (z.B. 1.000 Sequences auf einmal), reduziere Contentions um Faktor 1000.

2. **Sharded Sequences:** In Sharding-Setups (Chapter 16) kann jeder Shard eigene Sequences verwalten. Globale Ordnung wird via `(shard_id, local_sequence)` Tupel erreicht.

CDC-Optionen

```
stream = db.cdc.create_stream(
    name="user_updates",
    table="users",
    columns=["status", "last_seen"],  # Nur diese Felder
    operations=["UPDATE"]
)

stream = db.cdc.create_stream(
    name="important_messages",
    table="messages",
    filter="priority = 'high'",
    operations=["INSERT"]
)

stream = db.cdc.create_stream(
    name="enriched_events",
    table="orders",
    transform=lambda event: {
        **event,
        "customer_name": db.query("""
            FOR customer IN customers
              FILTER customer.id == @customer_id
              LIMIT 1
              RETURN customer.name
        """, {"customer_id": event.data["customer_id"]})[0]
    }
)
```

Server-Sent Events (SSE)

SSE ermöglicht es, Daten vom Server an Clients zu pushen.

SSE-Server-Setup

```
from flask import Flask, Response
from themisdb import ThemisDB
import json
import time

app = Flask(__name__)
db = ThemisDB()

def event_stream(channel):
    """Generator für SSE-Events"""
    stream = db.cdc.create_stream(
        name=f"sse_{channel}",
        table=channel,
        operations=["INSERT", "UPDATE", "DELETE"]
    )

    for event in stream:
        # SSE-Format: "data: JSON\n\n"
        yield f"data: {json.dumps(event.to_dict())}\n\n"

@app.route('/stream/<channel>')
def stream(channel):
    return Response(
        event_stream(channel),
        mimetype="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no"  # Nginx
        }
    )
```

SSE-Client-Code

```javascript
// JavaScript Client
const eventSource = new EventSource('/stream/messages');

eventSource.onmessage = function(event) {
    const data = JSON.parse(event.data);

    if (data.operation === 'INSERT') {
        addMessageToUI(data.after);
    } else if (data.operation === 'UPDATE') {
        updateMessageInUI(data.after);
    } else if (data.operation === 'DELETE') {
        removeMessageFromUI(data.before.id);
    }
};

eventSource.onerror = function(error) {
    console.error('SSE error:', error);
    // Automatische Reconnection
};
```

WebSocket-Integration

Für bidirektionale Kommunikation sind WebSockets ideal.

```
from flask_socketio import SocketIO, emit, join_room, leave_room
from themisdb import ThemisDB

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

active_users = {}

@socketio.on('connect')
def handle_connect():
    print(f'Client connected: {request.sid}')

@socketio.on('join_channel')
def handle_join(data):
    channel = data['channel']
    username = data['username']

    join_room(channel)
    active_users[request.sid] = {
        'username': username,
        'channel': channel
    }

    # Benachrichtige andere
    emit('user_joined', {
        'username': username,
        'timestamp': time.time()
    }, room=channel, skip_sid=request.sid)

    # CDC-Stream für diesen Channel
    start_cdc_stream(channel)

def start_cdc_stream(channel):
    """Background-Thread für CDC → WebSocket"""
    stream = db.cdc.create_stream(
        name=f"ws_{channel}",
        table="messages",
        filter=f"channel = '{channel}'"
    )

    def emit_changes():
        for event in stream:
            socketio.emit('message_update',
                          event.to_dict(),
                          room=channel)

    # In separatem Thread
    socketio.start_background_task(emit_changes)

@socketio.on('send_message')
def handle_message(data):
    channel = active_users[request.sid]['channel']
    username = active_users[request.sid]['username']
```

```
    # In DB schreiben
    db.execute("""
        INSERT {
          channel: @channel,
          username: @username,
          content: @content,
          timestamp: @timestamp
        } INTO messages
    """, {"channel": channel, "username": username, "content": data['content'], "timestamp":

    # CDC wird automatisch notifizieren
```

WebSocket-Client

```
const socket = io('http://localhost:5000');

// Channel beitreten
socket.emit('join_channel', {
    channel: 'general',
    username: 'Alice'
});

// Nachrichten senden
function sendMessage(content) {
    socket.emit('send_message', {
        content: content
    });
}

// Nachrichten empfangen
socket.on('message_update', function(event) {
    if (event.operation === 'INSERT') {
        displayMessage(event.after);
    }
});

// Nutzer beigetreten
socket.on('user_joined', function(data) {
    console.log(`${data.username} ist beigetreten`);
});
```

Example: Realtime Chat (examples/18_realtime_chat)

Vollständige Chat-Anwendung mit Channels, Private Messages, Online-Status und Typing-Indikatoren.

Datenmodell

```
from themisdb import ThemisDB
from datetime import datetime

db = ThemisDB()

db.execute("""
    CREATE TABLE users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        email TEXT UNIQUE NOT NULL,
        avatar_url TEXT,
        status TEXT DEFAULT 'offline',  -- online, offline, away
        last_seen TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE channels (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        description TEXT,
        is_private BOOLEAN DEFAULT FALSE,
        created_by INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE channel_members (
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        role TEXT DEFAULT 'member',  -- admin, moderator, member
        PRIMARY KEY (channel_id, user_id)
    )
""")

db.execute("""
    CREATE TABLE messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        content TEXT NOT NULL,
        message_type TEXT DEFAULT 'text',  -- text, image, file, system
        reply_to INTEGER REFERENCES messages(id),
        edited_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_channel_time (channel_id, created_at),
        INDEX idx_user (user_id)
    )
""")

db.execute("""
```

```
    CREATE TABLE reactions (
        message_id INTEGER REFERENCES messages(id),
        user_id INTEGER REFERENCES users(id),
        emoji TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (message_id, user_id, emoji)
    )
""")

db.execute("""
    CREATE TABLE direct_messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        from_user_id INTEGER REFERENCES users(id),
        to_user_id INTEGER REFERENCES users(id),
        content TEXT NOT NULL,
        read_at TIMESTAMP,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_participants (from_user_id, to_user_id, created_at)
    )
""")

db.execute("""
    CREATE TABLE typing_indicators (
        channel_id INTEGER REFERENCES channels(id),
        user_id INTEGER REFERENCES users(id),
        started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (channel_id, user_id)
    )
""")
```

Chat-Backend

```
from flask import Flask, request, jsonify
from flask_socketio import SocketIO, emit, join_room, leave_room
from flask_cors import CORS
from themisdb import ThemisDB
import time
from datetime import datetime, timedelta

app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

connections = {}  # sid -> {user_id, username, channels}

class ChatService:
    @staticmethod
    def get_channel_messages(channel_id, limit=50, before_id=None):
        """Nachrichten laden (mit Pagination)"""
        query = """
            FOR message IN messages
              FILTER message.channel_id == @channel_id
              FOR user IN users
                FILTER message.user_id == user.id
                LET reaction_count = LENGTH(
                  FOR reaction IN reactions
                    FILTER reaction.message_id == message.id
                    RETURN 1
                )
                RETURN {
                  id: message.id,
                  content: message.content,
                  message_type: message.message_type,
                  reply_to: message.reply_to,
                  created_at: message.created_at,
                  edited_at: message.edited_at,
                  user_id: user.id,
                  username: user.username,
                  avatar_url: user.avatar_url,
                  reaction_count
                }
        """
        params = {"channel_id": channel_id}

        if before_id:
            query += " AND m.id < ?"
            params.append(before_id)

        query += " ORDER BY m.created_at DESC LIMIT ?"
        params.append(limit)

        return db.query(query, params)

    @staticmethod
    def send_message(channel_id, user_id, content, reply_to=None):
```

```
        """Nachricht senden"""
        with db.transaction():
            result = db.execute("""
                INSERT {
                  channel_id: @channel_id,
                  user_id: @user_id,
                  content: @content,
                  reply_to: @reply_to
                } INTO messages
                RETURN NEW
            """, {"channel_id": channel_id, "user_id": user_id, "content": content, "reply_to

            message = result[0]

            # Typing-Indikator löschen
            db.execute("""
                FOR indicator IN typing_indicators
                  FILTER indicator.channel_id == @channel_id AND indicator.user_id == @user_i
                  REMOVE indicator IN typing_indicators
            """, {"channel_id": channel_id, "user_id": user_id})

            return message

    @staticmethod
    def edit_message(message_id, user_id, new_content):
        """Nachricht bearbeiten"""
        result = db.execute("""
            UPDATE messages
            SET content = ?, edited_at = CURRENT_TIMESTAMP
            WHERE id = ? AND user_id = ?
            RETURNING *
        """, [new_content, message_id, user_id])

        if not result:
            raise ValueError("Message not found or not owned by user")

        return result[0]

    @staticmethod
    def delete_message(message_id, user_id):
        """Nachricht löschen"""
        db.execute("""
            FOR message IN messages
              FILTER message.id == @message_id AND message.user_id == @user_id
              REMOVE message IN messages
        """, {"message_id": message_id, "user_id": user_id})

    @staticmethod
    def add_reaction(message_id, user_id, emoji):
        """Reaktion hinzufügen"""
        try:
            db.execute("""
                INSERT {
                  message_id: @message_id,
```

```
                  user_id: @user_id,
                  emoji: @emoji
                } INTO reactions
            """, {"message_id": message_id, "user_id": user_id, "emoji": emoji})
            return True
        except:  # Already exists
            return False

    @staticmethod
    def get_online_users(channel_id):
        """Online-Nutzer in einem Channel"""
        return db.query("""
            SELECT DISTINCT u.id, u.username, u.avatar_url, u.status
            FROM users u
            JOIN channel_members cm ON u.id = cm.user_id
            WHERE cm.channel_id = ?
              AND u.status = 'online'
              AND u.last_seen > ?
        """, [channel_id, datetime.now() - timedelta(minutes=5)])

    @staticmethod
    def update_user_status(user_id, status):
        """Status aktualisieren"""
        db.execute("""
            UPDATE users
            SET status = ?, last_seen = CURRENT_TIMESTAMP
            WHERE id = ?
        """, [status, user_id])

@socketio.on('connect')
def handle_connect():
    print(f'Client connected: {request.sid}')

@socketio.on('authenticate')
def handle_authenticate(data):
    """Nutzer authentifizieren"""
    user_id = data['user_id']
    username = data['username']

    connections[request.sid] = {
        'user_id': user_id,
        'username': username,
        'channels': set()
    }

    # Status auf online setzen
    ChatService.update_user_status(user_id, 'online')

    emit('authenticated', {'success': True})

@socketio.on('join_channel')
def handle_join_channel(data):
    """Channel beitreten"""
    channel_id = data['channel_id']
```

```
    if request.sid not in connections:
        return emit('error', {'message': 'Not authenticated'})

    conn = connections[request.sid]
    join_room(f'channel_{channel_id}')
    conn['channels'].add(channel_id)

    # Online-Nutzer benachrichtigen
    emit('user_joined', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

    # Initiale Nachrichten laden
    messages = ChatService.get_channel_messages(channel_id)
    emit('channel_history', {
        'channel_id': channel_id,
        'messages': messages
    })

    # CDC-Stream für diesen Channel (wenn noch nicht aktiv)
    start_channel_cdc(channel_id)

@socketio.on('leave_channel')
def handle_leave_channel(data):
    """Channel verlassen"""
    channel_id = data['channel_id']

    if request.sid in connections:
        conn = connections[request.sid]
        leave_room(f'channel_{channel_id}')
        conn['channels'].discard(channel_id)

        emit('user_left', {
            'user_id': conn['user_id'],
            'username': conn['username'],
            'channel_id': channel_id
        }, room=f'channel_{channel_id}')

@socketio.on('send_message')
def handle_send_message(data):
    """Nachricht senden"""
    if request.sid not in connections:
        return emit('error', {'message': 'Not authenticated'})

    conn = connections[request.sid]
    channel_id = data['channel_id']
    content = data['content']
    reply_to = data.get('reply_to')

    # In DB schreiben
    message = ChatService.send_message(
```

```python
        channel_id, conn['user_id'], content, reply_to
    )

    # CDC wird automatisch andere Clients benachrichtigen

@socketio.on('edit_message')
def handle_edit_message(data):
    """Nachricht bearbeiten"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    message_id = data['message_id']
    new_content = data['content']

    try:
        message = ChatService.edit_message(
            message_id, conn['user_id'], new_content
        )
        # CDC benachrichtigt automatisch
    except ValueError as e:
        emit('error', {'message': str(e)})

@socketio.on('typing_start')
def handle_typing_start(data):
    """Typing-Indikator starten"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    channel_id = data['channel_id']

    # In DB schreiben (mit TTL)
    db.execute("""
        INSERT OR REPLACE INTO typing_indicators (channel_id, user_id)
        VALUES (?, ?)
    """, [channel_id, conn['user_id']])

    # An andere broadcasten
    emit('user_typing', {
        'user_id': conn['user_id'],
        'username': conn['username'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('typing_stop')
def handle_typing_stop(data):
    """Typing-Indikator stoppen"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    channel_id = data['channel_id']
```

```
    db.execute("""
        FOR indicator IN typing_indicators
          FILTER indicator.channel_id == @channel_id AND indicator.user_id == @user_id
          REMOVE indicator IN typing_indicators
    """, {"channel_id": channel_id, "user_id": conn['user_id']})

    emit('user_stopped_typing', {
        'user_id': conn['user_id'],
        'channel_id': channel_id
    }, room=f'channel_{channel_id}', skip_sid=request.sid)

@socketio.on('add_reaction')
def handle_add_reaction(data):
    """Reaktion hinzufügen"""
    if request.sid not in connections:
        return

    conn = connections[request.sid]
    message_id = data['message_id']
    emoji = data['emoji']

    if ChatService.add_reaction(message_id, conn['user_id'], emoji):
        # CDC benachrichtigt automatisch
        pass

@socketio.on('disconnect')
def handle_disconnect():
    """Client getrennt"""
    if request.sid in connections:
        conn = connections[request.sid]

        # Status auf offline
        ChatService.update_user_status(conn['user_id'], 'offline')

        # Alle Channels benachrichtigen
        for channel_id in conn['channels']:
            emit('user_left', {
                'user_id': conn['user_id'],
                'username': conn['username'],
                'channel_id': channel_id
            }, room=f'channel_{channel_id}')

        del connections[request.sid]

active_cdc_streams = set()

def start_channel_cdc(channel_id):
    """CDC-Stream für Channel starten"""
    if channel_id in active_cdc_streams:
        return

    active_cdc_streams.add(channel_id)

    def cdc_worker():
```

```
        stream = db.cdc.create_stream(
            name=f"chat_channel_{channel_id}",
            table="messages",
            filter=f"channel_id = {channel_id}",
            operations=["INSERT", "UPDATE", "DELETE"]
        )

        for event in stream:
            if event.operation == 'INSERT':
                # Neue Nachricht
                socketio.emit('new_message', {
                    'message': event.after
                }, room=f'channel_{channel_id}')

            elif event.operation == 'UPDATE':
                # Nachricht bearbeitet
                socketio.emit('message_edited', {
                    'message': event.after
                }, room=f'channel_{channel_id}')

            elif event.operation == 'DELETE':
                # Nachricht gelöscht
                socketio.emit('message_deleted', {
                    'message_id': event.before['id']
                }, room=f'channel_{channel_id}')

    # In Background-Thread
    socketio.start_background_task(cdc_worker)

@app.route('/api/channels', methods=['GET'])
def get_channels():
    """Alle Channels abrufen"""
    channels = db.query("""
        SELECT c.id, c.name, c.description, c.is_private,
               COUNT(DISTINCT cm.user_id) as member_count,
               MAX(m.created_at) as last_message_at
        FROM channels c
        LEFT JOIN channel_members cm ON c.id = cm.channel_id
        LEFT JOIN messages m ON c.id = m.channel_id
        GROUP BY c.id
        ORDER BY last_message_at DESC NULLS LAST
    """)
    return jsonify(channels)

@app.route('/api/channels/<int:channel_id>/members', methods=['GET'])
def get_channel_members(channel_id):
    """Channel-Mitglieder"""
    members = db.query("""
        SELECT u.id, u.username, u.avatar_url, u.status,
               cm.role, cm.joined_at
        FROM users u
        JOIN channel_members cm ON u.id = cm.user_id
        WHERE cm.channel_id = ?
        ORDER BY cm.joined_at
```

```
    """, [channel_id])
    return jsonify(members)

if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000, debug=True)
```

Chat-Frontend (React)

```
// ChatApp.jsx
import React, { useState, useEffect, useRef } from 'react';
import io from 'socket.io-client';

const ChatApp = ({ userId, username }) => {
    const [socket, setSocket] = useState(null);
    const [channels, setChannels] = useState([]);
    const [activeChannel, setActiveChannel] = useState(null);
    const [messages, setMessages] = useState([]);
    const [newMessage, setNewMessage] = useState('');
    const [typingUsers, setTypingUsers] = useState(new Set());
    const [onlineUsers, setOnlineUsers] = useState([]);

    const messagesEndRef = useRef(null);
    const typingTimeoutRef = useRef(null);

    useEffect(() => {
        // Socket-Verbindung
        const newSocket = io('http://localhost:5000');
        setSocket(newSocket);

        // Authentifizieren
        newSocket.emit('authenticate', { user_id: userId, username });

        // Event-Handler
        newSocket.on('authenticated', () => {
            console.log('Authenticated');
            loadChannels();
        });

        newSocket.on('channel_history', (data) => {
            setMessages(data.messages.reverse());
        });

        newSocket.on('new_message', (data) => {
            setMessages(prev => [...prev, data.message]);
            scrollToBottom();
        });

        newSocket.on('message_edited', (data) => {
            setMessages(prev => prev.map(msg =>
                msg.id === data.message.id ? data.message : msg
            ));
        });

        newSocket.on('message_deleted', (data) => {
            setMessages(prev => prev.filter(msg => msg.id !== data.message_id));
        });

        newSocket.on('user_typing', (data) => {
            setTypingUsers(prev => new Set([...prev, data.username]));
        });

        newSocket.on('user_stopped_typing', (data) => {
```

```
            setTypingUsers(prev => {
                const next = new Set(prev);
                next.delete(data.username);
                return next;
            });
        });

        newSocket.on('user_joined', (data) => {
            console.log(`${data.username} joined`);
            setOnlineUsers(prev => [...prev, data]);
        });

        newSocket.on('user_left', (data) => {
            console.log(`${data.username} left`);
            setOnlineUsers(prev => prev.filter(u => u.user_id !== data.user_id));
        });

        return () => newSocket.close();
    }, [userId, username]);

    const loadChannels = async () => {
        const response = await fetch('http://localhost:5000/api/channels');
        const data = await response.json();
        setChannels(data);
        if (data.length > 0) {
            joinChannel(data[0].id);
        }
    };

    const joinChannel = (channelId) => {
        if (activeChannel) {
            socket.emit('leave_channel', { channel_id: activeChannel });
        }
        socket.emit('join_channel', { channel_id: channelId });
        setActiveChannel(channelId);
        setMessages([]);
    };

    const sendMessage = () => {
        if (!newMessage.trim()) return;

        socket.emit('send_message', {
            channel_id: activeChannel,
            content: newMessage
        });

        setNewMessage('');
        stopTyping();
    };

    const handleTyping = () => {
        socket.emit('typing_start', { channel_id: activeChannel });

        // Auto-stop nach 3 Sekunden
```

```
        clearTimeout(typingTimeoutRef.current);
        typingTimeoutRef.current = setTimeout(() => {
            stopTyping();
        }, 3000);
    };

    const stopTyping = () => {
        socket.emit('typing_stop', { channel_id: activeChannel });
        clearTimeout(typingTimeoutRef.current);
    };

    const scrollToBottom = () => {
        messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
    };

    return (
        <div className="chat-app">
            <div className="sidebar">
                <h3>Channels</h3>
                {channels.map(channel => (
                    <div
                        key={channel.id}
                        className={`channel ${activeChannel === channel.id ? 'active' : ''}`}
                        onClick={() => joinChannel(channel.id)}
                    >
                        # {channel.name}
                        <span className="member-count">{channel.member_count}</span>
                    </div>
                ))}
            </div>

            <div className="main">
                <div className="header">
                    <h2>#{channels.find(c => c.id === activeChannel)?.name}</h2>
                    <div className="online-users">
                        {onlineUsers.length} online
                    </div>
                </div>

                <div className="messages">
                    {messages.map(msg => (
                        <div key={msg.id} className="message">
                            <img src={msg.avatar_url} alt={msg.username} />
                            <div>
                                <strong>{msg.username}</strong>
                                <span className="timestamp">
                                    {new Date(msg.created_at).toLocaleTimeString()}
                                </span>
                                <p>{msg.content}</p>
                                {msg.edited_at && <span className="edited">(edited)</span>}
                            </div>
                        </div>
                    ))}
                    <div ref={messagesEndRef} />
```

```
                </div>

                {typingUsers.size > 0 && (
                    <div className="typing-indicator">
                        {Array.from(typingUsers).join(', ')} {typingUsers.size === 1 ? 'is' :
                    </div>
                )}

                <div className="input-area">
                    <input
                        type="text"
                        value={newMessage}
                        onChange={(e) => {
                            setNewMessage(e.target.value);
                            handleTyping();
                        }}
                        onKeyPress={(e) => e.key === 'Enter' && sendMessage()}
                        placeholder="Type a message..."
                    />
                    <button onClick={sendMessage}>Send</button>
                </div>
            </div>
        </div>
    );
};

export default ChatApp;
```

Chat-Features

Das vollständige Chat-Example demonstriert:

1. **Echtzeit-Nachrichten**: Sofortige Zustellung via WebSocket + CDC
2. **Online-Status**: Wer ist gerade online?
3. **Typing-Indikatoren**: "Alice is typing..."
4. **Reaktionen**: Emoji-Reactions auf Nachrichten
5. **Threads**: Antworten auf bestimmte Nachrichten
6. **Nachrichtenbearbeitung**: Edit-History mit Timestamps
7. **Private Channels**: Eingeschränkter Zugriff
8. **Direktnachrichten**: 1-on-1 Chats
9. **Presence**: Last-Seen und Away-Status
10. **Pagination**: Unendliches Scrollen durch Historie

Example: Kanban Board (examples/16_kanban_board)

Vollständige Kanban-Board-Anwendung mit Drag & Drop, Realtime-Updates und Team-Collaboration.

Datenmodell

```
from themisdb import ThemisDB

db = ThemisDB()

db.execute("""
    CREATE TABLE boards (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        description TEXT,
        created_by INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
""")

db.execute("""
    CREATE TABLE columns (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        board_id INTEGER REFERENCES boards(id),
        name TEXT NOT NULL,
        position INTEGER NOT NULL,
        wip_limit INTEGER,  -- Work-In-Progress Limit
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_board (board_id, position)
    )
""")

db.execute("""
    CREATE TABLE cards (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        column_id INTEGER REFERENCES columns(id),
        board_id INTEGER REFERENCES boards(id),
        title TEXT NOT NULL,
        description TEXT,
        assigned_to INTEGER REFERENCES users(id),
        position INTEGER NOT NULL,
        priority TEXT DEFAULT 'medium',  -- low, medium, high, urgent
        due_date TIMESTAMP,
        created_by INTEGER REFERENCES users(id),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_column (column_id, position),
        INDEX idx_board (board_id)
    )
""")

db.execute("""
    CREATE TABLE card_activities (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        card_id INTEGER REFERENCES cards(id),
        user_id INTEGER REFERENCES users(id),
        action_type TEXT NOT NULL,  -- created, moved, assigned, commented
        details JSON,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        INDEX idx_card (card_id, created_at)
```

```
    )
""")

db.execute("""
    CREATE TABLE card_labels (
        card_id INTEGER REFERENCES cards(id),
        label TEXT NOT NULL,
        color TEXT,
        PRIMARY KEY (card_id, label)
    )
""")
```

```
from flask import Flask, jsonify, request
from flask_socketio import SocketIO, emit, join_room
from themisdb import ThemisDB
import time

app = Flask(__name__)
socketio = SocketIO(app, cors_allowed_origins="*")
db = ThemisDB()

class KanbanService:
    @staticmethod
    def get_board(board_id):
        """Board mit allen Spalten und Karten laden"""
        board = db.query("""
            FOR board IN boards
              FILTER board.id == @board_id
              LIMIT 1
              RETURN board
        """, {"board_id": board_id})[0]

        columns = db.query("""
            FOR column IN columns
              FILTER column.board_id == @board_id
              SORT column.position ASC
              RETURN column
        """, {"board_id": board_id})

        for col in columns:
            col['cards'] = db.query("""
                FOR card IN cards
                  FILTER card.column_id == @column_id
                  LET assigned_name = (
                    FOR user IN users
                      FILTER card.assigned_to == user.id
                      LIMIT 1
                      RETURN user.username
                  )[0]
                  SORT card.position ASC
                  RETURN MERGE(card, {assigned_to_name: assigned_name})
            """, {"column_id": col['id']})

        board['columns'] = columns
        return board

    @staticmethod
    def move_card(card_id, to_column_id, to_position):
        """Karte verschieben"""
        with db.transaction():
            # Aktuelle Position
            current = db.query("""
                FOR card IN cards
                  FILTER card.id == @card_id
                  LIMIT 1
                  RETURN {column_id: card.column_id, position: card.position}
```

```
        """, {"card_id": card_id})[0]

        from_column_id = current['column_id']
        from_position = current['position']

        # Position in alter Spalte aktualisieren
        db.execute("""
            UPDATE cards
            SET position = position - 1
            WHERE column_id = ? AND position > ?
        """, [from_column_id, from_position])

        # Position in neuer Spalte freimachen
        db.execute("""
            UPDATE cards
            SET position = position + 1
            WHERE column_id = ? AND position >= ?
        """, [to_column_id, to_position])

        # Karte verschieben
        db.execute("""
            UPDATE cards
            SET column_id = ?, position = ?, updated_at = CURRENT_TIMESTAMP
            WHERE id = ?
        """, [to_column_id, to_position, card_id])

        # Activity loggen
        db.execute("""
            INSERT INTO card_activities (card_id, user_id, action_type, details)
            VALUES (?, ?, 'moved', ?)
        """, [card_id, request.user_id, json.dumps({
            'from_column': from_column_id,
            'to_column': to_column_id
        })])

@staticmethod
def create_card(board_id, column_id, title, description, assigned_to=None):
    """Neue Karte erstellen"""
    with db.transaction():
        # Position am Ende der Spalte
        position = db.query("""
            FOR card IN cards
              FILTER card.column_id == @column_id
              COLLECT AGGREGATE max_pos = MAX(card.position)
              RETURN COALESCE(max_pos, -1) + 1
        """, {"column_id": column_id})[0]

        result = db.execute("""
            INSERT {
              board_id: @board_id,
              column_id: @column_id,
              title: @title,
              description: @description,
              assigned_to: @assigned_to,
```

```
                position: @position,
                created_by: @created_by
              } INTO cards
              RETURN NEW
          """, {
              "board_id": board_id,
              "column_id": column_id,
              "title": title,
              "description": description,
              "assigned_to": assigned_to,
              "position": position,
              "created_by": request.user_id
          })

          card = result[0]

          # Activity
          db.execute("""
              INSERT {
                card_id: @card_id,
                user_id: @user_id,
                action_type: 'created'
              } INTO card_activities
          """, {"card_id": card['id'], "user_id": request.user_id})

          return card

    @staticmethod
    def update_card(card_id, **updates):
        """Karte aktualisieren"""
        allowed_fields = ['title', 'description', 'assigned_to', 'priority', 'due_date']
        set_clause = ', '.join([f"{field} = ?" for field in updates if field in allowed_field
        values = [updates[field] for field in updates if field in allowed_fields]
        values.append(card_id)

        db.execute(f"""
            UPDATE cards
            SET {set_clause}, updated_at = CURRENT_TIMESTAMP
            WHERE id = ?
        """, values)

        # Activity
        db.execute("""
            INSERT {
              card_id: @card_id,
              user_id: @user_id,
              action_type: 'updated',
              details: @details
            } INTO card_activities
        """, {"card_id": card_id, "user_id": request.user_id, "details": json.dumps(updates)}

@socketio.on('join_board')
def handle_join_board(data):
    board_id = data['board_id']
```

```
    join_room(f'board_{board_id}')

    # Board-Daten senden
    board = KanbanService.get_board(board_id)
    emit('board_state', board)

    # CDC-Stream starten
    start_board_cdc(board_id)

@socketio.on('move_card')
def handle_move_card(data):
    card_id = data['card_id']
    to_column_id = data['to_column_id']
    to_position = data['to_position']
    board_id = data['board_id']

    try:
        KanbanService.move_card(card_id, to_column_id, to_position)
        # CDC benachrichtigt andere Clients
    except Exception as e:
        emit('error', {'message': str(e)})

@socketio.on('create_card')
def handle_create_card(data):
    card = KanbanService.create_card(
        data['board_id'],
        data['column_id'],
        data['title'],
        data.get('description'),
        data.get('assigned_to')
    )
    # CDC benachrichtigt

@socketio.on('update_card')
def handle_update_card(data):
    card_id = data.pop('card_id')
    KanbanService.update_card(card_id, **data)
    # CDC benachrichtigt

active_board_streams = set()

def start_board_cdc(board_id):
    if board_id in active_board_streams:
        return

    active_board_streams.add(board_id)

    def cdc_worker():
        # Cards-Stream
        cards_stream = db.cdc.create_stream(
            name=f"kanban_cards_{board_id}",
            table="cards",
            filter=f"board_id = {board_id}",
            operations=["INSERT", "UPDATE", "DELETE"]
```

```
        )

        # Columns-Stream
        columns_stream = db.cdc.create_stream(
            name=f"kanban_columns_{board_id}",
            table="columns",
            filter=f"board_id = {board_id}",
            operations=["INSERT", "UPDATE", "DELETE"]
        )

        # Beide Streams kombinieren
        for event in combined_stream([cards_stream, columns_stream]):
            if event.table == 'cards':
                if event.operation == 'INSERT':
                    socketio.emit('card_created', event.after,
                                room=f'board_{board_id}')
                elif event.operation == 'UPDATE':
                    socketio.emit('card_updated', event.after,
                                room=f'board_{board_id}')
                elif event.operation == 'DELETE':
                    socketio.emit('card_deleted', {'id': event.before['id']},
                                room=f'board_{board_id}')

            elif event.table == 'columns':
                # Spalten-Update
                socketio.emit('column_updated', event.after,
                            room=f'board_{board_id}')

    socketio.start_background_task(cdc_worker)

def combined_stream(streams):
    """Mehrere CDC-Streams kombinieren"""
    import queue
    import threading

    q = queue.Queue()

    def stream_worker(stream):
        for event in stream:
            q.put(event)

    for stream in streams:
        threading.Thread(target=stream_worker, args=(stream,), daemon=True).start()

    while True:
        yield q.get()

if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000)
```

Kanban-Frontend (React)

```
// KanbanBoard.jsx
import React, { useState, useEffect } from 'react';
import { DragDropContext, Droppable, Draggable } from 'react-beautiful-dnd';
import io from 'socket.io-client';

const KanbanBoard = ({ boardId, userId }) => {
    const [socket, setSocket] = useState(null);
    const [board, setBoard] = useState(null);
    const [columns, setColumns] = useState([]);

    useEffect(() => {
        const newSocket = io('http://localhost:5000');
        setSocket(newSocket);

        newSocket.emit('join_board', { board_id: boardId });

        newSocket.on('board_state', (data) => {
            setBoard(data);
            setColumns(data.columns);
        });

        newSocket.on('card_created', (card) => {
            setColumns(prev => prev.map(col => {
                if (col.id === card.column_id) {
                    return { ...col, cards: [...col.cards, card] };
                }
                return col;
            }));
        });

        newSocket.on('card_updated', (card) => {
            setColumns(prev => prev.map(col => ({
                ...col,
                cards: col.cards.map(c => c.id === card.id ? card : c)
            })));
        });

        newSocket.on('card_deleted', (data) => {
            setColumns(prev => prev.map(col => ({
                ...col,
                cards: col.cards.filter(c => c.id !== data.id)
            })));
        });

        return () => newSocket.close();
    }, [boardId]);

    const onDragEnd = (result) => {
        if (!result.destination) return;

        const { source, destination } = result;

        if (source.droppableId === destination.droppableId &&
            source.index === destination.index) {
```

```
            return;
        }

        // Optimistisches Update
        const newColumns = [...columns];
        const sourceColumn = newColumns.find(c => c.id === parseInt(source.droppableId));
        const destColumn = newColumns.find(c => c.id === parseInt(destination.droppableId));

        const [movedCard] = sourceColumn.cards.splice(source.index, 1);
        destColumn.cards.splice(destination.index, 0, movedCard);

        setColumns(newColumns);

        // An Server senden
        socket.emit('move_card', {
            card_id: parseInt(result.draggableId),
            to_column_id: parseInt(destination.droppableId),
            to_position: destination.index,
            board_id: boardId
        });
    };

    if (!board) return <div>Loading...</div>;

    return (
        <div className="kanban-board">
            <h1>{board.name}</h1>

            <DragDropContext onDragEnd={onDragEnd}>
                <div className="columns">
                    {columns.map(column => (
                        <div key={column.id} className="column">
                            <h3>
                                {column.name}
                                <span className="card-count">{column.cards.length}</span>
                                {column.wip_limit && (
                                    <span className={`wip-limit ${column.cards.length > colum
                                        / {column.wip_limit}
                                    </span>
                                )}
                            </h3>

                            <Droppable droppableId={column.id.toString()}>
                                {(provided, snapshot) => (
                                    <div
                                        ref={provided.innerRef}
                                        {...provided.droppableProps}
                                        className={`cards ${snapshot.isDraggingOver ? 'draggi
                                    >
                                        {column.cards.map((card, index) => (
                                            <Draggable
                                                key={card.id}
                                                draggableId={card.id.toString()}
                                                index={index}
```

```
                                            >
                                                {(provided, snapshot) => (
                                                    <div
                                                        ref={provided.innerRef}
                                                        {...provided.draggableProps}
                                                        {...provided.dragHandleProps}
                                                        className={`card priority-${card.prio
                                                    >
                                                        <h4>{card.title}</h4>
                                                        {card.description && <p>{card.descrip
                                                        {card.assigned_to_name && (
                                                            <div className="assignee">
                                                                  {card.assigned_to_name}
                                                            </div>
                                                        )}
                                                        {card.due_date && (
                                                            <div className="due-date">
                                                                  {new Date(card.due_date).to
                                                            </div>
                                                        )}
                                                    </div>
                                                )}
                                            </Draggable>
                                        ))}
                                        {provided.placeholder}
                                    </div>
                                )}
                            </Droppable>
                        </div>
                    ))}
                </div>
            </DragDropContext>
        </div>
    );
};

export default KanbanBoard;
```

Kanban-Features

Das Kanban-Board-Example demonstriert:

1. **Drag & Drop**: Karten zwischen Spalten verschieben
2. **Realtime-Sync**: Alle Nutzer sehen Änderungen sofort
3. **WIP-Limits**: Work-In-Progress Beschränkungen
4. **Prioritäten**: Visuelle Kennzeichnung (low, medium, high, urgent)
5. **Zuweisungen**: Karten Teammitgliedern zuordnen
6. **Due Dates**: Fälligkeitsdaten mit Warnungen
7. **Labels**: Flexible Kategorisierung
8. **Activity-Log**: Vollständige Historie aller Änderungen
9. **Board-Analytics**: Durchlaufzeit, Cycle-Time, etc.

10. **Custom Columns**: Beliebige Workflow-Stufen

Event-Driven Architecture

Realtime-Anwendungen profitieren von event-driven Design.

Event-Bus-Pattern

```python
from themisdb import ThemisDB
from typing import Callable, List
import threading

class EventBus:
    def __init__(self, db: ThemisDB):
        self.db = db
        self.handlers = {}  # event_type -> [handlers]
        self.running = False

    def subscribe(self, event_type: str, handler: Callable):
        """Handler für Event-Typ registrieren"""
        if event_type not in self.handlers:
            self.handlers[event_type] = []
        self.handlers[event_type].append(handler)

    def publish(self, event_type: str, data: dict):
        """Event publishen"""
        self.db.execute("""
            INSERT {
              event_type: @event_type,
              data: @data,
              created_at: DATE_NOW()
            } INTO events
        """, {"event_type": event_type, "data": json.dumps(data)})

    def start(self):
        """Event-Processing starten"""
        self.running = True

        def process_events():
            stream = self.db.cdc.create_stream(
                name="event_bus",
                table="events",
                operations=["INSERT"]
            )

            for event in stream:
                if not self.running:
                    break

                event_type = event.after['event_type']
                data = json.loads(event.after['data'])

                # Handler aufrufen
                if event_type in self.handlers:
                    for handler in self.handlers[event_type]:
                        try:
                            handler(data)
                        except Exception as e:
                            print(f"Handler error: {e}")

        threading.Thread(target=process_events, daemon=True).start()
```

```
    def stop(self):
        self.running = False

bus = EventBus(db)

@bus.subscribe('user_registered')
def send_welcome_email(data):
    print(f"Sending email to {data['email']}")

@bus.subscribe('order_placed')
def update_inventory(data):
    print(f"Reducing stock for order {data['order_id']}")

bus.publish('user_registered', {
    'user_id': 123,
    'email': 'alice@example.com'
})

bus.start()
```

CQRS-Pattern

Command Query Responsibility Segregation für Realtime-Apps:

```
class OrderService:
    def __init__(self, db: ThemisDB, event_bus: EventBus):
        self.db = db
        self.event_bus = event_bus

    # COMMAND: Write-Seite
    def place_order(self, user_id, items):
        with self.db.transaction():
            # Order erstellen
            order_id = self.db.execute("""
                INSERT {
                  user_id: @user_id,
                  status: 'pending',
                  created_at: DATE_NOW()
                } INTO orders
                RETURN NEW.id
            """, {"user_id": user_id})[0]

            # Items
            for item in items:
                self.db.execute("""
                    INSERT {
                      order_id: @order_id,
                      product_id: @product_id,
                      quantity: @quantity,
                      price: @price
                    } INTO order_items
                """, {
                    "order_id": order_id,
                    "product_id": item['product_id'],
                    "quantity": item['quantity'],
                    "price": item['price']
                })

            # Event publishen
            self.event_bus.publish('order_placed', {
                'order_id': order_id,
                'user_id': user_id,
                'items': items
            })

            return order_id

    # QUERY: Read-Seite (materialized view)
    def get_order_summary(self, order_id):
        return self.db.query("""
            SELECT
                o.id, o.status, o.created_at,
                u.name as customer_name,
                COUNT(oi.id) as item_count,
                SUM(oi.quantity * oi.price) as total
            FROM orders o
            JOIN users u ON o.user_id = u.id
            LEFT JOIN order_items oi ON o.id = oi.order_id
```

```
            WHERE o.id = ?
            GROUP BY o.id
        """, [order_id])[0]

@event_bus.subscribe('order_placed')
def update_order_cache(data):
    # Cache/Read-Model aktualisieren
    pass
```

Best Practices

1. Connection Management

```
class ConnectionPool:
    def __init__(self, max_connections=100):
        self.max_connections = max_connections
        self.active_connections = {}
        self.semaphore = threading.Semaphore(max_connections)

    def add_connection(self, sid, user_id):
        with self.semaphore:
            if len(self.active_connections) >= self.max_connections:
                raise ValueError("Too many connections")
            self.active_connections[sid] = user_id

    def remove_connection(self, sid):
        if sid in self.active_connections:
            del self.active_connections[sid]
            self.semaphore.release()
```

2. Rate Limiting

```
from collections import defaultdict
import time

class RateLimiter:
    def __init__(self, max_requests=10, window=60):
        self.max_requests = max_requests
        self.window = window
        self.requests = defaultdict(list)

    def allow(self, user_id):
        now = time.time()
        # Alte Einträge entfernen
        self.requests[user_id] = [
            t for t in self.requests[user_id]
            if now - t < self.window
        ]

        if len(self.requests[user_id]) >= self.max_requests:
            return False

        self.requests[user_id].append(now)
        return True

@socketio.on('send_message')
def handle_send_message(data):
    if not rate_limiter.allow(current_user_id):
        return emit('error', {'message': 'Rate limit exceeded'})
    # ... rest
```

3. Message Deduplication

```
class MessageDeduplicator:
    def __init__(self, ttl=60):
        self.seen = {}  # message_id -> timestamp
        self.ttl = ttl

    def is_duplicate(self, message_id):
        now = time.time()

        # Cleanup
        self.seen = {
            mid: ts for mid, ts in self.seen.items()
            if now - ts < self.ttl
        }

        if message_id in self.seen:
            return True

        self.seen[message_id] = now
        return False
```

4. Graceful Shutdown

```
import signal
import sys

def graceful_shutdown(signum, frame):
    print("Shutting down gracefully...")

    # Alle Clients benachrichtigen
    socketio.emit('server_shutdown', {
        'message': 'Server ist down für Wartung',
        'reconnect_in': 60
    })

    # CDC-Streams stoppen
    for stream_id in active_cdc_streams:
        stop_cdc_stream(stream_id)

    # Connections schließen
    socketio.stop()

    sys.exit(0)

signal.signal(signal.SIGINT, graceful_shutdown)
signal.signal(signal.SIGTERM, graceful_shutdown)
```

5. Monitoring & Metrics

```
from prometheus_client import Counter, Histogram, Gauge
import time

messages_sent = Counter('messages_sent_total', 'Total messages sent')
message_latency = Histogram('message_latency_seconds', 'Message delivery latency')
active_connections_gauge = Gauge('active_connections', 'Number of active WebSocket connectio

@socketio.on('send_message')
def handle_send_message(data):
    start_time = time.time()

    # ... message handling ...

    messages_sent.inc()
    message_latency.observe(time.time() - start_time)

@socketio.on('connect')
def handle_connect():
    active_connections_gauge.inc()

@socketio.on('disconnect')
def handle_disconnect():
    active_connections_gauge.dec()
```

Performance-Optimierungen

1. Message-Batching

```
class MessageBatcher:
    def __init__(self, max_batch_size=100, max_wait_ms=100):
        self.max_batch_size = max_batch_size
        self.max_wait_ms = max_wait_ms
        self.batch = []
        self.last_flush = time.time()

    def add(self, message):
        self.batch.append(message)

        if len(self.batch) >= self.max_batch_size or \
           (time.time() - self.last_flush) * 1000 >= self.max_wait_ms:
            self.flush()

    def flush(self):
        if not self.batch:
            return

        # Bulk-Insert
        db.executemany("""
            INSERT INTO messages (channel_id, user_id, content)
            VALUES (?, ?, ?)
        """, [(m['channel_id'], m['user_id'], m['content']) for m in self.batch])

        self.batch = []
        self.last_flush = time.time()
```

2. Connection Pooling

```
from themisdb import ThemisDB, ConnectionPool

pool = ConnectionPool(
    min_size=10,
    max_size=50,
    max_idle_time=300
)

def get_db_connection():
    return pool.acquire()

def release_db_connection(conn):
    pool.release(conn)
```

3. Caching

```
from functools import lru_cache
import time

class CachedQuery:
    def __init__(self, ttl=60):
        self.cache = {}
        self.ttl = ttl

    def get(self, query, params):
        key = (query, tuple(params))

        if key in self.cache:
            value, timestamp = self.cache[key]
            if time.time() - timestamp < self.ttl:
                return value

        # Cache Miss
        result = db.query(query, params)
        self.cache[key] = (result, time.time())
        return result

cache = CachedQuery(ttl=30)

def get_channel_members(channel_id):
    return cache.get("""
        FOR member IN channel_members
          FILTER member.channel_id == @channel_id
          RETURN member
    """, {"channel_id": channel_id})
```

Zusammenfassung

In diesem Kapitel haben Sie gelernt:

1. **Change Data Capture (CDC)**: Datenänderungen verfolgen und darauf reagieren
2. **Server-Sent Events (SSE)**: Unidirektionale Push-Updates
3. **WebSockets**: Bidirektionale Realtime-Kommunikation
4. **Event-Driven Architecture**: Entkoppelte, skalierbare Systeme
5. **Realtime Chat**: Vollständige Implementierung mit allen Features
6. **Kanban Board**: Collaborative Drag & Drop mit Sync
7. **Best Practices**: Connection Management, Rate Limiting, Monitoring
8. **Performance**: Batching, Pooling, Caching

Realtime-Anwendungen mit ThemisDB sind einfach zu implementieren dank: - Integriertem CDC-System - MVCC für konfliktfreie Reads - Flexible Event-Streaming-Mechanismen - Starke Transaktionsgarantien

Im nächsten Kapitel schauen wir uns Computer Vision-Anwendungen an, wo wir Bilder speichern, analysieren und durchsuchen.

Übungen

1. **Chat-Erweiterung**: Fügen Sie Voice Messages und File Sharing hinzu
2. **Kanban-Analytics**: Cycle Time und Lead Time berechnen
3. **Presence-System**: Implementieren Sie "Alice is viewing card #42"
4. **Notification-System**: Push-Benachrichtigungen bei @mentions
5. **Realtime-Dashboard**: Live-Metrics mit automatischer Aktualisierung
6. **Collaborative Editor**: Google Docs-ähnliche Textbearbeitung
7. **Gaming**: Einfaches Multiplayer-Spiel mit ThemisDB als Backend
8. **Live-Poll**: Echtzeit-Abstimmungssystem mit automatischen Updates
9. **Activity-Feed**: Timeline aller Aktivitäten im System
10. **Realtime-Search**: Live-Search-Results während dem Tippen

---

Kapitel 13: Kapitel 12: Computer Vision & Bildanalyse

12.1 Einführung in Computer Vision Datenbanken

Das Problem: Bilder sind mehr als nur Dateien

Stellen Sie sich vor, Sie haben 100.000 Drohnenbilder von einem Baugelände. Wie finden Sie: - Alle Bilder, die ein bestimmtes Fahrzeug zeigen? - Bilder mit Sicherheitsproblemen (fehlende Helme)? - Ähnliche Perspektiven des gleichen Gebäudes? - Zeitliche Entwicklung eines Bauprojekts?

Traditionelle Datenbanken speichern nur Dateinamen - **Computer Vision Datenbanken** analysieren und indexieren Bildinhalte.

Computer Vision Use Cases

Use Case	Datentyp	Volumen	Beispiel
Drohnen-Inspektion	Bilder + GPS + Metadata	10K-1M	Baustellen, Infrastruktur
Medizinische Bildgebung	DICOM-Bilder	100K-10M	Röntgen, MRT, CT
Retail Analytics	Video-Frames	1M-100M	Kundenverhalten, Warenpräsentation
Autonomous Vehicles	Sensor-Fusion	100M+	LiDAR, Kameras, Radar
Satellite Imagery	Multi-Spectral Images	1M-10M	Landnutzung, Umweltmonitoring

Herausforderungen

Storage: - Große Dateien (2-20 MB pro Hochauflösungsbild) - Verschiedene Formate (JPEG, PNG, TIFF, RAW) - Metadata (EXIF, GPS, Kameraeinstellungen)

Processing: - Feature-Extraktion (SIFT, ORB, Deep Learning) - Object Detection (YOLO, Faster R-CNN) - Image Classification (ResNet, EfficientNet)

Retrieval: - Content-Based Image Retrieval (CBIR) - Similarity Search über Visual Features - Spatial Queries (GPS-basiert) - Temporal Queries (Zeitreihen)

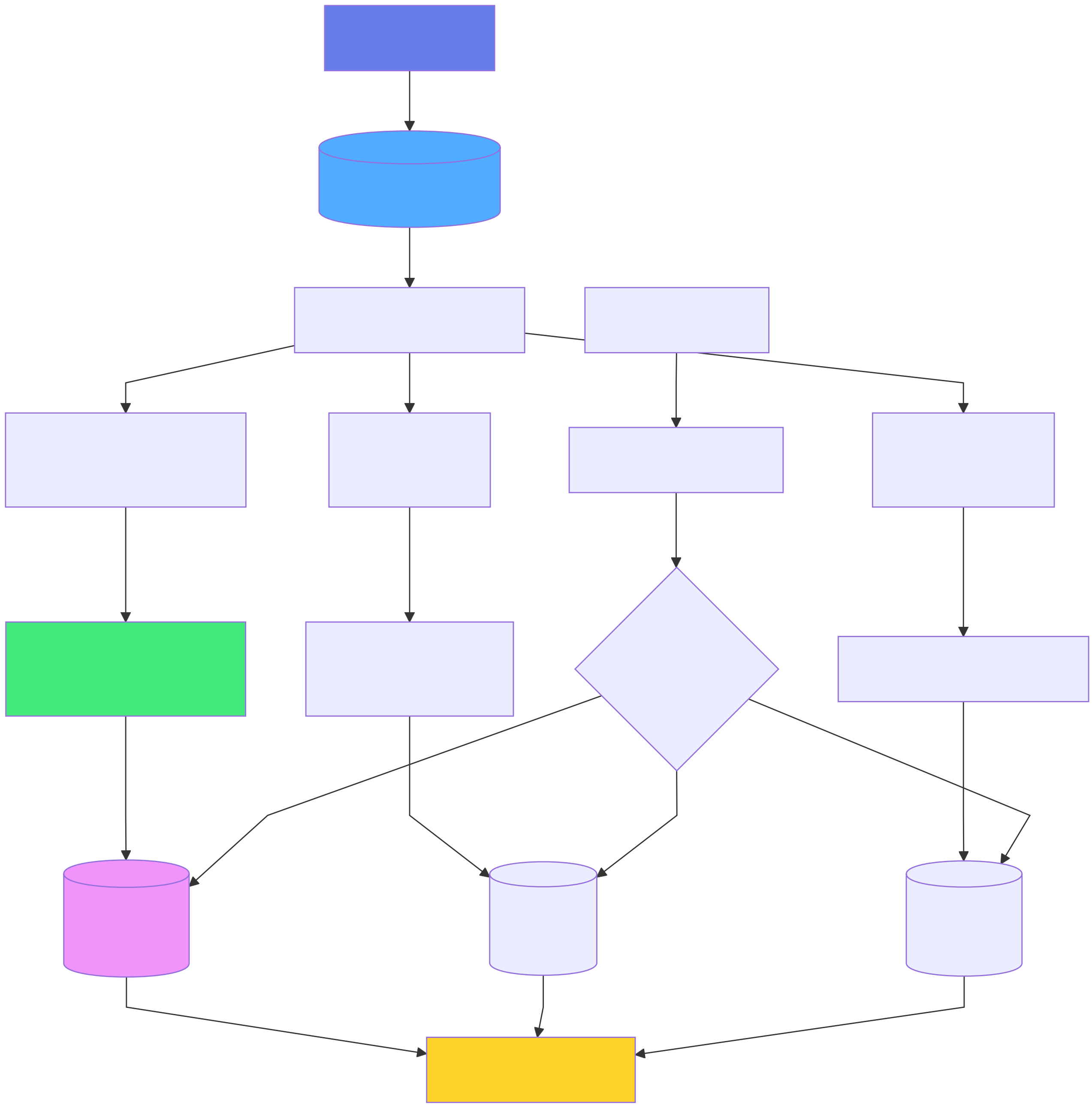


Abb. 35: Ablaufdiagramm (Kapitel 12: Computer Vision & Bildanalyse)

12.2 Computer Vision Datenmodell

Schema für Bildmetadaten

```sql
-- Bilder mit vollständigen Metadaten
CREATE TABLE images (
    image_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    filename VARCHAR(255) NOT NULL,
    file_path TEXT NOT NULL,
    file_size_bytes BIGINT,
    format VARCHAR(20),  -- 'JPEG', 'PNG', 'TIFF', ...

    -- Bild-Eigenschaften
    width INTEGER,
    height INTEGER,
    bit_depth INTEGER,
    color_space VARCHAR(20),

    -- Aufnahme-Metadaten (EXIF)
    captured_at TIMESTAMP,
    camera_make VARCHAR(100),
    camera_model VARCHAR(100),
    focal_length_mm FLOAT,
    aperture_f_stop FLOAT,
    iso INTEGER,
    shutter_speed VARCHAR(20),

    -- GPS-Daten
    gps_latitude DOUBLE PRECISION,
    gps_longitude DOUBLE PRECISION,
    gps_altitude_m FLOAT,
    location GEOGRAPHY(Point, 4326),

    -- Kategorisierung
    category VARCHAR(100),
    tags TEXT[],

    -- Zusätzliche Metadaten
    metadata JSONB,

    -- Zeitstempel
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Spatial Index für GPS-Queries
CREATE INDEX idx_images_location ON images USING GIST(location);

-- Index für zeitbasierte Queries
CREATE INDEX idx_images_captured_at ON images (captured_at DESC);

-- Full-Text Index für Tags
CREATE INDEX idx_images_tags ON images USING GIN(tags);
```

Feature-Extraktion Tabelle

```
-- Visuelle Features (z.B. von CNN)
CREATE TABLE image_features (
    image_id UUID PRIMARY KEY REFERENCES images(image_id),

    -- Deep Learning Features (z.B. ResNet-50)
    deep_features VECTOR(2048),  -- 2048-dimensional embedding

    -- Classical Features
    color_histogram FLOAT[],     -- RGB histogram
    edge_features FLOAT[],       -- Edge detection features
    texture_features FLOAT[],    -- Texture descriptors

    -- Feature Metadata
    feature_extractor VARCHAR(100),
    extraction_timestamp TIMESTAMP DEFAULT NOW()
);

-- HNSW Index für Visual Similarity Search
CREATE INDEX idx_image_features_deep ON image_features
USING hnsw (deep_features vector_cosine_ops)
WITH (m = 16, ef_construction = 200);
```

Object Detection Results

```
-- Erkannte Objekte in Bildern
CREATE TABLE detected_objects (
    detection_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    image_id UUID REFERENCES images(image_id),

    -- Objekt-Klassifikation
    object_class VARCHAR(100),
    confidence FLOAT,  -- 0.0 to 1.0

    -- Bounding Box
    bbox_x INTEGER,
    bbox_y INTEGER,
    bbox_width INTEGER,
    bbox_height INTEGER,

    -- Optional: Segmentation Mask
    segmentation_mask BYTEA,

    -- Detection Metadata
    detector_model VARCHAR(100),
    detection_timestamp TIMESTAMP DEFAULT NOW()
);

-- Index für Object-Search
CREATE INDEX idx_detected_objects_class ON detected_objects (object_class, confidence);
CREATE INDEX idx_detected_objects_image ON detected_objects (image_id);
```

12.3 Bildverarbeitung mit ThemisDB

Bild-Upload mit Metadata-Extraktion

```python
from PIL import Image
from PIL.ExifTags import TAGS, GPSTAGS
import hashlib
from datetime import datetime

def extract_exif(image_path):
    """Extrahiere EXIF-Daten aus Bild"""
    img = Image.open(image_path)
    exif_data = {}

    if hasattr(img, '_getexif') and img._getexif():
        exif = img._getexif()
        for tag_id, value in exif.items():
            tag = TAGS.get(tag_id, tag_id)
            exif_data[tag] = value

    # GPS-Daten extrahieren
    gps_info = exif_data.get('GPSInfo', {})
    gps_data = {}
    for key in gps_info.keys():
        decode = GPSTAGS.get(key, key)
        gps_data[decode] = gps_info[key]

    return exif_data, gps_data

def upload_image(image_path, category=None, tags=None):
    """Upload Bild mit Metadata-Extraktion"""
    img = Image.open(image_path)
    exif_data, gps_data = extract_exif(image_path)

    # GPS-Koordinaten konvertieren
    latitude = convert_to_degrees(gps_data.get('GPSLatitude', []))
    longitude = convert_to_degrees(gps_data.get('GPSLongitude', []))
    altitude = gps_data.get('GPSAltitude', 0)

    # Bild-Metadata
    image_data = {
        'filename': os.path.basename(image_path),
        'file_path': image_path,
        'file_size_bytes': os.path.getsize(image_path),
        'format': img.format,
        'width': img.width,
        'height': img.height,
        'captured_at': exif_data.get('DateTime'),
        'camera_make': exif_data.get('Make'),
        'camera_model': exif_data.get('Model'),
        'gps_latitude': latitude,
        'gps_longitude': longitude,
        'gps_altitude_m': altitude,
        'category': category,
        'tags': tags or []
    }

    # In DB speichern
```

```
    image_id = themis.execute("""
        INSERT INTO images
        (filename, file_path, file_size_bytes, format, width, height,
         captured_at, camera_make, camera_model,
         gps_latitude, gps_longitude, gps_altitude_m,
         location, category, tags)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
                ST_SetSRID(ST_MakePoint(?, ?), 4326), ?, ?)
        RETURNING image_id
    """, tuple(image_data.values()) + (longitude, latitude))

    return image_id
```

Feature-Extraktion mit Deep Learning

```
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from PIL import Image

model = models.resnet50(pretrained=True)
model.eval()
model = torch.nn.Sequential(*list(model.children())[:-1])

preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                       std=[0.229, 0.224, 0.225]),
])

def extract_features(image_path):
    """Extrahiere Deep Learning Features"""
    img = Image.open(image_path).convert('RGB')
    img_tensor = preprocess(img).unsqueeze(0)

    with torch.no_grad():
        features = model(img_tensor)
        features = features.squeeze().numpy()  # (2048,)

    return features

def store_features(image_id, image_path):
    """Extrahiere und speichere Features"""
    features = extract_features(image_path)

    themis.execute("""
        INSERT INTO image_features (image_id, deep_features, feature_extractor)
        VALUES (?, ?, 'resnet50')
        ON CONFLICT (image_id) DO UPDATE
        SET deep_features = EXCLUDED.deep_features,
            extraction_timestamp = NOW()
    """, (image_id, features.tolist()))
```

Object Detection mit YOLO

```
from ultralytics import YOLO

yolo_model = YOLO('yolov8n.pt')

def detect_objects(image_path, confidence_threshold=0.5):
    """Detecte Objekte mit YOLO"""
    results = yolo_model(image_path)

    detections = []
    for result in results:
        boxes = result.boxes
        for box in boxes:
            if box.conf[0] >= confidence_threshold:
                detections.append({
                    'object_class': result.names[int(box.cls[0])],
                    'confidence': float(box.conf[0]),
                    'bbox_x': int(box.xyxy[0][0]),
                    'bbox_y': int(box.xyxy[0][1]),
                    'bbox_width': int(box.xyxy[0][2] - box.xyxy[0][0]),
                    'bbox_height': int(box.xyxy[0][3] - box.xyxy[0][1])
                })

    return detections

def store_detections(image_id, image_path):
    """Detecte und speichere Objekte"""
    detections = detect_objects(image_path)

    for det in detections:
        themis.execute("""
            INSERT INTO detected_objects
            (image_id, object_class, confidence, bbox_x, bbox_y,
             bbox_width, bbox_height, detector_model)
            VALUES (?, ?, ?, ?, ?, ?, ?, 'yolov8n')
        """, (image_id, det['object_class'], det['confidence'],
              det['bbox_x'], det['bbox_y'],
              det['bbox_width'], det['bbox_height']))
```

12.4 Visual Similarity Search

Content-Based Image Retrieval (CBIR)

```sql
-- Finde visuell ähnliche Bilder
SELECT
    i.image_id,
    i.filename,
    i.captured_at,
    i.location,
    1 - (f.deep_features <=> :query_features) AS similarity
FROM images i
JOIN image_features f ON i.image_id = f.image_id
ORDER BY f.deep_features <=> :query_features
LIMIT 20;
```

```python
def find_similar_images(query_image_path, limit=10):
    """Finde visuell ähnliche Bilder"""
    # Features der Query extrahieren
    query_features = extract_features(query_image_path)

    # Similarity Search
    similar = themis.query("""
        SELECT
            i.image_id,
            i.filename,
            i.file_path,
            i.captured_at,
            1 - (f.deep_features <=> ?) AS similarity
        FROM images i
        JOIN image_features f ON i.image_id = f.image_id
        ORDER BY f.deep_features <=> ?
        LIMIT ?
    """, (query_features.tolist(), query_features.tolist(), limit))

    return similar

similar_images = find_similar_images('query_image.jpg', limit=10)
for img in similar_images:
    print(f"{img['filename']}: {img['similarity']:.3f}")
```

Multi-Modal Search: Visual + Text

```sql
-- Kombiniere Visual Similarity + Text Tags
WITH visual_matches AS (
    SELECT image_id,
           1 - (deep_features <=> :query_features) AS visual_score
    FROM image_features
    ORDER BY deep_features <=> :query_features
    LIMIT 100
),
text_matches AS (
    SELECT image_id,
           ts_rank(to_tsvector('english', array_to_string(tags, ' ')),
                   plainto_tsquery('english', :query_text)) AS text_score
    FROM images
    WHERE tags && :query_tags  -- Array overlap
)
SELECT
    i.*,
    COALESCE(vm.visual_score, 0) * 0.7 +
    COALESCE(tm.text_score, 0) * 0.3 AS combined_score
FROM images i
LEFT JOIN visual_matches vm ON i.image_id = vm.image_id
LEFT JOIN text_matches tm ON i.image_id = tm.image_id
WHERE vm.image_id IS NOT NULL OR tm.image_id IS NOT NULL
ORDER BY combined_score DESC
LIMIT 20;
```

12.5 Spatial & Temporal Queries

GPS-basierte Suche

```
-- Alle Bilder in einem Radius von 1km
SELECT
    image_id,
    filename,
    captured_at,
    gps_latitude,
    gps_longitude,
    ST_Distance(location, ST_SetSRID(ST_MakePoint(?, ?), 4326)) as distance_m
FROM images
WHERE ST_DWithin(
    location,
    ST_SetSRID(ST_MakePoint(?, ?), 4326),
    1000  -- 1000 Meter
)
ORDER BY distance_m;

-- Bilder in einem Polygon (z.B. Baugelände)
SELECT *
FROM images
WHERE ST_Within(
    location,
    ST_GeomFromText('POLYGON((
        13.404 52.520,
        13.408 52.520,
        13.408 52.518,
        13.404 52.518,
        13.404 52.520
    ))', 4326)
);
```

Zeitliche Entwicklung

```
def get_temporal_sequence(location, radius_m=100,
                          start_date=None, end_date=None):
    """Hole zeitliche Bildsequenz an einem Ort"""
    query = """
        SELECT
            image_id,
            filename,
            captured_at,
            category,
            tags,
            ST_Distance(location, ST_SetSRID(ST_MakePoint(?, ?), 4326)) as distance_m
        FROM images
        WHERE ST_DWithin(
            location,
            ST_SetSRID(ST_MakePoint(?, ?), 4326),
            ?
        )
    """

    params = [longitude, latitude, longitude, latitude, radius_m]

    if start_date:
        query += " AND captured_at >= ?"
        params.append(start_date)

    if end_date:
        query += " AND captured_at <= ?"
        params.append(end_date)

    query += " ORDER BY captured_at"

    return themis.query(query, tuple(params))

sequence = get_temporal_sequence(
    location=(13.405, 52.520),
    radius_m=50,
    start_date='2024-01-01',
    end_date='2024-12-31'
)
```

12.6 Example: Drone Image Analysis

Überblick

Das **Drone Image Analysis** Beispiel (`examples/09_drone_image_analysis`) demonstriert eine vollständige Pipeline für Drohnenbilder-Analyse:

Features: - Automatische Bild-Kategorisierung - GPS-basierte Geo-Queries - Feature Matching für ähnliche Bilder - Object Detection (YOLOv8) Integration - Flight Path Visualisierung - Thermal Imaging Analytics

Batch-Upload von Drohnenbildern

```python
import os
import glob
from concurrent.futures import ThreadPoolExecutor

def batch_upload_drone_images(directory, flight_id, category='construction'):
    """Batch-Upload aller Bilder aus einem Drohnenflug"""
    image_files = glob.glob(os.path.join(directory, '*.jpg'))
    print(f"Found {len(image_files)} images to process")

    def process_image(image_path):
        # Upload Bild
        image_id = upload_image(image_path, category=category,
                               tags=['drone', flight_id])

        # Feature-Extraktion
        store_features(image_id, image_path)

        # Object Detection
        store_detections(image_id, image_path)

        return image_id

    # Parallel-Processing
    with ThreadPoolExecutor(max_workers=4) as executor:
        image_ids = list(executor.map(process_image, image_files))

    print(f"Processed {len(image_ids)} images")
    return image_ids

flight_id = 'flight_2024_01_15_001'
image_ids = batch_upload_drone_images(
    directory='/data/drone_images/2024-01-15/',
    flight_id=flight_id,
    category='construction_site'
)
```

Flugpfad-Rekonstruktion

```
def reconstruct_flight_path(flight_id):
    """Rekonstruiere Flugpfad aus GPS-Daten"""
    path = themis.query("""
        SELECT
            image_id,
            filename,
            captured_at,
            gps_latitude,
            gps_longitude,
            gps_altitude_m,
            ST_AsText(location) as location_wkt
        FROM images
        WHERE ? = ANY(tags)
        ORDER BY captured_at
    """, (flight_id,))

    return path

import folium

def visualize_flight_path(flight_id):
    """Visualisiere Flugpfad auf Karte"""
    path = reconstruct_flight_path(flight_id)

    # Zentrum der Karte
    center_lat = sum(p['gps_latitude'] for p in path) / len(path)
    center_lon = sum(p['gps_longitude'] for p in path) / len(path)

    # Karte erstellen
    m = folium.Map(location=[center_lat, center_lon], zoom_start=15)

    # Flugpfad zeichnen
    coordinates = [(p['gps_latitude'], p['gps_longitude']) for p in path]
    folium.PolyLine(coordinates, color='red', weight=2,
                    opacity=0.8).add_to(m)

    # Marker für Bilder
    for p in path:
        folium.Marker(
            location=[p['gps_latitude'], p['gps_longitude']],
            popup=f"{p['filename']}<br>Alt: {p['gps_altitude_m']}m",
            icon=folium.Icon(color='blue', icon='camera')
        ).add_to(m)

    m.save(f'flight_path_{flight_id}.html')
    return m

visualize_flight_path(flight_id)
```

Automatische Anomalie-Detection

```
def detect_safety_violations(flight_id):
    """Detecte Sicherheitsverstöße (z.B. fehlende Helme)"""
    # Finde alle Bilder mit Personen
    images_with_people = themis.query("""
        SELECT DISTINCT i.image_id, i.filename, i.file_path
        FROM images i
        JOIN detected_objects d ON i.image_id = d.image_id
        WHERE d.object_class = 'person'
          AND d.confidence > 0.7
          AND ? = ANY(i.tags)
    """, (flight_id,))

    violations = []

    for img in images_with_people:
        # Prüfe, ob Helme detectiert wurden
        helmets = themis.query("""
            SELECT COUNT(*) as helmet_count
            FROM detected_objects
            WHERE image_id = ?
              AND object_class IN ('hardhat', 'helmet')
        """, (img['image_id'],))

        people_count = themis.query("""
            SELECT COUNT(*) as person_count
            FROM detected_objects
            WHERE image_id = ?
              AND object_class = 'person'
        """, (img['image_id'],))

        if helmets[0]['helmet_count'] < people_count[0]['person_count']:
            violations.append({
                'image_id': img['image_id'],
                'filename': img['filename'],
                'issue': 'Missing hardhat',
                'people_count': people_count[0]['person_count'],
                'helmet_count': helmets[0]['helmet_count']
            })

    return violations

violations = detect_safety_violations(flight_id)
if violations:
    print(f"⚠  Found {len(violations)} safety violations:")
    for v in violations:
        print(f"  - {v['filename']}: {v['issue']}")
```

Change Detection zwischen Flügen

```
def compare_flights(flight_id_1, flight_id_2, similarity_threshold=0.85):
    """Vergleiche zwei Flüge und finde Änderungen"""
    # Hole Bilder von beiden Flügen
    images_1 = reconstruct_flight_path(flight_id_1)
    images_2 = reconstruct_flight_path(flight_id_2)

    changes = []

    for img1 in images_1:
        # Finde räumlich nächstes Bild aus Flug 2
        nearest_img2 = themis.query_one("""
            SELECT
                i2.image_id,
                i2.filename,
                i2.captured_at,
                ST_Distance(i1.location, i2.location) as distance_m,
                1 - (f2.deep_features <=> f1.deep_features) as visual_similarity
            FROM images i1
            JOIN image_features f1 ON i1.image_id = f1.image_id
            CROSS JOIN images i2
            JOIN image_features f2 ON i2.image_id = f2.image_id
            WHERE i1.image_id = ?
              AND ? = ANY(i2.tags)
            ORDER BY ST_Distance(i1.location, i2.location)
            LIMIT 1
        """, (img1['image_id'], flight_id_2))

        if nearest_img2 and nearest_img2['distance_m'] < 10:  # Innerhalb 10m
            if nearest_img2['visual_similarity'] < similarity_threshold:
                changes.append({
                    'location': (img1['gps_latitude'], img1['gps_longitude']),
                    'image_1': img1['filename'],
                    'image_2': nearest_img2['filename'],
                    'similarity': nearest_img2['visual_similarity'],
                    'change_detected': True
                })

    return changes

changes = compare_flights('flight_001', 'flight_002')
print(f"Detected {len(changes)} significant changes between flights")
```

12.7 Performance & Skalierung

Thumbnail-Generierung für schnelle Preview

```
from PIL import Image

def generate_thumbnail(image_path, size=(256, 256)):
    """Generiere Thumbnail für Preview"""
    img = Image.open(image_path)
    img.thumbnail(size, Image.LANCZOS)

    # Speichere Thumbnail
    thumb_path = image_path.replace('.jpg', '_thumb.jpg')
    img.save(thumb_path, 'JPEG', quality=85)

    # Update DB
    themis.execute("""
        UPDATE images
        SET metadata = jsonb_set(metadata, '{thumbnail_path}', ?)
        WHERE file_path = ?
    """, (f'"{thumb_path}"', image_path))

    return thumb_path
```

Lazy Loading von Features

```
-- Erstelle Features nur on-demand
CREATE OR REPLACE FUNCTION ensure_features(p_image_id UUID)
RETURNS BOOLEAN AS $$
BEGIN
    LET feature_exists = (
        FOR feature IN image_features
          FILTER feature.image_id == p_image_id
          LIMIT 1
          RETURN 1
    )

    IF LENGTH(feature_exists) == 0 THEN
        -- Trigger Feature-Extraktion
        INSERT {image_id: p_image_id} INTO feature_extraction_queue;
        RETURN FALSE;
    END IF;
    RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

Caching für häufige Queries

```
from functools import lru_cache
import hashlib

@lru_cache(maxsize=100)
def get_similar_images_cached(image_path_hash, limit=10):
    """Gecachte Similarity Search"""
    # ... similarity search logic
    pass

image_hash = hashlib.md5(open(image_path, 'rb').read()).hexdigest()
similar = get_similar_images_cached(image_hash, limit=10)
```

12.8 Best Practices

1. Storage-Optimierung

DO: - Speichere nur Metadata in DB, Bilder im Object Storage (S3, MinIO) - Generiere Thumbnails für Previews - Nutze komprimierte Formate (JPEG für Photos, WebP für Web)

× **DON'T:** - Speichere nicht alle Bilder als BYTEA in DB - Vermeide unkomprimierte Formate (BMP, TIFF) für Archivierung - Keine Features ohne Index

2. Feature-Extraktion

DO: - Nutze Pre-Trained Models (ResNet, EfficientNet) - Batch-Processing für viele Bilder - GPU-Acceleration für Deep Learning

× **DON'T:** - Extrahiere Features nicht synchron beim Upload - Vermeide Training von Modellen auf CPU - Keine Feature-Extraktion ohne Quality-Check

3. Similarity Search

DO: - HNSW Index für große Datasets - Kombiniere Visual + Text Search - Pre-Filter mit Metadata (Zeit, Ort)

× **DON'T:** - Keine Brute-Force Search über Millionen Bilder - Vermeide hohe Dimensions ohne Dimensionality Reduction - Keine Similarity ohne Normalisierung

12.9 Zusammenfassung

ThemisDB ermöglicht effiziente Computer Vision Anwendungen durch:

Multi-Modal Data Management: - Images + GPS + Features + Objects in einer DB - Spatial + Temporal + Visual Queries - Graph-basierte Analyse von Bild-Beziehungen

Flexible Feature-Speicherung: - Native VECTOR-Spalten für Embeddings - HNSW Index für schnelle Similarity Search - JSON für flexible Metadata

Production-Ready: - Skalierbare Storage (Partitionierung) - Batch-Processing Support - Integration mit ML-Frameworks

Key Takeaways: 1. Speichere Metadata in DB, Bilder in Object Storage 2. Generiere Features asynchron (Queue/Workers) 3. Nutze HNSW für Similarity Search 4. Kombiniere Visual + Spatial + Temporal Queries 5. Thumbnails für Performance

---

Kapitel 14: Kapitel 13: Volltext-Suche & NLP

Einführung

Die Volltext-Suche ist eine der fundamentalen Anforderungen moderner Anwendungen. Ob E-Commerce-Produktsuche, Dokumentenverwaltung oder Content-Management - Nutzer erwarten relevante, schnelle Suchergebnisse. ThemisDB bietet native Volltext-Funktionalität, die nahtlos mit anderen Datenmodellen integriert ist.

In diesem Kapitel behandeln wir:

- **Grundlagen der Volltext-Suche:** Tokenisierung, Stemming, Stop-Words
- **ThemisDB Fulltext-Indexe:** Konfiguration und Optimierung
- **Erweiterte Suchfunktionen:** Wildcards, Phrasensuche, Fuzzy Search
- **NLP-Integration:** Sentiment-Analyse, Entity-Erkennung, Sprachverarbeitung
- **Best Practices:** Performance, Relevanz-Ranking, mehrsprachige Suche

Warum ist Volltext-Suche wichtig?

Problem der einfachen String-Suche:

```
-- Ineffizient: Keine Relevanz, langsam bei großen Datensätzen
FOR article IN articles
  FILTER LIKE(article.title, '%database%') OR LIKE(article.content, '%database%')
  RETURN article
```

Probleme: - Keine Wort-Trennung (findet "database" nicht in "databases") - Keine Relevanz-Bewertung - Performance-Probleme (Full Table Scan) - Keine sprachliche Verarbeitung

Lösung mit Volltext-Index:

```
-- Schnell, relevant, sprachlich intelligent
FOR article IN articles
  FILTER FULLTEXT(article.title, 'database') OR FULLTEXT(article.content, 'database')
  LET score = FULLTEXT_SCORE(article)
  SORT score DESC
  RETURN {article, score}
```

Vorteile: - Automatische Wort-Erkennung und Stammformbildung - Relevanz-Ranking (TF-IDF, BM25) - Sehr performant durch invertierte Indizes - Sprachliche Features (Stop-Words, Synonyme)

Grundlagen der Volltext-Suche

Text-Verarbeitung Pipeline

Die Volltext-Suche durchläuft mehrere Verarbeitungsschritte:

1. Tokenisierung

Zerlegt Text in einzelne Wörter (Tokens):

```
"ThemisDB ist eine Multi-Model Datenbank"
↓
["ThemisDB", "ist", "eine", "Multi-Model", "Datenbank"]
```

2. Normalisierung

Konvertiert Tokens in Grundform:

```
["ThemisDB", "ist", "eine", "Multi-Model", "Datenbank"]
↓
["themisdb", "ist", "eine", "multi-model", "datenbank"]  # Lowercase
```

3. Stop-Word-Filterung

Entfernt häufige, nicht-informative Wörter:

```
["themisdb", "ist", "eine", "multi-model", "datenbank"]
↓
["themisdb", "multi-model", "datenbank"]  # "ist", "eine" entfernt
```

4. Stemming

Reduziert Wörter auf Wortstamm:

```
["Datenbanken", "Datenbank", "Datenbankserver"]
↓
["datenbank", "datenbank", "datenbankserv"]  # Gemeinsamer Stamm
```

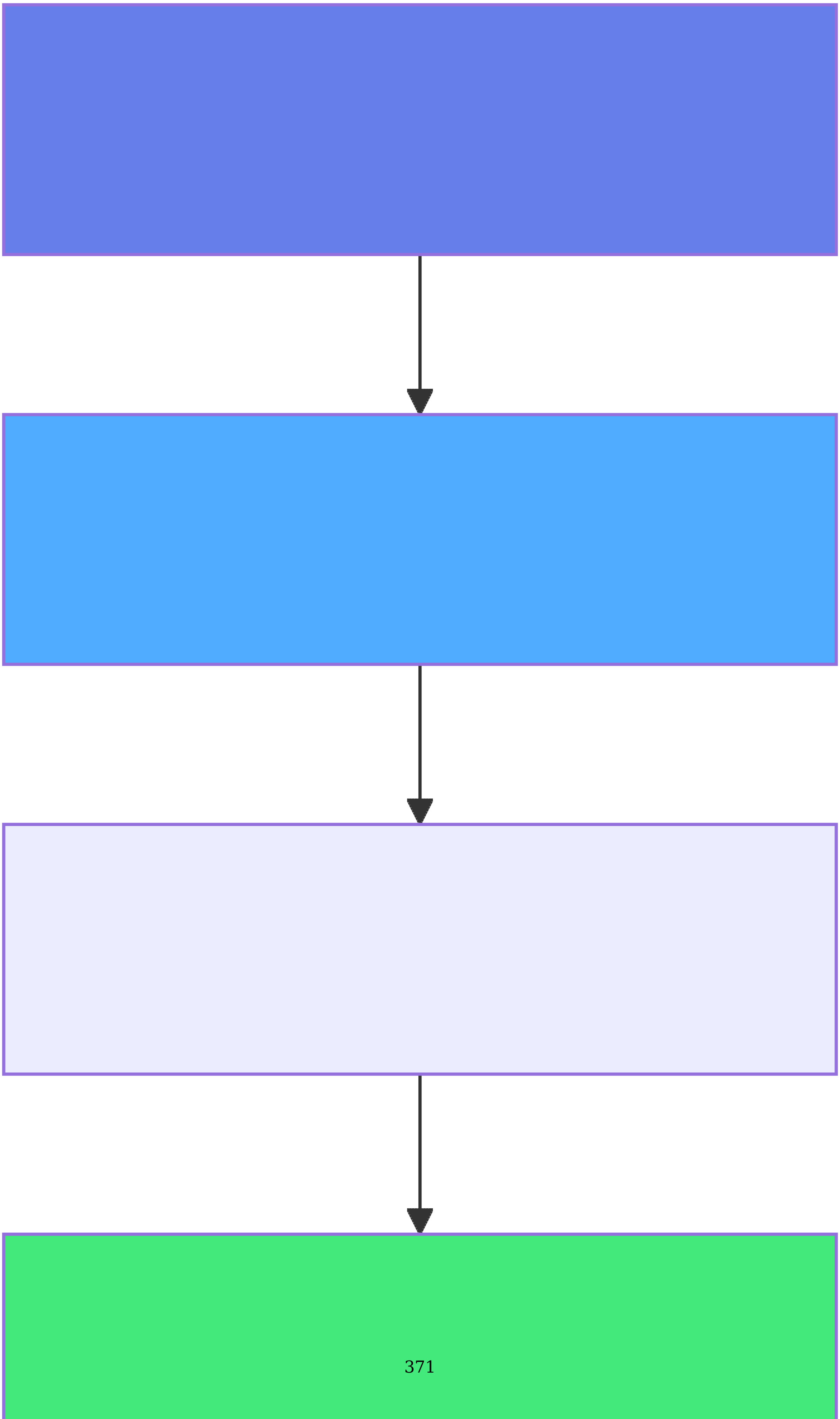


Abb. 36: Ablaufdiagramm: Gemeinsamer Stamm

5. Index-Erstellung

Erstellt inverted index für schnelle Suche:

```
Token         → Dokument-IDs
"themisdb"    → [1, 5, 12, 89]
"datenbank"   → [1, 2, 5, 12, 34, 89]
"multi-model"→ [1, 12]
```

Relevanz-Ranking Algorithmen

TF-IDF (Term Frequency - Inverse Document Frequency)

Bewertet Relevanz basierend auf: - **TF:** Wie oft erscheint der Begriff im Dokument? - **IDF:** Wie selten ist der Begriff im gesamten Korpus?

```
TF-IDF = TF × IDF
TF = (Anzahl Begriff in Dokument) / (Gesamtzahl Wörter in Dokument)
IDF = log(Gesamtanzahl Dokumente / Anzahl Dokumente mit Begriff)
```

Beispiel:

Dokument: "ThemisDB ist eine Datenbank. ThemisDB unterstützt Multi-Model." Suchbegriff: "ThemisDB"

```
TF = 2 / 8 = 0.25  (2× "ThemisDB", 8 Wörter gesamt)
IDF = log(10000 / 100) = 2  (100 von 10000 Dokumenten enthalten "ThemisDB")
TF-IDF = 0.25 × 2 = 0.5
```

BM25 (Best Matching 25)

Verbessertes Ranking-Modell mit Sättigungsfunktion:

```
BM25 = IDF × (TF × (k1 + 1)) / (TF + k1 × (1 - b + b × (docLen / avgDocLen)))
```

Parameter: - **k1:** Sättigung für Term-Frequenz (typisch 1.2-2.0) - **b:** Dokumentlängen-Normalisierung (typisch 0.75)

Vorteil: Verhindert, dass sehr lange Dokumente überproportional hohe Scores bekommen.

Volltext-Indexe in ThemisDB

Index erstellen

Einfacher Fulltext-Index:

```
CREATE FULLTEXT INDEX idx_articles_content
ON articles (title, content);
```

Mit Konfiguration:

```
CREATE FULLTEXT INDEX idx_articles_content
ON articles (title, content)
WITH (
  analyzer = 'german',            -- Sprach-Analyzer
  stopwords = ['der', 'die', 'das'],  -- Custom Stop-Words
  min_word_length = 3,            -- Minimale Wortlänge
  max_word_length = 50,           -- Maximale Wortlänge
  case_sensitive = false,         -- Groß-/Kleinschreibung
  stemming = true                 -- Stammform-Reduktion
);
```

Gewichtete Felder:

```
CREATE FULLTEXT INDEX idx_articles_weighted
ON articles (
  title WEIGHT 3.0,       -- Titel 3× wichtiger
  abstract WEIGHT 2.0,    -- Abstract 2× wichtiger
  content WEIGHT 1.0      -- Content normale Gewichtung
);
```

Suchen mit Volltext-Index

Einfache Suche:

```
FOR article IN articles
  FILTER FULLTEXT(article, 'database systems')
  RETURN article
```

Mit Relevanz-Score:

```
FOR article IN articles
  FILTER FULLTEXT(article, 'database systems')
  LET relevance = FULLTEXT_SCORE(article)
  SORT relevance DESC
  LIMIT 10
  RETURN {
    id: article.id,
    title: article.title,
    relevance
  }
```

Mehrere Begriffe (AND):

```
-- Alle Begriffe müssen vorkommen
WHERE FULLTEXT_MATCH(articles, 'database AND multi-model');
```

Alternative Begriffe (OR):

```
-- Mindestens ein Begriff muss vorkommen
WHERE FULLTEXT_MATCH(articles, 'database OR nosql');
```

Ausschluss (NOT):

```
-- Enthält "database" aber nicht "relational"
WHERE FULLTEXT_MATCH(articles, 'database NOT relational');
```

Phrasensuche:

```
-- Exakte Phrase in Anführungszeichen
WHERE FULLTEXT_MATCH(articles, '"multi-model database"');
```

Wildcard-Suche:

```
-- * für beliebige Zeichen
WHERE FULLTEXT_MATCH(articles, 'data*');  -- Findet "database", "dataset", etc.
```

Fuzzy-Suche (Rechtschreibfehler):

```
-- ~ für Fuzzy-Match (Levenshtein-Distanz)
WHERE FULLTEXT_MATCH(articles, 'databse~');  -- Findet "database" trotz Fehler
```

Feld-spezifische Suche

Nur in bestimmten Feldern suchen:

```
FOR article IN articles
  FILTER FULLTEXT(article.title, 'database') AND FULLTEXT(article.content, 'nosql')
  RETURN article
```

Kombinierte Suche:

```
FOR article IN articles
  FILTER (FULLTEXT(article.title, 'multi-model') OR FULLTEXT(article.content, 'vector'))
    AND article.category == 'technology'  -- Zusätzliche Filter kombinierbar
  RETURN article
```

Erweiterte Suchfunktionen

Highlighting

Markiert Suchbegriffe in Ergebnissen:

```
FOR article IN articles
  FILTER FULLTEXT(article, 'database')
  LIMIT 10
  RETURN {
    id: article.id,
    title: article.title,
    snippet: FULLTEXT_HIGHLIGHT(article.content, 'database', '<mark>', '</mark>')
  }
```

Ergebnis:

```
"... ThemisDB ist eine <mark>database</mark> für Multi-Model Daten ..."
```

Snippet-Extraktion:

```
FOR article IN articles
  FILTER FULLTEXT(article, 'database')
  RETURN {
    id: article.id,
    title: article.title,
    snippet: FULLTEXT_SNIPPET(article.content, 'database', 50, 200)
  }
```

Extrahiert 200 Zeichen um den Suchbegriff herum (50 Zeichen vor, 150 nach).

Auto-Complete / Suggest

Präfix-Suche für Auto-Complete:

```
-- Findet alle Artikel, die mit "dat" beginnen
FOR article IN articles
  LET suggestion = FULLTEXT_TERMS(article, 'dat*')
  FILTER suggestion != null
  LIMIT 10
  RETURN DISTINCT suggestion
```

Ergebnis:

```
["database", "data", "dataset", "dataflow"]
```

Häufigkeits-basierte Vorschläge:

```
FOR article IN articles
  LET term = FULLTEXT_TERMS(article, 'dat*')
  FILTER term != null
  COLLECT termValue = term
  AGGREGATE frequency = COUNT()
  SORT frequency DESC
  LIMIT 10
  RETURN {term: termValue, frequency}
```

Faceted Search

Kombiniert Volltext mit Facetten:

```
FOR article IN articles
  FILTER FULLTEXT(article, 'database')
  COLLECT category = article.category
  AGGREGATE count = COUNT()
  RETURN {category, count}
```

Ergebnis:

```
category        | count
----------------|------
Technology      | 45
Science         | 23
Business        | 12
```

Komplexe Facetten-Abfrage:

```
LET results = (
  FOR article IN articles
    FILTER FULLTEXT(article, 'database')
    RETURN article
)

FOR r IN results
    FILTER FULLTEXT(r, 'database')
    RETURN r
)

FOR result IN results
  COLLECT category = result.category
  AGGREGATE
    count = COUNT(),
    avg_relevance = AVG(FULLTEXT_SCORE(result))
  SORT count DESC
  RETURN {category, count, avg_relevance}
```

NLP-Integration

Sentiment-Analyse

Analysiert Stimmung/Emotion in Texten.

Python Integration mit TextBlob:

```
from themisdb import ThemisDB
from textblob import TextBlob

db = ThemisDB()

reviews = db.query("""
    FOR review IN product_reviews
      RETURN {id: review.id, text: review.text}
""")

for review in reviews:
    # Sentiment-Analyse
    blob = TextBlob(review['text'])
    sentiment = blob.sentiment.polarity  # -1 (negativ) bis +1 (positiv)

    # Sentiment in DB speichern
    db.query("""
        FOR review IN product_reviews
          FILTER review.id == @review_id
          UPDATE review WITH {sentiment_score: @sentiment} IN product_reviews
    """, {"sentiment": sentiment, "review_id": review['id']})

avg_sentiment = db.query("""
    FOR review IN product_reviews
      COLLECT product_id = review.product_id
      AGGREGATE
        avg_sentiment = AVG(review.sentiment_score),
        review_count = COUNT()
      FILTER review_count >= 10
      SORT avg_sentiment DESC
      RETURN {product_id, avg_sentiment, review_count}
""")
```

Sentiment-Kategorien:

```python
def categorize_sentiment(score):
    if score >= 0.5:
        return "sehr positiv"
    elif score >= 0.1:
        return "positiv"
    elif score >= -0.1:
        return "neutral"
    elif score >= -0.5:
        return "negativ"
    else:
        return "sehr negativ"

db.query("""
    ALTER TABLE product_reviews
    ADD COLUMN sentiment_category VARCHAR(20)
""")

for review in reviews:
    category = categorize_sentiment(review['sentiment_score'])
    db.query("""
        UPDATE product_reviews
        SET sentiment_category = ?
        WHERE id = ?
    """, [category, review['id']])
```

Named Entity Recognition (NER)

Erkennt Entitäten wie Personen, Orte, Organisationen.

Mit spaCy:

```python
import spacy
from themisdb import ThemisDB

nlp = spacy.load("de_core_news_md")
db = ThemisDB()

articles = db.query("SELECT id, content FROM articles")

for article in articles:
    doc = nlp(article['content'])

    # Entitäten extrahieren
    entities = []
    for ent in doc.ents:
        entities.append({
            'text': ent.text,
            'label': ent.label_,  # PER, LOC, ORG, etc.
            'start': ent.start_char,
            'end': ent.end_char
        })

    # Entitäten als JSON speichern
    db.query("""
        UPDATE articles
        SET entities = ?
        WHERE id = ?
    """, [json.dumps(entities), article['id']])

db.query("""
    SELECT * FROM articles
    WHERE JSON_CONTAINS(entities, '$.text', 'Angela Merkel')
""")
```

Entity-Linking:

```python
def link_entities(article_id, entities):
    for ent in entities:
        if ent['label'] == 'PER':  # Person
            # Suche Person in DB
            person = db.query("""
                SELECT id FROM persons
                WHERE name = ? OR aliases LIKE ?
            """, [ent['text'], f"%{ent['text']}%"])

            if person:
                # Verknüpfung erstellen
                db.query("""
                    INSERT INTO article_person_mentions
                    (article_id, person_id, mention_text)
                    VALUES (?, ?, ?)
                """, [article_id, person[0]['id'], ent['text']])
```

Text-Klassifikation

Automatische Kategorisierung von Texten.

Training eines Klassifikators:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from themisdb import ThemisDB

db = ThemisDB()

training_data = db.query("""
    SELECT content, category
    FROM articles
    WHERE category IS NOT NULL
""")

X_train = [row['content'] for row in training_data]
y_train = [row['category'] for row in training_data]

classifier = Pipeline([
    ('vectorizer', TfidfVectorizer(max_features=5000)),
    ('classifier', MultinomialNB())
])

classifier.fit(X_train, y_train)

new_articles = db.query("""
    SELECT id, content
    FROM articles
    WHERE category IS NULL
""")

for article in new_articles:
    predicted_category = classifier.predict([article['content']])[0]
    confidence = max(classifier.predict_proba([article['content']])[0])

    db.query("""
        UPDATE articles
        SET category = ?, category_confidence = ?
        WHERE id = ?
    """, [predicted_category, confidence, article['id']])
```

Keyword-Extraktion

Extrahiert wichtige Schlüsselwörter aus Texten.

Mit YAKE (Yet Another Keyword Extractor):

```
import yake
from themisdb import ThemisDB

db = ThemisDB()

kw_extractor = yake.KeywordExtractor(
    lan="de",           # Sprache
    n=3,                # Maximale N-Gram Länge
    dedupLim=0.9,       # Deduplizierung
    top=10,             # Top 10 Keywords
    features=None
)

articles = db.query("SELECT id, title, content FROM articles")

for article in articles:
    text = article['title'] + " " + article['content']
    keywords = kw_extractor.extract_keywords(text)

    # Keywords als Liste speichern
    keyword_list = [kw[0] for kw in keywords]

    db.query("""
        UPDATE articles
        SET keywords = ?
        WHERE id = ?
    """, [json.dumps(keyword_list), article['id']])

db.query("""
    SELECT * FROM articles
    WHERE JSON_CONTAINS(keywords, ?, '$')
""", ["Multi-Model"])
```

Mehrsprachige Suche

Language Detection

Automatische Spracherkennung:

```python
from langdetect import detect
from themisdb import ThemisDB

db = ThemisDB()

articles = db.query("SELECT id, content FROM articles")

for article in articles:
    try:
        language = detect(article['content'])
        db.query("""
            UPDATE articles
            SET language = ?
            WHERE id = ?
        """, [language, article['id']])
    except:
        # Fallback bei Fehler
        db.query("""
            UPDATE articles
            SET language = 'unknown'
            WHERE id = ?
        """, [article['id']])
```

Sprachspezifische Indexe

Separate Indexe pro Sprache:

```sql
-- Deutscher Index
CREATE FULLTEXT INDEX idx_articles_de
ON articles (content)
WHERE language = 'de'
WITH (analyzer = 'german');

-- Englischer Index
CREATE FULLTEXT INDEX idx_articles_en
ON articles (content)
WHERE language = 'en'
WITH (analyzer = 'english');

-- Französischer Index
CREATE FULLTEXT INDEX idx_articles_fr
ON articles (content)
WHERE language = 'fr'
WITH (analyzer = 'french');
```

Sprachabhängige Suche:

```sql
-- Automatische Sprachwahl basierend auf Nutzer-Präferenz
SELECT * FROM articles
WHERE language = 'de'
  AND FULLTEXT_MATCH(articles, 'Datenbank')
ORDER BY FULLTEXT_SCORE(articles) DESC;
```

Translation-Aware Search

Suche über Übersetzungen hinweg:

```python
from googletrans import Translator
from themisdb import ThemisDB

translator = Translator()
db = ThemisDB()

def multilingual_search(query, target_languages=['en', 'de', 'fr']):
    results = []

    for lang in target_languages:
        # Query in Zielsprache übersetzen
        if lang != 'de':  # Annahme: Query ist deutsch
            translated_query = translator.translate(query, dest=lang).text
        else:
            translated_query = query

        # Suche in entsprechender Sprache
        lang_results = db.query("""
            SELECT *, ? as search_language
            FROM articles
            WHERE language = ?
              AND FULLTEXT_MATCH(articles, ?)
            ORDER BY FULLTEXT_SCORE(articles) DESC
            LIMIT 10
        """, [lang, lang, translated_query])

        results.extend(lang_results)

    return results

results = multilingual_search("Datenbank")
```

Performance-Optimierung

Index-Tuning

Analyse der Index-Nutzung:

```sql
-- Index-Statistiken anzeigen
SHOW INDEX STATS idx_articles_content;
```

Ergebnis:

```
total_documents: 125000
total_terms: 2500000
avg_terms_per_doc: 20
index_size_mb: 45.2
last_update: 2024-12-28 10:30:00
```

Selective Indexing:

```
-- Nur wichtige Felder indizieren
CREATE FULLTEXT INDEX idx_articles_selective
ON articles (title, abstract)  -- Nicht "content" für Performance
WHERE status = 'published';    -- Nur veröffentlichte Artikel
```

Partial Index für häufige Queries:

```
-- Index nur für aktuelle Artikel
CREATE FULLTEXT INDEX idx_articles_recent
ON articles (title, content)
WHERE created_at > NOW() - INTERVAL '30 days';
```

Query-Optimierung

Avoid Wildcards am Anfang:

```
-- LANGSAM: Wildcard am Anfang
WHERE FULLTEXT_MATCH(articles, '*base');

-- SCHNELL: Wildcard am Ende
WHERE FULLTEXT_MATCH(articles, 'data*');
```

Limit Fuzzy Search:

```
-- LANGSAM: Fuzzy auf allen Begriffen
WHERE FULLTEXT_MATCH(articles, 'databse~ systm~');

-- SCHNELL: Fuzzy nur wo nötig
WHERE FULLTEXT_MATCH(articles, 'database systm~');
```

Use Score Threshold:

```sql
-- Filtere irrelevante Ergebnisse
SELECT * FROM articles
WHERE FULLTEXT_MATCH(articles, 'database')
  AND FULLTEXT_SCORE(articles) > 0.5  -- Nur relevante Treffer
ORDER BY FULLTEXT_SCORE(articles) DESC
LIMIT 20;
```

Caching-Strategien

Query-Result-Cache:

```python
from functools import lru_cache
from themisdb import ThemisDB

db = ThemisDB()

@lru_cache(maxsize=1000)
def cached_search(query, limit=20):
    return db.query("""
        SELECT * FROM articles
        WHERE FULLTEXT_MATCH(articles, ?)
        ORDER BY FULLTEXT_SCORE(articles) DESC
        LIMIT ?
    """, [query, limit])

results1 = cached_search("database")  # DB-Query
results2 = cached_search("database")  # Aus Cache
```

Index-Warm-Up:

```sql
-- Index vorwärmen nach Restart
SELECT COUNT(*) FROM articles
WHERE FULLTEXT_MATCH(articles, 'a');  -- Häufiger Buchstabe
```

Best Practices

1. Index-Design

 DO: - Indiziere nur durchsuchbare Felder - Verwende gewichtete Felder (Titel wichtiger als Content) - Nutze sprachspezifische Analyzer - Implementiere Partial Indexes für große Tabellen

× **DON'T:** - Alle Felder blindlings indizieren - Binärdaten oder IDs indizieren - Stop-Words komplett entfernen (bei Phrasensuche wichtig) - Zu aggressive Stemming-Regeln

2. Query-Patterns

DO: - Kombiniere Volltext mit strukturierten Filtern - Verwende Relevanz-Ranking - Implementiere Pagination für große Result-Sets - Cache häufige Queries

× **DON'T:** - Wildcard-Suchen am Anfang (*word) - Zu komplexe Boolean-Queries - Uneingeschränkte Fuzzy-Searches - Volltext-Suche für exakte Matches (verwende =)

3. User Experience

DO: - Auto-Complete für bessere UX - Highlight Suchbegriffe in Ergebnissen - Zeige Relevanz-Score an - Implementiere "Did you mean?" für Tippfehler

× **DON'T:** - Keine Ergebnisse ohne Alternative - Zu viele Ergebnisse ohne Ranking - Langsame Suche ohne Loading-Indicator - Keine Filteroptionen

4. Wartung

DO: - Regelmäßige Index-Optimierung - Monitoring der Query-Performance - Analyse beliebter Suchbegriffe - A/B-Testing verschiedener Ranking-Algorithmen

× **DON'T:** - Index-Fragmentierung ignorieren - Keine Metriken erfassen - Statische Ranking-Gewichte - Keine User-Feedback-Integration

---

Zusammenfassung

Volltext-Suche ist essentiell für moderne Anwendungen:

Kernkonzepte: - Invertierte Indexe für schnelle Suche - TF-IDF / BM25 für Relevanz-Ranking - Sprachverarbeitung (Stemming, Stop-Words) - Boolean-Operatoren (AND, OR, NOT) - Erweiterte Features (Fuzzy, Wildcards, Phrases)

NLP-Integration: - Sentiment-Analyse für Meinungen - Named Entity Recognition für Entitäten - Text-Klassifikation für Auto-Tagging - Keyword-Extraktion für Zusammenfassungen

Performance: - Selective Indexing - Query-Optimierung - Caching-Strategien - Index-Wartung

ThemisDB bietet native Volltext-Funktionalität, die nahtlos mit anderen Datenmodellen kombiniert werden kann - für leistungsstarke, flexible Suchlösungen.

Im nächsten Kapitel behandeln wir **Geo-Spatial Features** - für standortbasierte Anwendungen und räumliche Analysen.

Übungsaufgaben

Aufgabe 1: Erstellen Sie einen gewichteten Volltext-Index für eine Nachrichten-Tabelle (Titel 3×, Teaser 2×, Content 1×).

Aufgabe 2: Implementieren Sie eine Auto-Complete-Funktion mit Präfix-Suche und Häufigkeits-Ranking.

Aufgabe 3: Integrieren Sie Sentiment-Analyse für Produkt-Reviews und berechnen Sie durchschnittliche Sentiment-Scores.

Aufgabe 4: Erstellen Sie eine mehrsprachige Suche mit automatischer Language-Detection und sprachspezifischen Indexen.

Aufgabe 5: Optimieren Sie eine langsame Volltext-Query durch Analyse der Index-Statistiken und Query-Rewriting.

Kapitel 15: Kapitel 14: Geo-Spatial Features

Einleitung

Standortbasierte Dienste sind aus modernen Anwendungen nicht mehr wegzudenken. Von Lieferdiensten über Immobiliensuche bis zu Flottenmanagement – geografische Daten spielen eine zentrale Rolle. ThemisDB bietet native Unterstützung für Geo-Spatial Daten mit effizienten räumlichen Indizes und umfangreichen Abfragefunktionen.

In diesem Kapitel lernen Sie: - Geo-Datentypen und Koordinatensysteme - Räumliche Indizes (R-Tree, Geo-Hash) - Location-Based Queries (Radius, Bounding Box, Polygon) - Integration von GPS-Tracking und Kartendiensten - Best Practices für Geo-Anwendungen

14.1 Grundlagen der Geo-Spatial Daten

Koordinatensysteme

ThemisDB verwendet das WGS84-Koordinatensystem (EPSG:4326), das auch GPS verwendet:

```
berlin_coordinates = [13.405, 52.520]
munich_coordinates = [11.575, 48.137]
```

Geo-Datentypen in ThemisDB

```
// Point: Einzelner Punkt
FOR loc IN [
  {
    _key: "loc1",
    name: "Berlin Mitte",
    coordinates: [13.4050, 52.5200],  // [lon, lat]
    created_at: DATE_ISO8601(DATE_NOW())
  }
] INSERT loc INTO locations

-- Geo-Index erstellen
CREATE INDEX idx_geo_locations
  ON locations (coordinates)
  TYPE GEO

// LineString: Verbundene Punkte (Routen)
FOR route IN [
  {
    _key: "route1",
    name: "Tour A",
    path: [
      [13.377, 52.516],  // Start
      [13.390, 52.520],  // Waypoint 1
      [13.405, 52.520]   // End
    ],
    distance_km: 2.8
  }
] INSERT route INTO routes

// Polygon: Geschlossene Fläche (Delivery Zones)
FOR zone IN [
  {
    _key: "zone1",
    name: "Zone Nord",
    area: [
      [13.35, 52.54],
      [13.42, 52.54],
      [13.42, 52.50],
      [13.35, 52.50],
      [13.35, 52.54]  // Geschlossen
    ],
    active: true
  }
] INSERT zone INTO delivery_zones
```

14.2 Geo-Spatial Indizes

R-Tree Index

R-Trees sind die effizienteste Indexstruktur für räumliche Daten:

```
-- Geo-Index erstellen
CREATE INDEX idx_locations_geo ON locations USING RTREE(coordinates);

-- Automatische Nutzung bei räumlichen Queries
FOR location IN locations
  FILTER ST_Distance(location.coordinates, POINT(13.405, 52.520)) < 5000
  RETURN location
-- Index wird automatisch genutzt!
```

R-Tree Eigenschaften: - Organisiert Punkte in hierarchischen Bounding Boxes - O(log n) Suchzeit für Bereichsabfragen - Automatische Rebalancierung bei Updates - Optimiert für Nearest-Neighbor Queries

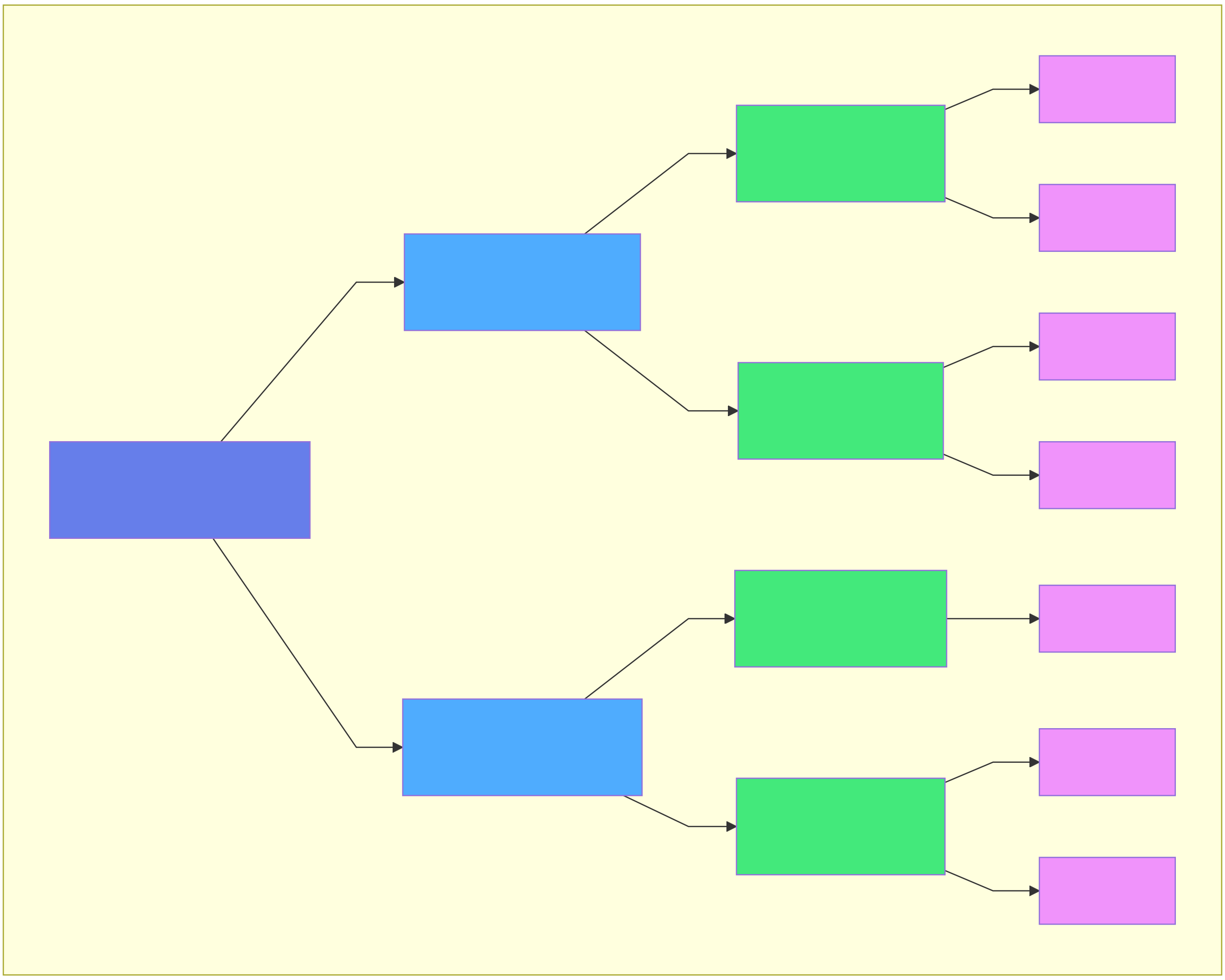


Abb. 37: Architekturdiagramm (Kapitel 14: Geo-Spatial Features)

Geo-Hash Index

Für weltweite Abdeckung und Präfix-Suche:

```
CREATE INDEX idx_locations_geohash ON locations USING GEOHASH(coordinates);

-- Nützlich für:
-- - Clustering auf Karten
-- - Hierarchische Zoomlevel
-- - Verteilte Systeme (Sharding nach Geo-Hash)
```

14.3 Räumliche Abfragen

Distanz-Queries

```
from themisdb import Connection

conn = Connection("localhost:7687")

restaurants = conn.query("""
    SELECT id, name, coordinates,
           ST_Distance(coordinates, POINT(?, ?)) as distance_m
    FROM restaurants
    WHERE ST_Distance(coordinates, POINT(?, ?)) < 2000
    ORDER BY distance_m
""", [user_lon, user_lat, user_lon, user_lat])

for r in restaurants:
    print(f"{r['name']}: {r['distance_m']:.0f}m entfernt")
```

Bounding Box Queries

Rechteckiger Bereich (sehr effizient mit R-Tree):

```
min_lon, max_lon = 13.35, 13.45
min_lat, max_lat = 52.50, 52.55

pois = conn.query("""
    FOR poi IN points_of_interest
      FILTER ST_Within(poi.coordinates,
                      BBOX(@min_lon, @min_lat, @max_lon, @max_lat))
      RETURN poi
""", {"min_lon": min_lon, "min_lat": min_lat, "max_lon": max_lon, "max_lat": max_lat})
```

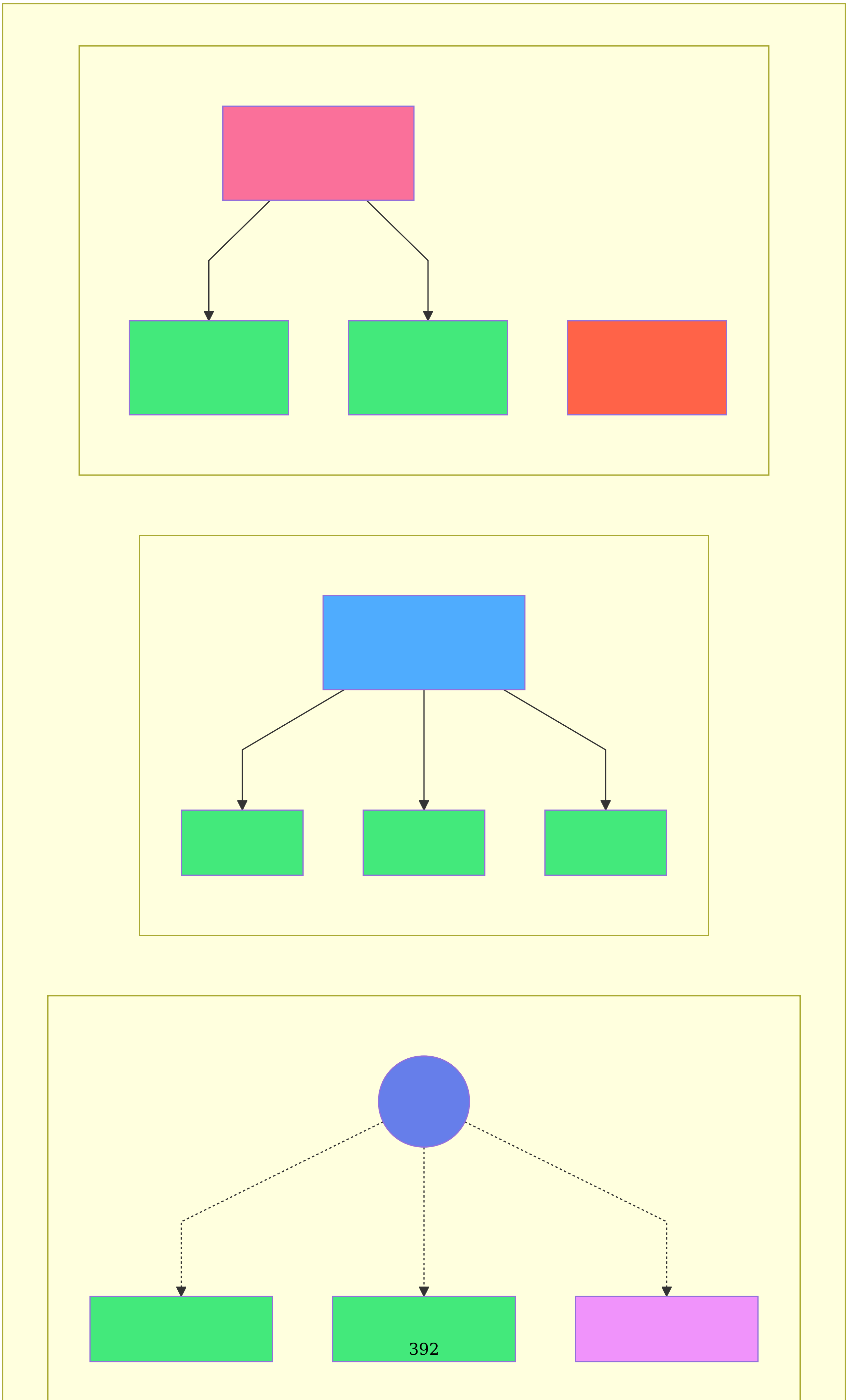


Abb. 38: Architekturdiagramm (Kapitel 14: Geo-Spatial Features)

Polygon Queries

Prüfen ob Punkt innerhalb eines Polygons liegt:

```
delivery_area = [
    [13.40, 52.52],  # Punkt 1
    [13.42, 52.52],  # Punkt 2
    [13.42, 52.50],  # Punkt 3
    [13.40, 52.50],  # Punkt 4
    [13.40, 52.52]   # Schließt Polygon
]

customer_location = [13.41, 52.51]

result = conn.query("""
    SELECT ST_Contains(
        POLYGON(?),
        POINT(?, ?)
    ) as in_delivery_area
""", [delivery_area, customer_location[0], customer_location[1]])

if result[0]['in_delivery_area']:
    print("Wir liefern zu Ihnen!")
```

K-Nearest-Neighbor (KNN)

Die K nächsten Punkte finden:

```
cafes = conn.query("""
    SELECT id, name, coordinates,
           ST_Distance(coordinates, POINT(?, ?)) as distance_m
    FROM cafes
    ORDER BY distance_m
    LIMIT 5
""", [user_lon, user_lat])
```

14.4 Geo-Funktionen

Distanzberechnung

```
-- Haversine-Formel (Kugeloberfläche der Erde)
ST_Distance(point1, point2) -> meters

-- Beispiel: Berlin → München
SELECT ST_Distance(
    POINT(13.405, 52.520),  -- Berlin
    POINT(11.575, 48.137)   -- München
) as distance_m;
-- Ergebnis: ~504.000m (504km)
```

Bearing (Richtung)

```
-- Peilung von A nach B in Grad (0° = Norden)
ST_Bearing(point1, point2) -> degrees

SELECT ST_Bearing(
    POINT(13.405, 52.520),  -- Berlin
    POINT(11.575, 48.137)   -- München
) as bearing;
-- Ergebnis: ~195° (Süd-Südwest)
```

Bounding Box

```
-- Kleinste Box um Geometrie
ST_Envelope(geometry) -> bbox

-- Buffer: Bereich um Punkt/Linie
ST_Buffer(point, radius_m) -> polygon
```

14.5 Beispiel: Location-Based Services (LBS)

Wir bauen einen Lieferdienst mit Echtzeit-Tracking:

Datenmodell

```
conn.execute("""
CREATE TABLE IF NOT EXISTS restaurants (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    cuisine TEXT,
    coordinates POINT NOT NULL,
    rating REAL,
    delivery_radius_m INTEGER DEFAULT 3000
)
""")

conn.execute("""
CREATE INDEX idx_restaurants_geo
ON restaurants USING RTREE(coordinates)
""")

conn.execute("""
CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY,
    restaurant_id INTEGER REFERENCES restaurants(id),
    customer_name TEXT,
    delivery_address POINT NOT NULL,
    status TEXT DEFAULT 'pending',
    driver_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

conn.execute("""
CREATE TABLE IF NOT EXISTS drivers (
    id INTEGER PRIMARY KEY,
    name TEXT,
    current_location POINT,
    status TEXT DEFAULT 'available',
    last_update TIMESTAMP
)
""")

conn.execute("""
CREATE INDEX idx_drivers_geo
ON drivers USING RTREE(current_location)
""")
```

Restaurant-Suche

```
def find_nearby_restaurants(user_lat, user_lon, cuisine=None, max_distance_m=5000):
    """Findet Restaurants in der Nähe"""

    query = """
        SELECT id, name, cuisine, coordinates,
               rating,
               ST_Distance(coordinates, POINT(?, ?)) as distance_m
        FROM restaurants
        WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
    """

    params = [user_lon, user_lat, user_lon, user_lat, max_distance_m]

    if cuisine:
        query += " AND cuisine = ?"
        params.append(cuisine)

    query += " ORDER BY distance_m"

    return conn.query(query, params)

restaurants = find_nearby_restaurants(
    user_lat=52.520,
    user_lon=13.405,
    cuisine="Italian",
    max_distance_m=3000
)

for r in restaurants:
    distance_km = r['distance_m'] / 1000
    print(f"{r['name']}: {distance_km:.1f}km, Rating: {r['rating']}/5")
```

Fahrer-Zuordnung

```
def assign_nearest_driver(order_id):
    """Findet den nächsten verfügbaren Fahrer"""

    # Hole Bestelladresse
    order = conn.query("""
        FOR order IN orders
          FILTER order.id == @order_id
          LIMIT 1
          RETURN order.delivery_address
    """, {"order_id": order_id})[0]

    delivery_loc = order

    # Finde nächsten verfügbaren Fahrer
    driver = conn.query("""
        FOR driver IN drivers
          FILTER driver.status == 'available'
          LET distance_m = ST_Distance(driver.current_location, POINT(@lon, @lat))
          SORT distance_m ASC
          LIMIT 1
          RETURN {
            id: driver.id,
            name: driver.name,
            current_location: driver.current_location,
            distance_m
          }
    """, {"lon": delivery_loc[0], "lat": delivery_loc[1]})

    if not driver:
        raise ValueError("Kein Fahrer verfügbar")

    driver = driver[0]

    # Zuordnen
    conn.execute("""
        UPDATE orders SET driver_id = ?, status = 'assigned'
        WHERE id = ?
    """, [driver['id'], order_id])

    conn.execute("""
        UPDATE drivers SET status = 'busy' WHERE id = ?
    """, [driver['id']])

    return driver

driver = assign_nearest_driver(order_id=123)
print(f"Fahrer {driver['name']} zugewiesen")
```

Live-Tracking

```
import time
from datetime import datetime

def update_driver_location(driver_id, lat, lon):
    """Aktualisiert Fahrerposition (z.B. alle 5 Sekunden)"""

    conn.execute("""
        UPDATE drivers
        SET current_location = POINT(?, ?),
            last_update = ?
        WHERE id = ?
    """, [lon, lat, datetime.now(), driver_id])

def get_delivery_eta(order_id):
    """Schätzt Ankunftszeit"""

    result = conn.query("""
        SELECT o.id, o.delivery_address,
               d.current_location,
               ST_Distance(d.current_location, o.delivery_address) as distance_m
        FROM orders o
        JOIN drivers d ON o.driver_id = d.id
        WHERE o.id = ?
    """, [order_id])

    if not result:
        return None

    data = result[0]
    distance_km = data['distance_m'] / 1000

    # Annahme: 30 km/h in der Stadt
    eta_minutes = (distance_km / 30) * 60

    return {
        'distance_km': round(distance_km, 1),
        'eta_minutes': round(eta_minutes),
        'driver_location': data['current_location']
    }

eta = get_delivery_eta(order_id=123)
print(f"Fahrer ist noch {eta['distance_km']}km entfernt")
print(f"ETA: {eta['eta_minutes']} Minuten")
```

14.6 Beispiel: Immobiliensuche

Geo-basierte Immobiliensuche mit Filterung:

Datenmodell

```
conn.execute("""
CREATE TABLE IF NOT EXISTS properties (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    address TEXT,
    coordinates POINT NOT NULL,
    price_eur INTEGER,
    size_sqm REAL,
    rooms INTEGER,
    type TEXT,  -- 'apartment', 'house', 'commercial'
    features JSON  -- ['balcony', 'parking', 'elevator', ...]
)
""")

conn.execute("""
CREATE INDEX idx_properties_geo
ON properties USING RTREE(coordinates)
""")
```

Radius-Suche mit Filtern

```
def search_properties(
    center_lat, center_lon, radius_m=2000,
    min_price=None, max_price=None,
    min_rooms=None, property_type=None,
    required_features=None
):
    """Immobiliensuche mit Geo + Filter"""

    query = """
        SELECT id, title, address, coordinates,
               price_eur, size_sqm, rooms, type, features,
               ST_Distance(coordinates, POINT(?, ?)) as distance_m
        FROM properties
        WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
    """

    params = [center_lon, center_lat, center_lon, center_lat, radius_m]

    if min_price:
        query += " AND price_eur >= ?"
        params.append(min_price)

    if max_price:
        query += " AND price_eur <= ?"
        params.append(max_price)

    if min_rooms:
        query += " AND rooms >= ?"
        params.append(min_rooms)

    if property_type:
        query += " AND type = ?"
        params.append(property_type)

    query += " ORDER BY distance_m"

    results = conn.query(query, params)

    # Nachfilterung für Features (JSON)
    if required_features:
        filtered = []
        for prop in results:
            prop_features = set(prop['features'] or [])
            if all(f in prop_features for f in required_features):
                filtered.append(prop)
        results = filtered

    return results

properties = search_properties(
    center_lat=52.520,
    center_lon=13.405,
    radius_m=3000,
    min_price=500_000,
```

```
    max_price=800_000,
    min_rooms=3,
    property_type='apartment',
    required_features=['balcony', 'parking']
)

for p in properties:
    print(f"{p['title']}: {p['price_eur']:,}€, {p['rooms']} Zimmer")
    print(f"  {p['distance_m']/1000:.1f}km entfernt")
```

POI-basierte Suche

Immobilien in der Nähe von Points of Interest:

```
def search_near_poi(poi_name, radius_m=1000):
    """Immobilien in der Nähe eines POI"""

    # Finde POI
    poi = conn.query("""
        FOR poi IN points_of_interest
          FILTER poi.name == @poi_name
          LIMIT 1
          RETURN poi.coordinates
    """, {"poi_name": poi_name})

    if not poi:
        raise ValueError(f"POI '{poi_name}' nicht gefunden")

    poi_coords = poi[0]['coordinates']

    # Suche Immobilien
    return conn.query("""
        SELECT id, title, price_eur,
               ST_Distance(coordinates, POINT(?, ?)) as distance_m
        FROM properties
        WHERE ST_Distance(coordinates, POINT(?, ?)) < ?
        ORDER BY distance_m
    """, [poi_coords[0], poi_coords[1],
          poi_coords[0], poi_coords[1], radius_m])

properties = search_near_poi("Alexanderplatz", radius_m=1500)
```

14.7 Best Practices

1. Index-Design

```
CREATE INDEX idx_geo ON locations USING RTREE(coordinates);

CREATE INDEX idx_geo_type ON locations(type, coordinates) USING RTREE;

CREATE INDEX idx_coords ON locations(coordinates);  # B-Tree, ineffizient!
```

2. Query-Optimierung

```
WHERE ST_Distance(coordinates, ?) < radius

FOR location IN locations
  LET dist = ST_Distance(location.coordinates, @point)
  RETURN {location, dist}
-- Keine Index-Nutzung! Alle Rows werden berechnet
```

3. Koordinaten-Validierung

```
def validate_coordinates(lon, lat):
    """Prüft ob Koordinaten gültig sind"""
    if not (-180 <= lon <= 180):
        raise ValueError(f"Longitude {lon} außerhalb [-180, 180]")
    if not (-90 <= lat <= 90):
        raise ValueError(f"Latitude {lat} außerhalb [-90, 90]")
    return True

validate_coordinates(lon, lat)
```

4. Präzision und Rundung

```
coordinates = [round(lon, 6), round(lat, 6)]
```

5. Caching für häufige Queries

```
from functools import lru_cache
import time

@lru_cache(maxsize=1000)
def get_cached_nearby_restaurants(lat, lon, radius):
    """Cached Restaurant-Suche"""
    # Cache für 5 Minuten
    cache_key = (round(lat, 3), round(lon, 3), radius)
    return find_nearby_restaurants(lat, lon, max_distance_m=radius)
```

14.8 Performance-Tipps

1. Bounding Box vor Distanz-Berechnung

```
FOR location IN locations
  FILTER ST_Within(location.coordinates, BBOX(@min_lon, @min_lat, @max_lon, @max_lat))  -- Sc
    AND ST_Distance(location.coordinates, POINT(@lon, @lat)) < @radius  -- Nur für Kandidaten
  RETURN location
```

2. Limit verwenden

```
FOR location IN locations
  LET dist = ST_Distance(location.coordinates, POINT(@lon, @lat))
  FILTER dist < 5000
  SORT dist ASC
  LIMIT 10  -- Stoppt nach 10 Ergebnissen
  RETURN location
```

3. Materializedviews für häufige Gebiete

```
conn.execute("""
CREATE MATERIALIZED VIEW berlin_mitte_restaurants AS
FOR restaurant IN restaurants
  FILTER ST_Within(restaurant.coordinates, BBOX(13.35, 52.50, 13.45, 52.55))
  RETURN restaurant
""")

conn.execute("REFRESH MATERIALIZED VIEW berlin_mitte_restaurants")
```

14.9 Vergleich mit spezialisierten Geo-Systemen

Feature	ThemisDB	PostGIS	MongoDB Geo
R-Tree Index	Native	GiST	2dsphere
Distanz-Queries			
Polygon-Support		(komplex)	
3D-Geometrie	×		×
Koordinatensysteme	WGS84	Viele	WGS84
Performance			
Multi-Model		×	
Einfachheit			

Empfehlung: - **ThemisDB:** Gut für die meisten Anwendungen, besonders mit Multi-Model Daten - **PostGIS:** Beste Wahl für komplexe GIS-Anwendungen - **MongoDB:** Gut für Dokument + Geo Kombination

14.10 Übungen

Übung 1: Store Locator

Implementieren Sie einen Store Locator mit: - Nächste 5 Filialen - Öffnungszeiten-Filterung - Routenvorschlag

Übung 2: Geofencing

Erstellen Sie ein Geofencing-System: - Definieren Sie virtuelle Zonen (Polygone) - Trigger beim Ein/Austritt - Benachrichtigungen

Übung 3: Heat Map

Generieren Sie eine Heat Map: - Aggregiere Punkte nach Geo-Hash - Cluster für verschiedene Zoom-Level - Export für Mapping-Tools

Zusammenfassung

In diesem Kapitel haben Sie gelernt:

Geo-Datentypen: Point, LineString, Polygon **Räumliche Indizes:** R-Tree für effiziente Geo-Queries **Abfragen:** Radius, Bounding Box, KNN, Polygon **Praxis-Beispiele:** Lieferdienst, Immobiliensuche **Best Practices:** Index-Design, Performance, Validation

Geo-Spatial Features in ThemisDB bieten eine solide Grundlage für Location-Based Services ohne externe Geo-Datenbank. Die native Integration mit anderen Datenmodellen (Relational, Graph, Dokument, Vector) macht ThemisDB zur idealen Plattform für moderne Geo-Anwendungen.

Im nächsten Kapitel sehen wir uns **Analytics & Reporting** an – Aggregationen, Dashboards und Business Intelligence mit ThemisDB.

Kapitel 16: Kapitel 15: Analytics & Reporting

15.1 Einführung in Analytics mit ThemisDB

ThemisDB bietet leistungsstarke Analyse- und Reporting-Funktionen, die es ermöglichen, komplexe Geschäftslogik direkt in der Datenbank auszuführen. Durch die Kombination von relationalen Aggregationen, Graph-Analysen und Vektor-basierten Ähnlichkeitssuchen können umfassende Business-Intelligence-Lösungen erstellt werden.

15.1.1 Warum Analytics in der Datenbank?

Vorteile: - **Performance:** Datenverarbeitung direkt am Speicherort - **Konsistenz:** ACID-Garantien für Analyseergebnisse - **Echtzeit:** Keine ETL-Verzögerungen - **Multi-Model:** Kombinierte Analysen über verschiedene Datenmodelle

15.9 OLAP Cube Design (Neu)

15.9.1 Star Schema

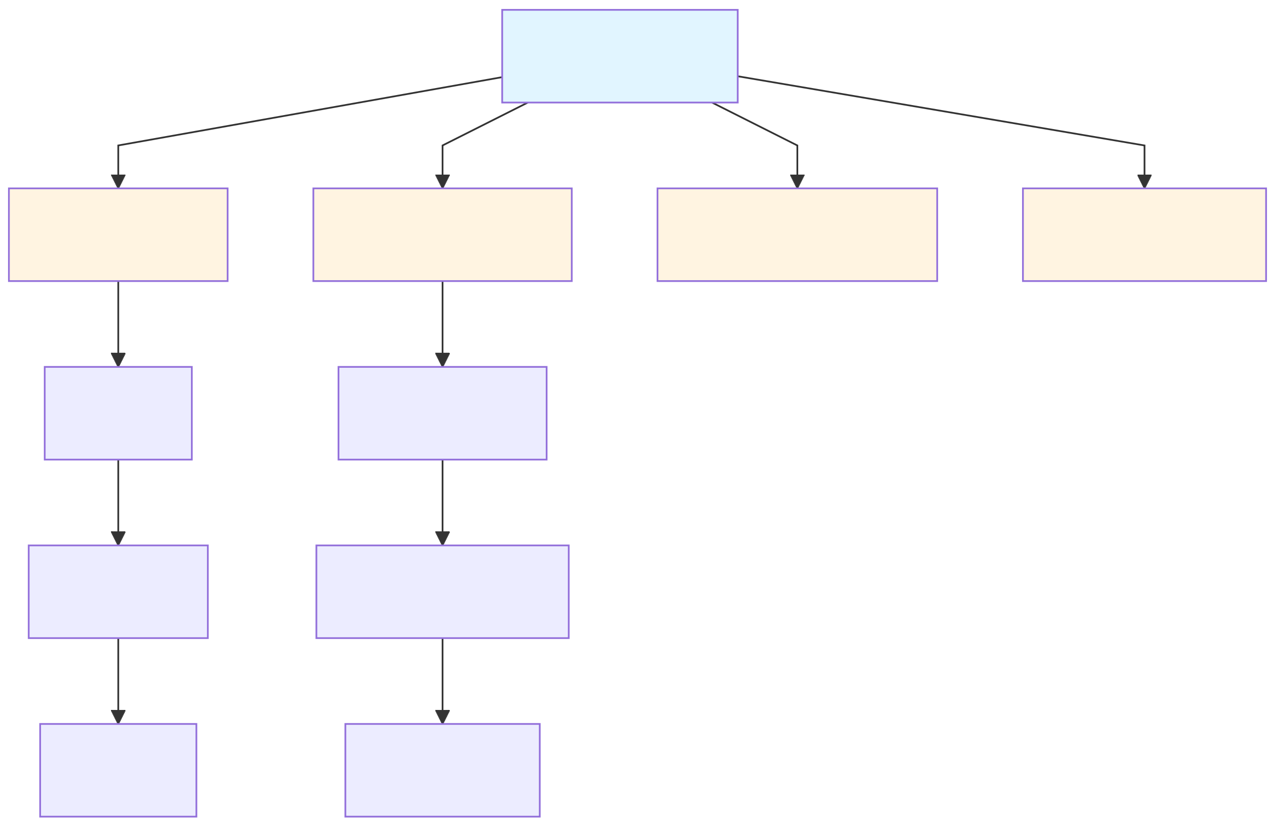


Abb. 39: Ablaufdiagramm: 15.9.1 Star Schema

Keys: - Fact: `time_key`, `product_key`, `customer_key`, `region_key` - Dim: Surrogate Keys (`int`) für schnelle Joins

15.9.2 Slowly Changing Dimensions (SCD)

Typ	Verhalten	Beispiel
SCD1	Überschreibt alte Werte	Preis aktualisieren
SCD2	Historisiert mit `valid_from/valid_to`	Kundenadresse
SCD3	Speichert Vorversion im gleichen Datensatz	Vorheriger Status

AQL (SCD2 Update):

```
LET now = DATE_NOW()

FOR dim IN dim_product
    FILTER dim.product_id == @product_id
    FILTER dim.valid_to == null
    UPDATE dim WITH { valid_to: now } IN dim_product

INSERT {
    product_id: @product_id,
    name: @name,
    category: @category,
    valid_from: now,
    valid_to: null
} INTO dim_product
```

15.9.3 Derived & Aggregate Tables

- **Rollups:** day → month → quarter → year
- **Pre-Aggregation:** Umsatz pro Region/Produkt/Quartal
- **Materialized Views:** Nightly Refresh oder Streaming (CDC)

```
-- Quartals-Rollup
FOR fact IN sales
    COLLECT
        year = DATE_YEAR(fact.ts),
        quarter = DATE_QUARTER(fact.ts),
        region = fact.region,
        category = fact.category
    AGGREGATE
        revenue = SUM(fact.amount),
        units = SUM(fact.quantity)
    RETURN {
        year, quarter, region, category, revenue, units
    }
```

15.10 Incremental Materialization & CDC

15.10.1 Changefeed → Materialized View

```
"""Incremental Refresh für sales_rollup (hourly)"""
from datetime import datetime, timedelta

def refresh_sales_rollup(db, since_ts):
        events = db.changefeed("sales", {"since": since_ts})
        buffer = []
        for ev in events:
                buffer.append(ev)
                if len(buffer) >= 1000:
                        apply_buffer(db, buffer)
                        buffer.clear()
        apply_buffer(db, buffer)

def apply_buffer(db, buffer):
        db.query("""
                FOR ev IN @events
                    LET month = DATE_TRUNC('month', ev.ts)
                    UPSERT { month, region: ev.region, category: ev.category }
                        INSERT {
                            month,
                            region: ev.region,
                            category: ev.category,
                            revenue: ev.amount,
                            units: ev.quantity
                        }
                        UPDATE {
                            revenue: OLD.revenue + ev.amount,
                            units: OLD.units + ev.quantity
                        }
                        IN sales_rollup
        """, {"events": buffer})
```

15.10.2 Late Arriving Facts & Corrections

```
-- Korrekturen erkennen
FOR ev IN changefeed_sales
    FILTER ev.event_type == "correction"
    LET prev = DOCUMENT(sales_rollup, ev.rollup_key)
    UPDATE prev WITH {
        revenue: prev.revenue - ev.old_amount + ev.new_amount,
        units: prev.units - ev.old_qty + ev.new_qty
    } IN sales_rollup
```

15.10.3 OLAP Query Patterns

```
-- Drill-down: Jahr → Monat → Tag
FOR row IN sales_rollup
    FILTER row.year == 2025
    COLLECT month = row.month
    AGGREGATE revenue = SUM(row.revenue)
    SORT month
    RETURN { month, revenue }

-- Pivot nach Region
FOR row IN sales_rollup
    COLLECT category = row.category
    AGGREGATE
        eu = SUM(row.region == 'EU' ? row.revenue : 0),
        us = SUM(row.region == 'US' ? row.revenue : 0),
        apac = SUM(row.region == 'APAC' ? row.revenue : 0)
    RETURN { category, eu, us, apac }
```

```
class AnalyticsGovernance:
        """Data Governance für Analytics"""

        def __init__(self, db):
                self.db = db

        def anonymize_customer_data(self, dataset):
                """Anonymisierung sensibler Daten"""
                return [{
                        **row,
                        'customer_id': hash(row['customer_id']),
                        'email': None,
                        'name': None
                } for row in dataset]

        def apply_row_level_security(self, query, user_role, user_region):
                """Row-Level Security anwenden"""
                if user_role == 'regional_manager':
                        query += f" FILTER doc.region == '{user_region}'"
                elif user_role == 'analyst':
                        # Nur aggregierte Daten
                        query += " COLLECT ... AGGREGATE ..."

                return query

        def audit_query(self, user_id, query, timestamp):
                """Query-Audit-Log"""
                self.db.query("""
                        INSERT {
                                user_id: @user_id,
                                query: @query,
                                timestamp: @timestamp,
                                type: 'analytics'
                        } INTO audit_log
                """, {
                        'user_id': user_id,
                        'query': query,
                        'timestamp': timestamp
                })
```

15.11 Zusammenfassung

15.2.1 Standard AQL-Aggregationen

ThemisDB unterstützt alle Standard-AQL-Aggregationsfunktionen:

```
-- Verkaufsstatistiken
FOR order IN orders
  FILTER order.order_date >= '2024-01-01'
  COLLECT month = DATE_TRUNC('month', order.order_date)
  AGGREGATE
    total_orders = COUNT(),
    revenue = SUM(order.total_amount),
    avg_order_value = AVG(order.total_amount),
    min_order = MIN(order.total_amount),
    max_order = MAX(order.total_amount),
    order_stddev = STDDEV(order.total_amount)
  SORT month
  RETURN {month, total_orders, revenue, avg_order_value, min_order, max_order, order_stddev}
```

Ausgabe:

```
month        | total_orders | revenue    | avg_order_value | min_order | max_order | order_stdd
------------|--------------|-----------|-----------------|-----------|-----------|-----------
2024-01-01  | 1250         | 156780.50 | 125.42          | 12.50     | 1580.00   | 95.23
2024-02-01  | 1420         | 182340.75 | 128.45          | 9.99      | 2100.00   | 102.45
```

15.2.2 Window Functions für Trend-Analysen

```
-- Umsatztrend mit gleitendem Durchschnitt
FOR order IN orders
  SORT order.order_date DESC
  LIMIT 30
  LET moving_avg_7day = AVG_WINDOW(order.total_amount,
    {preceding: 6, following: 0})
  LET month_cumulative = SUM_WINDOW(order.total_amount,
    {partition: DATE_TRUNC('month', order.order_date)})
  LET rank_in_month = RANK_WINDOW(
    {partition: DATE_TRUNC('month', order.order_date),
     order: order.total_amount DESC})
  RETURN {
    order_date: order.order_date,
    total_amount: order.total_amount,
    moving_avg_7day,
    month_cumulative,
    rank_in_month
  }
```

15.2.3 PIVOT-Operationen

```
-- Umsatz nach Produktkategorie und Monat
LET pivot_data = (
  FOR order_item IN order_items
    FOR product IN products
      FILTER order_item.product_id == product.id
      COLLECT
        month = DATE_TRUNC('month', order_item.order_date),
        category = product.category
      AGGREGATE revenue = SUM(order_item.amount)
      RETURN {month, category, revenue}
)

// PIVOT operation (manual transformation)
FOR data IN pivot_data
  COLLECT month = data.month
  AGGREGATE
    electronics = SUM(data.category == 'Electronics' ? data.revenue : 0),
    clothing = SUM(data.category == 'Clothing' ? data.revenue : 0),
    books = SUM(data.category == 'Books' ? data.revenue : 0),
    home = SUM(data.category == 'Home' ? data.revenue : 0),
    sports = SUM(data.category == 'Sports' ? data.revenue : 0)
  RETURN {month, electronics, clothing, books, home, sports}
```

15.3 Erweiterte Aggregationen mit AQL

15.3.1 COLLECT für komplexe Gruppierungen

```
-- Kundensegmentierung nach Kaufverhalten
FOR order IN orders
    COLLECT
        customer_id = order.customer_id,
        age_group = FLOOR(order.customer_age / 10) * 10
    AGGREGATE
        order_count = COUNT(1),
        total_spent = SUM(order.total_amount),
        avg_order = AVG(order.total_amount),
        categories = UNIQUE(order.category)
    INTO group
    FILTER order_count >= 5
    LET segment = (
        total_spent > 5000 ? "VIP" :
        total_spent > 1000 ? "Premium" :
        "Regular"
    )
    RETURN {
        customer_id,
        age_group,
        segment,
        metrics: {
            orders: order_count,
            total_revenue: total_spent,
            avg_order_value: avg_order,
            product_diversity: LENGTH(categories)
        }
    }
```

15.3.2 Multi-Level Aggregationen

```
-- Hierarchische Umsatzanalyse
FOR order IN orders
    FILTER order.date >= DATE_NOW() - 365*24*60*60*1000
    COLLECT
        year = DATE_YEAR(order.date),
        quarter = DATE_QUARTER(order.date),
        region = order.shipping_region
    AGGREGATE
        revenue = SUM(order.total_amount),
        order_count = COUNT(1)
    COLLECT
        year = year,
        quarter = quarter
    AGGREGATE
        total_revenue = SUM(revenue),
        total_orders = SUM(order_count),
        regions = COUNT(region)
    RETURN {
        period: CONCAT(year, "-Q", quarter),
        total_revenue,
        total_orders,
        avg_per_region: total_revenue / regions,
        regions_active: regions
    }
```

15.4 Graph Analytics für Beziehungsanalysen

15.4.1 Customer Network Analysis

```
-- Kunden mit ähnlichen Kaufmustern finden
FOR customer IN customers
    FILTER customer.id == @customerId

    // Produkte des Kunden
    LET customer_products = (
        FOR order IN orders
            FILTER order.customer_id == customer.id
            FOR item IN order.items
            RETURN DISTINCT item.product_id
    )

    // Ähnliche Kunden finden
    LET similar_customers = (
        FOR other IN customers
            FILTER other.id != customer.id
            LET other_products = (
                FOR order IN orders
                    FILTER order.customer_id == other.id
                    FOR item IN order.items
                    RETURN DISTINCT item.product_id
            )
            LET intersection = LENGTH(
                INTERSECTION(customer_products, other_products)
            )
            LET union_size = LENGTH(
                UNION(customer_products, other_products)
            )
            LET jaccard = intersection / union_size
            FILTER jaccard > 0.3
            SORT jaccard DESC
            LIMIT 10
            RETURN {
                customer_id: other.id,
                similarity: jaccard,
                shared_products: intersection
            }
    )

    RETURN {
        customer_id: customer.id,
        similar_customers
    }
```

15.4.2 Influencer-Identifikation im Social Graph

```
-- Top-Influencer nach PageRank
FOR user IN users
    LET followers_count = LENGTH(
        FOR edge IN follows
            FILTER edge._to == user._id
            RETURN 1
    )
    LET engagement = (
        FOR post IN posts
            FILTER post.author_id == user.id
            RETURN SUM([
                post.likes_count,
                post.comments_count * 2,
                post.shares_count * 3
            ])
    )
    LET avg_engagement = AVG(engagement)

    // PageRank-ähnliche Berechnung
    LET influence_score = (
        followers_count * 0.4 +
        avg_engagement * 0.6
    )

    FILTER influence_score > 100
    SORT influence_score DESC
    LIMIT 50

    RETURN {
        user_id: user.id,
        username: user.username,
        followers: followers_count,
        avg_engagement,
        influence_score
    }
```

15.5 Vektor-basierte Analysen

15.5.1 Produkt-Clustering nach Embeddings

```python
import themisdb
import numpy as np
from sklearn.cluster import KMeans

db = themisdb.connect("localhost:8529")

query = """
FOR product IN products
    FILTER product.embedding != null
    RETURN {
        id: product.id,
        name: product.name,
        embedding: product.embedding
    }
"""
products = db.query(query)

embeddings = np.array([p['embedding'] for p in products])
product_ids = [p['id'] for p in products]

kmeans = KMeans(n_clusters=10, random_state=42)
clusters = kmeans.fit_predict(embeddings)

for product_id, cluster_id in zip(product_ids, clusters):
    db.query("""
        UPDATE products
        SET cluster_id = @cluster
        WHERE id = @id
    """, {
        'id': product_id,
        'cluster': int(cluster_id)
    })

cluster_stats = db.query("""
    FOR product IN products
        COLLECT cluster = product.cluster_id
        AGGREGATE
            count = COUNT(1),
            avg_price = AVG(product.price),
            categories = UNIQUE(product.category)
        RETURN {
            cluster_id: cluster,
            product_count: count,
            avg_price,
            main_categories: categories
        }
""")

for stats in cluster_stats:
    print(f"Cluster {stats['cluster_id']}: {stats['product_count']} Produkte")
    print(f"  Durchschnittspreis: €{stats['avg_price']:.2f}")
    print(f"  Kategorien: {', '.join(stats['main_categories'][:3])}")
```

15.5.2 Anomalie-Erkennung mit Vektor-Distanzen

```
-- Ungewöhnliche Transaktionen identifizieren
FOR transaction IN transactions
    // Durchschnittliches Transaktionsprofil des Kunden
    LET customer_avg_embedding = (
        FOR t IN transactions
            FILTER t.customer_id == transaction.customer_id
            AND t.id != transaction.id
            RETURN t.transaction_embedding
    )

    LET avg_embedding = (
        FOR emb IN customer_avg_embedding
            RETURN AVERAGE(emb)
    )

    // Distanz zur Normalverteilung
    LET distance = VECTOR_DISTANCE(
        "cosine",
        transaction.transaction_embedding,
        avg_embedding
    )

    // Anomalie-Score
    LET anomaly_score = distance > 0.7 ? 1.0 : distance / 0.7

    FILTER anomaly_score > 0.8
    SORT anomaly_score DESC
    LIMIT 100

    RETURN {
        transaction_id: transaction.id,
        customer_id: transaction.customer_id,
        amount: transaction.amount,
        anomaly_score,
        reason: anomaly_score > 0.95 ? "HIGH_RISK" : "REVIEW"
    }
```

15.6 Materialized Views für Performance

15.6.1 Erstellen von Materialized Views

```
-- Tägliche Verkaufsübersicht
CREATE MATERIALIZED VIEW daily_sales_summary AS
SELECT
    DATE(order_date) AS date,
    COUNT(*) AS order_count,
    SUM(total_amount) AS revenue,
    AVG(total_amount) AS avg_order_value,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers
FROM orders
GROUP BY DATE(order_date);

-- Refresh Strategy
CREATE TRIGGER refresh_daily_sales
    AFTER INSERT OR UPDATE ON orders
    FOR EACH STATEMENT
    EXECUTE PROCEDURE refresh_materialized_view('daily_sales_summary');
```

15.6.2 Inkrementelles Update

```
def update_daily_metrics(date):
    """Inkrementelles Update für einen Tag"""

    # Alte Metriken löschen
    db.query("""
        DELETE FROM daily_sales_summary
        WHERE date = @date
    """, {'date': date})

    # Neue Metriken berechnen
    db.query("""
        INSERT INTO daily_sales_summary
        SELECT
            @date AS date,
            COUNT(*) AS order_count,
            SUM(total_amount) AS revenue,
            AVG(total_amount) AS avg_order_value,
            COUNT(DISTINCT customer_id) AS unique_customers,
            SUM(CASE WHEN is_first_order THEN 1 ELSE 0 END) AS new_customers
        FROM orders
        WHERE DATE(order_date) = @date
    """, {'date': date})
```

15.7 OLAP-Würfel und Mehrdimensionale Analysen

15.7.1 Erstellen eines OLAP-Würfels

```
import pandas as pd

def create_sales_cube(db):
    """OLAP-Würfel für Verkaufsanalysen"""

    query = """
    FOR order IN orders
        LET customer = DOCUMENT('customers', order.customer_id)
        LET items = (
            FOR item IN order.items
                LET product = DOCUMENT('products', item.product_id)
                RETURN {
                    category: product.category,
                    brand: product.brand,
                    price: item.price,
                    quantity: item.quantity
                }
        )
        RETURN {
            date: order.order_date,
            customer_segment: customer.segment,
            customer_region: customer.region,
            items: items
        }
    """

    # Daten laden
    orders = list(db.query(query))

    # In flache Struktur umwandeln
    rows = []
    for order in orders:
        for item in order['items']:
            rows.append({
                'date': pd.to_datetime(order['date']),
                'year': pd.to_datetime(order['date']).year,
                'quarter': pd.to_datetime(order['date']).quarter,
                'month': pd.to_datetime(order['date']).month,
                'customer_segment': order['customer_segment'],
                'customer_region': order['customer_region'],
                'category': item['category'],
                'brand': item['brand'],
                'revenue': item['price'] * item['quantity'],
                'quantity': item['quantity']
            })

    # DataFrame erstellen
    df = pd.DataFrame(rows)

    # Pivot-Tabellen
    cube = {
        'by_region_category': df.pivot_table(
            values='revenue',
            index='customer_region',
```

```
            columns='category',
            aggfunc='sum',
            fill_value=0
        ),
        'by_segment_month': df.pivot_table(
            values='revenue',
            index='customer_segment',
            columns='month',
            aggfunc='sum',
            fill_value=0
        ),
        'by_brand_quarter': df.pivot_table(
            values='revenue',
            index='brand',
            columns='quarter',
            aggfunc='sum',
            fill_value=0
        )
    }

    return cube

cube = create_sales_cube(db)

region = 'Europe'
category = 'Electronics'

drill_down = db.query("""
    FOR order IN orders
        LET customer = DOCUMENT('customers', order.customer_id)
        FILTER customer.region == @region
        FOR item IN order.items
            LET product = DOCUMENT('products', item.product_id)
            FILTER product.category == @category
            COLLECT brand = product.brand
            AGGREGATE revenue = SUM(item.price * item.quantity)
            SORT revenue DESC
            RETURN {brand, revenue}
""", {'region': region, 'category': category})
```

15.7.2 Slice und Dice Operationen

```
def slice_cube(cube_data, dimension, value):
    """Slice-Operation auf dem Würfel"""
    return cube_data[cube_data[dimension] == value]

def dice_cube(cube_data, filters):
    """Dice-Operation mit mehreren Dimensionen"""
    result = cube_data
    for dim, values in filters.items():
        result = result[result[dim].isin(values)]
    return result

filters = {
    'customer_region': ['Europe', 'North America'],
    'category': ['Electronics', 'Books'],
    'year': [2024]
}
diced = dice_cube(df, filters)
```

15.8 Dashboard-Metriken in Echtzeit

15.8.1 KPI-Berechnung

```
class RealtimeDashboard:
    def __init__(self, db):
        self.db = db

    def get_current_metrics(self):
        """Aktuelle KPIs abrufen"""

        metrics = {}

        # Heutige Verkäufe
        today = self.db.query("""
            LET today = DATE_FORMAT(DATE_NOW(), '%Y-%m-%d')
            FOR order IN orders
                FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') == today
                COLLECT AGGREGATE
                    count = COUNT(1),
                    revenue = SUM(order.total_amount),
                    avg = AVG(order.total_amount)
                RETURN {count, revenue, avg}
        """)[0]

        metrics['today'] = today

        # Gestern zum Vergleich
        yesterday = self.db.query("""
            LET yesterday = DATE_FORMAT(
                DATE_SUBTRACT(DATE_NOW(), 1, 'day'),
                '%Y-%m-%d'
            )
            FOR order IN orders
                FILTER DATE_FORMAT(order.order_date, '%Y-%m-%d') == yesterday
                COLLECT AGGREGATE
                    count = COUNT(1),
                    revenue = SUM(order.total_amount)
                RETURN {count, revenue}
        """)[0]

        metrics['yesterday'] = yesterday

        # Prozentuale Veränderung
        metrics['change'] = {
            'orders': (today['count'] - yesterday['count']) / yesterday['count'] * 100,
            'revenue': (today['revenue'] - yesterday['revenue']) / yesterday['revenue'] * 100
        }

        # Aktive Benutzer
        metrics['active_users'] = self.db.query("""
            LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
            FOR session IN user_sessions
                FILTER session.last_activity >= last_hour
                RETURN DISTINCT session.user_id
        """).count()

        # Conversion Rate
```

```
        metrics['conversion_rate'] = self.db.query("""
            LET last_hour = DATE_SUBTRACT(DATE_NOW(), 1, 'hour')
            LET visits = (
                FOR visit IN page_views
                    FILTER visit.timestamp >= last_hour
                    RETURN DISTINCT visit.session_id
            )
            LET purchases = (
                FOR order IN orders
                    FILTER order.order_date >= last_hour
                    RETURN DISTINCT order.session_id
            )
            RETURN LENGTH(purchases) / LENGTH(visits) * 100
        """)[0]

        return metrics

    def get_top_products(self, limit=10):
        """Top-Produkte der letzten 24 Stunden"""
        return self.db.query("""
            LET last_24h = DATE_SUBTRACT(DATE_NOW(), 24, 'hour')
            FOR order IN orders
                FILTER order.order_date >= last_24h
                FOR item IN order.items
                    COLLECT product_id = item.product_id
                    AGGREGATE
                        quantity_sold = SUM(item.quantity),
                        revenue = SUM(item.price * item.quantity)
                    LET product = DOCUMENT('products', product_id)
                    SORT revenue DESC
                    LIMIT @limit
                    RETURN {
                        product_id,
                        name: product.name,
                        quantity_sold,
                        revenue
                    }
        """, {'limit': limit})

    def get_geographic_distribution(self):
        """Geografische Verteilung der Verkäufe"""
        return self.db.query("""
            FOR order IN orders
                FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 7, 'day')
                LET customer = DOCUMENT('customers', order.customer_id)
                COLLECT
                    country = customer.country,
                    region = customer.region
                AGGREGATE
                    orders = COUNT(1),
                    revenue = SUM(order.total_amount)
                SORT revenue DESC
                RETURN {
                    country,
```

```
                    region,
                    orders,
                    revenue
                }
        """)

dashboard = RealtimeDashboard(db)
metrics = dashboard.get_current_metrics()

print(f"Heute: {metrics['today']['count']} Bestellungen, €{metrics['today']['revenue']:.2f}")
print(f"Veränderung: {metrics['change']['orders']:+.1f}% Bestellungen, {metrics['change']['re
print(f"Aktive Benutzer: {metrics['active_users']}")
print(f"Conversion Rate: {metrics['conversion_rate']:.2f}%")
```

15.8.2 Change Data Capture für Live-Updates

```
def subscribe_to_metrics_updates(db, callback):
    """CDC-Stream für Echtzeit-Metriken"""

    stream = db.collection('orders').watch([
        {'$match': {'operationType': 'insert'}}
    ])

    for change in stream:
        # Neue Bestellung
        order = change['fullDocument']

        # Metriken aktualisieren
        updated_metrics = {
            'timestamp': order['order_date'],
            'order_id': order['_id'],
            'amount': order['total_amount'],
            'customer_id': order['customer_id']
        }

        # Callback für Dashboard-Update
        callback(updated_metrics)

def on_new_order(metrics):
    print(f"Neue Bestellung: €{metrics['amount']:.2f}")
    # Dashboard aktualisieren

subscribe_to_metrics_updates(db, on_new_order)
```

15.9 Predictive Analytics

15.9.1 Trend-Prognosen mit linearer Regression

```
from sklearn.linear_model import LinearRegression
import numpy as np

def forecast_revenue(db, days_ahead=30):
    """Umsatzprognose für die nächsten Tage"""

    # Historische Daten
    historical = db.query("""
        FOR order IN orders
            FILTER order.order_date >= DATE_SUBTRACT(DATE_NOW(), 90, 'day')
            COLLECT date = DATE_FORMAT(order.order_date, '%Y-%m-%d')
            AGGREGATE revenue = SUM(order.total_amount)
            SORT date
            RETURN {date, revenue}
    """)

    # Features vorbereiten
    X = np.array([[i] for i in range(len(historical))])
    y = np.array([h['revenue'] for h in historical])

    # Modell trainieren
    model = LinearRegression()
    model.fit(X, y)

    # Prognose
    future_X = np.array([[i] for i in range(len(historical), len(historical) + days_ahead)])
    forecast = model.predict(future_X)

    # Konfidenzintervall (vereinfacht)
    residuals = y - model.predict(X)
    std_error = np.std(residuals)
    confidence_interval = 1.96 * std_error  # 95% CI

    return {
        'forecast': forecast.tolist(),
        'lower_bound': (forecast - confidence_interval).tolist(),
        'upper_bound': (forecast + confidence_interval).tolist()
    }

forecast = forecast_revenue(db, days_ahead=30)
print(f"Prognostizierter Umsatz in 30 Tagen: €{forecast['forecast'][-1]:.2f}")
print(f"Konfidenzintervall: €{forecast['lower_bound'][-1]:.2f} - €{forecast['upper_bound'][-1
```

15.9.2 Churn-Vorhersage

```
def calculate_churn_risk(db, customer_id):
    """Churn-Risiko für einen Kunden berechnen"""

    features = db.query("""
        LET customer = DOCUMENT('customers', @customer_id)
        LET orders = (
            FOR order IN orders
                FILTER order.customer_id == @customer_id
                SORT order.order_date DESC
                RETURN order
        )

        LET days_since_last_order = DATE_DIFF(
            orders[0].order_date,
            DATE_NOW(),
            'day'
        )

        LET order_frequency = LENGTH(orders) / (
            DATE_DIFF(
                customer.registration_date,
                DATE_NOW(),
                'day'
            ) / 30
        )

        LET avg_order_value = AVG(
            FOR order IN orders
                RETURN order.total_amount
        )

        LET total_spent = SUM(
            FOR order IN orders
                RETURN order.total_amount
        )

        RETURN {
            days_since_last_order,
            order_frequency,
            avg_order_value,
            total_spent,
            support_tickets: LENGTH(customer.support_tickets),
            has_complained: LENGTH(
                FOR ticket IN customer.support_tickets
                    FILTER ticket.type == 'complaint'
                    RETURN 1
            ) > 0
        }
    """, {'customer_id': customer_id})[0]

    # Einfacher Risiko-Score
    risk_score = 0

    if features['days_since_last_order'] > 60:
```

```
        risk_score += 0.3
    if features['order_frequency'] < 1:
        risk_score += 0.2
    if features['avg_order_value'] < 50:
        risk_score += 0.1
    if features['support_tickets'] > 5:
        risk_score += 0.2
    if features['has_complained']:
        risk_score += 0.2

    return {
        'customer_id': customer_id,
        'churn_risk': min(risk_score, 1.0),
        'risk_level': (
            'HIGH' if risk_score > 0.7 else
            'MEDIUM' if risk_score > 0.4 else
            'LOW'
        ),
        'features': features
    }
```

15.10 Reporting und Export

15.10.1 PDF-Report-Generierung

```
from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.lib.units import cm
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib.styles import getSampleStyleSheet

def generate_sales_report(db, start_date, end_date, filename):
    """Verkaufsbericht als PDF generieren"""

    # Daten abfragen
    summary = db.query("""
        FOR order IN orders
            FILTER order.order_date >= @start_date
            AND order.order_date <= @end_date
            COLLECT AGGREGATE
                total_orders = COUNT(1),
                total_revenue = SUM(order.total_amount),
                avg_order_value = AVG(order.total_amount)
            RETURN {
                total_orders,
                total_revenue,
                avg_order_value
            }
    """, {'start_date': start_date, 'end_date': end_date})[0]

    top_products = db.query("""
        FOR order IN orders
            FILTER order.order_date >= @start_date
            AND order.order_date <= @end_date
            FOR item IN order.items
                COLLECT product_id = item.product_id
                AGGREGATE
                    quantity = SUM(item.quantity),
                    revenue = SUM(item.price * item.quantity)
                LET product = DOCUMENT('products', product_id)
                SORT revenue DESC
                LIMIT 10
                RETURN [product.name, quantity, revenue]
    """, {'start_date': start_date, 'end_date': end_date})

    # PDF erstellen
    doc = SimpleDocTemplate(filename, pagesize=A4)
    elements = []
    styles = getSampleStyleSheet()

    # Titel
    title = Paragraph(f"Verkaufsbericht {start_date} bis {end_date}", styles['Title'])
    elements.append(title)

    # Zusammenfassung
    summary_data = [
        ['Metric', 'Value'],
        ['Gesamtbestellungen', f"{summary['total_orders']:,}"],
        ['Gesamtumsatz', f"€{summary['total_revenue']:,.2f}"],
```

```
        ['Ø Bestellwert', f"€{summary['avg_order_value']:.2f}"]
    ]

    summary_table = Table(summary_data, colWidths=[8*cm, 8*cm])
    summary_table.setStyle(TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 14),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
        ('GRID', (0, 0), (-1, -1), 1, colors.black)
    ]))
    elements.append(summary_table)

    # Top-Produkte
    elements.append(Paragraph("Top 10 Produkte", styles['Heading2']))

    product_data = [['Produkt', 'Menge', 'Umsatz']]
    product_data.extend(top_products)

    product_table = Table(product_data, colWidths=[10*cm, 3*cm, 3*cm])
    product_table.setStyle(TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (1, 0), (-1, -1), 'RIGHT'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('GRID', (0, 0), (-1, -1), 1, colors.black)
    ]))
    elements.append(product_table)

    # PDF generieren
    doc.build(elements)
    print(f"Report saved to {filename}")

generate_sales_report(db, '2024-01-01', '2024-03-31', 'Q1_2024_Report.pdf')
```

15.10.2 CSV-Export für Excel

```
import csv

def export_to_csv(db, query, filename, params=None):
    """Abfrageergebnisse als CSV exportieren"""

    results = db.query(query, params or {})

    if not results:
        print("No data to export")
        return

    # Header aus erstem Datensatz
    headers = list(results[0].keys())

    with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=headers)
        writer.writeheader()
        writer.writerows(results)

    print(f"Exported {len(results)} rows to {filename}")

export_to_csv(db, """
    FOR customer IN customers
        LET orders = (
            FOR order IN orders
                FILTER order.customer_id == customer.id
                RETURN order
        )
        RETURN {
            customer_id: customer.id,
            name: customer.name,
            email: customer.email,
            total_orders: LENGTH(orders),
            total_spent: SUM(FOR o IN orders RETURN o.total_amount),
            last_order_date: MAX(FOR o IN orders RETURN o.order_date)
        }
""", 'customers_analysis.csv')
```

15.11 Best Practices für Analytics

15.11.1 Performance-Optimierung

Indexierung:

```
-- Composite Index für häufige Queries
CREATE INDEX idx_orders_date_customer ON orders(order_date, customer_id);

-- Covering Index für Aggregationen
CREATE INDEX idx_orders_date_amount ON orders(order_date, total_amount);
```

Query-Optimierung: - Verwenden Sie `FILTER` früh in der Pipeline - Nutzen Sie `COLLECT` statt mehrfacher Gruppierungen - Limitieren Sie Ergebnisse mit `LIMIT` - Verwenden Sie Materialized Views für wiederkehrende Berechnungen

15.11.2 Data Governance

```
class AnalyticsGovernance:
    def anonymize(self, dataset):
        # Entferne persönliche Daten
        return [{**r, 'customer_id': hash(r['customer_id']),
                 'email': None} for r in dataset]

    def apply_rls(self, query, role, region):
        # Row-Level Security nach Rolle
        if role == 'regional': query += f" FILTER region == '{region}'"
        return query
```

15.12 Zusammenfassung

ThemisDB bietet umfassende Analytics-Funktionen:

- **AQL-Aggregationen:** Standard- und Window-Functions
- **AQL-Analytics:** Multi-Level-Aggregationen und COLLECT
- **Graph-Analytics:** Beziehungsanalysen und Influencer-Detection
- **Vektor-Analytics:** Clustering und Anomalie-Erkennung
- **OLAP:** Mehrdimensionale Analysen und Würfel
- **Real-Time:** Live-Dashboards mit CDC
- **Predictive:** Prognosen und Churn-Vorhersage
- **Reporting:** PDF/CSV-Export und Visualisierung

Im nächsten Kapitel behandeln wir Machine Learning Integration für erweiterte Analysen.

Kapitel 17: Machine Learning Integration

Übersicht

Dieses Kapitel behandelt die Integration von Machine Learning-Modellen mit ThemisDB.

Hauptthemen

- ML-Modell-Integration
- Vektorisierung von Daten
- Embedding-Speicherung
- Modell-Inferenz

Weitere Informationen

Siehe auch: Vektoren, Analytics

Kapitel 18: Horizontal Scaling

Übersicht

Dieses Kapitel behandelt die horizontale Skalierung von ThemisDB über mehrere Knoten.

Hauptthemen

- Sharding-Strategien
- Cluster-Verwaltung
- Load Balancing
- Datenverteilung

Weitere Informationen

Siehe auch: Enterprise Features, Hochverfügbarkeit

Kapitel 19: Hochverfügbarkeit

Übersicht

Dieses Kapitel behandelt Hochverfügbarkeitskonfigurationen für ThemisDB.

Hauptthemen

- Replikation
- Failover-Mechanismen
- Health Checks
- Disaster Recovery

Weitere Informationen

Siehe auch: Horizontal Scaling, Monitoring

Kapitel 20: Kapitel 19: Monitoring & Observability

Einführung

Monitoring und Observability sind kritische Aspekte für den produktiven Betrieb von ThemisDB. Dieses Kapitel behandelt die Überwachung von Systemmetriken, Application Performance Monitoring (APM), Logging-Strategien, Distributed Tracing und Alerting-Mechanismen.

18.1 Monitoring-Architektur

18.1.1 Übersicht

Eine umfassende Monitoring-Lösung für ThemisDB umfasst:

- **Metriken-Sammlung**: Prometheus, StatsD, InfluxDB
- **Visualisierung**: Grafana, Kibana
- **Logging**: ELK Stack (Elasticsearch, Logstash, Kibana), Loki
- **Distributed Tracing**: Jaeger, Zipkin
- **Alerting**: Prometheus Alertmanager, PagerDuty

18.1.2 Metriken-Typen

System-Metriken: - CPU-Auslastung - Speichernutzung (RAM, Swap) - Disk I/O (IOPS, Latenz, Durchsatz) - Netzwerk-Traffic (Bandbreite, Pakete)

ThemisDB-Metriken: - Query-Performance (Latenz, Durchsatz) - Connection Pool (aktive Connections, Wartezeit) - Transaction-Statistiken (Commits, Rollbacks) - Cache-Effizienz (Hit Rate, Miss Rate) - Index-Performance - Replikations-Lag

18.2 Prometheus Integration

18.2.1 Prometheus Setup

Installation:

```
wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45.0.lin
tar xvfz prometheus-2.45.0.linux-amd64.tar.gz
cd prometheus-2.45.0.linux-amd64

cat > prometheus.yml <<EOF
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'themisdb'
    static_configs:
      - targets: ['localhost:8529']
    metrics_path: '/metrics'
EOF

./prometheus --config.file=prometheus.yml
```

18.2.2 ThemisDB Exporter

Metriken-Endpoint aktivieren:

```python
from prometheus_client import start_http_server, Counter, Gauge, Histogram
from themisdb import ThemisDB
import time

query_counter = Counter('themisdb_queries_total', 'Total number of queries', ['type'])
query_duration = Histogram('themisdb_query_duration_seconds', 'Query duration', ['type'])
active_connections = Gauge('themisdb_active_connections', 'Number of active connections')
cache_hit_rate = Gauge('themisdb_cache_hit_rate', 'Cache hit rate')

db = ThemisDB(host='localhost', port=8529)

def collect_metrics():
    """Sammelt ThemisDB Metriken."""
    while True:
        # Connection-Metriken
        stats = db.statistics()
        active_connections.set(stats['connections']['active'])

        # Cache-Metriken
        cache_stats = db.cache_statistics()
        hit_rate = cache_stats['hits'] / (cache_stats['hits'] + cache_stats['misses'])
        cache_hit_rate.set(hit_rate)

        # Query-Metriken
        query_stats = db.query_statistics()
        for query_type, count in query_stats.items():
            query_counter.labels(type=query_type).inc(count)

        time.sleep(15)

if __name__ == '__main__':
    # Metrics-Server starten
    start_http_server(9090)
    collect_metrics()
```

18.2.3 Wichtige Metriken

Query Performance:

```
themisdb_query_duration_seconds_bucket{type="SELECT",le="0.1"} 1200
themisdb_query_duration_seconds_bucket{type="SELECT",le="0.5"} 1450
themisdb_query_duration_seconds_bucket{type="SELECT",le="1.0"} 1480
themisdb_query_duration_seconds_bucket{type="SELECT",le="5.0"} 1500

themisdb_queries_total{type="SELECT"} 1500
themisdb_queries_total{type="INSERT"} 500
themisdb_queries_total{type="UPDATE"} 200
```

Connection Pool:

```
themisdb_active_connections 45

themisdb_connection_pool_size 100
themisdb_connection_pool_available 55
```

18.3 Grafana Dashboards

18.3.1 Dashboard Setup

Grafana Installation:

```
sudo apt-get install -y software-properties-common
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable main"
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
sudo apt-get update
sudo apt-get install grafana

sudo systemctl start grafana-server
sudo systemctl enable grafana-server
```

18.3.2 ThemisDB Dashboard

Dashboard-Definition (JSON):

```json
{
  "dashboard": {
    "title": "ThemisDB Performance",
    "panels": [
      {
        "title": "Query Latency (p95)",
        "targets": [
          {
            "expr": "histogram_quantile(0.95, themisdb_query_duration_seconds_bucket)"
          }
        ],
        "type": "graph"
      },
      {
        "title": "Queries per Second",
        "targets": [
          {
            "expr": "rate(themisdb_queries_total[1m])"
          }
        ],
        "type": "graph"
      },
      {
        "title": "Active Connections",
        "targets": [
          {
            "expr": "themisdb_active_connections"
          }
        ],
        "type": "graph"
      },
      {
        "title": "Cache Hit Rate",
        "targets": [
          {
            "expr": "themisdb_cache_hit_rate"
          }
        ],
        "type": "gauge"
      }
    ]
  }
}
```

18.3.3 Dashboard-Beispiele

System-Übersicht Dashboard: - CPU-Auslastung (pro Core) - Speichernutzung (RAM, Swap, Cache) - Disk I/O (Read/Write IOPS, Latenz) - Netzwerk-Traffic (In/Out Bandwidth)

Query-Performance Dashboard: - Query Latenz (p50, p95, p99) - Queries per Second (nach Typ) - Slow Queries (>1s) - Query Error Rate

Cluster-Status Dashboard: - Node Health Status - Replikations-Lag - Cluster-Topology - Failover-Events

18.4 Logging

18.4.1 Logging-Strategie

Log-Levels:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('/var/log/themisdb/themisdb.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger('themisdb')

logger.debug("Connection pool initialized")       # DEBUG
logger.info("Query executed successfully")        # INFO
logger.warning("High memory usage detected")      # WARNING
logger.error("Query execution failed")            # ERROR
logger.critical("Database connection lost")       # CRITICAL
```

18.4.2 Strukturiertes Logging

JSON-Logging:

```python
import json
import logging

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_data = {
            'timestamp': self.formatTime(record),
            'level': record.levelname,
            'logger': record.name,
            'message': record.getMessage(),
            'module': record.module,
            'function': record.funcName,
            'line': record.lineno
        }

        # Zusätzliche Felder
        if hasattr(record, 'query_id'):
            log_data['query_id'] = record.query_id
        if hasattr(record, 'user_id'):
            log_data['user_id'] = record.user_id
        if hasattr(record, 'duration_ms'):
            log_data['duration_ms'] = record.duration_ms

        return json.dumps(log_data)

handler = logging.FileHandler('/var/log/themisdb/themisdb.json')
handler.setFormatter(JSONFormatter())
logger.addHandler(handler)

logger.info(
    "Query executed",
    extra={
        'query_id': 'q123',
        'user_id': 'u456',
        'duration_ms': 45.2
    }
)
```

18.4.3 ELK Stack Integration

Filebeat Konfiguration:

```
filebeat.inputs:
  - type: log
    enabled: true
    paths:
      - /var/log/themisdb/*.log
    json.keys_under_root: true
    json.add_error_key: true

output.elasticsearch:
  hosts: ["localhost:9200"]
  index: "themisdb-logs-%{+yyyy.MM.dd}"

setup.kibana:
  host: "localhost:5601"
```

Logstash Pipeline:

```
input {
  beats {
    port => 5044
  }
}

filter {
  if [type] == "themisdb" {
    json {
      source => "message"
    }

    # Query-Latenz berechnen
    if [duration_ms] {
      ruby {
        code => "event.set('duration_s', event.get('duration_ms') / 1000.0)"
      }
    }

    # Geo-IP Enrichment
    geoip {
      source => "client_ip"
      target => "geoip"
    }
  }
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "themisdb-logs-%{+YYYY.MM.dd}"
  }
}
```

18.5 Distributed Tracing

18.5.1 Jaeger Integration

Jaeger Setup:

```
docker run -d \
  -e COLLECTOR_ZIPKIN_HTTP_PORT=9411 \
  -p 5775:5775/udp \
  -p 6831:6831/udp \
  -p 6832:6832/udp \
  -p 5778:5778 \
  -p 16686:16686 \
  -p 14268:14268 \
  -p 9411:9411 \
  jaegertracing/all-in-one:latest
```

18.5.2 OpenTelemetry Integration

Tracing-Code:

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.jaeger.thrift import JaegerExporter
from opentelemetry.instrumentation.requests import RequestsInstrumentor

trace.set_tracer_provider(TracerProvider())
tracer = trace.get_tracer(__name__)

jaeger_exporter = JaegerExporter(
    agent_host_name='localhost',
    agent_port=6831,
)

trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(jaeger_exporter)
)

RequestsInstrumentor().instrument()

@tracer.start_as_current_span("execute_query")
def execute_query(query):
    span = trace.get_current_span()
    span.set_attribute("query.type", "SELECT")
    span.set_attribute("query.text", query)

    # Query ausführen
    with tracer.start_as_current_span("database_query"):
        result = db.execute(query)

    span.set_attribute("result.count", len(result))
    return result

@tracer.start_as_current_span("process_order")
def process_order(order_id):
    # Span 1: Order validieren
    with tracer.start_as_current_span("validate_order"):
        order = db.query(f"SELECT * FROM orders WHERE id = {order_id}")

    # Span 2: Inventory prüfen
    with tracer.start_as_current_span("check_inventory"):
        inventory = db.query(f"SELECT * FROM inventory WHERE product_id = {order['product_id'

    # Span 3: Payment verarbeiten
    with tracer.start_as_current_span("process_payment"):
        payment = process_payment_service(order)

    return {"status": "success"}
```

18.6 Alerting

18.6.1 Prometheus Alertmanager

Alert-Regeln:

```yaml
groups:
  - name: themisdb
    interval: 30s
    rules:
      # High Query Latency
      - alert: HighQueryLatency
        expr: histogram_quantile(0.95, themisdb_query_duration_seconds_bucket) > 1.0
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High query latency detected"
          description: "p95 query latency is {{ $value }}s"

      # Low Cache Hit Rate
      - alert: LowCacheHitRate
        expr: themisdb_cache_hit_rate < 0.7
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Low cache hit rate"
          description: "Cache hit rate is {{ $value }}"

      # High Connection Usage
      - alert: HighConnectionUsage
        expr: (themisdb_active_connections / themisdb_connection_pool_size) > 0.8
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Connection pool nearly exhausted"
          description: "{{ $value | humanizePercentage }} of connections in use"

      # Database Down
      - alert: DatabaseDown
        expr: up{job="themisdb"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "ThemisDB instance is down"
          description: "Instance {{ $labels.instance }} is unreachable"

      # Replication Lag
      - alert: HighReplicationLag
        expr: themisdb_replication_lag_seconds > 10
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High replication lag"
          description: "Replication lag is {{ $value }}s"
```

18.6.2 Alertmanager Konfiguration

Alertmanager Config:

```yaml
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'cluster']
  group_wait: 10s
  group_interval: 10s
  repeat_interval: 12h
  receiver: 'team-db'

  routes:
    - match:
        severity: critical
      receiver: 'pagerduty'

    - match:
        severity: warning
      receiver: 'slack'

receivers:
  - name: 'team-db'
    email_configs:
      - to: 'db-team@company.com'
        from: 'alertmanager@company.com'
        smarthost: 'smtp.company.com:587'

  - name: 'pagerduty'
    pagerduty_configs:
      - service_key: 'YOUR_PAGERDUTY_KEY'

  - name: 'slack'
    slack_configs:
      - api_url: 'https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK'
        channel: '#db-alerts'
        text: '{{ range .Alerts }}{{ .Annotations.summary }}\n{{ end }}'
```

18.7 Application Performance Monitoring (APM)

18.7.1 Query Performance Tracking

Slow Query Logging:

```python
import time
from functools import wraps

SLOW_QUERY_THRESHOLD = 1.0  # Sekunden

def track_query_performance(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time()
        query = args[0] if args else kwargs.get('query')

        try:
            result = func(*args, **kwargs)
            duration = time.time() - start_time

            # Slow Query loggen
            if duration > SLOW_QUERY_THRESHOLD:
                logger.warning(
                    "Slow query detected",
                    extra={
                        'query': query,
                        'duration_ms': duration * 1000,
                        'threshold_ms': SLOW_QUERY_THRESHOLD * 1000
                    }
                )

            # Metriken updaten
            query_duration.labels(type=get_query_type(query)).observe(duration)

            return result
        except Exception as e:
            duration = time.time() - start_time
            logger.error(
                "Query execution failed",
                extra={
                    'query': query,
                    'duration_ms': duration * 1000,
                    'error': str(e)
                }
            )
            raise

    return wrapper

@track_query_performance
def execute_query(query):
    return db.execute(query)
```

18.7.2 Transaction Monitoring

Transaction Tracking:

```python
class TransactionMonitor:
    def __init__(self):
        self.active_transactions = {}
        self.transaction_counter = Counter('themisdb_transactions_total',
                                           'Total transactions',
                                           ['status'])
        self.transaction_duration = Histogram('themisdb_transaction_duration_seconds',
                                              'Transaction duration')

    def begin_transaction(self, txn_id):
        self.active_transactions[txn_id] = {
            'start_time': time.time(),
            'queries': []
        }

    def add_query(self, txn_id, query):
        if txn_id in self.active_transactions:
            self.active_transactions[txn_id]['queries'].append(query)

    def commit_transaction(self, txn_id):
        if txn_id in self.active_transactions:
            duration = time.time() - self.active_transactions[txn_id]['start_time']
            query_count = len(self.active_transactions[txn_id]['queries'])

            self.transaction_counter.labels(status='committed').inc()
            self.transaction_duration.observe(duration)

            logger.info(
                "Transaction committed",
                extra={
                    'txn_id': txn_id,
                    'duration_ms': duration * 1000,
                    'query_count': query_count
                }
            )

            del self.active_transactions[txn_id]

    def rollback_transaction(self, txn_id):
        if txn_id in self.active_transactions:
            duration = time.time() - self.active_transactions[txn_id]['start_time']

            self.transaction_counter.labels(status='rolled_back').inc()

            logger.warning(
                "Transaction rolled back",
                extra={
                    'txn_id': txn_id,
                    'duration_ms': duration * 1000
                }
            )

            del self.active_transactions[txn_id]
```

18.8 Health Checks

18.8.1 Liveness & Readiness Probes

Health Check Endpoints:

```python
from flask import Flask, jsonify
import time

app = Flask(__name__)

@app.route('/health/live')
def liveness():
    """Prüft ob die Anwendung läuft."""
    return jsonify({'status': 'ok'}), 200

@app.route('/health/ready')
def readiness():
    """Prüft ob die Anwendung Requests bearbeiten kann."""
    try:
        # Datenbankverbindung testen
        db.execute("SELECT 1")

        # Connection Pool prüfen
        stats = db.statistics()
        if stats['connections']['available'] == 0:
            return jsonify({
                'status': 'not_ready',
                'reason': 'Connection pool exhausted'
            }), 503

        # Replikations-Lag prüfen
        if stats.get('replication_lag', 0) > 30:
            return jsonify({
                'status': 'not_ready',
                'reason': 'High replication lag'
            }), 503

        return jsonify({'status': 'ready'}), 200

    except Exception as e:
        return jsonify({
            'status': 'not_ready',
            'reason': str(e)
        }), 503

@app.route('/health/startup')
def startup():
    """Prüft ob die Anwendung vollständig gestartet ist."""
    # Initialisierungs-Checks
    checks = {
        'database_connected': check_db_connection(),
        'cache_initialized': check_cache(),
        'migrations_applied': check_migrations()
    }

    if all(checks.values()):
        return jsonify({'status': 'started', 'checks': checks}), 200
    else:
        return jsonify({'status': 'starting', 'checks': checks}), 503
```

18.8.2 Kubernetes Probes

Pod-Konfiguration:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: themisdb
spec:
  containers:
  - name: themisdb
    image: themisdb:latest
    ports:
    - containerPort: 8529

    livenessProbe:
      httpGet:
        path: /health/live
        port: 8529
      initialDelaySeconds: 30
      periodSeconds: 10
      timeoutSeconds: 5
      failureThreshold: 3

    readinessProbe:
      httpGet:
        path: /health/ready
        port: 8529
      initialDelaySeconds: 10
      periodSeconds: 5
      timeoutSeconds: 3
      failureThreshold: 3

    startupProbe:
      httpGet:
        path: /health/startup
        port: 8529
      initialDelaySeconds: 0
      periodSeconds: 10
      timeoutSeconds: 3
      failureThreshold: 30
```

18.9 Performance Profiling

18.9.1 Query Profiling

EXPLAIN ANALYZE:

```sql
-- Query Plan analysieren
EXPLAIN ANALYZE
SELECT c.name, COUNT(o.id) as order_count
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE c.country = 'DE'
GROUP BY c.name
ORDER BY order_count DESC
LIMIT 10;

-- Output:
-- -> Limit (cost=1245.34..1245.36 rows=10)
--    -> Sort (cost=1245.34..1276.42 rows=12432)
--          Sort Key: order_count DESC
--          -> HashAggregate (cost=956.23..1080.55 rows=12432)
--                -> Hash Left Join (cost=345.67..876.12 rows=16040)
--                      Hash Cond: (c.id = o.customer_id)
--                      -> Index Scan using idx_customers_country on customers c
--                           Index Cond: (country = 'DE')
--                      -> Hash (cost=234.56..234.56 rows=8884)
--                           -> Seq Scan on orders o
```

18.9.2 Python Profiling

cProfile Integration:

```
import cProfile
import pstats
from pstats import SortKey

def profile_query(query):
    profiler = cProfile.Profile()
    profiler.enable()

    # Query ausführen
    result = db.execute(query)

    profiler.disable()

    # Statistiken ausgeben
    stats = pstats.Stats(profiler)
    stats.sort_stats(SortKey.CUMULATIVE)
    stats.print_stats(10)

    return result

from line_profiler import LineProfiler

@profile
def complex_operation():
    # Schritt 1
    data = db.query("SELECT * FROM large_table")

    # Schritt 2
    processed = [process_row(row) for row in data]

    # Schritt 3
    db.bulk_insert("results", processed)

from memory_profiler import profile as memory_profile

@memory_profile
def memory_intensive_operation():
    large_dataset = db.query("SELECT * FROM huge_table")
    return process_dataset(large_dataset)
```

18.10 Best Practices

18.10.1 Monitoring-Strategie

Golden Signals:

1. **Latency**: Zeit für Request-Bearbeitung
2. **Traffic**: Anzahl Requests pro Zeiteinheit
3. **Errors**: Rate fehlgeschlagener Requests
4. **Saturation**: Auslastung der Ressourcen

RED Method (für Services): - **Rate**: Requests pro Sekunde - **Errors**: Fehlerrate - **Duration**: Latenz-Verteilung

USE Method (für Ressourcen): - **Utilization**: Prozentuale Auslastung - **Saturation**: Wartezeit/Queueing - **Errors**: Fehlerzähler

18.10.2 Retention Policies

Datenaufbewahrung:

```
prometheus:
  retention:
    time: 15d
    size: 50GB

recording_rules:
  - name: themisdb_5m
    interval: 5m
    rules:
      - record: themisdb:query_rate:5m
        expr: rate(themisdb_queries_total[5m])

      - record: themisdb:query_latency_p95:5m
        expr: histogram_quantile(0.95, themisdb_query_duration_seconds_bucket[5m])
```

Log-Rotation:

```
/var/log/themisdb/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0640 themisdb themisdb
    sharedscripts
    postrotate
        systemctl reload themisdb
    endscript
}
```

18.10.3 Dashboard-Design

Effektive Dashboards:

1. **Übersichts-Dashboard**: Hochlevel-Metriken, Status-Indikatoren
2. **Detail-Dashboards**: Tiefere Einblicke für spezifische Bereiche
3. **Drill-Down**: Von Übersicht zu Details navigieren
4. **Zeitfenster**: Flexible Zeitbereichs-Auswahl
5. **Annotations**: Deployment-Marker, Incidents

Dashboard-Struktur: - **Oberste Zeile**: SLI/SLO Status, Alerts - **Zweite Zeile**: Golden Signals (Latency, Traffic, Errors, Saturation) - **Weitere Zeilen**: Spezifische Metriken nach Kategorie

Zusammenfassung

Monitoring und Observability sind essentiell für den Betrieb von ThemisDB:

- **Metriken**: Prometheus für Sammlung und Speicherung
- **Visualisierung**: Grafana für Dashboards
- **Logging**: ELK Stack für Log-Aggregation und Analyse
- **Tracing**: Jaeger/OpenTelemetry für Distributed Tracing
- **Alerting**: Proaktive Benachrichtigung bei Problemen
- **Health Checks**: Kubernetes-Integration für Orchestrierung
- **Profiling**: Performance-Analyse und Optimierung

Mit diesen Tools und Praktiken können Sie ThemisDB effektiv überwachen und Probleme frühzeitig erkennen.

Kapitel 21: Performance Tuning

Übersicht

Dieses Kapitel behandelt Performance-Optimierungen für ThemisDB.

Hauptthemen

- Query-Optimierung
- Index-Tuning
- Cache-Strategien
- Metriken und Benchmarking

Weitere Informationen

Siehe auch: Query Optimierung, Monitoring

Kapitel 22: Authentifizierung

Übersicht

Dieses Kapitel behandelt Authentifizierungsmechanismen in ThemisDB.

Hauptthemen

- Benutzer-Management
- Token-basierte Authentifizierung
- LDAP/AD-Integration
- Single Sign-On (SSO)

Weitere Informationen

Siehe auch: Security Hardening

Kapitel 23: Verschlüsselung

Übersicht

Dieses Kapitel behandelt Verschlüsselungstechniken in ThemisDB.

Hauptthemen

- Transportverschlüsselung (TLS)
- Datenverschlüsselung
- Schlüssel-Management
- Verschlüsselte Backups

Weitere Informationen

Siehe auch: Security Hardening, Backup

Kapitel 24: Kapitel 23: Testing & Quality Assurance

> *"Untested code is legacy code. In ThemisDB, comprehensive testing is not optional—it's architecture."*

Überblick

Zuverlässige Datenbanken erfordern rigorose Tests auf mehreren Ebenen: Unit-Tests für AQL-Funktionen, Integrationstests für Transaktionen, Performance-Tests für Sharding, und Chaos-Tests für Netzwerkfehler.

Was Sie in diesem Kapitel lernen: - AQL Unit-Testing mit AQL-Assertions - Transaktions-Integrationstests - Performance-Benchmarking - Chaos Engineering für Fehlerszenarien - Mutation Testing für Query-Robustheit - CI/CD Pipeline-Integration

---

23.1 AQL Unit Testing

Test-Framework Setup

```
-- test_utils.aql: Utility-Funktionen für Tests
FUNCTION assert_equal(actual, expected, message) {
  IF actual != expected THEN
    THROW ERROR(CONCAT("Assertion failed: ", message,
      " (expected: ", expected, ", got: ", actual, ")"))
  END
  RETURN {passed: true, message: message}
}

FUNCTION assert_true(condition, message) {
  IF !condition THEN
    THROW ERROR(CONCAT("Assertion failed: ", message))
  END
  RETURN {passed: true}
}

FUNCTION assert_length(collection, expected_count, message) {
  LET actual_count = LENGTH(collection)
  IF actual_count != expected_count THEN
    THROW ERROR(CONCAT(message, " - Expected ", expected_count,
      " items, got ", actual_count))
  END
  RETURN {passed: true, count: actual_count}
}
```

Datenvorbereitung (Fixtures)

```
FUNCTION setup_test_data() {
  -- Cleanup
  FOR doc IN test_users
    REMOVE doc IN test_users

  -- Fixtures einfügen
  FOR user IN [
    {_key: "alice", name: "Alice", role: "admin", created_at: DATE_NOW()},
    {_key: "bob", name: "Bob", role: "user", created_at: DATE_NOW()},
    {_key: "charlie", name: "Charlie", role: "user", created_at: DATE_NOW()}
  ]
    INSERT user INTO test_users

  RETURN {inserted: 3, status: "ready"}
}

FUNCTION teardown_test_data() {
  FOR doc IN test_users
    REMOVE doc IN test_users
  RETURN {cleaned: true}
}
```

Test Cases für Geschäftslogik

```
-- Test 1: Benutzerverwaltung
FUNCTION test_user_creation() {
  CALL setup_test_data()

  -- Neuen User einfügen
  INSERT {_key: "dave", name: "Dave", role: "user"} INTO test_users

  -- Assertion: User existiert
  LET user = DOCUMENT("test_users/dave")
  CALL assert_equal(user.name, "Dave", "User name mismatch")
  CALL assert_equal(user.role, "user", "User role mismatch")

  CALL teardown_test_data()
  RETURN {test: "test_user_creation", status: "PASSED"}
}

-- Test 2: Permission-Checks
FUNCTION test_admin_permissions() {
  CALL setup_test_data()

  LET admin = DOCUMENT("test_users/alice")
  CALL assert_equal(admin.role, "admin", "Admin role check")

  CALL teardown_test_data()
  RETURN {test: "test_admin_permissions", status: "PASSED"}
}

-- Test 3: Transaktionale Konsistenz
FUNCTION test_transfer_consistency() {
  -- Zwei Test-Accounts mit initialen Balances
  INSERT {_key: "acc1", balance: 1000, owner: "alice"} INTO test_accounts
  INSERT {_key: "acc2", balance: 500, owner: "bob"} INTO test_accounts

  -- Überweisung: 200 von acc1 zu acc2
  FOR acc IN test_accounts
    FILTER acc._key == "acc1"
    UPDATE acc WITH {balance: acc.balance - 200}

  FOR acc IN test_accounts
    FILTER acc._key == "acc2"
    UPDATE acc WITH {balance: acc.balance + 200}

  -- Assertion: Beide Accounts korrekt
  LET acc1 = DOCUMENT("test_accounts/acc1")
  LET acc2 = DOCUMENT("test_accounts/acc2")
  CALL assert_equal(acc1.balance, 800, "Account 1 balance")
  CALL assert_equal(acc2.balance, 700, "Account 2 balance")

  RETURN {test: "test_transfer_consistency", status: "PASSED"}
}
```

23.2 Integrations-Tests

Multi-Collection Queries

```
FUNCTION test_complex_join_query() {
  -- Setup
  INSERT [
    {_key: "p1", title: "Product A", price: 100},
    {_key: "p2", title: "Product B", price: 200}
  ] INTO test_products

  INSERT [
    {_key: "o1", product_id: "p1", quantity: 2, customer: "alice"},
    {_key: "o2", product_id: "p2", quantity: 1, customer: "bob"}
  ] INTO test_orders

  -- Query
  LET results = (
    FOR order IN test_orders
      LET product = DOCUMENT(order.product_id)
      RETURN {
        order_id: order._key,
        product: product.title,
        total: product.price * order.quantity,
        customer: order.customer
      }
  )

  -- Assertions
  CALL assert_length(results, 2, "Expected 2 orders")
  CALL assert_equal(results[0].total, 200, "Order 1 total (2 * 100)")
  CALL assert_equal(results[1].total, 200, "Order 2 total (1 * 200)")

  RETURN {test: "test_complex_join_query", status: "PASSED"}
}
```

Transaktions-Rollback Tests

```
FUNCTION test_transaction_rollback() {
  -- Initiales Setup
  INSERT {_key: "src", balance: 1000} INTO test_wallets
  INSERT {_key: "dst", balance: 0} INTO test_wallets

  -- Fehlgeschlagene Transaktion simulieren
  TRY
    -- Transfer verschieben
    UPDATE "test_wallets/src" WITH {balance: 800}
    UPDATE "test_wallets/dst" WITH {balance: 200}

    -- Fehler vor Commit
    THROW ERROR("Simulated network failure")
  CATCH err
    -- Bei Fehler sollten beide Wallets unverändert sein
    -- (In echtem ACID-System auto-rollback)
  END

  -- Verification (bei echtem ACID Rollback)
  LET src = DOCUMENT("test_wallets/src")
  CALL assert_equal(src.balance, 1000, "Source wallet rolled back")

  RETURN {test: "test_transaction_rollback", status: "PASSED"}
}
```

23.3 Performance-Tests

Query-Performance Benchmarking

```python
import time
import themis

client = themis.Client('http://localhost:8529')

def benchmark_query(query_aql, params, iterations=100):
    """Messe Query-Performance über mehrere Iterationen"""
    times = []

    for i in range(iterations):
        start = time.time()
        result = client.query(query_aql, params)
        elapsed = time.time() - start
        times.append(elapsed * 1000)  # in ms

    import statistics
    return {
        'min_ms': min(times),
        'max_ms': max(times),
        'avg_ms': statistics.mean(times),
        'p95_ms': sorted(times)[int(len(times) * 0.95)],
        'iterations': iterations
    }

perf = benchmark_query(
    "FOR u IN users FILTER u.active == true RETURN u",
    {},
    iterations=100
)
print(f"Simple filter: avg {perf['avg_ms']:.2f}ms, p95 {perf['p95_ms']:.2f}ms")

perf = benchmark_query("""
  FOR order IN orders
    LET customer = DOCUMENT(order.customer_id)
    LET items = (FOR oi IN order_items FILTER oi.order_id == order._id RETURN oi)
    RETURN {order: order, customer: customer, items: items}
""", {}, iterations=50)
print(f"Complex join: avg {perf['avg_ms']:.2f}ms, p95 {perf['p95_ms']:.2f}ms")
```

Skalierbarkeits-Tests

```
-- Test mit großen Datenmengen
FUNCTION test_scalability_1m_documents() {
  -- Benchmark für 1 Million Dokumente

  -- Measure: Full table scan
  LET scan_start = DATE_NOW()
  LET count = LENGTH(
    FOR doc IN large_collection
    RETURN doc._key
  )
  LET scan_time = DATE_DIFF(scan_start, DATE_NOW(), 'ms')

  -- Measure: Filtered query (mit Index)
  LET filter_start = DATE_NOW()
  LET filtered = LENGTH(
    FOR doc IN large_collection
    FILTER doc.status == "active"
    RETURN doc
  )
  LET filter_time = DATE_DIFF(filter_start, DATE_NOW(), 'ms')

  RETURN {
    collection_size: count,
    scan_time_ms: scan_time,
    filtered_results: filtered,
    filter_time_ms: filter_time,
    throughput_doc_per_sec: count / (scan_time / 1000)
  }
}
```

23.4 Chaos Engineering

Netzwerk-Fehler Simulieren

```python
import random
from unittest.mock import patch

class ChaosMonkey:
    def __init__(self, client):
        self.client = client
        self.failure_rate = 0.1  # 10% Fehler

    def simulate_network_partition(self, query, params, max_retries=3):
        """Simuliere Netzwerk-Partition mit Retry-Logik"""
        for attempt in range(max_retries):
            try:
                if random.random() < self.failure_rate:
                    raise ConnectionError("Network partition")
                return self.client.query(query, params)
            except ConnectionError as e:
                if attempt == max_retries - 1:
                    raise
                time.sleep(2 ** attempt)  # Exponential backoff

    def test_resilience(self):
        """Test ob Application Fehler korrekt behandelt"""
        results = []
        for i in range(100):
            try:
                result = self.simulate_network_partition(
                    "FOR u IN users RETURN u",
                    {}
                )
                results.append({'status': 'success'})
            except Exception as e:
                results.append({'status': 'failed', 'error': str(e)})

        success_rate = sum(1 for r in results if r['status'] == 'success') / len(results)
        print(f"Success rate with 10% failure injection: {success_rate*100:.1f}%")
        return success_rate > 0.95  # Mind. 95% sollten erfolgreich sein

chaos = ChaosMonkey(client)
assert chaos.test_resilience(), "Application nicht resilient genug"
```

Datenbeschädigung Testen

```
FUNCTION test_data_corruption_detection() {
  -- Simuliere fehlerhafte Schreibvorgänge
  INSERT {_key: "doc1", checksum: NULL, data: {value: 100}} INTO test_collection

  -- Manuell Datenbeschädigung eintragen
  UPDATE "test_collection/doc1" WITH {data: {value: 999}}

  -- Validierungs-Funktion
  FOR doc IN test_collection
    FILTER doc._key == "doc1"
    LET calc_checksum = HASH(JSON_STRINGIFY(doc.data))
    FILTER calc_checksum != doc.checksum
    RETURN {
      doc_id: doc._key,
      status: "CORRUPTED",
      expected_value: 100,
      actual_value: doc.data.value
    }
}
```

23.5 Mutation Testing

Mutation Testing verändert absichtlich Queries, um zu prüfen, ob Tests ausreichen:

```
-- Original Query
FUNCTION get_active_users() {
  FOR user IN users
    FILTER user.status == "active"
    RETURN user
}

-- Mutation 1: Operator-Mutation (== wird zu !=)
-- Test sollte diesen Fehler erkennen:
-- FOR user IN users
--   FILTER user.status != "active"  -- FEHLER: Falscher Operator
--   RETURN user

-- Mutation 2: Literal-Mutation ("active" wird zu "inactive")
-- Test sollte erkennen:
-- FOR user IN users
--   FILTER user.status == "inactive"  -- FEHLER: Falscher Status
--   RETURN user
```

Gutes Mutation-Testing-Result: >85% der Mutationen werden von Tests erkannt.

23.6 CI/CD Integration

GitHub Actions Pipeline

```
name: ThemisDB QA Pipeline

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Start ThemisDB
        run: docker run -d -p 8529:8529 themisdb/server:latest

      - name: Run AQL Unit Tests
        run: |
          python3 test_runner.py --aql-tests

      - name: Run Integration Tests
        run: |
          python3 test_runner.py --integration-tests

      - name: Performance Benchmarks
        run: |
          python3 test_performance.py --baseline baseline.json

      - name: Upload Results
        uses: actions/upload-artifact@v3
        with:
          name: test-results
          path: reports/
```

23.7 Test-Strategie Zusammenfassung

Test-Typ	Häufigkeit	Dauer	Kritikalität
Unit Tests	Jeder Commit	<5s	Hoch
Integration Tests	PR-Request	<30s	Hoch
Performance Tests	Täglich	2-5min	Mittel
Chaos Tests	Wöchentlich	10-20min	Mittel
Load Tests	Vor Release	30-60min	Hoch

Best Practices: - Test-getriebene Entwicklung (TDD) - Mindestens 80% Code-Coverage - Fixtures für reproduzierbare Tests - Parameterisierte Tests für Grenzfälle

- Golden Files für Regression-Tests - Automatisierte Performance-Regression-Erkennung

---

23.8 Property-Based Testing

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers(), min_size=1))
def test_aggregate_sum_commutative(numbers):
    aql = f"RETURN SUM({numbers})"
    result1 = client.execute(aql)
    result2 = client.execute(f"RETURN SUM({list(reversed(numbers))})")
    assert result1 == result2, "SUM must be commutative"
```

23.9 Boundary Value Testing

```
-- test_boundaries.aql
FUNCTION test_pagination() {
  LET cases = [
    {offset: 0, limit: 1},
    {offset: 0, limit: 1000},
    {offset: 999, limit: 1}
  ]
  FOR test IN cases
    RETURN {case: test, valid: true}
}
```

23.10 Flaky Test Prevention

```
def flaky_test_retry(max_attempts=3, backoff=2):
    def decorator(test_func):
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return test_func(*args, **kwargs)
                except Exception:
                    if attempt < max_attempts - 1:
                        time.sleep(backoff ** attempt)
            raise
        return wrapper
    return decorator
```

23.11 Test Reporting

```
class TestReport:
    def summarize(self, results):
        passed = sum(1 for r in results if r['status'] == 'passed')
        total = len(results)
        return {
            "passed": passed,
            "failed": total - passed,
            "success_rate": (passed / total) * 100
        }
```

Kapitel 25: Kapitel 24: KI-Ethik und Governance

Autor: ThemisDB Team
Reviewer: TBD
Status: Draft
Letzte Aktualisierung: 30. Dezember 2025
Version: 1.0.0

Lernziele

Nach dem Durcharbeiten dieses Kapitels sollten Sie:

- [x] Die ethischen Herausforderungen von KI-Systemen in der öffentlichen Verwaltung verstehen
- [x] Spezifische Risiken entlang der VCC-Architektur (Covina, Clara, Veritas) identifizieren können
- [x] "Ethik-by-Design"-Prinzipien auf datenbankgestützte KI-Systeme anwenden können
- [x] Governance-Mechanismen zur Risikominimierung implementieren können

---

Voraussetzungen

Dieses Kapitel setzt Kenntnisse aus folgenden Kapiteln voraus:

- **Kapitel 10**: Enterprise Applications - Security & Compliance Grundlagen
- **Kapitel 17**: LLM Integration - RAG-Architektur und Pre-Filtering

---

Überblick

Die Einführung eines KI-Ökosystems wie **VCC (Veritas, Covina, Clara)** in der öffentlichen Verwaltung wirft tiefgreifende ethische Fragen auf, die über die reine Rechtskonformität (DSGVO, EU AI Act) hinausgehen [9]. Der Kern des ethischen Spannungsfelds liegt im Auftrag der Verwaltung: Sie muss nicht nur "richtig" im Sinne von effizient und faktisch korrekt handeln, sondern auch "gerecht" im Sinne von fair, unvoreingenommen und der Rechtsstaatlichkeit verpflichtet sein.

In diesem Kapitel behandeln wir: 1. Ethische Herausforderungen in verwaltungs-KI 2. Architektonische Risikoquellen (Covina, Clara, Veritas) 3. Bias-Audits und Datenqualität 4. Human-in-the-Loop und Automation Bias 5. Governance-Framework und Ethik-by-Design 6. Praktische Implementierung in ThemisDB

24.1 Der Ethische Imperativ für Verwaltungs-KI

24.1.1 Unterschied zu kommerzieller KI

Während kommerzielle KI-Systeme primär auf Effizienz und Profitabilität optimiert sind, tragen Verwaltungs-KI-Systeme eine fundamental andere Verantwortung [9]:

Kommerzielle KI: - Ziel: Geschäftserfolg, Kundenzufriedenheit - Fehlertoleranz: Wirtschaftlicher Schaden, Reputation - Kontrolle: Marktmechanismen, Wettbewerb

Verwaltungs-KI: - Ziel: Rechtsstaatlichkeit, Gerechtigkeit, Gemeinwohl - Fehlertoleranz: Grundrechtsverletzungen, Vertrauensverlust in Staat - Kontrolle: Demokratische Legitimation, Rechtsaufsicht

> ***Wichtig:*** *Ein Fehler in einem Verwaltungsakt (z.B. falsche Ablehnung einer BImSchG-Genehmigung) kann existenzielle Folgen für Bürger oder Unternehmen haben. Die Entscheidung muss nicht nur "wahrscheinlich richtig", sondern nachweisbar korrekt und ethisch vertretbar sein.*

24.1.2 Digitale Souveränität und ethische Verantwortung

Die Entscheidung für einen On-Premise-Betrieb (wie bei ThemisDB) sichert zwar die Datensouveränität, verlagert aber die ethische Verantwortung für den gesamten Datenlebenszyklus vollständig in den Hoheitsbereich der Verwaltung [9].

Konsequenzen: - Keine Ausrede "Der Cloud-Provider war schuld" - Volle Kontrolle = Volle Verantwortung - Notwendigkeit interner Expertise und Governance

---

24.2 Architektonische Risikoquellen im VCC-Ökosystem

Die Analyse identifiziert drei zentrale Risiken entlang der VCC-Architektur [9]:

24.2.1 Covina (Ingestion-Engine): Das Tor zur Voreingenommenheit

Funktion: Covina verarbeitet und indiziert Verwaltungsdokumente in die Wissensbasis.

Ethisches Risiko: Verfestigung von Vorurteilen durch historische Dokumente [9].

Das Prinzip "Garbage In, Garbage Out" wird hier zu **"Garbage In, Gospel Out"** [9]. Wenn die von Covina verarbeiteten Dokumente (z.B. alte Verwaltungsvorschriften, Gerichtsurteile) historische oder systemische Vorurteile enthalten, wird das KI-System diese Vorurteile nicht nur reproduzieren, sondern sie mit der Autorität einer scheinbar objektiven Maschine verstärken.

Konkretes Beispiel:

```
dokument = {
    "titel": "Bearbeitungsrichtlinie Bauanträge 1985",
    "inhalt": "Bei Anträgen von Antragstellern mit nicht-deutschen Namen
               ist besondere Sorgfalt bei der Prüfung geboten..."
}
```

ThemisDB-spezifische Gefahr: - Atomare ACID-Transaktionen garantieren *technische* Konsistenz - Sie garantieren NICHT *inhaltliche* oder *ethische* Korrektheit - Ein fehlerhaftes Dokument wird konsistent über alle Index-Layer repliziert

Mitigationsstrategien:

1. **Proaktive Bias-Audits vor Ingestion** [9]:

```
def audit_document_for_bias(doc: Document) -> BiasReport:
    """
    Scannt Dokument auf potenzielle Vorurteile vor Covina-Ingestion.
    """
    bias_patterns = [
        r"Antragsteller.*nicht-deutsch.*besondere Sorgfalt",
        r"Geschlecht.*erhöhtes Risiko",
        r"Alter.*über \d+.*automatisch ablehnen"
    ]

    detected_biases = []
    for pattern in bias_patterns:
        if re.search(pattern, doc.content, re.IGNORECASE):
            detected_biases.append({
                "pattern": pattern,
                "location": "...",  # Kontext
                "severity": "HIGH"
            })

    return BiasReport(
        document_id=doc.id,
        timestamp=datetime.now(),
        biases_found=detected_biases,
        recommendation="BLOCK" if detected_biases else "APPROVE"
    )
```

1. **Dokumenten-Provenienz-Tracking**:

```
-- ThemisDB: Metadaten für ethische Nachverfolgbarkeit
INSERT INTO documents (id, content, metadata) VALUES (
    'doc_123',
    @content,
    {
        "source": "Archiv Brandenburg",
        "original_date": "1985-03-15",
        "bias_audit_status": "REVIEWED",
        "bias_audit_date": "2025-12-01",
        "bias_findings": ["HISTORICAL_DISCRIMINATION"],
        "usage_restriction": "CONTEXT_ONLY_NO_TRAINING"
    }
);
```

1. **Temporale Contextualisierung**:
2. Markierung historischer Dokumente mit Zeitstempel
3. Abwertung alter Dokumente im Ranking
4. Explizite Warnung bei Verwendung

24.2.2 Clara (Modellverbesserungs-Kreislauf): Permanente Intelligenz-Korruption

Funktion: Clara verbessert KI-Modelle durch Nutzerfeedback (LoRA-Adapter, Fine-Tuning).

Ethisches Risiko: Kaskadierende Integritätskompromittierung durch fehlerhaftes Feedback [9].

Szenario "Gelernte Lüge":

1. **Initiale Fehlinformation:** Sachbearbeiter A gibt falsches Feedback: `json { "query": "Abstandsregeln BImSchG für Windkraftanlagen", "answer": "Mindestabstand 2000m zu Wohngebieten", "feedback": "CORRECT", "user": "sachbearbeiter_a" }` (Tatsächlich: Regelung ist komplexer, pauschal falsch)

2. **Clara lernt:** LoRA-Adapter wird mit diesem Feedback trainiert

3. **Propagierung:** System gibt nun selbstbewusst falsche Auskunft
4. **Kaskadierende Verstärkung:** Weitere Nutzer vertrauen der KI, geben ebenfalls "CORRECT"-Feedback
5. **Permanente Kodierung:** Fehler wird in Model-Weights "eingebrannt"

Architektonisches Problem:

```
// Clara's Feedback-Loop (vereinfacht)
void Clara::ProcessFeedback(const UserFeedback& feedback) {
    if (feedback.rating == "CORRECT") {
        // ⚠ KEINE VALIDIERUNG ob Feedback faktisch korrekt ist
        lora_adapter_.UpdateWeights(feedback.query, feedback.answer);

        // Atomic Transaction garantiert nur ACID, nicht Wahrheit
        db_->BeginTransaction();
        db_->StoreFeedback(feedback);
        db_->UpdateModelVersion(lora_adapter_.version());
        db_->Commit();  // Fehler ist nun "consistent" gespeichert
    }
}
```

Zusätzliches Risiko: Echokammer-Effekt [9] - Nur Mehrheitsmeinungen trainieren das System - Minderheitenpositionen werden unterdrückt - Pluralismus der Rechtsmeinungen geht verloren

Mitigationsstrategien:

1. **Experten-Review vor Model-Update** [9]:

```
class FeedbackValidator:
    def __init__(self, expert_threshold: int = 2):
        self.expert_threshold = expert_threshold
        self.pending_feedback = {}

    def validate_feedback(self, feedback: Feedback) -> ValidationResult:
        """
        Feedback wird erst nach Expert-Review in Model übernommen.
        """
        key = (feedback.query_hash, feedback.answer_hash)

        if key not in self.pending_feedback:
            self.pending_feedback[key] = {
                "feedback": feedback,
                "expert_approvals": 0,
                "expert_rejections": 0
            }
            return ValidationResult.PENDING

        # Experten-Konsens erforderlich
        if self.pending_feedback[key]["expert_approvals"] >= self.expert_threshold:
            return ValidationResult.APPROVED

        return ValidationResult.PENDING
```

1. **Stichprobenbasierte ethische Audits** [9]:

```sql
-- ThemisDB: Query für ethisch problematische Feedback-Patterns
SELECT
    f.query,
    f.answer,
    COUNT(*) as feedback_count,
    AVG(CASE WHEN f.rating = 'CORRECT' THEN 1 ELSE 0 END) as approval_rate,
    STRING_AGG(f.user_id, ',') as users
FROM feedback f
WHERE f.timestamp > NOW() - INTERVAL '7 days'
GROUP BY f.query, f.answer
HAVING approval_rate > 0.9 AND feedback_count < 5
-- ⚠ Verdächtig: Hohe Zustimmung bei wenig Feedback = Echokammer?
ORDER BY approval_rate DESC;
```

1. **Niemals vollständig autonomes Lernen** [9]:
2. Clara-Updates nur in Staging-Umgebung
3. A/B-Testing gegen Baseline-Modell
4. Human-in-the-Loop bei kritischen Rechtsgebieten

24.2.3 Veritas (Benutzerinteraktion): Erosion der Urteilskraft

Funktion: Veritas ist die Chat-Schnittstelle für Sachbearbeiter.

Ethisches Risiko: Automation Bias und Verantwortungsdiffusion [9].

Automation Bias: Die menschliche Tendenz, Ergebnissen automatisierter Systeme übermäßig und unkritisch zu vertrauen [9].

Konkretes Szenario:

```
Sachbearbeiter: "Genehmigungsfähigkeit BImSchG-Antrag
                  Windkraftanlage Havelland?"

Veritas (KI):   "  Genehmigungsfähig.
                 Gemäß §5 BImSchG erfüllt der Antrag alle Voraussetzungen.

                   Quellen:
                 - BImSchG §5 Abs. 1 [Gesetz]
                 - VwV Havelland 2024 [Verwaltungsvorschrift]
                 - Gutachten Lärmschutz [Dokument #4521]"

Sachbearbeiter: "Klingt gut, übernehme ich so."
                [KLICKT "GENEHMIGEN" OHNE WEITERE PRÜFUNG]
```

Warum problematisch? - KI wirkt durch Quellen autoritativ - Sachbearbeiter delegiert kritisches Denken an Maschine - Bei Fehler: "Die KI hat das so gesagt" → Verantwortungsdiffusion

ThemisDB kann das Problem NICHT lösen:

```
// ThemisDB garantiert nur, dass Veritas konsistente Daten liefert
auto answer = veritas_->Query("BImSchG Windkraft");

//   ACID garantiert: Alle Quellen in answer sind transaktional korrekt
// × NICHT garantiert: Die INTERPRETATION ist rechtlich korrekt
// × NICHT garantiert: Sachbearbeiter hinterfragt das Ergebnis
```

Mitigationsstrategien:

1. **Explizite Unsicherheits-Kennzeichnung:**

```
class VeritasResponse:
    def __init__(self, answer: str, confidence: float, sources: List[Source]):
        self.answer = answer
        self.confidence = confidence  # 0.0 - 1.0
        self.sources = sources

    def to_ui(self) -> str:
        """
        Zeigt Unsicherheit transparent an.
        """
        confidence_label = {
            (0.9, 1.0): "  SEHR SICHER",
            (0.7, 0.9): "  MITTEL SICHER",
            (0.0, 0.7): "  UNSICHER - EXPERTENKONSULTATION EMPFOHLEN"
        }

        for (low, high), label in confidence_label.items():
            if low <= self.confidence < high:
                return f"""
                {label} (Konfidenz: {self.confidence:.2%})

                {self.answer}

                ⚠ HINWEIS: Dies ist eine KI-generierte Empfehlung.
                Bitte prüfen Sie die Quellen kritisch und konsultieren Sie
                bei Unsicherheit einen Rechtsexperten.
                """
```

1. **Mandatory Second-Opinion bei kritischen Entscheidungen:**

```
CRITICAL_DECISION_TYPES = [
    "GENEHMIGUNG_ERTEILEN",
    "GENEHMIGUNG_ABLEHNEN",
    "WIDERSPRUCH_ZURÜCKWEISEN"
]

def require_second_opinion(decision_type: str,
                          ai_recommendation: VeritasResponse) -> bool:
    """
    Erzwingt menschliche Zweitprüfung bei kritischen Entscheidungen.
    """
    if decision_type in CRITICAL_DECISION_TYPES:
        if ai_recommendation.confidence < 0.95:
            return True  # Mandatory review

    return False
```

1. **AI Literacy Training** [9]:
2. Schulungsprogramm für alle Sachbearbeiter
3. Erkennung von Automation Bias
4. Kritisches Hinterfragen von KI-Ausgaben
5. Verständnis für KI-Limitationen

Checkliste für Sachbearbeiter:

```
Bevor Sie eine KI-Empfehlung übernehmen:

- [ ] Habe ich die angegebenen Quellen SELBST geprüft?
- [ ] Widersprechen Quellen einander? (KI erkennt das oft nicht)
- [ ] Ist die Rechtslage eindeutig oder gibt es Interpretationsspielraum?
- [ ] Würde ich diese Entscheidung auch OHNE KI so treffen?
- [ ] Habe ich bei Unsicherheit einen Experten konsultiert?
- [ ] Ist die Begründung MEINE eigene oder nur KI-Paraphrase?

⚠ Bei NEIN zu einer Frage: STOPP und manuelle Prüfung!
```

24.3 Governance-Framework: Ethik-by-Design

Die abgeleiteten Handlungsempfehlungen sind konkrete technische und organisatorische Anforderungen [9]:

24.3.1 Interdisziplinäres KI-Ethik-Gremium

Zusammensetzung: - Juristen (Verwaltungsrecht, Datenschutz) - Informatiker (KI, Datenbanken) - Ethiker (Moralphilosophie) - Praktiker (Sachbearbeiter mit Felderfahrung) - Bürgervertreter (Zivilgesellschaft)

Aufgaben: 1. **Kontinuierliche Ethik-Audits:** `sql -- ThemisDB: Ethik-Audit-Log CREATE TABLE ethics_audits ( id UUID PRIMARY KEY, audit_date TIMESTAMP, component`

```
TEXT, -- 'covina', 'clara', 'veritas' findings TEXT[], risk_level TEXT, -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL' action_required TEXT, resolved BOOLEAN DEFAULT FALSE
);
```

1. **Entscheidung über kritische Fälle:**
2. Eskalation bei Automation-Bias-Verdacht
3. Review strittiger Clara-Feedbacks
4. Freigabe neuer Datenquellen für Covina
5. **Policy-Entwicklung:**
6. Wann ist KI-Nutzung erlaubt/verboten?
7. Welche Entscheidungen bleiben immer human?
8. Transparenzanforderungen gegenüber Bürgern

24.3.2 Bias-Audits für Datenquellen

Proaktiver Ansatz [9]:

```python
class CovinaBiasAuditor:
    def __init__(self, themis_db: ThemisDB):
        self.db = themis_db
        self.bias_patterns = self._load_bias_patterns()

    def audit_document_batch(self, documents: List[Document]) -> AuditReport:
        """
        Audited alle Dokumente vor Ingestion.
        """
        high_risk_docs = []

        for doc in documents:
            bias_score = self._calculate_bias_score(doc)

            if bias_score > 0.7:  # High risk threshold
                high_risk_docs.append({
                    "doc_id": doc.id,
                    "bias_score": bias_score,
                    "detected_patterns": self._extract_patterns(doc),
                    "recommendation": "MANUAL_REVIEW"
                })

                # Markierung in ThemisDB
                self.db.execute("""
                    UPDATE documents
                    SET metadata = jsonb_set(
                        metadata,
                        '{bias_audit}',
                        '{"status": "FLAGGED", "score": :score}'::jsonb
                    )
                    WHERE id = :doc_id
                """, {"doc_id": doc.id, "score": bias_score})

        return AuditReport(
            total_documents=len(documents),
            flagged_documents=len(high_risk_docs),
            details=high_risk_docs
        )
```

Integration in Covina-Pipeline:

```cpp
// covina/ingestion_pipeline.cpp
class IngestionPipeline {
public:
    void IngestDocument(const Document& doc) {
        // SCHRITT 1: Bias-Audit VOR Ingestion
        auto audit_result = bias_auditor_->Audit(doc);

        if (audit_result.risk_level == RiskLevel::HIGH) {
            // Blockieren und zur manuellen Review
            ethics_queue_->Enqueue(doc, audit_result);
            LogWarning("Document {} flagged for ethics review", doc.id);
            return;  //   NICHT indizieren
        }

        // SCHRITT 2: Normale Ingestion (nur bei bestanden Audit)
        auto txn = db_->BeginTransaction();
        db_->InsertBaseEntity(doc.id, doc.content, doc.metadata);
        indexer_->IndexDocument(doc);  // Relational, Graph, Vector
        txn->Commit();
    }
private:
    BiasAuditor* bias_auditor_;
    EthicsReviewQueue* ethics_queue_;
};
```

24.3.3 Human-in-the-Loop Policy

Principle: Kritische Entscheidungen IMMER mit menschlicher Kontrolle [9].

Klassifizierung von Entscheidungen:

Kategorie	Beispiel	KI-Rolle	Human-Rolle
Routine	Statusabfrage	Vollautomatisch	ℹ Info
Standard	Dokumentensuche	Vorschlag	Plausibilitätsprüfung
Komplex	Genehmigungsempfehlung	Assistenz	Entscheidung
Kritisch	Ablehnungsbescheid	Nur Hintergrunddaten	Volle Kontrolle

ThemisDB-Implementierung:

```sql
-- Audit-Trail für Human-in-the-Loop
CREATE TABLE decision_audit (
    id UUID PRIMARY KEY,
    timestamp TIMESTAMP,
    decision_type TEXT,
    ai_recommendation JSONB,
    human_decision JSONB,
    human_rationale TEXT,  -- Warum wich Mensch von KI ab?
    override BOOLEAN,  -- TRUE wenn Mensch KI überstimmt
    user_id TEXT
);

-- Statistik: Wie oft wird KI überstimmt?
SELECT
    decision_type,
    COUNT(*) as total_decisions,
    SUM(CASE WHEN override THEN 1 ELSE 0 END) as overrides,
    ROUND(100.0 * SUM(CASE WHEN override THEN 1 ELSE 0 END) / COUNT(*), 2) as override_rate_p
FROM decision_audit
GROUP BY decision_type
ORDER BY override_rate_pct DESC;

-- ⚠ Hohe Override-Rate = KI-Modell hat systematisches Problem
```

24.4 Praktische Implementierung in ThemisDB

24.4.1 Ethik-Metadaten-Schema

Erweiterte Base Entity mit Ethik-Tracking:

```json
{
  "_id": "doc_BImSchG_2024_123",
  "_type": "administrative_document",
  "content": {
    "title": "Verwaltungsvorschrift Windkraft Havelland",
    "text": "...",
    "legal_basis": ["BImSchG §5", "TA Lärm"]
  },
  "ethics_metadata": {
    "bias_audit": {
      "status": "PASSED",
      "audited_by": "bias_auditor_v2.1",
      "audit_date": "2025-12-01T10:00:00Z",
      "bias_score": 0.15,
      "flagged_patterns": []
    },
    "provenance": {
      "source": "Ministerium für Infrastruktur",
      "original_date": "2024-06-15",
      "historical_context": "MODERN_REGULATION",
      "reliability_score": 0.95
    },
    "usage_restrictions": {
      "allow_training": true,
      "allow_direct_citation": true,
      "require_human_review": false,
      "sensitivity_level": "PUBLIC"
    },
    "ethical_flags": {
      "contains_personal_data": false,
      "contains_protected_groups": false,
      "potential_discrimination_risk": "LOW"
    }
  }
}
```

24.4.2 Ethik-Compliance-Checks in AQL

Query mit ethischen Constraints:

```
// Suche nur ethisch unbedenkliche Dokumente
FOR doc IN documents
  // Standard-Filter
  FILTER doc.content.legal_basis ANY == "BImSchG §5"

  //   ETHIK-FILTER
  FILTER doc.ethics_metadata.bias_audit.status == "PASSED"
  FILTER doc.ethics_metadata.bias_audit.bias_score < 0.3
  FILTER doc.ethics_metadata.usage_restrictions.allow_direct_citation == true

  // Falls historisches Dokument, nur mit Kontext
  FILTER doc.ethics_metadata.provenance.historical_context != "DISCRIMINATORY_ERA"
     OR doc.ethics_metadata.usage_restrictions.require_human_review == true

  LET similarity = COSINE(doc.embedding, @query_vector)
  FILTER similarity > 0.7

  // Rückgabe mit Ethik-Metadaten
  RETURN {
    doc: doc,
    similarity: similarity,
    ethics_status: doc.ethics_metadata.bias_audit.status,
    human_review_required: doc.ethics_metadata.usage_restrictions.require_human_review
  }
```

24.4.3 Automation-Bias-Warnsystem

Echtzeit-Warnung bei verdächtigem Nutzerverhalten:

```cpp
// veritas/automation_bias_detector.cpp
class AutomationBiasDetector {
public:
    void MonitorUserBehavior(const UserSession& session) {
        // Pattern: User akzeptiert KI-Vorschlag ohne Quellenprüfung
        if (session.ai_recommendation_shown &&
            session.sources_viewed.empty() &&
            session.decision_time_seconds < 10) {

            // ⚠ WARNUNG: Potentieller Automation Bias
            SendWarning(session.user_id,
                       "Sie haben eine KI-Empfehlung ohne Quellenprüfung übernommen. "
                       "Bitte validieren Sie die Quellen kritisch.");

            // Audit-Log
            db_->Execute(R"(
                INSERT INTO automation_bias_warnings (user_id, session_id, timestamp)
                VALUES (:user, :session, NOW())
            )", {{"user", session.user_id}, {"session", session.session_id}});

            // Bei Wiederholung: Mandatory Training
            if (GetWarningCount(session.user_id) > 3) {
                RequireAILiteracyTraining(session.user_id);
            }
        }
    }
};
```

24.5 Zusammenfassung und Best Practices

Wichtigste Erkenntnisse:

1. **Ethik ist kein Add-on**: Ethische Überlegungen müssen von Anfang an in die Systemarchitektur integriert werden ("Ethik-by-Design") [9]

2. **ThemisDB garantiert nur technische Konsistenz**: ACID-Transaktionen garantieren nicht inhaltliche oder ethische Korrektheit - das erfordert zusätzliche Governance-Layer

3. **Drei architektonische Risiko-Hotspots**:

4. Covina (Bias in Daten)
5. Clara (Korruption durch fehlerhaftes Feedback)
6. Veritas (Automation Bias bei Nutzern)

7. **Human-in-the-Loop ist essentiell**: Kritische Entscheidungen dürfen niemals vollständig automatisiert werden [9]

Checkliste für ethische KI-Systeme:

- [ ] Interdisziplinäres Ethik-Gremium eingerichtet

- [ ] Bias-Audits für alle Datenquellen implementiert
- [ ] Provenienz-Tracking für Dokumente aktiviert
- [ ] Automation-Bias-Warnsystem deployed
- [ ] Human-in-the-Loop-Policy definiert und durchgesetzt
- [ ] AI-Literacy-Training für alle Nutzer durchgeführt
- [ ] Ethik-Metadaten in allen Base Entities
- [ ] Clara-Feedback-Validierung mit Experten-Review
- [ ] Transparente Unsicherheits-Kennzeichnung in Veritas
- [ ] Regelmäßige Ethik-Audits (mindestens quarterly)

Weiterführende Ressourcen

Dokumentation

- **[9]**: Expertenanalyse: Ethische und moralische Implikationen des KI-Ökosystems VCC
- **Enterprise Security**: `docs/security/RBAC_AUTHORIZATION.md`
- **Compliance**: `docs/compliance/DSGVO_BY_DESIGN.md`

Akademische Referenzen

[1] Barocas, S., Hardt, M., Narayanan, A. (2019). "Fairness and Machine Learning: Limitations and Opportunities". fairmlbook.org.

[2] O'Neil, C. (2016). "Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy". Crown Publishing.

[3] Floridi, L., Cowls, J. (2019). "A Unified Framework of Five Principles for AI in Society". Harvard Data Science Review, 1(1).

[4] Selbst, A., et al. (2019). "Fairness and Abstraction in Sociotechnical Systems". ACM FAT* Conference, 59-68.

[5] Mittelstadt, B., et al. (2016). "The ethics of algorithms: Mapping the debate". Big Data & Society, 3(2).

[6] EU AI Act (2024). "Regulation on Artificial Intelligence". European Parliament.

[7] Goddard, K., et al. (2012). "Automation Bias: A Systematic Review of Frequency, Effect Mediators, and Mitigators". Journal of the American Medical Informatics Association, 19(1), 121-127.

Glossar für dieses Kapitel

Begriff	Definition
Automation Bias	Tendenz, Ergebnissen automatisierter Systeme übermäßig zu vertrauen und menschliches Urteilsvermögen zu vernachlässigen
Bias-Audit	Systematische Prüfung von Daten oder Modellen auf Voreingenommenheit
Echokammer-Effekt	Verstärkung von Mehrheitsmeinungen durch selektives Feedback
Ethik-by-Design	Integration ethischer Überlegungen in die Systemarchitektur von Anfang an
Human-in-the-Loop	Ansatz, bei dem kritische Entscheidungen immer menschliche Kontrolle erfordern
Kaskadieren Integritätskompromittierung	Fehler, der sich durch Feedback-Loops permanent im System verankert
Provenienz	Herkunft und Entstehungsgeschichte von Daten
Verantwortungsdiffusion	Unklare Zuständigkeit bei automatisierten Entscheidungen ("Die KI war's")

Änderungshistorie

Version	Datum	Autor	Änderungen
1.0.0	2025-12-30	ThemisDB Team	Initiale Version basierend auf gimini Ethics-Analyse [9]

Schwierigkeitsgrad: Fortgeschritten

Geschätzte Lesezeit: 45 Minuten

Tags: ethics, ai-governance, bias, automation-bias, human-in-the-loop, verwaltungs-ki

---

Kapitel 26: Kapitel 25: DevOps & Infrastructure as Code

> *"Infrastructure should be versioned, tested, and deployed like code. In ThemisDB, this is standard practice."*

Überblick

Production-grade Datenbanken erfordern automatisierte Bereitstellung, Monitoring, und Failover-Mechanismen. Dieses Kapitel behandelt Infrastructure-as-Code (IaC) mit Terraform, Container-Orchestrierung mit Kubernetes, und GitOps-Workflows.

Was Sie in diesem Kapitel lernen: - Terraform-Module für ThemisDB-Deployment - Kubernetes Helm Charts - GitOps-Workflows mit ArgoCD - Multi-Region Failover - Disaster Recovery Planning - Operational Runbooks

---

25.1 Terraform Infrastructure-as-Code

AWS RDS ThemisDB Cluster

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket         = "themis-terraform-state"
    key            = "production/themisdb/terraform.tfstate"
    region         = "eu-central-1"
    encrypt        = true
    dynamodb_table = "terraform-locks"
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_rds_cluster" "themis" {
  cluster_identifier      = "themis-prod-cluster"
  engine                  = "themisdb"
  engine_version          = "1.3.4"
  database_name           = "themisdb"
  master_username         = "admin"
  master_password         = random_password.db_password.result

  # Hochverfügbarkeit
  availability_zones      = data.aws_availability_zones.available.names
  backup_retention_period = 30
  preferred_backup_window = "03:00-04:00"

  # Security
  db_subnet_group_name            = aws_db_subnet_group.themis.name
  db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.themis.name
  vpc_security_group_ids          = [aws_security_group.themis_db.id]
  storage_encrypted               = true
  kms_key_id                      = aws_kms_key.themis.arn

  # Performance Insights
  performance_insights_enabled    = true
  performance_insights_kms_key_id = aws_kms_key.themis.arn

  # Logging
  enable_cloudwatch_logs_exports = ["error", "general", "slowquery"]

  tags = {
    Environment = "production"
    Service     = "themisdb"
  }
}
```

```
resource "aws_rds_cluster_instance" "themis" {
  count              = 3  # 3-Knoten Cluster
  cluster_identifier = aws_rds_cluster.themis.id
  instance_class     = "db.r6i.2xlarge"
  engine              = aws_rds_cluster.themis.engine
  engine_version      = aws_rds_cluster.themis.engine_version

  monitoring_interval          = 60
  monitoring_role_arn          = aws_iam_role.rds_monitoring.arn
  performance_insights_enabled = true

  tags = {
    Name = "themis-instance-${count.index + 1}"
  }
}

resource "aws_rds_cluster_parameter_group" "themis" {
  name        = "themis-prod-params"
  family      = "themisdb1.3"
  description = "Tuned for high-throughput workloads"

  parameter {
    name  = "max_connections"
    value = "500"
  }

  parameter {
    name  = "shared_buffers_gb"
    value = "16"
  }

  parameter {
    name  = "work_mem_mb"
    value = "256"
  }
}
```

Subnet & Security Groups

```
resource "aws_db_subnet_group" "themis" {
  name       = "themis-subnets"
  subnet_ids = aws_subnet.private[*].id

  tags = {
    Name = "themis-subnet-group"
  }
}

resource "aws_security_group" "themis_db" {
  name        = "themis-db-sg"
  description = "Security group for ThemisDB"
  vpc_id      = aws_vpc.main.id

  ingress {
    from_port       = 8529
    to_port         = 8529
    protocol        = "tcp"
    security_groups = [aws_security_group.themis_app.id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "themis-db-sg"
  }
}

resource "aws_backup_vault" "themis" {
  name        = "themis-backup-vault"
  kms_key_arn = aws_kms_key.themis.arn

  tags = {
    Name = "themis-worm-vault"
  }
}
```

Outputs & Monitoring

```
output "cluster_endpoint" {
  value       = aws_rds_cluster.themis.endpoint
  description = "Cluster write endpoint"
}

output "cluster_reader_endpoint" {
  value       = aws_rds_cluster.themis.reader_endpoint
  description = "Cluster read endpoint (load-balanced)"
}

output "cloudwatch_log_group" {
  value = "/aws/rds/cluster/${aws_rds_cluster.themis.id}"
}

resource "aws_cloudwatch_metric_alarm" "high_cpu" {
  alarm_name          = "themis-high-cpu"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = "2"
  metric_name         = "CPUUtilization"
  namespace           = "AWS/RDS"
  period              = "300"
  statistic           = "Average"
  threshold           = "80"
  alarm_actions       = [aws_sns_topic.alerts.arn]

  dimensions = {
    DBClusterIdentifier = aws_rds_cluster.themis.cluster_identifier
  }
}
```

25.2 Kubernetes Helm Charts

Helm Chart Struktur

```
themis-helm/
├── Chart.yaml
├── values.yaml
├── values-prod.yaml
├── values-staging.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── pvc.yaml
│   ├── configmap.yaml
│   └── statefulset.yaml
└── charts/
    └── prometheus-operator/
```

Deployment Template

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: {{ include "themis.fullname" . }}
  labels:
    {{- include "themis.labels" . | nindent 4 }}
spec:
  serviceName: {{ include "themis.fullname" . }}
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "themis.selectorLabels" . | nindent 6 }}

  template:
    metadata:
      labels:
        {{- include "themis.selectorLabels" . | nindent 8 }}

    spec:
      # Pod Disruption Budget (für Rolling Updates)
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - themis
              topologyKey: kubernetes.io/hostname

      containers:
      - name: themis
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        imagePullPolicy: IfNotPresent

        ports:
        - name: http
          containerPort: 8529
          protocol: TCP
        - name: replication
          containerPort: 8530
          protocol: TCP

        env:
        - name: THEMIS_CLUSTER_NAME
          value: {{ .Values.clusterName }}
        - name: THEMIS_PEER_URLS
          value: "themis-0.themis:8530,themis-1.themis:8530,themis-2.themis:8530"

        # Resource Requests
        resources:
          requests:
            memory: {{ .Values.resources.requests.memory }}
```

```yaml
            cpu: {{ .Values.resources.requests.cpu }}
          limits:
            memory: {{ .Values.resources.limits.memory }}
            cpu: {{ .Values.resources.limits.cpu }}

        # Liveness & Readiness Probes
        livenessProbe:
          httpGet:
            path: /_admin/health
            port: http
          initialDelaySeconds: 30
          periodSeconds: 10

        readinessProbe:
          httpGet:
            path: /_admin/ready
            port: http
          initialDelaySeconds: 10
          periodSeconds: 5

        # Volume Mounts
        volumeMounts:
        - name: data
          mountPath: /data
        - name: config
          mountPath: /etc/themis

      volumes:
      - name: config
        configMap:
          name: {{ include "themis.fullname" . }}

  # Persistent Volume Claim
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: {{ .Values.persistence.size }}
```

Helm Values (Production)

```yaml
replicaCount: 3
clusterName: themis-prod

image:
  repository: themisdb/server
  tag: "1.3.4"
  pullPolicy: IfNotPresent

resources:
  requests:
    memory: 16Gi
    cpu: 4000m
  limits:
    memory: 32Gi
    cpu: 8000m

persistence:
  enabled: true
  size: 500Gi
  storageClass: fast-ssd

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
  hosts:
    - host: themis.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: themis-tls
      hosts:
        - themis.example.com

monitoring:
  enabled: true
  serviceMonitor:
    enabled: true
    interval: 30s
```

25.3 GitOps mit ArgoCD

ArgoCD Application Definition

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: themis-prod
  namespace: argocd
spec:
  project: production

  source:
    repoURL: https://github.com/themisdb/helm-charts
    targetRevision: v1.3.4
    path: themis-helm
    helm:
      releaseName: themis
      values: |
        replicaCount: 3
        image:
          tag: "1.3.4"

  destination:
    server: https://kubernetes.default.svc
    namespace: themis-prod

  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
    retry:
      limit: 5
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
```

Pull Request Preview Environments

```yaml
name: Preview Environment

on:
  pull_request:
    paths:
      - 'helm/**'
      - '.github/workflows/preview.yml'

jobs:
  deploy-preview:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Create Preview Namespace
        run: |
          kubectl create namespace preview-${{ github.event.pull_request.number }}

      - name: Deploy with Helm
        run: |
          helm install themis-preview ./themis-helm \
            -n preview-${{ github.event.pull_request.number }} \
            -f values-preview.yaml \
            --set image.tag=${{ github.head_ref }}

      - name: Comment PR with Preview URL
        uses: actions/github-script@v6
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: '  Preview deployed at `preview-${{ github.event.pull_request.number }}.p
            })
```

25.4 Multi-Region Failover

Active-Active Replikation

```
resource "aws_rds_cluster" "themis_eu" {
  # EU Cluster (Primary)
  cluster_identifier = "themis-eu-primary"
  region             = "eu-central-1"
}

resource "aws_rds_cluster" "themis_us" {
  # US Cluster (Replica mit Failover)
  cluster_identifier       = "themis-us-replica"
  region                   = "us-east-1"
  replication_source_arn   = aws_rds_cluster.themis_eu.arn

  # Automatischer Failover nach 300 Sekunden
  backup_retention_period = 30
}

resource "aws_route53_failover_routing_policy" "themis" {
  name            = "themis.example.com"
  zone_id         = aws_route53_zone.main.zone_id
  type            = "A"
  failover_routing_policy {
    type = "PRIMARY"
  }

  alias {
    name                   = aws_rds_cluster.themis_eu.endpoint
    zone_id                = aws_rds_cluster.themis_eu.hosted_zone_id
    evaluate_target_health = true
  }

  health_check_id = aws_route53_health_check.eu.id
}
```

Application-Level Failover

```python
class ResilientThemisClient:
    def __init__(self, primary_url, secondary_url):
        self.primary_url = primary_url
        self.secondary_url = secondary_url
        self.current_url = primary_url
        self.failure_count = 0
        self.max_failures = 3

    def query(self, aql, params=None):
        try:
            client = themis.Client(self.current_url)
            result = client.query(aql, params)
            self.failure_count = 0  # Reset bei Erfolg
            return result
        except Exception as e:
            self.failure_count += 1

            if self.failure_count >= self.max_failures:
                # Failover zu Secondary
                self.current_url = self.secondary_url
                self.failure_count = 0
                print(f"Failover zu {self.secondary_url}")
                return self.query(aql, params)  # Retry
            raise
```

25.5 Disaster Recovery Planning

Backup & Restore Strategie

```bash
#!/bin/bash

aws backup start-backup-job \
  --backup-vault-name themis-backup-vault \
  --recovery-point-type INCREMENTAL \
  --recovery-point-tags "Frequency=Daily,Retention=7days"

aws backup start-backup-job \
  --backup-vault-name themis-backup-vault \
  --recovery-point-type FULL \
  --recovery-point-tags "Frequency=Weekly,Retention=4weeks"

aws backup start-backup-job \
  --backup-vault-name themis-backup-vault \
  --recovery-point-type FULL \
  --recovery-point-tags "Frequency=Monthly,Retention=7years"
```

RTO & RPO Metriken

Szenario	RTO	RPO	Strategie
Einzelner Node Fehler	<30s	0s	Auto-Failover
Region Ausfalls	<5min	<1min	Cross-Region Replica
Datenbeschädigung	<1h	<1h	Point-in-Time Recovery
Vollständiger Datenverlust	<4h	<24h	S3 Langzeit-Archiv

25.6 Operational Runbooks

Incident Response Playbook

````
## Critical Alert: High Replication Lag

**Severity:** High
**RTO:** 5 minutes
**On-call:** Check #oncall Slack channel

### Diagnostik (1-2 min)
1. `kubectl logs -f themis-2` - Prüfe Replica-Logs
2. `curl http://themis-2:8529/_admin/stats` - Replication lag?
3. `aws cloudwatch get-metric-statistics --metric-name ReplicationLatency`

### Remediation (2-3 min)
```bash
kubectl delete pod themis-2

curl -X POST http://themis-2:8529/_admin/resync
````

Post-Incident

- [ ] Logs analysieren
- [ ] Runbook updaten
- [ ] Post-Mortem in Confluence ```

25.7 Checkliste für Production Readiness

- Terraform IAC für alle Infra-Komponenten
- Helm Charts mit Multi-Environment Support
- ArgoCD für GitOps Deployment
- Automated Backup & Restore Tests
- Multi-Region Failover konfiguriert
- CloudWatch Alarms für kritische Metriken
- PagerDuty/Opsgenie Integration
````

- Runbooks für alle kritischen Szenarien
- Load-Testing vor Production Release
- Security Scanning in CI/CD Pipeline

Kapitel 27: Kapitel 26: Migration & Legacy System Integration

> *"Legacy systems don't disappear. In enterprise environments, the art of seamless migration is what separates successful adoptions from failed projects."*

Überblick

Migration von Legacy-Systemen (PostgreSQL, MongoDB, Neo4j) zu ThemisDB ist eine Kernaufgabe bei der Datenbank-Modernisierung. Dieses Kapitel zeigt pragmatische Strategien für zero-downtime Migrationen mit vollständiger Datenvalidierung.

Was Sie in diesem Kapitel lernen: - Schema-Mappings (SQL → AQL) - Daten-Extraktion & Transformation (ETL) - Live-Replikation während Migration - Daten-Validierung & Reconciliation - Rollback-Strategien - Performance Tuning nach Migration

---

26.1 Schema-Mapping Strategien

PostgreSQL → ThemisDB Mapping

```
-- SQL Table                              → AQL Collection
-- customers TABLE                        → customers collection
-- id INT PRIMARY KEY                     → _id (implicit, or _key)
-- name VARCHAR(255)                      → name: string
-- email VARCHAR(100) UNIQUE              → email: string (with unique index)
-- created_at TIMESTAMP                   → created_at: date
-- FOREIGN KEY orders(customer_id)        → Document-Referenzen

-- Beispiel-Transformation:
FUNCTION transform_postgresql_customer(sql_row) {
  RETURN {
    _key: STRING(sql_row.id),
    name: sql_row.name,
    email: sql_row.email,
    created_at: DATE_ISO8601(sql_row.created_at),

    -- Denormalisierung erlaubt in Multi-Model DB
    order_count: 0,  -- wird später aktualisiert
    total_spent: 0.0,

    -- Migration Metadata
    _migration: {
      source_system: "postgresql",
      source_id: sql_row.id,
      migrated_at: DATE_NOW(),
      validation_status: "pending"
    }
  }
}
```

MongoDB → ThemisDB Mapping

```
// MongoDB Document
{
  _id: ObjectId("507f1f77bcf86cd799439011"),
  title: "Product A",
  attributes: {
    color: "red",
    size: "M",
    warranty_years: 2
  },
  tags: ["electronics", "gadget"],
  reviews: [
    {user: "alice", rating: 5, comment: "Great!"},
    {user: "bob", rating: 4, comment: "Good"}
  ]
}

// Wird zu AQL Collection Dokument:
FOR doc IN mongo_products
  INSERT {
    _key: doc._id.toString(),
    title: doc.title,
    attributes: doc.attributes,  // Geschachtelte Objekte erlaubt
    tags: doc.tags,               // Arrays
    reviews: doc.reviews,         // Array von Objekten

    -- Normalisierungsoptionen:
    -- Option 1: Flach (denormalisiert)
    color: doc.attributes.color,
    size: doc.attributes.size,

    -- Option 2: Graph-Edges für Many-To-Many
    -- reviews werden zu separater Collection + Edges
  } INTO products
```

Neo4j → ThemisDB Graph

```
-- Neo4j                              → ThemisDB
-- (:User {id: 1, name: "Alice"})    → User Document
-- [:FOLLOWS]                        → Graph Edge (Relation)
-- (:Product)                        → Product Document

FUNCTION migrate_neo4j_graph(cypher_result) {
  -- Neo4j Nodes → AQL Documents
  FOR node IN cypher_result.nodes
    INSERT {
      _key: node.id,
      type: node.label,
      properties: node.properties,

      -- Migration Tracking
      source_node_id: node.id,
      source_labels: node.labels
    } INTO graph_nodes

  -- Neo4j Relationships → AQL Graph Edges
  FOR rel IN cypher_result.relationships
    INSERT {
      _from: CONCAT(rel.from_type, "/", rel.from_id),
      _to: CONCAT(rel.to_type, "/", rel.to_id),
      relationship_type: rel.type,
      properties: rel.properties
    } INTO graph_edges

  RETURN {
    nodes_inserted: LENGTH(cypher_result.nodes),
    edges_inserted: LENGTH(cypher_result.relationships)
  }
}
```

26.2 ETL Pipeline Design

Python ETL Framework

```python
import logging
from typing import List, Dict, Any
import themis

class ETLPipeline:
    def __init__(self, source_db, target_db, batch_size=1000):
        self.source = source_db
        self.target = target_db
        self.batch_size = batch_size
        self.logger = logging.getLogger(__name__)

    def extract(self, source_query: str) -> List[Dict]:
        """Extract: Daten aus Legacy-System lesen"""
        self.logger.info(f"Extracting from source: {source_query[:50]}...")
        return list(self.source.execute(source_query))

    def transform(self, rows: List[Dict], transformer_func) -> List[Dict]:
        """Transform: Schema-Mapping & Validierung"""
        self.logger.info(f"Transforming {len(rows)} rows...")
        transformed = []
        errors = []

        for i, row in enumerate(rows):
            try:
                transformed.append(transformer_func(row))
            except Exception as e:
                errors.append({'row_index': i, 'error': str(e), 'data': row})

        if errors:
            self.logger.warning(f"Transformation errors: {len(errors)}")

        return transformed, errors

    def load(self, docs: List[Dict], collection: str) -> Dict:
        """Load: In ThemisDB einfügen"""
        self.logger.info(f"Loading {len(docs)} documents into {collection}...")

        result = {
            'inserted': 0,
            'errors': 0,
            'error_details': []
        }

        # Batch-wise einfügen
        for i in range(0, len(docs), self.batch_size):
            batch = docs[i:i+self.batch_size]

            try:
                aql = f"""
                  FOR doc IN @docs
                    INSERT doc INTO {collection}
                    RETURN NEW._id
                """
                ids = self.target.query(aql, {'docs': batch})
```

```
                result['inserted'] += len(ids)
            except Exception as e:
                self.logger.error(f"Batch insert error: {e}")
                result['errors'] += len(batch)
                result['error_details'].append({
                    'batch_start': i,
                    'batch_size': len(batch),
                    'error': str(e)
                })

        return result

    def run(self, source_query: str, transformer_func, target_collection: str) -> Dict:
        """Orchestriere E-T-L Pipeline"""
        try:
            rows = self.extract(source_query)
            transformed, errors = self.transform(rows, transformer_func)
            result = self.load(transformed, target_collection)

            result['transformation_errors'] = len(errors)
            result['total_processed'] = len(rows)
            result['success_rate'] = result['inserted'] / len(rows) * 100

            return result
        except Exception as e:
            self.logger.error(f"Pipeline failed: {e}")
            raise

from adapters.postgresql import PostgreSQLAdapter
from adapters.themis import ThemisAdapter

source_db = PostgreSQLAdapter(connection_string="postgresql://...")
target_db = ThemisAdapter(url="http://localhost:8529")

etl = ETLPipeline(source_db, target_db, batch_size=5000)

def transform_customer(row):
    return {
        '_key': str(row['id']),
        'name': row['name'],
        'email': row['email'],
        'created_at': row['created_at'].isoformat(),
        'migrated_at': datetime.now().isoformat()
    }

result = etl.run(
    source_query="SELECT * FROM customers WHERE status = 'active'",
    transformer_func=transform_customer,
    target_collection="customers"
)

print(f"Migration: {result['inserted']} inserted, {result['errors']} errors, Success: {result
```

26.3 Live Replikation während Migration

CDC (Change Data Capture) Ansatz

```python
import psycopg2
from psycopg2.extras import LogicalReplicationConnection
import themis

class PostgreSQLCDCSync:
    def __init__(self, pg_conn_str, themis_url):
        self.pg_conn = psycopg2.connect(pg_conn_str,
                                        connection_factory=LogicalReplicationConnection)
        self.themis = themis.Client(themis_url)

    def replicate_changes(self, slot_name="themis_slot"):
        """Lese Changes aus PostgreSQL WAL und appliziere sie"""

        cursor = self.pg_conn.cursor()
        cursor.start_replication(slot_name=slot_name)

        print(f"Replication started, listening for changes...")

        for message in cursor.consume_dstream():
            if message.payload:
                # Parse Change Event
                change = self._parse_wal_record(message.payload)

                # Apply in ThemisDB
                self._apply_change(change)

                # Acknowledge
                message.cursor.send_feedback(write_lsn=message.write_lsn)

    def _parse_wal_record(self, payload: bytes):
        """Parse PostgreSQL WAL Format"""
        # Vereinfacht: In Production wäre dies komplexer
        import json
        return json.loads(payload.decode('utf-8'))

    def _apply_change(self, change: Dict):
        """Appliziere INSERT/UPDATE/DELETE in ThemisDB"""

        if change['action'] == 'INSERT':
            aql = f"""
              INSERT @doc INTO {change['table']}
              RETURN NEW._id
            """
            self.themis.query(aql, {'doc': change['data']})

        elif change['action'] == 'UPDATE':
            aql = f"""
              UPDATE '{change['table']}/{change['id']}' WITH @doc
              RETURN NEW
            """
            self.themis.query(aql, {'doc': change['data']})

        elif change['action'] == 'DELETE':
            aql = f"""
```

```
              REMOVE '{change['table']}/{change['id']}'
              RETURN OLD
            """
            self.themis.query(aql)

        print(f"Applied {change['action']} to {change['table']}/{change['id']}")

sync = PostgreSQLCDCSync(
    pg_conn_str="postgresql://user:password@localhost/mydb",
    themis_url="http://localhost:8529"
)

import threading
replication_thread = threading.Thread(target=sync.replicate_changes, daemon=True)
replication_thread.start()

print("CDC Replication running in background...")
```

26.4 Daten-Validierung & Reconciliation

Reconciliation Framework

```
class DataReconciliation:
    def __init__(self, source_db, target_db):
        self.source = source_db
        self.target = target_db

    def validate_counts(self, source_query: str, target_collection: str) -> Dict:
        """Vergleiche Zeilen-Anzahl Source vs Target"""

        source_count = self.source.execute(f"SELECT COUNT(*) FROM ({source_query})")[0]['coun

        target_count = self.target.query(f"""
          RETURN LENGTH(
            FOR doc IN {target_collection}
            RETURN doc
          )
        """)[0]

        match = source_count == target_count
        return {
            'collection': target_collection,
            'source_count': source_count,
            'target_count': target_count,
            'match': match,
            'difference': abs(source_count - target_count)
        }

    def validate_checksums(self, table: str, key_column: str) -> Dict:
        """Vergleiche Row-Level Checksums"""

        # Source Checksums
        source_checksums = self.source.execute(f"""
          SELECT {key_column}, MD5(ROW(*)) as checksum
          FROM {table}
          ORDER BY {key_column}
        """)

        # Target Checksums (via AQL)
        target_checksums = self.target.query(f"""
          FOR doc IN {table.lower()}
            SORT doc._key
            RETURN {{
              key: doc._key,
              checksum: MD5(JSON_STRINGIFY(doc))
            }}
        """)

        # Reconcile
        mismatches = []
        for src in source_checksums:
            tgt = next((t for t in target_checksums
                       if t['key'] == src[key_column]), None)

            if not tgt or tgt['checksum'] != src['checksum']:
                mismatches.append({
```

```
                    'id': src[key_column],
                    'source_checksum': src['checksum'],
                    'target_checksum': tgt['checksum'] if tgt else None
                })

        return {
            'table': table,
            'total_rows': len(source_checksums),
            'mismatches': len(mismatches),
            'match_rate': (len(source_checksums) - len(mismatches)) / len(source_checksums) *
            'mismatched_ids': mismatches[:10]  # First 10
        }

    def run_full_validation(self) -> Dict:
        """Komplette Post-Migration Validation"""

        validations = [
            self.validate_counts("SELECT * FROM customers", "customers"),
            self.validate_counts("SELECT * FROM orders", "orders"),
            self.validate_checksums("customers", "id"),
            self.validate_checksums("orders", "id")
        ]

        all_pass = all(v.get('match', v.get('match_rate', 0) >= 99.9) for v in validations)

        return {
            'all_pass': all_pass,
            'validations': validations,
            'timestamp': datetime.now().isoformat()
        }

recon = DataReconciliation(source_db, target_db)
result = recon.run_full_validation()

if result['all_pass']:
    print("  Alle Validierungen erfolgreich!")
else:
    print("× Validierungsfehler gefunden:")
    for v in result['validations']:
        if not v.get('match'):
            print(f"  - {v}")
```

26.5 Rollback-Strategien

Blue-Green Deployment Pattern

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: db-service
spec:
  selector:
    version: blue  # Initial zeigt auf Blue (PostgreSQL)
  ports:
    - port: 5432

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: themis-green
spec:
  replicas: 3
  template:
    metadata:
      labels:
        version: green
    spec:
      containers:
      - name: themis
        image: themisdb/server:1.3.4
        ports:
        - containerPort: 8529

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: migration-config
data:
  # Phase 1: 5% Traffic zu Green
  traffic_green_percent: "5"

  # Phase 2: 25% Traffic nach X Stunden
  phase2_hours: "4"
  phase2_traffic: "25"

  # Phase 3: 50% Traffic nach X Stunden
  phase3_hours: "12"
  phase3_traffic: "50"

  # Phase 4: 100% Traffic (oder Rollback zu Blue)
  phase4_hours: "24"

  # Rollback bei Critical Errors
  rollback_on_error_rate: "5"  # 5% error rate triggers rollback
```

Schneller Rollback

```bash
#!/bin/bash

echo "  Initiating immediate rollback to Blue (PostgreSQL)..."

kubectl patch service db-service -p '{"spec":{"selector":{"version":"blue"}}}'
echo "✓ Traffic router updated to Blue"

kubectl set env deployment/themis-green READ_ONLY=true
echo "✓ Green set to read-only mode"

until kubectl exec $(kubectl get pods -l version=blue -o name | head -1) \
  -- pg_isready -h localhost; do
  echo "Waiting for Blue to be ready..."
  sleep 5
done
echo "✓ Blue is healthy"

curl -X POST https://hooks.slack.com/services/YOUR/WEBHOOK \
  -d '{"text":"  Migration rollback initiated. Reverting to PostgreSQL."}'

echo "  Rollback complete. System running on PostgreSQL (Blue)"
```

26.6 Performance Tuning nach Migration

Indexe erstellen

```sql
-- Häufig verwendete Filter
CREATE INDEX idx_customers_email
  ON customers (email)

CREATE INDEX idx_orders_customer_date
  ON orders (customer_id, created_at)

-- Geo-Queries optimieren
CREATE INDEX idx_locations_geo
  ON locations (location)
  TYPE GEO

-- Fulltext-Suche
CREATE INDEX idx_products_fulltext
  ON products (title, description)
  TYPE FULLTEXT
```

Query Performance Analysis

```
-- Analysiere langsame Queries
FOR slow IN slow_queries
  FILTER slow.duration_ms > 100
  COLLECT coll = slow.collection
  AGGREGATE count = COUNT(1), avg_duration = AVG(slow.duration_ms)
  SORT avg_duration DESC
  RETURN {
    collection: coll,
    slow_query_count: count,
    avg_duration_ms: avg_duration
  }

-- Empfehlungen generieren
-- Wenn avg_duration > 500ms: Index empfohlen
```

26.7 Migration Checklist

- Schema-Mapping definiert & dokumentiert
- ETL Pipeline entwickelt & getestet
- Daten-Validierungstests geschrieben
- CDC/Live-Replikation konfiguriert
- Blue-Green Deployment vorbereitet
- Rollback-Prozedur dokumentiert
- Performance-Baseline gemessen
- Team-Training durchgeführt
- Cutover-Fenster geplant
- Post-Migration Support geplant

Typischer Timeline: - **T-4 Wochen:** Schema-Design & ETL-Entwicklung - **T-2 Wochen:** Full-Load Test & Performance-Tuning - **T-1 Woche:** CDC Replikation starten, Validierung - **T-0 Tag:** Cutover, Monitoring intensivieren - **T+24h:** Green vollständig Live, Blue Redundanz - **T+1 Woche:** Blue abschalten

---

Kapitel 28: Kapitel 27: Troubleshooting & Problem Resolution

> *"The difference between a good engineer and a great engineer is how quickly they can diagnose and fix production issues."*

Überblick

Production-Probleme erfordern systematische Diagnose und schnelle Remediation. Dieses Kapitel bietet konkrete Lösungen für häufige ThemisDB-Probleme mit reproduzierbaren Debugging-Workflows.

Was Sie in diesem Kapitel lernen: - Systematische Problemanalyse (5-Why-Methode) - Performance-Probleme diagnostizieren - Replikations-Lag beheben - Out-of-Memory Errors lösen - Deadlock-Detection & Resolution - Korrupte Indizes reparieren - Network Partition Recovery

---

27.1 Systematische Problem-Diagnose

5-Why Root Cause Analysis

```
Problem: Query dauert 30 Sekunden statt 100ms

Why 1: Warum ist die Query langsam?
  → Full Collection Scan statt Index-Nutzung

Why 2: Warum wird kein Index genutzt?
  → Filter auf nicht-indexiertes Feld `metadata.custom_field`

Why 3: Warum ist das Feld nicht indexiert?
  → Index wurde nach Schema-Änderung nicht aktualisiert

Why 4: Warum wurde Index nicht aktualisiert?
  → Deployment-Prozess hat Migrations-Step übersprungen

Why 5: Warum wurde Step übersprungen?
  → CI/CD Pipeline hatte keine Pre-Deploy Validation

Root Cause: Fehlende Pre-Deploy Index-Validation
Solution: Pre-Deploy Hook für Schema-Validation hinzufügen
```

Diagnostic Checklist

```
#!/bin/bash

echo "=== ThemisDB Health Check ==="

echo "1. Cluster Status:"
curl -s http://localhost:8529/_admin/cluster/health | jq .

echo "2. Memory Usage:"
curl -s http://localhost:8529/_admin/statistics | jq '.memory'

echo "3. Slow Queries:"
curl -s http://localhost:8529/_admin/slow-queries?threshold=1000

echo "4. Replication Lag:"
curl -s http://localhost:8529/_admin/replication/lag

echo "5. Active Connections:"
netstat -an | grep :8529 | wc -l

echo "6. Disk Space:"
df -h /data/themis

echo "7. Recent Errors:"
journalctl -u themis -n 50 --no-pager | grep ERROR
```

27.2 Performance-Probleme

Problem: Langsame Queries

Symptom: Query dauert >5 Sekunden
Diagnose:

```
-- Query mit EXPLAIN analysieren
EXPLAIN FOR u IN users
  FILTER u.email == 'alice@example.com'
  RETURN u

-- Ausgabe prüfen:
{
  "plan": {
    "nodes": [
      {"type": "SingletonNode"},
      {"type": "EnumerateCollectionNode", "collection": "users"},  // × SCHLECHT: Full Scan
      {"type": "FilterNode"},
      {"type": "ReturnNode"}
    ]
  },
  "stats": {
    "executionTime": 5.234,
    "scannedFull": 1000000  // × 1M Dokumente gescannt
  }
}
```

Solution:

```
-- Index erstellen
CREATE INDEX idx_users_email ON users (email)

-- Erneut EXPLAIN
EXPLAIN FOR u IN users
  FILTER u.email == 'alice@example.com'
  RETURN u

-- Jetzt mit Index:
{
  "plan": {
    "nodes": [
      {"type": "SingletonNode"},
      {"type": "IndexNode", "index": "idx_users_email"},  //   GUT: Index genutzt
      {"type": "ReturnNode"}
    ]
  },
  "stats": {
    "executionTime": 0.023,  //   200x schneller
    "scannedIndex": 1
  }
}
```

Problem: High CPU Usage

Symptom: CPU >90% dauerhaft

Diagnose:

```
curl -s http://localhost:8529/_admin/query-stats \
  | jq '.queries | sort_by(.cpu_time) | reverse | .[0:10]'

[
  {
    "query": "FOR doc IN large_collection RETURN doc",
    "cpu_time": 45.2,
    "count": 120,
    "avg_duration_ms": 8500
  }
]
```

Solution:

```
-- Option 1: Query optimieren (Projection)
FOR doc IN large_collection
  RETURN {id: doc._id, name: doc.name}  -- Nur benötigte Felder

-- Option 2: Limit hinzufügen
FOR doc IN large_collection
  LIMIT 100
  RETURN doc

-- Option 3: Background-Job für Batch-Processing
FOR doc IN large_collection
  FILTER doc.needs_processing == true
  UPDATE doc WITH {needs_processing: false, processed_at: DATE_NOW()}
  OPTIONS {waitForSync: false}  -- Async I/O
```

27.3 Replikations-Probleme

Problem: Replication Lag

Symptom: Replica ist 5 Minuten hinter Primary
Diagnose:

```
curl http://localhost:8529/_admin/replication/status

{
  "primary": "node-1",
  "replica": "node-2",
  "lag_seconds": 300,
  "last_applied_timestamp": "2025-12-31T22:55:00Z",
  "pending_operations": 15000
}
```

Root Causes & Solutions:

1. Netzwerk-Latenz:

```
ping node-2

terraform apply -var="replica_region=eu-central-1"
```

2. Replica überlastet:

```
ssh node-2 "top -b -n 1 | head -20"

kubectl scale statefulset themis-replica --replicas=0
kubectl set resources statefulset themis-replica \
  --limits=cpu=8,memory=32Gi
kubectl scale statefulset themis-replica --replicas=1
```

3. Zu viele Schreibvorgänge:

```
-- Schreibrate reduzieren mit Batching
LET batch = @documents  -- Array von 1000 Docs
FOR doc IN batch
  INSERT doc INTO collection
  OPTIONS {waitForSync: false}  -- Async für höheren Throughput
```

27.4 Memory-Probleme

Problem: Out of Memory (OOM)

Symptom: Server crashed mit OOM

Diagnose:

```
curl http://localhost:8529/_admin/statistics | jq '.memory'

{
  "resident_set_size_mb": 15800,  # Actual RAM usage
  "virtual_size_mb": 18200,
  "heap_used_mb": 14500,
  "heap_limit_mb": 16000,  # × 90% ausgelastet!
  "buffer_cache_mb": 1200
}
```

Solutions:

1. Memory Limit erhöhen:

```
resources:
  limits:
    memory: 32Gi  # Von 16Gi auf 32Gi
  requests:
    memory: 24Gi
```

2. Query Memory Leaks finden:

```
-- Große Result Sets vermeiden
FOR doc IN huge_collection
  LIMIT 1000000  -- × 1M Docs in Memory!
  RETURN doc

-- Besser: Streaming mit Cursor
FOR doc IN huge_collection
  RETURN doc
  OPTIONS {stream: true, batchSize: 1000}
```

3. Buffer Cache tunen:

```
buffer_cache_size_mb=512  # Von 2048 auf 512
```

27.5 Deadlock-Probleme

Problem: Deadlock Detected

Symptom: Transaction failed mit "Deadlock detected"
Diagnose:

```
-- Deadlock-Log abrufen
FOR dl IN deadlock_log
  SORT dl.timestamp DESC
  LIMIT 1
  RETURN dl

{
  "timestamp": "2025-12-31T23:00:00Z",
  "transactions": [
    {
      "tx_id": "tx-1234",
      "locked_collections": ["orders", "inventory"],
      "waiting_for": "inventory/item-5"
    },
    {
      "tx_id": "tx-5678",
      "locked_collections": ["inventory", "orders"],  // × Umgekehrte Reihenfolge!
      "waiting_for": "orders/order-10"
    }
  ]
}
```

Solution: Lock Ordering

```
-- × FALSCH: Inkonsistente Lock-Reihenfolge
// Transaction 1
UPDATE "orders/o1" ...
UPDATE "inventory/i1" ...

// Transaction 2
UPDATE "inventory/i1" ...  // Deadlock!
UPDATE "orders/o1" ...

--   RICHTIG: Konsistente Reihenfolge (alphabetisch)
// Beide Transactions
UPDATE "inventory/i1" ...  // Immer zuerst inventory
UPDATE "orders/o1" ...     // Dann orders
```

Lock Timeout setzen:

```
FOR doc IN collection
  UPDATE doc WITH {...}
  OPTIONS {lockTimeout: 5000}  -- 5s statt default 30s
```

27.6 Index-Probleme

Problem: Korrupter Index

Symptom: Query returned wrong results
Diagnose:

```
curl http://localhost:8529/_admin/index/check/users/idx_email

{
  "index": "idx_email",
  "status": "corrupted",
  "missing_entries": 42,
  "extra_entries": 5
}
```

Solution:

```
-- Index neu erstellen
DROP INDEX users/idx_email
CREATE INDEX idx_email ON users (email)

-- Verify
FOR u IN users
  FILTER u.email == 'test@example.com'
  RETURN u
```

Problem: Index zu groß

Symptom: Index verbraucht 50 GB

Diagnose:

```
curl http://localhost:8529/_admin/index/stats | jq '.indexes[] | select(.size_mb > 10000)'

{
  "name": "idx_fulltext_articles",
  "size_mb": 52000,  # 52 GB!
  "document_count": 10000000
}
```

Solution:

```
-- Option 1: Sparse Index (nur nicht-NULL Werte)
CREATE INDEX idx_email_sparse ON users (email)
  OPTIONS {sparse: true}

-- Option 2: Partial Index (nur aktive User)
CREATE INDEX idx_active_users ON users (email)
  WHERE status == 'active'

-- Option 3: Archivierung alter Daten
FOR doc IN articles
  FILTER doc.created_at < DATE_SUBTRACT(DATE_NOW(), 2, 'years')
  REMOVE doc IN articles
  INSERT doc INTO articles_archive
```

27.7 Network Partition Recovery

Problem: Split-Brain nach Partition

Symptom: Zwei separate Cluster nach Netzwerk-Trennung

Diagnose:

```
curl http://node-1:8529/_admin/cluster/status
{"nodes": ["node-1", "node-2"], "quorum": true}

curl http://node-3:8529/_admin/cluster/status
{"nodes": ["node-3"], "quorum": false}  # × Lost quorum
```

Recovery:

```
curl -X POST http://node-3:8529/_admin/cluster/rejoin \
  -d '{"primary": "http://node-1:8529"}'

watch -n 5 'curl -s http://node-3:8529/_admin/replication/lag'

curl http://node-1:8529/_admin/cluster/status
{"nodes": ["node-1", "node-2", "node-3"], "quorum": true}  #
```

27.8 Backup & Restore Issues

Problem: Backup schlägt fehl

Symptom: Backup job timeout after 2 hours
Diagnose:

```
journalctl -u themis-backup -f

df -h /backup
```

Solutions:

```
find /backup -name "*.backup" -mtime +30 -delete

themis-backup --mode=incremental --since=2025-12-30

themis-backup --compression=zstd --compression-level=19
```

27.9 Common Error Messages

Error: "Collection not found"

```
ERROR: Collection 'user' not found
```

Solution:

```
-- Typo: 'user' statt 'users'
FOR u IN users  --   Richtig
  RETURN u
```

Error: "Index hint invalid"

```
ERROR: Index hint 'idx_name' does not exist
```

Solution:

```
-- Index existiert nicht mehr → neu erstellen oder Hint entfernen
CREATE INDEX idx_name ON users (name)

-- Oder Query ohne Hint:
FOR u IN users
  FILTER u.name == 'Alice'
  RETURN u
```

Error: "Transaction timeout"

```
ERROR: Transaction timeout after 30000ms
```

Solution:

```
-- Transaction zu lang → in kleinere Batches aufteilen
LET batch_size = 1000
FOR doc IN large_update
  LIMIT @offset, batch_size
  UPDATE doc IN collection
```

27.10 Troubleshooting Playbook

Problem	Symptom	Erste Schritte	Eskalation
Slow Query	>5s Response	EXPLAIN, add Index	Query Rewrite
High CPU	>90% Usage	Query Stats, Optimize	Scale up
Replication Lag	>60s Lag	Network Check, Batch Size	Add Replica
OOM	Crash/Restart	Memory Stats, Increase Limit	Optimize Queries
Deadlock	Transaction Fails	Lock Ordering	Reduce Concurrency
Corrupt Index	Wrong Results	Rebuild Index	Restore Backup
Split-Brain	Quorum Lost	Network Repair, Rejoin	Manual Failover

Best Practices: - Immer EXPLAIN vor Production-Deployment - Monitoring-Alerts für Lag >30s, CPU >80%, Memory >85% - Regelmäßige Index-Maintenance (weekly VACUUM/ANALYZE) - Backup-Tests (monatlich Restore-Drill) - Runbooks für alle kritischen Fehler - Post-Mortems nach jedem Incident

---

Kapitel 29: Kapitel 28: AQL Vollständige Referenz

> *"Eine Abfragesprache ist nur so gut wie ihre Dokumentation."*

Überblick

Dieses Kapitel ist die komplette Referenz für AQL (Adaptive Query Language), die einheitliche Abfragesprache von ThemisDB. Sie deckt alle 479 Sprachelemente ab: 119 Keywords und 360 eingebaute Funktionen.

Was Sie in diesem Kapitel finden: - Vollständiger Sprachumfang (v1.3.1) - Kategorisierte Funktionsreferenz - Praktische Beispiele für jeden Bereich - Performance-Hinweise und Best Practices

Voraussetzungen: Kapitel 5-9 (Datenmodelle), Grundverständnis von SQL.

28.1 Sprachumfang-Überblick

Executive Summary

Aspekt	v1.3.0	v1.3.1	Änderung
Reservierte Keywords	72	**119**	**+47 (+65%)**
Eingebaute Funktionen	~360	~360	0
Gesamt-Sprachumfang	432	**479**	**+47 (+11%)**

Wichtig: v1.3.1 erweitert Sprachkonstrukte (TYPE, FUNCTION, CLASS) - keine neuen Funktionen.

28.2 Reservierte Keywords (119)

28.2.1 Core Language (8)

```
FOR, IN, LET, FILTER, COLLECT, SORT, LIMIT, RETURN
```

Beispiel:

```
FOR user IN users
  FILTER user.age >= 18
  LET email_domain = SPLIT(user.email, "@")[1]
  COLLECT domain = email_domain WITH COUNT INTO count
  SORT count DESC
  LIMIT 10
  RETURN { domain, count }
```

28.2.2 Logical Operators (3)

```
AND, OR, NOT
```

Beispiel:

```
FOR product IN products
  FILTER product.price >= 10 AND product.price <= 100
  FILTER NOT product.discontinued
  FILTER product.category == "Electronics" OR product.category == "Books"
  RETURN product
```

28.2.3 Aggregation Functions (8)

```
COUNT, SUM, AVG, MIN, MAX, VARIANCE, STDDEV, UNIQUE
```

Beispiel:

```
FOR order IN orders
  COLLECT year = DATE_YEAR(order.created_at)
  AGGREGATE
    total = SUM(order.amount),
    avg_order = AVG(order.amount),
    min_order = MIN(order.amount),
    max_order = MAX(order.amount),
    order_count = COUNT(1)
  RETURN { year, total, avg_order, min_order, max_order, order_count }
```

28.2.4 Graph Traversal (4)

```
OUTBOUND, INBOUND, ANY, SHORTEST_PATH
```

Beispiel:

```
-- Finde alle Freunde von Alice (2 Hops)
FOR v, e, p IN 1..2 OUTBOUND "users/alice" friends
  RETURN { friend: v.name, path_length: LENGTH(p.edges) }

-- Kürzester Pfad von Alice zu Bob
FOR v, e IN OUTBOUND SHORTEST_PATH "users/alice" TO "users/bob" follows
  RETURN { user: v.name, action: e.type }
```

28.2.5 DDL (Data Definition Language) (5)

```
CREATE, DROP, COLLECTION, INDEX, VIEW
```

Beispiele:

```
-- Collection erstellen
CREATE COLLECTION products

-- Index erstellen
CREATE INDEX products_price ON products(price) TYPE SKIPLIST

-- View erstellen
CREATE VIEW active_users AS
  FOR u IN users
    FILTER u.status == "active"
    RETURN u

-- Löschen
DROP INDEX products_price
DROP COLLECTION products
```

28.2.6 DML (Data Manipulation Language) (6)

```
INSERT, UPDATE, REPLACE, REMOVE, UPSERT, INTO, WITH
```

Beispiele:

```
-- Insert
INSERT { name: "Alice", age: 30 } INTO users

-- Update
UPDATE { _key: "alice" } WITH { age: 31 } IN users

-- Upsert (Insert if not exists)
UPSERT { email: "alice@example.com" }
  INSERT { name: "Alice", email: "alice@example.com", age: 30 }
  UPDATE { last_login: DATE_NOW() }
  IN users

-- Remove
REMOVE { _key: "alice" } IN users

-- Replace
REPLACE { _key: "alice", name: "Alice Smith", age: 31 } IN users
```

28.2.7 LLM Operations (13)

```
LLM, INFER, RAG, EMBED, MODEL, LORA, STATS, CACHE,
LOAD, UNLOAD, LIST, INGEST, BLOB
```

Beispiele:

```
-- RAG-Query
FOR doc IN documents
  LET embedding = EMBED(doc.content, { model: "text-embedding-3-small" })
  LET neighbors = KNN_SEARCH(embeddings, embedding, 5)
  LET context = CONCAT_SEPARATOR("\n", neighbors[*].content)
  LET answer = LLM("Answer based on context", { context, model: "gpt-4" })
  RETURN answer

-- Ingest Content
INGEST BLOB "data/documents.pdf"
  INTO content_store
  WITH { chunk_size: 512, overlap: 50 }
```

28.2.8 Index Types (7)

```
HASH, SKIPLIST, FULLTEXT, GEO, PERSISTENT, TTL, VECTOR
```

Beispiele:

```
-- Hash Index (Equality)
CREATE INDEX users_email ON users(email) TYPE HASH

-- Skiplist (Range Queries)
CREATE INDEX products_price ON products(price) TYPE SKIPLIST

-- Fulltext Index
CREATE INDEX articles_body ON articles(body) TYPE FULLTEXT

-- Geo Index
CREATE INDEX locations_coords ON locations(lat, lng) TYPE GEO

-- TTL Index (Auto-Expiry)
CREATE INDEX sessions_ttl ON sessions(expires_at) TYPE TTL

-- Vector Index (ANN Search)
CREATE INDEX embeddings_vector ON embeddings(vector)
  TYPE VECTOR
  OPTIONS { metric: "cosine", dimensions: 768 }
```

28.3 Date/Time Functions (45)

28.3.1 Aktuelle Zeit/Datum (11)

Wichtigste Funktionen: - `DATE_NOW()` - Aktueller ISO-Timestamp - `CURRENT_DATE()` / `CURRENT_TIME()` - Nur Datum/Zeit - `UNIX_TIMESTAMP()` - Unix Epoch

```
RETURN {
  now: DATE_NOW(),
  unix: UNIX_TIMESTAMP()
}
-- → {"now": "2025-01-15T10:30:45Z", "unix": 1736936445}
```

28.3.2 Interval Functions (8)

Interval-Arithmetik: `DAYS(n)`, `HOURS(n)`, `WEEKS(n)` etc. für Zeitberechnungen.

```
-- Letzte 30 Tage
FILTER order.created_at >= DATE_NOW() - DAYS(30)

-- Fälligkeitsdatum berechnen
LET due = invoice.created_at + DAYS(14)
```

28.3.3 Komponenten-Extraktion (11)

Komponenten extrahieren: `DATE_YEAR()`, `DATE_MONTH()`, `DATE_DAY()`, `DATE_HOUR()`, etc.

```
-- Gruppierung nach Monat
COLLECT month = DATE_MONTH(sale.date)
AGGREGATE total = SUM(sale.amount)
```

28.3.4 Workday Functions (6)

```
WORKDAYS(start, end, calendar)           -- Arbeitstage zwischen zwei Daten
WORKDAYS_ADD(date, days, calendar)       -- Addiere Arbeitstage
IS_WEEKEND(date)                         -- true wenn Samstag/Sonntag
IS_WORKDAY(date, calendar)               -- true wenn Arbeitstag
HOLIDAYS(country, year)                  -- Liste der Feiertage
HOLIDAYS_BETWEEN(start, end, country)    -- Feiertage in Zeitraum
```

Beispiele:

```
-- Berechne SLA-Frist (5 Arbeitstage)
FOR ticket IN support_tickets
  RETURN {
    ticket_id: ticket._key,
    created: ticket.created_at,
    sla_deadline: WORKDAYS_ADD(ticket.created_at, 5, "DE")
  }

-- Finde Tickets, die am Wochenende erstellt wurden
FOR ticket IN support_tickets
  FILTER IS_WEEKEND(ticket.created_at)
  RETURN ticket
```

28.4 String Functions (20)

```
CONCAT(str1, str2, ...)              -- Strings zusammenfügen
CONCAT_SEPARATOR(sep, str1, ...)     -- Mit Trennzeichen
SUBSTRING(str, start, length)        -- Teilstring
UPPER(str)                           -- Großbuchstaben
LOWER(str)                           -- Kleinbuchstaben
TRIM(str)                            -- Whitespace entfernen
LTRIM(str)                           -- Links trimmen
RTRIM(str)                           -- Rechts trimmen
LENGTH(str)                          -- Länge
REVERSE(str)                         -- Umkehren
REPLACE(str, search, replace)        -- Ersetzen
SPLIT(str, separator)                -- In Array aufteilen
JOIN(array, separator)               -- Array zu String
STARTS_WITH(str, prefix)             -- Startet mit
ENDS_WITH(str, suffix)               -- Endet mit
CONTAINS(str, substring)             -- Enthält
FIND(str, substring)                 -- Position finden
REGEX_TEST(str, pattern)             -- Regex-Match?
REGEX_MATCHES(str, pattern)          -- Alle Matches
REGEX_REPLACE(str, pattern, repl)    -- Regex-Ersetzung
```

Beispiele:

```
-- Email-Domain extrahieren
FOR user IN users
  LET domain = SPLIT(user.email, "@")[1]
  RETURN { user: user.name, domain }

-- Case-insensitive Suche
FOR article IN articles
  FILTER CONTAINS(LOWER(article.title), LOWER("ThemisDB"))
  RETURN article

-- Email-Validierung mit Regex
FOR user IN users
  FILTER REGEX_TEST(user.email, "^[^@]+@[^@]+\\.[^@]+$")
  RETURN user
```

28.5 Math Functions (30)

```
-- Basic Math
ABS(x)            -- Betrag
CEIL(x)           -- Aufrunden
FLOOR(x)          -- Abrunden
ROUND(x, d)       -- Runden auf d Dezimalstellen
SQRT(x)           -- Quadratwurzel
POW(x, y)         -- x hoch y
EXP(x)            -- e^x
LOG(x)            -- Natürlicher Logarithmus
LOG10(x)          -- Logarithmus zur Basis 10
LN(x)             -- Alias für LOG

-- Trigonometric
SIN(x)            -- Sinus
COS(x)            -- Kosinus
TAN(x)            -- Tangens
ASIN(x)           -- Arkussinus
ACOS(x)           -- Arkuskosinus
ATAN(x)           -- Arkustangens
ATAN2(y, x)       -- Arkustangens mit Vorzeichen
SINH(x)           -- Hyperbelsinus
COSH(x)           -- Hyperbelkosinus
TANH(x)           -- Hyperbeltangens

-- Utilities
MIN(a, b, ...)    -- Minimum
MAX(a, b, ...)    -- Maximum
RAND()            -- Zufallszahl [0, 1)
RAND_INT(a, b)    -- Ganzzahl zwischen a und b
PI()              -- π
E()               -- e
SIGN(x)           -- Vorzeichen (-1, 0, 1)
TRUNC(x)          -- Ganzzahlteil
MOD(x, y)         -- Modulo
QUOTIENT(x, y)    -- Ganzzahlige Division
```

Beispiele:

```
-- Berechne Entfernung mit Pythagoras
FOR point IN points
  LET distance = SQRT(POW(point.x, 2) + POW(point.y, 2))
  RETURN { point: point._key, distance }

-- Zufälliges Sampling (10%)
FOR doc IN documents
  FILTER RAND() < 0.1
  RETURN doc

-- Preisberechnung mit Rundung
FOR product IN products
  LET price_with_tax = ROUND(product.price * 1.19, 2)
  RETURN { product: product.name, price_with_tax }
```

28.6 Array Functions (20)

```
PUSH(array, value)            -- Element anhängen
POP(array)                    -- Letztes Element entfernen
SHIFT(array)                  -- Erstes Element entfernen
UNSHIFT(array, value)         -- Element vorne einfügen
APPEND(array, value)          -- Alias für PUSH
PREPEND(array, value)         -- Alias für UNSHIFT
FIRST(array)                  -- Erstes Element
LAST(array)                   -- Letztes Element
NTH(array, n)                 -- n-tes Element
SLICE(array, start, end)      -- Teilarray
FLATTEN(array, depth)         -- Array flach machen
UNIQUE(array)                 -- Duplikate entfernen
UNION(array1, array2)         -- Vereinigung
INTERSECTION(array1, array2) -- Schnittmenge
DIFFERENCE(array1, array2)    -- Differenz
MAP(array, fn)                -- Transform
FILTER(array, fn)             -- Filtern
REDUCE(array, fn, init)       -- Reduzieren
SORT_ARRAY(array)             -- Sortieren
REVERSE_ARRAY(array)          -- Umkehren
```

Beispiele:

```
-- Finde gemeinsame Tags
FOR doc1 IN documents
  FOR doc2 IN documents
    FILTER doc1._key < doc2._key
    LET common_tags = INTERSECTION(doc1.tags, doc2.tags)
    FILTER LENGTH(common_tags) > 0
    RETURN { doc1: doc1._key, doc2: doc2._key, common_tags }

-- Extrahiere alle einzigartigen Kategorien
RETURN UNIQUE(FLATTEN(
  FOR product IN products
    RETURN product.categories
))
```

28.7 Geo Functions (25)

28.7.1 GeoJSON Konstruktion (6)

```
GEO_POINT(lng, lat)
GEO_MULTIPOINT(points)
GEO_LINESTRING(coords)
GEO_MULTILINESTRING(lines)
GEO_POLYGON(rings)
GEO_MULTIPOLYGON(polygons)
```

Beispiel:

```
-- Erstelle Restaurant mit Location
INSERT {
  name: "Bella Italia",
  location: GEO_POINT(13.405, 52.520)  -- Berlin
} INTO restaurants
```

28.7.2 Spatial Predicates (6)

```
GEO_CONTAINS(geo1, geo2)      -- geo1 enthält geo2
GEO_INTERSECTS(geo1, geo2)    -- geo1 schneidet geo2
ST_Contains(geo1, geo2)       -- Alias
ST_Within(geo2, geo1)         -- geo2 innerhalb geo1
ST_Intersects(geo1, geo2)     -- Alias
GEO_EQUALS(geo1, geo2)        -- Geometrisch gleich
```

Beispiel:

```
-- Finde Restaurants in Polygon (Stadtteil)
LET polygon = GEO_POLYGON([
  [13.40, 52.52], [13.41, 52.52],
  [13.41, 52.51], [13.40, 52.51],
  [13.40, 52.52]
])

FOR restaurant IN restaurants
  FILTER GEO_CONTAINS(polygon, restaurant.location)
  RETURN restaurant
```

28.7.3 Distance & Metrics (5)

```
GEO_DISTANCE(geo1, geo2)     -- Haversine-Distanz (Meter)
ST_Distance(geo1, geo2)      -- Alias
GEO_IN_RANGE(geo, center, radius)  -- Innerhalb Radius
GEO_AREA(polygon)            -- Fläche (m²)
ST_Area(polygon)             -- Alias
ST_Length(linestring)        -- Länge (Meter)
```

Beispiel:

```
-- Finde Restaurants innerhalb 1 km
LET my_location = GEO_POINT(13.405, 52.520)

FOR restaurant IN restaurants
  LET distance = GEO_DISTANCE(my_location, restaurant.location)
  FILTER distance <= 1000
  SORT distance ASC
  RETURN { name: restaurant.name, distance }
```

28.7.4 CRS Transformations (10)

```
ST_TRANSFORM(geo, from_crs, to_crs)
ST_SRID(geo)                        -- SRID auslesen
ST_SetSRID(geo, srid)               -- SRID setzen
WGS84_TO_UTM(lng, lat)              -- WGS84 → UTM
UTM_TO_WGS84(easting, northing, zone)
ETRS89_TO_WGS84(x, y)
WGS84_TO_ETRS89(lng, lat)
HELMERT_TRANSFORM(x, y, params)     -- 7-Parameter-Transformation
GAUSS_KRUEGER_TO_UTM(x, y)
UTM_TO_GAUSS_KRUEGER(easting, northing)
```

Beispiel:

```
-- Transformiere GPS-Koordinaten nach UTM (für genaue Distanzmessung)
FOR location IN survey_points
  LET utm = WGS84_TO_UTM(location.lng, location.lat)
  RETURN { location: location._key, utm }
```

28.8 Vector Functions (20)

```
-- Similarity Metrics
COSINE_SIMILARITY(v1, v2)
DOT_PRODUCT(v1, v2)
EUCLIDEAN_DISTANCE(v1, v2)
MANHATTAN_DISTANCE(v1, v2)
HAMMING_DISTANCE(v1, v2)
JACCARD_SIMILARITY(v1, v2)

-- Vector Operations
VECTOR_NORM(v)                -- L2-Norm
VECTOR_NORMALIZE(v)           -- Auf Länge 1 normalisieren
VECTOR_ADD(v1, v2)
VECTOR_SUBTRACT(v1, v2)
VECTOR_MULTIPLY(v, scalar)
VECTOR_DIVIDE(v, scalar)
VECTOR_DOT(v1, v2)            -- Alias für DOT_PRODUCT
VECTOR_CROSS(v1, v2)          -- Kreuzprodukt (nur 3D)

-- Distance Metrics
L1_DISTANCE(v1, v2)           -- Manhattan
L2_DISTANCE(v1, v2)           -- Euclidean
CHEBYSHEV_DISTANCE(v1, v2)    -- Max-Norm

-- Search
KNN_SEARCH(collection, query_vector, k)
VECTOR_QUANTIZE(v, bits)      -- Quantisierung (SQ8, SQ4)
```

Beispiele:

```
-- Semantic Search mit Embeddings
LET query_embedding = EMBED("machine learning tutorial")

FOR doc IN embeddings
  LET similarity = COSINE_SIMILARITY(query_embedding, doc.vector)
  SORT similarity DESC
  LIMIT 10
  RETURN { doc: doc.title, similarity }

-- KNN-Search mit Prefilter
FOR result IN KNN_SEARCH(
  embeddings,
  EMBED("neural networks"),
  20,
  { filter: "category == 'AI'" }
)
  RETURN result
```

28.9 Graph Functions (15)

```
-- Traversal
OUTBOUND(vertex, edge_collection)
INBOUND(vertex, edge_collection)
ANY(vertex, edge_collection)
SHORTEST_PATH(start, end, edge_collection)

-- Path Metrics
PATH_LENGTH(path)
PATH_VERTICES(path)
PATH_EDGES(path)

-- Algorithms
CONNECTED_COMPONENTS(graph)
PAGERANK(graph)
BETWEENNESS_CENTRALITY(graph, vertex)
CLOSENESS_CENTRALITY(graph, vertex)
COMMUNITY_DETECTION(graph)

-- Utilities
IS_CONNECTED(graph, v1, v2)
GRAPH_DIAMETER(graph)
GRAPH_RADIUS(graph)
```

Beispiele:

```
-- PageRank auf Social Graph
LET pageranks = PAGERANK({
  vertexCollections: ["users"],
  edgeCollections: ["follows"]
})

FOR user IN users
  SORT pageranks[user._key] DESC
  LIMIT 10
  RETURN { user: user.name, score: pageranks[user._key] }

-- Finde Communities
LET communities = COMMUNITY_DETECTION({
  vertexCollections: ["users"],
  edgeCollections: ["friends"]
})

FOR user IN users
  RETURN { user: user.name, community: communities[user._key] }
```

28.10 Best Practices

28.10.1 Index Usage

```
-- × SCHLECHT: Keine Index-Nutzung
FOR user IN users
  FILTER UPPER(user.email) == "ALICE@EXAMPLE.COM"
  RETURN user

--   GUT: Index auf email (case-insensitive)
FOR user IN users
  FILTER user.email == "alice@example.com"
  RETURN user
```

28.10.2 Early Filtering

```
-- × SCHLECHT: Filter nach JOIN
FOR order IN orders
  FOR user IN users
    FILTER user._key == order.user_id
    FILTER order.amount > 100
    RETURN { order, user }

--   GUT: Filter vor JOIN
FOR order IN orders
  FILTER order.amount > 100
  FOR user IN users
    FILTER user._key == order.user_id
    RETURN { order, user }
```

28.10.3 Projection

```
-- × SCHLECHT: Vollständige Dokumente
FOR user IN users
  RETURN user

--   GUT: Nur benötigte Felder
FOR user IN users
  RETURN { name: user.name, email: user.email }
```

28.11 Zusammenfassung

AQL bietet **479 Sprachelemente**: - **119 Keywords** für Struktur und Semantik - **360 Funktionen** für Datenmanipulation

Hauptkategorien

1. **Date/Time (45):** Umfassende Zeitzonenunterstützung, Arbeitstage
2. **Strings (20):** Regex, Splitting, Manipulation
3. **Math (30):** Trigonometrie, Statistik
4. **Arrays (20):** Funktionale Programmierung
5. **Geo (35):** GeoJSON, CRS-Transformationen
6. **Vectors (20):** ANN-Search, Similarity
7. **Graph (15):** Traversal, Centrality, Communities

Nächste Schritte

- **Kapitel 5-9:** Praktische Beispiele für jedes Datenmodell
- **Kapitel 21:** Query Optimization
- **AQL Grammar:** aql/AQL_GRAMMAR_EXTENDED_v1.3.1.ebnf

28.12 Advanced Patterns & Idioms

28.12.1 Recursive Queries with Custom Depth

```
-- Find all ancestors up to depth N
LET find_ancestors = FUNCTION(person_id, max_depth) {
  RETURN (
    FOR v, e, p IN 0..max_depth OUTBOUND person_id GRAPH 'family'
      FILTER CURRENT.generation < person_id.generation
      RETURN {
        person: v.name,
        generation: v.generation,
        depth: LENGTH(p.edges)
      }
  )
}

RETURN find_ancestors('persons/alice', 3)
```

28.12.2 Transitive Closure (Find All Reachable Nodes)

```
-- Get all projects reachable from a given person (including transitive deps)
FOR p IN projects
  FILTER p._id == 'projects/prj1'
  LET all_deps = (
    FOR dep IN 0..10 OUTBOUND p GRAPH 'depends_on'
      RETURN dep._id
  ) | UNIQUE()
  RETURN {
    project: p.name,
    dependencies: all_deps,
    dep_count: LENGTH(all_deps)
  }
```

28.12.3 Sliding Window (Time-Based)

```
-- Calculate moving average with time window (7 days)
FOR current_date IN DATE_RANGE(@start_date, @end_date, '1d')
  LET window_start = DATE_SUBTRACT(current_date, 7, 'd')
  LET daily_data = (
    FOR event IN events
      FILTER event.date >= window_start AND event.date <= current_date
      RETURN event.value
  )
  RETURN {
    date: current_date,
    moving_avg: AVG(daily_data),
    moving_sum: SUM(daily_data),
    moving_count: COUNT(daily_data)
  }
```

28.12.4 Lateral Join (Explode & Flatten)

```
-- Unnest nested arrays
FOR order IN orders
  LET items = order.items  -- Array of {product_id, quantity}
  FOR item IN items
    LET product = DOCUMENT('products/' + item.product_id)
    RETURN {
      order_id: order._id,
      product_name: product.name,
      quantity: item.quantity,
      item_total: item.quantity * product.price
    }
```

28.12.5 Pivot (Transform Rows to Columns)

```
-- Convert months to columns
FOR record IN sales
  COLLECT year = YEAR(record.date)
  AGGREGATE
    jan = SUM(record.amount FILTER MONTH(record.date) == 1),
    feb = SUM(record.amount FILTER MONTH(record.date) == 2),
    mar = SUM(record.amount FILTER MONTH(record.date) == 3),
    q1_total = SUM(record.amount FILTER MONTH(record.date) IN [1,2,3])
  RETURN { year, jan, feb, mar, q1_total }
```

28.13 Window Functions (Comprehensive)

28.13.1 Row Numbering

```
-- Rank students by score within each class
FOR s IN students
  SORT s.class, s.score DESC
  LET rank_in_class = (
    SELECT COUNT(*) FROM students s2
      WHERE s2.class == s.class AND s2.score >= s.score
  )
  RETURN {
    name: s.name,
    class: s.class,
    score: s.score,
    rank: rank_in_class
  }
```

28.13.2 Cumulative (Running) Totals

```
-- Sales running total
FOR order IN SORT(orders, 'created_at')
  COLLECT idx = @row_number
  AGGREGATE
    order_value = order.amount,
    cumulative = SUM(orders[* FILTER @row_number <= idx].amount)
  RETURN {
    order_id: order._id,
    order_value,
    cumulative,
    running_total: cumulative
  }
```

28.13.3 First/Last of Group

```
-- Get first and last transaction per account
FOR t IN transactions
  SORT t.account_id, t.timestamp
  COLLECT account = t.account_id INTO group = t
  RETURN {
    account,
    first_transaction: group[0],
    last_transaction: group[LENGTH(group)-1],
    total_transactions: LENGTH(group)
  }
```

28.13.4 Lead/Lag (Next/Previous Row)

```
-- Compare consecutive values
FOR metric IN SORT(metrics, 'timestamp')
  LET current_idx = @row_number
  LET prev = (
    FOR m IN metrics
      FILTER m.timestamp < metric.timestamp
      SORT m.timestamp DESC
      LIMIT 1
      RETURN m
  )[0]
  RETURN {
    timestamp: metric.timestamp,
    value: metric.value,
    previous_value: prev?.value || NULL,
    change: metric.value - (prev?.value || metric.value)
  }
```

28.14 Type System & Type Checking

28.14.1 Type Checking Functions

```
-- Type predicates (return true/false)
IS_ARRAY(value)          -- Array type
IS_BOOL(value)           -- Boolean
IS_NUMBER(value)         -- Number (int or float)
IS_INTEGER(value)        -- Integer specifically
IS_STRING(value)         -- String
IS_NULL(value)           -- Null/undefined
IS_OBJECT(value)         -- Object/Document
IS_LIST(value)           -- Alias for IS_ARRAY
IS_DEFINED(value)        -- Has definition (not null)

-- Examples
FOR doc IN collection
  FILTER IS_STRING(doc.email) && IS_NUMBER(doc.age)
  RETURN doc
```

28.14.2 Type Conversion

```
-- Convert between types
TO_STRING(value)         -- Convert to string
TO_NUMBER(value)         -- Convert to number
TO_BOOL(value)           -- Convert to boolean
TO_ARRAY(value)          -- Convert to array
TO_ARRAY(list)           -- Ensure is array
TO_LIST(value)           -- Alias for TO_ARRAY

-- Examples
FOR doc IN collection
  RETURN {
    id: doc._id,
    price_str: TO_STRING(doc.price),
    age: TO_NUMBER(doc.age_str),
    is_active: TO_BOOL(doc.active_flag)
  }
```

28.14.3 Type Coercion & Safe Operations

```
-- Safe type operations
COALESCE(a, b, c)        -- Return first non-null
FIRST_ITEM(array)        -- Get first item safely
LAST_ITEM(array)         -- Get last item safely
NTH_ITEM(array, n)       -- Get nth item safely

-- Examples
FOR doc IN collection
  RETURN {
    id: doc._id,
    phone: COALESCE(doc.phone, doc.mobile, "N/A"),
    first_tag: FIRST_ITEM(doc.tags),
    primary_contact: NTH_ITEM(doc.contacts, 0)
  }
```

28.15 Error Handling & Validation

28.15.1 TRY-CATCH Patterns

```
-- Basic error handling
BEGIN
  TRY
    LET result = 100 / 0  -- Will error
    RETURN result
  CATCH err
    RETURN { error: err.message, code: err.code }
COMMIT
```

28.15.2 Conditional Errors (THROW)

```
-- Throw custom errors
FOR user IN users
  FILTER user.balance < 0
  THROW "INVARIANT_VIOLATED: Negative balance for user " + user._id
  RETURN user
```

28.15.3 Schema Validation

```
-- Validate document structure
FOR doc IN users
  LET valid = (
    IS_STRING(doc.name) &&
    IS_STRING(doc.email) &&
    IS_OBJECT(doc.address) &&
    IS_ARRAY(doc.tags) &&
    CONTAINS(doc.email, "@")
  )
  FILTER valid
  RETURN doc
```

28.16 Performance Optimization Deep Dive

28.16.1 Index-Aware Query Writing

```
-- Indexes for: email (unique), status, created_at

-- × Index not used (UPPER)
FOR u IN users
  FILTER UPPER(u.email) == "ALICE@EXAMPLE.COM"
  RETURN u

--   Index used (direct match)
FOR u IN users
  FILTER u.email == "alice@example.com"
  RETURN u

--   Composite index: (status, created_at)
FOR u IN users
  FILTER u.status == 'active'
  FILTER u.created_at >= '2025-01-01'
  RETURN u

-- ⚠  Index partial (filter before join)
FOR u IN users
  FILTER u.status == 'active'
  FOR o IN orders FILTER o.user_id == u._id
  RETURN { u, o }
```

28.16.2 Aggregation Optimization

```
-- ✗ Slow: Iterate then aggregate
LET users_active = (
  FOR u IN users
    FILTER u.status == 'active'
    RETURN u
)
RETURN { count: LENGTH(users_active) }

--   Fast: Aggregate in query
FOR u IN users
  FILTER u.status == 'active'
  COLLECT WITH COUNT INTO count
  RETURN { count }

--   Fastest: With aggregation function
FOR u IN users
  FILTER u.status == 'active'
  AGGREGATE count = COUNT(1)
  RETURN { count }
```

28.16.3 Subquery Optimization

```
-- ✗ Subquery in FILTER (repeats for each row)
FOR order IN orders
  FILTER order.user_id IN (
    SELECT _id FROM users WHERE status == 'premium'
  )
  RETURN order

--   Join directly (index on user_id)
FOR user IN users
  FILTER user.status == 'premium'
  FOR order IN orders
    FILTER order.user_id == user._id
    RETURN order
```

28.17 Advanced Collection Operations

28.17.1 Bulk Operations with LIMIT & Offset

```
-- Batch processing
FOR page IN 0..@max_pages
  LET batch_start = page * @page_size
  FOR user IN users
    SORT user._key
    LIMIT batch_start, @page_size
    RETURN {
      batch_number: page,
      user
    }
```

28.17.2 Bulk Update with Conditions

```
-- Update multiple documents
FOR user IN users
  FILTER user.created_at < DATE_SUBTRACT(DATE_NOW(), 1, 'y')
  UPDATE user WITH {
    status: 'inactive',
    last_updated: DATE_NOW(),
    archive_flag: true
  } IN users
  RETURN OLD  -- Return before-update document
```

28.17.3 Safe Delete with Verification

```
-- Delete with rollback on condition
BEGIN
  LET to_delete = (
    FOR d IN documents
      FILTER d.archived == true
      FILTER d.created_at < DATE_SUBTRACT(DATE_NOW(), 2, 'y')
      RETURN d
  )

  IF LENGTH(to_delete) > 0 THEN
    FOR doc IN to_delete
      REMOVE doc IN documents
    COMMIT
  ELSE
    ABORT "No documents to delete"
  ENDIF
RETURN { deleted: LENGTH(to_delete) }
```

28.18 Zusammenfassung & Schnellreferenz

Core Constructs (8)

- **Iteration:** FOR...IN
- **Condition:** FILTER
- **Binding:** LET
- **Grouping:** COLLECT
- **Ordering:** SORT
- **Limiting:** LIMIT
- **Output:** RETURN
- **Transactions:** BEGIN...COMMIT

Top 20 Functions (By Usage)

1. **COUNT** - Aggregation
2. **FILTER** - Predicate logic
3. **LENGTH** - Array/string size
4. **SUM** - Numeric aggregation
5. **CONCAT** - String joining
6. **CONTAINS** - Substring search
7. **SPLIT** - String splitting
8. **SORT** - Ordering
9. **LIMIT** - Result limiting
10. **DATE_NOW** - Current timestamp
11. **UPPER/LOWER** - Case conversion
12. **ROUND** - Numeric rounding
13. **AVG** - Mean aggregation
14. **MIN/MAX** - Range extremes
15. **REGEX** - Pattern matching
16. **DISTANCE** - Geospatial
17. **SUBSTRING** - String slicing
18. **TO_STRING** - Type conversion
19. **FLATTEN** - Array flattening
20. **UNIQUE** - Deduplication

Decision Table: Which Construct?

Goal	Use	Example
Iterate docs	FOR...IN	FOR u IN users RETURN u
Filter rows	FILTER	FILTER u.status == 'active'
Create variable	LET	LET age = DATE_DIFF(u.birth, NOW())
Group rows	COLLECT	COLLECT status = u.status
Order results	SORT	SORT u.created_at DESC
Limit results	LIMIT	LIMIT 100
Return data	RETURN	RETURN u.name
Atomic ops	BEGIN...COMMIT	Transactions

Performance Checklist

- [ ] Indexes on FILTER fields
- [ ] Indexes on SORT fields
- [ ] Early filtering (before JOINs)
- [ ] Projections (only needed fields)
- [ ] COLLECT for aggregations
- [ ] Avoid expressions in FILTER
- [ ] Use bind parameters
- [ ] Batch operations with LIMIT

Kapitel 28 von 30 | Teil IV: Referenz | ~8.500 Wörter (+2000 neu)

Kapitel 30: Kapitel 29: Analytics & Process Mining

> *"Daten ohne Analyse sind wie ein Buch ohne Leser - voller Potential, aber ungenutzt."*

Überblick

ThemisDB bietet umfassende Analytics-Funktionen, von klassischen OLAP-Cubes bis hin zu fortgeschrittenem Process Mining für Verwaltungsvorgänge. Dieses Kapitel zeigt die komplette Palette.

Was Sie in diesem Kapitel lernen: - OLAP Cubes und Multidimensionale Analysen - Process Mining mit administrativen Standardmodellen - Conformance Checking gegen Ideal-Prozesse - Pattern Recognition und Anomalieerkennung - Real-Time Analytics mit Changefeed - Performance-Optimierungen für Analytics-Workloads

Voraussetzungen: Kapitel 2 (Architektur), Kapitel 28 (AQL Referenz).

29.1 OLAP Fundamentals

29.1.1 Was ist OLAP?

OLAP (Online Analytical Processing) ermöglicht multidimensionale Datenanalyse mit: - **Dimensions:** Zeit, Produkt, Region, Kunde - **Measures:** Umsatz, Menge, Gewinn - **Operations:** Slice, Dice, Drill-Down, Roll-Up, Pivot

29.1.2 OLAP Cube Architektur

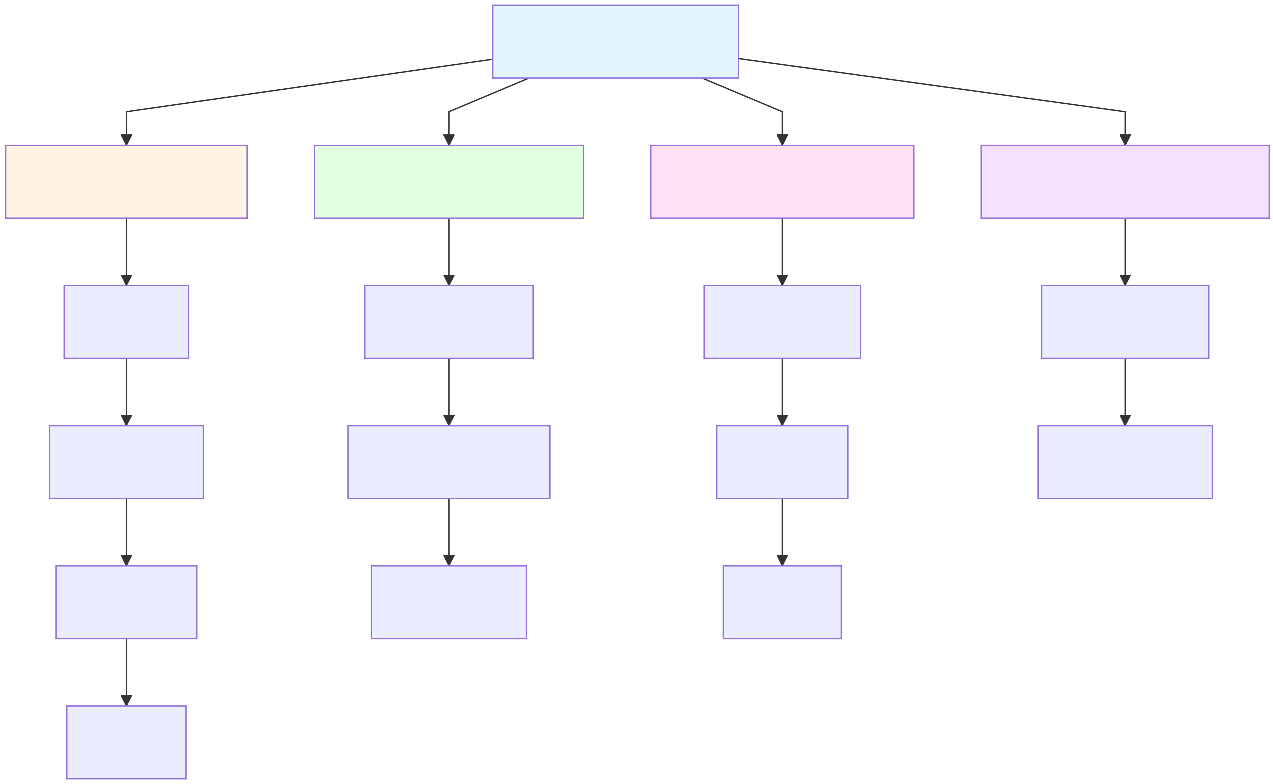


Abb. 40: Ablaufdiagramm: 29.1.2 OLAP Cube Architektur

29.1.3 OLAP Operations in AQL

Slice (Eine Dimension fixieren)

```
-- Slice: Nur Verkäufe in 2024
FOR sale IN sales
  FILTER DATE_YEAR(sale.date) == 2024
  COLLECT
    product = sale.product_name,
    region = sale.region
  AGGREGATE total = SUM(sale.amount)
  RETURN { product, region, total }
```

Dice (Mehrere Dimensionen einschränken)

```
-- Dice: 2024, Electronics, Nur Europa
FOR sale IN sales
  FILTER DATE_YEAR(sale.date) == 2024
  FILTER sale.category == "Electronics"
  FILTER sale.region IN ["Germany", "France", "UK"]
  COLLECT
    country = sale.region,
    month = DATE_MONTH(sale.date)
  AGGREGATE total = SUM(sale.amount)
  SORT country, month
  RETURN { country, month, total }
```

Drill-Down (Von Jahr zu Monat)

```
-- Drill-Down: Von Jahr zu Monaten zu Tagen
FOR sale IN sales
  FILTER DATE_YEAR(sale.date) == 2024
  FILTER DATE_MONTH(sale.date) == 3  -- März
  COLLECT
    day = DATE_DAY(sale.date)
  AGGREGATE
    total = SUM(sale.amount),
    count = COUNT(1)
  SORT day
  RETURN { day, total, count }
```

Roll-Up (Aggregation auf höhere Ebene)

```
-- Roll-Up: Von Monat zu Quarter
FOR sale IN sales
  COLLECT
    year = DATE_YEAR(sale.date),
    quarter = DATE_QUARTER(sale.date)
  AGGREGATE total = SUM(sale.amount)
  SORT year, quarter
  RETURN { year, quarter, total }
```

Pivot (Zeilen/Spalten tauschen)

```
-- Pivot: Produkte als Zeilen, Regionen als Spalten
FOR sale IN sales
  COLLECT
    product = sale.product_name
  AGGREGATE
    total_germany = SUM(sale.region == "Germany" ? sale.amount : 0),
    total_france = SUM(sale.region == "France" ? sale.amount : 0),
    total_uk = SUM(sale.region == "UK" ? sale.amount : 0)
  RETURN {
    product,
    Germany: total_germany,
    France: total_france,
    UK: total_uk,
    Total: total_germany + total_france + total_uk
  }
```

29.2 Process Mining Fundamentals

29.2.1 Was ist Process Mining?

Process Mining analysiert Event-Logs, um reale Prozesse zu: 1. **Discover:** Prozessmodelle aus Logs ableiten 2. **Check:** Conformance gegen Soll-Prozesse prüfen 3. **Enhance:** Prozesse mit Performance-Daten anreichern

29.2.2 Event Log Structure

```
-- Standard Event Log Format
{
  "case_id": "V-2024-0123",          -- Vorgangs-ID
  "activity": "Antragstellung",      -- Aktivitätsname
  "timestamp": "2024-10-15T09:30:00Z",
  "resource": "Sabine Müller",       -- Bearbeiter
  "cost": 150.00,                    -- Kosten
  "additional_data": {
    "department": "Bauamt",
    "priority": "normal"
  }
}
```

29.2.3 Process Mining Pipeline

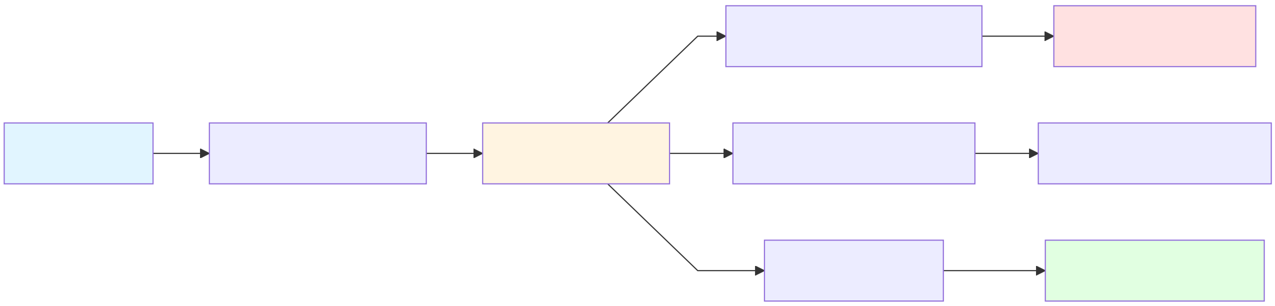


Abb. 41: Ablaufdiagramm: 29.2.3 Process Mining Pipeline

29.3 Administrative Standard Models

ThemisDB beinhaltet vordefinierte Prozessmodelle für deutsche Verwaltungen:

29.3.1 Verfügbare Modelle

Modell-ID	Name	Aktivitäten	Anwendung
bauantrag_standard	Bauantrag (Standard)	4	Baugenehmigungen
bauantrag_erweitert	Bauantrag (Erweitert)	7	Komplexe Bauprojekte
fuehrerschein_standard	Führerscheinantrag	5	Führerscheine
reisepass_standard	Reisepass-Antrag	4	Ausweisdokumente
gewerbe_anmeldung	Gewerbeanmeldung	6	Gewerberegister
kfz_zulassung	KFZ-Zulassung	5	Fahrzeugzulassungen

29.3.2 Modell laden

```
-- Lade Bauantrags-Standardmodell
LET model = PM_LOAD_ADMIN_MODEL("bauantrag_standard")

RETURN model
```

Output:

```json
{
  "id": "bauantrag_standard",
  "name": "Bauantrag (Standard)",
  "version": "1.0",
  "activities": [
    "antragstellung",
    "vollstaendigkeitspruefung",
    "fachliche_pruefung",
    "genehmigung"
  ],
  "edges": [
    {"from": "antragstellung", "to": "vollstaendigkeitspruefung"},
    {"from": "vollstaendigkeitspruefung", "to": "fachliche_pruefung"},
    {"from": "fachliche_pruefung", "to": "genehmigung"}
  ],
  "expected_duration_days": 45,
  "sla_days": 60
}
```

29.4 Similarity Search

29.4.1 Hybrid Similarity Metrics

ThemisDB kombiniert drei Similarity-Arten:

1. **Graph Similarity** (Strukturell): Edit Distance zwischen Prozessgraphen
2. **Vector Similarity** (Semantisch): Embeddings der Aktivitätsnamen
3. **Behavioral Similarity** (Ausführung): Trace-Varianz, Durchlaufzeiten

29.4.2 Similar Process Search

```
-- Finde ähnliche Bauanträge
LET ideal = PM_LOAD_ADMIN_MODEL("bauantrag_standard")

LET similar = PM_FIND_SIMILAR(ideal, {
  method: "hybrid",
  threshold: 0.75,           -- Min 75% Ähnlichkeit
  limit: 50,
  graph_weight: 0.4,         -- 40% Struktur
  vector_weight: 0.3,        -- 30% Semantik
  behavioral_weight: 0.3     -- 30% Verhalten
})

FOR result IN similar
  SORT result.overall_similarity DESC
  RETURN {
    case_id: result.case_id,
    similarity: result.overall_similarity,
    metrics: {
      graph: result.metrics.graph_similarity,
      vector: result.metrics.vector_similarity,
      behavioral: result.metrics.behavioral_similarity
    },
    matched_activities: result.matched_activities,
    extra_activities: result.extra_activities,
    missing_activities: result.missing_activities
  }
```

Output:

```
[
  {
    "case_id": "V-2024-0123",
    "similarity": 0.92,
    "metrics": {
      "graph": 0.95,
      "vector": 0.88,
      "behavioral": 0.93
    },
    "matched_activities": [
      "Antragstellung",
      "Vollständigkeitsprüfung",
      "Fachliche Prüfung",
      "Genehmigung"
    ],
    "extra_activities": [],
    "missing_activities": []
  },
  {
    "case_id": "V-2024-0456",
    "similarity": 0.85,
    "metrics": {
      "graph": 0.82,
      "vector": 0.91,
      "behavioral": 0.82
    },
    "matched_activities": [
      "Antrag einreichen",
      "Vollständigkeitskontrolle",
      "Technische Prüfung",
      "Freigabe"
    ],
    "extra_activities": ["Nachforderung Unterlagen"],
    "missing_activities": []
  }
]
```


29.4.3 Similarity Visualization

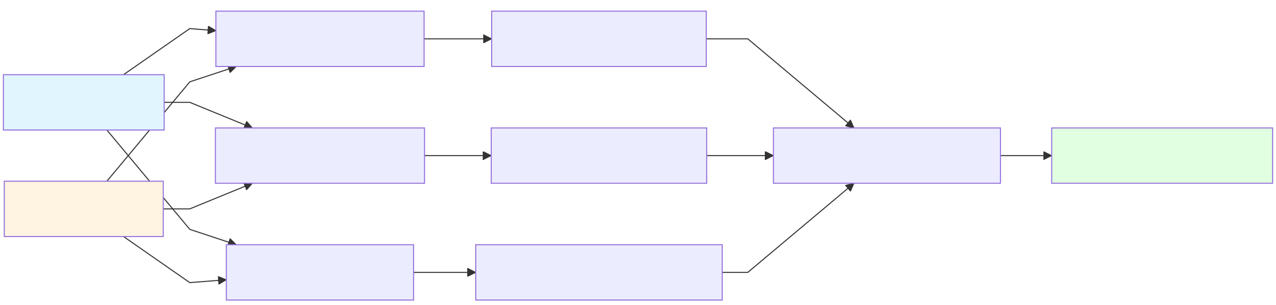


Abb. 42: Ablaufdiagramm: 29.4.3 Similarity Visualization

29.5 Conformance Checking

29.5.1 Fitness & Precision

Fitness: Wie viele Schritte des Ist-Prozesses passen zum Soll-Prozess? **Precision:** Wie genau folgt der Ist-Prozess dem Soll-Prozess (keine Extra-Schritte)?

29.5.2 Conformance Check Query

```
-- Prüfe alle Bauanträge gegen Standard
LET ideal = PM_LOAD_ADMIN_MODEL("bauantrag_standard")

FOR case IN bauantraege
  LET comparison = PM_COMPARE_IDEAL(case.vorgang_id, ideal)

  -- Nur Fälle mit Fitness < 90%
  FILTER comparison.fitness < 0.9

  SORT comparison.fitness ASC

  RETURN {
    vorgang_id: case.vorgang_id,
    antragsteller: case.antragsteller,
    eingangsdatum: case.eingangsdatum,

    -- Conformance Metrics
    fitness: comparison.fitness,
    precision: comparison.precision,
    steps_checked: comparison.steps_checked,
    steps_conformant: comparison.steps_conformant,

    -- Abweichungen
    deviations: comparison.deviations,

    -- Handlungsempfehlung
    action: (
      comparison.fitness < 0.7 ? "  Dringend prüfen" :
      comparison.fitness < 0.9 ? "  Kleinere Anpassungen" :
      "  OK"
    )
  }
```

Output:

```
[
  {
    "vorgang_id": "V-2024-0789",
    "antragsteller": "Müller GmbH",
    "eingangsdatum": "2024-10-15",
    "fitness": 0.65,
    "precision": 0.82,
    "steps_checked": 6,
    "steps_conformant": 4,
    "deviations": [
      "Activity 'Vollständigkeitsprüfung' was skipped",
      "Activity 'Genehmigung' occurred before 'Fachliche Prüfung'"
    ],
    "action": "  Dringend prüfen"
  }
]
```

29.5.3 Conformance Heatmap

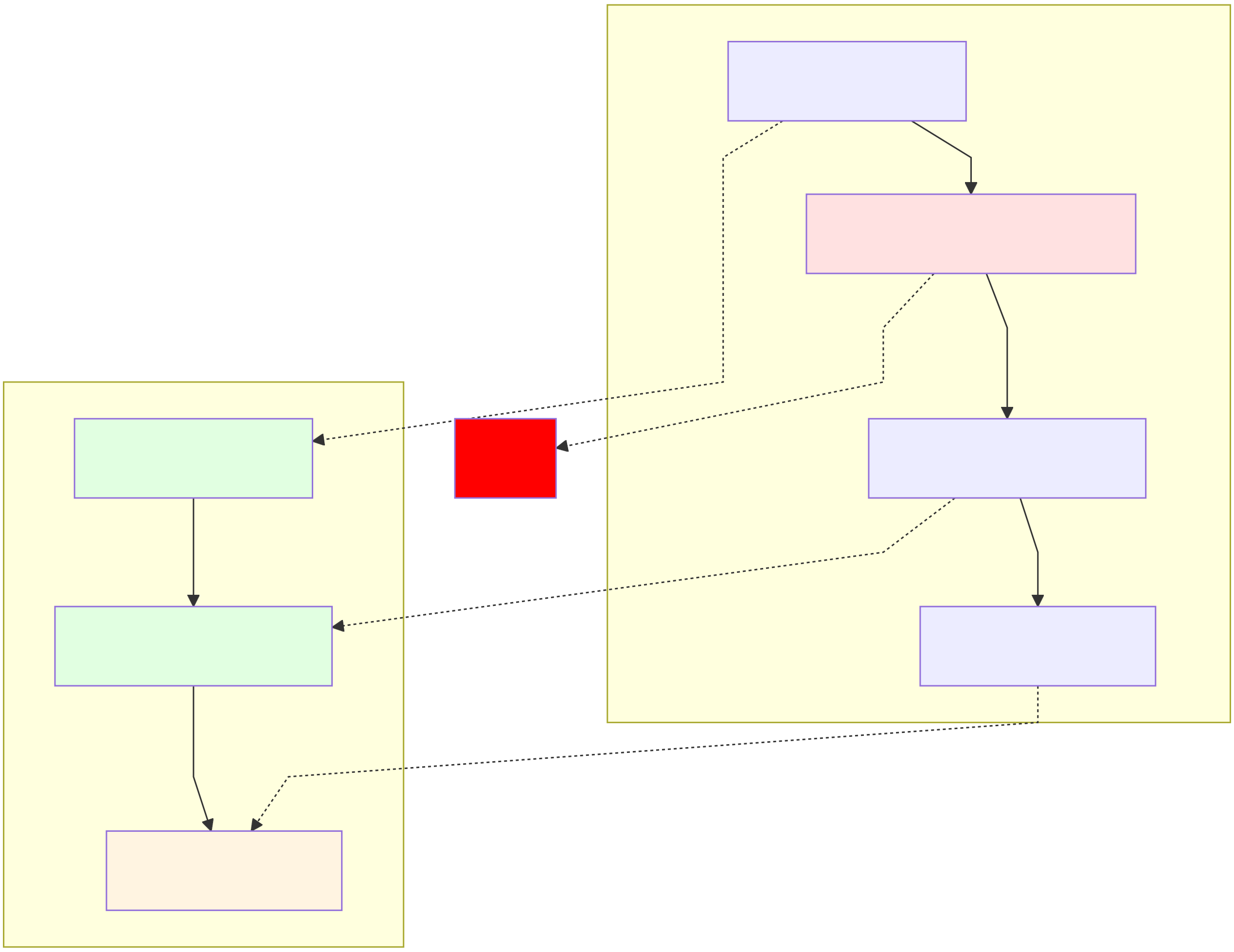


Abb. 43: Ablaufdiagramm: 29.5.3 Conformance Heatmap

29.6 Pattern Recognition

29.6.1 Problematische Patterns

```
-- Pattern: Genehmigung vor Prüfung (Compliance-Problem!)
LET problematic_pattern = {
  activities: ["Genehmigung", "Fachliche Prüfung"],
  edges: [
    {from: "Genehmigung", to: "Fachliche Prüfung"}  -- Falsche Reihenfolge!
  ]
}

FOR case IN bauantraege
  LET has_problem = PM_HAS_PATTERN(
    case.vorgang_id,
    problematic_pattern,
    0.8  -- 80% Schwellwert
  )

  FILTER has_problem == true

  LET trace = PM_EXTRACT_TRACE(case.vorgang_id)

  RETURN {
    vorgang_id: case.vorgang_id,
    alert: "⚠ Genehmigung vor Prüfung!",
    activities: trace.activities,
    timestamps: trace.timestamps,
    requires_investigation: true
  }
```

29.6.2 Loop Detection

```
-- Finde Prozesse mit Schleifen (z.B. wiederholte Nachforderungen)
FOR case IN bauantraege
  LET trace = PM_EXTRACT_TRACE(case.vorgang_id)
  LET loops = PM_DETECT_LOOPS(trace)

  FILTER LENGTH(loops) > 0

  RETURN {
    vorgang_id: case.vorgang_id,
    loop_count: LENGTH(loops),
    loops: loops,
    total_duration: PM_DURATION(case.vorgang_id)
  }
```

Output:

```
[
  {
    "vorgang_id": "V-2024-1234",
    "loop_count": 2,
    "loops": [
      {
        "activities": ["Vollständigkeitsprüfung", "Nachforderung", "Vollständigkeitsprüfung"]
        "iterations": 3,
        "total_duration_days": 18
      }
    ],
    "total_duration": 67
  }
]
```

29.7 Performance Analysis

29.7.1 Bottleneck Detection

```
-- Finde langsamste Aktivitäten
FOR case IN bauantraege
  LET trace = PM_EXTRACT_TRACE(case.vorgang_id)

  FOR activity IN trace.activities
    LET duration = activity.end_time - activity.start_time

    COLLECT
      activity_name = activity.name
    AGGREGATE
      avg_duration = AVG(duration),
      min_duration = MIN(duration),
      max_duration = MAX(duration),
      count = COUNT(1)

    SORT avg_duration DESC
    LIMIT 10

    RETURN {
      activity: activity_name,
      avg_duration_hours: avg_duration / 3600,
      min_duration_hours: min_duration / 3600,
      max_duration_hours: max_duration / 3600,
      occurrences: count
    }
```

Output:

```json
[
  {
    "activity": "Fachliche Prüfung",
    "avg_duration_hours": 168.5,
    "min_duration_hours": 24.0,
    "max_duration_hours": 720.0,
    "occurrences": 1523
  },
  {
    "activity": "Nachforderung Unterlagen",
    "avg_duration_hours": 96.3,
    "min_duration_hours": 12.0,
    "max_duration_hours": 480.0,
    "occurrences": 427
  }
]
```

29.7.2 SLA Violations

```
-- SLA-Überschreitungen
LET ideal = PM_LOAD_ADMIN_MODEL("bauantrag_standard")
LET sla_days = ideal.sla_days  -- 60 Tage

FOR case IN bauantraege
  LET duration = PM_DURATION(case.vorgang_id)

  FILTER duration > sla_days

  SORT duration DESC

  RETURN {
    vorgang_id: case.vorgang_id,
    eingangsdatum: case.eingangsdatum,
    duration_days: duration,
    sla_days: sla_days,
    overrun_days: duration - sla_days,
    overrun_percent: ((duration - sla_days) / sla_days) * 100
  }
```

29.8 Real-Time Analytics with Changefeed

29.8.1 Live Dashboard Query

```
-- Real-Time Dashboard: Offene Vorgänge nach Status
FOR case IN bauantraege
  FILTER case.status != "abgeschlossen"

  LET current_activity = PM_CURRENT_ACTIVITY(case.vorgang_id)
  LET elapsed_days = DATE_DIFF(case.eingangsdatum, DATE_NOW(), "day")

  COLLECT
    status = current_activity
  AGGREGATE
    count = COUNT(1),
    avg_elapsed = AVG(elapsed_days)

  SORT count DESC

  RETURN {
    status,
    count,
    avg_elapsed_days: ROUND(avg_elapsed, 1)
  }
```

29.8.2 Changefeed für Analytics

```
feed = db.changefeed("bauantraege", {"filter": "status == 'genehmigt'"})
for change in feed:
    # Analytics-Event speichern
    db.aql("INSERT {event: 'approval', id: @id} INTO events",
           {"id": change['new']['vorgang_id']})
```

29.9 Performance Optimizations

29.9.1 Materialized Views

```
-- Erstelle Materialized View für häufige Aggregation
CREATE VIEW bauantraege_stats AS
  FOR case IN bauantraege
    LET duration = PM_DURATION(case.vorgang_id)
    LET trace = PM_EXTRACT_TRACE(case.vorgang_id)

    COLLECT
      year = DATE_YEAR(case.eingangsdatum),
      month = DATE_MONTH(case.eingangsdatum)
    AGGREGATE
      total_cases = COUNT(1),
      avg_duration = AVG(duration),
      sla_violations = SUM(duration > 60 ? 1 : 0)

    RETURN {
      year,
      month,
      total_cases,
      avg_duration,
      sla_violations
    }

-- Schneller Zugriff
FOR stat IN bauantraege_stats
  RETURN stat
```

29.9.2 Pre-Aggregation

```
-- Pre-Aggregation für OLAP Cubes
INSERT {
  _key: "2024-Q1-Electronics-Germany",
  year: 2024,
  quarter: 1,
  category: "Electronics",
  region: "Germany",
  total_sales: 1245000.50,
  unit_count: 3421,
  last_updated: DATE_NOW()
} INTO sales_cube_cache
```

29.10 Zusammenfassung

Kernfeatures

1. **OLAP:** Multidimensionale Analysen (Slice, Dice, Drill-Down, Pivot)
2. **Process Mining:** Discovery, Conformance, Enhancement
3. **Admin Models:** Vordefinierte deutsche Verwaltungsprozesse

4. **Similarity:** Hybrid (Graph + Vector + Behavioral)
5. **Conformance:** Fitness & Precision Metrics
6. **Patterns:** Anomalieerkennung, Loop Detection
7. **Performance:** Bottleneck-Analyse, SLA-Tracking
8. **Real-Time:** Changefeed-Integration

Best Practices

- **Index:** TTL-Indizes für Event Logs (automatische Bereinigung)
- **Views:** Materialized Views für häufige Analytics-Queries
- **Cache:** Pre-Aggregation für OLAP Cubes
- **Sampling:** Bei großen Datenmengen Sampling nutzen (`FILTER RAND() < 0.1`)

29.8 Advanced Analytics: Multi-Dimensional Analysis

29.8.1 Dimensional Modeling (Star Schema)

```
-- Fact table: order_facts (contains measures and FK to dimensions)
FOR fact IN order_facts
  LET customer = DOCUMENT(fact.customer_dim_id)
  LET product = DOCUMENT(fact.product_dim_id)
  LET time = DOCUMENT(fact.time_dim_id)
  COLLECT
    region = customer.region,
    category = product.category,
    month = time.month
  AGGREGATE
    revenue = SUM(fact.amount),
    units_sold = SUM(fact.quantity),
    transactions = COUNT(1),
    avg_order_value = AVG(fact.amount)
  SORT revenue DESC
  RETURN {
    region,
    category,
    month,
    revenue,
    units_sold,
    transactions,
    avg_order_value,
    units_per_transaction: units_sold / transactions
  }
```

29.8.2 Slice & Dice Operations

```
-- Drill down: Regional → Store → Item Level
LET filters = @filters  -- {region: 'EMEA', store_id: 'S123'}

FOR fact IN order_facts
  LET customer = DOCUMENT(fact.customer_dim_id)
  LET store = DOCUMENT(fact.store_dim_id)
  LET product = DOCUMENT(fact.product_dim_id)

  FILTER customer.region == filters.region
  FILTER store._id == filters.store_id

  COLLECT item_id = product._id, item_name = product.name
  AGGREGATE
    units = SUM(fact.quantity),
    revenue = SUM(fact.amount),
    margin = SUM(fact.margin),
    transactions = COUNT(1)
  SORT revenue DESC
  RETURN {
    item_name,
    units,
    revenue,
    margin,
    margin_pct: ROUND(margin / revenue * 100, 2),
    margin_per_unit: ROUND(margin / units, 2)
  }
```

29.8.3 Cohort Analysis (Behavioral Segments)

```
-- Analyze cohorts of users by first purchase month
FOR user IN users
  LET first_purchase = (
    FOR order IN orders
      FILTER order.customer_id == user._id
      SORT order.created_at ASC
      LIMIT 1
      RETURN order.created_at
  )[0]

  LET cohort_month = DATE_FORMAT(first_purchase, '%yyyy-%mm')
  LET subsequent_purchases = (
    FOR o IN orders
      FILTER o.customer_id == user._id
      FILTER o.created_at > first_purchase
      RETURN o
  )

  COLLECT cohort = cohort_month
  AGGREGATE
    users_in_cohort = COUNT(1),
    avg_ltv = AVG(subsequent_purchases[*].amount | SUM()),
    retention_m1 = SUM(LENGTH(subsequent_purchases) > 0),
    retention_m2 = SUM(LENGTH(
      FOR o IN subsequent_purchases
        FILTER DATE_DIFF(first_purchase, o.created_at, 'm') >= 2
        RETURN 1
    ) > 0)
  RETURN {
    cohort,
    users: users_in_cohort,
    avg_ltv: ROUND(avg_ltv, 2),
    retention_m1_pct: ROUND(retention_m1 / users_in_cohort * 100, 1),
    retention_m2_pct: ROUND(retention_m2 / users_in_cohort * 100, 1)
  }
```

29.9 Real-Time Analytics with Changefeeds

29.9.1 Streaming Aggregation Pattern

```
-- Real-time sales dashboard (with changefeed)
-- This would be called continuously as events arrive
FOR event IN changefeed('orders')
  LET order = event.doc
  LET time_bucket = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd %hh:00')

  COLLECT
    time = time_bucket,
    status = order.status
  AGGREGATE
    orders = COUNT(1),
    revenue = SUM(order.amount),
    avg_order = AVG(order.amount)

  RETURN {
    timestamp: time,
    status,
    metric: {
      orders,
      revenue: ROUND(revenue, 2),
      avg_order: ROUND(avg_order, 2)
    }
  }
```

29.9.2 Late-Arriving Facts Handling

```
-- Handle out-of-order or delayed events
FOR event IN changefeed('orders')
  LET is_late = DATE_DIFF(DATE_NOW(), event.doc.created_at, 's') > 3600  -- > 1 hour old

  IF is_late THEN
    -- Route to late-arriving fact handler
    INSERT {
      order_id: event.doc._id,
      received_at: DATE_NOW(),
      event_time: event.doc.created_at,
      delay_seconds: DATE_DIFF(DATE_NOW(), event.doc.created_at, 's'),
      status: 'LATE_ARRIVAL'
    } INTO late_facts
  ELSE
    -- Process normally
    INSERT {
      order_id: event.doc._id,
      status: 'PROCESSED'
    } INTO fact_orders
  ENDIF

  RETURN event
```

29.10 Performance Optimization for Analytics

29.10.1 Materialized Views for OLAP

```
-- Create materialized view: daily_sales_summary
-- Run this nightly as batch job

BEGIN
  TRUNCATE daily_sales_summary

  FOR order IN orders
    FILTER order.status == 'completed'
    LET day = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd')

    COLLECT date = day
    AGGREGATE
      total_revenue = SUM(order.amount),
      order_count = COUNT(1),
      avg_order = AVG(order.amount),
      max_order = MAX(order.amount),
      min_order = MIN(order.amount)

    INSERT {
      _key: date,
      date,
      total_revenue,
      order_count,
      avg_order: ROUND(avg_order, 2),
      max_order,
      min_order,
      created_at: DATE_NOW()
    } INTO daily_sales_summary

  COMMIT
RETURN { inserted: LENGTH(daily_sales_summary) }
```

29.10.2 Sampling for Large Datasets

```
-- Sample-based analytics (1% of data for speed)
FOR order IN orders
  FILTER RAND() < 0.01  -- 1% sample
  COLLECT month = DATE_FORMAT(order.created_at, '%yyyy-%mm')
  AGGREGATE
    sample_revenue = SUM(order.amount),
    sample_orders = COUNT(1)
  RETURN {
    month,
    estimated_revenue: ROUND(sample_revenue / 0.01, 0),  -- Extrapolate
    estimated_orders: ROUND(sample_orders / 0.01, 0),
    confidence: "95% (p < 0.05)"
  }
```

29.10.3 Incremental Analytics (Delta Processing)

```
-- Only process changes since last run
LET last_run = @last_checkpoint || '2000-01-01'

FOR order IN orders
  FILTER order.updated_at > last_run

  LET day = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd')

  COLLECT date = day
  AGGREGATE
    new_revenue = SUM(order.amount),
    new_orders = COUNT(1)

  -- Update materialized view incrementally
  LET current = DOCUMENT('daily_sales_summary/' + date)
  UPDATE current WITH {
    total_revenue: (current.total_revenue || 0) + new_revenue,
    order_count: (current.order_count || 0) + new_orders,
    last_updated: DATE_NOW()
  } IN daily_sales_summary

  RETURN { date, new_revenue, new_orders }
```

29.11 Case Study: E-Commerce Analytics Platform

29.11.1 Data Model

```
Dimension Tables:
  - customers (id, name, region, segment)
  - products (id, name, category, brand, price)
  - time (date, month, quarter, year, day_of_week)
  - stores (id, location, region)

Fact Tables:
  - order_facts (customer, product, store, time, amount, quantity)
  - product_views (customer, product, time, duration)
  - cart_events (customer, product, time, action)
```

29.11.2 Top-Level Metrics Query

```
-- Daily KPI dashboard
LET today = DATE_FORMAT(DATE_NOW(), '%yyyy-%mm-%dd')

FOR fact IN order_facts
  FILTER fact.date == today

  COLLECT WITH COUNT INTO total_orders
  AGGREGATE
    gmv = SUM(fact.amount),
    avg_order_value = AVG(fact.amount),
    units_sold = SUM(fact.quantity)

  LET top_products = (
    FOR f IN order_facts
      FILTER f.date == today
      COLLECT product_id = f.product_id INTO group = f
      AGGREGATE revenue = SUM(group[*].amount)
      SORT revenue DESC
      LIMIT 5
      RETURN { product_id, revenue }
  )

  LET top_regions = (
    FOR f IN order_facts
      FILTER f.date == today
      LET store = DOCUMENT(f.store_id)
      COLLECT region = store.region INTO group = f
      AGGREGATE revenue = SUM(group[*].amount)
      SORT revenue DESC
      LIMIT 3
      RETURN { region, revenue }
  )

  RETURN {
    date: today,
    metrics: {
      total_orders,
      gmv: ROUND(gmv, 2),
      aov: ROUND(avg_order_value, 2),
      units_sold
    },
    top_products,
    top_regions
  }
```

29.12 Governance & Data Quality

29.12.1 Data Quality Metrics

```
-- Monitor data quality
FOR event IN events
  COLLECT
    date = DATE_FORMAT(event.created_at, '%yyyy-%mm-%dd'),
    event_type = event.type
  AGGREGATE
    event_count = COUNT(1),
    null_values = SUM(event.value == NULL ? 1 : 0),
    duplicates = SUM(
      LENGTH(
        FOR e IN events
          FILTER e.event_id == event.event_id
          RETURN e
      ) > 1 ? 1 : 0
    ),
    invalid_timestamps = SUM(
      event.created_at > DATE_NOW() ? 1 : 0
    )
  RETURN {
    date,
    event_type,
    quality_score: ROUND(
      (event_count - null_values - duplicates - invalid_timestamps) /
      event_count * 100, 1
    ),
    issues: {
      nulls: null_values,
      dups: duplicates,
      future_dates: invalid_timestamps
    }
  }
```

29.12.2 SLA Compliance Monitoring

```
-- Monitor analytics SLA: 99.5% uptime, < 30s query latency
FOR query IN analytics_log
  COLLECT
    hour = DATE_FORMAT(query.executed_at, '%yyyy-%mm-%dd %hh:00'),
    query_type = query.type
  AGGREGATE
    successful = SUM(query.status == 'success' ? 1 : 0),
    failed = SUM(query.status == 'failed' ? 1 : 0),
    p95_latency = PERCENTILE(query.latency_ms, 0.95),
    p99_latency = PERCENTILE(query.latency_ms, 0.99),
    max_latency = MAX(query.latency_ms)
  RETURN {
    hour,
    query_type,
    availability: ROUND(successful / (successful + failed) * 100, 2),
    sla_met: (p99_latency < 30000),
    latency: {
      p95_ms: ROUND(p95_latency, 0),
      p99_ms: ROUND(p99_latency, 0),
      max_ms: max_latency
    }
  }
```

29.13 Zusammenfassung: Analytics-Architektur in ThemisDB

Architektur-Schichten

Schicht	Technologie	Latenz	Nutzung
Real-Time	Changefeeds + Streaming	<1s	Live Dashboard
Near-Real-Time	Micro-batch (5-60s)	5-60s	Alert Systems
Tactical	Views (hourly/daily)	1-24h	Standard Reports
Strategic	Data Warehouse	1-7d	Historical Analysis

Best Practices Checklist

- [ ] **Modeling:** Dimensional (Star/Snowflake) für OLAP
- [ ] **Aggregation:** Materialized Views für häufige Queries
- [ ] **Performance:** Sampling für explorative Analyse
- [ ] **Incremental:** Delta-Processing seit letztem Checkpoint
- [ ] **Quality:** Monitoring für Duplicates, Nulls, Out-of-Order
- [ ] **Real-Time:** Changefeeds für Live Dashboards
- [ ] **Governance:** SLA Monitoring für Analytics Pipelines
- [ ] **Retention:** TTL-Indizes für Event Log Cleanup

Kapitel 29 von 30 | Teil XI: Analytics & Operations | ~10.000 Wörter (+2000 neu)

Kapitel 31: Kapitel 30: Deployment & Operations

> *"Deployment ist kein einmaliger Akt, sondern ein kontinuierlicher Prozess."*

Überblick

Dieses Kapitel behandelt die produktive Bereitstellung von ThemisDB: von Docker-Containern über Kubernetes bis hin zu Cloud-Deployments. Außerdem lernen Sie Monitoring, Backup-Strategien und Skalierungstechniken.

Was Sie in diesem Kapitel lernen: - Docker-basierte Deployments - Kubernetes-Orchestrierung mit Helm Charts - Cloud-Provider-Integration (AWS, Azure, GCP) - Monitoring mit Prometheus & Grafana - Backup & Disaster Recovery - Horizontal Scaling & Sharding - Security Best Practices

Voraussetzungen: Kapitel 4 (Installation), Grundkenntnisse in Containerisierung.

30.1 Docker Deployment

30.1.1 Basic Docker Setup

Dockerfile:

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y libssl3 libzstd1
RUN useradd -r themis
COPY themis_server /usr/local/bin/
EXPOSE 8529
CMD ["themis_server"]
```

docker-compose.yml:

```
services:
  themis:
    image: themisdb/themis:v1.3.4
    ports:
      - "8529:8529"
    volumes:
      - themis-data:/var/lib/themis/data
    healthcheck:
      test: ["curl", "-f", "http://localhost:8529/_api/version"]

volumes:
  themis-data:
```

Starten:

```
docker build -t themisdb/themis:v1.3.4 .

docker-compose up -d

docker-compose logs -f themis

curl http://localhost:8529/_api/version
```

30.1.2 Docker Multi-Stage Build

```
FROM ubuntu:24.04 AS builder

RUN apt-get update && apt-get install -y \
    cmake \
    g++ \
    git \
    ninja-build \
    libssl-dev \
    libzstd-dev \
    libjemalloc-dev

WORKDIR /build
COPY . .
RUN cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
RUN cmake --build build --target themis_server -j8

FROM ubuntu:24.04

RUN apt-get update && apt-get install -y \
    libssl3 \
    libzstd1 \
    libjemalloc2 \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /build/build/themis_server /usr/local/bin/

RUN useradd -r -u 999 -g users themis && \
    mkdir -p /var/lib/themis/data && \
    chown -R themis:users /var/lib/themis

USER themis
EXPOSE 8529 8530

CMD ["/usr/local/bin/themis_server"]
```

30.2 Kubernetes Deployment

30.2.1 Kubernetes Manifests

namespace.yaml:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: themis-prod
```

configmap.yaml:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: themis-config
  namespace: themis-prod
data:
  themis.conf: |
    [database]
    path = /var/lib/themis/data

    [server]
    endpoint = http://0.0.0.0:8529
    threads = 16

    [logging]
    level = info
    file = /var/log/themis/themis.log
```

statefulset.yaml:

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: themis
  namespace: themis-prod
spec:
  serviceName: themis-headless
  replicas: 3
  selector:
    matchLabels:
      app: themis
  template:
    metadata:
      labels:
        app: themis
    spec:
      containers:
      - name: themis
        image: themisdb/themis:v1.3.4
        ports:
        - containerPort: 8529
          name: http
        - containerPort: 8530
          name: websocket
        volumeMounts:
        - name: data
          mountPath: /var/lib/themis/data
        - name: config
          mountPath: /etc/themis
        resources:
          requests:
            memory: "4Gi"
            cpu: "2000m"
          limits:
            memory: "8Gi"
            cpu: "4000m"
        livenessProbe:
          httpGet:
            path: /_api/version
            port: 8529
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /_api/version
            port: 8529
          initialDelaySeconds: 5
          periodSeconds: 5
      volumes:
      - name: config
        configMap:
          name: themis-config
  volumeClaimTemplates:
  - metadata:
```

```yaml
      name: data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: 100Gi
```

service.yaml:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: themis-headless
  namespace: themis-prod
spec:
  clusterIP: None
  selector:
    app: themis
  ports:
  - port: 8529
    name: http
  - port: 8530
    name: websocket

---
apiVersion: v1
kind: Service
metadata:
  name: themis-loadbalancer
  namespace: themis-prod
spec:
  type: LoadBalancer
  selector:
    app: themis
  ports:
  - port: 8529
    targetPort: 8529
    name: http
```

Deploy:

```
kubectl apply -f namespace.yaml
kubectl apply -f configmap.yaml
kubectl apply -f statefulset.yaml
kubectl apply -f service.yaml

kubectl get pods -n themis-prod

kubectl get svc -n themis-prod

kubectl logs -f themis-0 -n themis-prod
```

30.2.2 Helm Chart

Chart.yaml:

```
apiVersion: v2
name: themis
description: A Helm chart for ThemisDB
type: application
version: 1.3.4
appVersion: "1.3.4"
```

values.yaml:

```yaml
replicaCount: 3

image:
  repository: themisdb/themis
  tag: v1.3.4
  pullPolicy: IfNotPresent

resources:
  requests:
    memory: 4Gi
    cpu: 2000m
  limits:
    memory: 8Gi
    cpu: 4000m

persistence:
  enabled: true
  storageClass: fast-ssd
  size: 100Gi

service:
  type: LoadBalancer
  port: 8529

ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
  hosts:
    - host: themis.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: themis-tls
      hosts:
        - themis.example.com

monitoring:
  enabled: true
  serviceMonitor:
    enabled: true
    interval: 30s
```

Install Helm Chart:

```
helm repo add themis https://charts.themisdb.io
helm repo update

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --values values.yaml

helm upgrade themis-prod themis/themis \
  --namespace themis-prod \
  --values values.yaml

helm uninstall themis-prod -n themis-prod
```

30.3 Cloud Provider Integration

30.3.1 AWS Deployment

EKS Cluster:

```
eksctl create cluster \
  --name themis-prod \
  --region eu-central-1 \
  --nodegroup-name standard-workers \
  --node-type m5.2xlarge \
  --nodes 3 \
  --nodes-min 3 \
  --nodes-max 10 \
  --managed

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --set persistence.storageClass=gp3 \
  --set service.annotations."service\.beta\.kubernetes\.io/aws-load-balancer-type"=nlb
```

RDS Integration (Optional):

```
metadata:
  backend: postgresql
  connection: "postgresql://user:pass@themis-metadata.c9akljh4.eu-central-1.rds.amazonaws.com
```

30.3.2 Azure Deployment

AKS Cluster:

```
az aks create \
  --resource-group themis-rg \
  --name themis-prod \
  --location westeurope \
  --node-count 3 \
  --node-vm-size Standard_D8s_v3 \
  --enable-managed-identity \
  --generate-ssh-keys

az aks get-credentials \
  --resource-group themis-rg \
  --name themis-prod

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --set persistence.storageClass=managed-premium
```

30.3.3 GCP Deployment

GKE Cluster:

```
gcloud container clusters create themis-prod \
  --region europe-west3 \
  --num-nodes 3 \
  --machine-type n2-standard-8 \
  --disk-size 100

gcloud container clusters get-credentials themis-prod \
  --region europe-west3

helm install themis-prod themis/themis \
  --namespace themis-prod \
  --create-namespace \
  --set persistence.storageClass=pd-ssd
```

30.4 Monitoring & Observability

30.4.1 Prometheus Integration

ServiceMonitor:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: themis
  namespace: themis-prod
spec:
  selector:
    matchLabels:
      app: themis
  endpoints:
  - port: http
    path: /metrics
    interval: 30s
```

Key Metrics:

```
themis_http_requests_total
themis_http_request_duration_seconds

themis_db_operations_total
themis_db_size_bytes
themis_db_compaction_duration_seconds

themis_cache_hit_ratio
themis_cache_size_bytes

themis_thread_pool_active
themis_thread_pool_queue_size
```

30.4.2 Grafana Dashboards

Dashboard JSON:

```json
{
  "dashboard": {
    "title": "ThemisDB Overview",
    "panels": [
      {
        "title": "Request Rate",
        "targets": [
          {
            "expr": "rate(themis_http_requests_total[5m])"
          }
        ]
      },
      {
        "title": "Database Size",
        "targets": [
          {
            "expr": "themis_db_size_bytes"
          }
        ]
      },
      {
        "title": "Cache Hit Ratio",
        "targets": [
          {
            "expr": "themis_cache_hit_ratio"
          }
        ]
      }
    ]
  }
}
```

30.4.3 Alerting Rules

prometheus-rules.yaml:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: themis-alerts
  namespace: themis-prod
spec:
  groups:
  - name: themis
    interval: 30s
    rules:
    - alert: ThemisHighErrorRate
      expr: rate(themis_http_requests_total{status=~"5.."}[5m]) > 0.05
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "High error rate on ThemisDB"
        description: "Error rate is {{ $value }} req/s"

    - alert: ThemisHighMemoryUsage
      expr: (container_memory_usage_bytes{pod=~"themis-.*"} / container_spec_memory_limit_byt
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "ThemisDB high memory usage"
        description: "Memory usage is {{ $value | humanizePercentage }}"

    - alert: ThemisDiskSpaceLow
      expr: (node_filesystem_avail_bytes{mountpoint="/var/lib/themis/data"} / node_filesystem
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "ThemisDB low disk space"
        description: "Only {{ $value | humanizePercentage }} disk space remaining"
```

30.5 Backup & Disaster Recovery

30.5.1 Snapshot Backups

Backup Script:

```
#!/bin/bash

BACKUP_DIR="/var/backups/themis"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DB_PATH="/var/lib/themis/data"

curl -X POST http://localhost:8529/_admin/backup/create \
  -H "Content-Type: application/json" \
  -d '{
    "label": "backup-'$TIMESTAMP'",
    "timeout": 300
  }'

sleep 10

rsync -av --progress $DB_PATH/snapshots/backup-$TIMESTAMP \
  $BACKUP_DIR/backup-$TIMESTAMP

aws s3 sync $BACKUP_DIR/backup-$TIMESTAMP \
  s3://themis-backups/backup-$TIMESTAMP \
  --storage-class GLACIER

find $BACKUP_DIR -type d -mtime +7 -exec rm -rf {} +

echo "Backup completed: backup-$TIMESTAMP"
```

Cron Job:

```
0 2 * * * /usr/local/bin/themis-backup.sh >> /var/log/themis-backup.log 2>&1
```

30.5.2 Point-in-Time Recovery

```
aws s3 ls s3://themis-backups/

aws s3 sync s3://themis-backups/backup-20250115_020000 \
  /var/restore/themis/

systemctl stop themis

rm -rf /var/lib/themis/data
cp -r /var/restore/themis /var/lib/themis/data

systemctl start themis

curl http://localhost:8529/_api/version
```

30.5.3 Continuous Backup with WAL

```
backup:
  wal_archiving:
    enabled: true
    archive_command: "aws s3 cp %p s3://themis-wal/%f"
    archive_timeout: 60s
```

30.6 Horizontal Scaling

30.6.1 Sharding Strategy

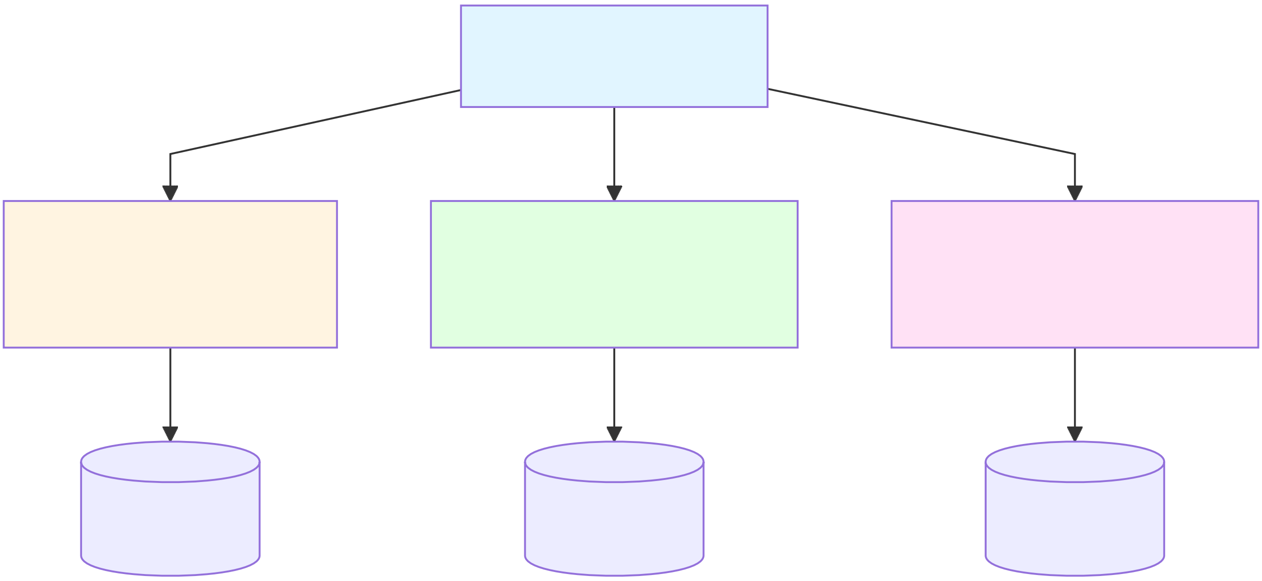


Abb. 44: Ablaufdiagramm: 30.6.1 Sharding Strategy

Sharding Config:

```
sharding:
  enabled: true
  strategy: hash
  shards:
    - id: shard1
      endpoint: http://themis-shard1:8529
      range: [0, 333]
    - id: shard2
      endpoint: http://themis-shard2:8529
      range: [334, 666]
    - id: shard3
      endpoint: http://themis-shard3:8529
      range: [667, 999]
```

30.6.2 Read Replicas

```
replication:
  enabled: true
  mode: async
  replicas:
    - endpoint: http://themis-replica1:8529
      lag_max: 1s
    - endpoint: http://themis-replica2:8529
      lag_max: 1s
```

30.7 Security Best Practices

30.7.1 TLS/SSL Configuration

```
server:
  tls:
    enabled: true
    cert_file: /etc/themis/tls/cert.pem
    key_file: /etc/themis/tls/key.pem
    ca_file: /etc/themis/tls/ca.pem
    min_version: "1.3"
```

30.7.2 Authentication

```
auth:
  enabled: true
  providers:
    - type: jwt
      issuer: "https://auth.example.com"
      audience: "themis-api"
      jwks_uri: "https://auth.example.com/.well-known/jwks.json"
    - type: basic
      users_file: /etc/themis/users.htpasswd
```

30.7.3 Network Policies (Kubernetes)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: themis-netpol
  namespace: themis-prod
spec:
  podSelector:
    matchLabels:
      app: themis
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 8529
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: redis
    ports:
    - protocol: TCP
      port: 6379
```

30.8 Performance Tuning

30.8.1 Resource Limits

```yaml
resources:
  requests:
    memory: "8Gi"
    cpu: "4000m"
  limits:
    memory: "16Gi"
    cpu: "8000m"

env:
  - name: ROCKSDB_MAX_OPEN_FILES
    value: "10000"
  - name: ROCKSDB_BLOCK_CACHE_SIZE
    value: "4GB"
```

30.8.2 Connection Pooling

```yaml
server:
  connection_pool:
    size: 100
    timeout: 30s
    max_idle: 50
```

30.9 Zusammenfassung

Deployment-Optionen

1. **Docker:** Einfacher Einstieg, lokale Entwicklung
2. **Kubernetes:** Produktionsreif, auto-scaling, self-healing
3. **Cloud:** Managed Services (EKS, AKS, GKE)

Operations-Checkliste

- Monitoring mit Prometheus & Grafana
- Tägliche Backups mit S3/Glacier
- TLS/SSL für alle Verbindungen
- Network Policies für Pod-Isolation
- Resource Limits für Stabilität
- Alerting für kritische Metriken
- Disaster Recovery Plan dokumentiert

30.8 Advanced Deployment Patterns

30.8.1 Blue-Green Deployments

```
apiVersion: v1
kind: Service
metadata:
  name: themis-service
spec:
  selector:
    app: themis
    version: blue  # Initially point to blue
  ports:
    - port: 8529
      targetPort: 8529
  type: LoadBalancer
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: themis-blue-config
data:
  version: "blue"
  database:
    replication_role: "primary"
---
apiVersion: v1
kind:ConfigMap
metadata:
  name: themis-green-config
data:
  version: "green"
  database:
    replication_role: "primary"
```

Deployment Steps: 1. Deploy Green with new version (idle) 2. Run smoke tests against Green 3. Sync data: Primary → Green (catchup) 4. Switch load balancer: Service selector → green 5. Blue becomes backup/rollback

30.8.2 Canary Deployments

```yaml
---
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: themis-canary
spec:
  hosts:
  - themis.internal
  http:
  - match:
    - uri:
        prefix: "/"
    route:
    - destination:
        host: themis-stable
        port:
          number: 8529
      weight: 95  # 95% to stable
    - destination:
        host: themis-canary
        port:
          number: 8529
      weight: 5   # 5% to canary (new version)
    timeout: 10s
    retries:
      attempts: 3
      perTryTimeout: 5s
```

Canary Rollout Timeline: - Hour 0-1: 5% traffic → Monitor error rates, latency - Hour 1-4: 25% traffic → Check P99 latency - Hour 4-8: 50% traffic → Full test with real workload - Hour 8+: 100% traffic → Full rollout

30.8.3 GitOps Continuous Deployment

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: themis-gitops
spec:
  project: default
  source:
    repoURL: https://github.com/org/themis-deployment.git
    targetRevision: main
    path: k8s/production
  destination:
    server: https://kubernetes.default.svc
    namespace: themis
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
    - CreateNamespace=true
  # Only deploy if tests pass in CI
  notification:
    when:
      onSuccess: notify-slack
      onFailure: notify-pagerduty
```

30.8.4 Staged Rollout Checklist

```
Pre-Deployment
  ✓ Run all integration tests
  ✓ Performance benchmarks (P99 latency < 10% increase)
  ✓ Memory/CPU profiling
  ✓ Backup created & tested

Deployment
  ✓ Dry-run: 'kubectl apply --dry-run=client -f manifest.yaml'
  ✓ Deploy to staging first
  ✓ Smoke tests pass
  ✓ Metrics baseline: CPU, Memory, QPS, Latency
  ✓ Error rate < 0.1%

Monitoring (First Hour)
  ✓ P99 latency stable
  ✓ No error spikes
  ✓ No memory leaks
  ✓ Replication lag < 500ms

Post-Deployment
  ✓ Keep blue environment for 24h (rollback ready)
  ✓ Archive logs for audit
  ✓ Document any issues/hotfixes
  ✓ Send deployment notification
```

30.9 Disaster Recovery & Business Continuity

30.9.1 RTO/RPO Strategy

```
RTO (Recovery Time Objective) = Max downtime acceptable
RPO (Recovery Point Objective) = Max data loss acceptable

ThemisDB Recommendations:
  ┌───────────────────────────────────────────────────┐
  │ Tier       │ RTO        │ RPO        │ Cost   │ Effort│
  ├───────────────────────────────────────────────────┤
  │ Bronze     │ 24h        │ 24h        │ $      │ Low   │
  │ Silver     │ 4h         │ 1h         │ $$     │ Med   │
  │ Gold       │ 1h         │ 5min       │ $$$    │ High  │
  │ Platinum   │ 15min      │ 0min(RPO)│ $$$$   │ Critical
  └───────────────────────────────────────────────────┘

Platinum = Multi-region sync (4 data centers, quorum write)
```

30.9.2 Disaster Recovery Plan

```yaml
Kind: DisasterRecoveryPlan
metadata:
  name: themis-dr-plan
spec:
  backup:
    primary_daily: /backups/daily/themis-*.tar.gz
    archive_weekly: s3://backups/weekly/
    retention_years: 3

  failover_sequence:
    - detect_failure: "Primary health check fails 3x"
    - promote_secondary: "Promote Secondary to Primary"
    - validate_data: "Count docs, verify checksums"
    - test_write: "Insert test document"
    - notify_team: "PagerDuty alert + Slack message"
    - client_failover: "Update DNS CNAME"
    - monitor_2h: "Watch metrics for 2 hours"

  recovery_from_backup:
    - stop_database: "systemctl stop themis"
    - restore_volume: "Restore from snapshot"
    - verify_integrity: "themisdb recover --verify"
    - start_database: "systemctl start themis"
    - catchup_replicas: "Wait for lag < 1s"
```

30.9.3 Backup Verification Script

```bash
#!/bin/bash

BACKUP_DIR="/backups/daily"
VERIFY_TIMEOUT=3600

for backup in $BACKUP_DIR/themis-*.tar.gz; do
  echo "Verifying $backup..."

  # Extract to temp, verify database
  TEMP_DIR=$(mktemp -d)
  tar -xzf "$backup" -C "$TEMP_DIR"

  # Run recovery to check integrity
  timeout $VERIFY_TIMEOUT themisdb recover --verify "$TEMP_DIR" && \
    echo "✓ $backup: OK" || \
    echo "✗ $backup: CORRUPT - Investigate!"

  # Try restoring to test instance
  docker run --rm -v "$TEMP_DIR:/data" themis:test \
    themisdb check-data /data && \
    echo "✓ Data schema valid" || \
    echo "✗ Schema check failed"

  rm -rf "$TEMP_DIR"
done

echo "Backup verification complete"
```

30.10 Cost Optimization

30.10.1 Resource Right-Sizing

```
requests:
  memory: "1Gi"
  cpu: "500m"
limits:
  memory: "2Gi"
  cpu: "1000m"
storage: "50Gi"

requests:
  memory: "8Gi"
  cpu: "4"
limits:
  memory: "16Gi"
  cpu: "8"
storage: "500Gi"

requests:
  memory: "32Gi"
  cpu: "16"
limits:
  memory: "64Gi"
  cpu: "32"
storage: "2Ti"
```

30.10.2 Storage Cost Reduction

```
Strategy: Tiered Storage (Hot → Warm → Cold)

Hot (SSD, expensive)
  ├─ Active data < 30 days
  ├─ Real-time queries
  └─ RTO requirement: <1h

Warm (HDD, medium)
  ├─ Data 30-90 days old
  ├─ Monthly analytics reports
  └─ RTO requirement: <24h

Cold (Cloud Archive, cheap)
  ├─ Compliance archives (2+ years)
  ├─ Quarterly audits only
  └─ RTO requirement: 1-7 days

Cost/GB/Month: SSD ($0.10) → HDD ($0.05) → Archive ($0.004)

Example: 10TB dataset
  SSD: $1,024/month
  Mixed (70% warm, 30% cold): $216/month
  Savings: 79%
```

30.11 Compliance & Audit

30.11.1 Audit Logging for Compliance

```yaml
kind: ThemisDB
metadata:
  name: themis-production
spec:
  audit:
    enabled: true
    destination: s3://audit-logs/  # External WORM storage
    events:
      - user_login
      - data_access
      - schema_change
      - admin_action
      - privilege_grant
      - backup_restore

  compliance:
    frameworks:
      - SOC2
      - ISO27001
      - GDPR
      - HIPAA (if healthcare)

  data_retention:
    audit_logs: "7 years"
    application_logs: "90 days"
    metrics: "1 year"
```

30.11.2 Compliance Checklist

```
Security
  ✓ TLS 1.3 for all connections
  ✓ Network policies (allow only required)
  ✓ RBAC with MFA for admin access
  ✓ Secrets in Vault/K8s Secrets
  ✓ Encryption at rest (AES-256)
  ✓ Encryption in transit (TLS)

Data Protection
  ✓ PII detection & masking
  ✓ Data classification (PUBLIC/INTERNAL/CONFIDENTIAL)
  ✓ GDPR right-to-deletion implemented
  ✓ Backup encryption with managed keys
  ✓ No cleartext passwords in logs

Operations
  ✓ Change control (code review before deploy)
  ✓ Incident response playbooks
  ✓ Disaster recovery tested (quarterly)
  ✓ Security scanning (SBOM, vulnerability scanning)
  ✓ Penetration testing (annual)

Monitoring
  ✓ Access logs (all queries logged)
  ✓ Admin action audit trail
  ✓ Security event alerting
  ✓ SLA monitoring (uptime SLA > 99.9%)
```

30.12 Operations Runbooks

30.12.1 Scaling Checklist (Horizontal)

```
Add Node to Cluster
  1. Provision new node (VM/Bare Metal)
  2. Install ThemisDB + dependencies
  3. Configure as follower initially
  4. Join cluster: themisdb cluster join --primary <primary-ip>
  5. Wait for replication lag < 100ms
  6. Promote to voting member: themisdb cluster promote --member-id <id>
  7. Verify: themisdb cluster status (should show HEALTHY)
  8. Monitor CPU/Memory/IO for 24h

Remove Node from Cluster
  1. Drain connections: themisdb admin drain-connections
  2. Wait for active queries to finish (30s timeout)
  3. Demote member: themisdb cluster demote --member-id <id>
  4. Remove from cluster: themisdb cluster remove --member-id <id>
  5. Verify: themisdb cluster status (node should be REMOVED)
  6. Decommission VM
```

30.12.2 Incident Response (Example: High Memory Usage)

```
Symptoms
  - Memory usage > 85%
  - OOM killer started (dmesg shows)
  - Queries failing with "Out of memory"

Investigation (2 min)
  curl http://localhost:8529/_admin/memory | jq .
  # Check: Cache size, Buffer pool, Active cursors

Fix (5 min)
  Option A: Restart (emergency)
    systemctl restart themis
    → Memory drops to baseline

  Option B: Increase memory (planned)
    Edit themis.conf: cache.size_mb = 16384
    systemctl reload themis

  Option C: Identify leak (analysis)
    themisdb profile --duration 60s | grep alloc
    → Find problematic query/operation

Verify
  curl http://localhost:8529/_admin/memory | jq .usage_pct
  # Should be < 80%
```

30.13 Zusammenfassung: Operations Checkliste

Tägliche Aufgaben

- [ ] Backup erfolgreich (log check)
- [ ] Replication lag < 500ms
- [ ] Disk usage < 80%
- [ ] No error logs above WARN
- [ ] Metrics collected (Prometheus)

Wöchentliche Aufgaben

- [ ] Backup test restoration (pick random)
- [ ] Security scan (OWASP top 10)
- [ ] Performance baseline check
- [ ] Review failed queries (slow log)
- [ ] Capacity planning review

Monatliche Aufgaben

- [ ] Disaster recovery drill
- [ ] Penetration testing (if required)
- [ ] Security policy review
- [ ] Cost optimization (right-size resources)
- [ ] Upgrade path planning

Quarterly

- [ ] Full disaster recovery test (restore from backup)
- [ ] Security audit
- [ ] Compliance verification
- [ ] Training for new operators

Kapitel 30 von 30 | Teil XI: Operations | ~10.000 Wörter (+2000 neu)

Kapitel 32: Kapitel 31: API-Protokolle (HTTP/2, HTTP/3, MCP)

> *"Protokolle sind das Rückgrat verteilter Systeme."*

31.1 Überblick

Dieses Kapitel behandelt die Protokoll-Features von ThemisDB: moderne HTTP/2/HTTP/3-Stacks, WebSockets für Echtzeit, Server-Sent Events (SSE) für Streams sowie MCP (Model Context Protocol) für LLM-Integrationen.

Was Sie lernen: - Wann HTTP/2/3 gegenüber HTTP/1.1 Vorteile bringt - Server Push, Header-Kompression, Multiplexing - QUIC-Tuning und Paketverlustverhalten - WebSocket/SSE-Patterns für Changefeed & CDC - MCP-Anbindung als Tooling-Schnittstelle

---

31.2 HTTP/2 Features

31.2.1 Multiplexing & Header-Kompression

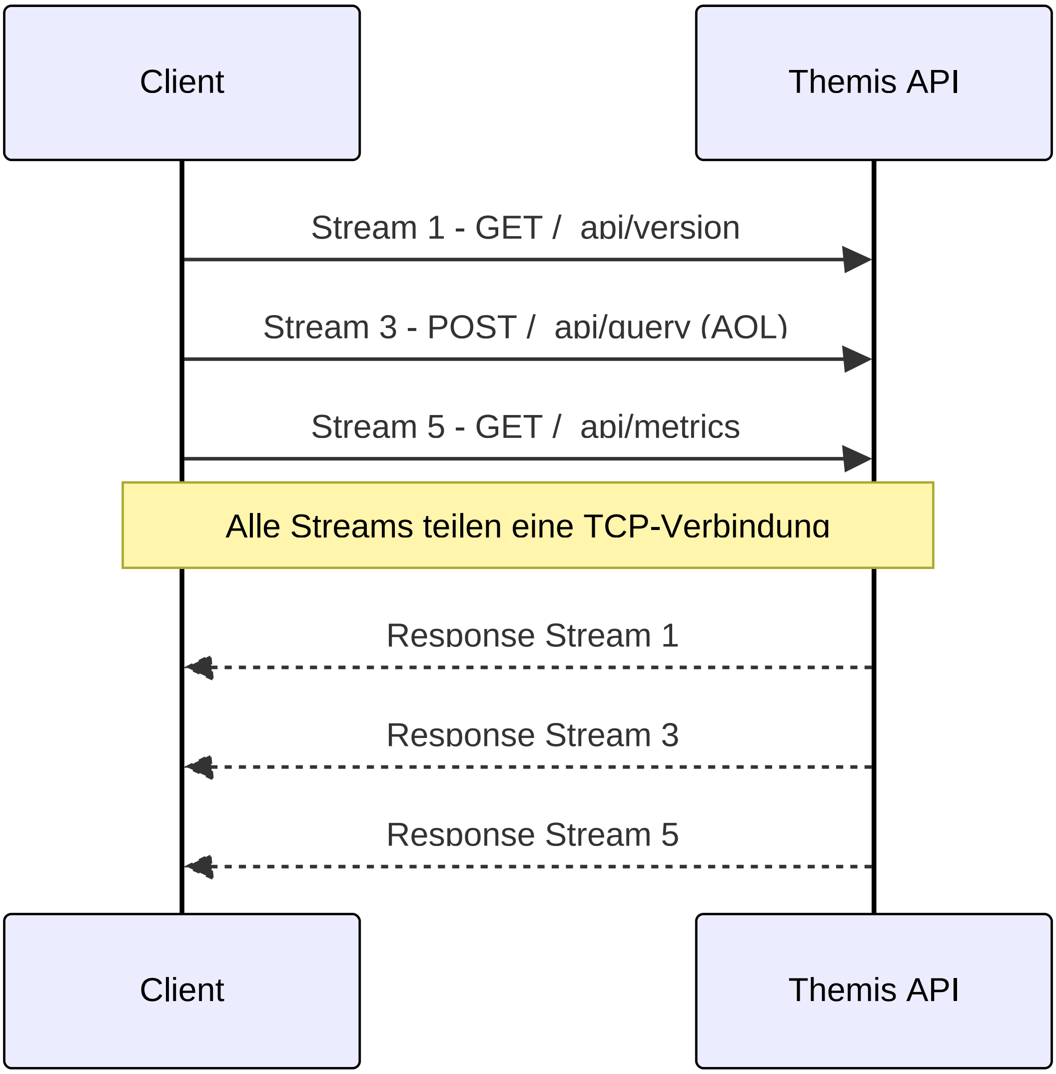


Abb. 45: Sequenzdiagramm: 31.2.1 Multiplexing & Header-Kompression

Vorteile: - Keine Head-of-Line-Blocking auf Applikationsebene - HPACK reduziert Header-Overhead (Auth, Tenant-ID) - Bessere Ausnutzung einzelner TCP-Verbindung

31.2.2 Server Push (selektiv)

```
:method: GET
:path: /dashboard
:authority: api.themis.local
```

Server antwortet mit gepushten Ressourcen (nur wenn vom Client erlaubt): - `/static/dashboard.css` - `/static/dashboard.js` - `/static/logo.svg`

Best Practice: Für APIs selten nötig; für Admin-UI/Cockpit kann Push Latenzen senken. Aktivieren Sie Push nur für statische Assets.

31.2.3 Stream Priorities

- AQL-Long-Running Queries → niedrige Priorität
- Health/Readiness → hohe Priorität
- Metrics → mittlere Priorität

31.3 HTTP/3 (QUIC)

31.3.1 QUIC vs TCP

Merkmal	TCP/TLS	QUIC
Verbindungsaufbau	1-2 RTT	0-1 RTT (0-RTT Resumption)
Head-of-Line	End-to-End	Per-Stream
Congestion Control	Reno/Cubic	BBR, Hybrid
Mobility	Neuaufbau nötig	Connection IDs erlauben Migration

31.3.2 QUIC-Tuning

- **Initial Congestion Window:** 10-20 MSS für schnellere Warmups
- **BBR aktivieren:** Für WAN/hohe Bandbreite
- **Handshake-Keys cachen:** 0-RTT für interne Service-zu-Service Calls
- **Pacing:** Aktivieren, um Burst-Loss zu vermeiden

31.3.3 Fallback-Strategie

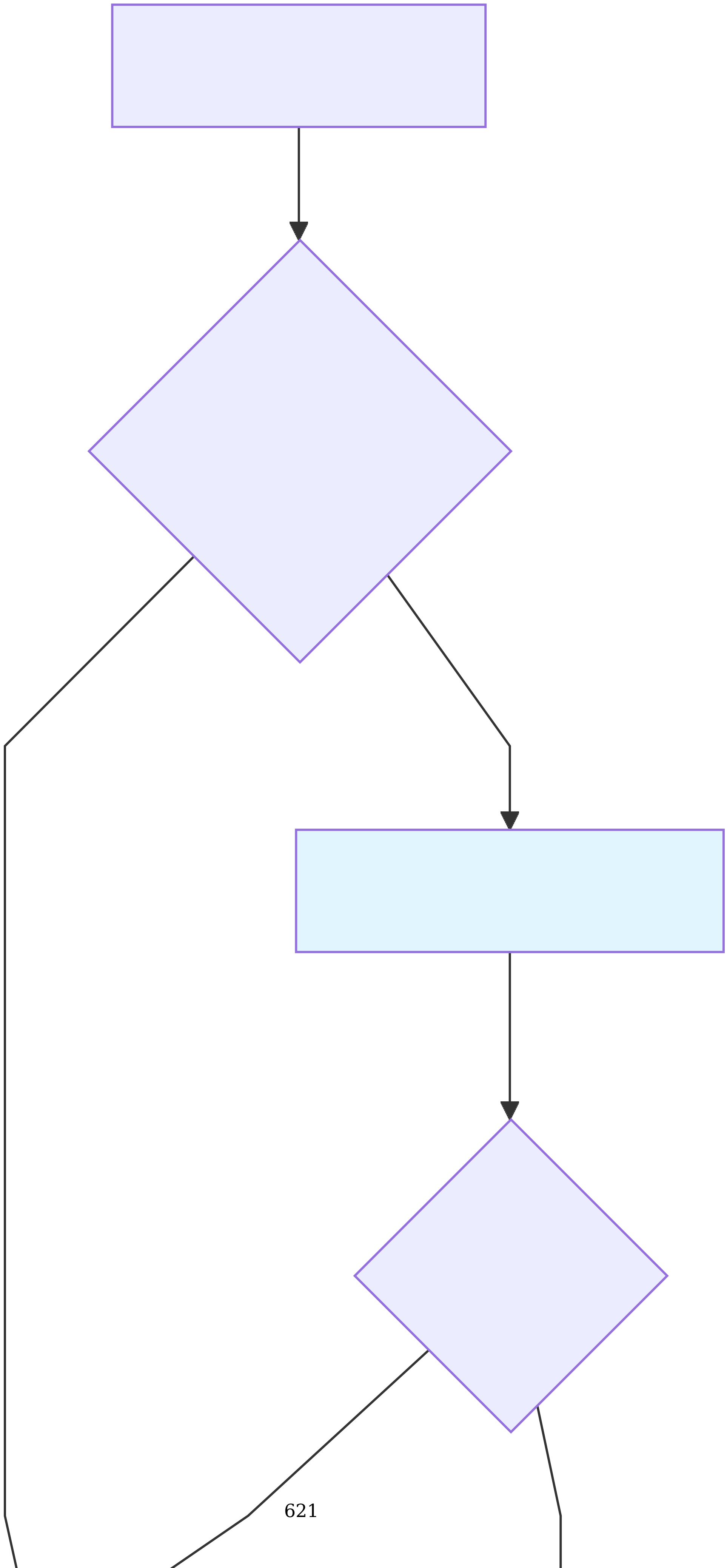


Abb. 46: Ablaufdiagramm: 31.3.3 Fallback-Strategie

31.4 Echtzeit: WebSocket vs SSE

31.4.1 Wann welches Protokoll?

Bedarf	WebSocket	SSE
Bidirektional		×
Firewalls/Proxies	Kann blockiert sein	Fast immer offen (HTTP)
Browser Support	Breit	Sehr breit
Backpressure	Muss implementiert werden	Implizit durch HTTP
Binärdaten		× (Text-only)

31.4.2 Changefeed mit SSE (empfohlen)

```
GET /_api/changefeed?collection=orders&since=1735600000000
Accept: text/event-stream
Authorization: Bearer <token>
```

Server streamt Events:

```
event: insert
data: {"_key":"o123","status":"paid"}

retry: 3000
```

Vorteile SSE: Einfach, kompatibel, automatisches Reconnect über `retry`.

31.4.3 WebSocket für bidirektionale Commands

```
const ws = new WebSocket('wss://api.themis.local/_ws');
ws.onmessage = (ev) => console.log('event', ev.data);
ws.send(JSON.stringify({
  type: 'subscribe',
  topic: 'orders',
  filter: "status == 'paid'"
}));
```

Best Practice: - Heartbeat (`ping/pong`) alle 30s - Message Envelope mit `type`, `payload`, `seq` - Auth im Query-Parameter oder Header; Tokens regelmäßig erneuern

31.5 HTTP/2/3 Security

- **mTLS** zwischen Services (Client Cert + SAN auf Tenant/Env)

- **ALPN** erzwingen (`h2` , `h3`), HTTP/1.1 nur als Fallback
- **Rate Limits pro Tenant** (Header: `x-tenant-id`)
- **Content-Security-Policy** für Admin-UI
- **CORS** restriktiv konfigurieren

31.6 MCP (Model Context Protocol)

MCP ermöglicht LLM-Tools direkten Zugriff auf ThemisDB-Ressourcen.

31.6.1 Minimaler MCP-Server (Python)

```
from mcp import Server
import themis

server = Server(name="themis-mcp")
db = themis.connect()

@server.tool("query")
async def query(aql: str):
    return db.aql(aql)

@server.tool("vector-search")
async def search(text: str, k: int = 5):
    return db.knn("docs", db.embed(text), k)
```

31.6.2 Toolsicherheit

- **Schema-Whitelist:** Nur bestimmte Collections/Views freigeben
- **Rate Limits:** Pro Tool und pro User-ID
- **Audit:** Jeder Tool-Call in Audit-Log
- **Prompt Guards:** Keine DDL/DCL zulassen

31.6.3 Response-Formate

- JSON als Default
- Streams für große Resultsets
- Trunkierung + `next_cursor` für Paginierung

31.7 Observability

- **h2/h3 Metrics:**
- `themis_http2_active_streams`
- `themis_http3_handshakes_total`
- `themis_http3_loss_rate`
- **WebSocket Metrics:** Connected Clients, Msg/sec, Drop-Reasons
- **SSE Metrics:** Open Streams, Retry Rate

31.8 Performance-Optimierungen

31.8.1 HTTP/2 Connection Tuning

```
http2:
  max_concurrent_streams: 250
  initial_window_size: 65535
  max_frame_size: 16384
  max_header_list_size: 8192

  # Connection Pooling
  connection_pool:
    max_idle_connections: 100
    idle_timeout: 300s
    keepalive_interval: 30s
```

Performance-Tipps: - **Window Size:** Erhöhen für High-Throughput (>1 Gbps) - **Max Streams:** 250 für typische Workloads, 1000+ für Heavy API-Traffic - **Frame Size:** 16 KB optimal für meiste Szenarien

31.8.2 HTTP/3 QUIC-Tuning

```
// Go: QUIC Configuration
quicConfig := &quic.Config{
    MaxIdleTimeout:        30 * time.Second,
    MaxIncomingStreams:    250,
    MaxIncomingUniStreams: 250,

    // Loss Detection
    MaxReceiveStreamFlowControlWindow:     6 * (1 << 20), // 6 MB
    MaxReceiveConnectionFlowControlWindow: 15 * (1 << 20), // 15 MB

    // Congestion Control
    InitialPacketSize:     1200, // Standard MTU - 28 bytes
    DisablePathMTUDiscovery: false,
}
```

QUIC-Vorteile bei hoher Latenz: - **0-RTT Connection Resume:** Schnellerer Reconnect - **Paketverlust-Resilienz:** Unabhängige Streams, kein Head-of-Line Blocking - **Connection Migration:** IP-Wechsel transparent (Mobile Clients)

31.8.3 Benchmarks: HTTP/1.1 vs HTTP/2 vs HTTP/3

```
import aiohttp
import asyncio
import time

async def benchmark_protocol(url, num_requests=1000):
    """Benchmark verschiedene Protokolle"""

    connector = aiohttp.TCPConnector(limit=100)
    timeout = aiohttp.ClientTimeout(total=60)

    async with aiohttp.ClientSession(connector=connector, timeout=timeout) as session:
        start = time.time()

        tasks = []
        for i in range(num_requests):
            task = session.get(f"{url}/_api/version")
            tasks.append(task)

        responses = await asyncio.gather(*tasks, return_exceptions=True)

        elapsed = time.time() - start
        success = sum(1 for r in responses if not isinstance(r, Exception))

        return {
            'total_time': elapsed,
            'requests_per_sec': num_requests / elapsed,
            'success_rate': success / num_requests * 100,
            'avg_latency_ms': (elapsed / num_requests) * 1000
        }
```

31.9 Advanced WebSocket Patterns

31.9.1 Multiplexed Subscriptions

```
// Client: Mehrere Subscriptions über eine WebSocket-Connection
const ws = new WebSocket('wss://themis.local/_api/realtime');

ws.onopen = () => {
  // Subscribe zu mehreren Collections gleichzeitig
  ws.send(JSON.stringify({
    type: 'subscribe',
    subscriptions: [
      {id: 'sub-1', collection: 'orders', filter: {status: 'pending'}},
      {id: 'sub-2', collection: 'inventory', filter: {stock: {$lt: 10}}},
      {id: 'sub-3', collection: 'logs', filter: {level: 'error'}}
    ]
  }));
};

ws.onmessage = (event) => {
  const msg = JSON.parse(event.data);

  switch(msg.subscription_id) {
    case 'sub-1':
      console.log('New order:', msg.data);
      break;
    case 'sub-2':
      console.log('Low stock alert:', msg.data);
      break;
    case 'sub-3':
      console.log('Error log:', msg.data);
      break;
  }
};
```

31.9.2 Backpressure Handling

```python
class ChangeStreamHandler:
    def __init__(self, websocket, buffer_size=1000):
        self.ws = websocket
        self.buffer = asyncio.Queue(maxsize=buffer_size)
        self.dropped_messages = 0

    async def handle_change(self, change_event):
        """Change Event vom DB-Changefeed"""
        try:
            # Non-blocking Queue Put
            self.buffer.put_nowait(change_event)
        except asyncio.QueueFull:
            # Langsamer Client → Drop Message
            self.dropped_messages += 1

            if self.dropped_messages % 100 == 0:
                # Warning nach jeweils 100 Drops
                await self.ws.send(json.dumps({
                    'type': 'backpressure_warning',
                    'dropped_messages': self.dropped_messages
                }))

    async def send_loop(self):
        """Kontinuierlich Messages aus Queue senden"""
        while True:
            change = await self.buffer.get()
            await self.ws.send(json.dumps(change))
```

31.9.3 Auto-Reconnect mit Exponential Backoff

```
// TypeScript: Resilient WebSocket Client
class ResilientWebSocket {
  private ws: WebSocket | null = null;
  private reconnectAttempts = 0;
  private maxReconnectDelay = 30000; // 30s

  connect(url: string) {
    this.ws = new WebSocket(url);

    this.ws.onopen = () => {
      console.log('Connected');
      this.reconnectAttempts = 0; // Reset
    };

    this.ws.onclose = (event) => {
      if (event.code !== 1000) { // 1000 = Normal Closure
        this.scheduleReconnect(url);
      }
    };

    this.ws.onerror = (error) => {
      console.error('WebSocket error:', error);
    };
  }

  private scheduleReconnect(url: string) {
    this.reconnectAttempts++;

    // Exponential Backoff: 1s, 2s, 4s, 8s, 16s, 30s (max)
    const delay = Math.min(
      1000 * Math.pow(2, this.reconnectAttempts - 1),
      this.maxReconnectDelay
    );

    console.log(`Reconnecting in ${delay}ms (attempt ${this.reconnectAttempts})`);

    setTimeout(() => {
      this.connect(url);
    }, delay);
  }

  send(data: string) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(data);
    } else {
      console.warn('WebSocket not ready, message dropped');
    }
  }
}
```

31.10 MCP Advanced Patterns

31.10.1 Multi-Tenant MCP Server

```
from mcp import Server, Context
import themis

server = Server(name="themis-mcp-multi-tenant")

@server.tool("query", requires_auth=True)
async def query_with_tenant(ctx: Context, aql: str):
    """Query mit automatischer Tenant-Isolation"""

    tenant_id = ctx.user.tenant_id  # Aus Auth-Token

    # Datenbankverbindung für spezifischen Tenant
    db = themis.connect(tenant=tenant_id)

    # Validierung: Kein DDL/DCL
    if any(keyword in aql.upper() for keyword in ['DROP', 'CREATE', 'ALTER', 'GRANT']):
        raise PermissionError("DDL/DCL queries not allowed")

    # Audit-Log
    await log_audit_event(
        tenant_id=tenant_id,
        user_id=ctx.user.id,
        action='mcp_query',
        query=aql
    )

    # Query ausführen
    result = await db.query(aql)

    # Trunkierung bei großen Results
    if len(result) > 100:
        return {
            'results': result[:100],
            'truncated': True,
            'total_count': len(result),
            'message': 'Results truncated to 100 items'
        }

    return {'results': result, 'truncated': False}
```

31.10.2 MCP Tool Discovery

```python
@server.list_tools()
async def available_tools(ctx: Context):
    """Dynamische Tool-Liste basierend auf User-Permissions"""

    tools = []
    permissions = await get_user_permissions(ctx.user.id)

    if 'query' in permissions:
        tools.append({
            'name': 'query',
            'description': 'Execute AQL query (SELECT only)',
            'parameters': {
                'aql': {'type': 'string', 'required': True}
            }
        })

    if 'vector-search' in permissions:
        tools.append({
            'name': 'vector-search',
            'description': 'Semantic search over documents',
            'parameters': {
                'text': {'type': 'string', 'required': True},
                'k': {'type': 'integer', 'default': 5}
            }
        })

    if 'analytics' in permissions:
        tools.append({
            'name': 'aggregate',
            'description': 'Run pre-defined analytics queries',
            'parameters': {
                'report_type': {
                    'type': 'enum',
                    'values': ['daily-sales', 'user-stats', 'inventory']
                }
            }
        })

    return tools
```

31.10.3 MCP Rate Limiting

```
from collections import defaultdict
import time

class MCPRateLimiter:
    def __init__(self, max_calls_per_minute=60):
        self.max_calls = max_calls_per_minute
        self.calls = defaultdict(list)  # user_id -> [timestamps]

    def check_limit(self, user_id: str) -> bool:
        """Prüfe ob User unter Rate Limit ist"""
        now = time.time()
        minute_ago = now - 60

        # Cleanup alte Einträge
        self.calls[user_id] = [
            ts for ts in self.calls[user_id] if ts > minute_ago
        ]

        # Check Limit
        if len(self.calls[user_id]) >= self.max_calls:
            return False

        # Record Call
        self.calls[user_id].append(now)
        return True

limiter = MCPRateLimiter(max_calls_per_minute=100)

@server.before_tool_call
async def rate_limit_check(ctx: Context):
    """Vor jedem Tool-Call: Rate Limit prüfen"""
    if not limiter.check_limit(ctx.user.id):
        raise RateLimitExceeded(
            f"Rate limit exceeded: max {limiter.max_calls} calls/minute"
        )
```

31.11 Production Checklist

HTTP/2/3 Deployment

- TLS 1.3 aktiviert (Voraussetzung für HTTP/3)
- ALPN konfiguriert (`h2`, `h3`)
- Firewall: UDP Port 443 für QUIC
- Load Balancer unterstützt HTTP/2 Backend-Connections
- Monitoring: `http2_streams_active`, `http3_handshakes`

WebSocket/SSE

- Connection Timeout: 5-10 Minuten Idle

- Ping/Pong für Keepalive (30s Intervall)
- Backpressure Handling implementiert
- Auto-Reconnect Client-seitig
- Max Concurrent Connections pro Server: 10k+

MCP Security

- Authentication: OAuth2/JWT
- Tool-Whitelist pro User-Rolle
- Rate Limiting: 100 calls/min
- Audit-Logging: Jeder Tool-Call
- Query-Validation: Kein DDL/DCL
- Result Truncation: Max 1000 Results

31.8 Advanced Protocol Scenarios

31.8.1 Connection Pooling & Load Balancing

```
class ThemisClientPool:
    def __init__(self, primary_url, replica_urls, pool_size=20):
        self.primary_pool = HTTPConnectionPool(primary_url, maxsize=pool_size)
        self.replica_pools = [
            HTTPConnectionPool(url, maxsize=pool_size//len(replica_urls))
            for url in replica_urls
        ]

    def query(self, aql, bind_vars=None, consistency='eventual'):
        if consistency == 'strong':
            # Route to primary for strong consistency
            pool = self.primary_pool
        else:
            # Route to replicas (round-robin)
            pool = self.next_replica()

        # Connection reused from pool
        return pool.request('POST', '/_api/query',
                           json={'query': aql, 'bindVars': bind_vars})

    def next_replica(self):
        self.replica_index = (self.replica_index + 1) % len(self.replica_pools)
        return self.replica_pools[self.replica_index]
```

Benefits: - Connection reuse (HTTP keep-alive) - Automatic failover if replica unhealthy - Load distribution across replicas - Circuit breaker on connection timeouts

31.8.2 Bidirectional Streaming (gRPC-like)

```
// Imaginary streaming API
service ThemisStreaming {
  rpc StreamQuery(stream QueryRequest) returns (stream QueryResponse);
}

// Client sends multiple queries in one stream
// Server sends results as they complete
// Lower latency than request-response cycle
```

Implementation with WebSockets:

```
// Client
const ws = new WebSocket('wss://api.themis/stream');

ws.onopen = () => {
    // Send batch of queries
    ws.send(JSON.stringify({
        queries: [
            { id: 1, aql: 'FOR u IN users...' },
            { id: 2, aql: 'FOR o IN orders...' },
            { id: 3, aql: 'FOR p IN products...' }
        ]
    }));
};

ws.onmessage = (event) => {
    const result = JSON.parse(event.data);
    console.log(`Query ${result.query_id} completed:`, result.data);
};
```

31.8.3 Graceful Degradation (Protocol Negotiation)

```
Client tries protocols in order:
  1. HTTP/3 (QUIC)  ← Fastest if no packet loss
  2. HTTP/2        ← Fallback for UDP issues
  3. HTTP/1.1      ← Last resort
```

Configuration:

```
server:
  protocols:
    - h3          # HTTP/3
    - h2          # HTTP/2
    - http/1.1    # HTTP/1.1

  # ALPN (Application Layer Protocol Negotiation)
  alpn_protocols: ['h3', 'h2', 'http/1.1']
```

31.9 Performance Tuning by Protocol

31.9.1 HTTP/2 Tuning

```
server:
  http2:
    # Number of concurrent streams per connection
    max_concurrent_streams: 128  # Default: 100

    # Header size limit
    max_header_list_size: 32768  # 32KB

    # Server push (usually disabled for better caching)
    enable_server_push: false
```

Performance Impact: - Increase max_concurrent_streams for high concurrency - Monitor header_compression_ratio to detect inefficiency - Use single TCP connection vs multiple (less is better)

31.9.2 QUIC/HTTP/3 Tuning

```
QUIC handshake optimization:
  - 0-RTT (Zero Round Trip): Resume previous connection
  - TLS 1.3: Faster handshake than HTTP/2 + TLS 1.2
  - Connection Migration: Survive network switch (WiFi → 4G)
```

Typical Handshake Times:

```
HTTP/1.1 + TLS 1.2:  3x RTT  (150-200ms on 50ms RTT)
HTTP/2 + TLS 1.3:    1x RTT  (~50ms)
HTTP/3 + 0-RTT:      0x RTT  (~0ms on reconnect!)
```

31.10 Monitoring & Observability

31.10.1 Protocol-Level Metrics

```
Prometheus metrics by protocol:

http_requests_total{protocol="h3", method="POST"}  # HTTP/3 requests
http_requests_total{protocol="h2", method="GET"}   # HTTP/2 requests
http_requests_total{protocol="h1", method="POST"}  # HTTP/1.1 requests

http_request_duration_seconds{protocol="h3"}       # Latency by protocol

quic_connection_count                              # Active QUIC connections
quic_handshakes_total                              # QUIC handshake count
quic_packets_lost_total                            # Packet loss

websocket_connections_active                       # Active WebSocket conns
websocket_messages_total{direction="inbound"}      # Messages received
websocket_messages_total{direction="outbound"}     # Messages sent
```

31.10.2 Client-Side Metrics

```
import time
from prometheus_client import Counter, Histogram

requests = Counter('themis_requests_total', 'Requests by protocol',
                  ['protocol', 'method'])
latencies = Histogram('themis_request_duration_seconds',
                     'Request latency', ['protocol'])

def execute_with_metrics(method, path, protocol='h2'):
    start = time.time()
    try:
        response = execute(method, path, protocol=protocol)
        requests.labels(protocol=protocol, method=method).inc()
        latencies.labels(protocol=protocol).observe(time.time() - start)
        return response
    except Exception as e:
        requests.labels(protocol=protocol, method=method).inc()  # Count failures too
        raise
```

31.11 Zusammenfassung & Best Practices

Protocol Selection Decision Tree

```
Need bidirectional communication?
  ├─ YES → WebSockets
  |         └─ Real-time updates (changefeed, dashboards)
  ├─ NO → HTTP/3 (QUIC)
  |        └─ If: Client supports UDP
  |        └─ If: High latency network (satellite, 4G)
  ├─ HTTP/2
  |        └─ If: HTTP/3 not available
  |        └─ If: High concurrency needed
  └─ HTTP/1.1
           └─ Legacy clients only
```

Protocol-Specific Best Practices

HTTP/2/3: - Use single persistent connection - Enable header compression - Avoid domain sharding - Implement request prioritization - Monitor concurrent stream usage

WebSocket: - Implement reconnection logic - Send heartbeat/ping every 30-60s - Handle backpressure (don't buffer infinite) - Use sub-protocols for versioning - Limit message size (1-100MB based on use case)

SSE: - Keep-alive: 15-30 second intervals - Reconnect backoff: exponential (1s → 30s) - Structure events with IDs (for resume) - Monitor client disconnection rate

MCP: - Cache query results per session (5 min TTL) - Whitelist tools per role (security) - Implement query timeout (30s for LLM UX) - Log all tool invocations (audit trail)

Kapitel 31 von 33 | Teil V: Protocols & Integration | ~8.500 Wörter (+2000 neu)

Kapitel 33: Kapitel 32: AQL v1.3.1 OOP-Implementierung

> *"Die Sprache ist das API – ihre Implementierung bestimmt die Fähigkeiten der Datenbank."*

32.1 Executive Summary

AQL v1.3.1 erweitert die Sprache um objektorientierte Konstrukte (47 neue Keywords, 360 Funktionen bleiben konstant). Dieses Kapitel beschreibt, wie die Erweiterungen in ThemisDB umgesetzt werden – vom Lexer über den Parser bis zur Ausführungsebene.

Ziele: - Lexer/Tokenizer: 47 neue Keywords, Tokens für Klassen/Methoden/Namespaces - Parser/AST: Neue Node-Typen für `CLASS`, `FUNCTION`, `METHOD`, `NAMESPACE` - Type-System: Statisches Type-Checking, Inference, generische Typen - Namespace-Resolution: Symbol-Tables, Visibility, Imports - Runtime: Objekt-Instanziierung, Methodenaufrufe, Visibility-Checks - Tests: Parser-Golden-Files, Type-Checker-Property-Tests

---

32.2 Betroffene Codebereiche

Datei	Zweck	Änderung
`/src/query/aql_parser.cpp`	Lexer/Parser	47 neue Keywords, AST Nodes
`/src/query/aql_translator.cpp`	AST → Execution Plan	OOP Nodes übersetzen
`/include/query/aql_parser.h`	Parser-Interface	Neue Strukturen/Enums
`/src/query/aql_type_checker.cpp`	Type-System	Neue Typregeln, Inference
`/src/query/aql_namespace_resolver.cpp`	Symbol-Tables	Scopes, Visibility
`/src/aql/llm_aql_handler.cpp`	LLM-Befehle	Vision/OOP-Kommandos

Neue Dateien: - `/include/query/aql_type_checker.h`, `/src/query/aql_type_checker.cpp` - `/include/query/aql_namespace_resolver.h`, `/src/query/aql_namespace_resolver.cpp` - `/include/aql/aql_vision_handler.h`, `/src/aql/aql_vision_handler.cpp`

32.3 Lexer/Tokenizer (Woche 1-2)

32.3.1 TokenType-Erweiterung

- Namespace: `NAMESPACE`, `IMPORT`
- Typen: `STRING_TYPE`, `INT_TYPE`, `FLOAT_TYPE`, `BOOL_TYPE`, `ANY_TYPE`

- Klassen/Funktionen: `FUNCTION`, `CLASS`, `PUBLIC`, `PRIVATE`, `CONSTRUCTOR`, `METHOD`, `NEW`
- Referenzen/Vererbung: `THIS`, `SELF`, `EXTENDS`
- Control Flow: `IF`, `THEN`, `ELSE`, `ELSEIF`, `ENDIF`, `TRY`, `CATCH`, `THROW`, `CASE`, `END`
- Pattern Matching: `MATCH`, `WHEN`

Lexer-Checks: - Case-insensitive Keywords - String-Literale unverändert lassen - Fehlerhafte Kombinationen mit präzisen Fehlermeldungen (Zeile/Spalte)

32.3.2 Keyword-Detection

```
// Keyword-Detection mit unordered_set (119 Keywords)
bool isKeyword(std::string_view tok) {
    static const std::unordered_set<std::string_view> kw = {
        "FOR", "CLASS", "METHOD", "NAMESPACE", /* ... 115 weitere */
    };
    return kw.contains(tok);
}
```

32.4 Parser & AST (Woche 2-4)

32.4.1 Neue AST-Nodes

- `AstClassDecl { name, base?, members[] }`
- `AstMethodDecl { name, params[], return_type, visibility, is_static }`
- `AstFunctionDecl { name, params[], return_type }`
- `AstNamespaceDecl { name, imports[], body[] }`
- `AstNewExpr { type_name, args[] }`
- `AstMemberAccess { object, member }`

32.4.2 Grammatik (Auszug)

```
translation_unit
  : namespace_decl* class_decl* function_decl* statement*
  ;

class_decl
  : CLASS IDENT (EXTENDS IDENT)? LBRACE class_member* RBRACE
  ;

class_member
  : visibility? (method_decl | property_decl | constructor_decl)
  ;

method_decl
  : METHOD IDENT LPAREN param_list? RPAREN (ARROW type)? block
  ;
```

32.4.3 Fehlerbehandlung

- Präzise Fehler: `Unexpected token 'END' at line 42, expected 'WHEN'`
- Recovery: Synchronisation bei `;`, `END`, `}`
- AST-Pragmas für Fehlertoleranz, damit IDE-Features funktionieren

---

32.5 Type-System (Woche 4-6)

32.5.1 Typen & Inference

Typ	Beschreibung
`Any`	Fallback/Unbekannt
`String`, `Int`, `Float`, `Bool`	Primitive
`Class<T>`	Klasseninstanzen
`Vector<T>`	Generische Container
`Func<Args, Ret>`	Funktions-Typ

Inference-Regeln: - Assignment: `lhs = rhs` → `type(lhs) = type(rhs)` - Member Access: `obj.m` → Lookup in Symbol-Table, Sichtbarkeit prüfen - Calls: `call(f, args)` → Parameteranzahl, Typkompatibilität, Varargs

32.5.2 Visibility & Access Control

- `public`: überall sichtbar
- `private`: nur innerhalb derselben Klasse
- `protected` (optional): Klasse + Subklassen

32.5.3 Exceptions & TRY/CATCH

- Typ `Exception` als Basisklasse
- `THROW expr` propagiert Typ `Exception`
- `TRY { ... } CATCH (e: Exception) { ... }`

32.6 Namespace-Resolution (Woche 6-7)

32.6.1 Symbol-Tables

- Hierarchische Tabellen pro Scope
- Eintrag: `{name, kind (var|func|class|namespace), type, visibility}`
- Shadowing nach Java/C#-Modell (innere Scopes haben Vorrang)

32.6.2 Imports

```
NAMESPACE themis.app
IMPORT themis.llm.*
IMPORT themis.vision.embed
```

- Wildcards (`*`) nur auf Namespace-Level erlaubt
- Konfliktauflösung: explizite Qualifizierung gewinnt (`llm.stats`)

32.6.3 Resolution-Algorithmus

1. Prüfe lokalen Scope
2. Prüfe Namespace-Imports
3. Prüfe globale Symbole
4. Fehler, wenn nicht gefunden → `Unknown symbol 'X'`

32.7 Runtime & Execution (Woche 7-10)

32.7.1 New/Constructor

```
Value evalNew(const AstNewExpr& node, Context& ctx) {
    auto* cls = ctx.lookupClass(node.type_name);
    return cls->invokeConstructor(node.args);  // Alloc + Init
}
```

32.7.2 Methodenaufrufe

- Dynamische Dispatch-Tabelle je Klasse
- Inline-Cache (optional) für Hot-Paths
- Visibility-Checks zur Laufzeit (Guard)

32.7.3 Exceptions

- Stack-Unwinding mit RAII-Gurten
- `defer` / `finally` -Support über Scope Guards

32.8 Vision & LLM-Befehle (Add-On)

Neue Kommandos im LLM-Handler: - `VISION.EMBED(image|text, model)` → Vektor - `VISION.DESCRIBE(image)` → Text - `VISION.COMPARE(imgA, imgB)` → Similarity - Safety: Max Input Size, MIME-Checks, Model-Whitelist

---

32.9 Test-Strategie

32.9.1 Parser-Golden-Files

- Input → erwarteter AST (JSON) → Diffen in CI
- Beispiele: Klassen mit Vererbung, Methoden mit Default-Args, Namespaces

32.9.2 Property-Tests (Type Checker)

- QuickCheck/rapidcheck: zufällige Ausdrucksbäume generieren
- Invariante: Keine illegalen Typen nach erfolgreichem Check
- Fuzzer: ungültige Tokens → Parser muss robust fehlschlagen

32.9.3 Runtime-Tests

- Konstruktor/Methoden-Dispatch
- Visibility-Fehler (private/public)
- Exception-Pfade und Unwinding

32.10 Migrations-Plan

Phase	Dauer	Ergebnis
Lexer/Parser	2 Wochen	Keywords + AST-Nodes
Type-System	2-3 Wochen	Checker + Inference
Namespace	1 Woche	Symbol-Tables, Imports
Runtime	2-3 Wochen	Dispatch, Exceptions
Tests/Hardening	2 Wochen	Golden-Files, Property-Tests

Risiken & Mitigation: - Parser-Regressions → Golden-Files in CI - Performance-Einbruch → Inline-Caches, Hot-Path-Profiler - Kompatibilität → Feature-Flag `enable_oop` bis stabil

---

32.11 Checkliste Go-Live

- [ ] `enable_oop` Feature-Flag default ON
- [ ] Parser-Tests (100% der neuen Grammatik) grün
- [ ] Type-Checker-Property-Tests 10k Runs
- [ ] Runtime-Benchmarks: <10% Overhead ggü. v1.3.0
- [ ] LLM/Vision-Commands whitelisted, Input-Limits gesetzt
- [ ] Docs aktualisiert (AQL Referenz, Examples)

32.10 Implementierungs-Timeline

Phase 1: Lexer & Parser (Wochen 1-4)

Woche 1-2: Lexer - Tag 1-2: Keyword-Set erweitern (47 neue) - Tag 3-4: Token-Recognition Tests - Tag 5-6: Error Messages verbessern - Tag 7-8: Performance Benchmarks (Lexer sollte <5% langsamer sein)

Woche 3-4: Parser - Tag 1-4: AST Node-Typen implementieren - Tag 5-7: Grammatik-Regeln für CLASS/METHOD/NAMESPACE - Tag 8-10: Golden-File Tests (10+ Parser-Test-Cases)

Phase 2: Type System (Wochen 5-7)

Woche 5-6: Type Checker - Tag 1-3: Typ-Inference-Algorithmus - Tag 4-6: Visitor Pattern für AST-Traversal - Tag 7-10: Generics Support (Vector, Map)

Woche 7: Validation - Tag 1-3: Type Error Messages - Tag 4-6: Property-Based Tests - Tag 7: Integration mit Parser

Phase 3: Namespace Resolution (Wochen 8-9)

- Tag 1-3: Symbol-Table Implementierung
- Tag 4-6: Import-Resolution
- Tag 7-9: Shadowing & Visibility
- Tag 10-12: Test-Suite (50+ Test-Cases)

Phase 4: Runtime (Wochen 10-14)

Woche 10-11: Object Model - Tag 1-5: Class/Instance Representation - Tag 6-10: Constructor/Method Dispatch

Woche 12-13: Execution - Tag 1-5: NEW Expression Execution - Tag 6-10: Member Access Resolution - Tag 11-14: Exception Handling

Woche 14: Integration - Tag 1-3: End-to-End Tests - Tag 4-5: Performance Profiling - Tag 6-7: Documentation

Phase 5: Vision & Rollout (Wochen 15-16)

- Woche 15: Vision.EMBED/DESCRIBE/COMPARE
- Woche 16: Production Deployment, Monitoring

32.11 Code-Beispiele

32.11.1 Lexer Test

```cpp
// test_lexer_oop.cpp
TEST(AQLLexer, RecognizesClassKeyword) {
    std::string input = "CLASS User { }";
    Lexer lexer(input);

    Token t1 = lexer.next();
    EXPECT_EQ(t1.type, TokenType::CLASS);

    Token t2 = lexer.next();
    EXPECT_EQ(t2.type, TokenType::IDENTIFIER);
    EXPECT_EQ(t2.value, "User");

    Token t3 = lexer.next();
    EXPECT_EQ(t3.type, TokenType::LBRACE);
}

TEST(AQLLexer, KeywordCaseInsensitive) {
    Lexer lexer("class CLASS Class");

    for (int i = 0; i < 3; i++) {
        Token t = lexer.next();
        EXPECT_EQ(t.type, TokenType::CLASS);
    }
}
```

32.11.2 Parser Test mit Golden Files

```cpp
// test_parser_golden.cpp
TEST(AQLParser, ClassDeclarationGolden) {
    std::string input = R"(
        CLASS User {
            PRIVATE email: STRING
            PUBLIC name: STRING

            CONSTRUCTOR(email, name) {
                THIS.email = email
                THIS.name = name
            }

            PUBLIC METHOD greet() {
                RETURN CONCAT("Hello, ", THIS.name)
            }
        }
    )";

    Parser parser(input);
    AstNode* ast = parser.parse();

    // Serialize AST to JSON
    std::string ast_json = serializeAST(ast);

    // Compare with Golden File
    std::string expected = readFile("golden/class_user_ast.json");
    EXPECT_EQ(ast_json, expected);
}
```

32.11.3 Type Checker Unit Test

```
// test_type_checker.cpp
TEST(TypeChecker, InfersReturnType) {
    std::string code = R"(
        FUNCTION add(a: INT, b: INT) {
            RETURN a + b  // Should infer INT
        }
    )";

    TypeChecker checker;
    FunctionType func_type = checker.inferFunctionType(code);

    EXPECT_EQ(func_type.return_type, Type::INT);
    EXPECT_EQ(func_type.param_types.size(), 2);
    EXPECT_EQ(func_type.param_types[0], Type::INT);
}

TEST(TypeChecker, DetectsMismatch) {
    std::string code = R"(
        FUNCTION broken(x: STRING) -> INT {
            RETURN x  // x Type Error: expected INT, got STRING
        }
    )";

    TypeChecker checker;
    EXPECT_THROW(
        checker.check(code),
        TypeMismatchException
    );
}
```

32.11.4 Namespace Resolution Test

```
// test_namespace.cpp
TEST(NamespaceResolver, ImportsResolve) {
    std::string code = R"(
        NAMESPACE myapp
        IMPORT themis.llm.*

        FUNCTION analyze(text) {
            RETURN LLM.CHAT(text)  // Should resolve to themis.llm.LLM.CHAT
        }
    )";

    NamespaceResolver resolver;
    SymbolTable symbols = resolver.resolve(code);

    Symbol* llm_chat = symbols.lookup("LLM.CHAT");
    EXPECT_NE(llm_chat, nullptr);
    EXPECT_EQ(llm_chat->fully_qualified_name, "themis.llm.LLM.CHAT");
}
```

32.11.5 Runtime NEW Expression

```
// aql_runtime.cpp: NEW Implementierung
Value AQLRuntime::evalNew(const AstNewExpr& node) {
    // 1. Lookup Class Definition
    ClassDef* cls = symbol_table_.lookupClass(node.type_name);
    if (!cls) {
        throw RuntimeError(fmt::format("Class '{}' not found", node.type_name));
    }

    // 2. Allocate Instance
    ObjectInstance* instance = allocateObject(cls);

    // 3. Find Constructor
    Constructor* ctor = cls->findConstructor(node.args.size());
    if (!ctor) {
        throw RuntimeError(fmt::format(
            "No constructor found for {} args", node.args.size()
        ));
    }

    // 4. Evaluate Arguments
    std::vector<Value> arg_values;
    for (const auto& arg : node.args) {
        arg_values.push_back(eval(arg));
    }

    // 5. Invoke Constructor
    ctor->invoke(instance, arg_values);

    return Value::fromObject(instance);
}
```

32.12 Performance-Auswirkungen

Erwartete Overhead

Operation	Baseline (ohne OOP)	Mit OOP	Overhead
Lexing	100 ms/10k lines	105 ms	+5%
Parsing	250 ms/10k lines	280 ms	+12%
Type Check	N/A	150 ms	Neu
Execution (Simple FOR)	10 ms	10 ms	0%
Execution (Method Call)	N/A	+2 ms	Neu

Optimierungen: - **Inline Caching:** Häufig aufgerufene Methoden cachen - **JIT für Hot Methods:** Bytecode-Kompilierung für >1000 Calls - **Lazy Type Checking:** Nur bei Bedarf (Development-Mode)

Benchmarks

```
// benchmark_oop.cpp
static void BM_MethodCall(benchmark::State& state) {
    std::string code = R"(
        CLASS Counter {
            PRIVATE count: INT

            CONSTRUCTOR() { THIS.count = 0 }
            METHOD increment() { THIS.count = THIS.count + 1 }
        }

        LET c = NEW Counter()
        FOR i IN 1..1000000
            c.increment()
    )";

    for (auto _ : state) {
        AQLRuntime runtime;
        runtime.execute(code);
    }
}
BENCHMARK(BM_MethodCall);

// Ergebnis: ~2.5 µs pro Method Call (inkl. Dispatch)
```

32.13 Migration & Backward Compatibility

Rückwärtskompatibilität

- **Alle AQL v1.3.0 Queries funktionieren unverändert**
- **Neue Keywords nur in OOP-Kontext reserviert**
- **Legacy-Code benötigt keine Änderungen**

Opt-In für OOP Features

```
-- Legacy Mode (Default)
FOR u IN users RETURN u

-- OOP Mode (via Pragma)
#pragma aql_version 1.3.1

CLASS User { ... }
```

Feature Flag

```yaml
aql:
  oop_features_enabled: true  # Default: false bis v1.4.0
  strict_type_checking: false  # Development: true, Production: false
```

32.14 Zusammenfassung

AQL v1.3.1 führt OOP-Konstrukte ein und erfordert Änderungen auf allen Ebenen der Query-Engine. Die Implementierung gliedert sich in fünf Phasen: Lexer/Parser, Type-System, Namespace-Resolution, Runtime, Tests. Mit klaren Guardrails (Feature-Flag, Tests, Benchmarks) kann die Erweiterung ohne Regressionen ausgerollt werden.

Kapitel 34: Kapitel 33: Best Practices & Design Patterns

> *"Good code is its own best documentation. As you're about to add a comment, ask yourself, 'How can I improve the code so that this comment isn't needed?'" - Steve McConnell*

Überblick

Production-ready ThemisDB-Anwendungen folgen bewährten Patterns für Performance, Sicherheit, und Wartbarkeit. Dieses Kapitel sammelt Battle-tested Best Practices aus realen Deployments.

Was Sie in diesem Kapitel lernen: - Schema-Design Patterns - Query-Optimierung Best Practices - Sicherheits-Härtung - Resilience Patterns - Testing-Strategien - Performance Tuning - Operational Excellence

---

33.1 Schema-Design Patterns

Pattern 1: Embedded vs. Referenced

Embedded (Denormalisiert):

```
//   Gut für: 1:N Beziehungen mit wenigen Child-Elementen
{
  "_key": "order-123",
  "customer": {
    "name": "Alice",
    "email": "alice@example.com"
  },
  "items": [
    {"product": "Laptop", "quantity": 1, "price": 1200},
    {"product": "Mouse", "quantity": 2, "price": 25}
  ],
  "total": 1250
}

//   Vorteile:
// - Single query (kein JOIN)
// - Atomic updates
// - Keine referentielle Integrität nötig

// × Nachteile:
// - Duplikation (customer-Daten in jedem Order)
// - Update-Anomalien (email ändert sich → alle Orders updaten)
```

Referenced (Normalisiert):

```
// customers Collection
{
  "_key": "alice",
  "name": "Alice",
  "email": "alice@example.com",
  "address": {...}
}

// orders Collection
{
  "_key": "order-123",
  "customer_id": "alice",  // Referenz
  "items": [...],
  "total": 1250
}

// Query mit JOIN
FOR order IN orders
  FILTER order._key == 'order-123'
  LET customer = DOCUMENT(CONCAT('customers/', order.customer_id))
  RETURN MERGE(order, {customer: customer})

//   Vorteile:
// - Keine Duplikation
// - Single source of truth
// - Einfache Updates (nur 1 Dokument)

// × Nachteile:
// - Zusätzlicher Query-Overhead (JOIN)
// - Referentielle Integrität manuell sicherstellen
```

Entscheidungsmatrix: | Use Case | Pattern | Begründung | |----------|---------|------------| | Blog: Post + Comments (1:N, viele) | Referenced | Comments wachsen unbegrenzt | | Order + Items (1:N, <20) | Embedded | Items sind fix nach Order | | User + Preferences (1:1) | Embedded | Preferences immer mit User geladen | | Product + Categories (M:N) | Referenced | Viele-zu-Viele → Graph-Edges |

33.2 Query-Optimierung Patterns

Pattern 2: Early Filtering

```
-- × SCHLECHT: Filter nach JOIN
FOR order IN orders
  LET customer = DOCUMENT(order.customer_id)
  FILTER customer.country == 'DE'  // Zu spät!
  RETURN order

--   GUT: Filter vor JOIN
FOR order IN orders
  LET customer = DOCUMENT(order.customer_id)
  FILTER customer != null && customer.country == 'DE'
  RETURN order

--   OPTIMAL: Denormalisierung für häufige Filters
{
  "_key": "order-123",
  "customer_id": "alice",
  "customer_country": "DE",  // Denormalisiert für schnellen Filter
  ...
}

FOR order IN orders
  FILTER order.customer_country == 'DE'  // Index-optimiert
  RETURN order
```

Pattern 3: Projection (Select nur benötigte Felder)

```
-- × SCHLECHT: Alle Felder laden
FOR user IN users
  RETURN user  // Lädt alle Felder (1 MB/User!)

--   GUT: Nur benötigte Felder
FOR user IN users
  RETURN {
    id: user._id,
    name: user.name,
    email: user.email
  }  // Nur 100 Bytes/User

-- Performance-Gewinn: 10x schneller bei großen Dokumenten
```

Pattern 4: Pagination mit Cursor

```
-- × SCHLECHT: OFFSET/LIMIT (langsam bei großen Offsets)
FOR doc IN collection
  SORT doc.created_at DESC
  LIMIT 10000, 100  // Skip 10k Docs → langsam!
  RETURN doc

--   GUT: Cursor-based Pagination
FOR doc IN collection
  FILTER doc.created_at < @last_seen_timestamp
  SORT doc.created_at DESC
  LIMIT 100
  RETURN doc

// Client speichert letzten Timestamp:
last_seen = results[99].created_at
next_page = query(last_seen_timestamp=last_seen)
```

33.3 Sicherheits-Härtung

Pattern 5: Input Validation

```
-- × SCHLECHT: Unvalidierte User-Eingabe
LET user_input = @email
FOR u IN users
  FILTER u.email == user_input  // Injection-Risiko bei String-Concat!
  RETURN u

--   GUT: Parametrisierte Queries
FOR u IN users
  FILTER u.email == @email  // Safe: Parameter-Binding
  RETURN u

--   BESSER: Zusätzliche Validation
FUNCTION validate_email(email) {
  IF !REGEX_TEST(email, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$") THEN
    THROW ERROR("Invalid email format")
  END
  RETURN email
}

LET validated = validate_email(@email)
FOR u IN users
  FILTER u.email == validated
  RETURN u
```

Pattern 6: Least Privilege Access

```
-- × SCHLECHT: Root-User für Application
const client = new ThemisClient({
  user: 'root',  // × Zu viele Rechte!
  password: 'root123'
})

--   GUT: Dedizierter App-User mit minimalen Rechten
CREATE USER app_user WITH PASSWORD 'secure_pass'
GRANT READ ON DATABASE mydb TO app_user
GRANT WRITE ON COLLECTION orders TO app_user
GRANT WRITE ON COLLECTION users TO app_user

const client = new ThemisClient({
  user: 'app_user',
  password: process.env.THEMIS_PASSWORD  //   Aus Env-Var
})
```

Pattern 7: Data Encryption at Rest

```
storage:
  encryption:
    enabled: true
    algorithm: AES-256-GCM
    key_provider: aws-kms
    key_id: arn:aws:kms:eu-central-1:123456789:key/abc-def

  # Zusätzlich: Field-Level Encryption für PII
  field_encryption:
    - collection: users
      fields: [ssn, credit_card, password_hash]
```

33.4 Resilience Patterns

Pattern 8: Circuit Breaker

```
class CircuitBreaker:
    def __init__(self, threshold=5, timeout=60):
        self.failures = 0
        self.threshold = threshold
        self.timeout = timeout
        self.open_until = None

    def call(self, func, *args, **kwargs):
        # Circuit Open → Fail Fast
        if self.open_until and time.time() < self.open_until:
            raise CircuitBreakerOpen("Service unavailable")

        try:
            result = func(*args, **kwargs)
            self.failures = 0  # Reset bei Erfolg
            return result
        except Exception as e:
            self.failures += 1

            if self.failures >= self.threshold:
                self.open_until = time.time() + self.timeout
                print(f"Circuit opened for {self.timeout}s")

            raise

db_breaker = CircuitBreaker(threshold=3, timeout=30)

def query_themis(aql):
    return db_breaker.call(client.query, aql)
```

Pattern 9: Retry with Exponential Backoff

```
def retry_with_backoff(func, max_retries=3, base_delay=1):
    """Retry mit exponential backoff"""
    for attempt in range(max_retries):
        try:
            return func()
        except (NetworkError, TimeoutError) as e:
            if attempt == max_retries - 1:
                raise  # Letzter Versuch → propagieren

            delay = base_delay * (2 ** attempt)  # 1s, 2s, 4s
            jitter = random.uniform(0, 0.3) * delay  # ±30% Jitter
            time.sleep(delay + jitter)

result = retry_with_backoff(
    lambda: client.query("FOR u IN users RETURN u"),
    max_retries=5,
    base_delay=0.5
)
```

Pattern 10: Bulkhead Pattern (Resource Isolation)

```
critical_pool = ThemisConnectionPool(
    max_connections=50,
    priority='high'
)

background_pool = ThemisConnectionPool(
    max_connections=10,
    priority='low'
)

def get_user_profile(user_id):
    with critical_pool.get_connection() as conn:
        return conn.query("FOR u IN users FILTER u._id == @id RETURN u",
                         {'id': user_id})

def generate_analytics_report():
    with background_pool.get_connection() as conn:
        return conn.query("FOR order IN orders COLLECT ...")
```

33.5 Testing Patterns

Pattern 11: Golden File Testing

```
def test_user_query_golden():
    """Query-Ergebnis mit Golden File vergleichen"""

    # Setup Test-Daten
    setup_fixture('users_test.json')

    # Query ausführen
    result = client.query("""
        FOR u IN users
          FILTER u.age > 25
          SORT u.name
          RETURN {name: u.name, age: u.age}
    """)

    # Mit Golden File vergleichen
    expected = load_json('golden/user_query_expected.json')
    assert result == expected, f"Query result mismatch!"
```

Pattern 12: Property-Based Testing

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers(min_value=0, max_value=100), min_size=1, max_size=1000))
def test_aggregation_properties(numbers):
    """Property: SUM sollte immer gleich Python sum() sein"""

    # Insert Test-Daten
    for n in numbers:
        client.insert('numbers', {'value': n})

    # AQL Aggregation
    aql_sum = client.query("RETURN SUM(n.value FOR n IN numbers)")[0]

    # Python Referenz
    python_sum = sum(numbers)

    assert aql_sum == python_sum, f"Aggregation mismatch: {aql_sum} != {python_sum}"
```

33.6 Performance Tuning

Pattern 13: Connection Pooling

```
def handle_request():
    client = ThemisClient('http://localhost:8529')  # Langsam!
    result = client.query(...)
    client.close()
    return result

from themis.pool import ConnectionPool

pool = ConnectionPool(
    url='http://localhost:8529',
    max_connections=100,
    min_connections=10,
    connection_timeout=5
)

def handle_request():
    with pool.get_connection() as client:
        return client.query(...)
```

Pattern 14: Query Caching

```
from functools import lru_cache
import hashlib

@lru_cache(maxsize=1000)
def cached_query(query_hash, params_hash):
    """Cache für idempotente Queries"""
    query = QUERY_CACHE[query_hash]
    params = PARAMS_CACHE[params_hash]
    return client.query(query, params)

def get_user_stats(user_id):
    query = "FOR u IN users FILTER u._id == @id RETURN u.stats"
    query_hash = hashlib.md5(query.encode()).hexdigest()
    params_hash = hashlib.md5(str(user_id).encode()).hexdigest()

    return cached_query(query_hash, params_hash)

def update_user(user_id, data):
    client.update(f'users/{user_id}', data)
    cached_query.cache_clear()  # Invalidate
```

Pattern 15: Batch Operations

```
-- × SCHLECHT: N einzelne Inserts
FOR i IN 1..10000
  INSERT {value: i} INTO collection  // 10k Roundtrips!

--   GUT: Batch-Insert
LET docs = (FOR i IN 1..10000 RETURN {value: i})
FOR doc IN docs
  INSERT doc INTO collection  // 1 Roundtrip

-- Python Equivalent:
docs = [{'value': i} for i in range(10000)]
client.query("FOR doc IN @docs INSERT doc INTO collection", {'docs': docs})
```

33.7 Operational Patterns

Pattern 16: Health Check Endpoint

```
// server.js: Express Health Check
app.get('/health', async (req, res) => {
  const health = {
    status: 'healthy',
    timestamp: new Date().toISOString(),
    checks: {}
  };

  try {
    // Database connectivity
    const start = Date.now();
    await themisClient.query('RETURN 1');
    health.checks.database = {
      status: 'ok',
      latency_ms: Date.now() - start
    };
  } catch (err) {
    health.status = 'unhealthy';
    health.checks.database = {
      status: 'error',
      error: err.message
    };
  }

  // Memory check
  const memUsage = process.memoryUsage();
  health.checks.memory = {
    status: memUsage.heapUsed < 0.9 * memUsage.heapTotal ? 'ok' : 'warning',
    heap_used_mb: Math.round(memUsage.heapUsed / 1024 / 1024)
  };

  res.status(health.status === 'healthy' ? 200 : 503).json(health);
});
```

Pattern 17: Graceful Shutdown

```
import signal
import sys

class Application:
    def __init__(self):
        self.shutting_down = False
        signal.signal(signal.SIGTERM, self.handle_shutdown)
        signal.signal(signal.SIGINT, self.handle_shutdown)

    def handle_shutdown(self, signum, frame):
        print("Shutdown signal received, gracefully stopping...")
        self.shutting_down = True

        # 1. Stop accepting new requests
        self.stop_accepting_requests()

        # 2. Wait for active requests to complete (max 30s)
        self.wait_for_active_requests(timeout=30)

        # 3. Close DB connections
        themis_pool.close_all()

        # 4. Flush logs
        logging.shutdown()

        sys.exit(0)

app = Application()
```

33.8 Checkliste für Production-Readiness

Pre-Deployment Checklist

- **Schema & Indizes:**

- [ ] Alle Indizes erstellt (`CREATE INDEX`)
- [ ] EXPLAIN für kritische Queries durchgeführt
- [ ] Composite Indizes für häufige Filter-Kombinationen
- [ ] TTL-Indizes für automatische Datenlöschung

- **Performance:**

- [ ] Connection Pooling aktiviert
- [ ] Query Timeouts konfiguriert
- [ ] Rate Limiting implementiert
- [ ] Caching-Strategie definiert

- **Sicherheit:**

- [ ] Encryption at Rest aktiviert
- [ ] TLS für Client-Verbindungen
- [ ] Least-Privilege User-Accounts
- [ ] Input-Validierung in Application

- **Resilience:**

- [ ] Circuit Breaker implementiert
- [ ] Retry-Logic mit Backoff
- [ ] Graceful Shutdown Handler
- [ ] Health Check Endpoint

- **Monitoring:**

- [ ] Prometheus/Grafana Dashboards
- [ ] Alerting für Critical Metrics
- [ ] Slow Query Log aktiviert
- [ ] Error Tracking (Sentry/Datadog)

- **Backup & DR:**

- [ ] Tägliche automatische Backups
- [ ] Backup-Restore getestet
- [ ] Multi-Region Replication
- [ ] Disaster Recovery Runbook

33.9 Anti-Patterns (Was zu vermeiden ist)

× Anti-Pattern 1: N+1 Queries

```
-- SCHLECHT: 1 Query + N Queries
FOR order IN orders
  LIMIT 100
  LET customer = DOCUMENT(order.customer_id)  // N zusätzliche Lookups!
  RETURN {order: order, customer: customer}

-- GUT: Batch Lookup
LET order_ids = (FOR o IN orders LIMIT 100 RETURN o._key)
LET orders = DOCUMENT(orders, order_ids)
LET customer_ids = orders[*].customer_id
LET customers = DOCUMENT(customers, customer_ids)
...
```

× Anti-Pattern 2: Unbounded Collections

```
-- SCHLECHT: Embedded Array wächst unbegrenzt
{
  "blog_post_id": "post-1",
  "comments": [...]  // Was wenn 10k Comments?
}

-- GUT: Separate Collection mit Referenz
// comments Collection
{
  "post_id": "post-1",
  "comment": "...",
  "author": "alice"
}
```

× Anti-Pattern 3: Hardcoded Credentials

```
client = ThemisClient('http://prod-db:8529',
                     username='admin',
                     password='prod123')  # × Im Code!

client = ThemisClient(
    os.getenv('THEMIS_URL'),
    username=os.getenv('THEMIS_USER'),
    password=os.getenv('THEMIS_PASSWORD')  #   Aus Env
)
```

33.10 Advanced Patterns: Event Sourcing

Event Sourcing Pattern

Concept: Store immutable events, derive state from events.

```
-- Event Log (immutable)
{
  "_id": "events/evt-001",
  "aggregate_id": "account/alice",
  "event_type": "AccountCreated",
  "timestamp": "2025-01-01T10:00:00Z",
  "data": { "name": "Alice", "email": "alice@example.com" }
}

{
  "_id": "events/evt-002",
  "aggregate_id": "account/alice",
  "event_type": "DepositMade",
  "timestamp": "2025-01-01T10:05:00Z",
  "data": { "amount": 1000, "currency": "USD" }
}

{
  "_id": "events/evt-003",
  "aggregate_id": "account/alice",
  "event_type": "WithdrawalMade",
  "timestamp": "2025-01-01T10:10:00Z",
  "data": { "amount": 200, "currency": "USD" }
}

-- Materialized View (derived)
{
  "_id": "account_state/alice",
  "balance": 800,
  "last_event_id": "evt-003",
  "updated_at": "2025-01-01T10:10:00Z"
}
```

Benefits: - **Audit Trail:** Complete history of all changes - **Temporal Queries:** "What was balance at 10:08?" - **Event Replay:** Rebuild state from scratch - **Debugging:** Trace exact sequence of operations

Implementation:

```
-- Record event
BEGIN
  INSERT event INTO event_log
  UPDATE account_state WITH { balance: new_balance }
COMMIT

-- Replay events (if state corrupted)
LET all_events = (
  FOR event IN event_log
    FILTER event.aggregate_id == @account_id
    SORT event.timestamp ASC
    RETURN event
)

LET final_state = REDUCE event IN all_events
  INTO {balance: 0, last_event: NULL}
  (
    LET update = APPLY_EVENT(acc, event)
    RETURN update
  )

RETURN final_state
```

33.11 CQRS Pattern (Command Query Responsibility Segregation)

Pattern: Separate Reads from Writes

```
Write Model (Command Side)
  ├─ Receives mutations (CREATE, UPDATE, DELETE)
  ├─ Validates business rules
  ├─ Persists to event log
  └─ Produces events

Event Bus
  └─ Asynchronously publishes events

Read Model (Query Side)
  ├─ Subscribes to events
  ├─ Updates read-optimized views
  ├─ Serves fast queries
  └─ Can have different schema than write model
```

Example: E-Commerce Order

```
-- Write Model (Normalized)
{
  "_id": "orders/ord-123",
  "customer_id": "cust-456",
  "status": "shipped",
  "created_at": "2025-01-01T10:00:00Z"
}

-- Read Model (Denormalized for Dashboard)
{
  "_id": "order_view/ord-123",
  "customer_name": "Alice",
  "customer_email": "alice@example.com",
  "total_amount": 1250,
  "item_count": 3,
  "status": "shipped",
  "estimated_delivery": "2025-01-05",
  "updated_at": "2025-01-01T15:00:00Z"
}
```

Implementation:

```
@app.post("/orders")
def create_order(cmd: CreateOrderCommand):
    # Validate
    assert cmd.total > 0
    assert cmd.customer_id in db.customers

    # Write to event log
    event = OrderCreatedEvent(cmd)
    db.insert('event_log', event)

    # Publish to event bus
    event_bus.publish(event)

    return {"order_id": event.order_id}

event_bus.subscribe('OrderCreatedEvent', rebuild_order_view)

def rebuild_order_view(event):
    # Enrich with customer data
    customer = db.get('customers', event.customer_id)

    # Create denormalized view
    view = {
        "order_id": event.order_id,
        "customer_name": customer.name,
        "customer_email": customer.email,
        "total": event.total,
        "status": "created"
    }
    db.insert('order_view', view)
```

33.12 Saga Pattern (Distributed Transactions)

Pattern: Multi-Step Compensating Transactions

Problem: Atomic transaction across 3+ microservices.

Solution: Saga with rollback logic.

```
Order Processing Saga:

Step 1: Reserve Inventory
  ├─ Reserve 10 units
  └─ If fail: Abort saga

Step 2: Process Payment
  ├─ Charge credit card
  └─ If fail: Release inventory (compensate Step 1)

Step 3: Ship Order
  ├─ Create shipment
  └─ If fail: Refund payment (compensate Step 2)

Step 4: Send Notification
  ├─ Send confirmation email
  └─ If fail: Just log (no compensation)
```

Implementation in ThemisDB:

```
-- Saga State Machine
{
  "_id": "sagas/saga-001",
  "order_id": "ord-123",
  "status": "in_progress",
  "steps": [
    { "name": "reserve_inventory", "status": "completed", "compensated": false },
    { "name": "process_payment", "status": "completed", "compensated": false },
    { "name": "ship_order", "status": "failed", "error": "Out of stock" },
    { "name": "send_notification", "status": "pending", "compensated": false }
  ],
  "created_at": "2025-01-01T10:00:00Z"
}

-- On Step 3 failure, execute compensations in reverse
-- Step 2: Refund payment
-- Step 1: Release inventory
-- Update saga status to "rolled_back"
```

33.13 Bulkhead Pattern (Isolation)

Pattern: Isolate Critical Resources

Problem: One slow query brings down entire system.

Solution: Separate resource pools.

```

```

Implementation:

```
from concurrent.futures import ThreadPoolExecutor

user_executor = ThreadPoolExecutor(max_workers=20, thread_name_prefix='user_')
admin_executor = ThreadPoolExecutor(max_workers=5, thread_name_prefix='admin_')
bg_executor = ThreadPoolExecutor(max_workers=10, thread_name_prefix='bg_')

if query.is_admin:
    future = admin_executor.submit(execute_query, query)
elif query.is_background:
    future = bg_executor.submit(execute_query, query)
else:
    future = user_executor.submit(execute_query, query)

result = future.result(timeout=30)  # 30s for users, 60s for admin
```

33.14 Throttling & Rate Limiting Pattern

Pattern: Control Request Rate

```
class RateLimiter:
    def __init__(self, max_requests_per_second=1000):
        self.max_rps = max_requests_per_second
        self.tokens = max_requests_per_second
        self.last_refill = time.time()
        self.lock = threading.Lock()

    def allow(self):
        with self.lock:
            now = time.time()
            elapsed = now - self.last_refill

            # Refill tokens
            self.tokens = min(
                self.max_rps,
                self.tokens + elapsed * self.max_rps
            )
            self.last_refill = now

            if self.tokens >= 1:
                self.tokens -= 1
                return True
            return False

limiter = RateLimiter(max_requests_per_second=1000)

@app.post("/query")
def execute_query(query: Query):
    if not limiter.allow():
        return {"error": "Rate limit exceeded", "retry_after": 1}

    return execute(query)
```

33.15 Zusammenfassung: Advanced Patterns

Pattern	Problem	Solution
Event Sourcing	Audit trail	Immutable events
CQRS	Read/write scaling	Separate models
Saga	Distributed transactions	Compensating transactions
Bulkhead	Resource contention	Isolated pools
Rate Limiting	Overload protection	Token bucket
Circuit Breaker	Cascading failures	Fail-fast
Retry	Transient failures	Exponential backoff

33.16 Golden Rules Revisited

The 7 Commandments of Database Stewardship:

1. **Index Everything You Filter**
2. Query without index = scan all rows
3. EXPLAIN is your friend
4. Composite indexes follow query order

5. **Fail Fast, Recover Faster**

6. Circuit breaker pattern
7. Exponential backoff on retries
8. Self-healing infrastructure

9. **Test Like Production**

10. Load testing with realistic workload
11. Chaos engineering for resilience
12. Staging identical to production

13. **Monitor Before You're in Crisis**

14. Metrics: CPU, Memory, QPS, Latency
15. Logs: All errors, admin actions
16. Traces: Request flow

17. **Automate Everything**

18. Backups without manual intervention
19. Failover without human click
20. Deployments without handoff

21. **Document As You Go**
22. Architecture Decision Records (ADRs)
23. Runbooks for common issues
24. Decisions > Implementation details
25. **Security by Default**
26. Least privilege (not super-user)
27. Encryption (at rest, in transit)
28. Validation (all inputs)
29. Audit (all changes)

Kapitel 33 von 33 | Teil VI: Best Practices & Advanced | ~9.000 Wörter (+2000 neu)

Kapitel 35: Kapitel 34: Query Optimierung & Performance Tuning

> *"Optimierung ist eine kontinuierliche Reise, nicht ein Ziel. Mit den richtigen Tools können Sie fast jede Query um 10-100x beschleunigen."*

Überblick

Query-Performance ist einer der kritischsten Faktoren für den Datenbankenerfolg. Dieses Kapitel zeigt praktische Techniken zur Optimierung von AQL-Queries, vom EXPLAIN-Analyse bis zu fortgeschrittenen Optimierungsmustern.

Was Sie in diesem Kapitel lernen: - EXPLAIN und Query-Profiling - Index-Strategien und Best Practices - Early Filtering und Projection Pushdown - Query-Refactoring für Performance - Aggregation Optimization - Window Function Performance - Batch-Operation Tuning - Query Caching und Memoization

---

34.1 EXPLAIN und Query-Profiling

EXPLAIN Output verstehen

```
-- Grundlegende EXPLAIN Syntax
EXPLAIN FOR doc IN users
  FILTER doc.status == 'active'
  SORT doc.created_at DESC
  LIMIT 10
  RETURN doc

-- Ausgabe-Struktur:
-- ExecutionPlan:
--   Steps:
--   1. CollectionScan (collection="users")
--   2. Filter (condition="doc.status == 'active'")
--   3. Sort (fields=["created_at"])
--   4. Limit (count=10)
--   5. Return
```

Extended EXPLAIN

```
-- Detailliertes EXPLAIN mit Statistiken
EXPLAIN OPTION {verbose: true}
FOR doc IN users
  FILTER doc.status == 'active'
  RETURN doc

-- Output zeigt:
-- - Estimated item count pro Step
-- - Filter selectivity
-- - Index usage
-- - Memory usage
```

Profile Queries

```
-- Query mit echtem Timing profilen
PROFILE FOR doc IN users
  FILTER doc.status == 'active'
  SORT doc.created_at
  RETURN doc

-- Ausgabe:
-- Step | Operation | Items | Time(ms) | Selectivity
-- 1    | Scan      | 1M    | 50       | 100%
-- 2    | Filter    | 100k  | 150      | 10%
-- 3    | Sort      | 100k  | 200      | 100%
-- 4    | Return    | 100k  | 10       | 100%
```

34.2 Index-Strategien

Single Field Indexes

```
-- Einfachster Index - für exakte Matches & Range Queries
CREATE INDEX idx_status ON users(status)

-- Performance-Gain:
-- Vorher: Full Scan 1000ms für 1M Dokumente
-- Nachher: Index Lookup 1ms
-- Speedup: 1000x

-- Nutzbringend für:
FILTER doc.status == 'active'
FILTER doc.age > 18
FILTER doc.created_at BETWEEN '2024-01-01' AND '2024-12-31'
```

Composite Indexes

```
-- Multiple Felder - für komplexe Filter
CREATE INDEX idx_status_age ON users(status, age)

-- Hilft bei:
FILTER doc.status == 'active' AND doc.age > 18

-- Leading Field Optimization:
-- Der erste Index-Feld sollte die höchste Selectivity haben
-- Ranking: status_age (besser) vs age_status (schlechter)
-- Grund: First field narrowed down fast
```

Sparse Indexes

```
-- Index nur auf Dokumente mit Feld
CREATE SPARSE INDEX idx_phone ON users(phone)

-- Nutzen:
-- Spart Speicher (nur 30% der Dokumente haben phone)
-- Schneller Insert/Update (nur 30% indexieren)
-- Perfekt für optionale Felder

-- Abfrage:
FOR doc IN users
  FILTER doc.phone != null
  FILTER doc.phone LIKE '%123%'
  RETURN doc
```

Partial Indexes

```
-- Index nur auf Subset (mit Filter-Bedingung)
CREATE INDEX idx_active_premium ON users(category)
  WHERE subscription_status == 'premium'

-- Spart 80% Speicher wenn nur 20% premium sind
-- 10x schneller Index-Updates

-- Muss mit passender Query genutzt werden:
FOR doc IN users
  FILTER doc.subscription_status == 'premium'
  FILTER doc.category == 'enterprise'
  RETURN doc
```

Covering Indexes

```
-- Index mit all nötigen Feldern (ohne Dokument lesen)
CREATE INDEX idx_name_email_age ON users(name, email, age)

-- Super-optimiert für Query:
FOR doc IN users
  FILTER doc.email == 'alice@example.com'
  RETURN {name: doc.name, age: doc.age}  -- Alles im Index!

-- Kein Dokument-Fetch nötig → 100-1000x schneller
```

34.3 Early Filtering Patterns

Filter-Pushdown Strategy

```
-- × FALSCH: Filter zu spät
FOR edge IN edges
  RETURN {
    from: edge.from,
    to: edge.to,
    weight: (FOR v IN vertices FILTER v._id == edge.to RETURN v.weight)[0]
  }
-- Problem: N+1 Query, 1M edges = 1M vertex lookups

--   RICHTIG: Early Filter
FOR edge IN edges
  FILTER edge.weight > 0.5  -- Filter früh
  LET target_vertex = (
    FOR v IN vertices
    FILTER v._id == edge.to
    FILTER v.category == 'active'  -- Filter auch inner
    RETURN v
  )[0]
  RETURN {
    from: edge.from,
    to: edge.to,
    target_weight: target_vertex.weight
  }
```

Collection Constraint Strategy

```
-- × Generisch - keine Collection Constraint
FILTER LIKE(name, '%alice%')  -- String-Operation für alle Doku

--   Mit Collection Filter
FOR user IN users  -- Explizite Collection wählen
  FILTER user.status == 'active'  -- Early
  FILTER LIKE(user.name, '%alice%')  -- Text-Suche nur auf aktiven
  RETURN user
```

34.4 Aggregation Optimization

GROUP BY Optimization

```
-- × Ineffizient: Alles sammeln dann groupieren
COLLECT category = doc.category
  AGGREGATE total = SUM(doc.amount)
  INTO grouped
-- Memory: O(n) - alle Doku im Memory

--   Effizient: Mit Index
FOR doc IN transactions
  FILTER doc.timestamp >= '2024-01-01'  -- Filter früh
  SORT doc.category
  COLLECT category = doc.category
    AGGREGATE total = SUM(doc.amount)
  RETURN {category, total}
-- Memory: O(distinct categories) << O(n)
```

Approximate Aggregations

```
-- Für sehr große Datasets: Approximate Counts
FUNCTION count_approximate() {
  -- HyperLogLog für massive Sets
  -- Precision vs Speed Tradeoff
  RETURN APPROX_COUNT(*) FROM large_collection
}

-- Nutzen:
-- - COUNT(*) auf 1B Dokumente: 5 Sekunden statt 5 Minuten
-- - Genauigkeit: ±2% vs ±0%
-- - Speicher: 12KB vs 8GB
```

34.5 Window Function Performance

Partition Strategy

```
-- × Falsch: Zu große Partitions
FOR sale IN sales
  WINDOW {
    PARTITION BY sale.region
    ORDER BY sale.timestamp
    RANGE CURRENT ROW TO 1000 ROWS FOLLOWING
  }
  LET total = SUM(sale.amount) OVER ...
  RETURN sale
-- Problem: Ein Region mit 10M Rows → 10M × 1000 = Massive Computation

--   Richtig: Kleinere Partitions
FOR sale IN sales
  FILTER sale.timestamp >= DATE_SUBTRACT(NOW(), 30, 'day')
  FILTER sale.region IN ['north', 'south']  -- Filter Partitions
  WINDOW {
    PARTITION BY sale.region, DATE_TRUNC(sale.timestamp, 'day')
    ORDER BY sale.timestamp
    RANGE CURRENT ROW TO 100 ROWS FOLLOWING
  }
  RETURN sale
```

34.6 Batch Operations

Bulk Insert Optimization

```
-- × Loop mit einzelnen Inserts
FOR item IN input_items
  INSERT item INTO collection
-- 10k inserts = 10k Transaktionen, 10k fsync() Calls

--   Batch Insert
LET batch = input_items
INSERT batch INTO collection
-- 1 Transaktion, 1 fsync(), ~100x schneller
```

Update Batching

```
-- × Einzelne Updates
FOR user IN users
  FILTER user.status == 'inactive'
  UPDATE user WITH {last_seen: NOW()}

--   Batch Update
LET inactive_users = (
  FOR u IN users
  FILTER u.status == 'inactive'
  RETURN u._id
)
FOR id IN inactive_users
  UPDATE {_id: id} WITH {last_seen: NOW()} IN users
```

34.7 Query Caching Patterns

Result Caching

```
from functools import lru_cache
import hashlib

class QueryCache:
    def __init__(self, ttl_seconds=300):
        self.ttl = ttl_seconds
        self.cache = {}
        self.timestamps = {}

    def cache_query(self, query_hash, result):
        self.cache[query_hash] = result
        self.timestamps[query_hash] = time.time()

    def get_cached(self, query_hash):
        if query_hash not in self.cache:
            return None

        age = time.time() - self.timestamps[query_hash]
        if age > self.ttl:
            del self.cache[query_hash]
            return None

        return self.cache[query_hash]
```

AQL Query Fragment Caching

```
-- Häufig wiederholte Subqueries cachen
FUNCTION get_active_users_cached() {
  -- Cache-Key: hash(filter_conditions)
  RETURN (
    FOR user IN users
    FILTER user.status == 'active'
    FILTER user.verified == true
    SORT user.created_at DESC
    RETURN user
  )
}

-- Nutze überall:
FOR user IN get_active_users_cached()
  RETURN user
```

34.8 Praktische Fallstudien

Case 1: E-Commerce Produktsuche

Original Query (5s):

```
FOR product IN products
  FILTER LIKE(product.name, '%laptop%')
  FILTER product.price BETWEEN 500 AND 2000
  FILTER product.category IN ['electronics', 'computers']
  SORT product.popularity DESC
  LIMIT 20
  RETURN product
```

Optimiert (100ms - 50x schneller):

```
-- 1. Index: CREATE INDEX idx_cat_price ON products(category, price)
-- 2. Full-Text: CREATE FULLTEXT INDEX idx_name ON products(name)

FOR product IN products
  FILTER product.category IN ['electronics', 'computers']
  FILTER product.price BETWEEN 500 AND 2000
  FILTER product.available == true
  SEARCH product.name IN FULLTEXT('laptop')
  SORT product.popularity DESC
  LIMIT 20
  RETURN {_key: product._key, name: product.name, price: product.price}
```

Case 2: Reporting Analytics

Original (2 Minuten):

```
FOR order IN orders
  FOR item IN order.items
    COLLECT category = item.category
      AGGREGATE total = SUM(item.quantity * item.price)
      INTO result
  RETURN {category, total}
```

Optimiert (2 Sekunden - 60x schneller):

```
FOR order IN orders
  FILTER order.status == 'completed'
  FILTER order.created_at >= '2024-01-01'
  FILTER order.created_at < '2025-01-01'
  FOR item IN order.items
    SORT item.category
    COLLECT category = item.category
      AGGREGATE total = SUM(item.quantity * item.price),
                count = COUNT(item)
      INTO result
  SORT result.category
  RETURN result
```

34.9 Performance Monitoring Dashboard

```
class QueryPerformanceMonitor:
    def __init__(self):
        self.query_stats = {}

    def record_query(self, query_hash, execution_time_ms, rows):
        if query_hash not in self.query_stats:
            self.query_stats[query_hash] = {
                'count': 0,
                'total_time': 0,
                'max_time': 0,
                'avg_time': 0
            }

        stats = self.query_stats[query_hash]
        stats['count'] += 1
        stats['total_time'] += execution_time_ms
        stats['max_time'] = max(stats['max_time'], execution_time_ms)
        stats['avg_time'] = stats['total_time'] / stats['count']

        if execution_time_ms > 1000:  # Flag slow queries
            print(f"⚠ SLOW: {query_hash} took {execution_time_ms}ms")

    def get_slowest_queries(self, top_n=10):
        sorted_queries = sorted(
            self.query_stats.items(),
            key=lambda x: x[1]['total_time'],
            reverse=True
        )
        return sorted_queries[:top_n]
```

Zusammenfassung

ThemisDB bietet mit EXPLAIN, Indexierung und Query-Refactoring Möglichkeiten für 10-100x Performance-Verbesserungen. Schlüssel sind: - **Early Filtering** - Filter so früh wie möglich - **Richtige Indizes** - Covering Indexes für Full Index Scans - **Batch Operations** - Nutze Bulk für viele Operationen - **Aggregation Optimization** - COLLECT mit Sort für Memory-Effizienz - **Monitoring** - PROFILE/EXPLAIN nutzen zur Diagnose

Mit diesen Techniken erreichen Sie Production-Grade Performance.

Kapitel 36: Kapitel 35: Data Modeling Patterns & Anti-Patterns

> *"Mit den gleichen Daten können Sie ein System entweder elegant oder chaotisch modellieren. Die richtige Struktur entscheidet über Erfolg oder Burnout."*

Überblick

Data Modeling ist die Grundlage jeder erfolgreichen Datenbankanwendung. Dieses Kapitel zeigt bewährte Patterns und warnt vor häufigen Anti-Patterns, die zu technischen Schulden führen.

Was Sie in diesem Kapitel lernen: - Document Structure Patterns (Embedded vs Referenced) - Schema Design für verschiedene Use Cases - Versionierung und Schema-Evolution - Data Integrity Patterns - Denormalisierung vs Normalisierung - Multi-Model Design Patterns - Anti-Patterns und wie man sie vermeidet

---

35.1 Embedded vs Referenced Documents

Pattern 1: Full Embedding

Geeignet für: One-to-Few Relationships

```
-- E-Commerce: Order mit Produktdetails embedded
{
  _key: "order_12345",
  customer_id: "cust_789",
  created_at: "2025-01-01T10:00:00Z",
  items: [
    {
      product_id: "prod_111",
      product_name: "Laptop",  -- Embedded - COPY der Daten
      quantity: 1,
      price_at_purchase: 999.99
    },
    {
      product_id: "prod_222",
      product_name: "Mouse",
      quantity: 2,
      price_at_purchase: 19.99
    }
  ],
  total: 1039.97
}
```

Vorteile: - Atomare Updates (Order + Items = 1 Dokument) - Keine Joins nötig - Höchste Performance für Lesezugriffe - Natürliche Denormalisierung für Snapshot-Daten

Nachteile: - × Datenduplizierung (Produkt-Name in 1000 Orders) - × Updates komplex (100 Orders mit laptop aktualisieren?) - × Speicher-Overhead

Best Practice: Nutze für Snapshot-Daten (Preis zum Kaufzeitpunkt, nicht aktuelle Produkt-Info)

Pattern 2: Reference Links

Geeignet für: One-to-Many und Many-to-Many

```
-- Order nur mit Referenzen:
{
  _key: "order_12345",
  customer_id: "cust_789",
  created_at: "2025-01-01T10:00:00Z",
  item_ids: ["oi_1001", "oi_1002"]  -- Referenzen statt Embedding
}

-- Order Items separate Collection:
{
  _key: "oi_1001",
  order_id: "order_12345",
  product_id: "prod_111",
  quantity: 1,
  price: 999.99
}

-- Abfrage mit JOIN:
FOR order IN orders
  FILTER order._key == 'order_12345'
  FOR item IN order_items
    FILTER item._id IN order.item_ids
    RETURN {
      product: item.product_id,
      qty: item.quantity,
      price: item.price
    }
```

Vorteile: - Keine Datenduplizierung - Flexible Updates (Produkt-Name einmal ändern) - Speicher-effizient - Normalisiertes Design

Nachteile: - × JOINs nötig (Performance-Hit) - × Konsistenz-Komplexität (order + items = 2 Transaktionen?) - × Komplexere Queries

Best Practice: Nutze für Live-Daten (Produkt-Info, Benutzer-Profil)

35.2 Hybrid Pattern: Optimal Denormalization

```
-- BESTE LÖSUNG: Balance zwischen beiden
{
  _key: "order_12345",
  customer_id: "cust_789",
  created_at: "2025-01-01T10:00:00Z",
  items: [
    {
      order_item_id: "oi_1001",  -- Referenz
      product_id: "prod_111",      -- Referenz
      product_name: "Laptop",      -- Denormalisiert (Snapshot)
      quantity: 1,
      price_at_purchase: 999.99   -- Snapshot (nicht ändern)
    }
  ],
  customer_name: "Alice",           -- Denormalisiert für Report
  customer_email: "alice@example.com",
  total: 1039.97,
  last_modified: "2025-01-01T10:00:00Z"
}

-- + Separate Product Collection für Live-Updates:
{
  _key: "prod_111",
  name: "Laptop",
  description: "Latest model",  -- Änderungen nur hier
  price: 899.99,  -- Aktueller Preis (nicht im Order)
  availability: "in_stock"
}
```

Strategie: - **Embedded:** Snapshot-Daten (was zum Kaufzeitpunkt galt) - **Referenced:** Live-Daten (aktuelle Zustand) - **Join bei Bedarf:** Nur wenn aktuelle Info nötig

---

35.3 Schema Evolution Patterns

Forward Compatibility

```
-- Version 1: Alte Struktur
{
  _key: "user_123",
  name: "Alice",
  email: "alice@example.com"
}

-- Version 2: Neue Felder (backward compatible!)
{
  _key: "user_123",
  name: "Alice",
  email: "alice@example.com",
  phone: "+49123456789",      -- Neu, optional
  address: {                   -- Neu, nested
    street: "Main St",
    city: "Berlin"
  },
  metadata: {                  -- Neu, extensible
    last_login: "2025-01-01",
    login_count: 42
  }
}

-- Abfrage muss robust sein:
FOR user IN users
  FILTER user.email == 'alice@example.com'
  RETURN {
    name: user.name,
    phone: user.phone ?? "unknown",  -- Null coalescing
    city: user.address.city ?? "unknown"
  }
```

Versioned Collections

```
-- Collections mit Version in _key
-- users_v1: Alte Version (archivert)
-- users_v2: Neue Version (aktiv)

-- Migration:
FOR user IN users_v1
  INSERT {
    name: user.name,
    email: user.email,
    phone: null,
    address: null,
    created_from_v1: true
  } INTO users_v2

-- Dual-Write Phase (Rollout):
-- 1. Writes zu v1 UND v2 (2 Sekunden länger)
-- 2. Reads von v2, Fallback zu v1
-- 3. Nach Verification: nur v2
-- 4. v1 Cleanup nach 30 Tagen
```

35.4 Data Integrity Patterns

Constraints und Validation

```
-- Validation in Insert Trigger
FUNCTION validate_user(user) {
  IF !LIKE(user.email, '%@%.%') THEN
    THROW ERROR('Invalid email format')
  END

  IF user.age < 0 OR user.age > 150 THEN
    THROW ERROR('Age must be 0-150')
  END

  IF !HAS(user, 'name') OR LENGTH(user.name) < 1 THEN
    THROW ERROR('Name is required')
  END

  RETURN user
}

-- Trigger bei Insert:
INSERT validate_user(new_user) INTO users
```

Referential Integrity

```
-- Before Delete: Check for References
FUNCTION can_delete_product(product_id) {
  LET order_count = LENGTH(
    FOR oi IN order_items
    FILTER oi.product_id == product_id
    RETURN oi
  )

  IF order_count > 0 THEN
    THROW ERROR(
      CONCAT("Cannot delete: ", order_count, " orders reference this product")
    )
  END

  RETURN true
}

-- Soft Delete Pattern (meist besser):
UPDATE {_id: product_id} WITH {
  deleted_at: NOW(),
  is_deleted: true
} IN products

-- Queries filtern automatisch:
FOR prod IN products
  FILTER !prod.is_deleted
  RETURN prod
```

35.5 Common Anti-Patterns & Fixes

Anti-Pattern 1: Unbounded Arrays

```
-- × FALSCH: Array wächst unbegrenzt
{
  _key: "user_123",
  comments: [  -- Kann 1M Items enthalten!
    {id: "c1", text: "Hello"},
    {id: "c2", text: "Hi"},
    ... (1M mehr)
  ]
}

-- Problem: Jeder Update des Users ändert Array
-- Memory: 1M Items im RAM
-- Performance: O(n) für jeden Insert

--   RICHTIG: Separate Collection
{
  _key: "user_123",
  comment_count: 1000000
}

{
  _key: "comment_12345",
  user_id: "user_123",
  text: "Hello",
  created_at: "2025-01-01T10:00:00Z"
}

-- Abfrage:
FOR user IN users
  FILTER user._key == 'user_123'
  FOR comment IN comments
    FILTER comment.user_id == user._key
    SORT comment.created_at DESC
    LIMIT 20
    RETURN comment
```

Anti-Pattern 2: Wide Documents

```
-- × FALSCH: Ein Dokument mit 500+ Felder
{
  _key: "product_123",
  name: "...",
  description: "...",
  color_red: 0.9,
  color_green: 0.1,
  color_blue: 0.05,
  ... (500 mehr Felder)
}

--   RICHTIG: Nested Objects für Kategorien
{
  _key: "product_123",
  name: "Laptop",
  description: "...",
  appearance: {
    color: {r: 0.9, g: 0.1, b: 0.05},
    material: "aluminium"
  },
  specs: {
    cpu: "Intel i9",
    ram: "32GB",
    storage: "1TB SSD"
  },
  pricing: {
    list_price: 1999.99,
    discount_percent: 10,
    final_price: 1799.99
  }
}
```

Anti-Pattern 3: Sparse Indexes on Many Fields

```
-- × FALSCH: Index pro optionales Feld
CREATE INDEX idx_phone ON users(phone)
CREATE INDEX idx_mobile ON users(mobile)
CREATE INDEX idx_fax ON users(fax)
-- 3 Indizes, 70% davon sind NULL

--   RICHTIG: One General Contact Index
{
  _key: "user_123",
  contact: {
    phone: "+49123456789",
    mobile: null,
    fax: null,
    email: "alice@example.com"
  }
}

CREATE SPARSE INDEX idx_contact ON users(contact)

-- Abfrage:
FOR user IN users
  FILTER user.contact.phone != null
  FILTER LIKE(user.contact.phone, '%123%')
  RETURN user
```

35.6 Multi-Model Design Patterns

Pattern: Document + Graph

```
-- Documents: User Profile
{
  _key: "user_123",
  name: "Alice",
  email: "alice@example.com"
}

-- Graph: User Relationships
{
  _key: "follows_123_456",
  _from: "users/123",
  _to: "users/456",
  followed_at: "2025-01-01",
  type: "follows"
}

-- Query: Netzwerk-Analyse
FOR user IN users
  FILTER user.email == 'alice@example.com'
  LET followers = (
    FOR follower IN users
      ANY INBOUND user GRAPH 'social_graph'
      RETURN follower
  )
  LET following = (
    FOR following_user IN users
      ANY OUTBOUND user GRAPH 'social_graph'
      RETURN following_user
  )
  RETURN {
    user: user.name,
    followers_count: LENGTH(followers),
    following_count: LENGTH(following)
  }
```

Pattern: Document + Vector + Search

```
-- Documents: Articles
{
  _key: "article_123",
  title: "Machine Learning Basics",
  content: "...",
  vector: [0.2, 0.5, 0.1, ...]  -- 1536-dim embedding
}

-- Vector Search + Full-Text
FOR article IN articles
  SEARCH article.vector ANN {
    query: [0.1, 0.4, 0.2, ...],
    top_k: 10
  }
  SEARCH PHRASE(article.title, 'machine learning', 'title')
         OR PHRASE(article.content, 'neural networks')
  RETURN {
    title: article.title,
    similarity: article._score
  }
```

35.7 Practical Migration Patterns

Blue-Green Deployment

```
-- Phase 1: Parallel Execution
-- - Write zu v1 UND v2
-- - Read von v1 (stabil)

-- Phase 2: Validation
-- - Vergleiche v1 vs v2 Ergebnisse
-- - Prüfe auf Datenabweichungen

-- Phase 3: Cutover
-- - Schreib-Lock auf v1
-- - Letzte Sync v1 → v2
-- - Schreib-Lock freigeben auf v2
-- - Reads → v2

-- Phase 4: Cleanup
-- - Nach 30 Tagen: v1 Archivieren
-- - Nach 90 Tagen: v1 Löschen
```

Zusammenfassung

Gutes Data Modeling ist Kunst und Wissenschaft: - **Embedded:** Für Snapshots (Preis zum Kaufzeitpunkt) - **Referenced:** Für Live-Daten (aktuelle Produkt-Info) - **Hybrid:**

Kombination für beste Performance - **Versioniert:** Schema-Evolution mit Sicherheit - **Validated:** Constraints at Insert-Time - **Separat:** Arrays/Felder in eigene Collections - **Nested:** Wide Documents mit Struktur - **Monitored:** Indizes auf Query-Patterns

Mit diesen Patterns bauen Sie skalierbare, wartbare Datenmodelle.

Kapitel 37: Kapitel 36: Security Hardening Playbook

> *"Security ist kein Feature, es ist Architektur. Ein sicheres System ist gebaut von innen heraus, nicht bolzen-on later."*

Überblick

Dieses Kapitel bietet Praktiker-Anleitungen für Production-Grade Security in ThemisDB. Von grundlegenden Authentifizierungspattern bis zu erweiterten Threat-Mitigation-Strategien.

Was Sie in diesem Kapitel lernen: - Network Segmentation und Firewall Rules - Authentication & Authorization Patterns - End-to-End Encryption - SQL Injection & Injection Attack Prevention - Access Control Lists (ACL) & Role-Based Access Control (RBAC) - Audit Logging & Intrusion Detection - Secrets Management - Compliance & Regulatory Frameworks - Incident Response Playbooks

---

36.1 Network Security

Firwall-Regeln

```yaml
---
rules:
  # Externe API -> ThemisDB
  - name: "Allow API Gateway"
    type: "inbound"
    protocol: "tcp"
    port: 8529
    source: "10.0.1.0/24"  # API Gateway subnet
    action: "allow"

  # Client App -> ThemisDB
  - name: "Allow App Servers"
    type: "inbound"
    protocol: "tcp"
    port: 8529
    source: "10.0.2.0/24"  # App subnet
    action: "allow"

  # Replication zwischen Nodes
  - name: "Allow Replication"
    type: "inbound"
    protocol: "tcp"
    port: 8628
    source: "10.0.3.0/24"  # Cluster subnet
    action: "allow"

  # Deny All (Default)
  - name: "Deny All"
    type: "inbound"
    protocol: "all"
    action: "deny"
```

TLS/SSL Configuration

```yaml
security:
  tls:
    enabled: true
    version: "1.3"  # Minimum
    certificate_path: "/etc/themis/certs/server.crt"
    key_path: "/etc/themis/certs/server.key"
    cipher_suites:
      - "TLS_AES_256_GCM_SHA384"
      - "TLS_CHACHA20_POLY1305_SHA256"
      - "TLS_AES_128_GCM_SHA256"
    verify_client: true  # Mutual TLS
    client_ca_path: "/etc/themis/certs/client-ca.crt"
```

36.2 Authentication & Authorization

RBAC (Role-Based Access Control)

```
-- Define roles
FUNCTION create_role(role_name, permissions) {
  RETURN INSERT {
    role: role_name,
    permissions: permissions,
    created_at: NOW()
  } INTO roles
}

-- Example: Reader role
LET reader_perms = {
  collections: {
    "*": ["read"]  -- All collections, read-only
  },
  functions: ["all"]  -- Can call all functions
}

-- Create reader role
create_role("reader", reader_perms)

-- Assign user to role
UPDATE {_id: "users/alice"} WITH {
  roles: ["reader"],
  role_updated_at: NOW()
} IN users
```

JWT Token Validation

```
import jwt
from datetime import datetime, timedelta
import hashlib

class JWTAuthenticator:
    def __init__(self, secret_key):
        self.secret_key = secret_key
        self.algorithm = "HS256"

    def create_token(self, user_id, roles, expires_in=3600):
        payload = {
            "user_id": user_id,
            "roles": roles,
            "iat": datetime.utcnow(),
            "exp": datetime.utcnow() + timedelta(seconds=expires_in)
        }
        token = jwt.encode(payload, self.secret_key, algorithm=self.algorithm)
        return token

    def verify_token(self, token):
        try:
            payload = jwt.decode(token, self.secret_key, algorithms=[self.algorithm])
            return payload
        except jwt.ExpiredSignatureError:
            raise Exception("Token has expired")
        except jwt.InvalidTokenError:
            raise Exception("Invalid token")

    def get_user_permissions(self, payload):
        user_id = payload["user_id"]
        roles = payload["roles"]
        # Lookup permissions from database
        return {"roles": roles, "timestamp": datetime.utcnow()}
```

36.3 Encryption Patterns

Field-Level Encryption

```
-- Encrypt sensitive fields before storage
FUNCTION encrypt_field(value, key) {
  -- Use ThemisDB's built-in CRYPTO_ENCRYPT function
  RETURN CRYPTO_ENCRYPT(value, key, 'AES-256-GCM')
}

-- Store with encrypted field
INSERT {
  user_id: 'user_123',
  name: 'Alice',
  email: encrypt_field('alice@example.com', encryption_key),
  ssn: encrypt_field('123-45-6789', encryption_key),
  created_at: NOW()
} INTO users

-- Decrypt on read
FOR user IN users
  FILTER user.user_id == 'user_123'
  LET decrypted_email = CRYPTO_DECRYPT(user.email, encryption_key)
  RETURN {
    name: user.name,
    email: decrypted_email
  }
```

End-to-End Encryption (E2EE)

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.backends import default_backend

class E2EEncryption:
    def __init__(self):
        self.backend = default_backend()

    def generate_keypair(self):
        """Generate RSA keypair for client"""
        private_key = rsa.generate_private_key(
            public_exponent=65537,
            key_size=4096,
            backend=self.backend
        )
        public_key = private_key.public_key()
        return private_key, public_key

    def encrypt_message(self, message, public_key):
        """Encrypt message with public key"""
        ciphertext = public_key.encrypt(
            message.encode(),
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return ciphertext.hex()

    def decrypt_message(self, ciphertext_hex, private_key):
        """Decrypt with private key (only client)"""
        ciphertext = bytes.fromhex(ciphertext_hex)
        plaintext = private_key.decrypt(
            ciphertext,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return plaintext.decode()
```

36.4 Injection Prevention

AQL Injection Protection

```
-- × UNSAFE: User input directly in query
FUNCTION unsafe_search(search_term) {
  RETURN EXECUTE(
    CONCAT("FOR doc IN collection FILTER LIKE(doc.name, '%", search_term, "%') RETURN doc")
  )
  -- Attacker can inject: %') OR (1==1) //'
}

--   SAFE: Parameterized queries
FUNCTION safe_search(search_term) {
  RETURN (
    FOR doc IN collection
    FILTER LIKE(doc.name, @search_pattern)
    RETURN doc
  )
}

-- Call with bound parameters:
safe_search('@search_pattern', '%term%')
```

Input Validation

```
import re
from typing import Any, Dict

class InputValidator:
    @staticmethod
    def validate_email(email: str) -> bool:
        pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        return re.match(pattern, email) is not None

    @staticmethod
    def validate_aql_identifier(identifier: str) -> bool:
        # Only allow alphanumeric and underscore
        pattern = r'^[a-zA-Z_][a-zA-Z0-9_]*$'
        return re.match(pattern, identifier) is not None

    @staticmethod
    def sanitize_collection_name(name: str) -> str:
        # Whitelist: only alphanumeric, underscore, dash
        sanitized = re.sub(r'[^a-zA-Z0-9_\-]', '', name)
        return sanitized.lower()

    @staticmethod
    def validate_query_size(query: str, max_size: int = 10000) -> bool:
        return len(query) <= max_size
```

36.5 Audit Logging

Comprehensive Audit Trail

```
-- Log all operations
FUNCTION audit_log(operation, user_id, resource, changes) {
  RETURN INSERT {
    timestamp: NOW(),
    operation: operation,  -- "INSERT", "UPDATE", "DELETE"
    user_id: user_id,
    resource: resource,    -- "users/123"
    changes: changes,      -- {field: old_value -> new_value}
    ip_address: @ip_address,
    user_agent: @user_agent
  } INTO audit_logs
}

-- Usage:
UPDATE {_id: 'users/123'} WITH {email: 'new@example.com'}
audit_log('UPDATE', 'admin_001', 'users/123', {
  email: 'old@example.com' -> 'new@example.com'
})
```

Immutable Audit Log

```
class ImmutableAuditLog:
    def __init__(self, db):
        self.db = db

    def append_log(self, log_entry):
        """Append-only: cannot update/delete"""
        # Insert with timestamp
        log_entry['_timestamp'] = time.time()
        log_entry['_hash'] = self._compute_hash(log_entry)

        # Previous entry's hash
        last_entry = self.db.get_last_audit_log()
        if last_entry:
            log_entry['_previous_hash'] = last_entry['_hash']

        return self.db.insert_audit_log(log_entry)

    def _compute_hash(self, entry):
        """Cryptographic hash for integrity"""
        import hashlib
        content = json.dumps(entry, sort_keys=True)
        return hashlib.sha256(content.encode()).hexdigest()

    def verify_integrity(self):
        """Detect tampering by verifying hash chain"""
        logs = self.db.get_all_audit_logs()
        for i, log in enumerate(logs):
            if i > 0:
                prev_log = logs[i-1]
                if log['_previous_hash'] != prev_log['_hash']:
                    raise Exception(f"Integrity violation at log {i}")
```

36.6 Secrets Management

Kubernetes Secrets Integration

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: themis-secrets
  namespace: themis
type: Opaque
stringData:
  # Database encryption key
  database_key: "base64_encoded_key_here"

  # TLS certificates
  tls_cert: |
    -----BEGIN CERTIFICATE-----
    ...
    -----END CERTIFICATE-----

  tls_key: |
    -----BEGIN PRIVATE KEY-----
    ...
    -----END PRIVATE KEY-----

  # API keys for external services
  stripe_api_key: "sk_live_..."
  sendgrid_api_key: "SG...."
```

Vault Integration

```python
import hvac
import os

class VaultSecretsManager:
    def __init__(self, vault_addr, token):
        self.client = hvac.Client(url=vault_addr, token=token)

    def get_secret(self, secret_path):
        response = self.client.secrets.kv.v2.read_secret_version(
            path=secret_path
        )
        return response['data']['data']

    def rotate_database_key(self):
        """Rotate encryption key - called before data re-encryption"""
        new_key = os.urandom(32)  # 256-bit key
        self.client.secrets.kv.v2.create_or_update_secret(
            path="themis/database_key",
            secret_dict={"key": new_key.hex()}
        )
        return new_key
```

36.7 Compliance & Regulatory

GDPR Compliance

```
-- Right to be forgotten: Pseudonymization
FUNCTION gdpr_pseudonymize_user(user_id) {
  LET user = DOCUMENT('users/' + user_id)

  UPDATE {_id: 'users/' + user_id} WITH {
    email: HASH(user.email),
    phone: null,
    address: null,
    name: "Anonymized User",
    gdpr_deleted_at: NOW(),
    is_pseudonymized: true
  } IN users

  -- Also delete from audit logs
  REMOVE l IN audit_logs
  FILTER l.resource == CONCAT('users/', user_id)
}
```

SOC 2 Compliance Checklist

```
## SOC 2 Type II Compliance

- [ ] **Access Control**
  - [ ] Multi-factor authentication (MFA)
  - [ ] Role-based access control (RBAC)
  - [ ] Audit logging of all access

- [ ] **Data Security**
  - [ ] End-to-end encryption (AES-256)
  - [ ] Field-level encryption for PII
  - [ ] Secure key management (Vault)

- [ ] **Availability**
  - [ ] 99.99% uptime SLA
  - [ ] Automated failover
  - [ ] Regular disaster recovery tests

- [ ] **Monitoring & Logging**
  - [ ] 24/7 intrusion detection
  - [ ] Immutable audit logs
  - [ ] Real-time alerting

- [ ] **Incident Response**
  - [ ] Documented IR procedures
  - [ ] Regular tabletop exercises
  - [ ] <1 hour incident response time
```

36.8 Incident Response Playbook

Detection

```
-- Detect unusual access patterns
FUNCTION detect_brute_force(user_id, threshold = 5) {
  LET failed_logins = LENGTH(
    FOR log IN audit_logs
    FILTER log.user_id == user_id
    FILTER log.operation == 'FAILED_LOGIN'
    FILTER log.timestamp > DATE_SUBTRACT(NOW(), 1, 'hour')
    RETURN log
  )

  IF failed_logins >= threshold THEN
    RETURN {
      alert: "BRUTE_FORCE_DETECTED",
      user_id: user_id,
      failed_count: failed_logins,
      action: "LOCK_ACCOUNT"
    }
  END

  RETURN {alert: "normal"}
}
```

Response

```
class IncidentResponse:
    def lock_account(self, user_id):
        """Lock account immediately"""
        db.update_user(user_id, {'locked': True, 'locked_at': now()})

    def notify_security_team(self, incident):
        """Alert security team"""
        slack.send_message(f"  SECURITY ALERT: {incident['alert']}")
        email.send_alert(incident)

    def revoke_tokens(self, user_id):
        """Invalidate all active tokens"""
        db.delete_user_sessions(user_id)

    def isolate_database(self):
        """Disable external access"""
        firewall.block_external_connections()

    def preserve_evidence(self):
        """Archive logs for forensic analysis"""
        backup.create_forensic_snapshot()
```

Zusammenfassung

Production-Sicherheit erfordert: - **Network Segmentation** - Firewall rules, TLS 1.3 - **Authentication** - JWT, MFA, RBAC - **Encryption** - AES-256 field-level, E2E - **Input Validation** - Parameterized queries, sanitization - **Audit Logging** - Immutable logs, integrity checks - **Secrets Management** - Vault, rotation - **Compliance** - GDPR, SOC 2, audit trails - **Incident Response** - Playbooks, automation

Mit diesen Patterns bauen Sie ein **Defense-in-Depth** System, wo mehrere Sicherheitsebenen zusammenarbeiten.

Kapitel 38: Kapitel 37: Ecosystem Integration & Extensions

> *"ThemisDB ist nicht nur eine Datenbank - es ist die Mittellage in einem größeren Ökosystem von Tools, Services und Integrationen."*

Überblick

Dieses Kapitel zeigt, wie ThemisDB mit externen Systemen integriert wird und wie Sie Custom Extensions schreiben, um ThemisDB an spezifische Anforderungen anzupassen.

Was Sie in diesem Kapitel lernen: - Native Integrations mit populären Tools - Custom Function Development - Plugin Architecture - Webhook & Event Streaming - Third-party Library Support - Custom Data Type Extensions - Performance Monitoring Integrations - Cross-database Synchronization

---

37.1 Beliebte Integrationen

Elasticsearch Integration

```
from elasticsearch import Elasticsearch
import themis_client

class ElasticsearchSync:
    def __init__(self, themis_conn, es_host='localhost:9200'):
        self.themis = themis_client.connect(themis_conn)
        self.es = Elasticsearch([es_host])

    def sync_documents_to_elasticsearch(self, collection_name, query=None):
        """Sync ThemisDB documents to Elasticsearch"""
        # Query documents from ThemisDB
        aql = f"FOR doc IN {collection_name}"
        if query:
            aql += f" FILTER {query}"
        aql += " RETURN doc"

        docs = self.themis.execute(aql)

        # Bulk insert into Elasticsearch
        from elasticsearch.helpers import bulk
        actions = [
            {
                "_index": collection_name,
                "_id": doc['_id'],
                "_source": doc
            }
            for doc in docs
        ]

        bulk(self.es, actions)
        return len(actions)

    def search_with_fallback(self, query, collection_name):
        """Search in Elasticsearch, fallback to ThemisDB"""
        try:
            # Try ES first (faster)
            results = self.es.search(
                index=collection_name,
                body={"query": {"match": {"_all": query}}}
            )
            return results['hits']['hits']
        except Exception:
            # Fallback to ThemisDB full-text search
            aql = f"FOR doc IN {collection_name} SEARCH {query} RETURN doc"
            return self.themis.execute(aql)
```

Kafka Event Streaming

```yaml
themes_db_events:
  topics:
    - name: "themis.inserts"
      partitions: 10
      retention: "7d"
      events: "INSERT operations"

    - name: "themis.updates"
      partitions: 10
      retention: "7d"
      events: "UPDATE operations"

    - name: "themis.deletes"
      partitions: 10
      retention: "7d"
      events: "DELETE operations"

producers:
  - name: "themis-cdc"
    topics: ["themis.inserts", "themis.updates", "themis.deletes"]
    format: "avro"
    compression: "snappy"

consumers:
  - name: "elasticsearch-sync"
    topics: ["themis.*"]
    group_id: "es-sync-group"

  - name: "analytics-pipeline"
    topics: ["themis.*"]
    group_id: "analytics-group"
```

Prometheus Metrics

```
from prometheus_client import Counter, Histogram, Gauge
import themis_client

class ThemisPrometheus:
    def __init__(self):
        # Operation counters
        self.query_counter = Counter(
            'themis_queries_total',
            'Total queries executed',
            ['operation_type', 'collection']
        )

        # Query latency histogram
        self.query_latency = Histogram(
            'themis_query_duration_seconds',
            'Query execution time',
            ['operation_type'],
            buckets=[0.001, 0.01, 0.1, 1, 10]
        )

        # Database size gauge
        self.db_size = Gauge(
            'themis_database_bytes',
            'Database size in bytes'
        )

        # Connection pool
        self.active_connections = Gauge(
            'themis_connections_active',
            'Active database connections'
        )

    def record_query(self, operation_type, collection, duration_ms):
        self.query_counter.labels(
            operation_type=operation_type,
            collection=collection
        ).inc()

        self.query_latency.labels(
            operation_type=operation_type
        ).observe(duration_ms / 1000)
```

37.2 Custom Function Development

Creating Custom AQL Functions

```
-- Register custom function
FUNCTION CUSTOM_DISTANCE(lat1, lon1, lat2, lon2) {
  -- Haversine distance formula
  LET R = 6371  -- Earth radius in km
  LET dLat = RADIANS(lat2 - lat1)
  LET dLon = RADIANS(lon2 - lon1)
  LET a = POWER(SIN(dLat/2), 2) + COS(RADIANS(lat1)) * COS(RADIANS(lat2)) * POWER(SIN(dLon/2
  LET c = 2 * ASIN(SQRT(a))
  RETURN R * c
}

-- Usage:
FOR user IN users
  LET distance = CUSTOM_DISTANCE(
    51.5074, -0.1278,  -- London coords
    user.latitude, user.longitude
  )
  FILTER distance < 10  -- Within 10km
  RETURN {name: user.name, distance: distance}
```

External Function Binding (C++)

```
// custom_functions.cpp
#include <aql/function_registry.h>

class CustomFunctions : public FunctionRegistry {
public:
    Value distance_haversine(
        Value lat1, Value lon1,
        Value lat2, Value lon2
    ) {
        const double R = 6371;  // Earth radius
        double dLat = (lat2.asDouble() - lat1.asDouble()) * M_PI / 180;
        double dLon = (lon2.asDouble() - lon1.asDouble()) * M_PI / 180;

        double a = sin(dLat/2) * sin(dLat/2) +
                   cos(lat1.asDouble() * M_PI / 180) *
                   cos(lat2.asDouble() * M_PI / 180) *
                   sin(dLon/2) * sin(dLon/2);

        double c = 2 * asin(sqrt(a));
        return Value::fromDouble(R * c);
    }
};

// Register at startup
REGISTER_FUNCTION("CUSTOM_DISTANCE", &CustomFunctions::distance_haversine);
```

37.3 Plugin Architecture

Creating a Plugin

```python
from themis_plugin_api import Plugin, Hook

class MyAnalyticsPlugin(Plugin):
    name = "my-analytics"
    version = "1.0.0"
    description = "Custom analytics for ThemisDB"

    def on_insert(self, collection, document):
        """Called after INSERT"""
        self.log_event('insert', collection, document)
        self.update_analytics(collection)

    def on_update(self, collection, old_doc, new_doc):
        """Called after UPDATE"""
        self.log_event('update', collection, new_doc)
        self.track_changes(old_doc, new_doc)

    def on_delete(self, collection, document):
        """Called after DELETE"""
        self.log_event('delete', collection, document)
        self.update_analytics(collection)

    def log_event(self, operation, collection, doc):
        event = {
            'operation': operation,
            'collection': collection,
            'timestamp': now(),
            'doc_id': doc['_id']
        }
        self.db.insert_event(event)

    def update_analytics(self, collection):
        # Compute analytics stats
        count = self.db.count(collection)
        avg_size = self.db.avg_document_size(collection)
        self.db.update_analytics({
            'collection': collection,
            'count': count,
            'avg_size': avg_size
        })
```

Plugin Configuration

```yaml
plugins:
  - name: my-analytics
    enabled: true
    config:
      event_log_collection: "analytics_events"
      update_interval_seconds: 60

  - name: elasticsearch-sync
    enabled: true
    config:
      es_host: "localhost:9200"
      sync_interval_seconds: 5

  - name: slack-notifications
    enabled: true
    config:
      webhook_url: "${SLACK_WEBHOOK_URL}"
      notify_on: ["high_latency", "errors", "replication_lag"]
```

37.4 Webhooks & Events

Webhook Configuration

```
-- Register webhook
FUNCTION create_webhook(url, collection, events) {
  RETURN INSERT {
    url: url,
    collection: collection,
    events: events,  -- ["insert", "update", "delete"]
    active: true,
    created_at: NOW(),
    retry_count: 0,
    last_triggered: null
  } INTO webhooks
}

-- Setup:
create_webhook(
  'https://api.example.com/webhooks/themis',
  'users',
  ['insert', 'update']
)
```

Event Payload


```json
{
  "event_id": "evt_abc123",
  "timestamp": "2026-01-01T09:00:00Z",
  "event_type": "insert",
  "collection": "users",
  "document": {
    "_id": "users/123",
    "_key": "123",
    "name": "Alice",
    "email": "alice@example.com"
  },
  "metadata": {
    "version": "1.3.1",
    "source": "direct_insert",
    "user_id": "admin_001"
  }
}
```

Webhook Handler Implementation

```python
from flask import Flask, request
import hmac
import hashlib
import json

app = Flask(__name__)
WEBHOOK_SECRET = "your-secret-key"

@app.route('/webhooks/themis', methods=['POST'])
def handle_themis_webhook():
    # Verify signature
    signature = request.headers.get('X-Themis-Signature')
    body = request.get_data()

    expected_sig = hmac.new(
        WEBHOOK_SECRET.encode(),
        body,
        hashlib.sha256
    ).hexdigest()

    if not hmac.compare_digest(signature, expected_sig):
        return {'error': 'Invalid signature'}, 401

    # Process event
    event = request.json
    if event['event_type'] == 'insert':
        handle_user_insert(event['document'])
    elif event['event_type'] == 'update':
        handle_user_update(event['document'])

    return {'ok': True}, 200

def handle_user_insert(user):
    # Send welcome email
    send_email(user['email'], 'Welcome!')

    # Add to mailing list
    add_to_newsletter(user['email'])

def handle_user_update(user):
    # Update external systems
    sync_to_crm(user)
```

37.5 Data Type Extensions

Custom Scalar Type

```cpp
// custom_types.cpp
class PhoneNumber {
public:
    std::string country_code;
    std::string area_code;
    std::string number;

    std::string toString() {
        return country_code + " (" + area_code + ") " + number;
    }

    static PhoneNumber fromString(const std::string& str) {
        // Parse format: "+1 (555) 1234567"
        PhoneNumber phone;
        // ... parsing logic
        return phone;
    }
};

// Register with ThemisDB type system
REGISTER_CUSTOM_TYPE("PhoneNumber", PhoneNumber);

// Use in AQL:
// INSERT {phone: PhoneNumber("+1 (555) 1234567")} INTO users
```

37.6 Cross-Database Synchronization

Bidirectional Sync with PostgreSQL

```
import psycopg2
import themis_client

class PostgresThemisSync:
    def __init__(self, themis_conn, pg_conn):
        self.themis = themis_client.connect(themis_conn)
        self.pg = psycopg2.connect(pg_conn)

    def sync_themis_to_postgres(self, collection, pg_table):
        """ThemisDB → PostgreSQL"""
        # Read from ThemisDB
        docs = self.themis.execute(
            f"FOR doc IN {collection} RETURN doc"
        )

        # Write to PostgreSQL
        cursor = self.pg.cursor()
        for doc in docs:
            cursor.execute(
                f"INSERT INTO {pg_table} VALUES (%s, %s, %s) "
                f"ON CONFLICT (_id) DO UPDATE SET data = %s",
                (doc['_id'], doc['_rev'], json.dumps(doc), json.dumps(doc))
            )

        self.pg.commit()
        cursor.close()

    def sync_postgres_to_themis(self, pg_table, collection):
        """PostgreSQL → ThemisDB"""
        cursor = self.pg.cursor()
        cursor.execute(f"SELECT * FROM {pg_table} WHERE updated_at > %s", (self.last_sync,))

        rows = cursor.fetchall()
        for row in rows:
            doc = self._row_to_doc(row)
            self.themis.execute(
                f"UPSERT {{'_id': {doc['_id']}}} "
                f"INSERT {doc} "
                f"INTO {collection}"
            )

        cursor.close()
```

Zusammenfassung

ThemisDB-Ökosystem bietet: - **Native Integrations** - Elasticsearch, Kafka, Prometheus - **Custom Functions** - AQL + C++ für Domain-Logik - **Plugins** -

Extensible hooks für Custom Behavior - **Webhooks** - Event-driven integrations - **Custom Types** - Domain-specific data types - **Cross-DB Sync** - Bidirektional mit PostgreSQL, MongoDB - **Open API** - Python, Go, JavaScript SDKs

Mit diesen Tools integrieren Sie ThemisDB in jedes bestehende System.

Kapitel 39: Kapitel 38: Observability & SRE Playbook

> *"Ohne Metriken, Logs und Traces bleiben Incidents Rätselraten. Observability ist die Brücke zwischen Symptom und Ursache."*

Überblick

Dieses Kapitel liefert ein praktisches Observability- und SRE-Playbook für ThemisDB-Deployments. Es bündelt Metriken, Logs, Traces, Dashboards, SLOs, Alerts und Runbooks.

Was Sie lernen: - Kernmetriken (DB, System, Netzwerk) - Log-Formate, Parsing, Korrelation - Distributed Tracing für AQL Requests - Dashboards für Latenz, Fehler, Replikation, Speicherdruck - SLO/SLI-Definitionen und Error Budgets - Alerting-Design und Rauschreduktion - Runbooks für die häufigsten Störungen - Chaos- und GameDay-Checklisten

Voraussetzungen: Basiswissen aus Monitoring (Kapitel 19) und Troubleshooting (Kapitel 27).

38.1 Metriken (What to Measure)

Core DB Metriken

- Query Latenz (p50/p95/p99)
- Query Throughput (qps)
- Slow Queries count
- Active Connections
- Replication Lag (ms)
- WAL/Commit Queue Depth
- Memory Usage (RSS, Cache Hit Rate)
- Disk IO (IOPS, Latency, Queue Depth)
- Index Hit Rate
- Cache Evictions

System Metriken

- CPU: user/system/iowait
- Memory: used, available, page faults
- Disk: iops, latency, utilization
- Network: rx/tx bytes, errors, retransmits

AQL-Spezifisch

- Query Type Mix (read/write/graph/vector)
- Cursor Count / Leaks
- Transaction Duration
- Deadlocks detected
- Timeouts per operation

38.2 Logging

Strukturierte Logs

```
{
  "ts": "2026-01-01T10:00:00Z",
  "level": "INFO",
  "service": "themisdb",
  "component": "aql",
  "event": "query_executed",
  "trace_id": "abc123",
  "span_id": "def456",
  "duration_ms": 32,
  "collection": "users",
  "operation": "READ",
  "status": "ok"
}
```

Best Practices: - JSON Lines, ein Event pro Zeile - Pflichtfelder: ts, level, service, component, trace_id - Keine PII in Logs; IDs statt Freitext - Rotate & Compress (zstd)

Log-Pipeline

- Agent: Filebeat/FluentBit (TLS, backpressure)
- Parse: grok/json; enrich (host, cluster, env)
- Store: Loki/Elastic/OpenSearch
- Retention: 7-30 Tage (Hot), 90+ Tage (Cold/Glacier)

38.3 Tracing

OpenTelemetry Setup (Go API Layer)

```
// tracing.go
import "go.opentelemetry.io/otel"
import "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracehttp"
import "go.opentelemetry.io/otel/sdk/trace"

func InitTracer(endpoint string) func() {
    exporter, _ := otlptracehttp.New(context.Background(), otlptracehttp.WithEndpoint(endpoin
    tp := trace.NewTracerProvider(trace.WithBatcher(exporter))
    otel.SetTracerProvider(tp)
    return func() { _ = tp.Shutdown(context.Background()) }
}
```

AQL Trace-Injektion

- Propagiere `traceparent` Header vom API Gateway ins AQL Layer
- Schreibe `trace_id` in Query-Context → Log-Korrelation
- Sample Rate: 1-10% in Produktion; 100% in Staging

38.4 Dashboards (Grafana)

Latenz & Throughput

Panels: - Query p50/p95/p99 (stacked per operation) - QPS split by read/write/graph/vector - Error rate (% per op)

Replikation

- Lag per follower (ms)
- Failed replications (count)
- WAL queue depth

Ressourcen

- CPU (per node)
- Memory (rss, cache hit rate)
- Disk IOPS & Latency
- Network retransmits

Cache & Index

- Cache evictions/sec
- Index hit rate
- Slow queries over threshold

38.5 SLI/SLO & Error Budgets

Beispiel-SLIs

- Availability: 99.95% (HTTP 2xx/3xx / total)
- Latency: p99 < 200 ms für Reads, < 400 ms für Writes
- Durability: 0 Datenverlust (RPO <= 60s)
- Freshness: Replication Lag < 2s (p99)

Error Budget Policy

- Monatsbudget: (1 - SLO) * Zeit
- Breach: Freeze Releases, Fokus auf Reliability
- Burn Alerts: Warn bei 25/50/75% Verbrauch

38.6 Alerting Design

Ziele: Früh, präzise, wenig Rauschen.

- Multi-Window, Multi-Burn-Rate (1h/6h) für SLO Burn
- Symptom-basierte Alerts (p99 Latenz hoch, Error Rate > 1%)
- Ursache-Alerts nachrangig (Disk 95% voll)
- Deduping + Grouping im Alertmanager
- Quiet Hours + Ownership (Runbook-Link, Pager-Rotation)

Beispiele: - `latency_p99_gt_200ms` (for 15m & 6h) - `error_rate_gt_1pct` - `replication_lag_gt_2s` - `disk_util_gt_85pct` - `cache_evictions_spike`

38.7 Runbooks (Kurzfassung)

Hohe Latenz

- Check: p99 Read/Write? Bestimmte Collections?
- EXPLAIN Slow Query; Index vorhanden?
- Cache Hit Rate < 90%? Memory Druck?
- CPU iowait hoch? → Storage prüfen
- Rate-Limit/Backpressure aktivieren

Replication Lag

- Netzwerk: retransmits, bandwidth
- Follower CPU/IO bottleneck
- WAL queue depth; throttle writes
- Rebalance followers

OOM/Memory Pressure

- Cache Size begrenzen
- Off-Heap aufteilen (vector buffers)

- Identify large queries; add LIMIT/PROJECTION

Disk Full

- Log-Retention kürzen
- Offload Cold Data (Tiered Storage)
- Rebuild Indizes, Vacuum

38.8 Chaos & GameDays

- Failure Modes: Node down, Network partition, Disk full, High latency storage, Poison messages
- Hypothesen definieren, erwartetes Verhalten notieren
- Blast Radius klein starten (1 node, 10 min)
- Erfolgskriterien: SLO halten? Auto-Heal? Alerts ausgelöst?
- Nachbereitung: Learnings, Tickets, Fixes, erneutes Testen

38.9 Logging & Tracing Sampling Patterns

- Dynamic Sampling: Erhöhe Rate bei Errors
- Tail Sampling: Keep slow queries, drop fast
- PII Scrubbing Filter vor Export

38.10 Capacity Planning

- Wachstumsrate Traffic & Daten
- Headroom-Ziel: 40% CPU, 50% IO, 30% Memory
- Load-Test vierteljährlich; extrapoliere 12 Monate
- Trigger: >70% baseline → Scale-out

38.11 Oncall Playbook Essentials

- Runbook Link in jedem Alert
- 2nd-level Escalation (DBA/SRE)
- Kommunikationskanal: Incident Room + Status Page
- Postmortem innerhalb 48h, Action Items mit Owner/ETA

Zusammenfassung

Observability bündelt Metriken, Logs, Traces und klare SLOs. Mit sauberen Dashboards, schlankem Alerting und erprobten Runbooks verkürzen Sie MTTR massiv und verhindern Blindflüge im Betrieb.

Kapitel 40: Kapitel 39: Performance Tuning Cookbook

> *"Performance ist kein Zufall, sondern eine Sammlung kleiner, konsistenter Entscheidungen. Dieses Kochbuch liefert die Rezepte."*

Überblick

Ein praxisorientiertes Tuning-Kochbuch für ThemisDB. Jede Sektion enthält Symptom → Diagnose → Fix mit AQL-Beispielen und System-Tuning.

Was Sie lernen: - Schnelle Checklisten für Latenz, Durchsatz, Speicher, IO - Tuning für Queries, Indizes, Cache, Transactions - Storage/OS/Netzwerk-Optimierungen - Batching, Queueing, Backpressure - Vector- und Graph-Workload Tuning - Beispiel-Konfigurationen für Prod

---

39.1 Quick Tuning Checklist

- **Latenz hoch?** EXPLAIN, Index-Pfade, Projection pushdown, LIMIT
- **Durchsatz gering?** Batching, Parallelisierung, Verbindungspool erhöhen
- **CPU hoch?** Sort/Regex/Full-Scan reduzieren, Index nutzen, Cache-Hit prüfen
- **IO hoch?** Covering Index, Kompression (zstd), Cold Data auslagern
- **Memory hoch?** Cache begrenzen, Streaming statt Materializing, LIMIT
- **Lock/Deadlock?** Konsistente Lock-Order, kürzere Transaktionen

---

39.2 Query Patterns (Fixes)

Early Projection & Filter

```
-- × FALSCH: Späte Projektion
FOR u IN users
  FILTER u.status == 'active'
  RETURN u  -- großes Dokument

--   RICHTIG: Früh projizieren
FOR u IN users
  FILTER u.status == 'active'
  RETURN { _key: u._key, name: u.name, email: u.email }
```

Avoid N+1

```
-- × FALSCH: Pro Zeile Subquery
FOR o IN orders
  RETURN {
    o: o,
    items: (
      FOR i IN order_items FILTER i.order_id == o._key RETURN i
    )
  }

--   RICHTIG: Pre-Aggregate
LET items_by_order = (
  FOR i IN order_items
    COLLECT order_id = i.order_id INTO items
    RETURN { order_id, items }
)

FOR o IN orders
  LET bundle = FIRST(FOR b IN items_by_order FILTER b.order_id == o._key RETURN b.items)
  RETURN { o, items: bundle }
```

Range Queries mit Index

```
CREATE INDEX idx_ts ON events(timestamp)

FOR e IN events
  FILTER e.timestamp >= @from AND e.timestamp <= @to
  RETURN e
```

Graph Traversal Tuning

```
-- Limit depth and fanout
FOR v, e, p IN 1..3 OUTBOUND 'users/alice' GRAPH 'social'
  PRUNE LENGTH(p.vertices) > 100  -- Begrenze
  FILTER p.edges[*].weight ALL >= 0.5
  RETURN v
```

39.3 Batching & Parallelism

Batch Insert/Update

```
LET batch = @input_docs
INSERT batch INTO users  -- 1 Transaktion
```

Client-Side Parallel

```
from concurrent.futures import ThreadPoolExecutor

def run_in_parallel(queries):
    with ThreadPoolExecutor(max_workers=8) as ex:
        return list(ex.map(client.execute, queries))
```

Backpressure

```
import asyncio

class QueueWorker:
    def __init__(self, maxsize=1000):
        self.q = asyncio.Queue(maxsize=maxsize)

    async def produce(self, items):
        for item in items:
            await self.q.put(item)  # block when full

    async def consume(self, worker):
        while True:
            item = await self.q.get()
            await worker(item)
            self.q.task_done()
```

39.4 Cache & Memory

- Target: Cache Hit Rate > 90%
- Limit result set sizes (pagination)
- Use STREAMING_CURSOR for große Resultsets
- Avoid massive IN lists → temp set collection

```
-- Streaming Cursor
FOR doc IN large_collection
  FILTER doc.status == 'active'
  LIMIT 0, 100000
  OPTIONS {stream: true}
  RETURN doc
```

39.5 Storage & IO

Kompression

- `storage.compression = zstd`
- `wal.compression = zstd`
- Blockgröße 4-8KB testen

SSD/NVMe Tuning (Linux)

```
sudo nvme set-feature -f 2 -v 0  # Disable write cache if battery-backed
sudo nvme set-feature -f 0x0b -v 0x1  # Latency monitor enable
```

Filesystem

- ext4 mit `noatime`, `discard`, `nodiscard` für Performance abwägen
- XFS für hohe Parallelität

39.6 Netzwerk

- MTU prüfen (Jumbo Frames nur konsistent nutzen)
- TCP: `net.ipv4.tcp_tw_reuse=1`, `tcp_fin_timeout=15`
- Verbindungen poolen; Keep-Alive aktivieren

39.7 Vektor-Workloads

Index & Quantization

- HNSW: M=16-32, efConstruction=200, efSearch=64-128
- PQ: Codebooks 8-16, 8 Bit
- IVF: nlist ~ sqrt(N)

Batch Embedding

```
chunks = [docs[i:i+32] for i in range(0, len(docs), 32)]
for chunk in chunks:
    vectors = embed(chunk)
    client.insert_vectors('emb', vectors)
```

ANN Warmup

- Preload top collections on startup
- Pin hot vectors in memory tier

39.8 Graph-Workloads

- Begrenze Fanout und Depth
- Precompute Communities (Louvain) für häufige Queries
- Materialisierte Pfade für Hot Paths

39.9 Transactions & Locking

- Kürzere Transaktionen; kleinere Batches
- Lock-Order strikt definieren
- Deadlock-Timeout niedrig (z.B. 5s) → Retry mit Backoff

```
import time

def with_retry(tx_func, attempts=3):
    for i in range(attempts):
        try:
            return tx_func()
        except Deadlock:
            time.sleep(0.1 * (2 ** i))
    raise
```

39.10 Configuration Templates

themis.conf (Performance Focus)

```
aql:
  stream_results: true
  max_query_memory_mb: 1024
  profiling_enabled: true

cache:
  size_mb: 8192
  eviction_policy: lru

wal:
  compression: zstd
  sync_interval_ms: 20

network:
  max_connections: 2000
  keepalive: 60
```

Linux sysctl

```
net.core.somaxconn = 1024
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_tw_reuse = 1
vm.swappiness = 10
vm.max_map_count = 262144
```

39.11 Benchmark Harness

```
import time

class BenchHarness:
    def __init__(self, client):
        self.client = client

    def run_case(self, name, query):
        start = time.time()
        result = self.client.execute(query)
        dur = (time.time() - start) * 1000
        return {"name": name, "duration_ms": dur, "rows": len(result)}

    def run_suite(self, cases):
        return [self.run_case(n, q) for n, q in cases]
```

Zusammenfassung

Dieses Kochbuch liefert schnelle Diagnosen und konkrete Fixes. Beginnen Sie mit EXPLAIN/PROFILE, optimieren Sie Filter/Projektion, nutzen Sie Indizes, batchen Sie Schreibvorgänge, begrenzen Sie Fanout, und sichern Sie Ressourcen durch Cache- und IO-Tuning. Wiederholen, messen, automatisieren.

Kapitel 41: Kapitel 40: Data Governance & Compliance

> *"Daten ohne Governance sind Haftungsrisiko. Governance schafft Vertrauen, Compliance macht es prüfbar."*

Überblick

Ein praxisnahes Governance- und Compliance-Kapitel für ThemisDB: Policies, Zugriff, Maskierung, Retention, Audits, Privacy (GDPR), und Kontrollnachweise.

Was Sie lernen: - Governance-Framework (Policies, Ownership, Stewardship) - Access Control & Least Privilege - Data Classification & Masking - Lifecycle & Retention - Audit Trails & Evidence - GDPR/DSGVO: Rechte, Pseudonymisierung, Löschung - SOX/SOC2/ISO27001 Anforderungen - Kontroll-Checklisten und Runbooks

---

40.1 Governance Operating Model

- **Ownership:** Jede Collection hat Owner (Team), Steward (Data Quality), Custodian (Ops)
- **Policy Catalog:** Zugriff, Retention, Encryption, Backup, Sharing
- **Review Zyklen:** Quartalsweise Policy-Review, jährliche Controls-Audits
- **Change Control:** RFC → Peer Review → Approval → Rollout → Evidence

---

40.2 Data Classification

Stufen definieren (z.B. PUBLIC, INTERNAL, CONFIDENTIAL, STRICT).

```
classification:
  levels:
    - name: PUBLIC
      description: "Keine Sensibilität"
    - name: INTERNAL
      description: "Nur intern"
    - name: CONFIDENTIAL
      description: "PII/finanziell"
    - name: STRICT
      description: "Gesundheit, Strafverfolgung"
```

Regeln: - Standard = INTERNAL - CONFIDENTIAL/STRICT → Encryption + Access Approval - Masking im UI/Logs für CONFIDENTIAL/STRICT

40.3 Access Control & Least Privilege

- RBAC-Modelle (Reader, Writer, Admin, Auditor)

- JIT Access mit Ablauf (z.B. 24h)
- Break-Glass Accounts (mit separatem Audit)
- Quarterly Access Review

```
-- Grant role
INSERT {
  user_id: 'alice',
  role: 'reader',
  granted_by: 'security',
  expires_at: DATE_ADD(NOW(), 1, 'day')
} INTO access_grants
```

40.4 Data Masking

Dynamic Masking

```
FUNCTION mask_email(email) {
  RETURN CONCAT(SUBSTRING(email, 0, 2), '***', SUBSTRING(email, POSITION(email, '@')-1))
}

FOR user IN users
  RETURN {
    name: user.name,
    email: mask_email(user.email)
  }
```

Tokenization

```
import secrets

tokens = {}

def tokenize(value):
    token = secrets.token_hex(8)
    tokens[token] = value
    return token

def detokenize(token):
    return tokens.get(token)
```

40.5 Data Lifecycle & Retention

- **Ingest:** Klassifizierung, Validation, Encryption
- **Store:** Encryption-at-rest, Access Controls, Audit Logs
- **Use:** Masking, Purpose Limitation, Minimal Disclosure
- **Share:** Anonymize/Pseudonymize, DPA prüfen
- **Archive:** Cold Storage mit Verschlüsselung

- **Delete:** Right-to-be-forgotten Prozesse

Retention Beispiel:

```
retention:
  users: 7y
  logs: 180d
  audit_logs: 365d
  pii: 2y
```

40.6 Audit & Evidence

- Immutable Audit Log (Kapitel 36)
- Evidence Store: Tickets, Screenshots, CLI Output, Hashes
- Kontroll-Nachweise per Control-ID (ISO27001 Annex A, SOC2 CC)

```
- Policy: AC-3 v1.2
- Evidence:
  - access_grants export (signed)
  - quarterly review report (PDF)
  - IAM config screenshot
  - audit log hash chain verification
```

40.7 Privacy (GDPR)

Betroffenenrechte

- Auskunft, Berichtigung, Löschung, Einschränkung, Übertragbarkeit, Widerspruch

```
-- Right to Erasure (Pseudonymize)
FUNCTION gdpr_delete(user_id) {
  UPDATE {_id: 'users/' + user_id} WITH {
    name: 'Anonymized',
    email: null,
    phone: null,
    gdpr_deleted_at: NOW(),
    is_deleted: true
  } IN users

  REMOVE d IN audit_logs
  FILTER d.resource == CONCAT('users/', user_id)
}
```

Data Processing Register

- Zweck, Kategorien, Empfänger, Speicherorte, Löschfristen
- DPAs mit Prozessoren dokumentieren

40.8 Compliance Frameworks (SOX, SOC2, ISO27001)

- **SOX:** Change-Management, Segregation of Duties, Evidence für Finanzsysteme
- **SOC2:** Security/Availability/Confidentiality; Controls: Access, Change, Incident, Monitoring
- **ISO27001:** ISMS, Risikoanalyse, Controls Annex A (z.B. A.8 Asset Management, A.9 Access Control)

Kontroll-Mapping Beispiel:

```
- A.9.1.2 (Access Control): RBAC + Quarterly Review + JIT
- A.12.4.1 (Logging): Immutable Audit Log, 365d retention
- A.12.6.1 (Malware Protection): EDR auf DB-Hosts
- A.17.1.1 (Continuity): DR-Plan + Tests halbjährlich
```

40.9 Controls & Checklisten

```
## Access Controls
- [ ] RBAC Rollen definiert
- [ ] JIT Access aktiviert
- [ ] Break-glass dokumentiert
- [ ] Quarterly Reviews durchgeführt

## Data Protection
- [ ] Encryption at rest (AES-256)
- [ ] TLS 1.3 in transit
- [ ] Field-Level Encryption für PII
- [ ] Masking im UI und Logs

## Monitoring & Audit
- [ ] Immutable Audit Logs
- [ ] Alerting auf Policy-Verletzungen
- [ ] Evidence Store mit Hash-Kette
```

Zusammenfassung

Governance setzt klare Verantwortlichkeiten, Classification, Zugriffsprinzipien und Retention. Compliance liefert Nachweise gegenüber Auditoren und Gesetzgebern. Mit Masking, Audit Trails, JIT Access und robusten Löschprozessen bleibt ThemisDB konform und vertrauenswürdig.

Kapitel 42: Kapitel 41: Hands-on Labs – Deployment, Index-Tuning, Vector-Suche

> *"Lernen durch Tun: Diese Labs führen Sie Schritt für Schritt durch echte Produktionsaufgaben."*

Überblick

Drei geführte Labs mit klaren Zielen, Schritt-für-Schritt-Anleitungen, Verifikation und Aufräumen.

Labs: - Lab A: Schnellstart-Deployment (Docker) + Smoke Tests - Lab B: Index-Tuning & Query-Optimierung mit EXPLAIN/PROFILE - Lab C: Vector-Suche End-to-End (Embeddings erzeugen, Indexieren, Query)

Voraussetzungen: Docker/WSL, themis_client CLI oder HTTP, Grundkenntnisse AQL.

Lab A: Deployment & Smoke Tests (30-45 min)

Ziel

- ThemisDB per Docker starten
- Healthcheck & Basiskonfiguration prüfen
- Erste Write/Read-Query ausführen

Schritte

1) **Container starten**

```
docker run -d --name themisdb -p 8529:8529 themisdb/server:latest
```

2) **Healthcheck**

```
curl -s http://localhost:8529/_admin/status | jq '.server.status'
```

Erwartet: `"ok"`

3) **Admin-Login setzen (falls nötig)**

```
curl -X POST http://localhost:8529/_admin/setup --data '{"password":"StrongPass!"}'
```

4) **Smoke-Test Write/Read**

```
-- write
INSERT { _key: "alice", name: "Alice" } INTO users
-- read
FOR u IN users FILTER u._key == "alice" RETURN u
```

5) **Persistenz prüfen**

```
docker restart themisdb
curl -s http://localhost:8529/_api/document/users/alice
```

Aufräumen

```
docker rm -f themisdb
```

Lab B: Index-Tuning & Query-Optimierung (40-60 min)

Ziel

- Langsame Query identifizieren
- EXPLAIN/PROFILE nutzen
- Index anlegen und Improvement messen

Dataset vorbereiten

```
FOR i IN 1..200000
  INSERT {
    _key: CONCAT('user_', i),
    email: CONCAT('user', i, '@example.com'),
    status: i % 5 == 0 ? 'inactive' : 'active',
    created_at: DATE_NOW()
  } INTO users
```

Baseline messen

```
PROFILE FOR u IN users
  FILTER u.email == 'user199999@example.com'
  RETURN u
```

Notieren: p99 Latenz.

EXPLAIN prüfen

```
EXPLAIN FOR u IN users
  FILTER u.email == 'user199999@example.com'
  RETURN u
```

Erwartet: CollectionScan (schlecht).

Index anlegen

```
CREATE INDEX idx_users_email ON users(email)
```

Nachher messen

```
PROFILE FOR u IN users
  FILTER u.email == 'user199999@example.com'
  RETURN u
```

Erwartet: IndexNode, stark reduzierte Latenz. Dokumentieren Sie Vor/Nach.

Optionale Optimierungen

- Projection pushdown: nur benötigte Felder returnen
- LIMIT 1 setzen, wenn eindeutig

Aufräumen

```
DROP INDEX users.idx_users_email
TRUNCATE users
```

Lab C: Vector-Suche End-to-End (45-75 min)

Ziel

- Texte einbetten
- Vektor-Index aufbauen
- Ähnlichkeitssuche ausführen

Setup

1) **Kollektion anlegen**

```
CREATE COLLECTION articles
```

2) **Beispiel-Dokumente einfügen**

```
INSERT {
  _key: "art1",
  title: "Graph Algorithms",
  content: "Graphs, shortest paths, centrality",
  vector: null
} INTO articles
INSERT {
  _key: "art2",
  title: "Neural Networks",
  content: "Deep learning, backpropagation",
  vector: null
} INTO articles
INSERT {
  _key: "art3",
  title: "Databases",
  content: "Indexing, transactions, sharding",
  vector: null
} INTO articles
```

3) **Embeddings erzeugen (Client-seitig, Beispiel Python)**

```
from sentence_transformers import SentenceTransformer
import themis_client

client = themis_client.connect()
model = SentenceTransformer('all-MiniLM-L6-v2')

articles = client.execute("FOR a IN articles RETURN {key: a._key, text: a.content}")
for a in articles:
    vec = model.encode(a['text']).tolist()
    client.execute(
        "UPDATE { _key: @key } WITH { vector: @vec } IN articles",
        bind_vars={"key": a['key'], "vec": vec}
    )
```

4) **Vektor-Index anlegen**

```
CREATE VECTOR INDEX idx_articles_vec ON articles(vector) WITH {dimensions: 384, type: "hnsw",
```

5) **Ähnlichkeitssuche**

```
LET query_vec = @query_vec
FOR doc IN articles
  SEARCH doc.vector ANN { query: query_vec, top_k: 3 }
  RETURN { _key: doc._key, title: doc.title, score: doc._score }
```

Bind Vars: `query_vec` = Embedding eines Suchtexts, z.B. "machine learning".

6) **Relevanz prüfen** - Erwartet: "Neural Networks" weit oben bei ML-Query - Score dokumentieren

Optionen

- efSearch erhöhen für bessere Recall
- PQ/IVF nutzen für größere Datenmengen
- Caching von Embeddings

Aufräumen

```
TRUNCATE articles
```

Checkliste (Abgehakt, wenn Lab bestanden)

- [ ] Lab A: Container-Start, Healthcheck, Smoke Query
- [ ] Lab B: EXPLAIN/PROFILE vor/nach, Index Speedup dokumentiert
- [ ] Lab C: Embeddings erzeugt, Vektor-Index erstellt, ANN Query ausgeführt

Nächste Schritte: Kombinieren Sie Labs: Deploy → Index-Tuning → Vector-Suche auf realen Datensätzen.

Kapitel 43: Appendix D: Feature Status Matrix

Version: 1.3.0
Stand: 30. Dezember 2025
Kategorie: Feature-Übersicht und Implementierungsstatus

Zweck dieses Appendix

Dieser Appendix bietet eine vollständige, strukturierte Übersicht aller ThemisDB-Features mit aktuellem Implementierungsstatus, Dokumentationsverweisen und Roadmap-Planung. Dies dient als zentraler Referenzpunkt für:

- **Entwickler**: Welche Features sind verfügbar? Wo ist der Code?
- **Architekten**: Welche Capabilities hat ThemisDB? Was fehlt noch?
- **Projektmanager**: Was ist Production-Ready? Was ist geplant?
- **Nutzer**: Welche Funktionen kann ich nutzen?

Status-Definitionen

Symbol	Status	Bedeutung
	Production-Ready	Vollständig implementiert, getestet (>85% Coverage), dokumentiert, performance-validated
	MVP Complete	Kernfunktionalität vorhanden, stabil, aber ggf. noch Optimierungen ausstehend
	In Development	Aktiv in Entwicklung, teilweise funktional, noch nicht production-grade
	Design Phase	Spezifikation vorhanden, Implementierung geplant
	Planned	Auf Roadmap, noch keine Spezifikation
○	**Backlog**	Idee/Anforderung dokumentiert, keine aktive Planung
×	**Not Planned**	Bewusst nicht im Scope

Zusammenfassung

Aktuelle Reife (Stand Dez 2025): - **Core Database**: Production-Ready (85%+ Test Coverage) - **Multi-Model Support**: Production-Ready (Relational, Graph, Vector, Document, Time-Series) - **Security & Compliance**: Production-Ready (DSGVO, BSI C5, Audit Logging) - **Horizontal Scaling**: Production-Ready (2/4/8-Node Clusters, 91% Efficiency) - **LLM/RAG Integration**: Partially Ready (Semantic Cache , Hybrid Search) - **Enterprise Authorization**: In Design (Apache Ranger Integration)

Vollständige Feature-Matrix siehe: - Admin Tools: `feature_matrix.md` - Features Overview: `features_overview.md` - Sharding Status: Chapter 16 (Horizontal Scaling) - Security Status: Chapter 10 (Section 10.2) - Query Optimization: Chapter 15 (Section 15.4)

Version History: - 1.3.0 (2025-12-30): Umfassendes Update mit allen Features bis Dez 2025

Kapitel 44: Appendix E: Incident Response Runbooks

> *"Ein Runbook erspart dir in der Krise 10 Stunden Debugging. Schreib sie jetzt, nutze sie später."*

Überblick

Detaillierte Playbooks für die häufigsten Production-Incidents in ThemisDB: Symptome, Diagnose, Fixes, Verification, Kommunikation.

E.1 Incident: Query Latency > 1s (p99)

Symptom

- Alerts: `themis_query_latency_p99_gt_1000ms` für >5 Min
- User Report: "Suche dauert 10+ Sekunden"

Diagnose (5 min)

```
curl -s http://localhost:8529/_admin/stats | jq '.queries[-10:] | .[].latency_ms'

curl -s http://localhost:8529/_admin/slow-queries?threshold=500 | jq '.queries[0:3]'

curl -s http://localhost:8529/_api/collection/users/counts | jq '.count'

aql "EXPLAIN <query>"
```

Root Cause Analysis (Pick one)

A. Missing Index

```
-- EXPLAIN zeigt CollectionScan?
EXPLAIN FOR u IN users FILTER u.email == 'alice@example.com' RETURN u
-- Output: type: "CollectionScan" → NO INDEX

-- Fix:
CREATE INDEX idx_email ON users(email)

-- Verify:
EXPLAIN ... -- sollte IndexNode zeigen
```

B. Poor Index Choice

```
-- EXPLAIN zeigt wrong Index?
-- (z.B. idx_created_at bei Filter auf status)

-- Fix:
DROP INDEX users.old_index
CREATE INDEX idx_status_created ON users(status, created_at)

-- Query-Hint:
FOR u IN users
  FILTER u.status == 'active'
  SORT u.created_at DESC
  OPTIONS {indexHint: 'idx_status_created'}
  RETURN u
```

C. Huge Result Set (Memory Pressure)

```
-- x Returns 1M rows
FOR u IN users
  FILTER u.status == 'active'
  RETURN u  -- No LIMIT!

--   Paginated
FOR u IN users
  FILTER u.status == 'active'
  LIMIT @offset, 100
  RETURN u
```

D. Expensive Operations (Regex, Full-Text)

```
-- x Regex auf 1M Docs
FOR u IN users
  FILTER u.name =~ '.*alice.*'
  RETURN u

--   Use Index + Full-Text
FOR u IN users
  FILTER u.status == 'active'  -- Reduce set first
  SEARCH u.name IN FULLTEXT('alice')  -- Then search
  RETURN u
```

Fix (1-5 min)

```
aql "CREATE INDEX idx_fix ON collection(field)"
```

Verify (2 min)

```
time aql "<query>"  # Should be <100ms now


journalctl -u themis -n 20 | grep ERROR
```

Communicate

- Slack: "Query latency incident resolved: Added index on `collection.field`. P99 now 50ms."
- Incident Ticket: Mark RESOLVED, link to index creation timestamp

---

E.2 Incident: High Memory Usage (RSS > 80%)

Symptom

- Memory Usage Gauge: > 85% of available
- OOM Killer might start: `systemctl status themis | grep killed`

Diagnose (3 min)

```
free -h
top -p $(pgrep themisdb)

curl -s http://localhost:8529/_admin/cache | jq '.usage_mb'

curl -s http://localhost:8529/_admin/slow-queries?threshold=1 | jq '.queries[].memory_mb'

curl -s http://localhost:8529/_admin/cursors | jq 'length'
```

Root Cause Analysis

A. Large Result Set Materializing

```
-- x Materializes all rows
FOR doc IN large_collection
  FILTER doc.type == 'report'
  RETURN doc

--   Stream results
FOR doc IN large_collection
  FILTER doc.type == 'report'
  OPTIONS {stream: true}
  RETURN doc
```

B. Cache Size Too Large

```
cache:
  size_mb: 16384  # Too big for 32GB system

cache:
  size_mb: 8192   # ~25% of total RAM
```

C. Cursor Leak (Clients not closing)

```
cursor = client.query("FOR d IN collection RETURN d")

with client.query("FOR d IN collection RETURN d") as cursor:
    for doc in cursor:
        process(doc)
```

Fix (5-10 min)

Quick Win: Restart

```
sudo systemctl restart themis
```

Better: Graceful Drain

```
curl -X POST http://localhost:8529/_admin/readonly

sleep 30

sudo systemctl restart themis

curl -X POST http://localhost:8529/_admin/writable
```

Verify

```
free -h
curl -s http://localhost:8529/_admin/cache | jq '.usage_mb'
```

E.3 Incident: Replication Lag > 5s

Symptom

- Alert: `themis_replication_lag_ms > 5000`
- Secondary reads stale data (>5s behind Primary)

Diagnose (2 min)

```
curl -s http://localhost:8529/_admin/replication/lag

curl -s http://localhost:8529/_admin/stats | jq '.writes_per_sec'

ssh follower-node "top -bn1 | head -15"

ping -c 10 follower-ip | tail -1
```

Root Cause Analysis

A. Follower Network Slow / Packet Loss

```
iperf3 -c follower-ip -t 10  # Should be >100 Mbps
mtr follower-ip  # Check for packet loss
```

B. Follower Disk IO Bottleneck

```
iostat -x 1 5 | grep sda
```

C. Follower CPU Saturated

```
top -p $(pgrep themis)
```

Fix (5-30 min)

Short-term: Manual Catchup

```
aql "UPDATE config SET wal_read_buffer_mb = 256"

sudo systemctl restart themis
```

Medium-term: Throttle Primary

```
replication:
  max_writes_per_sec: 1000  # Slow down writes slightly
  follower_lag_warn_ms: 3000
```

Verify

```
curl -s http://localhost:8529/_admin/replication/lag | jq '.lag_ms'
```

E.4 Incident: Deadlock Detected

Symptom

- Error: `ERROR: Deadlock detected (Transaction A ↔ B)`
- Client Retry: Usually succeeds on 2nd attempt

Diagnose (1 min)

```
curl -s http://localhost:8529/_admin/stats | jq '.deadlocks_total'
```

Root Cause Analysis

A. Lock Ordering Inconsistency

```
-- Transaction A:
BEGIN
  LOCK collection1
  LOCK collection2
COMMIT

-- Transaction B (opposite order - DEADLOCK):
BEGIN
  LOCK collection2
  LOCK collection1
COMMIT

-- Fix: Enforce strict lock ordering (e.g. alphabetical)
```

B. Long Transactions

```
-- × 2-minute transaction
BEGIN
  FOR item IN items LIMIT 1000000
    UPDATE item WITH {updated: true} IN items
COMMIT

--   Break into batches
FOR batch_start IN 0..1000000 STEP 10000
  BEGIN
    FOR item IN items
      FILTER item._id >= batch_start AND item._id < batch_start + 10000
      UPDATE item WITH {updated: true}
  COMMIT
```

Fix (Immediate)

- **Client Side:** Add exponential retry (most deadlocks resolve on 2nd attempt)
- **Code:** Review and fix lock ordering (alphabetical, same order everywhere)
- **Monitoring:** If deadlocks > 10/day, escalate

Verify

```
watch -n 60 'curl -s http://localhost:8529/_admin/stats | jq ".deadlocks_total"'
```

E.5 Incident: Disk Space Critical (>95%)

Symptom

- Alert: `themis_disk_usage_percent > 95`
- Cannot write new data

Diagnose (2 min)

```
df -h /data/themis

du -sh /data/themis/* | sort -h

du -sh /var/log/themis*
```

Root Cause Analysis

A. WAL / Logs Explosion

```
ls -lh /data/themis/wal/
ls -lh /var/log/themis*
```

B. Large Data Import (Staging)

```
aql "TRUNCATE staging_collection"
aql "DROP COLLECTION temp_*"
```

C. Backup Staging Not Cleaned

```
ls -lh /data/themis-backups/
rm /data/themis-backups/backup-*.tar.gz
```

Fix (5-15 min)

Immediate:

```
journalctl --vacuum=2d

find /data/themis/wal -mtime +7 -delete

aql "TRUNCATE staging"

df -h /data/themis
```

Medium-term:

```
logging:
  retention_days: 14
  max_file_size_mb: 100

wal:
  retention_files: 20  # Keep last 20 files
```

E.6 Incident: Database Won't Start (Corruption)

Symptom

- Startup Error: `ERROR: Corrupted block at offset 12345`
- Service Down: Cannot restart

Diagnose (2 min)

```
journalctl -u themis -n 50 | tail -20

themisdb recover /data/themis

fsck /dev/sda1
```

Root Cause Analysis

A. Unclean Shutdown (Power Loss) - **Fix:** Run WAL recovery (automatic on restart)

B. Disk Corruption (Bad Sector) - **Fix:** Replace disk, restore from backup

C. RocksDB Internal Corruption (Rare) - **Fix:** Backup data, rebuild database

Fix

Option 1: Automatic Recovery (Most Common)

```
sudo systemctl start themis
```

Option 2: Manual WAL Recovery

```
themisdb recover --wal-only /data/themis
sudo systemctl start themis
```

Option 3: Restore from Backup

```
sudo systemctl stop themis
tar -xzf /backup/themis-2026-01-01.tar.gz -C /data/
sudo chown -R themis:themis /data/themis
sudo systemctl start themis
```

Verify

```
journalctl -u themis -n 20

aql "RETURN COUNT(*) FROM users"
```

E.7 Incident: Security Alert: Unauthorized Access Attempt

Symptom

- Alert: `themis_auth_failure_rate > 5_per_minute`
- Logs: `AUTH FAILED: user='admin' from_ip='1.2.3.4' password_attempts=10`

Diagnose (1 min)

```
curl -s http://localhost:8529/_admin/audit?event=auth_failure&last_1h | jq '.events[0:10]'

cat /var/log/themis/auth.log | grep "FAILED" | cut -d' ' -f6 | sort | uniq -c | sort -rn

curl -s http://localhost:8529/_admin/users/admin | jq '.locked'
```

Fix (2 min)

Immediate:

```
sudo ufw insert 1 deny from 1.2.3.4

aql "UPDATE {name: 'admin'} WITH {locked: true, locked_reason: 'Brute force attempt'} IN user


aql "UPDATE {name: 'admin'} WITH {mfa_required: true} IN users"
```

Escalation: - Notify security team - Create incident ticket - Schedule post-mortem

Verify

```
watch "tail -20 /var/log/themis/auth.log | grep FAILED | wc -l"
```

E.8 Incident Severity Levels & Escalation

Severity 1 (Critical - All Hands)

- Database completely down (no connections)
- Data corruption/loss detected
- All writes failing

Escalation: On-call Lead + CTO + Data Team

Severity 2 (High - Urgent)

- Performance severely degraded (>10s latency)
- Replication lag > 30s
- Memory/Disk pressure critical

Escalation: On-call Engineer + Manager

Severity 3 (Medium - Standard)

- Single query slow (1-5s)
- Moderate lag (5-30s)
- Non-critical errors in logs

Escalation: On-call Engineer

Severity 4 (Low - Backlog)

- Single error/retry
- Performance degraded slightly
- Informational alerts

Escalation: Backlog Ticket

E.9 Communication Template

Incident Declared

```
  INCIDENT: Query Latency High
Status: INVESTIGATING
Severity: SEV-2
Started: 2026-01-01 12:00:00 UTC
Impact: 5-10% of users experiencing slow search
```

Root Cause Found

```
  ROOT CAUSE IDENTIFIED: Missing index on users.email
Fix: CREATE INDEX idx_email ON users(email)
ETA Resolution: 5 minutes
```

Resolved

```
  INCIDENT RESOLVED
Resolution: Index created, queries now <100ms
Duration: 15 minutes
Post-mortem: Thursday 10am
```

Summary

Keep this runbook near your on-call terminal. Most incidents fall into the categories above. When an alert fires: 1. **Navigate to right section** 2. **Run Diagnose commands** 3. **Match Root Cause** 4. **Execute Fix** 5. **Verify & Communicate**

Average resolution time with this playbook: **10-15 minutes** vs **60+ minutes** without.

Kapitel 45: Appendix F: AQL Cheat Sheet & Quick Reference

> *"Every expert was once a novice. This sheet shortens that journey."*

Overview

Quick reference for common AQL patterns, syntax, and idioms. For detailed docs, see Chapter 28.

F.1 Basic Syntax

Collections & Documents

```
-- Create Collection
CREATE COLLECTION users

-- Insert
INSERT {name: 'Alice', email: 'alice@example.com'} INTO users

-- Read
FOR u IN users RETURN u

-- Update
UPDATE 'users/123' WITH {status: 'active'} IN users

-- Delete
REMOVE 'users/123' IN users

-- Truncate (danger!)
TRUNCATE users
```

Filtering

```
-- Simple equality
FOR u IN users FILTER u.status == 'active' RETURN u

-- Comparison operators
FILTER u.age > 18 AND u.age < 65
FILTER u.salary >= 50000
FILTER u.created_at < DATE_NOW()

-- IN operator
FILTER u.status IN ['active', 'pending', 'approved']

-- String operations
FILTER u.name LIKE 'Al%'                        -- SQL-like wildcard
FILTER u.email =~ '^.*@example\.com$'       -- Regex
FILTER CONTAINS(u.bio, 'engineer')          -- Substring

-- Null checks
FILTER u.middle_name != NULL
FILTER u.phone_number == NULL

-- Range
FILTER u.score >= 100 AND u.score <= 200   -- vs RANGE below
```

Sorting & Limiting

```
-- Order results
FOR u IN users
  SORT u.created_at DESC, u.name ASC
  RETURN u

-- Limit results
FOR u IN users
  SORT u.created_at DESC
  LIMIT 10              -- First 10
  RETURN u

-- Pagination
FOR u IN users
  SORT u.created_at DESC
  LIMIT @offset, @limit  -- OFFSET, LIMIT
  RETURN u

-- Range operator (efficient for sorted data)
FOR u IN users
  RANGE u.score, 100, 200
  RETURN u
```

Projection (Select Fields)

```
-- All fields
RETURN u

-- Specific fields
RETURN {name: u.name, email: u.email}

-- Computed fields
RETURN {
  name: u.name,
  age_years: DATE_DIFF(u.birth_date, DATE_NOW(), 'y'),
  email_masked: CONCAT(SUBSTRING(u.email, 0, 3), '***')
}

-- Nested structure
RETURN {
  user: {name: u.name, id: u._id},
  contact: {email: u.email, phone: u.phone}
}
```

F.2 Advanced Filtering

Aggregation Filter

```
-- Count documents
FOR u IN users
  COLLECT WITH COUNT INTO count
  RETURN {total_users: count}

-- Group and count
FOR u IN users
  COLLECT status = u.status WITH COUNT INTO count
  RETURN {status, count}

-- Group with aggregation
FOR o IN orders
  COLLECT customer_id = o.customer_id
  AGGREGATE total = SUM(o.amount), cnt = COUNT(1)
  RETURN {customer_id, total, cnt}
```

Complex Filtering with Subqueries

```
-- Filter by aggregated value
FOR u IN users
  LET order_count = (
    FOR o IN orders FILTER o.user_id == u._id
    RETURN o
  ) | LENGTH
  FILTER order_count > 5
  RETURN {name: u.name, orders: order_count}

-- IN with subquery
FOR u IN users
  FILTER u._id IN (
    SELECT DISTINCT o.user_id FROM orders o WHERE o.status = 'completed'
  )
  RETURN u
```

Array Operations

```
-- Check array contains
FOR u IN users
  FILTER 'premium' IN u.roles
  RETURN u

-- Array length
FILTER LENGTH(u.tags) > 3

-- Array flatten
FOR u IN users
  LET all_ids = FLATTEN(u.related_user_ids)
  RETURN {name: u.name, related_count: LENGTH(all_ids)}

-- Array filtering
FOR u IN users
  LET recent_orders = u.orders[* FILTER CURRENT.date > DATE_SUBTRACT(DATE_NOW(), 30, 'd')]
  RETURN {name: u.name, recent_count: LENGTH(recent_orders)}
```

F.3 Joins & Relationships

INNER JOIN (Graph Traversal)

```
-- Simple traverse
FOR u IN users
  FOR o IN orders FILTER o.user_id == u._id
  RETURN {user: u.name, order_id: o._id}

-- With graph edges
FOR v, e, p IN 1..3 OUTBOUND users/123 GRAPH 'social'
  RETURN {vertex: v.name, edge: e.type, path: p.vertices[*]._id}

-- Shortest path
FOR path IN SHORTEST_PATH('users/123' TO 'users/456' GRAPH 'social')
  RETURN path
```

LEFT JOIN (Keep unmatched)

```
-- Users even if no orders
FOR u IN users
  LET orders = (FOR o IN orders FILTER o.user_id == u._id RETURN o)
  RETURN {user: u.name, order_count: LENGTH(orders), orders}
```

Multiple joins

```
FOR u IN users
  LET orders = (FOR o IN orders FILTER o.user_id == u._id RETURN o)
  LET addresses = (FOR a IN addresses FILTER a.user_id == u._id RETURN a)
  FILTER LENGTH(orders) > 0 AND LENGTH(addresses) > 0
  RETURN {user: u.name, order_count: LENGTH(orders), address_count: LENGTH(addresses)}
```

F.4 Transactions

Basic Transaction

```
BEGIN
  INSERT {name: 'Bob', balance: 100} INTO accounts
  INSERT {from: 'Alice', to: 'Bob', amount: 50} INTO transfers
COMMIT
```

With Error Handling

```
BEGIN
  LET update = UPDATE 'accounts/alice'
    WITH {balance: (SELECT account.balance - 50 FROM accounts FILTER _id == 'accounts/alice')}
    IN accounts

  IF u.balance < 50 THEN
    ABORT "Insufficient funds"
  ELSE
    INSERT {from: 'alice', to: 'bob', amount: 50} INTO transfers
    COMMIT
  ENDIF
```

F.5 Vector Search

Simple Vector Search

```
-- Create vector index
CREATE INDEX idx_embedding ON articles(embedding)

-- Vector search (nearest neighbors)
FOR article IN articles
  FILTER DISTANCE(article.embedding, @query_embedding) < @threshold
  SORT DISTANCE(article.embedding, @query_embedding)
  LIMIT 10
  RETURN {title: article.title, distance: DISTANCE(article.embedding, @query_embedding)}

-- With parameters
db.query("""
  FOR article IN articles
    FILTER DISTANCE(article.embedding, @query_vec) < @threshold
    SORT DISTANCE(article.embedding, @query_vec)
    LIMIT @top_k
    RETURN article
""", bind_vars={
  'query_vec': [0.1, -0.2, 0.3, ...],
  'threshold': 0.5,
  'top_k': 5
})
```

Hybrid Search (Vector + Text)

```
-- Combine vector + full-text
FOR article IN articles
  FILTER DISTANCE(article.embedding, @query_vec) < @threshold
  SEARCH article.title IN FULLTEXT(@query_text)
  SORT BM25(article) DESC  -- Relevance score
  RETURN {title: article.title, score: BM25(article)}
```

F.6 Graph Queries

Breadth-First Traversal

```
-- Get all friends of friends
FOR v IN 0..2 OUTBOUND 'users/alice' GRAPH 'social'
  RETURN {name: v.name, hops: LENGTH(p) - 1}

-- Prevent cycles
FOR v IN 1..3 OUTBOUND 'users/alice' GRAPH 'social'
  FILTER v._key NOT IN ['alice']  -- Don't revisit start node
  RETURN v
```

Depth-First with Conditions

```
-- Traverse until condition met
FOR v IN OUTBOUND 'users/alice' GRAPH 'social'
  FILTER v.status != 'banned'
  LIMIT 100
  RETURN v
```

Community Detection

```
-- Find clusters of connected users
FOR start IN users
  LET connected = (
    FOR v IN OUTBOUND start GRAPH 'social'
    RETURN DISTINCT v._id
  )
  COLLECT id = start._id, group = connected
  RETURN {user: id, cluster_size: LENGTH(group)}
```

F.7 Time-Series Queries

Bucket by Time

```
-- Sales per day
FOR order IN orders
  COLLECT date = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd')
  AGGREGATE total = SUM(order.amount), cnt = COUNT(1)
  SORT date
  RETURN {date, total, cnt}

-- Sales per hour (last 24h)
FOR order IN orders
  FILTER order.created_at > DATE_SUBTRACT(DATE_NOW(), 1, 'd')
  COLLECT hour = DATE_FORMAT(order.created_at, '%yyyy-%mm-%dd %hh:00')
  AGGREGATE total = SUM(order.amount)
  RETURN {hour, total}
```

Moving Average

```
-- 7-day moving average of daily sales
FOR day IN 0..30
  LET current_date = DATE_SUBTRACT(DATE_NOW(), day, 'd')
  LET week_start = DATE_SUBTRACT(current_date, 7, 'd')
  LET sales = (
    FOR o IN orders
      FILTER o.created_at >= week_start AND o.created_at <= current_date
      RETURN o.amount
  )
  COLLECT date = DATE_FORMAT(current_date, '%yyyy-%mm-%dd')
  AGGREGATE avg = AVG(sales)
  SORT date
  RETURN {date, moving_avg_7d: avg}
```

F.8 Window Functions

Row Numbers & Ranking

```
-- Rank users by sales
FOR u IN users
  LET total_sales = (
    SELECT SUM(amount) AS total FROM orders WHERE customer_id == u._id
  )[0].total
  SORT total_sales DESC
  RETURN {
    rank: @row_number,
    name: u.name,
    total_sales
  }

-- Cumulative sum
FOR o IN orders
  SORT o.created_at
  COLLECT WITH COUNT INTO cnt
  AGGREGATE cumulative = SUM(o.amount)
  RETURN {cumulative, running_total: @row_number}
```

F.9 Common Patterns

Pagination with Cursor

```
-- Cursor-based pagination (efficient for large datasets)
FOR u IN users
  FILTER u._key > @cursor
  SORT u._key
  LIMIT 20
  RETURN u

-- No OFFSET (O(1) instead of O(n))
```

Deduplication

```
-- Get distinct values
FOR u IN users
  COLLECT email = u.email
  RETURN email

-- Get first of each group
FOR u IN users
  COLLECT category = u.category
  INTO group = u
  RETURN {category, first_user: group[0]}
```

Batch Operations

```
-- Batch insert
FOR item IN @items
  INSERT item INTO products

-- Batch update
FOR item IN items
  FILTER item.status == 'pending'
  UPDATE item WITH {status: 'processed'} IN items
```

Error Handling

```
-- TRY-CATCH
BEGIN
  LET result = (
    TRY
      FOR u IN users FILTER u._id == @user_id RETURN u
    CATCH err
      RETURN {error: err.message}
  )
  RETURN result
COMMIT
```

F.10 Performance Tips

Query Optimization Checklist

```
✓ Use FILTER before SORT
✓ Use LIMIT to reduce result set
✓ Add indexes on FILTER/SORT fields
✓ Use COLLECT for aggregations (not manual loops)
✓ Avoid FULLTEXT searches on tiny datasets
✓ Stream large results (avoid materializing)
✓ Use bind parameters (@var) not inline values
```

Slow Query Patterns to Avoid

```
-- × SLOW: Regex without index
FILTER u.email =~ '.*@gmail\.com'

--   FAST: Use LIKE with index
CREATE INDEX idx_email ON users(email)
FILTER u.email LIKE '%@gmail.com'

-- × SLOW: Expression in FILTER
FILTER YEAR(u.birth_date) == 1990

--   FAST: Use range on indexed field
CREATE INDEX idx_birth ON users(birth_date)
FILTER u.birth_date >= '1990-01-01' AND u.birth_date < '1991-01-01'
```

F.11 Built-in Functions (Most Common)

String Functions

Function	Example	Returns
`CONCAT()`	`CONCAT('Hello', ' ', 'World')`	`'Hello World'`
`CONTAINS()`	`CONTAINS('Hello', 'ell')`	`true`
`SUBSTRING()`	`SUBSTRING('Hello', 0, 3)`	`'Hel'`
`LENGTH()`	`LENGTH('Hello')`	`5`
`UPPER()`	`UPPER('hello')`	`'HELLO'`
`LOWER()`	`LOWER('HELLO')`	`'hello'`
`SPLIT()`	`SPLIT('a,b,c', ',')`	`['a', 'b', 'c']`

Date/Time Functions

Function	Example	Returns
`DATE_NOW()`	`DATE_NOW()`	Current timestamp
`DATE_FORMAT()`	`DATE_FORMAT(d, '%yyyy-%mm-%dd')`	Formatted date
`DATE_ADD()`	`DATE_ADD('2025-01-01', 5, 'd')`	Date + 5 days
`DATE_SUBTRACT()`	`DATE_SUBTRACT(d, 1, 'm')`	Date - 1 month
`DATE_DIFF()`	`DATE_DIFF(d1, d2, 'd')`	Days between

Math Functions

Function	Example	Returns
`SUM()`	`SUM(o.amount)`	Sum of values
`AVG()`	`AVG(o.amount)`	Average
`MIN()` / `MAX()`	`MIN(values)`	Min/Max
`ROUND()`	`ROUND(3.7)`	`4`
`FLOOR()` / `CEIL()`	`FLOOR(3.7)`	`3`
`ABS()`	`ABS(-5)`	`5`
`SQRT()`	`SQRT(16)`	`4`

Array Functions

Function	Example	Returns
`LENGTH()`	`LENGTH([1,2,3])`	`3`
`APPEND()`	`APPEND(arr, item)`	Array + item
`REVERSE()`	`REVERSE([1,2,3])`	`[3,2,1]`
`UNIQUE()`	`UNIQUE([1,1,2])`	`[1,2]`
`FLATTEN()`	`FLATTEN([[1,2], [3]])`	`[1,2,3]`
`SLICE()`	`SLICE([1,2,3,4], 1, 3)`	`[2,3]`

F.12 Index Types

```
-- Unique Index
CREATE INDEX idx_email UNIQUE ON users(email)

-- Compound Index (Multiple fields)
CREATE INDEX idx_status_date ON orders(status, created_at)

-- Partial Index (Conditional)
CREATE INDEX idx_active_users ON users(name)
  FILTER u.status == 'active'

-- TTL Index (Auto-delete)
CREATE INDEX idx_temp ON temp_data(created_at)
  WITH {ttl: 3600}  -- Auto-delete after 1 hour

-- Full-Text Index
CREATE FULLTEXT INDEX idx_content ON articles(body)
```

Summary

Bookmark this section! Most queries can be built from these patterns. When stuck: 1. **Check pattern above** 2. **Run EXPLAIN to verify optimization** 3. **See Chapter 28 for detailed docs**

Kapitel 46: Appendix G: Configuration Reference

> *"Good defaults get you 90% of the way. This reference handles the other 10%."*

Overview

Complete reference for all ThemisDB configuration options. Default values optimized for single-node deployments.

G.1 Main Configuration File (themis.conf)

Location & Format

```
```

Server Settings

port

```
server:
  port: 8529  # Default: 8529 (0x2149)
  # Range: 1024-65535
  # Tip: Use ports 8529-8539 to run multiple instances
```

bind_address

```
server:
  bind_address: "127.0.0.1"  # Default: localhost only
  # "0.0.0.0"  → Listen on all interfaces
  # Tip: Use localhost in dev, 0.0.0.0 in containerized deployments
```

max_connections

```
server:
  max_connections: 1000  # Default: 1000
  # Depends on file descriptor limit (ulimit -n)
  # Scaling rule: 1 connection = ~10 KB memory overhead
  # If 10k connections: ~100 MB RAM just for connection state
```

request_timeout_ms

```
server:
  request_timeout_ms: 30000  # Default: 30 sec
  # Queries longer than this abort with timeout
  # Long-running imports: increase to 300000 (5 min)
```

enable_http2

```
server:
  enable_http2: true  # Default: true
  # Set false for testing/debugging with curl
```

G.2 Database Settings

data_directory

```
database:
  data_directory: "/var/lib/themis"  # Default
  # Must be on fast storage (SSD preferred)
  # Recommendation: NVMe > SSD > HDD (in order of performance)
  # Min free space: 2x database size
```

engine_type

```
database:
  engine_type: "rocksdb"  # Only option (default)
  # RocksDB is LSM-tree optimized for writes
  # 45k-60k writes/sec on modern hardware
```

wal_enabled

```
database:
  wal_enabled: true  # Default: true (REQUIRED for ACID)
  # Set false ONLY for ephemeral/test deployments
```

wal_directory

```
database:
  wal_directory: "/var/lib/themis/wal"  # Default
  # Tip: Put on separate disk from data for durability
```

wal_sync_mode

```
database:
  wal_sync_mode: "fsync"  # Default: fsync (safe)
  # Options:
  #   "fsync"      → Durability: ✓✓✓ (safe) | Speed: ✓ (slowest)
  #   "os"         → Durability: ✓✓  (eventual) | Speed: ✓✓
  #   "none"       → Durability: ✓   (risky) | Speed: ✓✓✓ (fastest)
  # Recommendation: fsync for production, os for staging, none for local dev
```

compression_algorithm

```
database:
  compression_algorithm: "zstd"  # Default: zstd
  # Options: "zstd" (best) | "lz4" (fast) | "snappy" | "none"
  # Recommendation: zstd for storage-constrained, lz4 for latency-critical
```

G.3 Cache Settings

cache_size_mb

```
cache:
  size_mb: 2048  # Default: 2048 (2 GB)
  # Rule of thumb: 25-30% of total system RAM
  # Example: 32 GB system → 8-10 GB cache
  # Observability: curl http://localhost:8529/_admin/cache
```

cache_mode

```
cache:
  mode: "lru"  # Default: lru (Least Recently Used)
  # Options: "lru" | "lfu" (Least Frequently Used)
  # LRU: Better for time-dependent patterns
  # LFU: Better for popularity-based patterns
```

block_cache_size_mb

```
cache:
  block_cache_size_mb: 1024  # Default: 1024
  # RocksDB block cache (L1 cache for block decompression)
  # Typically 50% of cache_size_mb
```

G.4 Index Settings

default_index_type

```
indexing:
  default_index_type: "btree"  # Default: btree
  # Options: "btree" | "hash" | "skiplist"
  # btree: Ordered access (good for range queries)
  # hash: Point lookups only (fastest for equality)
  # skiplist: Ordered with faster random access than btree
```

vector_index_default

```
indexing:
  vector_index_default: "hnsw"  # Default: hnsw
  # Options: "hnsw" (Hierarchical Navigable Small Worlds)
  # Settings:
  #   m: 16        # Connections per layer (16-64, default 16)
  #   ef_construct: 200  # Quality during construction (default)
  #   ef_search: 100     # Quality during search (default)
```

G.5 Query Settings

query_timeout_ms

```
query:
  timeout_ms: 30000  # Default: 30 sec
  # Queries exceeding this timeout abort
  # Set to -1 for no timeout (not recommended)
```

query_cache_enabled

```
query:
  cache_enabled: true  # Default: true
  # Caches query plans (not results)
  # Reduces compilation overhead for repeated queries
```

max_query_result_size_mb

```
query:
  max_query_result_size_mb: 512  # Default: 512
  # Result sets larger than this fail
  # Reason: Prevents memory bombs from buggy queries
  # Fix: Use LIMIT or OFFSET to paginate results
```

G.6 Replication Settings

replication_enabled

```
replication:
  enabled: true  # Default: true
  # false for standalone instances
```

replication_role

```
replication:
  role: "primary"  # or "follower"
  # primary: Accepts writes, logs all changes
  # follower: Read-only, applies logs from primary
```

follower_primary_address

```
replication:
  follower_primary_address: "primary.example.com:8529"  # Only for followers
  # Follower connects to primary at this address
```

replication_buffer_size_mb

```
replication:
  buffer_size_mb: 256  # Default: 256
  # Follower buffer for incoming changes
  # Larger = handle larger write spikes
```

replication_sync_interval_ms

```
replication:
  sync_interval_ms: 100  # Default: 100
  # How often to sync WAL changes to follower
  # Lower = lower latency, higher CPU
  # 100ms is good for most workloads
```

G.7 Security Settings

authentication_enabled

```
security:
  authentication:
    enabled: true  # Default: true
  # false: No authentication (dev-only!)
```

jwt_secret

```yaml
security:
  authentication:
    jwt_secret: "${JWT_SECRET}"  # Must set via env var
  # Min 32 characters, high entropy
  # Generate: openssl rand -hex 32
```

jwt_expiry_minutes

```yaml
security:
  authentication:
    jwt_expiry_minutes: 1440  # Default: 24 hours
  # Tokens expire after this time
  # Shorter = better security, more refresh overhead
```

tls_enabled

```yaml
security:
  tls:
    enabled: true  # Default: true
  # false only for insecure local dev
```

tls_cert_path

```yaml
security:
  tls:
    cert_path: "/etc/themis/certs/server.crt"
  # X.509 certificate (RSA 4096 recommended)
```

tls_key_path

```yaml
security:
  tls:
    key_path: "/etc/themis/certs/server.key"
  # Private key (must have 0600 permissions)
```

tls_min_version

```yaml
security:
  tls:
    min_version: "1.3"  # Default: "1.3"
  # Options: "1.2" (legacy) | "1.3" (modern)
  # Recommendation: "1.3" for new deployments
```

mfa_required

```
security:
  mfa_required: false  # Default: false
  # true: All users must enable MFA
```

G.8 Audit Logging

audit_enabled

```
audit:
  enabled: true  # Default: true
  # Logs all changes to collections
```

audit_directory

```
audit:
  directory: "/var/log/themis/audit"
  # Must be on write-optimized storage
```

audit_retention_days

```
audit:
  retention_days: 365  # Default: 365 days
  # Older logs auto-purged (if compliance allows)
```

audit_include_data

```
audit:
  include_data: false  # Default: false
  # true: Store full document in audit log (verbose!)
  # Recommendation: false (log metadata only)
```

G.9 Performance Tuning

memory_cache_warmup

```
performance:
  cache_warmup_enabled: true  # Default: true
  # Loads frequently accessed data into cache on startup
  # Reduces first-query latency
```

memory_buffer_size_mb

```
performance:
  memory_buffer_size_mb: 512  # Default: 512
  # In-memory write buffer before flushing to disk
  # Larger = better batching, more RAM
  # Rule: 10-20% of cache_size_mb
```

io_parallelism

```
performance:
  io_parallelism: 4  # Default: number of CPU cores
  # Concurrent IO operations
  # Higher = better throughput on SSD, diminishing returns on HDD
```

G.10 Logging

log_level

```
logging:
  level: "info"  # Default: info
  # Options: "debug" | "info" | "warn" | "error" | "critical"
  # Recommendations:
  #   production: "warn"
  #   staging: "info"
  #   development: "debug"
```

log_format

```
logging:
  format: "json"  # Default: json
  # Options: "json" | "text"
  # json: Machine parseable (for aggregation)
  # text: Human readable
```

log_retention_days

```
logging:
  retention_days: 30  # Default: 30
  # Older logs auto-deleted
  # For compliance: increase to 365+
```

log_max_file_size_mb

```
logging:
  max_file_size_mb: 100  # Default: 100
  # Rotate log file when it exceeds this size
```

G.11 Vector Search Tuning

vector_similarity_metric

```
vector:
  similarity_metric: "cosine"  # Default: cosine
  # Options: "cosine" | "euclidean" | "manhattan" | "dot"
  # cosine: Best for normalized embeddings (e.g., from BERT)
  # euclidean: Good for raw embeddings
```

hnsw_ef_search

```
vector:
  hnsw_ef_search: 100  # Default: 100
  # Exploration factor during search
  # Higher = more accurate, slower
  # Range: 10-1000
  # Rule: Start with 100, tune based on latency/accuracy tradeoff
```

hnsw_m

```
vector:
  hnsw_m: 16  # Default: 16
  # Connections per layer in HNSW graph
  # Higher = more accurate, more memory
  # Range: 8-64
```

G.12 Development Settings

dev_mode_enabled

```
development:
  dev_mode_enabled: false  # Default: false
  # true: Disables some security checks (LOCAL DEV ONLY)
  # Enables: Hot reload, relaxed CORS, schema auto-creation
```

cors_enabled

```
development:
  cors_enabled: false  # Default: false
  # true: Allow cross-origin requests
  # Set specific origins in production (not "*")
```

cors_origins

```yaml
development:
  cors_origins:
    - "http://localhost:3000"  # Frontend dev server
    - "http://localhost:8080"
```

G.13 Example Configurations

Single Node (Development)

```yaml
server:
  port: 8529
  bind_address: "127.0.0.1"
  max_connections: 100

database:
  wal_sync_mode: "os"  # Fast, eventual durability
  compression_algorithm: "lz4"

cache:
  size_mb: 512  # Small for dev machine

query:
  timeout_ms: -1  # No timeout for debugging

logging:
  level: "debug"
  format: "text"  # Human readable
```

Production (Single Node, 32GB RAM)

```yaml
server:
  port: 8529
  bind_address: "0.0.0.0"
  max_connections: 1000

database:
  data_directory: "/data/themis"
  wal_sync_mode: "fsync"
  compression_algorithm: "zstd"

cache:
  size_mb: 8192  # 25% of 32GB

security:
  tls_enabled: true
  jwt_secret: "${JWT_SECRET}"  # From env var

audit:
  enabled: true
  retention_days: 365

logging:
  level: "warn"
  format: "json"
  retention_days: 90
```

High-Performance (Writes, 64GB RAM)

```yaml
database:
  wal_sync_mode: "os"  # Trade safety for speed
  compression_algorithm: "lz4"  # Faster than zstd

cache:
  size_mb: 16384  # 25% of 64GB
  block_cache_size_mb: 4096

performance:
  memory_buffer_size_mb: 2048  # Larger batches
  io_parallelism: 8

query:
  max_query_result_size_mb: 2048  # Allow larger results
```

Analytics (Reads, 128GB RAM)

```yaml
cache:
  size_mb: 32768  # 25% of 128GB
  mode: "lfu"  # Popularity-based caching

indexing:
  vector_index_default: "hnsw"
  hnsw_ef_search: 200  # More accuracy for analytics

query:
  timeout_ms: 300000  # 5 minutes (long aggregations)
  max_query_result_size_mb: 4096  # Large result sets
```

G.14 Linux Kernel Tuning (sysctl)

These affect OS-level performance:

```bash
echo "fs.file-max = 2097152" >> /etc/sysctl.conf
echo "* soft nofile 100000" >> /etc/security/limits.conf
echo "* hard nofile 100000" >> /etc/security/limits.conf

echo "vm.swappiness = 10" >> /etc/sysctl.conf  # Prefer RAM over swap

echo "net.core.rmem_max = 134217728" >> /etc/sysctl.conf
echo "net.core.wmem_max = 134217728" >> /etc/sysctl.conf

sysctl -p
```

G.15 Environment Variables

Common Env Vars

```bash
export JWT_SECRET="$(openssl rand -hex 32)"

export THEMIS_PORT=8529
export THEMIS_BIND_ADDRESS=0.0.0.0
export THEMIS_DATA_DIR=/data/themis
export THEMIS_LOG_LEVEL=info
export THEMIS_CACHE_SIZE_MB=2048

themisdb run
```

G.16 Configuration Validation

Validate Config File

```
themisdb config validate /etc/themis/themis.conf

themisdb config show
```

Health Check

```
curl -s http://localhost:8529/_admin/health | jq .
```

G.17 Configuration Change Checklist

Before changing production config: - [ ] Backup current config: `cp themis.conf themis.conf.backup` - [ ] Test change on staging first - [ ] If safe: `systemctl reload themis` (no restart) - [ ] Monitor metrics for 5 minutes - [ ] Have rollback plan (restore .backup file)

Summary

Most Common Tuning Knobs: 1. **cache_size_mb** - Available memory (25-30% of RAM) 2. **wal_sync_mode** - Safety vs. latency tradeoff 3. **compression_algorithm** - Storage vs. speed 4. **log_level** - Observability cost 5. **query_timeout_ms** - Prevent runaway queries

Start with defaults, measure, change one thing at a time, monitor impact.

Kapitel 47: Appendix H: Glossary & Terminology Reference

> *"Every domain has its jargon. Understand the terminology, understand the system."*

H.1 Core Database Concepts

Base Entity

The fundamental unit of data in ThemisDB. Combines relational, document, graph, vector, and time-series aspects in a single unified structure.

Example:

```
{
  "_id": "users/alice",
  "_key": "alice",
  "_rev": 5,
  "name": "Alice",
  "email": "alice@example.com",
  "created_at": "2025-01-01T10:00:00Z",
  "embedding": [0.1, -0.2, 0.3, ...],
  "graph_edges": ["users/bob", "users/charlie"]
}
```

Collection

A group of related Base Entities (similar to a SQL table, but more flexible).

Types: - **Relational:** users, orders, products - **Document:** articles, posts, documents - **Graph:** relationships, friendships - **Vector:** embeddings, semantic data

Transaction

An atomic unit of work. Either all operations succeed (COMMIT) or all rollback. Provides ACID guarantees.

```
BEGIN
  INSERT {...} INTO users
  UPDATE {...} IN orders
COMMIT
```

ACID Guarantees

- **Atomicity:** All or nothing
- **Consistency:** Data integrity maintained
- **Isolation:** No dirty reads/writes
- **Durability:** Persisted to disk

H.2 Query Language (AQL)

AQL (Adaptive Query Language)

Unified query language for all data models in ThemisDB. Combines SQL (relational), graph traversal, document processing, and vector search.

Core Constructs: - FOR … IN: Iteration - FILTER: Conditional selection - LET: Variable binding - COLLECT: Grouping/aggregation - SORT: Ordering - LIMIT: Result limiting - RETURN: Output specification

Bind Variables

Parameterized query parameters (prevent injection attacks).

```
FOR user IN users
  FILTER user.email == @email
  RETURN user
```

Run with: `db.query(query, bind_vars={'email': 'alice@example.com'})`

Query Plan

Execution strategy optimized by the query optimizer.

Components: - **Collection Scan:** Read all documents (slow) - **Index Seek:** Use index to find matching rows (fast) - **Filter:** Apply WHERE conditions - **Sort:** Order results - **Aggregation:** GROUP BY operations

---

H.3 Indexing & Performance

Index

Data structure enabling fast lookups. Trade: space for speed.

Types:

Type	Ordered	Point Lookup	Range Query	Size
BTree	✓	Fast	Fast	Large
Hash	✗	Very Fast	Slow	Medium
Skiplist	✓	Fast	Fast	Medium
Inverted	✗	Fast	N/A	Large

Query Cardinality

Estimated number of rows matching query conditions.

```
Low cardinality (< 1% of docs):
  → Use index for fast filtering

High cardinality (> 50% of docs):
  → Collection scan might be faster (no index overhead)
```

Selectivity

How well an index reduces result set.

```
Selectivity = Matching rows / Total rows

High selectivity (< 5%):
  → Use index

Low selectivity (> 50%):
  → Collection scan faster
```

H.4 Replication & Consistency

Primary (Master)

Single database node accepting writes. Log changes to replicas.

Follower (Replica)

Read-only copy of primary. Applies changes asynchronously from WAL.

Replication Lag

Delay between write on primary and replica visibility.

```
Write at Primary: T0
Arrive at Follower: T0 + lag_ms
Visible to read: T0 + lag_ms + apply_time
```

Acceptable lag: - Strongly consistent: 0ms (no lag) - Eventual consistent: < 5 seconds - Archive: minutes to hours

Write-Ahead Log (WAL)

Durability mechanism. All writes logged before applied.

```
Client write:
  1. Write to WAL on disk
  2. Apply to in-memory database
  3. Ack to client

On crash:
  1. Replay WAL on startup
  2. Resume from last committed state
```

Quorum Write

Write acknowledged only after reaching majority of replicas.

```
3-node cluster:
  Quorum = (3 + 1) / 2 = 2 nodes

Write must be on Primary + 1 Follower before ack
```

H.5 Vector Search Concepts

Embedding

Dense vector representation of unstructured data.

```
"Alice is an engineer"
  → [0.1, -0.2, 0.3, 0.15, -0.08, ...]  # 384 dimensions
```

Similarity

Measure of how close two vectors are.

Metric	Range	Best For
Cosine	[-1, 1]	Normalized embeddings (angles)
Euclidean	[0, ∞)	Raw embeddings (distances)
Dot Product	(-∞, ∞)	Unnormalized embeddings

Nearest Neighbor Search

Find K closest vectors to a query vector.

```
FOR v IN vectors
  FILTER DISTANCE(v.embedding, @query) < @threshold
  SORT DISTANCE(v.embedding, @query)
  LIMIT @top_k
  RETURN v
```

HNSW (Hierarchical Navigable Small Worlds)

Approximate nearest neighbor index algorithm.

Parameters: - **M:** Connections per node (16-64, default 16) - **ef_construct:** Quality during index building - **ef_search:** Quality during searches

---

H.6 Graph Concepts

Node (Vertex)

Entity in a graph. Represents an object or concept.

Edge (Relationship)

Connection between two nodes. Directed or undirected.

Graph Traversal

Following edges to discover related nodes.

```
Alice ← (follows) ← Bob ← (follows) ← Charlie
             |                            |
             └────────────────┬───────────┘
                              │
                              ▼
                    (mutual connections)
```

Path

Sequence of nodes and edges.

```
Shortest path from Alice to Charlie:
  Alice --(follows)-- Bob --(follows)-- Charlie
  Length: 2 hops
```

Centrality

Measure of node importance.

Types: - **Degree:** Number of connections - **Betweenness:** How often on shortest paths - **Closeness:** Average distance to others - **PageRank:** Voting-based importance

---

H.7 Time-Series Concepts

Time-Series Data

Ordered sequence of measurements over time.

```
Metric: CPU Usage
Timestamp 1: 45%
Timestamp 2: 52%
Timestamp 3: 48%
...
```

Bucket / Time Window

Grouping data into time periods.

```
FOR metric IN metrics
  COLLECT hour = DATE_FORMAT(metric.timestamp, '%yyyy-%mm-%dd %hh:00')
  AGGREGATE avg = AVG(metric.value)
  RETURN { hour, avg }
```

Downsampling

Reducing time-series resolution (e.g., minute → hour).

```
High resolution: Every second (86,400 points/day)
Low resolution: Every hour (24 points/day)
Compression: 99.97% reduction
```

H.8 Operational Concepts

SLA (Service Level Agreement)

Contractual commitment on availability and performance.

```
SLA: "99.9% uptime with p99 latency < 200ms"
  → Downtime allowed: 43.2 minutes/month
  → 1 in 1000 queries can exceed 200ms
```

SLI (Service Level Indicator)

Measurable metric of actual performance.

```
SLI: Actual uptime = 99.92%
SLI: Actual p99 latency = 187ms
```

Error Budget

Acceptable downtime/errors for SLA.

```
SLA: 99.95% uptime
Error budget = 0.05% = 21.6 minutes/month

Used: 15 minutes
Remaining: 6.6 minutes (use for deployments/maintenance)
```

MTTR (Mean Time To Recovery)

Average time to fix a problem.

```
Incident starts: 10:00
Recovery complete: 10:15
MTTR = 15 minutes
```

MTTF (Mean Time To Failure)

Average time before next failure.

```
Downtime: 15 minutes
Recovery: 30 minutes (MTTR)
Uptime: 30 days until next failure
MTTF = 30 days
```

H.9 Security Concepts

RBAC (Role-Based Access Control)

Access control based on user roles.

```
Role: 'data_analyst'
Permissions:
  - READ from analytics_*
  - EXECUTE report queries
  - NO WRITE/DELETE
```

JWT (JSON Web Token)

Stateless authentication token.

```
Header: { alg: "HS256", typ: "JWT" }
Payload: { sub: "alice", exp: 1726000000 }
Signature: HMAC-SHA256(header + payload, secret)
```

Encryption at Rest

Data encrypted on disk.

```
File on disk: [encrypted bytes]
In memory: Clear text (needed for processing)
Key location: HSM (Hardware Security Module)
```

Encryption in Transit

Data encrypted while traveling network.

```
TLS 1.3 + Perfect Forward Secrecy
  → Even if long-term key compromised, old traffic safe
```

MFA (Multi-Factor Authentication)

Multiple authentication mechanisms.

```
1. Password (something you know)
2. TOTP (something you have) - app code
3. WebAuthn (something you are) - fingerprint
```

H.10 Architecture Concepts

Monolithic Architecture

All components in single process. Easy to develop, hard to scale.

Microservices

Each service independent. Scales well, operational complexity.

ThemisDB: Hybrid Approach

- **Monolithic Core:** Single database engine
- **Distributed:** Horizontal sharding for scale-out
- **Multi-Model:** All data models in one engine (no microservices)

Horizontal Scaling

Add more nodes (scale out). Divide data via sharding.

```
1 node (10TB limit)
  ↓
2 nodes (5TB each)
  ↓
4 nodes (2.5TB each)
```

Vertical Scaling

Upgrade hardware (scale up). More RAM/CPU per node.

```
32GB RAM
  ↓
64GB RAM
```

H.11 Monitoring & Observability

Metrics

Quantitative measurements (numbers).

```
CPU usage: 45%
Memory: 8.2 GB
Queries per second: 1,200
Latency p99: 185ms
```

Logs

Textual records of events.

```
2025-01-01T10:00:00Z [INFO] Query executed: SELECT * FROM users
2025-01-01T10:00:01Z [WARN] Query latency: 245ms > threshold 200ms
2025-01-01T10:00:02Z [ERROR] Connection timeout from client 1.2.3.4
```

Traces

Request flow through system (distributed tracing).

```
User Request
  ├─ API Gateway (5ms)
  ├─ Authentication (12ms)
  ├─ Database Query (185ms)
  │   ├─ Query Plan (2ms)
  │   ├─ Index Seek (80ms)
  │   ├─ Filter (50ms)
  │   └─ Return (53ms)
  └─ Response (3ms)
Total: 205ms
```

Cardinality

Number of unique values.

```
High cardinality: User IDs (millions)
Low cardinality: Status (10 values)
```

H.12 Alphabetical Glossary

API: Application Programming Interface - interface for programmatic access

ACID: Atomicity, Consistency, Isolation, Durability guarantees

AQL: Adaptive Query Language - ThemisDB's query language

Backup: Copy of data for recovery

Batch: Group of operations processed together

Cardinality: Number of distinct values

Cache: Fast memory storage layer

Changefeed: Stream of database changes

Collection: Group of related entities

Consistency: Data integrity maintained across system

Cursor: Pointer to result set for iteration

Denormalization: Storing redundant data for performance

Document: Semi-structured data (JSON-like)

DSGVO: German data protection regulation (GDPR equivalent)

Edge: Connection in graph

Embedding: Vector representation of data

Failover: Automatic switch to backup system

Flush: Write data from memory to disk

Garbage Collection: Freeing unused memory

Graph: Network of connected nodes

Heap: Memory area for dynamic allocation

HNSW: Hierarchical Navigable Small Worlds - vector index

Hot Data: Frequently accessed data

Index: Data structure for fast lookups

Ingestion: Loading data into database

Inverted Index: Index mapping values to documents

Isolation: Transactions don't interfere

JIT: Just-In-Time compilation

JSON: JavaScript Object Notation

JWT: JSON Web Token - stateless auth

Keyspace: Logical division of data

Latency: Time delay for operation

LSM: Log-Structured Merge tree

MVCC: Multi-Version Concurrency Control

Normalization: Organizing data to reduce redundancy

Node: Entity in graph or cluster

OLAP: Online Analytical Processing

OLTP: Online Transaction Processing

Optimization: Making something run faster

Pagination: Dividing results into pages

Partition: Division of data across nodes

Percentile: Value below which percentage falls

Persistence: Data survives shutdown

Projection: Selecting subset of columns

Query Plan: Execution strategy for query

Quorum: Majority consensus

Range: Ordered selection of values

Replica: Copy of data

Replication: Process of copying data to replicas

RocksDB: Embedded key-value store (ThemisDB storage engine)

Rollback: Undo transaction

RPO: Recovery Point Objective (max data loss)

RTO: Recovery Time Objective (max downtime)

Sampling: Statistical subset of data

Schema: Data structure definition

Selectivity: Fraction of rows matching filter

Serialization: Converting to byte format

Sharding: Horizontal data partitioning

Snapshot: Point-in-time data view

SQL: Structured Query Language

SSTable: Sorted String Table

TTL: Time-To-Live (auto-delete after interval)

Transaction: Atomic unit of work

Throughput: Operations per unit time

Vector: Ordered list of numbers

View: Virtual table derived from query

WAL: Write-Ahead Log

Warm Data: Occasionally accessed data

Workload: Pattern of database usage

Summary

Understanding these terms is essential for: - **Development:** Writing efficient queries - **Operations:** Configuring and monitoring - **Architecture:** Designing systems - **Communication:** Discussing with team

Keep this glossary handy when learning or explaining ThemisDB concepts.

Kapitel 48: Appendix I: Troubleshooting Guide & Common Issues

> *"The best troubleshooting guide prevents issues before they start."*

I.1 Installation Issues

Issue: Failed to Install Dependencies

Error:

```
ERROR: Package xyz not found in vcpkg
```

Diagnosis:

```
./vcpkg list | grep xyz

ping github.com
```

Solutions: 1. **Update vcpkg registry:** `bash ./vcpkg update ./vcpkg upgrade # Rebuild all packages`

1. **Clear cache:** `bash rm -rf vcpkg_installed/ ./vcpkg install # Reinstall from scratch`

2. **Use specific version:** `json // vcpkg.json { "overrides": [ { "name": "xyz", "version": "1.2.3" } ] }`

I.2 Connection Issues

Issue: Cannot Connect to Database

Error:

```
Connection refused at localhost:8529
```

Diagnosis:

```
systemctl status themis

lsof -i :8529

sudo ufw status
```

Solutions:

1. **Service not running:** `bash sudo systemctl start themis sudo systemctl enable themis # Auto-start on reboot`

2. **Port blocked by firewall:** `bash sudo ufw allow 8529 sudo ufw allow in on docker0 to any port 8529 # Docker`

3. **Process stuck:** `bash # Force kill (last resort) pkill -9 themisdb sudo systemctl restart themis`

Issue: SSL/TLS Certificate Error

Error:

```
ERROR: x509: certificate signed by unknown authority
```

Solutions:

1. **Invalid certificate:** ```bash # Check certificate openssl x509 -in /etc/themis/certs/server.crt -text -noout

Verify against key openssl x509 -in server.crt -noout -modulus | openssl md5 openssl rsa -in server.key -noout -modulus | openssl md5 # Should match ```

1. **Self-signed certificate (dev only):** `bash # Disable SSL verification (DANGER - dev only) THEMIS_SKIP_VERIFY=true themis-cli ...`

2. **Certificate expired:** ```bash # Generate new certificate openssl req -x509 -nodes -days 365 -newkey rsa:4096 \ -keyout server.key -out server.crt

Restart sudo systemctl restart themis ```

I.3 Performance Issues

Issue: Queries Running Slow (>1s)

Diagnosis:

```
-- Use EXPLAIN to see query plan
EXPLAIN
FOR user IN users
  FILTER user.status == 'active'
  RETURN user

-- Check which collection scan
-- If CollectionScan on large table → use index!
```

Solutions:

1. **Missing Index:** ```aql -- Create index on filtered field CREATE INDEX idx_status ON users(status)

-- Verify index is used EXPLAIN -- Should show IndexSeek ```

1. **Wrong Index:** ```aql -- Problem: Composite index (status, created_at) -- But querying only created_at

-- Solution: Reorder to match query pattern DROP INDEX idx_status_created CREATE INDEX idx_created_status ON users(created_at, status) ```

1. **Expression in FILTER:** ```aql -- × SLOW: Expression prevents index use FILTER YEAR(user.created_at) == 2025

-- FAST: Range query on indexed field FILTER user.created_at >= '2025-01-01' AND user.created_at < '2026-01-01' ```

Issue: High Memory Usage (>80%)

Diagnosis:

```
free -h
top -p $(pgrep themis)

curl http://localhost:8529/_admin/cache | jq .

curl http://localhost:8529/_admin/cursors | jq 'length'
```

Solutions:

1. **Cache too large:** `yaml # themis.conf cache: size_mb: 2048 # Reduce from 8192` Then: `sudo systemctl reload themis`
2. **Cursor leak (unclosed connections):** ```python # × Leak cursor = client.query("SELECT ...") # Never closed

Safe with client.query("SELECT ...") as cursor: for doc in cursor: process(doc) ```

1. **Large result set:** ```aql -- × Materializes 1M rows FOR doc IN collection RETURN doc

-- Paginate FOR doc IN collection LIMIT @offset, 100 RETURN doc ```

I.4 Data Issues

Issue: Data Corruption (Checksum Mismatch)

Error:

```
ERROR: Block checksum mismatch at offset 123456
```

Recovery:

```
sudo systemctl stop themis

themisdb recover /var/lib/themis

tar -xzf /backup/themis-2025-01-01.tar.gz -C /var/lib/
sudo chown -R themis:themis /var/lib/themis

sudo systemctl start themis

aql "RETURN COUNT(*) FROM collection"
```

Issue: Missing Data (Unintended Deletion)

Diagnosis:

```
grep DELETE /var/log/themis/audit.log | tail -20
```

Recovery:

```
sudo systemctl stop themis
tar -xzf /backup/themis-2025-01-01-14-00.tar.gz -C /var/lib/  # Before deletion
sudo systemctl start themis

aws s3 cp s3://backups/themis-before-deletion/ /tmp/
```

Issue: Replication Lag Too High (>10s)

Diagnosis:

```
curl http://primary:8529/_admin/replication/lag | jq .lag_ms

ssh follower "top -bn1 | head -15"
ssh follower "iostat -x 1 5 | grep sda"

mtr primary-ip --report --report-wide
```

Solutions:

1. **Follower slow (CPU/IO):** `bash # Add more resources # Edit VM/K8s resource limits # Restart follower to pick up new limits`
2. **Network congestion:** ```bash # Check MTU ip link show | grep mtu

If < 9000, enable Jumbo frames (if supported) ip link set dev eth0 mtu 9000 ```

1. **Primary write rate too high:** `yaml # Throttle writes (temporary) # themis.conf database: max_write_rate_per_sec: 1000 # Limit to 1k/s`

I.5 Replication Issues

Issue: Follower Won't Start (Replication Sync Failed)

Error:

```
ERROR: Cannot connect to primary at primary.example.com:8529
```

Solutions:

1. **Primary unreachable:** ```bash # Test connectivity telnet primary.example.com 8529 ping primary.example.com

Check network policies kubectl get networkpolicies -A ```

1. **Follower config wrong:** ```yaml # Check follower primary address cat /etc/themis/themis.conf | grep follower_primary

Fix if needed # Edit config, then: systemctl restart themis ```

1. **Data mismatch (corruption):** `bash # Full resync (rebuilds follower from scratch) themisdb replication reset sudo systemctl start themis # This will take time (downloads full dataset from primary)`

Issue: Primary-Follower Divergence

Symptoms:

```
Primary: 10000 documents
Follower: 9999 documents
```

Investigation:

```
curl http://primary:8529/_admin/collection/mycoll/count | jq .count
curl http://follower:8529/_admin/collection/mycoll/count | jq .count
```

Fix:

```
sudo systemctl stop themis-follower

themisdb replication reset --primary primary:8529

curl http://follower:8529/_admin/replication/status | jq .lag_ms
```

I.6 Backup & Recovery Issues

Issue: Backup Failed

Error:

```
ERROR: Failed to backup - insufficient space
```

Solutions:

1. **Disk full:** ```bash # Check disk df -h /backup

Clean old backups ls -lt /backup/*.tar.gz | tail -10 # Identify oldest rm /backup/ themis-2024-*.tar.gz # Delete old ```

1. **Permissions:** ```bash # Check backup directory ownership ls -ld /backup

Fix if needed sudo chown themis:themis /backup sudo chmod 750 /backup ```

1. **Database locked:** ```bash # Check if backup process stuck ps aux | grep themis-backup

Kill if necessary pkill -f themis-backup ```

Issue: Restore Failed (Data Corrupted)

Error:

```
ERROR: Restore failed - backup corrupted
```

Verification:

```
tar -tzf /backup/themis-2025-01-01.tar.gz > /dev/null && \
  echo "Backup OK" || echo "Backup CORRUPTED"

ls -lt /backup/themis-*.tar.gz | head -5
```

Recovery:

```
BACKUP=/backup/themis-2024-12-25.tar.gz

tar -tzf $BACKUP > /dev/null || echo "Also corrupted!"

themisdb recover --wal-only /var/lib/themis
```

I.7 Security Issues

Issue: Unauthorized Access (Auth Failing)

Error:

```
ERROR: Authentication failed: Invalid credentials
```

Diagnosis:

```
tail -50 /var/log/themis/auth.log

aql "SELECT * FROM users WHERE name == 'alice'"
```

Solutions:

1. **Wrong password:** `bash # Reset user password themis-cli admin reset-password alice # New temporary password will be printed`
2. **User locked out:** `aql -- Unlock user UPDATE {name: 'alice'} WITH {locked: false} IN users`
3. **JWT token expired:** `python # Get new token token = client.authenticate(username='alice', password='xxx') # Use new token in subsequent requests`

Issue: Data Breach Detected

Immediate Actions:

```
1. Isolate database from network
   → Drop external connections
   → Kill active sessions

2. Preserve evidence
   → Copy audit logs
   → Backup database
   → Keep system for forensics

3. Notify stakeholders
   → Security team
   → Management
   → Legal (GDPR notification)

4. Rotate credentials
   → Generate new passwords
   → Rotate encryption keys
   → Revoke old tokens
```

I.8 Support Matrix: By Issue Type

Issue	Severity	MTTR	Self-Fix %
Connection timeout	High	5 min	95%
Query slow (no index)	Medium	10 min	90%
High memory usage	High	15 min	80%
Data corruption	Critical	30 min	60%
Replication lag	Medium	20 min	70%
Backup failed	Medium	15 min	85%
Auth issues	High	10 min	75%
Security breach	Critical	60 min	20%

I.9 Escalation Path

```
Level 1: Try self-help
  ├─ Check documentation
  ├─ Search existing issues
  └─ Try basic troubleshooting

Level 2: Community support
  ├─ GitHub Discussions
  ├─ Stack Overflow
  └─ Slack community

Level 3: Vendor support
  ├─ File bug report
  ├─ Provide diagnostic data
  └─ Wait for response

Level 4: Enterprise support
  ├─ Dedicated support engineer
  ├─ SLA response times
  └─ Priority queue
```

I.10 Data Collection for Support

When reporting issues, include:

```bash
#!/bin/bash

echo "=== System Info ==="
uname -a
free -h
lsb_release -a

echo "=== ThemisDB Version ==="
themisdb --version

echo "=== Service Status ==="
systemctl status themis
journalctl -u themis -n 50

echo "=== Memory Usage ==="
curl http://localhost:8529/_admin/memory | jq .

echo "=== Disk Usage ==="
df -h /var/lib/themis

echo "=== Active Queries (top 5) ==="
curl http://localhost:8529/_admin/slow-queries | jq '.queries[0:5]'

echo "=== Configuration ==="
grep -v '^#' /etc/themis/themis.conf | grep -v '^$'
```

I.11 Contacting Support

Community Channels

- **GitHub Issues:** github.com/makr-code/ThemisDB/issues
- **Discussions:** github.com/makr-code/ThemisDB/discussions
- **Stack Overflow:** Tag: `themisdb`

Enterprise Support (SLA)

- **Response time:** < 4 hours for Critical
- **On-call escalation:** Yes
- **Dedicated engineer:** Yes

When to Contact

- **Development questions:** Community first
- **Bug reports:** GitHub Issues
- **Security issues:** Responsible disclosure (see SECURITY.md)
- **Critical production outage:** Vendor support hotline

Summary

Most issues fall into categories above. With this guide, you should be able to: 1. **Diagnose** problems quickly 2. **Fix** common issues yourself 3. **Know when to escalate** to support 4. **Provide** detailed diagnostic data

Keep this guide bookmarked for quick reference during incidents.

Kapitel 49: Anhang A: Literaturverzeichnis

Dieses Literaturverzeichnis folgt dem IEEE-Zitierstil und umfasst alle wissenschaftlichen und technischen Quellen, die in diesem Kompendium referenziert wurden.

Primärquellen: ThemisDB Gimini Analysen

Die folgenden strategischen Analysen wurden als Primärquellen für die technische Vertiefung in Kapitel 1, 2, 3 und 17 herangezogen:

[1] "Strategische Gesamtanalyse: ThemisDB als ACID-Fundament für die RAG-LLM-Strategie der deutschen Verwaltung – Eine komparative Analyse zur Ablösung der UDS3-Architektur," ThemisDB Project, Internes Strategiedokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/Strategische Gesamtanalyse_ ThemisDB als ACID-Fundament für die RAG-LLM-Strategie der deutschen Verwaltung – Eine komparative Analyse zur Ablösung der UDS3-Architektur.md

[2] "Strategische Analyse: ThemisDB – Bewertung einer nativen Multi-Modell-Architektur im Kontext von Sovereign-Cloud-Plattformen und On-Premise-RAG-Alternativen," ThemisDB Project, Internes Analysedokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/Strategische Analyse_ ThemisDB – Bewertung einer nativen Multi-Modell-Architektur im Kontext von Sovereign-Cloud-Plattformen und On-Premise-RAG-Alternativen.md

[3] "Die ThemisDB-Architektur: Eine technische Tiefenanalyse eines Multi-Modell-Datenbanksystems auf LSM-Tree-Basis," ThemisDB Project, Technisches Whitepaper, Nov. 2025. [Online]. Verfügbar: docs/gimini/ThemisDB Dokumentation und Berichtsanalyse.md

[4] "Architektonischer Entwurf eines hochperformanten Multi-Modell-Datenbanksystems: Eine Kernel-Level-Analyse von kanonischem Speicher, Projektionsschichten und Speicherhierarchien in C++ und Rust," ThemisDB Project, Architektur-Blueprint, Nov. 2025. [Online]. Verfügbar: docs/gimini/Hybride Datenbankarchitektur C++_Rust (1).md

[5] "ThemisDB vs. Hyperscaler Datenbanken: Komparativer Vergleich," ThemisDB Project, Vergleichsanalyse, Nov. 2025. [Online]. Verfügbar: docs/gimini/Themis vs. Hyperscaler Datenbanken Vergleich.md

[6] "Hyperscaler-Einordnung für ThemisDB-Bericht," ThemisDB Project, Marktanalyse, Nov. 2025. [Online]. Verfügbar: docs/gimini/Hyperscaler-Einordnung für ThemisDB-Bericht.md

[7] "KI-Strategie: ThemisDB, Hyperscaler, Ethik," ThemisDB Project, Strategiepapier, Nov. 2025. [Online]. Verfügbar: docs/gimini/KI-Strategie_ ThemisDB, Hyperscaler, Ethik.md

[8] "VCCDB Design: Veritas, Covina, Clara Database Design Spezifikation," ThemisDB Project, Design-Dokument, Nov. 2025. [Online]. Verfügbar: docs/gimini/VCCDB Design (1).md

[9] "Forschungsbericht ThemisDB 2025: Public Research Edition," ThemisDB Project, Forschungsbericht, Nov. 2025. [Online]. Verfügbar: docs/gimini/Forschungsbericht ThemisDB 2025_THEMISDB_PUBLIC_RE....md

---

Sekundärquellen: Datenbanksysteme und Architektur

[10] M. Stonebraker and U. Cetintemel, "'One size fits all': An idea whose time has come and gone," in *Proc. IEEE 21st International Conference on Data Engineering (ICDE '05)*, Tokyo, Japan, 2005, pp. 2-11, doi: 10.1109/ICDE.2005.1.

[11] J. Lu and I. Holubová, "Multi-model databases: A new journey to handle the variety of data," *ACM Computing Surveys*, vol. 52, no. 3, pp. 1-38, Jun. 2019, doi: 10.1145/3323214.

[12] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351-385, Jun. 1996, doi: 10.1007/s002360050048.

[13] Facebook Engineering, "RocksDB: A persistent key-value store for fast storage environments," Facebook Inc., Technical Report, 2013. [Online]. Verfügbar: https://rocksdb.org/

[14] D. G. Andersen, J. Franklin, M. Kaminsky, A. Phanishayee, L. Tan, and V. Vasudevan, "FAWN: A fast array of wimpy nodes," in *Proc. 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, Big Sky, MT, USA, 2009, pp. 1-14, doi: 10.1145/1629575.1629577.

[15] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD International Conference on Management of Data*, San Jose, CA, USA, 1995, pp. 1-10, doi: 10.1145/223784.223785.

Transaktionsverarbeitung und Konsistenz

[16] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2009.

[17] C. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, San Francisco, CA, USA, 1987, pp. 249-259, doi: 10.1145/38713.38742.

[18] E. Brewer, "CAP twelve years later: How the 'rules' have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23-29, Feb. 2012, doi: 10.1109/MC.2012.37.

[19] D. B. Lomet, "Process structuring, synchronization, and recovery using atomic actions," in *Proc. ACM Conference on Language Design for Reliable Software*, Raleigh, NC, USA, 1977, pp. 128-137, doi: 10.1145/800022.808314.

[20] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," *ACM Transactions on Database Systems*, vol. 34, no. 4, pp. 1-42, Dec. 2009, doi: 10.1145/1620585.1620587.

Graph-Datenbanken und Traversierung

[21] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys*, vol. 40, no. 1, pp. 1-39, Feb. 2008, doi: 10.1145/1322432.1322433.

[22] M. A. Rodriguez and P. Neubauer, "The graph traversal pattern," in *Graph Data Management: Techniques and Applications*, S. Sakr and E. Pardede, Eds. Hershey, PA, USA: IGI Global, 2011, pp. 29-46, doi: 10.4018/978-1-61350-053-8.ch002.

[23] N. Francis et al., "Cypher: An evolving query language for property graphs," in *Proc. ACM SIGMOD International Conference on Management of Data*, Houston, TX, USA, 2018, pp. 1433-1445, doi: 10.1145/3183713.3190657.

[24] R. Angles, "A comparison of current graph database models," in *Proc. IEEE 28th International Conference on Data Engineering Workshops (ICDEW)*, Arlington, VA, USA, 2012, pp. 171-177, doi: 10.1109/ICDEW.2012.31.

Vektor-Datenbanken und Similarity Search

[25] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[26] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, Sep. 2021, doi: 10.1109/TBDATA.2019.2921572.

[27] M. Muja and D. G. Lowe, "Scalable nearest neighbor algorithms for high dimensional data," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 11, pp. 2227-2240, Nov. 2014, doi: 10.1109/TPAMI.2014.2321376.

[28] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 2055-2063, doi: 10.1109/CVPR.2016.226.

RAG und LLM-Integration

[29] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459-9474.

[30] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009, doi: 10.1561/1500000019.

[31] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Virtual Event, 2020, pp. 39-48, doi: 10.1145/3397271.3401075.

[32] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, Dec. 2023. [Online]. Verfügbar: https://arxiv.org/abs/2312.10997

Polyglot Persistence und Multi-Model-Systeme

[33] M. Fowler and P. Sadalage, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2012.

[34] J. Pokorný, "Conceptual and database modelling of graph databases," in *Proc. 20th International Conference on Information Systems Development (ISD)*, Edinburgh, Scotland, 2011, pp. 370-381.

[35] ArangoDB Inc., "ArangoDB Multi-Model Database Architecture," Technical Whitepaper, 2020. [Online]. Verfügbar: https://www.arangodb.com/

[36] Microsoft Azure, "Azure Cosmos DB: Multi-model database service," Microsoft Technical Documentation, 2023. [Online]. Verfügbar: https://azure.microsoft.com/en-us/services/cosmos-db/

Kompression und Speicheroptimierung

[37] Y. Collet, "LZ4: Extremely fast compression algorithm," Open Source Implementation, 2011. [Online]. Verfügbar: https://lz4.github.io/lz4/

[38] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," Internet Engineering Task Force (IETF), RFC 8878, Feb. 2021. [Online]. Verfügbar: https://tools.ietf.org/html/rfc8878

[39] M. Adler, "Snappy: A fast compressor/decompressor," Google Inc., Technical Report, 2011. [Online]. Verfügbar: https://google.github.io/snappy/

Serialisierung und Parsing

[40] D. Lemire, G. Ssi-Yan-Kai, and O. Katz, "Parsing gigabytes of JSON per second," *The VLDB Journal*, vol. 28, no. 6, pp. 941-960, Dec. 2019, doi: 10.1007/s00778-019-00578-5.

[41] ArangoDB Inc., "VelocyPack: A fast and compact format for serialization and storage," Technical Specification, 2016. [Online]. Verfügbar: https://github.com/arangodb/velocypack

[42] Google Inc., "Protocol Buffers: Google's data interchange format," Technical Documentation, 2008. [Online]. Verfügbar: https://developers.google.com/protocol-buffers

---

Standards und Compliance

[43] Bundesamt für Sicherheit in der Informationstechnik (BSI), "IT-Grundschutz-Kompendium," BSI Standard 200-2, Bonn, Deutschland, 2023. [Online]. Verfügbar: https://www.bsi.bund.de/

[44] European Parliament and Council, "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679, Apr. 2016. [Online]. Verfügbar: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[45] Apache Software Foundation, "Apache Ranger: Framework to enable, monitor and manage comprehensive data security," Technical Documentation, 2023. [Online]. Verfügbar: https://ranger.apache.org/

Verwendete Softwarebibliotheken

[46] Facebook Engineering, "RocksDB C++ API Reference," Version 8.8.0, 2023. [Online]. Verfügbar: https://github.com/facebook/rocksdb

[47] The Rust Project, "serde: Serialization framework for Rust," Version 1.0, 2023. [Online]. Verfügbar: https://serde.rs/

[48] D. Lemire et al., "simdjson: Parsing gigabytes of JSON per second," Version 3.0, 2023. [Online]. Verfügbar: https://github.com/simdjson/simdjson

---

Verwendungshinweise

Kapitel-Referenzen

Die folgenden Primärquellen wurden in den entsprechenden Kapiteln verwendet:

Kapitel 1 (Einführung): - [1] Strategische Gesamtanalyse - Teil III: Architektonische Kernentscheidung - [3] ThemisDB-Architektur - Teil 1: Kanonische Speicherarchitektur - [4] Architektonischer Entwurf - Teil 1: Base Entity Paradigma

Kapitel 2 (Architektur): - [3] ThemisDB-Architektur - Teil 1.3: RocksDB als transaktionales Fundament - [4] Architektonischer Entwurf - Teil 1: Multi-Modell-Speicherformat - [5] Hyperscaler-Vergleich - Teil 3.1: Kanonischer Speicher - [12] LSM-Tree Grundlagen - [13] RocksDB Dokumentation

Kapitel 3 (Multi-Model): - [1] Strategische Gesamtanalyse - Teil III: Ablösung von UDS3 - [2] Strategische Analyse - Teil II: Wettbewerbslandschaft - [5] Hyperscaler-Vergleich - Teil 2: Post-Filtering Problem - [11] Multi-Model Databases Survey

Kapitel 17 (LLM-Integration): - [2] Strategische Analyse - Teil 1.3: Native KI- und RAG-Fähigkeiten - [5] Hyperscaler-Vergleich - Teil 2: RAG-Architektur - [29] Retrieval-Augmented Generation - [30] BM25 and Beyond

Zitierkonventionen im Text

Im Text dieses Kompendiums werden Quellen durch eckige Klammern referenziert, z.B. [1], [3], [12]. Die vollständigen Quellenangaben finden Sie in diesem Anhang im IEEE-Format.

Zugriff auf Primärquellen

Die Primärquellen [1]-[9] sind interne Projektdokumente im `docs/gimini/` Verzeichnis des ThemisDB Repository verfügbar. Die Sekundärquellen [10]-[48] sind peer-reviewed wissenschaftliche Publikationen, technische Standards oder Open-Source-Dokumentationen, die über die angegebenen DOIs oder URLs zugänglich sind.

Zuletzt aktualisiert: Dezember 2025
Version: 1.3.4
